

BIS Complete Source Code Reference

Generated on: 2026-05-06 10:41:36

This document is auto-generated from project source files.

apphosting.yaml

```yaml

runtime: python3.12

runConfig:

minInstances: 0

maxInstances: 50

concurrency: 80

cpu: 1

memoryMiB: 512

entrypoint: uvicorn backend.app.main:app --host 0.0.0.0 --port \$PORT

env:

# Firebase bucket

- variable: FIREBASE\_STORAGE\_BUCKET

value: barangay-1721d.appspot.com

availability:

- RUNTIME

```
PayMongo secrets
```

```
- variable: PAYMONGO_SECRET_KEY
```

```
secret: paymongoSecretKeyRef
```

```
availability:
```

```
- RUNTIME
```

```
- variable: PAYMONGO_PUBLIC_KEY
```

```
secret: paymongoPublicKeyRef
```

```
availability:
```

```
- RUNTIME
```

```
- variable: PAYMONGO_WEBHOOK_SECRET
```

```
secret: paymongoWebhookSecretRef
```

```
availability:
```

```
- RUNTIME
```

```
...
```

```
backend__init__.py
```

```
```python
```

```
# backend package
```

```
...
```

```
## backend\app\__init__.py
```

```
```python
```

```
...
```

```
backend\app\config.py
```

```
```python
```

```
import os
```

```
class Settings:
```

```
    PAYMONGO_SECRET_KEY = os.environ.get("PAYMONGO_SECRET_KEY")
```

```
    PAYMONGO_PUBLIC_KEY = os.environ.get("PAYMONGO_PUBLIC_KEY")
```

```
    PAYMONGO_WEBHOOK_SECRET = os.environ.get("PAYMONGO_WEBHOOK_SECRET")
```

```
    FIREBASE_STORAGE_BUCKET = os.environ.get("FIREBASE_STORAGE_BUCKET")
```

```
settings = Settings()
```

```
```
```

```
backend\app\core\auth.py
```

```
```python
```

```
# backend/app/core/auth.py
```

```
from firebase_admin import auth, firestore
```

```
from fastapi import Depends, HTTPException, Header, status
```

```
from backend.app.core.roles import get_permissions
```

```
from backend.app.core.firebase import ensure_firebase_initialized #  centralized init
```

```
import logging
```

```
logger = logging.getLogger("unicorn.error")
```

```
def get_db() -> firestore.Client:
```

```
    """Return Firestore client, ensuring Firebase is initialized."""
```

```
    ensure_firebase_initialized()
```

```
    return firestore.client()
```

```
def _verify_token(authorization: str) -> dict:
```

```
    if not authorization or not authorization.startswith("Bearer "):
```

```
        raise HTTPException(status_code=401, detail="Missing or malformed Bearer token")
```

```
    token = authorization.removeprefix("Bearer ").strip()
```

```
    try:
```

```
        decoded = auth.verify_id_token(token)
```

```
        logger.info("✅ Token verified: uid=%s, role=%s, aud=%s, iss=%s",
```

```
                    decoded.get("uid"), decoded.get("role"), decoded.get("aud"), decoded.get("iss"))
```

```
        return decoded
```

```
    except Exception as e:
```

```
        # Log unverified claims for debugging
```

```
        try:
```

```
            import jwt
```

```
            unverified = jwt.decode(token, options={"verify_signature": False})
```

```
            logger.error("❌ Token verification failed: %s | Claims=%s", e, unverified)
```

```
        except Exception:
```

```
            logger.error("❌ Token verification failed: %s | Could not decode claims", e)
```

```
raise HTTPException(status_code=401, detail=f"Authentication failed: {str(e)}")
```

```
async def get_current_user(authorization: str = Header(...)) -> dict:
```

```
    """Return decoded token payload for the current user, resolving role from Firestore if missing."""
```

```
    decoded = _verify_token(authorization)
```

```
    uid = decoded.get("uid")
```

```
    if not uid:
```

```
        raise HTTPException(status_code=401, detail="Invalid token payload: UID missing")
```

```
    role = decoded.get("role")
```

```
    logger.debug("🔍 Resolving role for uid=%s: token role=%s", uid, role)
```

```
    # 🔍 Derive role if not present in claims
```

```
    if not role:
```

```
        logger.warning("⚠️ No role claim in token for uid=%s. Falling back to Firestore.", uid)
```

```
        db = get_db()
```

```
        user_doc = db.collection("users").document(uid).get()
```

```
        if user_doc.exists:
```

```
            role = user_doc.to_dict().get("role")
```

```
        elif db.collection("residents").document(uid).get().exists:
```

```
            role = "resident"
```

```
    decoded["role"] = str(role or "resident").strip().lower()
```

```
    return decoded
```

```

async def get_admin_uid(user: dict = Depends(get_current_user)) -> str:
    """Require that the user has role=admin and return UID."""
    uid = user.get("uid")
    role = user.get("role")
    if role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return uid

```

```

def require_permission(permission: str | list[str]):
    """
    Factory that returns a dependency requiring one or more permissions.
    Accepts either a single permission string or a list of permission strings.
    """

```

```

    async def dependency(user: dict = Depends(get_current_user)) -> str:
        uid = user.get("uid")
        role = user.get("role")

        # Prefer explicit claims if present
        permissions = user.get("permissions")

        # Force certain staff roles to always use JSON permissions
        if role in ("admin", "staff", "secretary", "treasurer", "sk", "dilig"):
            permissions = get_permissions(role)

```

```
# Residents fallback to JSON if token has no permissions
```

```
elif permissions is None:
```

```
    permissions = get_permissions(role)
```

```
# Handle single vs. multiple permissions
```

```
if isinstance(permission, list):
```

```
    if not any(permissions.get(p, False) for p in permission):
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_403_FORBIDDEN,
```

```
            detail=f"One of {permission} required"
```

```
        )
```

```
else:
```

```
    if not permissions.get(permission, False):
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_403_FORBIDDEN,
```

```
            detail=f"Permission '{permission}' required"
```

```
        )
```

```
    return uid
```

```
    return dependency
```

```
def set_user_role(uid: str, role: str):
```

```
    auth.set_custom_user_claims(uid, {"role": role})
```

```
    return {"uid": uid, "role": role}
```

```
```
```

```
backend\app\core\firebase.py
```

```
```python
```

```
import os, logging, json, firebase_admin
```

```
from firebase_admin import credentials, firestore, storage
```

```
from google.cloud.storage.bucket import Bucket
```

```
from google.cloud import exceptions
```

```
from datetime import timedelta
```

```
from urllib.parse import quote
```

```
from uuid import uuid4
```

```
logger = logging.getLogger("uvicorn.error")
```

```
def ensure_firebase_initialized() -> firebase_admin.App:
```

```
    try:
```

```
        app = firebase_admin.get_app()
```

```
        logger.debug("  Firebase already initialized. Options: %s", app.options.__dict__)
```

```
        return app
```

```
    except ValueError:
```

```
        bucket = os.environ.get("FIREBASE_STORAGE_BUCKET")
```

```
        if not bucket:
```



```
raise RuntimeError("❌ FIREBASE_STORAGE_BUCKET environment variable is not set")
```

```
# Case 1: Service account JSON injected directly into env var
```

```
service_account_json = os.environ.get("FIREBASE_SERVICE_ACCOUNT")
```

```
if service_account_json:
```

```
    logger.info("🔑 Using service account JSON from FIREBASE_SERVICE_ACCOUNT env var")
```

```
    cred = credentials.Certificate(json.loads(service_account_json))
```

```
    app = firebase_admin.initialize_app(cred, {"storageBucket": bucket})
```

```
else:
```

```
# Case 2: Fallback to GOOGLE_APPLICATION_CREDENTIALS file path (local dev)
```

```
cred_path = os.environ.get("GOOGLE_APPLICATION_CREDENTIALS")
```

```
if cred_path and os.path.exists(cred_path):
```

```
    logger.info("🔑 Using service account key at: %s", cred_path)
```

```
    cred = credentials.Certificate(cred_path)
```

```
    app = firebase_admin.initialize_app(cred, {"storageBucket": bucket})
```

```
else:
```

```
    logger.info("🔑 Using default application credentials")
```

```
    app = firebase_admin.initialize_app(options={"storageBucket": bucket})
```

```
logger.info("✅ Firebase initialized successfully with bucket: %s", bucket)
```

```
return app
```

```
def get_firestore() -> firestore.Client:
```

```
ensure_firebase_initialized()

return firestore.client()
```

```
def get_storage_bucket() -> Bucket:

    app = ensure_firebase_initialized()

    bucket_name = app.options.get("storageBucket")

    logger.info("🔍 Firebase app storageBucket option = %s", bucket_name)

    logger.info("🔍 Bucket repr = %r", bucket_name)

    if not bucket_name:

        raise RuntimeError("❌ Firebase app has no storageBucket configured")

    return storage.bucket(bucket_name)
```

```
def upload_file(file_obj, path: str, public: bool = True) -> dict:

    bucket = get_storage_bucket()

    blob = bucket.blob(path)

    try:

        file_obj.file.seek(0)

        content_type = getattr(file_obj, "content_type", "application/octet-stream")

        download_token = None

        if public:

            download_token = str(uuid4())

            blob.metadata = {
```

```
    "firebaseStorageDownloadTokens": download_token,  
}
```

```
blob.upload_from_file(file_obj.file, content_type=content_type)
```

```
if public:
```

```
    encoded_path = quote(path, safe="")
```

```
    url = (
```

```
        f"https://firebasestorage.googleapis.com/v0/b/{bucket.name}/o/"
```

```
        f"{encoded_path}?alt=media&token={download_token}"
```

```
    )
```

```
else:
```

```
    url = blob.generate_signed_url(expiration=timedelta(hours=24))
```

```
logger.info(" 📁 Uploaded file %s → %s", getattr(file_obj, "filename", "<unknown>"), url)
```

```
return {"url": url, "path": path}
```

```
except exceptions.GoogleCloudError as gce:
```

```
    logger.error(" ❌ Google Cloud error uploading file %s: %s", getattr(file_obj, "filename",  
"<unknown>"), gce)
```

```
    raise RuntimeError("File upload failed") from gce
```

```
except Exception as e:
```

```
    logger.exception(" ❌ Unexpected error uploading file %s: %s", getattr(file_obj,  
"filename", "<unknown>"), e)
```

```
    raise RuntimeError("File upload failed") from e
```

```

def delete_file(path: str) -> None:

    bucket = get_storage_bucket()

    blob = bucket.blob(path)

    try:

        blob.delete()

        logger.info("🗑 Deleted file at path=%s", path)

    except exceptions.NotFound:

        logger.warning("⚠ File not found in storage: %s", path)

    except exceptions.GoogleCloudError as gce:

        logger.error("❌ Google Cloud error deleting file %s: %s", path, gce)

        raise RuntimeError("File deletion failed") from gce

    except Exception as e:

        logger.exception("❌ Unexpected error deleting file %s: %s", path, e)

        raise RuntimeError("File deletion failed") from e

    ...

```

```

## backend\app\core\roles.py

```

```

```python

```

```

backend/app/core/roles.py

```

```

import json

```

```

from pathlib import Path

```

```
from typing import Dict
```

```
CONFIG_PATH = Path(__file__).resolve().parents[3] / "config" / "role_permissions.json"
```

```
def load_role_permissions(path: Path = CONFIG_PATH) -> Dict[str, Dict[str, bool]]:
```

```
 with open(path, "r", encoding="utf-8") as f:
```

```
 overrides = json.load(f)
```

```
 # Collect all permissions mentioned across roles
```

```
 all_perms = {perm for keys in overrides.values() for perm in keys}
```

```
 # Build full map: each role gets True/False for every permission
```

```
 role_maps = {
```

```
 role: {perm: perm in keys for perm in all_perms}
```

```
 for role, keys in overrides.items()
```

```
 }
```

```
 return role_maps, all_perms
```

```
ROLE_PERMISSIONS, ALL_PERMISSIONS = load_role_permissions()
```

```
def get_permissions(role: str) -> Dict[str, bool]:
```

```
 role = role.lower().strip()
```

```
 return ROLE_PERMISSIONS.get(role, {perm: False for perm in ALL_PERMISSIONS})
```

```
```
```

```
## backend\app\core\websocket_manager.py
```

```
```python
```

```
import logging
```

```
from typing import Dict
```

```
from fastapi import WebSocket
```

```
from fastapi.encoders import jsonable_encoder
```

```
from starlette.websockets import WebSocketState
```

```
logger = logging.getLogger("uvicorn.error")
```

```
class ConnectionManager:
```

```
 def __init__(self):
```

```
 # Map WebSocket -> user info (role, user_id, auth_method, etc.)
```

```
 self.active_connections: Dict[WebSocket, dict] = {}
```

```
 async def connect(self, websocket: WebSocket, user_info: dict):
```

```
 """
```

```
 Accept and store a new WebSocket connection with user metadata.
```

```
 user_info should include: uid, role, user_id, and optionally auth_method.
```

```
 """
```

```
 if websocket.client_state == WebSocketState.CONNECTING:
```

```
 await websocket.accept()
```

```
 self.active_connections[websocket] = user_info
```

```

logger.info(
 " 🍷 WebSocket connected (role=%s, user_id=%s, auth=%s). Total connections: %d",
 user_info.get("role"),
 user_info.get("user_id"),
 user_info.get("auth_method", "unknown"),
 len(self.active_connections),
)

```

```

def disconnect(self, websocket: WebSocket):
 """Remove a WebSocket connection."""
 if websocket in self.active_connections:
 info = self.active_connections.pop(websocket, None)
 logger.info(
 " ❌ WebSocket disconnected (role=%s, user_id=%s, auth=%s). Total connections: %d",
 info.get("role") if info else None,
 info.get("user_id") if info else None,
 info.get("auth_method") if info else None,
 len(self.active_connections),
)

```

```

async def send_personal_message(self, message: dict, websocket: WebSocket):
 """Send a message to a specific WebSocket client."""
 try:
 await websocket.send_json(jsonable_encoder(message))
 except Exception as e:

```

```
logger.error(" ⚠ Failed to send personal message: %s", e)
```

```
async def broadcast(self, message: dict, role: str = None, user_id: str = None):
```

```
 """
```

```
 Broadcast a message to all connected clients.
```

```
 Optionally filter by role and/or user_id.
```

```
 """
```

```
 logger.debug(
```

```
 " 📡 Broadcasting message (role=%s, user_id=%s) to %d clients: %s",
```

```
 role,
```

```
 user_id,
```

```
 len(self.active_connections),
```

```
 message,
```

```
)
```

```
 for connection, info in list(self.active_connections.items()):
```

```
 try:
```

```
 if role and info.get("role") != role:
```

```
 continue
```

```
 if user_id and info.get("user_id") != user_id:
```

```
 continue
```

```
 await connection.send_json(jsonable_encoder(message))
```

```
 except Exception as e:
```

```
 logger.error(" ⚠ Failed to send message to client: %s", e)
```

```
 self.disconnect(connection)
```



```
manager = ConnectionManager()
```

```
...
```

```
backend\app\main.py
```

```
```python
```

```
import os
```

```
import logging
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
from backend.app.config import settings
```

```
from contextlib import asynccontextmanager
```

```
from fastapi import FastAPI, Request
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from fastapi.encoders import ENCODERS_BY_TYPE
```

```
from backend.app.core.firebase import ensure_firebase_initialized
```

```
from backend.app.routes import (
```

```
    resident_routes,
```

```
    dashboard,
```

```
    business_routes,
```

```
    payment_routes,
```

```
paymongo_routes,  
document_routes,  
incident_routes,  
complaint_routes,  
account_routes,  
audit_routes,  
fee_routes,  
disbursement_routes,  
role_routes,  
password_routes,  
ws_routes,  
notification_routes,  
)
```

```
logger = logging.getLogger("barangay")
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
    # Startup logic
```

```
    try:
```

```
        bucket_env = os.environ.get("FIREBASE_STORAGE_BUCKET")
```

```
        logger.info("🔍 FIREBASE_STORAGE_BUCKET env var = %s", bucket_env)
```

```
        logger.info("🚀 Initializing Firebase...")
```

```
        ensure_firebase_initialized()
```

```
    except Exception as e:
```

```
logger.error(f" ❌ Firebase initialization failed: {e}")
```

```
raise
```

```
yield # <-- app runs here
```

```
# Shutdown logic
```

```
logger.info(" 🛑 Shutting down Barangay API")
```

```
def create_app() -> FastAPI:
```

```
    app = FastAPI(
```

```
        title="Barangay Information System API",
```

```
        version="1.0.0",
```

```
        docs_url="/docs",
```

```
        redoc_url="/redoc",
```

```
        lifespan=lifespan, # ✅ attach lifespan handler
```

```
        root_path="/", # optional
```

```
        proxy_headers=True
```

```
    )
```

```
# 🌐 CORS Configuration
```

```
cors_env = os.environ.get("CORS_ORIGINS", "")
```

```
allowed_origins = [origin.strip() for origin in cors_env.split(";") if origin] or [
```

```
    "http://localhost:3000",
```

```
    "https://barangay-1721d.web.app",
```

```
]
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=allowed_origins,  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

📦 Route Registration

```
api_prefix = "/api"  
  
app.include_router(resident_routes.router, prefix=api_prefix, tags=["Residents"])  
  
app.include_router(document_routes.router, prefix=f"{api_prefix}/documents",  
tags=["Documents"])  
  
app.include_router(incident_routes.router, prefix=f"{api_prefix}/incidents",  
tags=["Incidents"])  
  
app.include_router(complaint_routes.router, prefix=f"{api_prefix}/complaints",  
tags=["Complaints"])  
  
app.include_router(account_routes.router, prefix=f"{api_prefix}", tags=["Accounts"])  
  
app.include_router(audit_routes.router, prefix=f"{api_prefix}/document_audit",  
tags=["Audit"])  
  
app.include_router(dashboard.router, prefix="/dashboard", tags=["Dashboard"])  
  
app.include_router(business_routes.router, prefix=api_prefix, tags=["Business"])  
  
app.include_router(payment_routes.router, prefix=api_prefix, tags=["Payments"])  
  
app.include_router(paymongo_routes.router, prefix=f"{api_prefix}/paymongo",  
tags=["PayMongo"])  
  
app.include_router(fee_routes.router, prefix=f"{api_prefix}", tags=["Fees"])  
  
app.include_router(disbursement_routes.router, prefix=f"{api_prefix}",  
tags=["Disbursements"])
```

```
app.include_router(role_routes.router, prefix=f"{api_prefix}", tags=["Roles"])

app.include_router(password_routes.router, prefix=f"{api_prefix}", tags=["Password
Reset"])

app.include_router(ws_routes.router, tags=["websocket"])

app.include_router(notification_routes.router, prefix=f"{api_prefix}",
tags=["notifications"])
```

🩺 Health Check

```
@app.get(f"{api_prefix}/status", tags=["Health"])

def status():

    return {"status": "ok", "message": "API is running"}
```

📄 Request Logger

```
@app.middleware("http")

async def log_requests(request: Request, call_next):

    if "paymongo" not in request.url.path:

        body = await request.body()

        if request.headers.get("content-type", "").startswith("application/json"):

            try:

                body_text = body.decode("utf-8")

            except UnicodeDecodeError:

                body_text = "<invalid utf-8>"

            else:

                body_text = f"<non-text body, length={len(body)}>"

            logger.info("📦 %s %s → %s", request.method, request.url.path, body_text or
"<empty>")

        return await call_next(request)
```

```
return app
```

```
# 🌀 Entrypoint for Uvicorn
```

```
app = create_app()
```

```
# 🛠 Override FastAPI's default bytes encoder
```

```
ENCODERS_BY_TYPE[bytes] = lambda o: "<binary data>"
```

```
...
```

```
## backend\app\models\__init__.py
```

```
```python
```

```
from .resident import ResidentCreate, ResidentUpdate, ResidentOut
```

```
...
```

```
backend\app\models\account.py
```

```
```python
```

```
from pydantic import BaseModel, EmailStr, Field
```

```
from enum import Enum
```

```
from typing import Optional
```

```
from datetime import datetime
```

🎯 Role definitions

```
class RoleEnum(str, Enum):
```

```
    staff = "staff"
```

```
    secretary = "secretary"
```

```
    treasurer = "treasurer"
```

```
    sk = "sk"
```

```
    dilg = "dilg"
```

```
    admin = "admin"
```

📄 Base account schema

```
class AccountBase(BaseModel):
```

```
    email: EmailStr
```

```
    full_name: str = Field(..., min_length=2, max_length=100)
```

```
    role: RoleEnum
```

🆕 Account creation schema

```
class AccountCreate(AccountBase):
```

```
    password: str = Field(..., min_length=8, max_length=128)
```

📡 Account response schema

```
class AccountResponse(AccountBase):
```

```
    uid: str
```

```
    created_by: str
```

```
    created_at: datetime
```

```
    updated_at: Optional[datetime] = None
```

🛠️ Firestore payload schema

```
class AccountFirestorePayload(AccountBase):  
    created_by: str = Field(..., alias="createdBy")  
    created_at: datetime = Field(..., alias="createdAt")  
    updated_at: Optional[datetime] = Field(None, alias="updatedAt")
```

```
class Config:  
    validate_by_name = True  
    json_encoders = {  
        datetime: lambda v: v.isoformat()  
    }
```

⬅️ Account update schema

```
class AccountUpdate(BaseModel):  
    full_name: Optional[str] = Field(None, min_length=2, max_length=100)  
    role: Optional[RoleEnum] = None  
    updated_at: datetime = Field(default_factory=datetime.utcnow)
```

```

## backend\app\models\business.py

```python

from pydantic import BaseModel, Field

from typing import Optional, Dict


```
class BusinessDetails(BaseModel):
```

```
    name: str
```

```
    type: str
```

```
    barangay: str
```

```
    address: str
```

```
    registration_date: str
```

```
class BusinessDocuments(BaseModel):
```

```
    valid_id: str
```

```
    proof_of_address: str
```

```
    dti_cert: Optional[str] = None
```

```
    business_logo: Optional[str] = None
```

```
class BusinessApplication(BaseModel):
```

```
    owner_uid: str
```

```
    owner_name: str
```

```
    contact_number: str
```

```
    email: str
```

```
    business: BusinessDetails
```

```
    documents: BusinessDocuments
```

```
    ``
```

```
## backend\app\models\complaint.py
```

```
``python
```

```
from pydantic import BaseModel, Field, StringConstraints
```

```
from typing import Optional, Annotated
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
# 🎯 Controlled vocabularies
```

```
class ComplaintCategory(str, Enum):
```

```
    noise = "Noise"
```

```
    service = "Service"
```

```
    neighbor = "Neighbor"
```

```
    other = "Other"
```

```
class ComplaintStatus(str, Enum):
```

```
    open = "open"
```

```
    in_review = "in_review" # ✅ underscore for consistency
```

```
    resolved = "resolved"
```

```
# 📋 Base complaint schema
```

```
class ComplaintBase(BaseModel):
```

```
    category: ComplaintCategory
```

```
    description: Annotated[str, StringConstraints(min_length=5, max_length=500)]
```

```
    location: str
```

```
    filed_by: Annotated[str, StringConstraints(min_length=28, max_length=28)] = Field(
```

```
        ..., description="UID of user who entered the complaint (resident self-filing or  
staff/admin proxy)"
```

```
)
```

```

    filed_for: Optional[Annotated[str, StringConstraints(min_length=28, max_length=28)]] =
Field(

    None, description="Resident UID the complaint is about (defaults to filed_by if resident
self-filing)"

)

```

 Complaint creation schema

```
class ComplaintCreate(ComplaintBase):
```

```
    """
```

Used when a complaint is filed.

- filed_by: who entered the complaint (resident or staff/admin)

- filed_for: resident the complaint is about (optional if resident self-filing)

Timestamp is set by backend (Firestore SERVER_TIMESTAMP).

```
    """
```

```
    pass
```

 Complaint response schema

```
class Complaint(BaseModel):
```

```
    id: str
```

```
    category: ComplaintCategory
```

```
    description: str
```

```
    location: str
```

```
    filed_by: str
```

```
    filed_for: Optional[str] = None
```

```
    timestamp: datetime
```

```
    status: ComplaintStatus = ComplaintStatus.open
```

resolution_notes: Optional[str] = None

updated_at: Optional[datetime] = None

class Config:

validate_by_name = True

from_attributes = True #  allows dict/ORM integration

@classmethod

def from_firestore(cls, snapshot) -> "Complaint":

"""

Helper to instantiate from Firestore DocumentSnapshot.

Ensures Firestore timestamps are converted to Python datetime.

"""

data = snapshot.to_dict() or {}

if "timestamp" in data and hasattr(data["timestamp"], "to_datetime"):

data["timestamp"] = data["timestamp"].to_datetime()

if "updated_at" in data and hasattr(data["updated_at"], "to_datetime"):

data["updated_at"] = data["updated_at"].to_datetime()

return cls(id=snapshot.id, **data)

def to_dict(self) -> dict:

"""

Convert to dict for Firestore writes, excluding None values.

"""

return self.dict(exclude_none=True)

```
# 🗨️ Complaint with resident + filer details
```

```
class ComplaintWithResident(Complaint):
```

```
    filed_for_name: str = Field(..., description="Resident full name for display")
```

```
    filed_by_name: Optional[str] = Field(None, description="Name of user who filed  
(staff/admin or resident)")
```

```
    ...
```

```
## backend\app\models\document.py
```

```
```python
```

```
from pydantic import BaseModel, Field
```

```
from typing import Optional, Dict, Any
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
class DocumentStatus(str, Enum):
```

```
 pending = "pending"
```

```
 for_payment = "for_payment"
```

```
 payment_submitted = "payment_submitted"
```

```
 paid = "paid"
```

```
 approved = "approved"
```

```
 rejected = "rejected"
```

```
class Attachment(BaseModel):
```

```
 url: str = Field(..., description="Public or signed URL to the uploaded file")
```

```
path: str = Field(..., description="Storage path inside Firebase bucket")
```

```
class Document(BaseModel):
```

```
 # 🔑 Firestore document ID
```

```
 id: str = Field(..., description="Firestore auto-generated document ID")
```

```
 # 🆔 Human-readable sequential ID
```

```
 documentId: str = Field(..., description="Type-based sequential identifier, e.g.
Barangay_Clearance-0001")
```

```
 # 👤 Resident info
```

```
 residentId: str = Field(..., description="Resident ID who requested the document")
```

```
 residentName: Optional[str] = Field(None, description="Full name of the resident")
```

```
 authUid: Optional[str] = Field(None, description="Auth UID if available")
```

```
 # 📄 Document details
```

```
 documentType: str = Field(..., description="Type of document requested")
```

```
 purpose: Optional[str] = Field(None, description="Purpose of the document")
```

```
 remarks: Optional[str] = Field(None, description="Remarks from secretary/admin")
```

```
 # ⬅️ Status lifecycle
```

```
 status: DocumentStatus = Field(..., description="Current status of the document")
```

```
 resubmitted: Optional[bool] = Field(False, description="Whether a rejected document
was resubmitted")
```

```
 # 🕒 Timestamps
```

```
createdAt: datetime = Field(..., description="When the document was created")
updatedAt: datetime = Field(..., description="When the document was last updated")
issuedAt: Optional[datetime] = Field(None, description="When the document was issued")
```

# 📎 Attachments (now objects with url + path)

```
attachments: Optional[Dict[str, Attachment]] = Field(
 None,
 description="Uploaded file metadata including URL and storage path"
)
```

# 💵 Payment info

```
amount: Optional[int] = Field(None, description="Payment amount if required")
paymentStatus: Optional[str] = Field(None, description="Payment status string")
referenceNumber: Optional[str] = Field(None, description="Payment reference number")
paymentIntentId: Optional[str] = Field(None, description="PayMongo Payment Intent ID")
transactionId: Optional[str] = Field(None, description="PayMongo Transaction ID")
```

# 📄 Issuance info

```
issuedBy: Optional[str] = Field(None, description="Secretary/Admin who issued the document")
fileUrl: Optional[str] = Field(None, description="URL to the issued document file")
```

# ✂ Flexible extra fields for type-specific data

```
extraFields: Optional[Dict[str, Any]] = Field(
 None,
```

```

 description="Additional fields depending on document type"
)

Common extras
occupation: Optional[str] = Field(None, description="Resident occupation")
voterStatus: Optional[str] = Field(None, description="Resident voter status")

...

backend\app\models\fee.py

```python
from pydantic import BaseModel, Field
from typing import Optional

# -----
# 🗝 Base Models
# -----

class BaseFee(BaseModel):
    """Common fields shared by all fee types."""
    fee: int = Field(..., ge=0, description="Base/base fee amount")
    enabled: bool = Field(default=True, description="Enable/disable this fee")
    miscType: Optional[str] = Field(
        default=None,
        description="Reference to a miscellaneous fee type (from misc_fees)"
    )

```



```
# -----
```

```
# 📄 Document Fee Models
```

```
# -----
```

```
class DocumentFee(BaseFee):
```

```
    """Update model for existing document fees."""
```

```
    documentType: Optional[str] = Field(None, min_length=1, description="Type of document")
```

```
class NewDocumentFee(BaseFee):
```

```
    """Creation model for new document fees."""
```

```
    documentType: str = Field(..., min_length=1, description="Type of document")
```

```
# -----
```

```
# 🏢 Business Fee Models
```

```
# -----
```

```
class BusinessFee(BaseFee):
```

```
    """Update model for existing business fees."""
```

```
    registrationFee: Optional[int] = Field(default=None, ge=0, description="Registration fee")
```

```
    annualFee: Optional[int] = Field(default=None, ge=0, description="Annual fee")
```

```
    businessType: Optional[str] = Field(None, min_length=1, description="Type of business")
```

```
class NewBusinessFee(BusinessFee):
```

```
    """Creation model for new business fees."""
```

```
    businessType: str = Field(..., min_length=1, description="Type of business")
```

```
# -----
```

```
#  Miscellaneous Fee Models
```

```
# -----
```

```
class MiscFee(BaseModel):
```

```
    """Update model for miscellaneous fees."""
```

```
    fee: int = Field(..., ge=0, description="Miscellaneous fee amount")
```

```
    enabled: bool = Field(default=True, description="Enable/disable this fee")
```

```
class NewMiscFee(MiscFee):
```

```
    """Creation model for new miscellaneous fees."""
```

```
    miscType: str = Field(..., min_length=1, description="Type of miscellaneous fee")
```

```
...
```

```
## backend\app\models\incident.py
```

```
```python
```

```
from pydantic import BaseModel, Field, StringConstraints
```

```
from typing import Optional, Annotated
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
 Controlled vocabularies
```

```
class IncidentType(str, Enum):
```

```
 theft = "Theft"
```

```
 dispute = "Dispute"
```

```
accident = "Accident"
```

```
other = "Other"
```

```
class IncidentStatus(str, Enum):
```

```
 pending = "pending"
```

```
 resolved = "resolved"
```

```
 escalated = "escalated"
```

```
📦 Base incident schema
```

```
class IncidentBase(BaseModel):
```

```
 type: IncidentType
```

```
 description: Annotated[str, StringConstraints(min_length=5, max_length=500)]
```

```
 location: str
```

```
 authUid: Optional[str] = Field(
```

```
 None, description="UID of the user (resident or staff) who logged the incident"
```

```
)
```

```
 residentId: Optional[str] = Field(
```

```
 None, description="Resident UID who is the subject of the incident"
```

```
)
```

```
🆕 Incident creation schema
```

```
class IncidentCreate(IncidentBase):
```

```
 authUid: str = Field(..., description="UID of the user creating the incident")
```

```
 residentId: str = Field(..., description="Resident UID involved in the incident")
```

```
 timestamp: Optional[datetime] = None
```

# 🚒 Incident response schema

```
class Incident(BaseModel):
```

```
 id: str
```

```
 type: IncidentType
```

```
 description: str
```

```
 location: str
```

```
 authUid: Optional[str] = None
```

```
 residentId: Optional[str] = None
```

```
 timestamp: datetime
```

```
 updated_at: Optional[datetime] = None
```

```
 status: IncidentStatus = IncidentStatus.pending
```

```
class Config:
```

```
 validate_by_name = True
```

```
 json_encoders = {
```

```
 datetime: lambda v: v.isoformat() if v else None
```

```
 }
```

# 🚒 Incident with enriched display details

```
class IncidentWithResident(Incident):
```

```
 reported_by_name: Optional[str] = Field(None, description="Resident full name")
```

```
 logged_by_officer: Optional[str] = Field(None, description="Officer full name")
```

```
 assigned_to_name: Optional[str] = Field(None, description="Assigned staff name")
```

```
...
```

```
backend\app\models\notification.py
```

```
```python
```

```
from pydantic import BaseModel, Field
```

```
from typing import Optional, Literal
```

```
from datetime import datetime, timezone
```

```
import uuid
```

```
class Notification(BaseModel):
```

```
    id: str = Field(default_factory=lambda: str(uuid.uuid4())) # unique per instance
```

```
    # Who should see this notification
```

```
    role: Literal["admin", "staff", "secretary", "treasurer", "resident"]
```

```
    # What type of event triggered it
```

```
    type: Literal[
```

```
        "login", "logout",
```

```
        "incident", "incident_update",
```

```
        "complaint", "complaint_update",
```

```
        "business", "business_update",
```

```
        "document", "document_update",
```

```
        "payment", "payment_update"
```

```
    ]
```

```
    # Scope clarifies login/logout context
```

```
    scope: Optional[Literal["resident", "officer"]] = None
```

Aggregated count (e.g., 3 residents logged in)

count: Optional[int] = None

Officer/staff name if applicable

user: Optional[str] = None

Resident UID for personal notifications

user_id: Optional[str] = None

Human-readable message

message: str

Timestamp defaults to UTC now

timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

Whether the notification has been read

read: bool = False

...

backend\app\models\password.py

```python

from pydantic import BaseModel, Field, field\_validator, StringConstraints, ValidationInfo

```
from typing import Annotated, Optional
```

```
🗝 Strong password type
```

```
PasswordStr = Annotated[str, StringConstraints(min_length=8, max_length=128)]
```

```
📧 Request model
```

```
class ResetRequest(BaseModel):
```

```
 email: str
```

```
class ResetApply(BaseModel):
```

```
 token: str
```

```
 new_password: PasswordStr
```

```
 confirm_password: str
```

```
@field_validator("new_password")
```

```
def validate_password(cls, value: str):
```

```
 if not any(c.islower() for c in value):
```

```
 raise ValueError("Password must contain a lowercase letter")
```

```
 if not any(c.isupper() for c in value):
```

```
 raise ValueError("Password must contain an uppercase letter")
```

```
 if not any(c.isdigit() for c in value):
```

```
 raise ValueError("Password must contain a digit")
```

```
 if not any(c in "!@#$%^&*()-_+=[]{};:.,<>?/\\" for c in value):
```

```
 raise ValueError("Password must contain a special character")
```

```
 return value
```

```
@field_validator("confirm_password")
def passwords_match(cls, v: str, info: ValidationInfo):
 new_password = info.data.get("new_password")

 if new_password and v != new_password:
 raise ValueError("Passwords do not match")

 return v
```

# 🏠 Address model

```
class Address(BaseModel):
 barangay: Optional[str] = None
 city: Optional[str] = None
 province: Optional[str] = None
 street: Optional[str] = None
 house_number: Optional[str] = Field(default=None, alias="houseNumber")
 purok: Optional[str] = None
 zip_code: Optional[str] = Field(default=None, alias="zipCode")
```

# 👤 Unified user model

```
class UserOut(BaseModel):
 uid: str
 email: str

 full_name: str = Field(alias="fullName") # ✅ maps both fullName (resident) and
full_name (account)

 role: Optional[str] = None
 barangay: Optional[str] = None
```



```
address: Optional[Address] = None
```

```
...
```

```
backend\app\models\payment.py
```

```
```python
```

```
from pydantic import BaseModel
```

```
class PaymentInit(BaseModel):
```

```
    business_id: str
```

```
    amount: int
```

```
...
```

```
## backend\app\models\paymongo.py
```

```
```python
```

```
from pydantic import BaseModel, Field
```

```
from typing import Optional, Dict
```

```

```

```
Request Models
```

```

```

```
class DocumentPaymentRequest(BaseModel):
```

documentId: str

documentType: str # e.g. "Barangay Clearance"

remarks: str = ""

class BusinessPaymentRequest(BaseModel):

businessId: str

businessType: str # e.g. "Retail Store"

feeType: str # "registrationFee" or "annualFee"

remarks: str = ""

class BillingInfo(BaseModel):

name: str = Field(..., description="Resident's full name")

email: str = Field(..., description="Resident's email address")

class AttachPaymentRequest(BaseModel):

paymentIntentId: str = Field(..., description="Payment Intent ID from PayMongo")

paymongoClientKey: str = Field(..., description="Client key from PayMongo intent creation")

method: str = Field(..., description="Payment method type (e.g., 'gcash', 'grab\_pay')")

billing: BillingInfo = Field(..., description="Billing information for the resident")

type: str = Field(..., description="business or document")

return\_url: Optional[str] = Field(  
 None,  
 description="URL to redirect after payment success/failure"  
)

```
` ``
```

```
backend\app\models\resident.py
```

```
` `` python
```

```
from pydantic import BaseModel, Field, EmailStr, HttpUrl, StringConstraints, ConfigDict
```

```
from typing import Optional, Annotated
```

```
from datetime import date, datetime
```

```
from enum import Enum
```

```
🎯 Controlled vocabularies
```

```
class Gender(str, Enum):
```

```
 Male = "Male"
```

```
 Female = "Female"
```

```
 Other = "Other"
```

```
class CivilStatus(str, Enum):
```

```
 Single = "Single"
```

```
 Married = "Married"
```

```
 Widowed = "Widowed"
```

```
 Separated = "Separated"
```

```
class VoterStatus(str, Enum):
```

```
 Yes = "yes"
```

```
 No = "no"
```

```
 Unknown = "unknown"
```

# 🖐️ Fingerprints model

```
class Fingerprints(BaseModel):
```

```
 left: Optional[str] = Field(None, alias="left")
```

```
 right: Optional[str] = Field(None, alias="right")
```

# 🏠 Address model

```
class Address(BaseModel):
```

```
 model_config = ConfigDict(populate_by_name=True)
```

```
 house_number: str = Field(..., alias="houseNumber", example="123")
```

```
 street: str = Field(..., example="Main St")
```

```
 purok: str = Field(..., example="3")
```

```
 barangay: str = Field(..., example="Moonwalk")
```

```
 city: str = Field(..., example="Parañaque")
```

```
 province: str = Field(..., example="Metro Manila")
```

```
 zip_code: Optional[str] = Field(None, alias="zipCode", example="1700")
```

# 🗳️ Resident creation model

```
class ResidentCreate(BaseModel):
```

```
 model_config = ConfigDict(populate_by_name=True)
```

```
 full_name: str = Field(..., min_length=2, max_length=100, alias="fullName")
```

```
 middle_name: Optional[str] = Field(None, alias="middleName")
```

```
 suffix: Optional[str] = Field(None, alias="suffix")
```

```
 birth_date: date = Field(..., alias="birthDate")
```

```

gender: Gender

civil_status: CivilStatus = Field(..., alias="civilStatus")

contact_number: Annotated[str, StringConstraints(pattern=r"^\d{9}$")] = Field(...,
alias="contactNumber", example="09171234567")

email: Optional[EmailStr]

address: Address

household_id: Optional[str] = Field(None, alias="householdId")

is_head_of_family: bool = Field(..., alias="isHeadOfFamily")

voter_status: VoterStatus = Field(..., alias="voterStatus")

occupation: Optional[str]

photo_url: Optional[str] = Field(None, alias="photoUrl")

fingerprints: Optional[Fingerprints] = Field(None, alias="fingerprints") # ✅ nested object

signature_url: Optional[str] = Field(None, alias="signatureUrl")

remarks: Optional[str] = None

```

# 📦 Resident output model

```
class ResidentOut(BaseModel):
```

```

 model_config = ConfigDict(
 populate_by_name=True,
 json_encoders={
 datetime: lambda v: v.isoformat() if v else None,
 date: lambda v: v.isoformat() if v else None,
 },
)

```

```
id: str
```

```
full_name: str = Field(..., alias="fullName")
birth_date: Optional[date] = Field(None, alias="birthDate")
gender: Optional[Gender]
civil_status: Optional[CivilStatus] = Field(None, alias="civilStatus")
contact_number: Optional[str] = Field(None, alias="contactNumber")
email: Optional[EmailStr]
address: Optional[Address]
household_id: Optional[str] = Field(None, alias="householdId")
is_head_of_family: Optional[bool] = Field(False, alias="isHeadOfFamily")
voter_status: Optional[VoterStatus] = Field(None, alias="voterStatus")
occupation: Optional[str]
photo_url: Optional[str] = Field(None, alias="photoUrl")
fingerprints: Optional[Fingerprints] = Field(None, alias="fingerprints")
signature_url: Optional[str] = Field(None, alias="signatureUrl")
remarks: Optional[str]
created_at: Optional[datetime] = Field(None, alias="createdAt")
updated_at: Optional[datetime] = Field(None, alias="updatedAt")
```

#  Partial update model

```
class ResidentUpdate(BaseModel):
```

```
 model_config = ConfigDict(populate_by_name=True)

 full_name: Optional[str] = Field(None, alias="fullName")
 middle_name: Optional[str] = Field(None, alias="middleName")
 suffix: Optional[str] = Field(None, alias="suffix")
 birth_date: Optional[date] = Field(None, alias="birthDate")
```

```

gender: Optional[Gender]

civil_status: Optional[CivilStatus] = Field(None, alias="civilStatus")

contact_number: Optional[Annotated[str, StringConstraints(pattern=r"^09\d{9}$")]] =
Field(None, alias="contactNumber", example="09171234567")

email: Optional[EmailStr]

address: Optional[Address]

household_id: Optional[str] = Field(None, alias="householdId")

is_head_of_family: Optional[bool] = Field(None, alias="isHeadOfFamily")

voter_status: Optional[VoterStatus] = Field(None, alias="voterStatus")

occupation: Optional[str]

photo_url: Optional[str] = Field(None, alias="photoUrl")

fingerprints: Optional[Fingerprints] = Field(None, alias="fingerprints") # ✅ nested object

signature_url: Optional[str] = Field(None, alias="signatureUrl")

remarks: Optional[str]

updated_at: datetime = Field(default_factory=datetime.utcnow, alias="updatedAt")

```

```

```
## backend\app\models\role.py
```

```
```python
```

```
from pydantic import BaseModel, field_validator
```

```
ALLOWED_ROLES = {"admin", "staff", "secretary", "treasurer", "sk", "dilg", "resident"}
```

```
class RoleUpdate(BaseModel):
```

```
role: str
```

```
@field_validator("role")
```

```
def validate_role(cls, v: str) -> str:
```

```
 if v not in ALLOWED_ROLES:
```

```
 raise ValueError(f"Invalid role: {v}")
```

```
 return v
```

```
class RoleResponse(BaseModel):
```

```
 uid: str
```

```
 role: str
```

```
 ...
```

```
backend\app\models\settings.py
```

```
```python
```

```
from pydantic import BaseModel, Field, condecimal
```

```
from typing import Dict
```

```
from typing import Annotated
```

```
# Define a reusable constrained type
```

```
FeeAmount = Annotated[condecimal(gt=0, lt=10000), Field(...)]
```



```

class RolePermission(BaseModel):
    role: str = Field(
        ...,
        example="staff",
        description="Role name to assign permissions to (e.g., admin, staff, resident)"
    )
    permissions: Dict[str, bool] = Field(
        ...,
        example={
            "viewDashboard": True,
            "fileComplaints": True,
            "manageResidents": False
        },
        description="Full permission map for the role. Keys must match known permission
identifiers."
    )

```

```

class DocumentFee(BaseModel):
    document_type: str = Field(
        ...,
        example="barangay_clearance",
        description="Type of document (e.g., barangay_clearance, certificate_of_indigency)"
    )
    fee: FeeAmount = Field(
        ...,
        example="50.0", # Pydantic prefers Decimal-compatible strings here
    )

```

```

        description="Fee amount in PHP (must be greater than 0 and less than 10,000)"
    )

    ...

## backend\app\routes\__init__.py

```python
...

backend\app\routes\account_routes.py

```python
import logging

from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel

from backend.app.models.account import AccountCreate, AccountResponse, RoleEnum
from backend.app.services.account_service import (
    create_barangay_account,
    delete_barangay_account,
    update_user_role,
    list_barangay_accounts,
)

from backend.app.core.auth import get_admin_uid, get_current_user, require_permission

```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(tags=["Accounts"])
```

```
# =====
```

```
# 📦 Response Models
```

```
# =====
```

```
class ActionResponse(BaseModel):
```

```
    detail: str
```

```
class RoleUpdatePayload(BaseModel):
```

```
    role: RoleEnum
```

```
# =====
```

```
# 🔧 Helper: wrap service calls
```

```
# =====
```

```
async def safe_service_call(service_func, *args, **kwargs):
```

```
    try:
```

```
        return await service_func(*args, **kwargs)
```

```
    except ValueError as ve:
```

```
        logger.warning("⚠️ Conflict: %s", str(ve))
```

```
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(ve))
```

```
    except PermissionError as pe:
```

```

        logger.warning("🚫 Permission denied: %s", str(pe))
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail=str(pe))
except HTTPException as he:
    logger.error("❌ HTTP error: %s", he.detail)
    raise he
except Exception as e:
    logger.exception("❌ Unexpected error")
    msg = str(e)
    if isinstance(msg, (dict, list)):
        msg = "; ".join(map(str, msg))
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"Operation failed: {msg}",
    )

```

```
# =====
```

```
# 🚀 Routes
```

```
# =====
```

```

@router.post(
    "/admin/create-account",
    response_model=AccountResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create a new barangay account",
    description="Accessible only to admins. Creates a new account with role-based access."
)

```

```

)

async def create_account_handler(
    payload: AccountCreate,
    admin_uid: str = Depends(get_admin_uid),
    _: None = Depends(require_permission("createAccount")),
) -> AccountResponse:

    logger.info("🚀 Account creation requested by admin: %s", admin_uid)

    account = await safe_service_call(create_barangay_account, payload,
    created_by=admin_uid)

    logger.info("✅ Account created successfully: %s", account.uid)

    return account

```

```

@router.delete(
    "/admin/delete-account/{uid}",
    response_model=ActionResponse,
    status_code=status.HTTP_200_OK,
    summary="Delete a barangay account",
    description="Accessible only to admins. Deletes both Firestore and Firebase Auth user."
)

async def delete_account_handler(
    uid: str,
    admin_uid: str = Depends(get_admin_uid),
    _: None = Depends(require_permission("deleteAccount")),
) -> ActionResponse:

    logger.info("🗑️ Account deletion requested by admin: %s for user: %s", admin_uid, uid)

```

```
await safe_service_call(delete_barangay_account, uid, deleted_by=admin_uid)

logger.info("✅ Account deleted successfully: %s", uid)

return ActionResult(detail=f"Account {uid} deleted successfully")
```

```
@router.put(
    "/admin/update-role/{uid}",
    response_model=AccountResponse,
    status_code=status.HTTP_200_OK,
    summary="Update a user's role",
    description="Accessible only to admins. Updates role in Firestore, Firebase Auth claims, and logs the change."
)

async def update_role_handler(
    uid: str,
    payload: RoleUpdatePayload,
    admin_uid: str = Depends(get_admin_uid),
    _: None = Depends(require_permission("updateRole")),
) -> AccountResponse:
    logger.info("👉 Role update requested by admin: %s for user: %s", admin_uid, uid)

    account = await safe_service_call(update_user_role, uid, payload.role,
    changed_by=admin_uid)

    logger.info("✅ Role updated to %s for UID: %s", payload.role, uid)

    return account

@router.get(
```

```

"/admin/accounts",
response_model=list[AccountResponse],
status_code=status.HTTP_200_OK,
summary="List all barangay accounts",
description="Accessible to admins and treasurers."
)

async def list_accounts_handler(
    user_uid: str = Depends(get_current_user),
    _: None = Depends(require_permission("manageUsers")),
    role: RoleEnum | None = None,
    limit: int = 20,
    offset: int = 0,
):
    logger.info("📄 Account list requested by user: %s", user_uid)

    # You'd implement a service function to query Firestore

    accounts = await safe_service_call(list_barangay_accounts, role=role, limit=limit,
offset=offset)

    return accounts

```

...

backend\app\routes\audit_routes.py

```python

# app/routes/audit\_routes.py

from fastapi import APIRouter, HTTPException

```

from backend.app.utils.firestore_utils import get_db

import logging

router = APIRouter()

logger = logging.getLogger("uvicorn.error")

@router.get("/", tags=["Audit"])
def list_audit_logs(limit: int = 50):
 """
 Return the latest document audit logs.
 """
 try:
 logs = (
 get_db().collection("document_audit")
 .order_by("timestamp", direction="DESCENDING")
 .limit(limit)
 .stream()
)
 return [log.to_dict() for log in logs]
 except Exception as e:
 logger.error(" ❌ Error fetching audit logs: %s", str(e), exc_info=True)
 raise HTTPException(status_code=500, detail="Failed to fetch audit logs")
 ...

backend\app\routes\business_routes.py

```



```

```python

from fastapi import APIRouter

from backend.app.core.firebase import delete_file

from backend.app.models.business import BusinessApplication

from backend.app.services.business_service import create_business_application

import logging

from backend.app.utils.firestore_utils import get_db


logger = logging.getLogger("uvicorn.error")


router = APIRouter(prefix="/businesses", tags=["Business"])


@router.post("/applications")
def create_application(payload: BusinessApplication):
    return create_business_application(payload)


@router.get("")
def list_businesses(ownerUid: str = None, ownerName: str = None):
    ref = get_db().collection("businesses")

    if ownerUid:
        docs = ref.where("ownerUid", "==", ownerUid).stream()

    elif ownerName:
        docs = ref.where("ownerName", "==", ownerName).stream()

    else:
        docs = ref.stream()

```

```
return [doc.to_dict() for doc in docs]
```

```
@router.delete("/{business_id}")
```

```
def delete_business(business_id: str):
```

```
    """Delete a business, related payments/receipts, and attachments in Storage."""
```

```
    business_docs = get_db().collection("businesses").where("businessId", "==",  
business_id).get()
```

```
    if not business_docs:
```

```
        return {"success": False, "message": "Business not found"}
```

```
    business_doc = business_docs[0]
```

```
    business_data = business_doc.to_dict()
```

```
# --- Delete attachments from Storage using stored paths ---
```

```
if business_data.get("documents"):
```

```
    for key, doc in business_data["documents"].items():
```

```
        # Expecting each doc to be a dict with {"url": ..., "path": ...}
```

```
        path = None
```

```
        if isinstance(doc, dict):
```

```
            path = doc.get("path")
```

```
        elif isinstance(doc, str):
```

```
            # Fallback for legacy records that only stored URL
```

```
            logger.warning(" ⚠ Document %s has only URL, no path. Skipping storage  
deletion.", key)
```

```
        if path:
```

```

try:
    delete_file(path)
except Exception as e:
    logger.warning(" ⚠ Failed to delete storage file %s: %s", path, e)

# --- Delete business doc ---
business_doc.reference.delete()

logger.info(" 🗑 Deleted business %s", business_id)

# --- Delete related payments ---
payments = get_db().collection("payments").where("businessId", "==", business_id).get()
for pay in payments:
    pay.reference.delete()

    logger.info(" 🗑 Deleted payment %s for business %s", pay.id, business_id)

# --- Delete related receipts ---
receipts = get_db().collection("receipts").where("businessId", "==", business_id).get()
for rec in receipts:
    rec.reference.delete()

    logger.info(" 🗑 Deleted receipt %s for business %s", rec.id, business_id)

return {"success": True, "message": f"Business {business_id}, related records, and
attachments deleted"}

...

```

```
## backend\app\routes\complaint_routes.py

```python
import logging

from typing import Optional, List

from fastapi import APIRouter, HTTPException, status, Depends, Query
from pydantic import BaseModel

from backend.app.models.complaint import (
 ComplaintCreate,
 Complaint,
 ComplaintWithResident,
 ComplaintStatus,
)
from backend.app.services.complaint_service import (
 file_complaint,
 get_complaint_by_id,
 list_complaints_with_residents,
 list_complaints_by_resident_id,
 update_complaint_status,
 delete_complaint,
)
from backend.app.core.auth import require_permission
from backend.app.services.notification_service import NotificationService
```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(tags=["Complaints"])
```

```

```

```
 Shared Models
```

```

```

```
class ActionResponse(BaseModel):
```

```
 message: str
```

```
class StatusUpdateRequest(BaseModel):
```

```
 status: ComplaintStatus
```

```
 notes: Optional[str] = None
```

```
 resolution_notes: Optional[str] = None
```

```

```

```
 1. Resident submits a complaint
```

```

```

```
@router.post(
```

```
 "/",
```

```
 response_model=Complaint,
```

```

 status_code=status.HTTP_201_CREATED,
 summary="File a new complaint (resident or staff on behalf of resident)",
)
 async def submit_complaint(
 complaint: ComplaintCreate,
 current_user=Depends(require_permission(["fileComplaints",
 "fileComplaintsForResidents"])),
):
 """
 Submit a complaint.
 - Residents: filed_by == filed_for (self-filing)
 - Staff/Admin: filed_by = staff/admin UID, filed_for = resident UID
 """
 try:
 # Ensure filed_for is set: if not provided, default to self-filing
 if not complaint.filed_for:
 complaint.filed_for = complaint.filed_by

 created = file_complaint(complaint)
 if not created:
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Failed to file complaint",
)

 logger.info(

```

```

" 📄 Complaint submitted: %s (filed_by=%s, filed_for=%s)",
created.id,
complaint.filed_by,
complaint.filed_for,
)

try:
 await NotificationService.notify(
 role="admin",
 type="complaint",
 message=f"New complaint filed ({created.category.value})",
)
 await NotificationService.notify(
 role="staff",
 type="complaint",
 message=f"New complaint filed ({created.category.value})",
)
except Exception as notify_err:
 logger.warning(" ⚠️ Complaint submit notification failed: %s", notify_err)

return created

except HTTPException:
 raise

except Exception as e:
 logger.error(" ❌ Failed to file complaint: %s", e)

```

```
raise HTTPException(
 status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
 detail="Unexpected error while filing complaint",
)
```

```

```

```
 2. Resident lists their own complaints
```

```

```

```
@router.get(
 "/mine",
 response_model=List[Complaint],
 summary="Resident lists their own complaints",
)

def get_my_complaints(
 resident_uid: str = Depends(require_permission("viewOwnComplaints")),
 limit: Optional[int] = Query(None, ge=0, le=100),
):
 return list_complaints_by_resident_id(resident_uid, limit)
```

```

```

```
 3. Admin/staff lists ALL complaints
```

```
(STATIC ROUTE — must come BEFORE /{complaint_id})
```

```

```



```

@router.get(
 "/all",
 response_model=List[ComplaintWithResident],
 summary="Admin/staff lists all complaints with resident + filer info",
)

def get_all_complaints(
 _: None = Depends(require_permission("viewAllComplaints")),
 limit: Optional[int] = Query(None, ge=0, le=100),
 status: Optional[ComplaintStatus] = Query(None),
):
 return list_complaints_with_residents(limit, status)

```

# -----

#  4. Get a specific complaint by ID

# -----

```

@router.get(
 ("/{complaint_id}",
 summary="Get a specific complaint by ID",
)

def get_complaint(
 complaint_id: str,
 current_user=Depends(require_permission(["viewOwnComplaints",
"viewAllComplaints"])),
):
 complaint = get_complaint_by_id(complaint_id)

```

```
if complaint is None:
 raise HTTPException(
 status_code=status.HTTP_404_NOT_FOUND,
 detail="Complaint not found",
)
```

```
Residents → plain Complaint
```

```
if getattr(current_user, "role", None) == "resident":
 return Complaint(**complaint.dict())
```

```
Staff/Admin → enriched ComplaintWithResident
return complaint
```

```

 5. Update complaint status (admin/staff)

```

```
@router.patch(
 ("/{complaint_id}/status",
 response_model=ComplaintWithResident,
 summary="Admin updates complaint status",
)

async def update_status(
 complaint_id: str,
 payload: StatusUpdateRequest,
 _: None = Depends(require_permission("manageComplaints")),
```

```

):

effective_notes = payload.notes

if effective_notes is None:

 effective_notes = payload.resolution_notes

updated = update_complaint_status(complaint_id, payload.status, effective_notes)

if updated is None:

 raise HTTPException(

 status_code=status.HTTP_404_NOT_FOUND,

 detail="Complaint not found",

)

try:

 status_label = payload.status.value.replace("_", " ")

 await NotificationService.notify(

 role="admin",

 type="complaint_update",

 message=f"Complaint status updated to {status_label}",

)

 await NotificationService.notify(

 role="staff",

 type="complaint_update",

 message=f"Complaint status updated to {status_label}",

)

resident_uid = getattr(updated, "filed_for", None) or getattr(updated, "filed_by", None)

```

```

if resident_uid:
 await NotificationService.notify(
 role="resident",
 type="complaint_update",
 message=f"Your complaint status was updated to {status_label}",
 user_id=resident_uid,
)
except Exception as notify_err:
 logger.warning(" ⚠️ Complaint status notification failed: %s", notify_err)

return updated

✅ 6. Delete a complaint (admin only)

@router.delete(
 ("/{complaint_id}",
 response_model=ActionResponse,
 summary="Admin deletes a complaint",
 status_code=status.HTTP_200_OK,
)

def delete_complaint_route(
 complaint_id: str,
 _: None = Depends(require_permission("manageComplaints")),
):

```

```

deleted = delete_complaint(complaint_id)

if deleted is None:

 raise HTTPException(

 status_code=status.HTTP_404_NOT_FOUND,

 detail="Complaint not found",

)

return ActionResponse(message=f"Complaint {complaint_id} deleted successfully")

...

```

```
backend\app\routes\dashboard.py
```

```

```python
import logging

from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel

from backend.app.utils.firestore_utils import get_db

```

```

logger = logging.getLogger("uvicorn.error")

router = APIRouter(tags=["Dashboard"])

```

```
# 📦 Response model
```

```

class DashboardSummary(BaseModel):

    residents: int

    businesses: int

```

complaints: int

officials: int

incidents: int

```
@router.get("/dashboard-summary", response_model=DashboardSummary)
```

```
async def get_dashboard_summary():
```

```
    try:
```

```
        collections = ["residents", "businesses", "complaints", "officials", "incidents"]
```

```
        counts = {}
```

```
        for col in collections:
```

```
            docs = get_db().collection(col).get()
```

```
            counts[col] = len(docs)
```

```
        logger.info("📊 Dashboard summary fetched successfully")
```

```
        return DashboardSummary(**counts)
```

```
    except Exception as e:
```

```
        logger.error("❌ Failed to fetch dashboard summary: %s", str(e), exc_info=True)
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
```

```
            detail="Failed to fetch dashboard summary"
```

```
        )
```

```
    ...
```

```
## backend\app\routes\disbursement_routes.py
```

```
` ``python
```

```
from fastapi import APIRouter, HTTPException
```

```
from backend.app.services import disbursement_service
```

```
router = APIRouter(tags=["Disbursements"])
```

```
@router.post("/disbursements")
```

```
async def create_disbursement(payload: dict):
```

```
    return disbursement_service.create_disbursement(payload)
```

```
@router.get("/disbursements")
```

```
async def list_disbursements():
```

```
    return disbursement_service.list_disbursements()
```

```
@router.get("/disbursements/{id}")
```

```
async def get_disbursement(id: str):
```

```
    result = disbursement_service.get_disbursement(id)
```

```
    if not result:
```

```
        raise HTTPException(status_code=404, detail="Disbursement not found")
```

```
    return result
```

```
@router.patch("/disbursements/{id}/status")
```

```
async def update_status(id: str, payload: dict):
```

```
result = disbursement_service.update_status(id, payload.get("status"))
if not result:
    raise HTTPException(status_code=404, detail="Disbursement not found")
return result
```

```
@router.put("/disbursements/{id}")
```

```
async def update_disbursement(id: str, payload: dict):
```

```
    result = disbursement_service.update_disbursement(id, payload)
```

```
    if not result:
```

```
        raise HTTPException(status_code=404, detail="Disbursement not found")
```

```
    return result
```

```
@router.delete("/disbursements/{id}")
```

```
async def delete_disbursement(id: str):
```

```
    success = disbursement_service.delete_disbursement(id)
```

```
    if not success:
```

```
        raise HTTPException(status_code=404, detail="Disbursement not found")
```

```
    return {"id": id, "deleted": True}
```

```
...
```

```
## backend\app\routes\document_routes.py
```

```
```python
```

```
from fastapi import APIRouter, Query, UploadFile, File, Form, Depends
```

```
from typing import List, Optional
```



```
from pydantic import BaseModel
from datetime import datetime
from backend.app.models.document import Document, DocumentStatus
from backend.app.core.auth import require_permission
from backend.app.services import document_service
from backend.app.services.notification_service import NotificationService
import logging
```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(tags=["Documents"])
```

```
async def _notify_document_submitted(doc: Document):
 try:
 suffix = f" ({doc.documentType})" if doc.documentType else ""
 await NotificationService.notify(
 role="admin",
 type="document",
 message=f"New document request submitted{suffix}",
)
 await NotificationService.notify(
 role="secretary",
 type="document",
 message=f"New document request submitted{suffix}",
)
```

```
except Exception as notify_err:
```

```
 logger.warning(" ⚠ Document submit notification failed: %s", notify_err)
```

```
async def _notify_document_status_change(doc: Document):
```

```
 try:
```

```
 status_value = doc.status.value if hasattr(doc.status, "value") else str(doc.status)
```

```
 status_label = str(status_value).replace("_", " ")
```

```
 suffix = f" ({doc.documentType})" if doc.documentType else ""
```

```
 await NotificationService.notify(
```

```
 role="admin",
```

```
 type="document_update",
```

```
 message=f"Document status updated to {status_label}{suffix}",
```

```
)
```

```
 await NotificationService.notify(
```

```
 role="secretary",
```

```
 type="document_update",
```

```
 message=f"Document status updated to {status_label}{suffix}",
```

```
)
```

```
 if doc.residentId:
```

```
 await NotificationService.notify(
```

```
 role="resident",
```

```
 type="document_update",
```

```
 message=f"Your document status was updated to {status_label}{suffix}",
```

```

 user_id=doc.residentId,
)
except Exception as notify_err:
 logger.warning(" ⚠ Document status notification failed: %s", notify_err)

=====
📁 List Documents
=====

@router.get("", response_model=List[Document])
async def list_documents(
 residentId: Optional[str] = Query(None),
 documentType: Optional[str] = Query(None),
 issuedBy: Optional[str] = Query(None),
 fromDate: Optional[datetime] = Query(None),
 toDate: Optional[datetime] = Query(None),
 uid: str = Depends(require_permission("viewDocuments"))
) -> List[Document]:
 return document_service.list_documents(
 uid=uid,
 residentId=residentId,
 documentType=documentType,
 issuedBy=issuedBy,
 fromDate=fromDate,
 toDate=toDate,
)

```

```
@router.get("/my", response_model=List[Document])
```

```
async def list_my_documents(resident_id: str) -> List[Document]:
```

```
 return document_service.list_my_documents(resident_id)
```

```
@router.get("/my/active", response_model=List[Document])
```

```
async def list_active_documents(resident_id: str) -> List[Document]:
```

```
 return document_service.list_active_documents(resident_id)
```

```
@router.get("/my/history", response_model=List[Document])
```

```
async def list_history_documents(resident_id: str) -> List[Document]:
```

```
 return document_service.list_history_documents(resident_id)
```

```
@router.get("/{doc_id}", response_model=Document)
```

```
async def get_document(doc_id: str) -> Document:
```

```
 return document_service.get_document(doc_id)
```

```
=====
```

```
 Mark Document as Resubmitted
```

```
=====
```

```
class ResubmissionPayload(BaseModel):
```

```
 resubmitted: bool = True
```

```
@router.patch("/{doc_id}/resubmission", response_model=Document)
```

```
async def mark_document_resubmitted(doc_id: str, payload: ResubmissionPayload) -> Document:
```

```
 return await document_service.mark_resubmitted(doc_id)
```

```

=====

📄 Create Document (Resident)

=====

@router.post("", response_model=Document, status_code=201)
async def create_document(
 resident_id: str = Form(...),
 document_type: str = Form(...),
 purpose: Optional[str] = Form(None),
 remarks: Optional[str] = Form(None),

 # Attachments
 idAttachment: UploadFile = File(None),
 residencyAttachment: UploadFile = File(None),
 medicalAttachment: UploadFile = File(None),
 photoAttachment: UploadFile = File(None),
 activityPlan: UploadFile = File(None),
 businessPermit: UploadFile = File(None),

 # Extra fields
 complainant: Optional[str] = Form(None),
 respondent: Optional[str] = Form(None),
 incident: Optional[str] = Form(None),
 businessName: Optional[str] = Form(None),
 activityName: Optional[str] = Form(None),

```

```
activityDate: Optional[str] = Form(None),
occupation: Optional[str] = Form(None),
voterStatus: Optional[str] = Form(None),
yearsOfStay: Optional[int] = Form(None),
```

```
 Location fields
```

```
locationBarangay: Optional[str] = Form(None),
locationStreet: Optional[str] = Form(None),
locationCity: Optional[str] = Form(None),
locationProvince: Optional[str] = Form(None),
```

```
) -> Document:
```

```
created = await document_service.create_document(
 resident_id=resident_id,
 resident_name=None, # Will be populated in service layer based on residentId
 document_type=document_type,
 purpose=purpose,
 remarks=remarks,
 idAttachment=idAttachment,
 residencyAttachment=residencyAttachment,
 medicalAttachment=medicalAttachment,
 photoAttachment=photoAttachment,
 activityPlan=activityPlan,
 businessPermit=businessPermit,
 complainant=complainant,
 respondent=respondent,
 incident=incident,
```

```

 businessName=businessName,
 activityName=activityName,
 activityDate=activityDate,
 occupation=occupation,
 voterStatus=voterStatus,
 yearsOfStay=yearsOfStay,
 locationBarangay=locationBarangay,
 locationStreet=locationStreet,
 locationCity=locationCity,
 locationProvince=locationProvince,
)
 await _notify_document_submitted(created)
 return created

```

```

=====

```

```

 Update Status

```

```

=====

```

```

class StatusUpdatePayload(BaseModel):

```

```

 newStatus: DocumentStatus

```

```

 remarks: Optional[str] = None

```

```

@router.patch("/{doc_id}/status", response_model=Document)

```

```

async def update_document_status(doc_id: str, payload: StatusUpdatePayload) ->
Document:

```

```

 updated = await document_service.update_status(doc_id, payload.newStatus,
payload.remarks)

```

```
await _notify_document_status_change(updated)

return updated
```

```
=====
```

```
🏠 Confirm Payment
```

```
=====
```

```
@router.patch("/{doc_id}/payment", response_model=Document)
```

```
async def confirm_payment(doc_id: str) -> Document:
```

```
 updated = await document_service.confirm_payment(doc_id)
```

```
 await _notify_document_status_change(updated)
```

```
 return updated
```

```
=====
```

```
📄 Issue Document
```

```
=====
```

```
class IssuePayload(BaseModel):
```

```
 issued_by: str
```

```
 file_url: Optional[str] = None
```

```
 remarks: Optional[str] = None
```

```
@router.patch("/{doc_id}/issue", response_model=Document)
```

```
async def issue_document(doc_id: str, payload: IssuePayload) -> Document:
```

```
 updated = await document_service.issue_document(
```

```
 doc_id,
```

```
 payload.issued_by,
```

```
 payload.file_url,
```



```

 payload.remarks
)
 await _notify_document_status_change(updated)
 return updated

```

```

@router.delete("/{doc_id}", response_model=Document)

async def delete_document(doc_id: str, uid: str =
Depends(require_permission("manageDocuments"))) -> Document:
 return await document_service.delete_document(doc_id, uid)

```

```

@router.get("/count/issued")

async def get_issued_count(documentType: Optional[str] = Query(None)) -> dict:
 count = document_service.count_issued_documents(documentType)
 return {"documentType": documentType, "issuedCount": count}

```

```

...

```

```

backend\app\routes\fee_routes.py

```

```

```python
from fastapi import APIRouter, Depends, HTTPException, Body
from backend.app.core.auth import get_admin_uid
from backend.app.services.paymongo_service import (
    create_payment_link,
    create_payment_intent #  new helper for e-wallets

```

```
)  
from backend.app.models.fee import (  
    DocumentFee, BusinessFee, MiscFee,  
    NewDocumentFee, NewBusinessFee, NewMiscFee  
)
```

```
from backend.app.utils.firestore_utils import (  
    create_document,  
    update_document,  
    delete_document,  
    get_db  
)
```

```
)  
import re  
import logging  
from typing import List, Dict, Optional, Type  
from pydantic import BaseModel
```

```
router = APIRouter(prefix="/fees", tags=["Fees"])  
logger = logging.getLogger("uvicorn.error")
```

```
# -----
```

```
# 🛠 Utility: Normalize IDs
```

```
# -----
```

```
def normalize_id(value: str) -> str:
```

```
return re.sub(r"^[a-z0-9_]", "_", value.strip().lower())
```

```
# -----
```

```
# 🔧 Shared Firestore Helpers
```

```
# -----
```

```
def list_collection(collection: str) -> List[Dict]:
```

```
    docs = get_db().collection(collection).get()
```

```
    return [doc.to_dict() | {"id": doc.id} for doc in docs]
```

```
def list_with_misc(collection: str) -> List[Dict]:
```

```
    docs = get_db().collection(collection).get()
```

```
    misc_map = {
```

```
        normalize_id(m.id): m.to_dict()
```

```
        for m in get_db().collection("misc_fees").get()
```

```
    }
```

```
    result = []
```

```
    for doc in docs:
```

```
        data = doc.to_dict()
```

```
        misc_type_raw = data.get("miscType")
```

```
        if misc_type_raw:
```

```
            misc_type_key = normalize_id(misc_type_raw)
```

```
            misc_entry = misc_map.get(misc_type_key)
```

```
            if misc_entry and misc_entry.get("enabled") and data.get("enabled"):
```

```
                data["miscFeeResolved"] = misc_entry["fee"]
```

```
            else:
```

```
                data["miscFeeResolved"] = None
```

```
    else:

        data["miscFeeResolved"] = None

    result.append(data | {"id": doc.id})

return result
```

```
def get_business_ref(identifier: str):

    # Try Firestore doc ID

    ref = get_db().collection("businesses").document(identifier)

    if ref.get().exists:

        return ref

    # Try custom businessId field

    docs = get_db().collection("businesses").where("businessId", "==", identifier).limit(1).get()

    if docs:

        return docs[0].reference

    raise HTTPException(status_code=404, detail=f"Business {identifier} not found")
```

```
def get_document_ref(identifier: str):

    ref = get_db().collection("documents").document(identifier)

    if ref.get().exists:

        return ref

    docs = get_db().collection("documents").where("documentId", "==",
identifier).limit(1).get()

    if docs:

        return docs[0].reference

    raise HTTPException(status_code=404, detail=f"Document {identifier} not found")
```

```

# -----
# 🛠 Router Factory
# -----

def make_fee_routes(
    collection: str,
    prefix: str,
    new_model: Type[BaseModel],
    update_model: Type[BaseModel],
    id_field: str,
    extra_fields: Optional[List[str]] = None,
    resolve_misc: bool = False,
):
    if extra_fields is None:
        extra_fields = []

    @router.get(f"/{prefix}")
    def list_fees(admin=Depends(get_admin_uid)):
        return list_with_misc(collection) if resolve_misc else list_collection(collection)

    @router.post(f"/{prefix}")
    def create_fee(payload: new_model = Body(...), admin=Depends(get_admin_uid)): #
type: ignore
        """
        Create a new fee entry in Firestore.

        FastAPI will validate `payload` against the `new_model` schema.
        """

```

```
logger.info("Creating fee with payload=%s", payload.dict())
```

```
fee_id = normalize_id(getattr(payload, id_field))
```

```
data = {id_field: getattr(payload, id_field).strip(), "fee": payload.fee}
```

```
for field in extra_fields:
```

```
    data[field] = getattr(payload, field, None)
```

```
return create_document(collection, fee_id, data)
```

```
@router.put("/{prefix}/{fee_id}")
```

```
def update_fee(fee_id: str, payload: update_model = Body(...),  
admin=Depends(get_admin_uid)): # type: ignore
```

```
    """
```

Update an existing fee entry in Firestore.

FastAPI will validate `payload` against the `update\_model` schema.

```
    """
```

```
update_data = {"fee": payload.fee}
```

```
for field in extra_fields:
```

```
    value = getattr(payload, field, None)
```

```
    if value is not None:
```

```
        update_data[field] = value
```

```
return update_document(collection, normalize_id(fee_id), update_data)
```

```
@router.delete("/{prefix}/{fee_id}")
```

```
def delete_fee(fee_id: str, admin=Depends(get_admin_uid)):
```

```
    """
```

Delete a fee entry from Firestore.

```
"""
```

```
    return delete_document(collection, normalize_id(fee_id))
```

```
# -----
```

```
# 📄 Document Fee Routes
```

```
# -----
```

```
make_fee_routes(
```

```
    collection="document_types",
```

```
    prefix="documents",
```

```
    new_model=NewDocumentFee,
```

```
    update_model=DocumentFee,
```

```
    id_field="documentType",
```

```
    extra_fields=["miscType", "enabled"],
```

```
    resolve_misc=True,
```

```
)
```

```
# -----
```

```
# 🏢 Business Fee Routes
```

```
# -----
```

```
make_fee_routes(
```

```
    collection="business_types",
```

```
    prefix="businesses",
```

```
    new_model=NewBusinessFee,
```

```
    update_model=BusinessFee,
```

```
    id_field="businessType",
```

```
    extra_fields=["registrationFee", "annualFee", "miscType", "enabled"],
```

```

        resolve_misc=True,
    )

# -----
#  Miscellaneous Fee Routes
# -----

make_fee_routes(
    collection="misc_fees",
    prefix="misc",
    new_model=NewMiscFee,
    update_model=MiscFee,
    id_field="miscType",
    extra_fields=["enabled"],
)

# -----
#  Public Business Fee View
# -----

@router.get("/public/businesses")

def list_public_business_types():
    all_types = list_with_misc("business_types")
    result = []
    for bt in all_types:
        total = (bt.get("fee", 0) +
                 bt.get("registrationFee", 0) +
                 (bt.get("miscFeeResolved") or 0))

```



```
        bt["totalFee"] = total
    result.append(bt)
return result
```

🌐 Public Document Fee View

```
@router.get("/public/documents")
```

```
def list_public_document_types():
```

```
    all_docs = list_with_misc("document_types")
```

```
    result = []
```

```
    for doc in all_docs:
```

```
        total = (doc.get("fee", 0) +
```

```
                  (doc.get("miscFeeResolved") or 0))
```

```
        doc["totalFee"] = total
```

```
        result.append(doc)
```

```
    return result
```

💰 Fee Computation Helpers

```
def resolve_misc_fee(bt: dict) -> int:
```

```
    misc_type_raw = bt.get("miscType")
```

```
    if misc_type_raw:
```

```
        misc_type_key = normalize_id(misc_type_raw)
```

```
        misc_entry = get_db().collection("misc_fees").document(misc_type_key).get()
```

```
        if misc_entry.exists:
```

```
    misc = misc_entry.to_dict()

    if misc.get("enabled") and bt.get("enabled"):

        return misc.get("fee", 0)

    return 0
```

```
def compute_document_fee(document_type: str) -> int:

    docs = get_db().collection("document_types").where("documentType", "==",
document_type).limit(1).get()

    if not docs:

        raise HTTPException(status_code=404, detail=f"No fee configured for document type:
{document_type}")

    doc = docs[0].to_dict()

    # ✅ Align with frontend: base + misc if enabled

    total = doc.get("fee", 0)

    total += resolve_misc_fee(doc)

    return total
```

```
def compute_business_registration_fee(business_type: str) -> int:

    docs = get_db().collection("business_types").where("businessType", "==",
business_type).limit(1).get()

    if not docs:

        raise HTTPException(status_code=404, detail=f"No fee configured for business type:
{business_type}")

    bt = docs[0].to_dict()
```

 Align with frontend: base + registration + misc if enabled

```
total = (bt.get("fee", 0) + bt.get("registrationFee", 0))
```

```
total += resolve_misc_fee(bt)
```

```
return total
```

```
def compute_business_annual_fee(business_type: str) -> int:
```

```
    docs = get_db().collection("business_types").where("businessType", "==",  
business_type).limit(1).get()
```

```
    if not docs:
```

```
        raise HTTPException(status_code=404, detail=f"No fee configured for business type:  
{business_type}")
```

```
    bt = docs[0].to_dict()
```

 Align with frontend: base + annual + misc if enabled

```
total = (bt.get("fee", 0) + bt.get("annualFee", 0))
```

```
total += resolve_misc_fee(bt)
```

```
return total
```

```
@router.post("/businesses/{business_id}/payment")
```

```
def create_business_payment(business_id: str, payload: dict = Body(...)):
```

```
    """
```

```
    Create a PayMongo payment for a business.
```

```
    Decides between registration vs annual renewal based on payload["paymentType"].
```

```
    """
```

```
payment_type = payload.get("paymentType", "registration") # default to registration
remarks = payload.get("remarks", f"Business {payment_type} fee")
ref = get_business_ref(business_id)
business = ref.get().to_dict()
```

```
#  Decide which fee to compute
```

```
if payment_type == "annual":
```

```
    fee = compute_business_annual_fee(business.get("businessType"))
```

```
    description = f"Annual Business Fee for {business_id}"
```

```
else:
```

```
    fee = compute_business_registration_fee(business.get("businessType"))
```

```
    description = f"Registration Business Fee for {business_id}"
```

```
if fee <= 0:
```

```
    raise HTTPException(status_code=400, detail="Invalid fee amount")
```

```
#  Decide API based on fee amount
```

```
if fee < 100:
```

```
    result = create_payment_intent(
```

```
        amount=fee,
```

```
        description=description,
```

```
        remarks=remarks,
```

```
        metadata={
```

```
            "businessId": business_id,
```

```
            "businessType": business.get("businessType"),
```

```
            "paymentType": payment_type,
```

```

    },
    success_url="https://your-app.com/business/payment-success",
    cancel_url="https://your-app.com/business/payment-cancel"
)
else:
    result = create_payment_link(
        amount=fee,
        description=description,
        remarks=remarks,
        metadata={
            "businessId": business_id,
            "businessType": business.get("businessType"),
            "paymentType": payment_type,
        },
        success_url="https://your-app.com/business/payment-success",
        cancel_url="https://your-app.com/business/payment-cancel"
    )

```

 Update Firestore with checkout details

```

ref.update({
    "checkoutUrl": result["checkout_url"],
    "paymongoLinkId": result.get("link_id"),
    "paymentIntentId": result.get("intent_id"),
    "status": "for_payment",
    "fee": fee,
    "paymentType": payment_type, # store type for audit clarity

```

```
})
```

```
return result
```

```
@router.post("/documents/{document_id}/payment")
```

```
def create_document_payment(document_id: str, payload: dict = Body(...)):
```

```
    remarks = payload.get("remarks", "Document fee")
```

```
    ref = get_document_ref(document_id)
```

```
    document = ref.get().to_dict()
```

```
    doc_type = document.get("documentType")
```

```
    fee = compute_document_fee(doc_type)
```

```
    if fee < 0:
```

```
        raise HTTPException(status_code=400, detail="Invalid fee amount")
```

```
    if fee == 0:
```

```
        ref.update({
```

```
            "status": "paid",
```

```
            "fee": 0,
```

```
            "paymentType": "free"
```

```
        })
```

```
        return {"message": f"{doc_type} is free, no payment required"}
```

```
description = f"{doc_type} Request {document_id}"
```

 Decide API based on fee

if fee < 100:

```
    result = create_payment_intent(  
        amount=fee,  
        description=description,  
        remarks=remarks,  
        metadata={"documentId": document_id, "documentType": doc_type},  
        success_url="https://your-app.com/documents/payment-success",  
        cancel_url="https://your-app.com/documents/payment-cancel"  
    )
```

else:

```
    result = create_payment_link(  
        amount=fee,  
        description=description,  
        remarks=remarks,  
        metadata={"documentId": document_id, "documentType": doc_type},  
        success_url="https://your-app.com/documents/payment-success",  
        cancel_url="https://your-app.com/documents/payment-cancel"  
    )
```

```
ref.update({  
    "checkoutUrl": result["checkout_url"],  
    "paymongoLinkId": result.get("link_id"),  
    "paymentIntentId": result.get("intent_id"),  
    "status": "awaiting_payment",
```

```

        "fee": fee
    })

    logger.info("Updated Firestore with new checkoutUrl=%s fee=%s", result["checkout_url"],
fee)

    return result

    ...

## backend\app\routes\incident_routes.py

```python
import logging

from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel
from typing import List, Optional

from backend.app.models.incident import (
 IncidentCreate,
 Incident,
 IncidentWithResident,
 IncidentStatus,
)

from backend.app.services.incident_service import (
 create_incident,

```



```
 get_incident_by_id,
 list_incidents_with_residents,
 update_incident_status,
 delete_incident,
)

from backend.app.services.notification_service import NotificationService

from backend.app.utils.firestore_utils import get_db
```

```
logger = logging.getLogger("uvicorn.error")
router = APIRouter(tags=["Incidents"])
```

```
📦 Response models
```

```
class ActionResponse(BaseModel):
 message: str
```

```
🔧 Request models
```

```
class AdminStatusUpdateRequest(BaseModel):
 status: IncidentStatus
 assigned_to: Optional[str] = None
```

```
def _resolve_incident_owner_resident_uid(incident_obj: Incident) -> Optional[str]:
 resident_uid = getattr(incident_obj, "residentId", None)
 if resident_uid:
 return resident_uid
```

```
auth_uid = getattr(incident_obj, "authUid", None)
```

```
if not auth_uid:
```

```
 return None
```

```
try:
```

```
 if get_db().collection("residents").document(auth_uid).get().exists:
```

```
 return auth_uid
```

```
except Exception:
```

```
 return None
```

```
return None
```

```
📄 Report a new incident
```

```
@router.post("", response_model=Incident, status_code=status.HTTP_201_CREATED)
```

```
async def report_incident(incident: IncidentCreate):
```

```
 try:
```

```
 created = create_incident(incident)
```

```
 logger.info(" 📄 Incident reported with ID: %s by resident %s", created.id,
incident.authUid)
```

```
 try:
```

```
 await NotificationService.notify(

```

```
 role="admin",

```

```
 type="incident",

```

```
 message=f"New incident filed ({created.type.value})",

```

```

)

 await NotificationService.notify(
 role="staff",
 type="incident",
 message=f"New incident filed ({created.type.value})",
)

except Exception as notify_err:

 logger.warning(" ⚠ Incident submit notification failed: %s", notify_err)

return created

except Exception as e:

 logger.error(" ❌ Failed to create incident: %s", e, exc_info=True)

 raise HTTPException(status_code=400, detail="Failed to create incident")

```

# 🔍 Get a specific incident

```

@router.get("/{incident_id}", response_model=Incident)
def get_incident(incident_id: str):

 try:

 incident = get_incident_by_id(incident_id)

 if not incident:

 raise HTTPException(status_code=404, detail="Incident not found")

 logger.info(" 🔍 Incident retrieved: %s", incident_id)

 return incident

 except HTTPException:

 raise

```

except Exception as e:

```
logger.error(" ❌ Error retrieving incident %s: %s", incident_id, e, exc_info=True)
```

```
raise HTTPException(status_code=500, detail="Internal server error")
```

# 📄 List all incidents with resident info

```
@router.get("", response_model=List[IncidentWithResident])
```

```
def get_all_incidents(status: Optional[str] = None):
```

```
 try:
```

```
 incidents = list_incidents_with_residents(status=status)
```

```
 logger.info(" 📄 Retrieved %d incidents (status=%s)", len(incidents), status)
```

```
 return incidents
```

except Exception as e:

```
logger.error(" ❌ Error listing incidents: %s", e, exc_info=True)
```

```
raise HTTPException(status_code=500, detail="Internal server error")
```

# 🛠 Admin: update incident status + assignment

```
@router.patch("/{incident_id}/status", response_model=ActionResponse)
```

```
async def admin_update_status(incident_id: str, payload: AdminStatusUpdateRequest):
```

```
 try:
```

```
 existing = get_incident_by_id(incident_id)
```

```
 if not existing:
```

```
 raise HTTPException(status_code=404, detail="Incident not found")
```

```

success = update_incident_status(
 incident_id,
 payload.status.value,
 assigned_to=payload.assigned_to,
)

if not success:
 raise HTTPException(status_code=404, detail="Incident not found")

logger.info(
 "🔧 Incident %s updated: status=%s, assigned_to=%s",
 incident_id,
 payload.status.value,
 payload.assigned_to,
)

try:
 status_label = payload.status.value.replace("_", " ")
 await NotificationService.notify(
 role="admin",
 type="incident_update",
 message=f"Incident status updated to {status_label}",
)
 await NotificationService.notify(
 role="staff",
 type="incident_update",
 message=f"Incident status updated to {status_label}",
)

```

```

 resident_uid = _resolve_incident_owner_resident_uid(existing)
 if resident_uid:
 await NotificationService.notify(
 role="resident",
 type="incident_update",
 message=f"Your incident status was updated to {status_label}",
 user_id=resident_uid,
)
 except Exception as notify_err:
 logger.warning(" ⚠ Incident status notification failed: %s", notify_err)

 return ActionResponse(message="Incident updated successfully")
except HTTPException:
 raise
except Exception as e:
 logger.error(" ❌ Error updating incident %s: %s", incident_id, e, exc_info=True)
 raise HTTPException(status_code=500, detail="Internal server error")

```

# 🗑 Delete an incident

```
@router.delete("/{incident_id}", response_model=ActionResponse)
```

```
def delete_incident_route(incident_id: str):
```

```
 try:
```

```
 success = delete_incident(incident_id)
```

```
 if not success:
```

```

 raise HTTPException(status_code=404, detail="Incident not found")

 logger.info(" 🗑 Incident deleted: %s", incident_id)

 return ActionResponse(message="Incident deleted successfully")

except HTTPException:

 raise

except Exception as e:

 logger.error(" ❌ Error deleting incident %s: %s", incident_id, e, exc_info=True)

 raise HTTPException(status_code=500, detail="Internal server error")

...

```

```

backend\app\routes\notification_routes.py

```

```

```python

```

```

# backend/app/routes/notification_routes.py

```

```

from fastapi import APIRouter, Depends, HTTPException, Path, Query
from typing import List, Set, Optional
from pydantic import BaseModel
from datetime import datetime, timezone
import logging

from backend.app.services.notification_service import NotificationService
from backend.app.models.notification import Notification
from backend.app.core.auth import get_current_user
from backend.app.core.websocket_manager import manager
from backend.app.utils.firestore_utils import get_db

```

```
router = APIRouter(prefix="/notifications", tags=["notifications"])
```

```
logger = logging.getLogger("uvicorn.error")
```

```
class ResidentLoginPayload(BaseModel):
```

```
    count: int
```

```
class OfficerLoginPayload(BaseModel):
```

```
    name: str
```

```
    role: str
```

```
class BusinessSubmittedPayload(BaseModel):
```

```
    resident_name: str
```

```
    business_name: str | None = None
```

```
class BusinessStatusUpdatePayload(BaseModel):
```

```
    status: str
```

```
    resident_uid: str | None = None
```

```
    business_id: str | None = None
```

```
    firestore_id: str | None = None
```

```
    business_name: str | None = None
```

```
def _resident_message(event_type: str, count: int) -> str:
```



```
if event_type == "login":  
    return f"{count} resident logged in" if count == 1 else f"{count} residents logged in"  
return f"{count} resident logged out" if count == 1 else f"{count} residents logged out"
```

```
def _record_login_event(  
    *,  
    scope: str,  
    actor_role: str,  
    actor_uid: Optional[str],  
    actor_name: Optional[str],  
    count: int = 1,  
):  
    try:  
        now = datetime.now(timezone.utc)  
        payload = {  
            "type": "login",  
            "scope": scope,  
            "role": actor_role,  
            "user_id": actor_uid,  
            "user": actor_name,  
            "count": max(1, int(count or 1)),  
            "timestamp": now,  
            "createdAt": now,  
        }  
        get_db().collection("logins").add(payload)
```

```
except Exception as err:
```

```
    logger.warning(" ⚠ Failed to record login event: %s", err)
```

```
def _first_or_none(stream):
```

```
    for item in stream:
```

```
        return item
```

```
    return None
```

```
def _normalize_role(value: str | None) -> str:
```

```
    return str(value or "").strip().lower()
```

```
def _resolve_audience_user_ids(data: dict) -> Set[str]:
```

```
    """
```

```
    Resolve intended recipient UIDs for a notification document.
```

```
    - user_id present => single-recipient notification
```

```
    - role-targeted => all users with that role in users collection
```

```
    """
```

```
    explicit_uid = data.get("user_id")
```

```
    if explicit_uid:
```

```
        return {str(explicit_uid)}
```

```
    target_role = _normalize_role(data.get("role"))
```

```
    if not target_role:
```

```
    return set()
```

```
if target_role == "resident":
```

```
    return set()
```

```
audience: Set[str] = set()
```

```
try:
```

```
    users = get_db().collection("users").where("role", "=", target_role).stream()
```

```
    for doc in users:
```

```
        audience.add(doc.id)
```

```
except Exception:
```

```
    return set()
```

```
return audience
```

```
def _is_notification_visible_to_user(data: dict, role: str, uid: str) -> bool:
```

```
    target_role = _normalize_role(data.get("role"))
```

```
    if role == "admin":
```

```
        return target_role == "admin"
```

```
    if role == "resident":
```

```
        return str(data.get("user_id") or "") == str(uid)
```

```
    return target_role == _normalize_role(role)
```

```
def _iter_scoped_notification_docs(role: str, uid: str):
```

```
query = get_db().collection("notifications")

if role == "admin":

    return query.where("role", "==", "admin").stream()

if role == "resident":

    return query.where("user_id", "==", uid).stream()

return query.where("role", "==", role).stream()
```

```
def _resolve_business_owner_uid(payload: BusinessStatusUpdatePayload) ->
Optional[str]:
```

```
    if payload.resident_uid:

        return str(payload.resident_uid)
```

```
db = get_db()
```

```
business_data = None
```

```
try:
```

```
    if payload.firestore_id:
```

```
        doc = db.collection("businesses").document(payload.firestore_id).get()
```

```
        if doc.exists:
```

```
            business_data = doc.to_dict() or {}
```

```
    elif payload.business_id:
```

```
        docs = db.collection("businesses").where("businessId", "==",
payload.business_id).limit(1).stream()
```

```
        first = _first_or_none(docs)
```

```
        if first:
```

```
            business_data = first.to_dict() or {}
```

```
except Exception:
```

```
    business_data = None
```

```
if not business_data:
```

```
    return None
```

```
owner_uid = business_data.get("ownerUid")
```

```
if owner_uid:
```

```
    return str(owner_uid)
```

```
email = business_data.get("email")
```

```
if not email:
```

```
    return None
```

```
try:
```

```
    users = db.collection("users").where("email", "==", email).limit(1).stream()
```

```
    user_doc = _first_or_none(users)
```

```
    if user_doc:
```

```
        return str(user_doc.id)
```

```
except Exception:
```

```
    pass
```

```
try:
```

```
    residents = db.collection("residents").where("email", "==", email).limit(1).stream()
```

```
    resident_doc = _first_or_none(residents)
```

```
    if resident_doc:
```

```
        return str(resident_doc.id)
except Exception:
    pass
```

```
return None
```

```
def _delete_for_user_or_globally(doc, uid: str):
    data = doc.to_dict() or {}
    deleted_by = {str(item) for item in (data.get("deleted_by") or [])}

    if str(uid) in deleted_by:
        return {"status": "already_deleted_for_user", "notification_id": doc.id}

    deleted_by.add(str(uid))
    audience_user_ids = _resolve_audience_user_ids(data)

    if audience_user_ids and audience_user_ids.issubset(deleted_by):
        doc.reference.delete()
        return {
            "status": "deleted_globally",
            "notification_id": doc.id,
            "deleted_by_count": len(deleted_by),
        }

    doc.reference.update({"deleted_by": sorted(deleted_by)})
```

```
return {  
    "status": "deleted_for_user",  
    "notification_id": doc.id,  
    "deleted_by_count": len(deleted_by),  
}
```

```
async def _broadcast_admin_notification(data: dict):  
    payload = dict(data)  
    await manager.broadcast(payload, role="admin")
```

```
async def _upsert_unread_resident_aggregate(event_type: str, delta: int) -> dict:  
    collection = get_db().collection("notifications")  
    existing = _first_or_none(  
        collection  
        .where("role", "==", "admin")  
        .where("scope", "==", "resident")  
        .where("type", "==", event_type)  
        .where("read", "==", False)  
        .limit(1)  
        .stream()  
    )
```

```
if existing:  
    current = existing.to_dict() or {}
```

```
next_count = max(0, int(current.get("count") or 0) + int(delta))

updates = {
    "count": next_count,
    "message": _resident_message(event_type, next_count),
    "timestamp": datetime.now(timezone.utc),
    "read": False,
}

existing.reference.update(updates)
updated = {**current, **updates, "id": existing.id}
await _broadcast_admin_notification(updated)
return updated
```

```
if delta <= 0:
    return {"count": 0}
```

```
notif = await NotificationService.notify(
    role="admin",
    type=event_type,
    message=_resident_message(event_type, delta),
    scope="resident",
    count=delta,
)

return notif.model_dump()
```

```
async def _decrement_unread_resident_logins(delta: int):
```



```
collection = get_db().collection("notifications")
```

```
login_doc = _first_or_none(  
    collection  
    .where("role", "==", "admin")  
    .where("scope", "==", "resident")  
    .where("type", "==", "login")  
    .where("read", "==", False)  
    .limit(1)  
    .stream()  
)
```

```
if not login_doc:
```

```
    return
```

```
current = login_doc.to_dict() or {}
```

```
current_count = int(current.get("count") or 0)
```

```
next_count = max(0, current_count - int(delta))
```

```
updates = {
```

```
    "count": next_count,
```

```
    "message": _resident_message("login", next_count),
```

```
    "timestamp": datetime.now(timezone.utc),
```

```
    "read": next_count == 0,
```

```
}
```

```
login_doc.reference.update(updates)
```

```
updated = {**current, **updates, "id": login_doc.id}
```

```
await_broadcast_admin_notification(updated)
```

```
async def _notify_multiple(notifications: List[dict]) -> List[Notification]:
```

```
    """
```

```
    Helper to notify multiple roles/types in one call.
```

```
    Returns the list of Notification objects created.
```

```
    """
```

```
    results = []
```

```
    for n in notifications:
```

```
        notif = await NotificationService.notify(**n)
```

```
        results.append(notif)
```

```
    return results
```

```
@router.get("/", response_model=List[Notification])
```

```
async def get_notifications(user: dict = Depends(get_current_user)):
```

```
    """
```

```
    Fetch notifications for the current user.
```

```
    - Admin: see all notifications.
```

```
    - Resident: only own notifications (user_id filter).
```

```
    - Other roles: only notifications addressed to their role.
```

```
    """
```

```
    role = user.get("role")
```

```
    uid = user.get("uid")
```

```
    try:
```

```

query = get_db().collection("notifications")

if role == "admin":
    docs = query.where("role", "==", "admin").stream()
elif role == "resident":
    docs = query.where("user_id", "==", uid).stream()
else:
    docs = query.where("role", "==", role).stream()

notifications = []
for doc in docs:
    data = doc.to_dict()
    deleted_by = {str(item) for item in (data.get("deleted_by") or [])}
    if uid and str(uid) in deleted_by:
        continue
    try:
        notifications.append(Notification(**data))
    except Exception as e:
        # Skip malformed documents
        import logging
        logging.getLogger("uvicorn.error").warning(" ⚠ Skipping invalid notification doc:
%s", e)

notifications.sort(key=lambda n: n.timestamp, reverse=True)

return notifications

```

```
except Exception as e:
```

```
    raise HTTPException(status_code=500, detail=f"Failed to fetch notifications: {str(e)}")
```

```
@router.patch("/{notification_id}/read", response_model=Notification)
```

```
async def mark_notification_read(
```

```
    notification_id: str = Path(..., description="Notification ID"),
```

```
    user: dict = Depends(get_current_user)
```

```
):
```

```
    """
```

```
    Mark a notification as read.
```

```
    - Residents can only mark their own notifications.
```

```
    - Admin/staff/secretary can mark any notification.
```

```
    """
```

```
    role = user.get("role")
```

```
    uid = user.get("uid")
```

```
    try:
```

```
        doc_ref = get_db().collection("notifications").document(notification_id)
```

```
        doc = doc_ref.get()
```

```
    if not doc.exists:
```

```
        raise HTTPException(status_code=404, detail="Notification not found")
```

```
    data = doc.to_dict()
```

```

# Residents can only mark their own notifications
if role == "resident" and data.get("user_id") != uid:
    raise HTTPException(status_code=403, detail="Not authorized to modify this
notification")

# Update read status
doc_ref.update({"read": True})
data["read"] = True

return Notification(**data)

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to update notification: {str(e)}")

@router.post("/resident-login", response_model=Notification)
async def resident_login(payload: ResidentLoginPayload, user: dict =
Depends(get_current_user)):
    """Admin unread resident-login aggregate increments until read."""
    step = max(1, int(payload.count or 1))
    _record_login_event(
        scope="resident",
        actor_role="resident",
        actor_uid=user.get("uid"),
        actor_name=user.get("fullName") or user.get("name") or user.get("email"),
        count=step,

```

```

)

updated = await _upsert_unread_resident_aggregate("login", step)

return Notification(**updated)


@router.post("/resident-logout", response_model=Notification)

async def resident_logout(payload: ResidentLoginPayload, user: dict =
Depends(get_current_user)):

    """Admin unread resident-logout aggregate increments and unread login aggregate
    decrements."""

    step = max(1, int(payload.count or 1))

    await _decrement_unread_resident_logins(step)

    updated = await _upsert_unread_resident_aggregate("logout", step)

    return Notification(**updated)


@router.post("/officer-login", response_model=Notification)

async def officer_login(payload: OfficerLoginPayload, user: dict =
Depends(get_current_user)):

    """Admin receives officer login notifications with names."""

    officer_role = (payload.role or "officer").replace("_", " ").title()

    _record_login_event(

        scope="officer",

        actor_role=(payload.role or user.get("role") or "officer"),

        actor_uid=user.get("uid"),

        actor_name=payload.name,

        count=1,

    )

    return await NotificationService.notify(

```

```
    role="admin",
    type="login",
    message=f"{officer_role} {payload.name} logged in",
    scope="officer",
    user=payload.name,
)
```

```
@router.post("/officer-logout", response_model=Notification)

async def officer_logout(payload: OfficerLoginPayload, user: dict =
Depends(get_current_user)):
    """Admin receives officer logout notifications with names."""
    officer_role = (payload.role or "officer").replace("_", " ").title()
    return await NotificationService.notify(
        role="admin",
        type="logout",
        message=f"{officer_role} {payload.name} logged out",
        scope="officer",
        user=payload.name,
    )
```

```
@router.post("/incident", response_model=List[Notification])

async def incident_submitted(resident_name: str, user: dict = Depends(get_current_user)):
    """Admin + staff receive incident submission notifications."""
    return await _notify_multiple([
        {"role": "admin", "type": "incident", "message": f"Incident submitted by
{resident_name}"},
```

```
        {"role": "staff", "type": "incident", "message": "New incident submitted"},
    ])
)
```

```
@router.post("/complaint", response_model=List[Notification])
```

```
async def complaint_submitted(resident_name: str, user: dict =
Depends(get_current_user)):
```

```
    """Admin + staff receive complaint submission notifications."""
```

```
    return await _notify_multiple([
```

```
        {"role": "admin", "type": "complaint", "message": f"Complaint submitted by
{resident_name}"},
```

```
        {"role": "staff", "type": "complaint", "message": "New complaint submitted"},
```

```
    ])
```

```
@router.post("/business", response_model=List[Notification])
```

```
async def business_registration(resident_name: str, user: dict =
Depends(get_current_user)):
```

```
    """Admin + staff receive business registration notifications."""
```

```
    return await _notify_multiple([
```

```
        {"role": "admin", "type": "business", "message": f"Business registration submitted by
{resident_name}"},
```

```
        {"role": "staff", "type": "business", "message": "New business registration submitted"},
```

```
    ])
```

```
@router.post("/business-submitted", response_model=List[Notification])
```

```
async def business_submitted(payload: BusinessSubmittedPayload, user: dict =
Depends(get_current_user)):
```



```

"""Admin + staff receive business submission notifications."""

business_suffix = f" ({payload.business_name})" if payload.business_name else ""

return await _notify_multiple([

    {

        "role": "admin",

        "type": "business",

        "message": f"Business registration submitted by
{payload.resident_name}{business_suffix}",

    },

    {

        "role": "staff",

        "type": "business",

        "message": f"New business registration submitted{business_suffix}",

    },

])

```

```

@router.post("/business-status-update", response_model=List[Notification])

async def business_status_update(payload: BusinessStatusUpdatePayload, user: dict =
Depends(get_current_user)):

    """Admin + staff + owner resident receive business status update notifications."""

    status_label = payload.status.replace("_", " ")

    business_suffix = f" ({payload.business_name})" if payload.business_name else ""

    resolved_resident_uid = _resolve_business_owner_uid(payload)

    notifications = [

        {

            "role": "admin",

```

```

        "type": "business_update",
        "message": f"Business status updated to {status_label}{business_suffix}",
    },
    {
        "role": "staff",
        "type": "business_update",
        "message": f"Business status updated to {status_label}{business_suffix}",
    },
]

```

```

if resolved_resident_uid:

```

```

    notifications.append({
        "role": "resident",
        "type": "business_update",
        "message": f"Your business status was updated to {status_label}{business_suffix}",
        "user_id": resolved_resident_uid,
    })

```

```

return await _notify_multiple(notifications)

```

```

@router.post("/document", response_model=List[Notification])

```

```

async def document_request(resident_name: str, user: dict = Depends(get_current_user)):

```

```

    """Admin + secretary receive document request notifications."""

```

```

    return await _notify_multiple([

```

```

        {"role": "admin", "type": "document", "message": f"Document request submitted by {resident_name}"},

```

```
    {"role": "secretary", "type": "document", "message": "New document request  
submitted"},  
    ])
```

```
@router.delete("/actions/bulk-delete", response_model=dict)
```

```
async def bulk_delete_notifications_actions(
```

```
    only_read: bool = Query(False, description="Delete only read notifications"),
```

```
    user: dict = Depends(get_current_user)
```

```
):
```

```
    """
```

```
    Bulk delete notifications for current user scope.
```

```
    - Applies per-user hide ( `deleted_by` ) for this account.
```

```
    - Permanently deletes only when all intended recipients have deleted.
```

```
    - Optionally restrict to only read notifications.
```

```
    """
```

```
    role = user.get("role")
```

```
    uid = user.get("uid")
```

```
    try:
```

```
        docs = _iter_scoped_notification_docs(role, uid)
```

```
        deleted_ids = []
```

```
        globally_deleted_ids = []
```

```
        for doc in docs:
```

```
            data = doc.to_dict() or {}
```

```
            if only_read and not bool(data.get("read")):
```

```
                continue
```

```

        result = _delete_for_user_or_globally(doc, str(uid))

        if result.get("status") in {"deleted_for_user", "deleted_globally"}:
            deleted_ids.append(doc.id)

        if result.get("status") == "deleted_globally":
            globally_deleted_ids.append(doc.id)

    return {
        "status": "bulk_deleted_for_user",
        "count": len(deleted_ids),
        "deleted_ids": deleted_ids,
        "globally_deleted_ids": globally_deleted_ids,
    }

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to bulk delete notifications: {str(e)}")

@router.delete("/actions/delete-all", response_model=dict)
async def delete_all_notifications_actions(
    user: dict = Depends(get_current_user)
):
    """
    Delete all notifications for current user scope.

    - Applies per-user hide ( ` deleted_by` ) for this account.

    - Permanently deletes only when all intended recipients have deleted.
    """

```

```

role = user.get("role")

uid = user.get("uid")

try:

    docs = _iter_scoped_notification_docs(role, uid)

    deleted_ids = []

    globally_deleted_ids = []

    for doc in docs:

        result = _delete_for_user_or_globally(doc, str(uid))

        if result.get("status") in {"deleted_for_user", "deleted_globally"}:

            deleted_ids.append(doc.id)

        if result.get("status") == "deleted_globally":

            globally_deleted_ids.append(doc.id)

    return {

        "status": "all_deleted_for_user",

        "count": len(deleted_ids),

        "deleted_ids": deleted_ids,

        "globally_deleted_ids": globally_deleted_ids,

    }

except Exception as e:

    raise HTTPException(status_code=500, detail=f"Failed to delete all notifications: {str(e)}")

@router.patch("/actions/mark-all-read", response_model=dict)

```

```

async def mark_all_notifications_read_actions(
    user: dict = Depends(get_current_user)
):
    """
    Mark all unread notifications as read for the caller scope.

    - Admin: mark admin-targeted notifications.
    - Resident: mark only own notifications.
    - Other roles: mark notifications addressed to their role.
    """

    role = user.get("role")
    uid = user.get("uid")

    try:
        query = get_db().collection("notifications").where("read", "==", False)

        if role == "admin":
            query = query.where("role", "==", "admin")
        elif role == "resident":
            query = query.where("user_id", "==", uid)
        else:
            query = query.where("role", "==", role)

        docs = query.stream()
        updated_ids = []
        now = datetime.now(timezone.utc)
        for doc in docs:

```

```

        ref = get_db().collection("notifications").document(doc.id)
        ref.update({"read": True, "timestamp": now})
        updated_ids.append(doc.id)

    return {"status": "marked_read", "count": len(updated_ids), "updated_ids":
updated_ids}

except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to mark all as read: {str(e)}")

@router.delete("/{notification_id}", response_model=dict)
async def delete_notification(
    notification_id: str = Path(..., description="Notification ID"),
    user: dict = Depends(get_current_user)
):
    """
    Delete a notification for the current user account.

    - Marks notification hidden for this user using `deleted_by`.
    - Permanently deletes from Firestore only when all intended recipients deleted it.
    """
    role = user.get("role")
    uid = user.get("uid")

    try:
        doc_ref = get_db().collection("notifications").document(notification_id)
        doc = doc_ref.get()

```

```

if not doc.exists:
    raise HTTPException(status_code=404, detail="Notification not found")

data = doc.to_dict()

# Role scoping guard
if not _is_notification_visible_to_user(data, role, uid):
    raise HTTPException(status_code=403, detail="Not authorized to delete this
notification")

return _delete_for_user_or_globally(doc, str(uid))

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to delete notification: {str(e)}")

@router.delete("/bulk", response_model=dict)
async def bulk_delete_notifications(
    only_read: bool = Query(False, description="Delete only read notifications"),
    user: dict = Depends(get_current_user)
):
    """

```


Legacy bulk delete endpoint.

Uses same per-user delete semantics as /actions/bulk-delete.

```
"""
```

```
role = user.get("role")
```

```
uid = user.get("uid")
```

```
try:
```

```
    docs = _iter_scoped_notification_docs(role, uid)
```

```
    deleted_ids = []
```

```
    globally_deleted_ids = []
```

```
    for doc in docs:
```

```
        data = doc.to_dict() or {}
```

```
        if only_read and not bool(data.get("read")):
```

```
            continue
```

```
        result = _delete_for_user_or_globally(doc, str(uid))
```

```
        if result.get("status") in {"deleted_for_user", "deleted_globally"}:
```

```
            deleted_ids.append(doc.id)
```

```
        if result.get("status") == "deleted_globally":
```

```
            globally_deleted_ids.append(doc.id)
```

```
    return {
```

```
        "status": "bulk_deleted_for_user",
```

```
        "count": len(deleted_ids),
```

```
        "deleted_ids": deleted_ids,
```

```
        "globally_deleted_ids": globally_deleted_ids,
```

```
    }
```

```
except Exception as e:

    raise HTTPException(status_code=500, detail=f"Failed to bulk delete notifications:
{str(e)}")
```

```
```\n
```

```
backend\\app\\routes\\password_routes.py
```

```
```python
import logging

from fastapi import APIRouter, HTTPException
from backend.app.models.password import ResetApply, ResetRequest
from backend.app.services.password_service import (
    create_reset_token,
    verify_reset_token,
    apply_password_reset,
    find_user_by_email
)

from firebase_admin import auth

import requests

logger = logging.getLogger("uvicorn.error")

router = APIRouter(prefix="/password", tags=["Password Reset"])
```

```
MAIL_URL = "https://sendemailasia-phy3kbzjda-as.a.run.app"
```

```
@router.post("/request")
```

```
async def request_reset(data: ResetRequest):
```

```
    try:
```

```
        user = auth.get_user_by_email(data.email)
```

```
    except Exception:
```

```
        raise HTTPException(status_code=404, detail="User not found")
```

```
    user_record = find_user_by_email(data.email)
```

```
    if not user_record:
```

```
        raise HTTPException(status_code=404, detail="No matching resident or account  
found")
```

```
    token = create_reset_token(data.email)
```

```
    reset_link = f"https://barangay-1721d.web.app/reset-password?token={token}"
```

```
    full_name = user_record.full_name or user.display_name or "User"
```

```
    barangay = user_record.barangay or (user_record.address.barangay if  
user_record.address else "Unknown")
```

```
    payload = {
```

```
        "type": "reset",
```

```
        "fullName": full_name,
```

```
        "email": data.email,
```

```
        "barangay": barangay,
```

```
    "resetLink": reset_link,  
}
```

```
resp = requests.post(MAIL_URL, json=payload)  
if resp.status_code != 200:  
    logger.error("✗ Email service error [%s]: %s", resp.status_code, resp.text)  
    raise HTTPException(status_code=500, detail="Failed to send reset email")  
  
return {"success": True, "message": "Reset email sent"}
```

```
@router.get("/verify/{token}")  
async def verify_reset(token: str):  
    """Verify if a reset token is valid and not expired."""  
    data = verify_reset_token(token)  
    return {"valid": True, "email": data["email"]}
```

```
@router.post("/apply")  
async def apply_reset(data: ResetApply):  
    """Apply a new password if token is valid."""  
    apply_password_reset(data.token, data.new_password)  
    return {"success": True, "message": "Password reset successful"}
```

```
...
```

```
## backend\app\routes\payment_routes.py
```

```
` ``python
```

```
import logging
```

```
import hmac
```

```
import hashlib
```

```
import os
```

```
from backend.app.services.payment_service import log_payment_record,  
_next_receipt_number, _get_business_doc
```

```
from backend.app.services.notification_service import NotificationService
```

```
from fastapi import APIRouter, Request
```

```
from fastapi.responses import JSONResponse
```

```
from google.cloud import firestore
```

```
from fastapi.concurrency import run_in_threadpool
```

```
from backend.app.utils.firestore_utils import get_db
```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(prefix="/paymongo", tags=["Payments"])
```

```
PAYMONGO_WEBHOOK_SECRET = os.getenv("PAYMONGO_WEBHOOK_SECRET", "")
```

```
async def _notify_payment_roles(message: str, event_type: str = "payment_update"):
```

```
    for target_role in ("admin", "treasurer", "staff"):
```

```
        try:
```

```

        await NotificationService.notify(
            role=target_role,
            type=event_type,
            message=message,
        )
    except Exception as notify_err:
        logger.warning(" ⚠️ Payment notification failed for role=%s: %s", target_role,
            notify_err)

def verify_signature(raw_body: bytes, header_signature: str) -> bool:
    if not PAYMONGO_WEBHOOK_SECRET:
        logger.error(" ❌ PAYMONGO_WEBHOOK_SECRET not set")
        return False

    parts = {}
    for item in header_signature.split(","):
        if "=" in item:
            k, v = item.split("=", 1)
            parts[k.strip()] = v.strip()

    timestamp = parts.get("t")
    provided = parts.get("s") or parts.get("te") or parts.get("li")

    if not timestamp or not provided:
        logger.error(" ❌ Missing timestamp or signature field in header: %s", header_signature)
        return False

```

```

signed_payload = f"{timestamp}.{raw_body.decode('utf-8')}".encode("utf-8")

computed = hmac.new(
    PAYMONGO_WEBHOOK_SECRET.encode("utf-8"),
    signed_payload,
    hashlib.sha256
).hexdigest()

valid = hmac.compare_digest(computed, provided)

if not valid:
    logger.warning(" ⚠️ Signature mismatch. computed=%s provided=%s", computed,
provided)

    return valid


def _next_transaction_id():
    counter_ref = get_db().collection("counters").document("transactions")
    transaction = get_db().transaction()

    @firestore.transactional
    def increment_counter(transaction):
        snapshot = counter_ref.get(transaction=transaction)
        current = snapshot.get("value") if snapshot.exists else 0
        new_value = current + 1
        transaction.set(counter_ref, {"value": new_value})
        return new_value

```

```
new_number = increment_counter(transaction)
```

```
return f"TXN-{new_number:05d}"
```

```
@router.post("/webhook")
```

```
async def paymongo_webhook(request: Request):
```

```
    try:
```

```
        raw_body = await request.body()
```

```
        header_signature = request.headers.get("Paymongo-Signature", "")
```

```
        logger.debug("🔴 Raw webhook body=%s", raw_body.decode("utf-8"))
```

```
        logger.debug("🔴 Signature header=%s", header_signature)
```

```
        if not header_signature or not verify_signature(raw_body, header_signature):
```

```
            return JSONResponse(status_code=400, content={"success": False, "message":  
"Invalid signature"})
```

```
        payload = await request.json()
```

```
        attributes = payload.get("data", {}).get("attributes", {})
```

```
        event_type = attributes.get("type")
```

```
        inner_data = attributes.get("data", {})
```

```
        inner_attrs = inner_data.get("attributes", {})
```

```
        status = inner_attrs.get("status")
```

```
        metadata = inner_attrs.get("metadata", {}) or {}
```

```
        reference_number = (
```

```
            inner_attrs.get("reference_number")
```



```

        or inner_attrs.get("externalReferenceNumber")
        or metadata.get("pmReferenceNumber")
    )
    paid_at = inner_attrs.get("paidAt")

    transaction_id = inner_data.get("id")
    intent_id = inner_attrs.get("paymentIntentId")

    # logger.info("📦 Webhook event=%s status=%s metadata=%s ref=%s",
    #             event_type, status, metadata, reference_number)

    allowed_events = {
        "link.payment.paid",
        "payment.paid",
        "payment.failed",
        "payment.cancelled",
        "payment.refunded",
        "source.chargeable",
        "source.consumed"
    }

    if event_type not in allowed_events:
        return JSONResponse(status_code=200, content={"success": True, "message":
f"Ignored event {event_type}"})

    if not status:

```

```
    return JSONResponse(status_code=400, content={"success": False, "message":
"Invalid payload"})
```

```
workflow_map = {
    "paid": "payment_submitted",
    "failed": "payment_failed",
    "cancelled": "payment_cancelled",
    "refunded": "payment_refunded"
}
workflow_status = workflow_map.get(status, status)
```

```
update_data = {
    "paymentStatus": status,
    "status": workflow_status,
    "transactionId": transaction_id,
    "paymentIntentId": intent_id,
    "paymentDate": paid_at or firestore.SERVER_TIMESTAMP,
    "eventType": event_type
}
```

```
# --- Business update ---
```

```
if "businessId" in metadata:
```

```
    docs = get_db().collection("businesses").where("businessId", "==",
metadata["businessId"]).limit(1).get()
```

```
    if docs:
```

```
        await run_in_threadpool(docs[0].reference.update, update_data)
```

```
logger.info("✅ Updated business=%s status=%s", metadata["businessId"], status)
```

```
business_data = docs[0].to_dict()
```

```
log_payment_record(  
    reference_number=reference_number or transaction_id,  
    transaction_id=transaction_id,  
    amount=(inner_attrs.get("amount") or 0) / 100,  
    status=status,  
    fee_type=metadata.get("feeType"),  
    business_id=metadata.get("businessId"),  
    owner_name=business_data.get("ownerName"),  
    business_name=business_data.get("businessName"),  
    business_type=business_data.get("businessType"),  
    event_type=event_type,  
    paid_at=paid_at,  
    method="paymongo"  
)
```

```
await _notify_payment_roles(  
    f"Business payment {status} ({metadata.get('businessId')})",  
    "payment_update",  
)
```

```
# --- Document update via Firestore ID ---
```

```
elif "documentId" in metadata:
```

```

docs = get_db().collection("documents").where("documentId", "==",
metadata["documentId"]).limit(1).get()

if docs:

    await run_in_threadpool(docs[0].reference.update, update_data)

    logger.info("✅ Updated document via documentId=%s status=%s",
metadata["documentId"], status)

```

```

doc_data = docs[0].to_dict()

```

```

# 🖱️ Log payment + receipt here

```

```

log_payment_record(
    reference_number=reference_number or transaction_id,
    transaction_id=transaction_id,
    amount=(inner_attrs.get("amount") or 0) / 100,
    status=status,
    fee_type=metadata.get("feeType"),
    document_id=metadata.get("documentId"),
    owner_name=doc_data.get("ownerName"),
    business_name=doc_data.get("businessName"),
    document_type=doc_data.get("documentType"),
    event_type=event_type,
    paid_at=paid_at,
    method="paymongo"
)

await _notify_payment_roles(
    f"Document payment {status} ({metadata.get('documentId')})",
    "payment_update",

```

```

    )

    else:

        logger.warning(" ⚠️ No document found for documentId=%s",
            metadata["documentId"])

    # --- Fallback: referenceNumber ---

    elif reference_number:

        # Try businesses first

        docs = get_db().collection("businesses").where("referenceNumber", "=",
            reference_number).limit(1).get()

        if docs:

            await run_in_threadpool(docs[0].reference.update, update_data)

            logger.info(" ✅ Updated business via referenceNumber=%s status=%s",
                reference_number, status)

    business_data = docs[0].to_dict()

    log_payment_record(
        reference_number=reference_number or transaction_id,
        transaction_id=transaction_id,
        amount=(inner_attrs.get("amount") or 0) / 100,
        status=status,
        fee_type=metadata.get("feeType"),
        business_id=business_data.get("businessId"),
        owner_name=business_data.get("ownerName"),
        business_name=business_data.get("businessName"),
        business_type=business_data.get("businessType"),
        event_type=event_type,

```

```

        paid_at=paid_at,

        method="paymongo"

    )

    await _notify_payment_roles(

        f"Business payment {status} ({business_data.get('businessId') or
reference_number})",

        "payment_update",

    )

else:

    # Then try documents

    docs = get_db().collection("documents").where("referenceNumber", "=",
reference_number).limit(1).get()

    if docs:

        await run_in_threadpool(docs[0].reference.update, update_data)

        logger.info("✅ Updated document via referenceNumber=%s status=%s",
reference_number, status)


    doc_data = docs[0].to_dict()

    log_payment_record(

        reference_number=reference_number or transaction_id,

        transaction_id=transaction_id,

        amount=(inner_attrs.get("amount") or 0) / 100,

        status=status,

        fee_type=metadata.get("feeType"),

        document_id=doc_data.get("documentId"),

        owner_name=doc_data.get("ownerName"),

        business_name=doc_data.get("businessName"),

```

```

        document_type=doc_data.get("documentType"),
        event_type=event_type,
        paid_at=paid_at,
        method="paymongo"
    )

    await _notify_payment_roles(
        f"Document payment {status} ({doc_data.get('documentId') or
reference_number})",
        "payment_update",
    )

else:

    logger.warning(" ⚠️ No record found for referenceNumber=%s",
reference_number)

    return JSONResponse(status_code=200, content={"success": False, "message":
"Unmatched webhook"})

else:

    logger.warning(" ⚠️ No identifiers in webhook payload: %s", payload)

    return JSONResponse(status_code=200, content={"success": False, "message":
"Unmatched webhook"})

return {"success": True}

except Exception as e:

    logger.exception(" ❌ Webhook processing failed: %s", e)

    return JSONResponse(status_code=500, content={"success": False, "message":
"Webhook error"})

```

```
@router.post("/payments/business")

async def record_business_payment(payload: dict):

    business_id = payload["businessId"]

    amount = payload["amount"]

    method = payload.get("method")


    # fetch business doc

    doc = _get_business_doc(business_id)

    if not doc:

        return {"success": False, "message": "Business not found"}


    business_data = doc.to_dict()


    transaction_id = payload.get("transactionId") or _next_transaction_id()

    receipt_number = _next_receipt_number()


    log_payment_record(

        reference_number=business_data.get("referenceNumber") or business_id,

        transaction_id=transaction_id,

        amount=amount,

        status="paid",

        fee_type="business_fee",

        business_id=business_id,

        business_name=business_data.get("businessName"),

        owner_name=business_data.get("ownerName"),
```



```
business_type=business_data.get("businessType"),
event_type="staff.payment",
paid_at=firestore.SERVER_TIMESTAMP,
method=method,
receipt_number=receipt_number # pass explicitly
)
```

```
response = {
    "success": True,
    "receiptNumber": receipt_number,
    "transactionId": transaction_id,
    "businessId": business_id,
    "businessName": business_data.get("businessName"),
    "ownerName": business_data.get("ownerName"),
    "businessType": business_data.get("businessType"),
    "barangay": business_data.get("barangay"),
    "method": method
}
```

```
await _notify_payment_roles(
    f"Business payment paid ({business_id})",
    "payment",
)
```

```
return response
```

```
@router.post("/payments/document")
```

```
async def record_document_payment(payload: dict):

    try:

        document_id = payload["documentId"]

        amount = payload["amount"]

        method = payload.get("method")


        # fetch document doc

        doc_ref = get_db().collection("documents").document(document_id)

        snapshot = doc_ref.get()

        if not snapshot.exists:

            return {"success": False, "message": "Document not found"}


        doc_data = snapshot.to_dict()


        transaction_id = payload.get("transactionId") or _next_transaction_id()

        receipt_number = _next_receipt_number()


        # 🗝 Update document payment status immediately

        update_data = {

            "paymentStatus": "paid",

            "status": "paid", # secretary payments can be final

            "transactionId": transaction_id,

            "paymentDate": firestore.SERVER_TIMESTAMP,

            "method": method,

            "eventType": "staff.payment"

        }
```

```
doc_ref.update(update_data)
```

```
# log payment + receipt
```

```
log_payment_record(  
    reference_number=doc_data.get("referenceNumber") or document_id,  
    transaction_id=transaction_id,  
    amount=amount,  
    status="paid",  
    fee_type="document_fee",  
    document_id=document_id,  
    owner_name=doc_data.get("ownerName"),  
    business_name=doc_data.get("businessName"),  
    document_type=doc_data.get("documentType"),  
    event_type="staff.payment",  
    paid_at=firestore.SERVER_TIMESTAMP,  
    method=method,  
    receipt_number=receipt_number  
)
```

```
response = {  
    "success": True,  
    "receiptNumber": receipt_number,  
    "transactionId": transaction_id,  
    "documentId": document_id,  
    "documentType": doc_data.get("documentType"),  
    "ownerName": doc_data.get("ownerName"),
```

```

        "businessName": doc_data.get("businessName"),
        "method": method
    }
    await _notify_payment_roles(
        f"Document payment paid ({document_id})",
        "payment",
    )
    return response

except Exception as e:
    logger.exception(" ❌ Document payment failed: %s", e)
    return JSONResponse(status_code=500, content={"success": False, "message":
"Payment error", "details": str(e)})

...

## backend\app\routes\paymongo_routes.py

```python
import base64
import logging
import os
import requests
from fastapi import APIRouter, HTTPException
from fastapi.concurrency import run_in_threadpool
from backend.app.utils.firestore_utils import get_db

```

```
from backend.app.services.paymongo_service import create_payment_link,
create_payment_intent, attach_payment_method
```

```
from backend.app.models.paymongo import DocumentPaymentRequest,
BusinessPaymentRequest, AttachPaymentRequest
```

```
from backend.app.routes.fee_routes import (
 compute_document_fee,
 compute_business_registration_fee,
 compute_business_annual_fee,
)
```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(tags=["Payments"])
```

```

```

```
Firestore helpers
```

```

```

```
def _get_document_doc(document_id: str):
```

```
 docs = get_db().collection("documents").where("documentId", "==",
document_id).limit(1).get()
```

```
 return docs[0] if docs else None
```

```
def _get_business_doc(business_id: str):
```

```
 docs = get_db().collection("businesses").where("businessId", "==",
business_id).limit(1).get()
```

```
 return docs[0] if docs else None
```

```

Shared payment creation logic

def _create_payment(fee: int, description: str, remarks: str, metadata: dict,
 success_url: str, cancel_url: str):
 """Create either a Payment Intent (< ₱100) or Payment Link (≥ ₱100)."""
 if fee < 100:
 result = create_payment_intent(
 amount=fee, description=description, remarks=remarks,
 metadata=metadata
)
 return {
 "checkoutUrl": result.get("checkoutUrl"),
 "referenceNumber": result.get("referenceNumber"),
 "paymentStatus": result.get("paymentStatus") or "awaiting_payment" or
"for_payment",
 "paymentIntentId": result.get("paymentIntentId"),
 "paymongoClientKey": result.get("paymongoClientKey"),
 "type": "intent"
 }
 else:
 result = create_payment_link(
 amount=fee, description=description, remarks=remarks,
 metadata=metadata, success_url=success_url, cancel_url=cancel_url
)
 return {

```

```

 "checkoutUrl": result.get("checkoutUrl"),
 "referenceNumber": result.get("referenceNumber"),
 "paymentStatus": result.get("paymentStatus") or "awaiting_payment" or
"for_payment",
 "paymongoLinkId": result.get("paymongoLinkId"),
 "type": "link"
 }

```

```

```

```

Document Payment Route

```

```

```

```

@router.post("/create-document-link")

```

```

async def create_document_payment_link(payload: DocumentPaymentRequest) -> dict:

```

```

 try:

```

```

 fee = compute_document_fee(payload.documentType)

```

```

 if fee <= 0:

```

```

 raise HTTPException(status_code=400, detail=f"Invalid fee for document type:
{payload.documentType}")

```

```

 description = f"{payload.documentType} Request {payload.documentId}"

```

```

 metadata = {"documentId": payload.documentId, "documentType":
payload.documentType}

```

```

 result = _create_payment(

```

```

 fee, description, payload.remarks, metadata,

```

```

 success_url="http://localhost:3000/payment-success?type=document",

```

```

 cancel_url="http://localhost:3000/documents/payment-cancel"

```

)

```
doc = _get_document_doc(payload.documentId)
```

```
if doc:
```

```
 update_data = {
 "checkoutUrl": result["checkoutUrl"],
 "paymentStatus": result["paymentStatus"],
 "status": "for_payment",
 "fee": fee,
 "referenceNumber": result.get("referenceNumber"),
 "paymentIntentId": result.get("paymentIntentId"),
 "paymongoClientKey": result.get("paymongoClientKey"),
 "paymongoLinkId": result.get("paymongoLinkId")
 }
```

```
 await run_in_threadpool(doc.reference.update, update_data)
```

```
else:
```

```
 logger.warning(" ⚠ No Firestore document found for %s", payload.documentId)
```

```
return {"success": True, "fee": fee, **result}
```

```
except Exception as e:
```

```
 logger.exception(" ❌ Failed to create document payment: %s", e)
```

```
 raise HTTPException(status_code=500, detail="Document payment creation failed")
```

```

```

```
Business Payment Route
```



```

@router.post("/create-business-link")
async def create_business_payment_link(payload: BusinessPaymentRequest) -> dict:

 try:

 fee = compute_business_annual_fee(payload.businessType) if payload.feeType ==
"annual" \

 else compute_business_registration_fee(payload.businessType)

 if fee <= 0:

 raise HTTPException(status_code=400, detail=f"Invalid fee for {payload.feeType} of
{payload.businessType}")

 description = f"{payload.feeType} for {payload.businessType} ({payload.businessId})"

 metadata = {"businessId": payload.businessId, "businessType": payload.businessType,
"feeType": payload.feeType}

 result = _create_payment(

 fee, description, payload.remarks, metadata,

 success_url="http://localhost:3000/payment-success?type=business",

 cancel_url="http://localhost:3000/business/payment-cancel"

)

 doc = _get_business_doc(payload.businessId)

 if doc:

 update_data = {

 "fee": fee,

 "feeType": payload.feeType,

 "status": "awaiting_payment",

```

```

 "paymentStatus": result["paymentStatus"],
 "checkoutUrl": result["checkoutUrl"],
 "referenceNumber": result.get("referenceNumber"),
 "paymentIntentId": result.get("paymentIntentId"),
 "paymongoClientKey": result.get("paymongoClientKey"),
 "paymongoLinkId": result.get("paymongoLinkId")
 }

 await run_in_threadpool(doc.reference.update, update_data)
else:
 logger.warning(" ⚠️ No Firestore business found for %s", payload.businessId)

 return {"success": True, "fee": fee, **result}

except Exception as e:
 logger.exception(" ❌ Failed to create business payment: %s", e)
 raise HTTPException(status_code=500, detail="Business payment creation failed")

Attach Payment Method Route

@router.post("/attach-payment-method")

async def attach_payment_method(payload: AttachPaymentRequest) -> dict:
 """
 Attach a payment method (GCash/GrabPay) to a PayMongo Payment Intent.
 Supports both business and document flows by setting the correct return_url.
 """

```

```

try:
 PAYMONGO_SECRET_KEY = os.getenv("PAYMONGO_SECRET_KEY")
 if not PAYMONGO_SECRET_KEY:
 raise HTTPException(status_code=500, detail="PayMongo secret key not configured")

 headers = {
 "Authorization": f"Basic
{base64.b64encode(PAYMONGO_SECRET_KEY.encode()).decode()}",
 "Content-Type": "application/json"
 }

 # Step 1: Create payment method
 pm_payload = {
 "data": {
 "attributes": {
 "type": payload.method, # "gcash" or "grab_pay"
 "billing": payload.billing.model_dump()
 }
 }
 }

 pm_res = requests.post("https://api.paymongo.com/v1/payment_methods",
json=pm_payload, headers=headers)

 pm_data = pm_res.json()

 # logger.info(" 📄 Payment method response: %s", pm_data)

 if "errors" in pm_data:

```

```

logger.error(" ❌ Payment method creation failed: %s", pm_data)

raise HTTPException(status_code=400, detail="Payment method creation failed")

payment_method_id = pm_data["data"]["id"]

Step 2: Attach to intent
if payload.type == "business":
 return_url = "http://localhost:3000/payment-success?type=business"
else:
 return_url = "http://localhost:3000/payment-success?type=document"

attach_payload = {
 "data": {
 "attributes": {
 "payment_method": payment_method_id,
 "client_key": payload.paymongoClientKey,
 "return_url": return_url
 }
 }
}

intent_res = requests.post(
 f"https://api.paymongo.com/v1/payment_intents/{payload.paymentIntentId}/attach",
 json=attach_payload, headers=headers
)

intent_data = intent_res.json()

logger.info(" 📥 Attach response: %s", intent_data)

```

```

if "errors" in intent_data:

 logger.error(" ❌ Payment intent attach failed: %s", intent_data)

 raise HTTPException(status_code=400, detail="Payment intent attach failed")

attrs = intent_data["data"]["attributes"]

redirect_url = attrs.get("next_action", {}).get("redirect", {}).get("url")

if not redirect_url:

 logger.error(" ⚠️ No redirect URL in intent attach response: %s", intent_data)

 raise HTTPException(status_code=500, detail="No redirect URL returned from PayMongo")

return {

 "redirectUrl": redirect_url,

 "status": attrs.get("status"),

 "referenceNumber": attrs.get("reference_number"),

 "paymentIntentId": payload.paymentIntentId

}

except Exception as e:

 logger.exception(" ❌ Failed to attach payment method: %s", e)

 raise HTTPException(status_code=500, detail="Attach payment method failed")

...

backend\app\routes\resident_routes.py

```

```
```python
```

```
import logging
```

```
from fastapi import APIRouter, Query, Body, HTTPException, status, Request
```

```
from typing import Optional, List
```

```
from starlette.concurrency import run_in_threadpool
```

```
from backend.app.models import ResidentCreate, ResidentUpdate, ResidentOut
```

```
from backend.app.services import resident_service
```

```
from backend.app.services.resident_service import ResidentError
```

```
from pydantic import BaseModel, ValidationError
```

```
logger = logging.getLogger("uvicorn.error")
```

```
router = APIRouter(tags=["Residents"])
```

```
# 📦 Response models
```

```
class BulkResidentResponse(BaseModel):
```

```
    householdId: str
```

```
    count: int
```

```
    items: List[ResidentOut]
```

```
    message: str
```

```
class DeleteResponse(BaseModel):
```

```
    id: Optional[str] = None
```

```
    householdId: Optional[str] = None
```

```
    message: str
```

```

# 🛠️ Safe service call wrapper with detailed error logging

async def safe_service_call(context: str, func, *args, **kwargs):
    try:
        return await run_in_threadpool(func, *args, **kwargs)
    except ValidationError as e:
        # Log detailed validation errors
        logger.error("❌ Validation error in %s: %s", context, e.errors())
        raise HTTPException(status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=e.errors())
    except ResidentError as e:
        logger.warning("⚠️ %s: %s", context, str(e))
        raise HTTPException(status_code=400, detail=str(e))
    except RuntimeError as e:
        logger.warning("⚠️ %s not found: %s", context, str(e))
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(e))
    except HTTPException:
        raise
    except Exception as e:
        # Log full stack trace for unexpected errors
        logger.error("❌ %s failed: %s", context, str(e), exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Service call '{context}' failed"
        )

```

```

# 🚀 GET /residents

```

```
@router.get("/residents", response_model=List[ResidentOut])
```

```
async def list_residents(
```

```
    limit: int = Query(50, ge=1, le=100),
```

```
    start_after_id: Optional[str] = Query(None),
```

```
    full_name: Optional[str] = Query(None, alias="fullName"),
```

```
    birth_date: Optional[str] = Query(None, alias="birthDate")
```

```
):
```

```
    if full_name and birth_date:
```

```
        return await safe_service_call(
```

```
            "find duplicates",
```

```
            resident_service.find_duplicates,
```

```
            full_name,
```

```
            birth_date
```

```
        )
```

```
    return await safe_service_call(
```

```
        "list residents",
```

```
        resident_service.get_all_residents,
```

```
        limit,
```

```
        start_after_id
```

```
    )
```

```
@router.get("/residents/{id}", response_model=ResidentOut)
```

```
async def get_resident(id: str):
```

```
    logger.info("🔴 Fetching resident with ID: %s", id)
```

```
    return await safe_service_call("get resident", resident_service.get_resident_by_id, id)
```



```
# 🚀 POST /residents

@router.post("/residents", response_model=ResidentOut,
status_code=status.HTTP_201_CREATED)

async def add_resident(data: ResidentCreate = Body(...)) -> ResidentOut:

    logger.debug("📥 Incoming resident payload: %s", data.model_dump(by_alias=True))

    return await safe_service_call(

        "create resident",

        resident_service.add_resident,

        data.model_dump(by_alias=True)

    )
```

```
# 🚀 PUT /residents/{id}

@router.put("/residents/{id}", response_model=ResidentOut)

async def update_resident(id: str, data: ResidentUpdate = Body(...)) -> ResidentOut:

    logger.debug("📥 Update resident %s payload: %s", id,
data.model_dump(by_alias=True))

    return await safe_service_call(

        "update resident",

        resident_service.update_resident,

        id,

        data.model_dump(by_alias=True)

    )
```

```
# 🚀 PATCH /residents/{id}

@router.patch("/residents/{id}", response_model=ResidentOut)
```

```
async def patch_resident(id: str, data: ResidentUpdate = Body(...)) -> ResidentOut:
```

```
    logger.debug("🔧 Patch resident %s payload: %s", id,  
data.model_dump(exclude_unset=True, by_alias=True))  
  
    return await safe_service_call(  
        "patch resident",  
        resident_service.patch_resident,  
        id,  
        data.model_dump(exclude_unset=True, by_alias=True)  
    )
```

```
# 🚀 DELETE /residents/{id}
```

```
@router.delete("/residents/{id}", response_model=DeleteResponse)
```

```
async def delete_resident(id: str):
```

```
    logger.info("🗑️ Deleting resident with ID: %s", id)  
  
    return await safe_service_call("delete resident", resident_service.delete_resident, id)
```

```
# 🚀 GET /households/{householdId}
```

```
@router.get("/households/{householdId}", response_model=List[ResidentOut])
```

```
async def get_household_residents(householdId: str):
```

```
    logger.info("🔧 Fetching residents for household %s", householdId)  
  
    return await safe_service_call("fetch household residents",  
resident_service.get_residents_by_household, householdId)
```

```
# 🚀 DELETE /households/{householdId}
```

```
@router.delete("/households/{householdId}", response_model=DeleteResponse)
```

```
async def delete_household_residents(householdId: str):
```

```

logger.info(" 🗑 Deleting residents in household %s", householdId)

return await safe_service_call("delete household residents",
resident_service.delete_by_household, householdId)

# 🚀 POST /residents/bulk

@router.post("/residents/bulk", response_model=BulkResidentResponse)
async def add_residents_bulk(
    data: List[ResidentCreate] = Body(...),
    household_id: Optional[str] = Query(None, alias="householdId")
):
    logger.debug(" 📦 Bulk resident payload count: %d", len(data))
    result = await safe_service_call(
        "bulk create residents",
        resident_service.add_residents_bulk,
        [d.model_dump(by_alias=True) for d in data],
        household_id
    )
    if "message" not in result:
        result["message"] = "Bulk residents created successfully"
    return result

```

```

# 🚀 DEBUG /residents/debug

@router.post("/residents/debug")
async def debug_resident(request: Request):
    """

```

Debug endpoint: manually validate incoming payload against ResidentCreate.

Useful for catching validation errors that FastAPI swallows.

```
"""
```

```
body = await request.json()
```

```
logger.debug("🚫 Raw incoming payload: %s", body)
```

```
try:
```

```
    resident = ResidentCreate.model_validate(body)
```

```
    logger.info("✅ ResidentCreate validated successfully")
```

```
    return {"parsed": resident.model_dump()}
```

```
except ValidationError as e:
```

```
    logger.error("❌ Debug validation error: %s", e.errors())
```

```
    raise HTTPException(status_code=422, detail=e.errors())
```

```
...
```

```
## backend\app\routes\role_routes.py
```

```
```python
```

```
from fastapi import APIRouter, Depends
```

```
from backend.app.core.auth import set_user_role, get_admin_uid
```

```
from backend.app.models.role import RoleUpdate, RoleResponse
```

```
router = APIRouter()
```

```
@router.post("/users/{uid}/role", response_model=RoleResponse, tags=["Roles"])
```

```
async def assign_role(uid: str, update: RoleUpdate):
```

```
 # If no admins exist yet, allow bootstrap
```

```

 if update.role == "admin":
 return set_user_role(uid, "admin")

 # Otherwise require admin
 _ = Depends(get_admin_uid)

 return set_user_role(uid, update.role)

...

backend\app\routes\settings_routes.py

```python
from fastapi import APIRouter, HTTPException, status, Depends
from backend.app.models.settings import RolePermission, DocumentFee
from backend.app.services.settings_service import SettingsService

router = APIRouter(tags=["Settings"])

# 🗝️ Centralized role-permission map
ALLOWED_ROLE_KEYS = {
    "admin": ["viewDashboard", "manageResidents", "approveClearance",
"generateCertificates"],
    "staff": ["viewDashboard", "fileComplaints", "generateCertificates"],
    "resident": ["viewDashboard", "fileComplaints"],
    # Extend as needed
}

```

🛠️ Dependency injection

```
def get_settings_service():  
    return SettingsService()
```

🗝️ Get all role permissions

```
@router.get("/permissions", response_model=dict)
```

```
def get_permissions(service: SettingsService = Depends(get_settings_service)):  
    try:  
        return service.get_permissions()  
    except Exception as e:  
        raise HTTPException(  
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,  
            detail=f"Failed to fetch permissions: {str(e)}"  
        )
```

🗝️ Update permissions for a specific role

```
@router.post("/permissions")
```

```
def update_permissions(  
    data: RolePermission,  
    service: SettingsService = Depends(get_settings_service)  
):  
    role = data.role  
    permission_map = data.permissions
```

🚫 Prevent admin override

```
if role == "admin":
```

```
raise HTTPException(
    status_code=status.HTTP_403_FORBIDDEN,
    detail="Admin permissions cannot be overridden."
)
```

```
# 🔍 Validate role
```

```
allowed_keys = ALLOWED_ROLE_KEYS.get(role)
```

```
if not allowed_keys:
```

```
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"Unknown role: '{role}'"
    )
```

```
# 🔍 Validate keys in payload
```

```
invalid_keys = [key for key in permission_map.keys() if key not in allowed_keys]
```

```
if invalid_keys:
```

```
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"Invalid permission keys for role '{role}': {invalid_keys}"
    )
```

```
try:
```

```
    success = service.update_permissions(role, permission_map)
```

```
if not success:
```

```
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
```

```

        detail="Permission update failed"
    )

    return {"message": f"✅ Permissions updated for role: {role}"}
except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to update permissions: {str(e)}"
    )

```

💰 Get all document fees

```

@router.get("/fees", response_model=dict)
def get_fees(service: SettingsService = Depends(get_settings_service)):
    try:
        return service.get_fees()
    except Exception as e:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Failed to fetch fees: {str(e)}"
        )

```

💰 Update fee for a specific document type

```

@router.post("/fees")
def update_fee(
    data: DocumentFee,
    service: SettingsService = Depends(get_settings_service)
):

```



```

try:
    success = service.update_fee(data.document_type, data.fee)
    if not success:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Fee update failed"
        )
    return {"message": f"✅ Fee updated for document: {data.document_type}"}
except Exception as e:
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to update fee: {str(e)}"
    )
...

```

backend\app\routes\ws_routes.py

```python

# backend/app/routes/ws\_routes.py

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Query
from starlette.websockets import WebSocketState
from backend.app.core.websocket_manager import manager
from backend.app.core.auth import _verify_token, get_db, require_permission
from fastapi import Depends

```

```
import logging
```

```
import json
```

```
router = APIRouter(tags=["websocket"])
```

```
logger = logging.getLogger("uvicorn.error")
```

```
@router.get("/api/ws/presence/roles")
```

```
async def get_role_presence(_uid: str = Depends(require_permission("manageUsers"))):
```

```
 role_counts = {}
```

```
 for info in manager.active_connections.values():
```

```
 role = str(info.get("role") or "resident").strip().lower()
```

```
 role_counts[role] = role_counts.get(role, 0) + 1
```

```
tracked_roles = ["admin", "secretary", "staff", "treasurer", "sk", "dilig", "resident"]
```

```
roles = {
```

```
 role: {
```

```
 "online": role_counts.get(role, 0) > 0,
```

```
 "count": role_counts.get(role, 0),
```

```
 }
```

```
 for role in tracked_roles
```

```
}
```

```
for role, count in role_counts.items():
```

```
 if role not in roles:
```

```
 roles[role] = {"online": count > 0, "count": count}
```

```
return {
 "roles": roles,
 "total_active_connections": len(manager.active_connections),
}
```

```
@router.websocket("/ws/notifications")
```

```
async def websocket_notifications(websocket: WebSocket, token: str = Query(None)):
```

```
 user_info = None
```

```
 try:
```

```
 await websocket.accept()
```

```
 token_value = None
```

```
 auth_method = None
```

```
 # First, check query string
```

```
 if token:
```

```
 token_value = token
```

```
 auth_method = "query"
```

```
 else:
```

```
 # Otherwise, expect the client to send token as first message
```

```
 try:
```

```
 initial_msg = await websocket.receive_text()
```

```
 payload = json.loads(initial_msg)
```

```
 token_value = payload.get("token")
```

```
 auth_method = "message"
```

except Exception:

    await websocket.close(code=4001, reason="Missing authentication token")

    return

if not token\_value:

    await websocket.close(code=4001, reason="Missing authentication token")

    return

logger.info("🔑 Auth method=%s", auth\_method)

# Verify token

try:

    decoded = \_verify\_token(f"Bearer {token\_value}")

except Exception as e:

    logger.error("❌ Token verification failed: %s", e)

    await websocket.close(code=4001, reason="Invalid token")

    return

uid = decoded.get("uid")

role = decoded.get("role")

if not role and uid:

    db = get\_db()

    user\_doc = db.collection("users").document(uid).get()

    if user\_doc.exists:

        role = user\_doc.to\_dict().get("role")

```

elif db.collection("residents").document(uid).get().exists:
 role = "resident"

role = (str(role or "resident").strip().lower())
user_info = {"uid": uid, "role": role, "user_id": uid, "auth_method": auth_method}

await manager.connect(websocket, user_info)

logger.info("✅ WebSocket connected for uid=%s role=%s via %s", uid, role,
auth_method)

while True:
 await websocket.receive_text()

except WebSocketDisconnect:
 manager.disconnect(websocket)
 logger.info("❌ WebSocket disconnected")

except Exception as e:
 logger.error("❌ WebSocket error uid=%s: %s", user_info.get("user_id") if user_info else
"unknown", e)
 if websocket.client_state != WebSocketState.CLOSED:
 await websocket.close(code=1011)
 manager.disconnect(websocket)

...

```

```
backend\app\scripts__init__.py
```

```
```python
```

```
```
```

```
backend\app\scripts\bootstrap_admin.py
```

```
```python
```

```
# backend/app/scripts/bootstrap_admin.py
```

```
from firebase_admin import auth, credentials, initialize_app
```

```
def main():
```

```
    # Initialize Firebase Admin SDK with your service account
```

```
    cred = credentials.Certificate(r"C:\Projects\BIS\backend\serviceAccountKey.json")
```

```
    initialize_app(cred)
```

```
    uid = "kGC89j9mSWb2jt8FcFvqj7DRTZb2"
```

```
    auth.set_custom_user_claims(uid, {"role": "admin"})
```

```
    print("✅ Admin role set")
```

```
if __name__ == "__main__":
```

```
    main()
```

```
```
```

```
backend\app\services__init__.py
```

```
```python
```

```
```
```

```
backend\app\services\account_service.py
```

```
```python
```

```
import logging
```

```
from typing import Optional
```

```
from datetime import datetime, timezone
```

```
from fastapi import HTTPException, status
```

```
from firebase_admin import auth
```

```
from firebase_admin.exceptions import FirebaseError
```

```
from firebase_admin._auth_utils import UserNotFoundError
```

```
from google.cloud import firestore
```

```
from backend.app.utils.firestore_utils import get_db
```

```
from backend.app.models.account import AccountCreate, AccountResponse, RoleEnum
```

```
from backend.app.core.roles import ROLE_PERMISSIONS
```

```
from backend.app.core.firebase import get_firestore
```

```
logger: logging.Logger = logging.getLogger("uvicorn.error")
```

```
# =====
```

```
# 🔧 Helpers
```

```
# =====
```

```
def sanitize_account_payload(data: AccountCreate, created_by: str) -> dict:
```

```
    """Prepare Firestore payload with consistent metadata."""
```

```
    return {
```

```
        "full_name": data.full_name,
```

```
        "email": data.email,
```

```
        "role": data.role.value,
```

```
        "createdBy": created_by,
```

```
        "createdAt": firestore.SERVER_TIMESTAMP,
```

```
        "updatedAt": firestore.SERVER_TIMESTAMP,
```

```
    }
```

```
def create_firebase_user(data: AccountCreate) -> str:
```

```
    """Create a Firebase Auth user and return UID."""
```

```
    try:
```

```
        user = auth.create_user(email=data.email, password=data.password)
```

```
        logger.info("🔑 Firebase Auth user created: %s", user.uid)
```

```
        return user.uid
```

```
    except FirebaseError as e:
```

```
        if "EMAIL_EXISTS" in str(e).upper():
```

```
            logger.warning("⚠️ Email already in use: %s", data.email)
```

```
            raise HTTPException(
```

```
                status_code=status.HTTP_409_CONFLICT,
```



```

        detail="Email already in use. Please choose a different one.",
    )
    logger.error(" ❌ Firebase Auth creation failed: %s", str(e), exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"Failed to create Firebase user: {str(e)}",
    )

```

```

def set_user_claims(uid: str, role: RoleEnum):

```

```

    """Assign custom claims for Firestore rules enforcement."""

```

```

    try:

```

```

        permissions = ROLE_PERMISSIONS.get(str(role), {})

```

```

        auth.set_custom_user_claims(uid, {

```

```

            "role": role.value,

```

```

            "permissions": permissions

```

```

        })

```

```

        logger.info(" 🛡️ Custom claims set for UID %s → role=%s, permissions=%s", uid, role,
permissions)

```

```

    except Exception as e:

```

```

        logger.error(" ❌ Failed to set custom claims for UID %s: %s", uid, str(e), exc_info=True)

```

```

        raise HTTPException(

```

```

            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,

```

```

            detail=f"Failed to set custom claims: {str(e)}"

```

```

        )

```

```

def delete_firebase_user(uid: str):
    """Delete a Firebase Auth user by UID."""
    try:
        auth.delete_user(uid)

        logger.info("🗑️ Firebase Auth user deleted: %s", uid)
    except UserNotFoundError:
        logger.warning("⚠️ Tried to delete non-existent Firebase Auth user: %s", uid)
    except FirebaseError as e:
        logger.error("❌ Firebase Auth deletion failed: %s", str(e), exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Failed to delete Firebase Auth user: {str(e)}",
        )

```

```

def write_firestore_profile(uid: str, payload: dict):
    """Write user profile to Firestore."""
    try:
        get_db().collection("users").document(uid).set(payload, merge=True)

        logger.info("✅ Firestore profile created for UID: %s", uid)
    except Exception as e:
        logger.error("❌ Firestore write failed: %s", str(e), exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,

```

```
        detail=f"Failed to write user profile: {str(e)}",
    )
```

```
def delete_firestore_profile(uid: str, deleted_by: str):
    """Delete user profile from Firestore and log audit trail."""
    try:
        get_db().collection("users").document(uid).delete()

        logger.info("🗑️ Firestore profile deleted for UID: %s", uid)

        get_db().collection("role_changes").add({
            "action": "delete",
            "target_user": uid,
            "changed_by": deleted_by,
            "timestamp": firestore.SERVER_TIMESTAMP,
        })
    except Exception as e:
        logger.error("❌ Firestore deletion failed: %s", str(e), exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Failed to delete Firestore profile: {str(e)}",
        )
```

```
def update_user_role(uid: str, new_role: RoleEnum, changed_by: str) -> AccountResponse:
    """Update a user's role in Firestore and Firebase Auth, log the change."""
```

try:

```
user_ref = get_db().collection("users").document(uid)
```

```
snapshot = user_ref.get()
```

if not snapshot.exists:

```
    logger.warning(" ⚠️ Tried to update role for non-existent user: %s", uid)
```

```
    raise HTTPException(
```

```
        status_code=status.HTTP_404_NOT_FOUND,
```

```
        detail=f"User {uid} not found"
```

```
    )
```

```
data = snapshot.to_dict()
```

 Update Firestore role

```
user_ref.update({
```

```
    "role": new_role.value,
```

```
    "updatedAt": firestore.SERVER_TIMESTAMP,
```

```
})
```

```
logger.info(" ✅ Role updated to %s for UID: %s", new_role, uid)
```

 Update Firebase Auth custom claims

```
set_user_claims(uid, new_role)
```

 Log role change

```
get_db().collection("role_changes").add({
```

```
        "action": "update_role",
        "target_user": uid,
        "new_role": new_role.value,
        "changed_by": changed_by,
        "timestamp": firestore.SERVER_TIMESTAMP,
    })
```

```
    return AccountResponse(
        uid=uid,
        email=data.get("email"),
        full_name=data.get("full_name"),
        role=new_role,
        created_by=data.get("createdBy", changed_by),
        created_at=data.get("createdAt", datetime.now(timezone.utc)),
        updated_at=datetime.now(timezone.utc),
    )
```

```
except Exception as e:
```

```
    logger.error(" ❌ Failed to update role for UID %s: %s", uid, str(e), exc_info=True)
```

```
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to update role: {str(e)}"
    )
```

```
# =====
```

🚀 Public Service Functions

=====

async def create_barangay_account(data: AccountCreate, created_by: str) ->
AccountResponse:

"""Create a Firebase Auth user, Firestore profile, and set claims."""

uid = create_firebase_user(data)

payload = sanitize_account_payload(data, created_by)

write_firestore_profile(uid, payload)

set_user_claims(uid, data.role)

📄 Log account creation in audit trail

try:

```
get_db().collection("role_changes").add({  
    "action": "create",  
    "target_user": uid,  
    "new_role": data.role.value,  
    "changed_by": created_by,  
    "timestamp": firestore.SERVER_TIMESTAMP,  
})
```

logger.info(" 📄 Audit trail logged for account creation: %s", uid)

except Exception as e:

logger.error(" ❌ Failed to log account creation audit trail: %s", str(e), exc_info=True)

return AccountResponse(
 uid=uid,
 email=data.email,

```

    full_name=data.full_name,
    role=data.role,
    created_by=created_by,
    created_at=datetime.now(timezone.utc),
    updated_at=datetime.now(timezone.utc),
)

```

```

async def delete_barangay_account(uid: str, deleted_by: str):
    """Delete a Firebase Auth user and Firestore profile."""
    delete_firebase_user(uid)
    delete_firestore_profile(uid, deleted_by)
    return {"detail": f"Account {uid} deleted successfully"}

```

```

async def list_barangay_accounts(
    role: RoleEnum | None = None,
    limit: int = 20,
    offset: int = 0,
    order_by: str = "createdAt"
) -> list[AccountResponse]:
    """List all barangay accounts, optionally filtered by role."""
    try:
        query = get_db().collection("users").order_by(order_by)
        if role:
            query = query.where("role", "==", role.value)
        snapshots = query.limit(limit).offset(offset).stream()

```

```

accounts = []
for snap in snapshots:
    data = snap.to_dict()
    accounts.append(AccountResponse(
        uid=snap.id,
        email=data.get("email"),
        full_name=data.get("full_name"),
        role=RoleEnum(data.get("role")),
        created_by=data.get("createdBy"),
        created_at=data.get("createdAt", datetime.now(timezone.utc)),
        updated_at=data.get("updatedAt", datetime.now(timezone.utc)),
    ))
return accounts
except Exception as e:
    logger.error(" ❌ Failed to list accounts: %s", str(e), exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to list accounts: {str(e)}"
    )

```

```

def find_account_by_email(email: str) -> Optional[dict]:
    clean_email = email.strip().lower()
    docs = get_db().collection("users").where("email", "==", clean_email).stream()

    for doc in docs:

```



```
    return {**doc.to_dict(), "uid": doc.id}
```

```
    return None
```

```
    ...
```

```
## backend\app\services\business_service.py
```

```
```python
```

```
import uuid
```

```
from datetime import datetime, timezone
```

```
import logging
```

```
from backend.app.core.firebase import delete_file
```

```
from backend.app.services.paymongo_service import create_payment_link
```

```
from backend.app.services.fee_service import resolve_business_fee,
determine_business_fee_type
```

```
from backend.app.utils.firestore_utils import get_db
```

```
logger = logging.getLogger("uvicorn.error")
```

```
def create_business_application(data):
```

```
 business = data.business
```

```
 documents = data.documents
```

```
 # Decide fee type (registration vs annual)
```

```
 fee_type = determine_business_fee_type(business.dict())
```

```

fee_breakdown = resolve_business_fee(business.type, fee_type)
amount = fee_breakdown["totalFee"]

doc_ref = get_db().collection("businesses").document()
year = datetime.now().year
business_id = f"BIZ-{business.barangay.upper()}-{year}-{uuid.uuid4().hex[:4]}"

Create PayMongo link
paymongo = create_payment_link(
 amount=amount,
 description=f"{fee_type} for {business.name}",
 remarks=f"business_id:{business_id}"
)

biz_data = business.dict()

Store documents with both URL and path
documents_data = {}
for key, doc in documents.dict().items():
 if isinstance(doc, dict):
 # Already has url/path structure
 documents_data[key] = doc
 else:
 # Fallback if only URL was provided
 documents_data[key] = {"url": doc, "path": None}

```

```

doc_ref.set({
 "ownerUid": data.owner_uid,
 "ownerName": data.owner_name,
 "contactNumber": data.contact_number,
 "email": data.email,
 "businessId": business_id,
 "businessName": biz_data.get("name"),
 "businessType": biz_data.get("type"),
 "barangay": biz_data.get("barangay"),
 "street": biz_data.get("street"),
 "city": biz_data.get("city"),
 "province": biz_data.get("province"),
 "address": f'{biz_data.get('street', "")}, Brgy. {biz_data.get('barangay', "")},
{biz_data.get('city', "")}, {biz_data.get('province', "")}',
 "documents": documents_data,
 "amount": amount,
 "feeType": fee_type,
 "status": "awaiting_payment",
 "paymentStatus": "unpaid",
 "paymongoLinkId": paymongo["paymongoLinkId"],
 "checkoutUrl": paymongo["checkoutUrl"],
 "referenceNumber": paymongo["referenceNumber"]
})

return {
 "business_id": business_id,

```

```
"checkout_url": paymongo["checkoutUrl"],
"fee_breakdown": fee_breakdown
}
```

```
def update_businesses_for_annual_renewal():
```

```
 """Scan businesses and move those past their anniversary into for_payment with annual
 fee."""
```

```
 now = datetime.now(timezone.utc)
```

```
 businesses = get_db().collection("businesses").stream()
```

```
 for biz in businesses:
```

```
 data = biz.to_dict()
```

```
 fee_type = determine_business_fee_type(data)
```

```
 # If the fee type is annual, update status and amount
```

```
 if fee_type == "annualFee":
```

```
 fee_breakdown = resolve_business_fee(data["businessType"], fee_type)
```

```
 amount = fee_breakdown["totalFee"]
```

```
 biz.reference.update({
```

```
 "status": "for_payment",
```

```
 "paymentStatus": "unpaid",
```

```
 "feeType": fee_type,
```

```
 "amount": amount,
```

```
 "updatedAt": now
```

```
)
```

```
def delete_business_and_related(business_id: str):

 """Delete a business and all related payments, receipts, and storage attachments."""

 business_docs = get_db().collection("businesses").where("businessId", "==",
business_id).limit(1).get()

 if not business_docs:

 logger.warning(" ⚠️ No business found for businessId=%s", business_id)

 return {"success": False, "message": "Business not found"}

 business_doc = business_docs[0]

 business_data = business_doc.to_dict()

 # --- Delete attachments from Storage using stored paths ---

 if business_data.get("documents"):

 for key, doc in business_data["documents"].items():

 path = doc.get("path")

 if path:

 try:

 delete_file(path)

 except Exception as e:

 logger.warning(" ⚠️ Failed to delete storage file %s: %s", path, e)

 # --- Delete business doc ---

 business_doc.reference.delete()
```

```

logger.info(" 🗑 Deleted business %s", business_id)

--- Delete related payments ---

payments = get_db().collection("payments").where("businessId", "=", business_id).get()
for pay in payments:
 pay.reference.delete()

 logger.info(" 🗑 Deleted payment %s for business %s", pay.id, business_id)

--- Delete related receipts ---

receipts = get_db().collection("receipts").where("businessId", "=", business_id).get()
for rec in receipts:
 rec.reference.delete()

 logger.info(" 🗑 Deleted receipt %s for business %s", rec.id, business_id)

return {"success": True, "message": f"Business {business_id} and related records +
attachments deleted"}

...

```

```

backend\app\services\complaint_service.py

```

```

```python
import logging

from typing import Optional, List

from google.cloud import firestore

from backend.app.utils.firestore_utils import get_db

```

```
from backend.app.models.complaint import (  
    ComplaintCreate,  
    Complaint,  
    ComplaintWithResident,  
    ComplaintStatus,  
)
```

```
logger = logging.getLogger("unicorn.error")
```

```
COMPLAINT_COLLECTION = "complaints"
```

```
RESIDENT_COLLECTION = "residents"
```

```
#  Timestamp normalization helper
```

```
def _to_datetime(value):
```

```
    try:
```

```
        return value.to_datetime() if hasattr(value, "to_datetime") else value
```

```
    except Exception as e:
```

```
        logger.warning("  Failed to convert timestamp: %s", e)
```

```
    return value
```

```
#  Shared Firestore → Complaint dict normalization
```

```
def _normalize_complaint(doc) -> dict:
```

```
    data = doc.to_dict()
```

```
    data["id"] = doc.id
```

```

if "timestamp" in data:
    data["timestamp"] = _to_datetime(data["timestamp"])

if "updated_at" in data:
    data["updated_at"] = _to_datetime(data["updated_at"])

return data

```

```

# ✅ Enrich complaint with resident info (admin/staff view)

def _enrich_with_resident(data: dict) -> ComplaintWithResident:

```

```

    # Resident info (complaint subject)

    resident_id = data.get("filed_for") or data.get("filed_by")

    filed_for_name = "Unknown"

    if resident_id:

        try:

            doc = get_db().collection(RESIDENT_COLLECTION).document(resident_id).get()

            if doc.exists:

                filed_for_name = doc.to_dict().get("fullName", "Unknown")

        except Exception as e:

            logger.warning("⚠️ Failed to fetch resident %s: %s", resident_id, e)

```

```

    # Filer info (always resolve, even if same as filed_for)

    filed_by_name = "Unknown"

    filer_id = data.get("filed_by")

```



```

if filer_id:

    try:

        doc = get_db().collection(RESIDENT_COLLECTION).document(filer_id).get()

        if doc.exists:

            filed_by_name = doc.to_dict().get("fullName", "Unknown")

    except Exception as e:

        logger.warning(" ⚠️ Failed to fetch filer %s: %s", filer_id, e)

```

```

return ComplaintWithResident(

    **{

        **data,

        "filed_for_name": filed_for_name,

        "filed_by_name": filed_by_name,

        "residentName": filed_for_name,

    }

)

```

📄 File a complaint (resident or staff on behalf of resident)

```
def file_complaint(data: ComplaintCreate) -> Optional[Complaint]:
```

```
    """
```

Create a complaint record.

- filed_by: the ID of the user who entered the complaint (resident or staff/admin)
- filed_for: the resident ID the complaint is about (required if staff/admin files)

```
    """
```

```
    doc_ref = get_db().collection(COMPLAINT_COLLECTION).document()
```

```
# Ensure filed_for is set: if not provided, default to filed_by (resident self-filing)
```

```
filed_for = data.filed_for or data.filed_by
```

```
payload = {
```

```
    **data.model_dump(),
```

```
    "filed_by": data.filed_by,    # who entered the complaint
```

```
    "filed_for": filed_for,    # resident the complaint is about
```

```
    "timestamp": firestore.SERVER_TIMESTAMP,
```

```
    "updated_at": None,
```

```
    "status": ComplaintStatus.open.value,
```

```
}
```

```
try:
```

```
    doc_ref.set(payload)
```

```
    snapshot = doc_ref.get()
```

```
    logger.info(
```

```
        "✅ Complaint filed with ID: %s (filed_by=%s, filed_for=%s)",
```

```
        doc_ref.id,
```

```
        data.filed_by,
```

```
        filed_for,
```

```
    )
```

```
    return Complaint.from_firestore(snapshot)
```

```
except Exception as e:
```

```
    logger.error("❌ Failed to file complaint: %s", e)
```

```
    return None
```

```
# 🔍 Get a specific complaint (enriched with resident + filer info)

def get_complaint_by_id(complaint_id: str) -> Optional[ComplaintWithResident]:
    try:
        doc = get_db().collection(COMPLAINT_COLLECTION).document(complaint_id).get()
        if doc.exists:
            normalized = _normalize_complaint(doc)
            enriched = _enrich_with_resident(normalized) # ✅ add resident + filer names
            return enriched
        logger.warning(" ⚠️ Complaint %s not found", complaint_id)
    except Exception as e:
        logger.error(" ❌ Failed to fetch complaint %s: %s", complaint_id, e)
    return None
```

```
# 👤 Resident: list complaints filed for them (self or by staff)

def list_complaints_by_resident_id(auth_uid: str, limit: Optional[int] = None):
    results: List[Complaint] = []

    try:
        query = get_db().collection(COMPLAINT_COLLECTION).where("filed_for", "==",
auth_uid)
        if limit:
            query = query.limit(limit)

        for doc in query.stream():
            results.append(Complaint(**_normalize_complaint(doc)))
```

```
logger.info("👤 Resident %s retrieved %d complaints", auth_uid, len(results))
```

```
except Exception as e:
```

```
logger.error("❌ Failed to list complaints for resident %s: %s", auth_uid, e)
```

```
return results
```

```
# 📁 Admin/Staff: list all complaints with resident + filer info
```

```
def list_complaints_with_residents(
```

```
    limit: Optional[int] = None,
```

```
    status: Optional[ComplaintStatus] = None,
```

```
) -> List[ComplaintWithResident]:
```

```
    results: List[ComplaintWithResident] = []
```

```
try:
```

```
    query = get_db().collection(COMPLAINT_COLLECTION).order_by(
```

```
        "timestamp", direction=firestore.Query.DESCENDING
```

```
)
```

```
if status:
```

```
    query = query.where("status", "=", status.value)
```

```
if limit:
```

```
    query = query.limit(limit)
```

```
for doc in query.stream():
```

```
    normalized = _normalize_complaint(doc)
```

```
    enriched = _enrich_with_resident(normalized) # ✅ directly enrich
```

```
results.append(enriched)
```

```
logger.info("📄 Admin listed %d complaints", len(results))
```

```
except Exception as e:
```

```
logger.error("❌ Failed to list complaints: %s", e)
```

```
return results
```

```
# 🛠️ Update complaint status (admin)
```

```
def update_complaint_status(
```

```
    complaint_id: str,
```

```
    status: ComplaintStatus,
```

```
    notes: Optional[str] = None,
```

```
) -> Optional[Complaint]:
```

```
    doc_ref = get_db().collection(COMPLAINT_COLLECTION).document(complaint_id)
```

```
    try:
```

```
        snapshot = doc_ref.get()
```

```
        if not snapshot.exists:
```

```
            logger.warning("⚠️ Complaint %s not found for update", complaint_id)
```

```
            return None
```

```
    update_data = {
```

```
        "status": status.value,
```

```
        "updated_at": firestore.SERVER_TIMESTAMP,
```

```
    }
```

```
if notes:
```

```
    update_data["resolution_notes"] = notes
```

```
doc_ref.update(update_data)
```

```
logger.info(
```

```
    "🔧 Complaint %s status updated to %s (notes=%s)",
```

```
    complaint_id,
```

```
    status.value,
```

```
    notes,
```

```
)
```

```
return get_complaint_by_id(complaint_id)
```

```
except Exception as e:
```

```
    logger.error("❌ Failed to update complaint %s: %s", complaint_id, e)
```

```
return None
```

```
# 🗑️ Delete complaint (admin only)
```

```
def delete_complaint(complaint_id: str) -> Optional[Complaint]:
```

```
    doc_ref = get_db().collection(COMPLAINT_COLLECTION).document(complaint_id)
```

```
try:
```

```
    snapshot = doc_ref.get()
```

```
    if not snapshot.exists:
```

```

        logger.warning(" ⚠ Complaint %s not found for deletion", complaint_id)

        return None

    doc_ref.delete()

    logger.info(" 🗑 Complaint %s deleted successfully", complaint_id)

    # Optionally return the deleted complaint data for confirmation
    normalized = _normalize_complaint(snapshot)
    return Complaint(**normalized)

except Exception as e:

    logger.error(" ❌ Failed to delete complaint %s: %s", complaint_id, e)

    return None

...

## backend\app\services\disbursement_service.py

```python
from backend.app.utils.firestore_utils import get_db

def create_disbursement(payload: dict) -> dict:

 ref = get_db().collection("disbursements").document()

 ref.set(payload)

 return {"id": ref.id, **payload}

```

```

def list_disbursements() -> list[dict]:
 docs = get_db().collection("disbursements").stream()
 return [{"id": doc.id, **doc.to_dict()} for doc in docs]

def get_disbursement(id: str) -> dict | None:
 doc = get_db().collection("disbursements").document(id).get()
 return {"id": doc.id, **doc.to_dict()} if doc.exists else None

def update_disbursement(id: str, payload: dict) -> dict | None:
 ref = get_db().collection("disbursements").document(id)
 if not ref.get().exists:
 return None
 ref.update(payload)
 return {"id": id, **payload}

def update_status(id: str, status: str) -> dict | None:
 ref = get_db().collection("disbursements").document(id)
 if not ref.get().exists:
 return None
 ref.update({"status": status})
 return {"id": id, "status": status}

def delete_disbursement(id: str) -> bool:
 ref = get_db().collection("disbursements").document(id)
 if not ref.get().exists:

```



```
 return False

 ref.delete()

 return True
```

```
    ````
```

```
## backend\app\services\document_service.py
```

```
````python
```

```
import logging
```

```
from datetime import datetime, timedelta, timezone
```

```
from fastapi import HTTPException, UploadFile
```

```
from fastapi.concurrency import run_in_threadpool
```

```
from typing import Optional
```

```
from backend.app.models.document import Document, DocumentStatus
```

```
import json
```

```
from backend.app.services.resident_service import get_resident_by_id
```

```
from backend.app.services.fee_service import resolve_document_fee
```

```
from backend.app.core.firebase import upload_file, get_storage_bucket
```

```
from backend.app.utils.firestore_utils import get_db
```

```
from backend.app.utils.barangay_documents import (
```

```
 generate_barangay_clearance_pdf,
```

```
 generate_residency_certificate_pdf,
```

```
 generate_indigency_certificate_pdf,
```

```
 generate_good_moral_certificate_pdf,
```

```
 generate_business_clearance_pdf,
```

```
generate_activity_permit_pdf,
generate_blotter_report_pdf,
generate_health_certificate_pdf,
generate_barangay_id_pdf,
)

logger = logging.getLogger("uvicorn.error")
```

```
📦 Document type dispatch
```

```
DOCUMENT_GENERATORS = {
 "Barangay Clearance": lambda data, issued_by, issued_at, doc_id:
 generate_barangay_clearance_pdf(data, issued_by, issued_at, doc_id),

 "Resident Certificate": lambda data, issued_by, issued_at, doc_id:
 generate_residency_certificate_pdf(data, issued_by, issued_at, doc_id),

 "Indigency Certificate": lambda data, issued_by, issued_at, doc_id:
 generate_indigency_certificate_pdf(data, issued_by, issued_at, doc_id),

 "Good Moral Certificate": lambda data, issued_by, issued_at, doc_id:
 generate_good_moral_certificate_pdf(data, issued_by, issued_at, doc_id),

 "Business Clearance": lambda data, issued_by, issued_at, doc_id:
 generate_business_clearance_pdf(data, issued_by, issued_at, doc_id),
```

```

"Activity Permit": lambda data, issued_by, issued_at, doc_id:
 generate_activity_permit_pdf(data, issued_by, issued_at, doc_id),

"Blotter Report": lambda data, issued_by, issued_at, doc_id:
 generate_blotter_report_pdf(data, issued_by, issued_at, doc_id),

"Health Certificate": lambda data, issued_by, issued_at, doc_id:
 generate_health_certificate_pdf(data, issued_by, issued_at, doc_id),

"Barangay ID": lambda data, issued_by, issued_at, doc_id:
 generate_barangay_id_pdf(data, issued_by, issued_at, doc_id),
}

```

```

def prepare_generator_data(doc: Document) -> dict:
 resident = {}

 if getattr(doc, "residentId", None):
 try:
 resident_out = get_resident_by_id(doc.residentId)
 resident = resident_out.model_dump(by_alias=True)
 logger.info("Fetched resident from Firestore: %s", resident_out.full_name)
 except Exception as e:
 logger.warning("Failed to fetch resident %s: %s", doc.residentId, e)

 # fallback if resident already embedded
 if not resident:
 resident = getattr(doc, "resident", {}) or {}

```

```
logger.info("Using embedded resident: %s", resident)
```

```
address = resident.get("address", {}) or {}
```

```
normalized_address = {}
```

```
key_map = {
```

```
 "houseNumber": "house_number",
```

```
 "street": "street",
```

```
 "purok": "purok",
```

```
 "barangay": "barangay",
```

```
 "city": "city",
```

```
 "province": "province",
```

```
 "zipCode": "zip_code",
```

```
}
```

```
for k, v in address.items():
```

```
 if v:
```

```
 normalized_address[key_map.get(k, k.lower())] = v
```

```
def safe_field(resident_dict, key, default="N/A"):
```

```
 val = resident_dict.get(key)
```

```
 return val if val not in (None, "", "null") else default
```

```
def clean_enum(val, default="N/A"):
```

```
 if val is None or val in ("null", ""):
```

```
 return default
```

```
 val_str = str(val) # force string conversion
```

```
Strip enum prefixes like "CivilStatus.", "Gender.", "VoterStatus."
```

```
if "." in val_str:
```

```
 return val_str.split(".")[-1]
```

```
return val_str
```

```
def format_address(addr: dict) -> str:
```

```
 if not addr:
```

```
 return "N/A"
```

```
 line1 = " ".join([addr.get("houseNumber", ""), addr.get("street", "")]).strip()
```

```
 line2_parts = []
```

```
 if addr.get("barangay"):
```

```
 line2_parts.append(f"Brgy. {addr['barangay']}")
```

```
 if addr.get("city"):
```

```
 line2_parts.append(addr["city"])
```

```
 if addr.get("province"):
```

```
 line2_parts.append(addr["province"])
```

```
 if addr.get("zipCode"):
```

```
 line2_parts.append(addr["zipCode"])
```

```
 line2 = " ".join(line2_parts)
```

```
 return f"{line1}\n{line2}" if line2 else line1
```

```
data = {
```

```
 "resident": {
```

```
 "fullName": safe_field(resident, "fullName", "Unnamed") or resident.get("full_name",
"Unnamed"),
```

```

"address": normalized_address, # ✅ keep dict for generator
"address_str": format_address(resident.get("address", {})),
"birthDate": safe_field(resident, "birthDate"),
"gender": clean_enum(safe_field(resident, "gender")),
"civilStatus": clean_enum(safe_field(resident, "civilStatus")),
"occupation": safe_field(resident, "occupation"),
"contactNumber": safe_field(resident, "contactNumber", ""),
"voterStatus": clean_enum(safe_field(resident, "voterStatus")),
"photoUrl": None,
"signatureUrl": None,
},
"purpose": getattr(doc, "purpose", None),
"remarks": getattr(doc, "remarks", None),
"occupation": getattr(doc, "occupation", None),
"voterStatus": getattr(doc, "voterStatus", None),
"attachments": getattr(doc, "attachments", {}),
}

```

```

✅ Prioritize photoAttachment over photoUrl
photo = doc.attachments.get("photoAttachment")
if photo:
 if isinstance(photo, str):
 data["resident"]["photoUrl"] = photo
 elif isinstance(photo, dict) and "url" in photo:
 data["resident"]["photoUrl"] = photo["url"]
 elif hasattr(photo, "url"):

```

```

 data["resident"]["photoUrl"] = photo.url
 elif resident.get("photoUrl"):
 data["resident"]["photoUrl"] = resident["photoUrl"]
 elif getattr(doc, "extraFields", {}).get("photoUrl"):
 data["resident"]["photoUrl"] = doc.extraFields["photoUrl"]

✅ SignatureUrl directly from resident or extraFields
if resident.get("signatureUrl"):
 data["resident"]["signatureUrl"] = resident["signatureUrl"]
elif getattr(doc, "extraFields", {}).get("signatureUrl"):
 data["resident"]["signatureUrl"] = doc.extraFields["signatureUrl"]

Merge extraFields if present
if getattr(doc, "extraFields", None):
 logger.info("Merging extraFields: %s", doc.extraFields)
 data.update(doc.extraFields)

Normalize keys (map camelCase → snake_case)
key_map = {
 "businessName": "business_name",
 "activityName": "activity_name",
 "activityDate": "activity_date",
 "dateReported": "date_reported",
 "yearsOfStay": "years_of_stay",
 "medicalAttachment": "medical_attachment",
}

```

```
for old_key, new_key in key_map.items():
 if old_key in data:
 data[new_key] = data[old_key]
 logger.info("Normalized key %s → %s: %s", old_key, new_key, data[new_key])
```

```
 Normalize location
```

```
loc = data.get("location", None) or normalized_address
```

```
if isinstance(loc, str):
 try:
 loc = json.loads(loc)
 logger.info("Parsed location string into dict: %s", loc)
 except Exception:
 logger.warning("Failed to parse location string: %s", loc)
 loc = {}
elif not isinstance(loc, dict):
 logger.warning("Location is unexpected type: %s", type(loc))
 loc = {}
```

```
data["location"] = {k.lower(): v for k, v in loc.items() if v}
logger.info("Normalized location: %s", data["location"])
```

```
Health certificate specific
```

```
data["health_purpose"] = getattr(doc, "purpose", None)
```

```
return data
```



```

=====

🛠️ Serialization Helpers

=====

def _serialize(snapshot) -> Document:
 if not snapshot or not snapshot.exists:
 raise HTTPException(status_code=404, detail="Document not found")

 data = snapshot.to_dict() or {}
 data.pop("id", None)

 # Convert status string -> Enum
 if "status" in data and isinstance(data["status"], str):
 try:
 data["status"] = DocumentStatus(data["status"])
 except ValueError:
 pass

 # Normalize extraFields
 extra_fields = data.get("extraFields", {}) or {}

 # Promote common extraFields to top-level
 for key in ["complainant", "respondent", "incident", "location", "dateReported"]:
 if key in extra_fields and key not in data:
 data[key] = extra_fields[key]

```

```

✅ Ensure resident object is preserved

if "resident" in data and isinstance(data["resident"], dict):

 # Promote fullName to residentName

 data["residentName"] = data["resident"].get("fullName", "Unnamed")

Normalize address keys inside resident

address = data["resident"].get("address", {}) or {}

normalized_address = {}

key_map = {

 "houseNumber": "house_number",

 "street": "street",

 "purok": "purok",

 "barangay": "barangay",

 "city": "city",

 "province": "province",

 "zipCode": "zip_code",

}

for k, v in address.items():

 if v:

 normalized_address[key_map.get(k, k.lower())] = v

data["resident"]["address"] = normalized_address

Provide safe defaults for missing fields

for field, default in {

 "fullName": "Unnamed",

 "birthDate": "N/A",

```

```

 "civilStatus": "N/A",
 "gender": "N/A",
 "occupation": "N/A",
 "voterStatus": "N/A",
 "photoUrl": "",
 }.items():
 if not data["resident"].get(field):
 data["resident"][field] = default

```

```

Ensure extraFields is always present

```

```

data["extraFields"] = extra_fields

```

```

return Document(id=snapshot.id, **data)

```

```

def get_and_serialize(doc_id: str) -> Document:

```

```

 snapshot = get_db().collection("documents").document(doc_id).get()

```

```

 if not snapshot.exists:

```

```

 raise HTTPException(status_code=404, detail="Document not found")

```

```

 return _serialize(snapshot)

```

```

async def update_document(doc_id: str, update_data: dict) -> Document:

```

```

 update_data["updatedAt"] = datetime.now(timezone.utc)

```

```

 await run_in_threadpool(

```

```

 get_db().collection("documents").document(doc_id).update,

```

```

 update_data

```

```

)

```

```

return get_and_serialize(doc_id)

=====

📁 List Documents

=====

def list_documents(
 uid: str,
 residentId: Optional[str] = None,
 documentType: Optional[str] = None,
 issuedBy: Optional[str] = None,
 fromDate: Optional[datetime] = None,
 toDate: Optional[datetime] = None,
) -> list[Document]:
 user_doc = get_db().collection("users").document(uid).get()
 role = user_doc.to_dict().get("role") if user_doc.exists else None

 query = get_db().collection("documents")

 # Role-based filtering
 if role in ("admin", "secretary") and residentId:
 query = query.where("residentId", "==", residentId)
 elif role not in ("admin", "secretary"):
 query = query.where("residentId", "==", uid)

 # Apply filters
 if documentType:

```

```

 query = query.where("documentType", "=", documentType)
if issuedBy:
 query = query.where("issuedBy", "=", issuedBy)
if fromDate:
 query = query.where("issuedAt", ">=", fromDate)
if toDate:
 query = query.where("issuedAt", "<=", toDate)

return [_serialize(s) for s in query.stream()]

def list_my_documents(resident_id: str) -> list[Document]:
 return [_serialize(s) for s in get_db().collection("documents")
 .where("residentId", "=", resident_id).stream()]

def list_active_documents(resident_id: str) -> list[Document]:
 active_statuses = {DocumentStatus.pending, DocumentStatus.for_payment,
DocumentStatus.paid}

 return [doc for s in get_db().collection("documents")
 .where("residentId", "=", resident_id).stream()
 if (doc := _serialize(s)).status in active_statuses]

def list_history_documents(resident_id: str) -> list[Document]:
 return [doc for s in get_db().collection("documents")
 .where("residentId", "=", resident_id).stream()
 if (doc := _serialize(s)).status == DocumentStatus.approved
 or (doc.status == DocumentStatus.rejected and doc.resubmitted)]

```

```
def get_document(doc_id: str) -> Document:
```

```
 return get_and_serialize(doc_id)
```

```
def count_issued_documents(document_type: Optional[str] = None) -> int:
```

```
 """
```

```
 Count how many documents have been issued (status = approved).
```

```
 If document_type is provided, filter by that type.
```

```
 """
```

```
 query = get_db().collection("documents").where("status", "=",
DocumentStatus.approved.value)
```

```
 if document_type:
```

```
 query = query.where("documentType", "=", document_type)
```

```
 return len([s for s in query.stream()])
```

```
=====
```

```
 Create Document with Type-Based Counter
```

```
=====
```

```
async def create_document(
```

```
 resident_id: str,
```

```
 resident_name: Optional[str],
```

```
 document_type: str,
```

```
 purpose: Optional[str] = None,
```

```
remarks: Optional[str] = None,
idAttachment: UploadFile = None,
residencyAttachment: UploadFile = None,
complainant: Optional[str] = None,
respondent: Optional[str] = None,
incident: Optional[str] = None,
locationBarangay: Optional[str] = None,
locationStreet: Optional[str] = None,
locationCity: Optional[str] = None,
locationProvince: Optional[str] = None,
businessName: Optional[str] = None,
activityName: Optional[str] = None,
activityDate: Optional[str] = None,
occupation: Optional[str] = None,
voterStatus: Optional[str] = None,
yearsOfStay: Optional[int] = None, # NEW
medicalAttachment: UploadFile = None, # NEW
photoAttachment: UploadFile = None,
activityPlan: UploadFile = None, # NEW
businessPermit: UploadFile = None, # NEW
) -> Document:
```

```
try:
```

```
 doc_ref = get_db().collection("documents").document()
```

```
 now = datetime.now(timezone.utc)
```

```
 # Upload attachments if provided
```

```
id_url, residency_url, medical_url, photo_url, activity_plan_url, business_permit_url =
None, None, None, None, None, None
```

```
if idAttachment:
```

```
 id_url = await run_in_threadpool(
 upload_file, idAttachment, f"documents/{doc_ref.id}/id_{idAttachment.filename}"
)
```

```
if residencyAttachment:
```

```
 residency_url = await run_in_threadpool(
 upload_file, residencyAttachment,
f"documents/{doc_ref.id}/residency_{residencyAttachment.filename}"
)
```

```
if medicalAttachment:
```

```
 medical_url = await run_in_threadpool(
 upload_file, medicalAttachment,
f"documents/{doc_ref.id}/medical_{medicalAttachment.filename}"
)
```

```
if photoAttachment:
```

```
 photo_url = await run_in_threadpool(
 upload_file, photoAttachment,
f"documents/{doc_ref.id}/photo_{photoAttachment.filename}"
)
```

```
if activityPlan:
```

```
 activity_plan_url = await run_in_threadpool(
 upload_file, activityPlan,
f"documents/{doc_ref.id}/activity_plan_{activityPlan.filename}"
)
```

```
if businessPermit:
```



```
business_permit_url = await run_in_threadpool(
 upload_file, businessPermit,
 f"documents/{doc_ref.id}/business_permit_{businessPermit.filename}"
)
```

```
🟡 Fetch resident details
```

```
resident_snapshot = get_db().collection("residents").document(resident_id).get()
```

```
if not resident_snapshot.exists:
```

```
 raise HTTPException(status_code=404, detail="Resident not found")
```

```
resident_data = resident_snapshot.to_dict()
```

```
Counter for sequential IDs
```

```
counter_ref = get_db().collection("counters").document(document_type)
```

```
counter_snapshot = counter_ref.get()
```

```
Check if there are any existing documents of this type
```

```
docs_exist = get_db().collection("documents").where("documentType", "=",
document_type).limit(1).get()
```

```
if not docs_exist:
```

```
 # Reset counter if no documents of this type exist
```

```
 last_number = 0
```

```
else:
```

```
 last_number = counter_snapshot.to_dict().get("last_number", 0) if
counter_snapshot.exists else 0
```

```
new_number = last_number + 1
```

```
await run_in_threadpool(counter_ref.set, {"last_number": new_number})
```

```
safe_type = document_type.replace(" ", "_")
```

```
document_id = f"{safe_type}-{new_number:04d}"
```

```
fee_info = resolve_document_fee(document_type)
```

```
amount = fee_info["totalFee"]
```

```
Build attachments dynamically (only include non-None values)
```

```
attachments = {}
```

```
if id_url:
```

```
 attachments["idAttachment"] = id_url
```

```
if residency_url:
```

```
 attachments["residencyAttachment"] = residency_url
```

```
if medical_url:
```

```
 attachments["medicalAttachment"] = medical_url
```

```
if photo_url:
```

```
 attachments["photoAttachment"] = photo_url
```

```
if activity_plan_url:
```

```
 attachments["activityPlan"] = activity_plan_url
```

```
if business_permit_url:
```

```
 attachments["businessPermit"] = business_permit_url
```

```
def safe_field(data: dict, key: str, default="N/A"):
```

```
 val = data.get(key)
```

```
 return val if val not in (None, "", "null") else default
```

#  Base document data with embedded resident

```
document_data = {
 "documentId": document_id,
 "residentId": resident_id,
 "residentName": resident_name or safe_field(resident_data, "fullName",
"Unnamed"),
 "resident": {
 "fullName": safe_field(resident_data, "fullName", "Unnamed"),
 "address": resident_data.get("address") or {},
 "birthDate": safe_field(resident_data, "birthDate"),
 "gender": safe_field(resident_data, "gender"),
 "civilStatus": safe_field(resident_data, "civilStatus"),
 "contactNumber": safe_field(resident_data, "contactNumber", ""),
 "photoUrl": photo_url or resident_data.get("photoUrl") or "",
 },
 "documentType": document_type,
 "purpose": purpose,
 "remarks": remarks,
 "status": DocumentStatus.paid.value if amount == 0 else
DocumentStatus.pending.value,
 "occupation": occupation,
 "voterStatus": voterStatus,
 "createdAt": now,
 "updatedAt": now,
 "attachments": attachments,
```

```
 "amount": amount,
}
```

```
Type-specific handling
```

```
if document_type == "Blotter Report":
```

```
 if not all([complainant, respondent, incident]):
```

```
 raise HTTPException(
 status_code=422,
 detail="Complainant, respondent, and incident are required for blotter reports"
)
```

```
 Rebuild location dict
```

```
location_dict = {
 "barangay": locationBarangay,
 "street": locationStreet,
 "city": locationCity,
 "province": locationProvince,
}
```

```
location_dict = {k: v for k, v in location_dict.items() if v}
```

```
if not location_dict:
```

```
 raise HTTPException(
 status_code=422,
 detail="Location is required for blotter reports"
)
```

```
document_data["extraFields"] = {
 "complainant": complainant,
 "respondent": respondent,
 "incident": incident,
 "location": location_dict,
 "dateReported": datetime.now().strftime("%Y-%m-%d"),
}
```

```
elif document_type == "Business Clearance":
 if not businessName:
 raise HTTPException(
 status_code=422,
 detail="Business name is required for business clearance"
)
```

```
 Rebuild location dict
```

```
location_dict = {
 "barangay": locationBarangay,
 "street": locationStreet,
 "city": locationCity,
 "province": locationProvince,
}
```

```
location_dict = {k: v for k, v in location_dict.items() if v}
```

```
if not location_dict:
```

```
raise HTTPException(
 status_code=422,
 detail="Business address/location is required for business clearance"
)
```

```
document_data["extraFields"] = {
 "businessName": businessName,
 "location": location_dict,
}
```

```
elif document_type == "Activity Permit":
```

```
if not all([activityName, activityDate]):
```

```
 raise HTTPException(
 status_code=422,
 detail="Activity name and date are required for activity permits"
)
```

```
location_dict = {
 "barangay": locationBarangay,
 "street": locationStreet,
 "city": locationCity,
 "province": locationProvince,
}
```

```
location_dict = {k: v for k, v in location_dict.items() if v}
```

```
if not location_dict:

 raise HTTPException(

 status_code=422,

 detail="Location is required for activity permits"

)

document_data["extraFields"] = {

 "activityName": activityName,

 "activityDate": activityDate,

 "location": location_dict,

}

elif document_type == "Resident Certificate":

 if not yearsOfStay:

 raise HTTPException(status_code=422, detail="Years of residency required")

 document_data["extraFields"] = {"yearsOfStay": yearsOfStay}

elif document_type == "Health Certificate":

 if not medical_url:

 raise HTTPException(status_code=422, detail="Medical result attachment
required")

 document_data["extraFields"] = {"medicalAttachment": medical_url}

elif document_type == "Barangay ID":

 if not photo_url:

 raise HTTPException(status_code=422, detail="Resident photo required")
```

```

Embed photo into resident snapshot
document_data["resident"]["photoUrl"] = photo_url

elif document_type == "Barangay Clearance":

 # ✅ Always embed resident address into extraFields.location
 address = resident_data.get("address", {}) or {}

 # Normalize keys to lowercase
 location_dict = {k.lower(): v for k, v in address.items() if v}

 document_data["extraFields"] = {"location": dict(location_dict)}

await run_in_threadpool(doc_ref.set, document_data)

snapshot = doc_ref.get()

if not snapshot.exists:

 raise HTTPException(status_code=500, detail="Document not saved")

return _serialize(snapshot)

except Exception as e:

 logger.exception("❌ Error creating document: %s", e)

 raise HTTPException(status_code=500, detail="Failed to create document")

=====

🏠 Workflow Functions

=====

async def mark_resubmitted(doc_id: str) -> Document:

 doc = get_and_serialize(doc_id)

 if doc.status != DocumentStatus.rejected:

```



```
 raise HTTPException(status_code=400, detail="Only rejected documents can be resubmitted")
```

```
 return await update_document(doc_id, {"resubmitted": True})
```

```
async def update_status(doc_id: str, new_status: DocumentStatus, remarks: Optional[str])
-> Document:
```

```
 doc = get_and_serialize(doc_id)
```

```
 if doc.amount == 0 and new_status in [DocumentStatus.for_payment,
DocumentStatus.payment_submitted]:
```

```
 raise HTTPException(status_code=400, detail="Free documents do not require payment")
```

```
 valid_transitions = {
```

```
 DocumentStatus.pending: [DocumentStatus.for_payment, DocumentStatus.rejected],
```

```
 DocumentStatus.for_payment: [DocumentStatus.paid, DocumentStatus.rejected],
```

```
 DocumentStatus.payment_submitted: [DocumentStatus.paid,
DocumentStatus.rejected],
```

```
 DocumentStatus.paid: [DocumentStatus.approved],
```

```
 }
```

```
 if new_status not in valid_transitions.get(doc.status, []):
```

```
 raise HTTPException(status_code=400, detail=f"Invalid transition {doc.status.value} → {new_status.value}")
```

```
 if new_status == DocumentStatus.rejected and not remarks:
```

```
 raise HTTPException(status_code=422, detail="Rejection reason required")
```

```
 return await update_document(doc_id, {"status": new_status.value, "remarks": remarks})
```

```
async def confirm_payment(doc_id: str) -> Document:
```

```
 doc = get_and_serialize(doc_id)
```

```
 if doc.status not in (DocumentStatus.for_payment,
DocumentStatus.payment_submitted):

 raise HTTPException(status_code=400, detail="Payment can only be confirmed from
for_payment or payment_submitted")

 return await update_document(doc_id, {"status": DocumentStatus.paid.value,
"paymentStatus": "paid"})
```

```
async def issue_document(doc_id: str, issued_by: str, file_url: Optional[str] = None,
remarks: Optional[str] = None) -> Document:
```

```
 doc = get_and_serialize(doc_id)
```

```
 if doc.amount > 0 and doc.paymentStatus != "paid":
```

```
 raise HTTPException(status_code=400, detail="Payment must be confirmed before
issuance")
```

```
 issued_at = datetime.now(timezone.utc)
```

```
 if not file_url:
```

```
 generator = DOCUMENT_GENERATORS.get(doc.documentType)
```

```
 if not generator:
```

```
 raise HTTPException(status_code=400, detail=f"Unsupported document type:
{doc.documentType}")
```

```
 try:
```

```
 data = prepare_generator_data(doc)
```

```
 pdf_bytes = generator(data, issued_by, issued_at, doc.documentId)
```

```
 bucket = get_storage_bucket()
```

```

 blob = bucket.blob(f"documents/{doc_id}.pdf")

 await run_in_threadpool(blob.upload_from_string, pdf_bytes,
content_type="application/pdf")

 file_url = await run_in_threadpool(

 blob.generate_signed_url, expiration=issued_at + timedelta(days=365*50)

)
 except Exception as e:

 logger.exception(" ❌ PDF generation/upload failed: %s", e)

 raise HTTPException(status_code=500, detail="Failed to generate or upload PDF")

```

```

def _clean_text(value: Optional[str]) -> Optional[str]:

```

```

 if value is None:

```

```

 return None

```

```

 text = str(value).strip()

```

```

 if not text:

```

```

 return None

```

```

 if text.lower() in {"undefined", "null", "none", "nan"}:

```

```

 return None

```

```

 return text

```

```

normalized_remarks = _clean_text(remarks)

```

```

existing_remarks = _clean_text(getattr(doc, "remarks", None))

```

```

final_remarks = normalized_remarks or existing_remarks or f"Issued by {issued_by}"

```

```

update_data = {

```

```

 "status": DocumentStatus.approved.value,

```

```
"issuedBy": issued_by,
"issuedAt": issued_at,
"fileUrl": file_url,
"remarks": final_remarks,
}
```

```
if doc.referenceNumber:
```

```
 update_data["referenceNumber"] = doc.referenceNumber
```

```
return await update_document(doc_id, update_data)
```

```
async def delete_document(doc_id: str, uid: str):
```

```
 """Hard delete a document by ID, along with related payments and receipts."""
```

```
 doc_ref = get_db().collection("documents").document(doc_id)
```

```
 snapshot = doc_ref.get()
```

```
 if not snapshot.exists:
```

```
 raise HTTPException(status_code=404, detail="Document not found")
```

```
 # Serialize before deleting so we can return the deleted record
```

```
 from backend.app.services.document_service import _serialize
```

```
 deleted_doc = _serialize(snapshot)
```

```
 # Perform deletion
```

```
 doc_ref.delete()
```

```
 # --- Delete related payments ---
```

```
payments = get_db().collection("payments").where("documentId", "==", doc_id).get()
for pay in payments:
 pay.reference.delete()
```

```
--- Delete related receipts ---
```

```
receipts = get_db().collection("receipts").where("documentId", "==", doc_id).get()
for rec in receipts:
 rec.reference.delete()
```

```
return deleted_doc
```

```
` ``
```

```
backend\app\services\fee_logic.py
```

```
` `` `python
```

```
from datetime import datetime, timezone
```

```
from dateutil.relativedelta import relativedelta
```

```
def determine_business_fee_type(business_doc: dict) -> str:
```

```
 """
```

```
 Pure business rule: decide registrationFee vs annualFee.
```

```
 No Firestore or external dependencies.
```

```
 """
```

```
 status = business_doc.get("status")
```

```
 reg_date = business_doc.get("registrationDate")
```

```
payments = business_doc.get("payments", [])
```

```
if status == "for_registration" or not payments:
```

```
 return "registrationFee"
```

```
if reg_date:
```

```
 try:
```

```
 if isinstance(reg_date, str):
```

```
 reg_dt = datetime.fromisoformat(reg_date).replace(tzinfo=timezone.utc)
```

```
 elif isinstance(reg_date, datetime):
```

```
 reg_dt = reg_date if reg_dt.tzinfo else reg_date.replace(tzinfo=timezone.utc)
```

```
 else:
```

```
 reg_dt = reg_date.to_datetime().replace(tzinfo=timezone.utc)
```

```
 next_anniversary = reg_dt + relativedelta(years=1)
```

```
 now = datetime.now(timezone.utc)
```

```
 if now >= next_anniversary:
```

```
 return "annualFee"
```

```
except Exception:
```

```
 return "annualFee"
```

```
return "annualFee"
```

```
...
```

```
backend\app\services\fee_service.py
```

```

```python

from backend.app.routes.fee_routes import list_with_misc, list_collection

from datetime import datetime, timezone

from dateutil.relativedelta import relativedelta


# -----

# 🛠️ Utility

# -----

def _normalize_key(value: str) -> str:

    """Normalize type strings for comparison (lowercase, underscores)."""

    return value.strip().lower().replace(" ", "_")


def _find_fee_record(collection: str, id_field: str, type_value: str) -> dict | None:

    """Find a fee record in Firestore collections with misc support."""

    fees = list_with_misc(collection) if collection in ["business_types", "document_types"]
    else list_collection(collection)

    key = _normalize_key(type_value)

    for fee in fees:

        if _normalize_key(fee[id_field]) == key:

            return fee

    return None


# -----

# 💰 Generic Fee Resolver

# -----

```

```
def resolve_fee(collection: str, id_field: str, type_value: str, fee_type: str | None = None) -> dict:
```

```
    """
```

```
    Generic fee resolver for business, document, and misc types.
```

```
    Returns a breakdown dict with base, main, misc, and total.
```

```
    """
```

```
    fee_record = _find_fee_record(collection, id_field, type_value)
```

```
    if not fee_record:
```

```
        raise ValueError(f"Fee not found for {type_value} in {collection}")
```

```
    misc = fee_record.get("miscFeeResolved") or 0
```

```
    if collection == "business_types":
```

```
        if not fee_type:
```

```
            raise ValueError("fee_type required for business_types (registrationFee or annualFee)")
```

```
            base = fee_record.get("fee", 0) # common base fee
```

```
            main = fee_record.get(fee_type, 0) # registrationFee or annualFee
```

```
    elif collection == "document_types":
```

```
        base = fee_record.get("fee", 0) # document base fee
```

```
        main = 0 # avoid double-counting
```

```
    else: # misc_fees
```

```
        base = 0
```

```
        main = fee_record.get("fee", 0)
```

```
    total = base + main + misc
```



```

return {
    "baseFee": base,
    "mainFee": main,
    "miscFee": misc,
    "totalFee": total,
    "feeType": fee_type or "standard",
    "typeValue": type_value
}

```

```

# -----

```

```

# 📄 Specific Resolvers

```

```

# -----

```

```

def resolve_document_fee(document_type: str) -> dict:

```

```

    """Resolve fee for a document type, including misc if enabled."""

```

```

    return resolve_fee("document_types", "documentType", document_type)

```

```

def resolve_business_fee(business_type: str, fee_type: str) -> dict:

```

```

    """

```

```

    Resolve fee for a business type.

```

```

    fee_type must be "registrationFee" or "annualFee".

```

```

    """

```

```

    return resolve_fee("business_types", "businessType", business_type, fee_type=fee_type)

```

```

def resolve_misc_fee(misc_type: str) -> dict:

```

```

    """Resolve fee for a standalone misc type."""

```

```

    return resolve_fee("misc_fees", "miscType", misc_type)

```

```

# -----
# 🏢 Business Fee Type Decision
# -----

def determine_business_fee_type(business_doc: dict) -> str:
    """
    Decide whether to charge registrationFee or annualFee
    based on status and calendar anniversaries.
    """

    status = business_doc.get("status")
    reg_date = business_doc.get("registrationDate")
    payments = business_doc.get("payments", [])

    # Case 1: First-time registration
    if status == "for_registration" or not payments:
        return "registrationFee"

    # Case 2: Annual renewal check (calendar anniversary)
    if reg_date:
        try:
            if isinstance(reg_date, str):
                reg_dt = datetime.fromisoformat(reg_date)

            elif isinstance(reg_date, datetime):
                reg_dt = reg_date

            elif hasattr(reg_date, "to_datetime"):
                reg_dt = reg_date.to_datetime() # Firestore Timestamp

```

```

    else:
        return "annualFee"

    next_anniversary = reg_dt + relativedelta(years=1)
    now = datetime.now(timezone.utc) # timezone-aware UTC datetime

    if now >= next_anniversary:
        return "annualFee"
    else:
        return "registrationFee"
except Exception:
    return "annualFee"

# Case 3: Default for active businesses
return "annualFee"

` ``

## backend\app\services\incident_service.py

` `` python
import logging

from typing import Optional, List

from google.cloud import firestore

from backend.app.utils.firestore_utils import get_db

from backend.app.models.incident import (

```

```
    IncidentCreate,  
    Incident,  
    IncidentWithResident,  
    IncidentStatus,  
)
```

```
logger = logging.getLogger("unicorn.error")
```

```
INCIDENT_COLLECTION = "incidents"
```

```
def _convert_timestamps(data: dict) -> dict:
```

```
    """Convert Firestore Timestamp objects to Python datetime."""
```

```
    for field in ["createdAt", "updatedAt"]:
```

```
        if field in data and hasattr(data[field], "to_datetime"):
```

```
            data[field] = data[field].to_datetime()
```

```
    return data
```

```
def _normalize_incident(doc) -> dict:
```

```
    data = doc.to_dict() or {}
```

```
    data = _convert_timestamps(data)
```

```
    return {
```

```
        "id": doc.id,
```

```
        "type": data.get("type"), # IncidentType enum
```

```
        "description": data.get("description"),
```

```
        "location": data.get("location"),
```

```
"status": data.get("status"),  
  
#  Align with model field names  
  
"timestamp": data.get("createdAt"),  
  
"updated_at": data.get("updatedAt"),  
  
"authUid": data.get("authUid"),  
  
"residentId": data.get("residentId") or data.get("filed_for"),  
  
"assigned_to_name": data.get("assigned_to_name"),  
  
}
```

 Create an incident

```
def create_incident(data: IncidentCreate) -> Incident:
```

```
    doc_ref = get_db().collection(INCIDENT_COLLECTION).document()
```

```
    payload = data.dict()
```

```
    payload.update({
```

```
        "id": doc_ref.id,
```

```
        "status": IncidentStatus.pending.value,
```

```
        "createdAt": firestore.SERVER_TIMESTAMP,
```

```
        "updatedAt": firestore.SERVER_TIMESTAMP,
```

```
    })
```

```
    doc_ref.set(payload)
```

```
    snapshot = doc_ref.get()
```

```
    return Incident(**_normalize_incident(snapshot))
```

🔍 Get a specific incident

```
def get_incident_by_id(incident_id: str) -> Optional[Incident]:  
  
    doc = get_db().collection(INCIDENT_COLLECTION).document(incident_id).get()  
  
    if not doc.exists:  
  
        return None  
  
    return Incident(**_normalize_incident(doc))
```

🛠️ Helper: enrich with resident info

```
def _enrich_with_resident(data: dict) -> IncidentWithResident:  
  
    resident_id = data.get("residentId") or data.get("authUid")  
    data["residentId"] = resident_id  
  
    reported_by_name = "Unknown"  
  
    if resident_id:  
  
        try:  
  
            resident_doc = get_db().collection("residents").document(resident_id).get()  
  
            if resident_doc.exists:  
  
                reported_by_name = resident_doc.to_dict().get("fullName", "Unknown")  
  
        except Exception as e:  
  
            logger.warning(" ⚠️ Failed to enrich resident info: %s", e)  
  
    logged_by_officer = "Unknown"  
  
    auth_uid = data.get("authUid")
```

```

if auth_uid:
    try:
        staff_doc = get_db().collection("users").document(auth_uid).get()
        if staff_doc.exists:
            logged_by_officer = staff_doc.to_dict().get("full_name", "Unknown")
    except Exception as e:
        logger.warning(" ⚠️ Failed to enrich staff info: %s", e)

assigned_to_name = data.get("assigned_to_name") or "—"
if assigned_to_name not in (None, "—"):
    try:
        staff_doc = get_db().collection("users").document(assigned_to_name).get()
        if staff_doc.exists:
            assigned_to_name = staff_doc.to_dict().get("full_name", assigned_to_name)
    except Exception:
        logger.info(" ⓘ Assigned_to is external or free-form: %s", assigned_to_name)

data.update({
    "reported_by_name": reported_by_name,
    "logged_by_officer": logged_by_officer,
    "assigned_to_name": assigned_to_name,
})

return IncidentWithResident(**_convert_timestamps(data))

```

📄 List all incidents with resident info (admin view)

```

def list_incidents_with_residents(
    status: Optional[str] = None, limit: int = 50, start_after_id: Optional[str] = None
) -> List[IncidentWithResident]:

    query = get_db().collection(INCIDENT_COLLECTION)

    if status:
        query = query.where("status", "==", status)

    query = query.order_by("createdAt").limit(limit)

    if start_after_id:
        last_doc = get_db().collection(INCIDENT_COLLECTION).document(start_after_id).get()
        if last_doc.exists:
            query = query.start_after(last_doc)

    incidents: List[IncidentWithResident] = []

    try:
        for doc in query.stream():
            normalized = _normalize_incident(doc)
            incidents.append(_enrich_with_resident(normalized))

        logger.info("📄 Listed %d incidents (status=%s)", len(incidents), status)
    except Exception as e:
        logger.error("🔥 Error listing incidents: %s", e, exc_info=True)
        raise

```



```
return incidents
```

```
# 📋 List incidents assigned to a specific staff (staff view)
```

```
def list_staff_incidents(staff_name: str, limit: int = 50) -> List[IncidentWithResident]:
```

```
    query = (  
        get_db().collection(INCIDENT_COLLECTION)  
        .where("assigned_to_name", "==", staff_name)  
        .order_by("createdAt")  
        .limit(limit)  
    )
```

```
    incidents: List[IncidentWithResident] = []
```

```
    for doc in query.stream():
```

```
        normalized = _normalize_incident(doc)
```

```
        incidents.append(_enrich_with_resident(normalized))
```

```
    logger.info(" 📋 Listed %d incidents for staff %s", len(incidents), staff_name)
```

```
    return incidents
```

```
# 🔧 Update incident status
```

```
def update_incident_status(  
    incident_id: str, status: str, assigned_to: Optional[str] = None
```

```
) -> bool:
```

```
    doc_ref = get_db().collection(INCIDENT_COLLECTION).document(incident_id)
```

```
    try:
```

```
if not doc_ref.get().exists:
```

```
    return False
```

```
update_data = {
```

```
    "status": status,
```

```
    "updatedAt": firestore.SERVER_TIMESTAMP,
```

```
}
```

```
if assigned_to:
```

```
    update_data["assigned_to_name"] = assigned_to
```

```
doc_ref.update(update_data)
```

```
logger.info(
```

```
    "🔧 Incident %s updated: status=%s, assigned_to=%s",
```

```
    incident_id,
```

```
    status,
```

```
    assigned_to,
```

```
)
```

```
return True
```

```
except Exception as e:
```

```
    logger.error("🔥 Failed to update incident %s: %s", incident_id, e, exc_info=True)
```

```
    return False
```

```
# 🗑️ Delete an incident
```

```
def delete_incident(incident_id: str) -> bool:
```

```
    doc_ref = get_db().collection(INCIDENT_COLLECTION).document(incident_id)
```

```
if not doc_ref.get().exists:
```

```
    return False
```

```
doc_ref.delete()
```

```
logger.info("🗑 Incident deleted with ID: %s", incident_id)
```

```
return True
```

```
```\n
```

```
backend\\app\\services\\notification_service.py
```

```
```python
```

```
# backend/app/services/notification_service.py
```

```
import logging
```

```
from typing import Optional
```

```
from datetime import datetime, timezone
```

```
from backend.app.models.notification import Notification
```

```
from backend.app.utils.firestore_utils import get_db
```

```
from backend.app.core.websocket_manager import manager
```

```
logger = logging.getLogger("uvicorn.error")
```

```
class NotificationService:
```

```
    @staticmethod
```

```

def create_notification(
    role: str,
    type: str,
    message: str,
    scope: Optional[str] = None,
    count: Optional[int] = None,
    user: Optional[str] = None,
    user_id: Optional[str] = None,
) -> Notification:
    """Factory to build a Notification object."""
    return Notification(
        role=role,
        type=type,
        scope=scope,
        count=count,
        user=user,
        user_id=user_id,
        message=message,
        timestamp=datetime.now(timezone.utc), # timezone-aware UTC
        read=False,
    )

```

```

@staticmethod

```

```

def save_to_firestore(notification: Notification) -> str:

```

```

    """Persist notification in Firestore."""

```

```

    try:

```

```

doc_ref = get_db().collection("notifications").document(notification.id)

payload = notification.model_dump() # ✅ use model_dump instead of dict

doc_ref.set(payload)

logger.info(
    "✅ Notification saved (role=%s, type=%s, message=%s)",
    notification.role,
    notification.type,
    notification.message,
)

return notification.id

except Exception as e:

    logger.error("❌ Failed to save notification: %s", e)

    raise

```

@staticmethod

```

async def broadcast(notification: Notification):

    """Send notification to all connected WebSocket clients."""

    try:

        await manager.broadcast(
            notification.model_dump(),
            role=notification.role,
            user_id=notification.user_id,
        )

        logger.info("📢 Notification broadcasted: %s", notification.message)

    except Exception as e:

```

```
logger.error(" ❌ Failed to broadcast notification: %s", e)
raise
```

```
@classmethod
```

```
async def notify(
```

```
    cls,
```

```
    role: str,
```

```
    type: str,
```

```
    message: str,
```

```
    scope: Optional[str] = None,
```

```
    count: Optional[int] = None,
```

```
    user: Optional[str] = None,
```

```
    user_id: Optional[str] = None,
```

```
) -> Notification:
```

```
    """
```

```
    High-level helper: create, save, and broadcast a notification.
```

```
    """
```

```
    notif = cls.create_notification(role, type, message, scope, count, user, user_id)
```

```
    cls.save_to_firestore(notif)
```

```
    await cls.broadcast(notif)
```

```
    return notif
```

```
...
```

```
## backend\app\services\password_service.py
```

```
```python
```

```
import uuid
```

```
from typing import Optional
```

```
from datetime import datetime, timedelta, timezone
```

```
from fastapi import HTTPException
```

```
from firebase_admin import auth
```

```
from backend.app.utils.firestore_utils import get_db
```

```
from backend.app.models.password import UserOut
```

```
RESET_EXPIRY_MINUTES = 30
```

```
def create_reset_token(email: str) -> str:
```

```
 """Generate and store a reset token for the given email."""
```

```
 token = str(uuid.uuid4())
```

```
 expires_at = datetime.now(timezone.utc) + timedelta(minutes=RESET_EXPIRY_MINUTES)
```

```
 db = get_db()
```

```
 db.collection("passwordResets").document(token).set({
```

```
 "email": email,
```

```
 "expiresAt": expires_at.isoformat()
```

```
 })
```

```
 return token
```

```
def verify_reset_token(token: str) -> dict:
 """Verify if a reset token exists and is still valid."""
 db = get_db()
 doc = db.collection("passwordResets").document(token).get()
 if not doc.exists:
 raise HTTPException(status_code=404, detail="Invalid token")

 data = doc.to_dict()
 expires_at = datetime.fromisoformat(data["expiresAt"])

 if datetime.now(timezone.utc) > expires_at:
 raise HTTPException(status_code=400, detail="Token expired")

 return data
```

```
def delete_reset_token(token: str) -> None:
 """Delete a reset token after use or expiry."""
 db = get_db()
 db.collection("passwordResets").document(token).delete()
```

```
def apply_password_reset(token: str, new_password: str) -> None:
 """Apply a new password if the token is valid."""
 token_data = verify_reset_token(token)
```



```

Update password in Firebase Auth
user = auth.get_user_by_email(token_data["email"])
auth.update_user(user.uid, password=new_password)

Delete token after successful reset
delete_reset_token(token)

def find_user_by_email(email: str) -> Optional[UserOut]:
 clean_email = email.strip().lower()

 # Residents
 resident_docs = get_db().collection("residents").where("email", "==",
clean_email).stream()

 for doc in resident_docs:
 return UserOut(uid=doc.id, **doc.to_dict())

 # Accounts
 account_docs = get_db().collection("users").where("email", "==", clean_email).stream()

 for doc in account_docs:
 return UserOut(uid=doc.id, **doc.to_dict())

 return None

...

backend\app\services\payment_service.py

```

```

```python
import logging

from google.cloud import firestore

from backend.app.utils.firestore_utils import get_db


logger = logging.getLogger("uvicorn.error")


def _get_business_doc(business_id: str):

    docs = get_db().collection("businesses").where("businessId", "=",
business_id).limit(1).get()

    return docs[0] if docs else None


def _next_receipt_number():

    counter_ref = get_db().collection("counters").document("receipts")

    transaction = get_db().transaction()


@firestore.transactional
def increment_counter(transaction):

    snapshot = counter_ref.get(transaction=transaction)


    # Check if receipts collection is empty

    receipts_exist = get_db().collection("receipts").limit(1).get()

    if not receipts_exist:

        # Reset counter if no receipts exist

```

```

        transaction.set(counter_ref, {"value": 0})

        current = 0

    else:

        current = snapshot.get("value") if snapshot.exists else 0

    new_value = current + 1

    transaction.set(counter_ref, {"value": new_value})

    return new_value


new_number = increment_counter(transaction)
return f"RCPT-{new_number:05d}"


def update_payment_status(business_id: str, event_type: str, status: str,
                          transaction_id: str, payment_intent_id: str = None, paid_at=None):
    doc = _get_business_doc(business_id)
    if not doc:
        logger.warning(" ⚠️ No business found for businessId=%s", business_id)
        return {"success": False, "message": "Business not found"}


# Map PayMongo statuses to internal statuses
status_map = {
    "chargeable": "payment_submitted",
    "consumed": "payment_submitted", # add this
    "unpaid": "for_payment",

```

```
"paid": "payment_submitted",    # staff will verify before final "paid"
"failed": "payment_failed",
"cancelled": "payment_cancelled",
"refunded": "payment_refunded"
}
```

```
new_status = status_map.get(status, status)
```

```
update_data = {
    "paymentStatus": status,
    "status": new_status,
    "transactionId": transaction_id,
    "eventType": event_type,
    "paymentDate": paid_at or firestore.SERVER_TIMESTAMP
}
```

```
if payment_intent_id:
```

```
    update_data["paymentIntentId"] = payment_intent_id
```

```
doc.reference.update(update_data)
```

```
#  Extract businessName from the document
```

```
business_data = doc.to_dict()
```

```
business_name = business_data.get("businessName")
```

```
owner_name = business_data.get("ownerName")
```

```
amount = business_data.get("amount") or business_data.get("amountDue")
```

```
#  Log payment with businessName
```

```
log_payment_record(  
    reference_number=business_data.get("referenceNumber") or business_id,  
    transaction_id=transaction_id,  
    amount=amount,  
    status=status,  
    fee_type="business_fee",  
    business_id=business_id,  
    business_name=business_name,  
    owner_name=owner_name,  
    business_type=business_data.get("businessType"),  
    event_type=event_type,  
    paid_at=paid_at,  
    method="paymongo"  
)
```

```
logger.info("✅ Updated business %s with status=%s event=%s intent=%s",  
            business_id, status, event_type, payment_intent_id)  
return {"success": True, "status": new_status}
```

```
def update_document_payment_status(event_type: str,  
    status: str,  
    transaction_id: str,  
    paymongo_link_id: str = None,  
    payment_intent_id: str = None,  
    source_id: str = None,  
    paid_at=None):
```

"""

Unified document payment status updater.

Handles PayMongo Link, Intent, and Source events.

Ensures 'paid' maps to 'payment_submitted' (secretary must verify).

"""

--- Find matching document ---

docs = []

if paymongo_link_id:

docs = get_db().collection("documents").where("paymongoLinkId", "=",
paymongo_link_id).get()

elif payment_intent_id:

docs = get_db().collection("documents").where("paymongoIntentId", "=",
payment_intent_id).get()

elif source_id:

docs = get_db().collection("documents").where("paymongoSourceId", "=",
source_id).get()

if not docs:

logger.warning(" ⚠️ No document found for link=%s intent=%s source=%s",

paymongo_link_id, payment_intent_id, source_id)

return {"success": False, "message": "Document not found"}

--- Map PayMongo status to internal status ---

status_map = {

"chargeable": "awaiting_payment",

"consumed": "awaiting_payment",

"unpaid": "awaiting_payment",

```
"paid": "payment_submitted", # secretary must verify before final 'paid'
"failed": "payment_failed",
"cancelled": "payment_cancelled",
"refunded": "payment_refunded"
}
```

```
new_status = status_map.get(status, status)
```

```
# --- Build update payload ---
```

```
update_data = {
    "paymentStatus": status,
    "status": new_status,
    "transactionId": transaction_id,
    "eventType": event_type,
    "paymentDate": paid_at or firestore.SERVER_TIMESTAMP
}
```

```
if payment_intent_id:
```

```
    update_data["paymentIntentId"] = payment_intent_id
```

```
if source_id:
```

```
    update_data["paymongoSourceId"] = source_id
```

```
# --- Update all matching docs ---
```

```
for doc in docs:
```

```
    doc.reference.update(update_data)
```

```
    doc_data = doc.to_dict()
```

```
    owner_name = doc_data.get("ownerName")
```

```
    business_name = doc_data.get("businessName")
```

```
amount = doc_data.get("amount") or doc_data.get("amountDue")
```

```
log_payment_record(  
    reference_number=doc_data.get("referenceNumber") or transaction_id,  
    transaction_id=transaction_id,  
    amount=amount,  
    status=status,  
    fee_type="document_fee",  
    document_id=doc_data.get("documentId"),  
    owner_name=owner_name,  
    business_name=business_name,  
    document_type=doc_data.get("documentType"),  
    event_type=event_type,  
    paid_at=paid_at,  
    method="paymongo"  
)
```

```
logger.info("✅ Updated document %s with status=%s event=%s intent=%s  
source=%s",
```

```
    doc.id, status, event_type, payment_intent_id, source_id)
```

```
return {"success": True, "status": new_status}
```

```
def log_payment_record(reference_number, transaction_id, amount, status, fee_type,  
    business_id=None, document_id=None, owner_name=None,  
    business_name=None,
```



```

        business_type=None, document_type=None, receipt_number=None,
        event_type=None, paid_at=None, method=None):

    """Log payment into payments and receipts collections."""

    # Decide entity type

    entity_type = "business" if business_id else "document"

    entity_category = business_type if business_id else document_type


    payment_data = {
        "referenceNumber": reference_number,
        "transactionId": transaction_id,
        "amount": amount,
        "status": status,
        "feeType": fee_type,
        "businessId": business_id,
        "documentId": document_id,
        "ownerName": owner_name,
        "businessName": business_name,
        "entityType": entity_type,
        "entityCategory": entity_category,
        "datePaid": paid_at or firestore.SERVER_TIMESTAMP,
        "eventType": event_type,
        "method": method
    }

    get_db().collection("payments").add(payment_data)


    receipt_data = {

```

```

    "receiptNumber": receipt_number or _next_receipt_number(),
    **payment_data,
    "issuedBy": "system-webhook"
}
get_db().collection("receipts").add(receipt_data)

```

```

    logger.info("✅ Logged payment and receipt for reference=%s type=%s",
reference_number, entity_category)

    return receipt_data["receiptNumber"]

```

```

` ``

```

```

## backend\app\services\paymongo_service.py

```

```

` `` python

```

```

import os

```

```

import base64

```

```

import requests

```

```

import logging

```

```

logger = logging.getLogger("uvicorn.error")

```

```

BASE_URL = "https://api.paymongo.com/v1"

```

```

def _get_secret_key() -> str:

```

```

    """Retrieve PayMongo secret key from environment at runtime."""

```

```

key = os.getenv("PAYMONGO_SECRET_KEY")

if not key:

    logger.error(" ❌ PAYMONGO_SECRET_KEY not set in environment")

    raise RuntimeError("PAYMONGO_SECRET_KEY missing")

return key

```

```

def _make_headers(secret_key: str) -> dict:

```

```

    """Build headers for PayMongo API requests."""

    encoded_key = base64.b64encode(f"{secret_key}:".encode()).decode()

    return {

        "Authorization": f"Basic {encoded_key}",

        "Content-Type": "application/json"

    }

```

```

def create_payment_link(amount: int, description: str, remarks: str = "", metadata: dict =
None,

```

```

        success_url: str = "https://your-app.com/payment-success",

        cancel_url: str = "https://your-app.com/payment-cancel") -> dict:

    """Create a PayMongo payment link with redirect URLs."""

    headers = _make_headers(_get_secret_key())

    payload = {

        "data": {

            "attributes": {

                "amount": amount * 100,

                "description": description,

                "remarks": remarks,

```

```

        "metadata": metadata or {},
        "success_url": success_url,
        "cancel_url": cancel_url
    }
}
}

```

```

# logger.info(" 🚀 Sending PayMongo link payload: %s", payload)

```

```

try:

```

```

    response = requests.post(f"{BASE_URL}/links", json=payload, headers=headers)
    response.raise_for_status()
    data = response.json().get("data", {})
    attrs = data.get("attributes", {})
    return {
        "checkoutUrl": attrs.get("checkout_url"),
        "referenceNumber": attrs.get("reference_number"),
        "paymentStatus": attrs.get("status") or "awaiting_payment",
        "paymongoLinkId": data.get("id")
    }

```

```

except requests.exceptions.HTTPError as e:

```

```

    logger.error(" ❌ PayMongo API error: %s", e)
    logger.error(" 📦 Response status=%s body=%s", response.status_code, response.text)
    raise RuntimeError("Failed to create payment link")

```

```

except Exception as e:

```

```

    logger.exception(" ❌ Unexpected error creating payment link")

```

```
raise RuntimeError("Failed to create payment link")
```

```
def get_payment_link(link_id: str) -> dict:
```

```
    """Retrieve an existing PayMongo payment link by ID."""
```

```
    headers = _make_headers(_get_secret_key())
```

```
    try:
```

```
        response = requests.get(f"{BASE_URL}/links/{link_id}", headers=headers)
```

```
        response.raise_for_status()
```

```
        payload = response.json()
```

```
        data = payload.get("data", {})
```

```
        attrs = data.get("attributes", {})
```

```
        return {
```

```
            "checkoutUrl": attrs.get("checkout_url"),
```

```
            "referenceNumber": attrs.get("reference_number"),
```

```
            "paymentStatus": attrs.get("status"),
```

```
            "paymongoLinkId": data.get("id"),
```

```
            "raw": payload
```

```
        }
```

```
    except requests.exceptions.HTTPError as e:
```

```
        logger.error(" ❌ PayMongo API error when fetching link %s: %s", link_id, e)
```

```
        logger.error(" 📦 Response status=%s body=%s", response.status_code, response.text)
```

```
        raise RuntimeError("Failed to fetch payment link")
```

```
    except Exception as e:
```

```
        logger.exception(" ❌ Unexpected error fetching payment link %s", link_id)
```

```
        raise RuntimeError("Failed to fetch payment link")
```

```

def create_payment_intent(amount: int, description: str, remarks: str = "",
                           metadata: dict = None) -> dict:
    headers = _make_headers(_get_secret_key())
    payload = {
        "data": {
            "attributes": {
                "amount": amount * 100,
                "currency": "PHP",
                "payment_method_allowed": ["gcash", "grab_pay"],
                "description": description,
                "statement_descriptor": remarks or description,
                "metadata": metadata or {},
                "capture_type": "automatic"
            }
        }
    }

    # logger.info("📩 Sending PayMongo intent payload: %s", payload)

    try:
        resp = requests.post(f"{BASE_URL}/payment_intents", json=payload, headers=headers)
        resp.raise_for_status()
        response_json = resp.json()

        # logger.info("📩 PayMongo response: %s", response_json)

```

```
if "errors" in response_json:
```

```
    logger.error(" ❌ PayMongo intent creation failed: %s", response_json["errors"])
```

```
    raise RuntimeError("Failed to create payment intent")
```

```
data = response_json.get("data", {})
```

```
attrs = data.get("attributes", {})
```

```
next_action = attrs.get("next_action") or {}
```

```
redirect = next_action.get("redirect") or {}
```

```
checkout_url = redirect.get("url") or ""
```

```
return {
```

```
    "checkoutUrl": checkout_url,
```

```
    "referenceNumber": attrs.get("reference_number"),
```

```
    "paymentStatus": attrs.get("status"),
```

```
    "paymentIntentId": data.get("id"),
```

```
    "paymongoClientKey": attrs.get("client_key"),
```

```
    "amount": (attrs.get("amount") or 0) / 100,
```

```
    "description": attrs.get("description"),
```

```
    "metadata": attrs.get("metadata", {})
```

```
}
```

```
except requests.exceptions.HTTPError as e:
```

```
    logger.error(" ❌ PayMongo API error. Payload=%s Response=%s", payload, resp.text)
```

```
    raise RuntimeError("Failed to create payment intent")
```

```
except Exception as e:
```

```
    logger.exception(" ❌ Unexpected error creating payment intent")
```

```
    raise RuntimeError("Failed to create payment intent")
```

```
def attach_payment_method(
```

```
    payment_intent_id: str,
```

```
    client_key: str,
```

```
    method: str,
```

```
    billing: dict,
```

```
    return_url: str
```

```
) -> dict:
```

```
    """
```

```
    Create a payment method and attach it to a Payment Intent.
```

```
    Must include a valid return_url so PayMongo generates a redirect URL.
```

```
    """
```

```
    headers = _make_headers(_get_secret_key())
```

```
# Step 1: Create payment method
```

```
pm_payload = {
```

```
    "data": {
```

```
        "attributes": {
```

```
            "type": method, # "gcash" or "grab_pay"
```

```
            "billing": billing
```

```
        }
```

```
    }
```

```
}
```

```
pm_res = requests.post(f"{BASE_URL}/payment_methods", json=pm_payload,  
headers=headers)
```

```
pm_res.raise_for_status()
```



```
pm_data = pm_res.json()

logger.info("📄 Payment method response: %s", pm_data)

if "errors" in pm_data:
    raise RuntimeError(f"Payment method creation failed: {pm_data['errors']}")

payment_method_id = pm_data["data"]["id"]

# Step 2: Attach to intent
attach_payload = {
    "data": {
        "attributes": {
            "payment_method": payment_method_id,
            "client_key": client_key,
            "return_url": return_url # must be a real frontend route
        }
    }
}

intent_res = requests.post(
    f"{BASE_URL}/payment_intents/{payment_intent_id}/attach",
    json=attach_payload,
    headers=headers
)

intent_res.raise_for_status()

intent_data = intent_res.json()

logger.info("📄 Attach response: %s", intent_data)
```

```
if "errors" in intent_data:
```

```
    raise RuntimeError(f"Payment intent attach failed: {intent_data['errors']}")
```

```
attrs = intent_data["data"]["attributes"]
```

```
redirect_url = attrs.get("next_action", {}).get("redirect", {}).get("url")
```

```
if not redirect_url:
```

```
    # Helpful log to see why redirect is missing
```

```
    logger.warning(" ⚠️ No redirect URL returned. Status=%s", attrs.get("status"))
```

```
return {
```

```
    "status": attrs.get("status"),
```

```
    "redirectUrl": redirect_url,
```

```
    "referenceNumber": attrs.get("reference_number"),
```

```
    "paymentIntentId": payment_intent_id
```

```
}
```

```
...
```

```
## backend\app\services\resident_service.py
```

```
```python
```

```
import logging, random
```

```
from datetime import date, datetime
```

```
from typing import List, Dict, Optional, Any
```

```
from google.cloud import firestore
from fastapi.encoders import jsonable_encoder
from firebase_admin import auth
from backend.app.utils.firestore_utils import get_db
from backend.app.models import ResidentOut
from backend.app.core.roles import ROLE_PERMISSIONS # ✅ import role permissions
```

```
logger = logging.getLogger("uvicorn.error")
```

```
🔑 Household ID generator
```

```
def generate_household_id(barangay: str = "GEN") -> str:
```

```
 year = datetime.utcnow().year
```

```
 random_seq = str(random.randint(0, 999999)).zfill(6)
```

```
 return f"HH-{year}-{barangay.upper()}-{random_seq}"
```

```
✂ Payload sanitizer for writes
```

```
def sanitize_resident_payload(data: dict, is_update: bool = False) -> dict:
```

```
 if isinstance(data.get("birthDate"), date):
```

```
 data["birthDate"] = data["birthDate"].isoformat()
```

```
 for key in ["fullName", "middleName", "suffix"]:
```

```
 if key in data and isinstance(data[key], str):
```

```
 data[key] = data[key].strip().title() or None
```

```
 for key in ["email", "remarks", "occupation"]:
```

```
 if key in data:
```

```
val = data[key].strip() if isinstance(data[key], str) else data[key]
data[key] = val or None
```

```
if not is_update:
 data["createdAt"] = firestore.SERVER_TIMESTAMP
else:
 data.pop("createdAt", None)
```

```
data["updatedAt"] = firestore.SERVER_TIMESTAMP
```

```
if not is_update and not data.get("householdId"):
 barangay = data.get("address", {}).get("barangay", "GEN")
 data["householdId"] = generate_household_id(barangay)
```

```
return data
```

```
def encode_for_firestore(data: dict) -> dict:
 encoded = {}
 for k, v in data.items():
 if v is firestore.SERVER_TIMESTAMP:
 encoded[k] = v
 else:
 encoded[k] = jsonable_encoder(v)
 return encoded
```

```
class ResidentError(Exception):
```

pass

```
def _get_resident_doc(id: str):
 snapshot = get_db().collection("residents").document(id).get()
 if not snapshot.exists:
 raise ResidentError(f"Resident {id} not found")
 return snapshot
```

```
def to_resident_out(doc: Dict[str, Any], id: Optional[str] = None) -> ResidentOut:
 data = **doc
 if id:
 data["id"] = id
 for key in ["createdAt", "updatedAt"]:
 if key in data and not isinstance(data[key], datetime):
 data.pop(key)
 for key in ["email", "remarks", "occupation"]:
 if key in data and data[key] == "":
 data[key] = None
 return ResidentOut(**data)
```

#  Create (with Firebase Auth integration + claims)

```
def add_resident(data: dict) -> ResidentOut:
 payload = sanitize_resident_payload(data)
 payload = encode_for_firestore(payload)

 if not data.get("email"):
```

```
raise ResidentError("Resident must have an email to create an Auth account")
```

```
clean_email = data["email"].strip().lower()
```

```
try:
```

```
 user = auth.create_user(
```

```
 email=clean_email,
```

```
 password="123456",
```

```
 display_name=data.get("fullName")
```

```
)
```

```
 payload["authUid"] = user.uid
```

```
 payload["id"] = user.uid
```

```
 payload["passwordResetRequired"] = True
```

```
 logger.info("✅ Firebase Auth user created for %s", clean_email)
```

```
 # 🔑 Assign resident claims immediately
```

```
 auth.set_custom_user_claims(user.uid, {"role": "resident"})
```

```
 logger.info("🔑 Claims set for resident UID %s", user.uid)
```

```
except Exception as e:
```

```
 logger.error("❌ Failed to create Firebase Auth user for %s: %s", clean_email, str(e))
```

```
 raise ResidentError("Failed to create Firebase Auth user")
```

```
doc_ref = get_db().collection("residents").document(user.uid)
```

```
doc_ref.set({
```

```
 **payload,
```

```
 "role": "resident",
 "permissions": ROLE_PERMISSIONS["resident"] # safe in Firestore
})
```

```
snapshot = doc_ref.get()

logger.info("✅ Resident added with ID: %s (Barangay: %s)", doc_ref.id,
payload.get("address", {}).get("barangay"))

return to_resident_out(snapshot.to_dict(), id=doc_ref.id)
```

# ➕ Bulk Create (with Firebase Auth integration + claims)

```
def add_residents_bulk(residents: List[dict], householdId: Optional[str] = None) -> Dict[str,
Any]:
```

```
 if not residents:
```

```
 raise ResidentError("No residents provided for bulk add")
```

```
 if not householdId:
```

```
 barangay = residents[0].get("address", {}).get("barangay", "GEN")
```

```
 householdId = generate_household_id(barangay)
```

```
 batch = get_db().batch()
```

```
 created = []
```

```
 for data in residents:
```

```
 if not data.get("email"):
```

```
 logger.warning("⚠️ Skipping resident without email: %s", data.get("fullName"))
```

```
 continue
```

```
data["householdId"] = householdId
payload = sanitize_resident_payload(data)
payload = encode_for_firestore(payload)
```

```
clean_email = data["email"].strip().lower()
```

```
try:
```

```
✅ Create Firebase Auth user
```

```
user = auth.create_user(
 email=clean_email,
 password="123456",
 display_name=data.get("fullName")
)
```

```
payload["authUid"] = user.uid
```

```
payload["id"] = user.uid
```

```
payload["passwordResetRequired"] = True
```

```
logger.info("✅ Firebase Auth user created for %s", clean_email)
```

```
✅ Assign only minimal claim
```

```
auth.set_custom_user_claims(user.uid, {"role": "resident"})
```

```
logger.info("🔑 Role claim set for resident UID %s", user.uid)
```

```
except Exception as e:
```

```
logger.error("❌ Failed to create Firebase Auth user for %s: %s", clean_email, str(e))
```



continue

# ✅ Store full profile + permissions in Firestore

```
doc_ref = get_db().collection("residents").document(user.uid)
```

```
batch.set(doc_ref, {
```

```
 **payload,
```

```
 "role": "resident",
```

```
 "permissions": ROLE_PERMISSIONS["resident"] # safe in Firestore
```

```
})
```

```
created.append(to_resident_out(payload, id=user.uid))
```

if not created:

```
 raise ResidentError("No residents were successfully created")
```

```
batch.commit()
```

```
logger.info("✅ Bulk added %d residents to household %s", len(created), householdId)
```

```
return {"householdId": householdId, "count": len(created), "items": created}
```

# 📖 Read single resident by ID

```
def get_resident_by_id(id: str) -> ResidentOut:
```

```
 snapshot = _get_resident_doc(id)
```

```
 return to_resident_out(snapshot.to_dict(), id=id)
```

# 📖 Read all residents

```
def get_all_residents(limit: int = 50, start_after_id: Optional[str] = None) ->
```

```
List[ResidentOut]:
```

```

query = get_db().collection("residents").order_by("createdAt").limit(limit)

if start_after_id:
 last_doc = get_db().collection("residents").document(start_after_id).get()

 if last_doc.exists:
 query = query.start_after(last_doc)
 else:
 logger.warning("⚠️ start_after_id not found: %s", start_after_id)

docs = query.get()

return [to_resident_out(doc.to_dict(), id=doc.id) for doc in docs]

```

#  Update (PUT)

```

def update_resident(id: str, data: dict) -> ResidentOut:
 doc_ref = get_db().collection("residents").document(id)
 snapshot = _get_resident_doc(id) # returns snapshot

 payload = sanitize_resident_payload(data, is_update=True)
 payload = encode_for_firestore(payload)

 doc_ref.update(payload)
 snapshot = doc_ref.get()

 return to_resident_out(snapshot.to_dict(), id=id)

```

#  Patch (PATCH)

```

def patch_resident(id: str, data: dict) -> ResidentOut:
 doc_ref = get_db().collection("residents").document(id)

```

```
snapshot = _get_resident_doc(id)
```

```
payload = sanitize_resident_payload(
 {k: v for k, v in data.items() if v is not None},
 is_update=True
)
```

```
payload = encode_for_firestore(payload)
```

```
doc_ref.update(payload)
```

```
snapshot = doc_ref.get()
```

```
return to_resident_out(snapshot.to_dict(), id=id)
```

```
🗑 Delete single resident
```

```
def delete_resident(id: str) -> Dict[str, str]:
```

```
 doc_ref = get_db().collection("residents").document(id)
```

```
 snapshot = _get_resident_doc(id)
```

```
 logger.info("Deleting resident: %s", snapshot.to_dict())
```

```
Delete Firestore document
```

```
doc_ref.delete()
```

```
Delete Firebase Auth user
```

```
try:
```

```
 auth.delete_user(id)
```

```
 logger.info("✅ Deleted Firebase Auth user with UID: %s", id)
```

except Exception as e:

logger.warning(" ⚠️ Failed to delete Firebase Auth user %s: %s", id, str(e))

return {"id": id, "message": "Resident deleted successfully"}

# 🔍 Duplicate check

def find\_duplicates(fullName: str, birthDate: str, middleName: Optional[str] = None, suffix: Optional[str] = None) -> List[ResidentOut]:

query = get\_db().collection("residents").where("fullName", "==",  
fullName.strip()).where("birthDate", "==", birthDate)

if middleName:

query = query.where("middleName", "==", middleName.strip())

if suffix:

query = query.where("suffix", "==", suffix.strip())

docs = query.stream()

return [to\_resident\_out(doc.to\_dict(), id=doc.id) for doc in docs]

# 🗑️ Bulk Delete by household

def delete\_by\_household(householdId: str) -> Dict[str, Any]:

docs = get\_db().collection("residents").where("householdId", "==", householdId).stream()

deleted\_count = 0

for doc in docs:

doc.reference.delete()

deleted\_count += 1

```

if deleted_count == 0:

 raise ResidentError(f"No residents found for householdId {householdId}")

 return {"householdId": householdId, "deletedCount": deleted_count, "message":
f"Deleted {deleted_count} resident(s)"}

📦 Bulk Fetch by household
def get_residents_by_household(householdId: str) -> List[ResidentOut]:

 docs = get_db().collection("residents").where("householdId", "==", householdId).stream()

 residents = [to_resident_out(doc.to_dict(), id=doc.id) for doc in docs]

 if not residents:

 raise ResidentError(f"No residents found for householdId {householdId}")

 return residents

def find_by_email(email: str) -> Optional[ResidentOut]:

 clean_email = email.strip().lower()

 docs = get_db().collection("residents").where("email", "==", clean_email).stream()

 for doc in docs:

 return to_resident_out(doc.to_dict(), id=doc.id)

 return None

```

```
` ``
```

```
backend\app\services\settings_service.py
```

```
` `` python
```

```
from typing import Dict
```

```
from firebase_admin import firestore
```

```
from google.cloud.firestore import Client
```

```
 Centralized permission key registry
```

```
ALL_PERMISSION_KEYS = [
```

```
 "viewDashboard",
```

```
 "fileComplaints",
```

```
 "generateCertificates",
```

```
 "manageResidents",
```

```
 "approveClearance",
```

```
 # Extend as needed
```

```
]
```

```
class SettingsService:
```

```
 def __init__(self):
```

```
 db: Client = firestore.client()
```

```
 self.settings_ref = db.collection("settings")
```

```
 self.permissions_doc = self.settings_ref.document("permissions")
```

```
 self.fees_doc = self.settings_ref.document("fees")
```

# 🗝️ Role Permissions

```
def get_permissions(self) -> Dict[str, Dict[str, bool]]:
```

```
 try:
```

```
 doc = self.permissions_doc.get()
```

```
 return doc.to_dict() if doc.exists else {}
```

```
 except Exception as e:
```

```
 print(f"❌ [get_permissions] Firestore error: {e}")
```

```
 return {}
```

```
def update_permissions(self, role: str, permission_map: Dict[str, bool]) -> bool:
```

```
 try:
```

```
 # ✅ Validate keys against known registry
```

```
 unknown_keys = [key for key in permission_map if key not in ALL_PERMISSION_KEYS]
```

```
 if unknown_keys:
```

```
 print(f"⚠️ [update_permissions] Unknown keys for role '{role}': {unknown_keys}")
```

```
 # ✅ Filter and normalize permission map
```

```
 valid_permissions = {
```

```
 key: bool(permission_map.get(key, False))
```

```
 for key in ALL_PERMISSION_KEYS
```

```
 }
```

```
 self.permissions_doc.update({role: valid_permissions})
```

```
 print(f"✅ [update_permissions] Permissions saved for role '{role}':
{valid_permissions}")
```

```
 return True
```

```
except Exception as e:
```

```
 print(f"❌ [update_permissions] Failed for role '{role}': {e}")
```

```
 return False
```

```
💰 Document Fees
```

```
def get_fees(self) -> Dict[str, float]:
```

```
 try:
```

```
 doc = self.fees_doc.get()
```

```
 return doc.to_dict() if doc.exists else {}
```

```
 except Exception as e:
```

```
 print(f"❌ [get_fees] Firestore error: {e}")
```

```
 return {}
```

```
def update_fee(self, document_type: str, fee: float) -> bool:
```

```
 if not isinstance(fee, (int, float)) or fee <= 0:
```

```
 print(f"⚠️ [update_fee] Invalid fee value for '{document_type}': {fee}")
```

```
 return False
```

```
 try:
```

```
 self.fees_doc.update({document_type: fee})
```

```
 print(f"✅ [update_fee] Updated fee for '{document_type}' to ₪{fee}")
```

```
 return True
```

```
 except Exception as e:
```

```
 print(f"❌ [update_fee] Failed for '{document_type}': {e}")
```

```
 return False
```



```
```
```

```
## backend\app\utils\__init__.py
```

```
```python
```

```
utils module
```

```
```
```

```
## backend\app\utils\barangay_documents.py
```

```
```python
```

```
from reportlab.lib.pagesizes import LETTER
```

```
from reportlab.pdfgen import canvas
```

```
from reportlab.lib.units import inch
```

```
from reportlab.graphics.barcode import qr
```

```
from reportlab.graphics.shapes import Drawing
```

```
from urllib.request import urlopen
```

```
from reportlab.lib.utils import ImageReader
```

```
from reportlab.graphics import renderPDF
```

```
from io import BytesIO
```

```
import base64
```

```
from datetime import timedelta
```

```
import textwrap
```

```
from backend.app.utils.date_utils import parse_date
```

```
import json
```

```
import logging
```

```
logger = logging.getLogger("uvicorn.error")
```

```
def safe_text(value):
```

```
 return str(value) if value is not None else ""
```

```
def format_address(address):
```

```
 if isinstance(address, str):
```

```
 try:
```

```
 parsed = json.loads(address)
```

```
 address = {k.lower(): v for k, v in parsed.items()}
```

```
 except Exception:
```

```
 return safe_text(address), "Unknown"
```

```
 elif isinstance(address, dict):
```

```
 #  normalize dict keys too
```

```
 address = {k.lower(): v for k, v in address.items() if v}
```

```
 else:
```

```
 return "Unknown address", "Unknown"
```

```
barangay = safe_text(address.get("barangay", "Unknown")).title()
```

```
street = safe_text(address.get("street", "")).title()
```

```
city = safe_text(address.get("city", "")).title()
```

```
province = safe_text(address.get("province", "")).title()
```

```
house_number = safe_text(address.get("house number", "")).strip()
```

```
purok = safe_text(address.get("purok", "").strip())
```

```
parts = []
```

```
if house_number:
```

```
 parts.append(house_number)
```

```
if street:
```

```
 parts.append(street)
```

```
if purok:
```

```
 parts.append(f"Purok {purok}")
```

```
if barangay and not barangay.lower().startswith("barangay"):
```

```
 parts.append(f"Barangay {barangay}")
```

```
elif barangay:
```

```
 parts.append(barangay)
```

```
if city:
```

```
 parts.append(city)
```

```
if province:
```

```
 parts.append(province)
```

```
full_address = ", ".join(parts)
```

```
return full_address, barangay
```

```
def draw_qr_code(c, doc_id, margin, width):
```

```
 try:
```

```
 qr_code = qr.QrCodeWidget(doc_id)
```

```
 size = 60
```

```
 d = Drawing(size, size)
```

```

 d.add(qr_code)

 renderPDF.draw(d, c, width - margin - size, margin + 10)
except Exception:

 c.setFont("Helvetica-Oblique", 8)

 c.drawString(width - margin - 60, margin + 10, "(QR error)")

def render_document(title, body_text, issued_by, issued_at, doc_id, barangay):

 buffer = BytesIO()

 c = canvas.Canvas(buffer, pagesize=LETTER)

 width, height = LETTER

 margin = inch

 # Border frame

 c.setLineWidth(2)

 c.rect(margin/2, margin/2, width - margin, height - margin)

 # Seals (left: Republic, right: Barangay)

 try:

 c.drawImage("backend/app/assets/seals/ph_seal.png",

 margin, height - margin - 50,

 width=60, height=60, preserveAspectRatio=True, mask='auto')
except Exception:

 c.setFont("Helvetica-Oblique", 8)

 c.drawString(margin, height - margin - 10, "(PH seal missing)")

 try:

```

```

c.drawImage("backend/app/assets/seals/barangay_seal.png",
 width - margin - 60, height - margin - 50,
 width=60, height=60, preserveAspectRatio=True, mask='auto')
except Exception:
 c.setFont("Helvetica-Oblique", 8)
 c.drawString(width - margin - 50, height - margin - 10, "(Barangay seal missing)")

Header text centered between seals
c.setFont("Times-Bold", 16)
c.drawCentredString(width/2, height - margin - 10, "Republic of the Philippines")
c.setFont("Times-Bold", 14)
c.drawCentredString(width/2, height - margin - 35, f"Barangay {barangay}")
c.setFont("Times-Bold", 18)
c.drawCentredString(width/2, height - margin - 75, title)

Body text block
text_obj = c.beginText(margin + 40, height - margin - 140)
text_obj.setFont("Times-Roman", 12)
text_obj.setLeading(18)

Calculate usable width (page width minus left/right margins)
usable_width = width - 2 * margin

Estimate characters per line (roughly 6 points per character at 12pt font)
chars_per_line = int(usable_width / 6)

for paragraph in body_text.split("\n"):

```

```
wrapped_lines = textwrap.wrap(paragraph, width=chars_per_line)

for line in wrapped_lines:
 text_obj.textLine(line)

text_obj.textLine("") # add blank line between paragraphs

c.drawText(text_obj)

Certification
c.setFont("Times-Roman", 12)
c.drawString(margin, margin + 100, "Certified by:")
c.setFont("Times-Bold", 12)
c.drawString(margin, margin + 80, issued_by)

Footer + QR
c.setFont("Helvetica-Oblique", 8)
c.drawString(margin, margin, "This document is system-generated and valid without
signature.")

draw_qr_code(c, doc_id, margin, width)

c.showPage()
c.save()
buffer.seek(0)
return buffer.read()

def generate_barangay_clearance_pdf(data, issued_by, issued_at, doc_id):
 resident = data.get("resident", {})
```

```
full_name = safe_text(resident.get("fullName") or resident.get("full_name") or
"Unnamed")
```

```
✅ Use normalized location from prepare_generator_data
```

```
location = data.get("location") or resident.get("address", {})
```

```
🔧 Ensure location is a dict
```

```
if isinstance(location, str):
```

```
 try:
```

```
 location = json.loads(location)
```

```
 except Exception:
```

```
 logger.warning("Failed to parse location string: %s", location)
```

```
 location = {}
```

```
elif not isinstance(location, dict):
```

```
 location = {}
```

```
full_address, barangay = format_address(location)
```

```
purpose = safe_text(data.get("purpose", "lawful purposes"))
```

```
body = (
```

```
 f"This is to certify that {full_name}, of legal age, currently residing at {full_address}, "
```

```
 f"is of good moral character and has no derogatory record filed in this barangay.\n\n"
```

```
 f"This clearance is issued upon request for {purpose}. "
```

```
 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"
```

```
 f"Document ID: {doc_id}"
```

```
)

return render_document("BARANGAY CLEARANCE", body, issued_by, issued_at, doc_id,
barangay)
```

```
def generate_residency_certificate_pdf(data, issued_by, issued_at, doc_id):

 resident = data.get("resident", {})

 full_name = safe_text(resident.get("fullName", "Unnamed"))

 full_address, barangay = format_address(resident.get("address", {}))

 years = safe_text(data.get("years_of_stay", "")) # ✅ normalized

 body = (

 f"This is to certify that {full_name}, of legal age, is a bonafide resident of Barangay
{barangay}, "

 f"currently residing at {full_address}.\n\n"

 + (f"Resident has lived in the barangay for {years} years.\n\n" if years else "")

 + f"This certificate is issued upon request for the purpose of establishing residency. "

 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"

 f"Document ID: {doc_id}"

)

 return render_document("CERTIFICATE OF RESIDENCY", body, issued_by, issued_at,
doc_id, barangay)
```

```
def generate_indigency_certificate_pdf(data, issued_by, issued_at, doc_id):

 resident = data.get("resident", {})

 full_name = safe_text(resident.get("fullName", "Unnamed"))
```



```

full_address, barangay = format_address(resident.get("address", {}))
remarks = safe_text(data.get("remarks", "financial assistance"))

body = (
 f"This is to certify that {full_name}, of legal age, currently residing at {full_address}, "
 f"is recognized by Barangay {barangay} as a person of indigent status.\n\n"
 f"This certificate is issued upon request for the purpose of {remarks}. "
 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"
 f"Document ID: {doc_id}"
)

return render_document("CERTIFICATE OF INDIGENCY", body, issued_by, issued_at,
doc_id, barangay)

```

```

def generate_good_moral_certificate_pdf(data, issued_by, issued_at, doc_id):
 resident = data.get("resident", {})
 full_name = safe_text(resident.get("fullName", "Unnamed"))
 full_address, barangay = format_address(resident.get("address", {}))
 purpose = safe_text(data.get("purpose", "school/employment"))

 body = (
 f"This is to certify that {full_name}, of legal age, residing at {full_address}, "
 f"is known to possess good moral character and has no record of misconduct in this "
 f"barangay.\n\n"
 f"This certificate is issued upon request for {purpose}. "
 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"
 f"Document ID: {doc_id}"
)

```

```
 return render_document("CERTIFICATE OF GOOD MORAL CHARACTER", body,
issued_by, issued_at, doc_id, barangay)
```

```
def generate_business_clearance_pdf(data, issued_by, issued_at, doc_id):
```

```
 business_name = safe_text(data.get("business_name", "Unnamed Business"))
```

```
 resident = data.get("resident", {})
```

```
 owner = safe_text(resident.get("fullName", "Unnamed"))
```

```
 full_address, barangay = format_address(data.get("location", {}) or resident.get("address",
{}))
```

```
 body = (
```

```
 f"This is to certify that the business named '{business_name}', owned by {owner}, "
```

```
 f"located at {full_address}, is duly recognized and permitted to operate within Barangay
{barangay}.\n\n"
```

```
 f"This clearance is issued for registration, renewal, or legal compliance. "
```

```
 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"
```

```
 f"Document ID: {doc_id}"
```

```
)
```

```
 return render_document("BARANGAY BUSINESS CLEARANCE", body, issued_by,
issued_at, doc_id, barangay)
```

```
def generate_activity_permit_pdf(data, issued_by, issued_at, doc_id):
```

```
 resident = data.get("resident", {})
```

```
 organizer = safe_text(resident.get("fullName", "Unnamed"))
```

```
 activity_name = safe_text(data.get("activity_name", "Unnamed Activity"))
```

```
 location = data.get("location", {})
```

```
 full_address, barangay = format_address(location)
```

```

✅ Ensure date is always valid

raw_date = data.get("activity_date")

date = parse_date(raw_date, issued_at) if raw_date else issued_at

if date is None:

 date = issued_at

body = (

 f"This is to certify that {organizer} is granted permission to conduct the activity titled "

 f"'{activity_name}', to be held at {full_address} on {date.strftime('%B %d, %Y')}. \n\n"

 f"This permit is issued in accordance with barangay regulations and public safety

protocols. "

 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}. \n\n"

 f"Document ID: {doc_id}"

)

return render_document("PERMIT TO CONDUCT ACTIVITY", body, issued_by, issued_at,
doc_id, barangay)

def generate_blotter_report_pdf(data, issued_by, issued_at, doc_id):

 complainant = safe_text(data.get("complainant", "Unnamed"))

 respondent = safe_text(data.get("respondent", "Unnamed"))

 incident = safe_text(data.get("incident", "No details"))

 location = data.get("location", {})

 full_address, barangay = format_address(location)

 date_reported = parse_date(data.get("date_reported"), issued_at)

```

```

body = (
 f"This is to certify that a blotter report was filed by {complainant} against {respondent},
 "

 f"regarding the following incident: {incident}.\n\n"

 f"The incident occurred at {full_address} and was reported on
 {date_reported.strftime('%B %d, %Y')}. "

 f"This report is recorded in the official barangay blotter log. "

 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"

 f"Document ID: {doc_id}"

)

return render_document("BLOTTER REPORT", body, issued_by, issued_at, doc_id,
barangay)

```

```

def generate_health_certificate_pdf(data, issued_by, issued_at, doc_id):
 resident = data.get("resident", {})

 full_name = safe_text(resident.get("fullName", "Unnamed"))

 full_address, barangay = format_address(resident.get("address", {}))

 purpose = safe_text(data.get("health_purpose", "medical purposes")) #  normalized

```

```

body = (
 f"This is to certify that {full_name}, residing at {full_address}, has undergone a medical
 check-up "

 f"and is found to be in good health condition.\n\n"

 f"This certificate is issued for the purpose of {purpose}. "

 f"Issued this {issued_at.strftime('%B %d, %Y')} at Barangay {barangay}.\n\n"

 f"Document ID: {doc_id}"

```

```
)

return render_document("HEALTH CERTIFICATE", body, issued_by, issued_at, doc_id,
barangay)
```

```
def generate_barangay_id_pdf(data, issued_by, issued_at, doc_id):
```

```
 # Work at 4x scale for easier layout
```

```
 scale_factor = 4
```

```
 large_width = scale_factor * 3.375 * inch
```

```
 large_height = scale_factor * 2.125 * inch
```

```
 buffer = BytesIO()
```

```
 c = canvas.Canvas(buffer, pagesize=(large_width, large_height))
```

```
 # --- Resident data ---
```

```
 resident = data.get("resident", {})
```

```
 full_name = safe_text(resident.get("fullName", "Unnamed"))
```

```
 birth_date = safe_text(resident.get("birthDate", "N/A"))
```

```
 civil_status = safe_text(resident.get("civilStatus", "N/A"))
```

```
 gender = safe_text(resident.get("gender", "N/A"))
```

```
 occupation = safe_text(data.get("occupation") or "N/A")
```

```
 voter_status = safe_text(data.get("voterStatus") or "N/A")
```

```
 photo_url = resident.get("photoUrl", "")
```

```
 address_dict = resident.get("address", {}) or {}
```

```
 barangay = safe_text(address_dict.get("barangay", "Unknown"))
```

```
 # --- Watermark ---
```

```
 try:
```

```

c.saveState()

c.setFillAlpha(0.15)

c.drawImage("backend/app/assets/seals/barangay_seal.png",
 large_width/2 - 2.4*inch, large_height/2 - 2.4*inch,
 width=4.8*inch, height=4.8*inch,
 preserveAspectRatio=True, mask='auto')

c.restoreState()

except Exception as e:

 logger.warning("Watermark render failed: %s", e)

--- Header ---

try:

 c.drawImage("backend/app/assets/seals/ph_seal.png",
 0.2*inch, large_height - 1.4*inch,
 width=1.2*inch, height=1.2*inch, preserveAspectRatio=True, mask='auto')

except Exception:

 c.setFont("Helvetica-Oblique", 20)

 c.drawString(0.2*inch, large_height - 1.0*inch, "(PH seal)")

try:

 c.drawImage("backend/app/assets/seals/barangay_seal.png",
 large_width - 1.4*inch, large_height - 1.4*inch,
 width=1.2*inch, height=1.2*inch, preserveAspectRatio=True, mask='auto')

except Exception:

 c.setFont("Helvetica-Oblique", 20)

 c.drawString(large_width - 1.4*inch, large_height - 1.0*inch, "(Barangay seal)")

```

```
c.setFont("Helvetica-Bold", 28)
c.drawCentredString(large_width/2, large_height - 0.8*inch, "Republic of the Philippines")
c.drawCentredString(large_width/2, large_height - 1.3*inch, f"Barangay {barangay}")
c.line(0.35*inch, large_height - 1.50*inch, large_width - 0.35*inch, large_height -
1.50*inch)
```

```
--- Gold banner ---
banner_y = large_height - 2.2*inch
banner_height = 0.6*inch
c.setFillColorRGB(1.0, 0.84, 0.0) # gold
c.rect(0.35*inch, banner_y, large_width - 0.7*inch, banner_height, fill=True, stroke=False)
c.setStrokeColorRGB(0, 0, 0)
c.setLineWidth(1)
c.rect(0.35*inch, banner_y, large_width - 0.7*inch, banner_height, fill=False, stroke=True)
c.setFillColorRGB(0, 0, 0)
c.setFont("Helvetica-Bold", 28)
c.drawCentredString(large_width/2, banner_y + banner_height/2 - 10, "BARANGAY
IDENTIFICATION CARD")
```

```
--- Barangay ID number ---
id_y = large_height - 3.0*inch
c.setFont("Helvetica-Bold", 18)
c.setFillColorRGB(0, 0, 0)
c.drawString(0.6*inch, id_y, f"ID #: {doc_id}")
```

```
--- Photo ---
```

```

if photo_url:
 try:
 if photo_url.startswith("http"):
 img_data = urlopen(photo_url).read()
 img = ImageReader(BytesIO(img_data))
 elif photo_url.startswith("data:image"):
 encoded = photo_url.split(",", 1)[1]
 img_data = base64.b64decode(encoded)
 img = ImageReader(BytesIO(img_data))
 else:
 img = ImageReader(photo_url)

 c.drawImage(img, 0.6*inch, large_height - 6.0*inch,
 width=2.4*inch, height=2.4*inch,
 preserveAspectRatio=True, mask='auto')
 except Exception as e:
 logger.warning("Photo render failed: %s", e)
 c.setFont("Helvetica-Oblique", 20)
 c.drawString(0.6*inch, large_height - 3.6*inch, "(Photo unavailable)")

--- Signature area below photo ---
signature_x = 0.6*inch
signature_y = large_height - 6.8*inch # adjust relative to photo bottom
signature_width = 2.4*inch
signature_height = 0.8*inch

```



```

signature_url = resident.get("signatureUrl", None)
if signature_url:
 try:
 if signature_url.startswith("http"):
 sig_data = urlopen(signature_url).read()
 sig_img = ImageReader(BytesIO(sig_data))
 elif signature_url.startswith("data:image"):
 encoded = signature_url.split(",", 1)[1]
 sig_data = base64.b64decode(encoded)
 sig_img = ImageReader(BytesIO(sig_data))
 else:
 sig_img = ImageReader(signature_url)

 # Draw signature image
 c.drawImage(sig_img, signature_x, signature_y,
 width=signature_width, height=signature_height,
 preserveAspectRatio=True, mask='auto')
 except Exception as e:
 logger.warning("Signature render failed: %s", e)

--- Always draw underline below signature area ---
underline_y = signature_y - 0.1*inch
c.setStrokeColorRGB(0, 0, 0)
c.setLineWidth(1)
c.line(signature_x, underline_y, signature_x + signature_width, underline_y)

```

```

--- Label below underline ---

c.setFont("Helvetica", 14)

c.setFill(0, 0, 0)

c.drawCentredString(signature_x + signature_width/2, underline_y - 0.3*inch,
"Signature")

--- Details ---

details_x = 4.0*inch

fields = [

 ("Name:", full_name),

 ("Birth Date:", birth_date),

 ("Civil Status:", civil_status),

 ("Gender:", gender),

 ("Occupation:", occupation),

 ("Voter Status:", "Registered Voter" if voter_status.lower() == "yes" else voter_status),

]

--- Address split into 3 lines ---

line1 = " ".join([address_dict.get("house_number", ""), address_dict.get("street",
"")]).strip()

line2_parts = []

if address_dict.get("barangay"):

 line2_parts.append(f"Brgy. {address_dict['barangay']}")

if address_dict.get("city"):

 line2_parts.append(address_dict["city"])

line2 = " ".join(line2_parts)

```

```

line3_parts = []

if address_dict.get("province"):
 line3_parts.append(address_dict["province"])

if address_dict.get("zip_code"):
 line3_parts.append(address_dict["zip_code"])

line3 = ", ".join(line3_parts)

address_lines = [line1 or "N/A", line2 or "", line3 or ""]

Add each line of address as its own field
fields.append(("Address:", address_lines[0]))

for extra_line in address_lines[1:]:
 fields.append(("", extra_line)) # continuation lines

--- Layout for details ---
start_y = banner_y - 1.0*inch
bottom_margin = 0.5*inch
available_height = start_y - bottom_margin
line_height = available_height / len(fields)
current_y = start_y

for label, value in fields:
 if label: # normal label/value pair
 c.setFont("Helvetica", 26)
 c.setFillcolorRGB(0, 0, 0) # black for labels
 c.drawString(details_x, current_y, label)

```

```

 label_width = c.stringWidth(label, "Helvetica", 26)
 c.setFont("Helvetica-Bold", 26)
 c.setFillcolorRGB(0, 0, 1) # blue for values
 c.drawString(details_x + label_width + 12, current_y, value)
else: # continuation line (no label, just value)
 c.setFont("Helvetica-Bold", 26)
 c.setFillcolorRGB(0, 0, 1) # blue for values
 c.drawString(details_x, current_y, value)
current_y -= line_height

--- QR code (right side, scaled up) ---
try:
 qr_code = qr.QrCodeWidget(doc_id)
 target_size = 3.0*inch
 bounds = qr_code.getBounds()
 width = bounds[2] - bounds[0]
 height = bounds[3] - bounds[1]
 scale_x = target_size / width
 scale_y = target_size / height
 d = Drawing(target_size, target_size, transform=[scale_x, 0, 0, scale_y, 0, 0])
 d.add(qr_code)
 qr_x = large_width - target_size - 0.6*inch
 qr_y = large_height - 6.0*inch
 renderPDF.draw(d, c, qr_x, qr_y)
except Exception as e:
 logger.warning("QR render failed: %s", e)

```

```
c.setFont("Helvetica-Oblique", 20)
c.setFillColorRGB(0, 0, 0)
c.drawString(large_width - 2.0*inch, large_height - 3.6*inch, "(QR error)")
```

```
--- Validity under QR code ---
```

```
valid_until = issued_at + timedelta(days=365)
```

```
c.setFont("Helvetica-Bold", 20)
```

```
c.setFillColorRGB(0, 0, 0)
```

```
c.drawRightString(large_width - 0.6*inch, large_height - 7.4*inch,
```

```
 f"Valid until: {valid_until.month} / {valid_until.day} / {valid_until.strftime('%y')}")
```

```
--- Scale down to ID size ---
```

```
scale_x = (3.375 * inch) / large_width
```

```
scale_y = (2.125 * inch) / large_height
```

```
c.scale(scale_x, scale_y)
```

```
c.showPage()
```

```
c.save()
```

```
buffer.seek(0)
```

```
return buffer.read()
```

```
...
```

```
backend\app\utils\date_utils.py
```

```
```python
```

```
# backend/app/utils/date_utils.py
```

```
from datetime import datetime
```

```
from typing import Optional
```

```
def parse_date(date_str: Optional[str], fallback: Optional[datetime] = None, fmt: str = "%Y-%m-%d") -> datetime:
```

```
    """
```

Parse a string into a datetime object. Falls back to provided datetime or today if parsing fails.

Args:

date_str (str): The date string to parse.

fallback (datetime): A fallback datetime if parsing fails. Defaults to today.

fmt (str): The expected format of the date string. Defaults to "%Y-%m-%d".

Returns:

datetime: A valid datetime object (parsed or fallback).

```
    """
```

```
    try:
```

```
        if date_str:
```

```
            return datetime.strptime(date_str, fmt)
```

```
    except (ValueError, TypeError):
```

```
        pass
```

```
    return fallback or datetime.today()
```

```
def format_date(date_obj: datetime, fmt: str = "%Y-%m-%d") -> str:
```

```
return date_obj.strftime(fmt)
```

```
def today(fmt: str = "%Y-%m-%d") -> str:
```

```
    return datetime.today().strftime(fmt)
```

```
    ````
```

```
backend\app\utils\fee_calculator.py
```

```
````python
```

```
def compute_business_fee(business_type: str) -> int:
```

```
    base_fee = 50000 # ₱500 in centavos
```

```
    # You can expand this later
```

```
    return base_fee
```

```
    ````
```

```
backend\app\utils\firestore_utils.py
```

```
````python
```

```
# backend/app/utils/firestore_utils.py
```

```
import logging
```

```
from fastapi import HTTPException
```

```
from backend.app.core.firebase import get_firestore
```

```
from datetime import datetime
```

```
logger = logging.getLogger("unicorn.error")
```

```
def get_db():
```

```
    """Return a Firestore client lazily."""
```

```
    return get_firestore()
```

```
# -----
```

```
# 🛠 Unified Fee Validator
```

```
# -----
```

```
def validate_fee(data: dict):
```

```
    """
```

```
    Ensure fee is non-negative.
```

```
    Accepts 0 as valid, rejects None or negative values.
```

```
    """
```

```
    fee = data.get("fee")
```

```
    if fee is None or fee < 0:
```

```
        raise HTTPException(status_code=400, detail="Fee must be a non-negative number")
```

```
# -----
```

```
# 📄 Create Document
```

```
# -----
```

```
def create_document(collection: str, doc_id: str, data: dict):
```

```
    logger.info("Firestore create_document data=%s", data)
```



```

try:

    validate_fee(data) #  unified check

    ref = get_db().collection(collection).document(doc_id)

    data["createdAt"] = datetime.utcnow().isoformat()

    ref.set(data)

    return {"id": doc_id, "data": data}

except Exception as e:

    raise HTTPException(status_code=500, detail=f"Failed to create {collection}/{doc_id}: {str(e)}")

# -----

#  Update Document

# -----

def update_document(collection: str, doc_id: str, data: dict):

    try:

        validate_fee(data) #  unified check

        ref = get_db().collection(collection).document(doc_id)

        if not ref.get().exists:

            raise HTTPException(status_code=404, detail=f"{collection}/{doc_id} not found")

        data["updatedAt"] = datetime.utcnow().isoformat()

        ref.update(data)

        return {"id": doc_id, "updated": data}

    except Exception as e:

        raise HTTPException(status_code=500, detail=f"Failed to update {collection}/{doc_id}: {str(e)}")

```

```
# -----
```

```
# ❌ Delete Document
```

```
# -----
```

```
def delete_document(collection: str, doc_id: str):
```

```
    try:
```

```
        ref = get_db().collection(collection).document(doc_id)
```

```
        if not ref.get().exists:
```

```
            raise HTTPException(status_code=404, detail=f"{collection}/{doc_id} not found")
```

```
        ref.delete()
```

```
        return {"id": doc_id, "status": "deleted"}
```

```
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=f"Failed to delete {collection}/{doc_id}:  
{str(e)}")
```

```
# -----
```

```
# 🔍 Get Document
```

```
# -----
```

```
def get_document(collection: str, doc_id: str):
```

```
    try:
```

```
        ref = get_db().collection(collection).document(doc_id)
```

```
        doc = ref.get()
```

```
        if not doc.exists:
```

```
            raise HTTPException(status_code=404, detail=f"{collection}/{doc_id} not found")
```

```
        return doc.to_dict() | {"id": doc.id}
```

```
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=f"Failed to fetch {collection}/{doc_id}:  
{str(e)}")
```

```
    ``
```

```
## backend\app\utils\sanitize.py
```

```
``python
```

```
``
```

```
## backend\app\utils\seed_fees.py
```

```
``python
```

```
import logging
```

```
from google.cloud import firestore
```

```
logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger("seed")
```

```
# Initialize Firestore client
```

```
db = firestore.Client()
```

```
#  Seed data
```

```
DOCUMENT_TYPES = [
```

```
    {"id": "barangay_clearance", "documentType": "Barangay Clearance", "fee": 100},
```

```
    {"id": "residency_certificate", "documentType": "Residency Certificate", "fee": 150},
```

```
{ "id": "building_permit", "documentType": "Building Permit", "fee": 500},  
]
```

```
BUSINESS_TYPES = [  
    { "id": "retail_store", "businessType": "Retail Store", "registrationFee": 500, "annualFee":  
1000},  
    { "id": "food_stall", "businessType": "Food Stall", "registrationFee": 300, "annualFee": 600},  
    { "id": "service_provider", "businessType": "Service Provider", "registrationFee": 400,  
"annualFee": 800},  
]
```

```
def seed_document_types():  
    for doc_type in DOCUMENT_TYPES:  
        doc_id = doc_type.pop("id")  
        db.collection("document_types").document(doc_id).set(doc_type)  
        logger.info(f"✅ Seeded document type: {doc_type['documentType']}  
(fee={doc_type['fee']})")
```

```
def seed_business_types():  
    for biz_type in BUSINESS_TYPES:  
        doc_id = biz_type.pop("id")  
        db.collection("business_types").document(doc_id).set(biz_type)  
        logger.info(  
            f"✅ Seeded business type: {biz_type['businessType']} "  
            f"(registrationFee={biz_type['registrationFee']}, annualFee={biz_type['annualFee']})"
```

)

```
def run_seed():
```

```
    logger.info("🚀 Starting Firestore seeding...")
```

```
    seed_document_types()
```

```
    seed_business_types()
```

```
    logger.info("🎉 Seeding complete!")
```

```
if __name__ == "__main__":
```

```
    run_seed()
```

```
```\n
```

```
backend\\assign_claims.py
```

```
```python
```

```
import logging
```

```
import os
```

```
import sys
```

```
import argparse
```

```
from firebase_admin import auth
```

```
from backend.app.core.roles import ROLE_PERMISSIONS
```

```
from backend.app.core.firebase import ensure_firebase_initialized, get_firestore # ✅  
centralized helpers
```

```
# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s: %(message)s"
)
logger = logging.getLogger("uvicorn.error")
```

```
def assign_role_claims(dry_run: bool = False,
                       filter_role: str = None,
                       filter_uid: str = None):
    """Assign custom claims to users based on their role."""
    ensure_firebase_initialized()
    db = get_firestore()

    users_ref = db.collection("users")
    docs = users_ref.stream()

    updated, skipped, failed = 0, 0, 0
    role_counts = {}

    for doc in docs:
        uid = doc.id
        data = doc.to_dict()
        role = (data.get("role") or "").strip().lower()
```

```
logger.debug("📄 Firestore doc for UID %s: %s", uid, data)
```

```
if not role:
```

```
    logger.warning("⚠️ No role found for UID: %s", uid)
```

```
    skipped += 1
```

```
    continue
```

```
if filter_role and role != filter_role.lower():
```

```
    skipped += 1
```

```
    continue
```

```
if filter_uid and uid != filter_uid:
```

```
    skipped += 1
```

```
    continue
```

```
permissions = ROLE_PERMISSIONS.get(role)
```

```
if not permissions:
```

```
    logger.warning("⚠️ Unknown role '%s' for UID: %s", role, uid)
```

```
    skipped += 1
```

```
    continue
```

```
role_counts[role] = role_counts.get(role, 0) + 1
```

```
try:
```

```
    current_claims = auth.get_user(uid).custom_claims or {}
```

```
desired_claims = {"role": role, "permissions": permissions}
```

```
if (current_claims.get("role") == role and
```

```
    current_claims.get("permissions") == permissions):
```

```
    logger.info(" ➡️ Claims already correct for UID: %s", uid)
```

```
    skipped += 1
```

```
    continue
```

```
if dry_run:
```

```
    logger.info(" 🔍 Dry run: would set claims for UID: %s", uid)
```

```
else:
```

```
    auth.set_custom_user_claims(uid, desired_claims)
```

```
    logger.info(" ✅ Claims set for UID: %s (role=%s)", uid, role)
```

```
    updated += 1
```

```
except Exception as e:
```

```
    logger.error(" ❌ Failed to set claims for UID %s: %s", uid, str(e))
```

```
    failed += 1
```

```
logger.info(" 📊 Roles processed: %s", role_counts)
```

```
logger.info(" ⏏️ Sync complete: %s updated, %s skipped, %s failed.",
```

```
    updated, skipped, failed)
```

```
sys.exit(1 if failed > 0 else 0)
```



```

if __name__ == "__main__":

    parser = argparse.ArgumentParser(description="Sync Firebase custom claims with
Firestore roles")

    parser.add_argument("--dry-run", action="store_true", help="Preview changes without
applying")

    parser.add_argument("--filter-role", type=str, help="Only process users with this role")
    parser.add_argument("--filter-uid", type=str, help="Only process this specific UID")
    parser.add_argument("--debug", action="store_true", help="Enable debug logging")

    args = parser.parse_args()

    if args.debug:
        logger.setLevel(logging.DEBUG)

    assign_role_claims(
        dry_run=args.dry_run,
        filter_role=args.filter_role,
        filter_uid=args.filter_uid
    )

    ...

## backend\backendStructure.txt

```text

```

BIS/

└─ backend/

| └─ app/

| | └─ core/

| | | └─ auth.py

| | | └─ roles.py

| | | └─ firebase.py

| | └─ models/

| | | └─ \_\_init\_\_.py

| | | └─ account.py

| | | └─ business.py

| | | └─ complaint.py

| | | └─ document.py

| | | └─ incident.py

| | | └─ payment.py

| | | └─ resident.py

| | | └─ settings.py

| | └─ routes/

| | | └─ \_\_init\_\_.py

| | | └─ account\_routes.py

| | | └─ audit\_routes.py

| | | └─ business\_routes.py

| | | └─ complaint\_routes.py

| | | └─ dashboard.py

| | | └─ document\_routes.py

| | | └─ incident\_routes.py

- | | | └─ payment\_routes.py
- | | | └─ paymongo\_routes.py
- | | | └─ resident\_routes.py
- | | | └─ settings\_routes.py
- | | └─ services/
- | | | └─ \_\_init\_\_.py
- | | | └─ account\_services.py
- | | | └─ business\_services.py
- | | | └─ complaint\_services.py
- | | | └─ document\_services.py
- | | | └─ incident\_services.py
- | | | └─ payment\_services.py
- | | | └─ paymongo\_services.py
- | | | └─ resident\_services.py
- | | | └─ settings\_services.py
- | | └─ utils/
- | | | └─ \_\_init\_\_.py
- | | | └─ barangay\_documents.py
- | | | └─ fee\_calculator.py
- | | | └─ sanitize.py
- | | └─ \_\_init\_\_.py
- | | └─ main.py
- | └─ v/
- | └─ venv/
- | └─ \_\_init\_\_.py
- | └─ assign\_claims.py

```
| |— Dockerfile
| |— requirements.txt
| |— serviceAccountKey.json
```

```
...
```

```
backend\backfill_resident_claims.py
```

```
```python
```

```
import logging
```

```
import sys
```

```
from firebase_admin import auth
```

```
from backend.app.core.roles import ROLE_PERMISSIONS
```

```
from backend.app.core.firebase import ensure_firebase_initialized, get_firestore # 
centralized helpers
```

```
# Configure logging
```

```
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
```

```
logger = logging.getLogger("uvicorn.error")
```

```
def backfill_resident_claims():
```

```
    """Assign resident claims to all existing residents in Firestore."""
```

```
    ensure_firebase_initialized()
```

```
    db = get_firestore()
```

```
residents_ref = db.collection("residents")
```

```
docs = residents_ref.stream()
```

```
updated, failed = 0, 0
```

```
for doc in docs:
```

```
    data = doc.to_dict()
```

```
    uid = doc.id # resident doc ID is the auth UID
```

```
    try:
```

```
        auth.set_custom_user_claims(uid, {
```

```
            "role": "resident",
```

```
            "permissions": ROLE_PERMISSIONS["resident"]
```

```
        })
```

```
        logger.info("✅ Claims set for resident UID %s (%s)", uid, data.get("fullName"))
```

```
        updated += 1
```

```
    except Exception as e:
```

```
        logger.error("❌ Failed to set claims for UID %s: %s", uid, str(e))
```

```
        failed += 1
```

```
logger.info("🏁 Backfill complete: %s updated, %s failed", updated, failed)
```

```
sys.exit(1 if failed > 0 else 0)
```

```
if __name__ == "__main__":
```

```
    backfill_resident_claims()
```

```
## backend\decode_jwt.py
```

```
```python
```

```
import jwt
```

```
Paste your JWT here
```

token =

"eyJhbGciOiJIUzU1NiIsImtpZCI6IjJmMjdhdGZmY1YzIktGU1MzVkNWRjMmMwNWw1YTE2N2FlMmY1NjgxYzliLCJ0eXAiOiJKV1QiIiwiaWF0IjoxNjY1OTU0OTU0LCJpc3MiOiJodHRwczovL3NlY3V5ZXRva2VuLmdvb2dsZS5jb20vYmFyYW5nYXktMTcyMWQlLCJhdWQiOiJiYXJhbmdheS0xNzlxZCIsImF1dGhfdGltZSI6MTc3MjE5Mjk4MywidXNlcl9pZCI6ImtHQzg5ajltU1diMmp0OEZjRnZxajdEUIRaYjliLCJzdWliOiJrR0M4OWo5bVNXYjJqdDhGY0Z2cWo3RFJlUWw1Y2liwWV0ljoXNzcyMTkyOTg1LCJleHAiOiJlY3NzlxOTY1ODUsImVtYWlsIjoiYWRtaW5AYmFyYW5nYXkuZ292LnBoliwiZW1haWxfdmVyaWZpZWQiOmZhbHNlLCJmaXJlYmFzZSI6eyJpZGVudGl0aWVzIjp7ImVtYWlsIjpbImFkbWluQGJhcmFuZ2F5Lmdvdi5waCJdfSwic2lnbl9pbl9wcm92aWRlcil6InBhc3N3b3JkIn19.Z4fadWobU\_hxz1kqv2P-mt30IEZX8Rh4mohy-A-ceAVRIYikao0rwZwFVQtn74g1nb2esu5jx9VObYXOy\_OoPzVvRfOZyen5fKaMrvyd\_vQUgy15pmvAFF3l0RojB5DMjyE-fvB7p\_6TmFvanlvty46urPx-8bxeuh\_j5tkKNfqcQefcKenSUjr9l-2Y45AnHeP9bZfC8RdFIOTWm61DU4dfdrMse\_p2U374ob4CGSMBSsvtWmvny0OgK\_D8EX136ID5AzHPLGPfstdrYhuwnJ-J6s888SuiHouTkC7bo-1W\_Dx4l-Ev2DOZh4vRtOEjygMPqf3WlyuzKQToUWTlw"

```
Decode without verifying signature
```

```
decoded = jwt.decode(token, options={"verify_signature": False})
```

```
print(decoded)
```

...

```
backend\requirements.txt
```

```
```text
```

```
pydantic[email]
```

```
pytest
```

```
python-decouple
```

```
fastapi==0.129.0
```

```
uvicorn==0.41.0
```

```
wsproto==1.2.0
```

```
firebase-admin==7.1.0
```

```
google-cloud-firestore==2.23.0
```

```
google-cloud-storage==3.9.0
```

```
python-dotenv==1.2.1
```

```
email-validator==2.3.0
```

```
python-dateutil==2.9.0.post0
```

```
reportlab==4.4.10
```

```
python-multipart==0.0.22
```

```
requests==2.31.0
```

```
```
```

```
backend\tests__init__.py
```

```
```python
```

```
```
```

```
backend\tests\test_document.py
```

```
```python
```

```
import pytest
```

```
from datetime import datetime
```

```
from backend.app.utils.barangay_documents import (
```

```
    generate_barangay_clearance_pdf,
```

```
    generate_residency_certificate_pdf,
```

```
    generate_indigency_certificate_pdf,
```

```
    generate_good_moral_certificate_pdf,
```

```
    generate_business_clearance_pdf,
```

```
    generate_activity_permit_pdf,
```

```
    generate_blotter_report_pdf,
```

```
    generate_health_certificate_pdf,
```

```
    generate_barangay_id_pdf,
```

```
)
```

```
# Common test data
```

```
resident = {
```

```
    "fullName": "Juan Dela Cruz",
```

```
    "address": {
```

```
        "houseNumber": "1243",
```

```
        "street": "St. Paul St",
```

```
        "purok": "1",
```

```
        "city": "Parañaque",
```

```
        "barangay": "Moonwalk",
```



```
    "province": "Metro Manila",  
},  
    "gender": "Male",  
    "birthDate": "1990-01-01",  
    "contactNumber": "09123456789",  
    "occupation": "Driver",  
    "voterStatus": "Registered",  
}
```

```
issued_by = "Sanya Lopez"  
issued_at = datetime.now()  
doc_id = "TEST-0001"
```

```
@pytest.mark.parametrize("generator,args", [  
    (generate_barangay_clearance_pdf, [resident, issued_by, issued_at, doc_id]),  
    (generate_residency_certificate_pdf, [resident, issued_by, issued_at, doc_id]),  
    (generate_indigency_certificate_pdf, [resident, issued_by, issued_at, doc_id]),  
    (generate_good_moral_certificate_pdf, [resident, issued_by, issued_at, doc_id]),  
    (generate_business_clearance_pdf, ["Sample Business", "Juan Dela Cruz",  
resident["address"], issued_by, issued_at, doc_id]),  
    (generate_activity_permit_pdf, ["Juan Dela Cruz", "Community Cleanup",  
resident["address"], issued_at, issued_by, issued_at, doc_id]),  
    (generate_blotter_report_pdf, ["Complainant", "Respondent", "Noise Complaint",  
resident["address"], issued_at, issued_by, issued_at, doc_id]),  
    (generate_health_certificate_pdf, [resident, "Employment Requirement", issued_by,  
issued_at, doc_id]),  
    (generate_barangay_id_pdf, [resident, issued_by, issued_at, doc_id]),
```

```

])

def test_generators_return_pdf_bytes(generator, args):
    pdf_bytes = generator(*args)
    assert isinstance(pdf_bytes, (bytes, bytearray))
    assert len(pdf_bytes) > 1000 # sanity check: not empty

```

```

` ` `

```

```

## backend\tests\test_fee_logic.py

```

```

` ` `python

```

```

from datetime import datetime, timezone
from dateutil.relativedelta import relativedelta
from backend.app.services.fee_logic import determine_business_fee_type

```

```

def make_business_doc(status="active", reg_date=None, payments=None):
    return {
        "status": status,
        "registrationDate": reg_date,
        "payments": payments or []
    }

```

```

def test_first_time_registration():
    doc = make_business_doc(status="for_registration", payments=[])
    assert determine_business_fee_type(doc) == "registrationFee"

```

```

def test_due_for_annual_fee():

    reg_date = datetime.now(timezone.utc) - relativedelta(years=1, days=1)

    doc = make_business_doc(status="active", reg_date=reg_date,
payments=[{"status":"verified"}])

    assert determine_business_fee_type(doc) == "annualFee"


def test_not_yet_due():

    reg_date = datetime.now(timezone.utc) - relativedelta(months=11)

    doc = make_business_doc(status="active", reg_date=reg_date,
payments=[{"status":"verified"}])

    assert determine_business_fee_type(doc) == "annualFee" # default for active

...

```

```

## backend\tests\test_fee_service.py

```

```

```python
import pytest

from datetime import datetime, timezone

from dateutil.relativedelta import relativedelta

from backend.app.services.fee_service import determine_business_fee_type

def make_business_doc(status="active", reg_date=None, payments=None):

 return {

 "status": status,

 "registrationDate": reg_date,

 "payments": payments or []
 }

```

```
}
```

```
def test_first_time_registration_no_payments():
```

```
 doc = make_business_doc(status="for_registration", payments=[])
```

```
 assert determine_business_fee_type(doc) == "registrationFee"
```

```
def test_first_time_registration_status_only():
```

```
 doc = make_business_doc(status="for_registration")
```

```
 assert determine_business_fee_type(doc) == "registrationFee"
```

```
def test_annual_due_on_anniversary():
```

```
 reg_date = datetime.now(timezone.utc) - relativedelta(years=1, days=1)
```

```
 doc = make_business_doc(status="active", reg_date=reg_date,
payments=[{"status": "verified"}])
```

```
 assert determine_business_fee_type(doc) == "annualFee"
```

```
def test_not_yet_due_before_anniversary():
```

```
 reg_date = datetime.now(timezone.utc) - relativedelta(months=11)
```

```
 doc = make_business_doc(status="active", reg_date=reg_date,
payments=[{"status": "verified"}])
```

```
 assert determine_business_fee_type(doc) == "annualFee" # default for active, but not
triggered by anniversary yet
```

```
def test_leap_year_anniversary():
```

```
 # Registered on Feb 29, 2024 → renewal should be Feb 28, 2025
```

```
 reg_date = datetime(2024, 2, 29, tzinfo=timezone.utc)
```

```

 doc = make_business_doc(status="active", reg_date=reg_date,
payments=[{"status":"verified"}])

 # Simulate current date after Feb 28, 2025

 now = datetime(2025, 3, 1, tzinfo=timezone.utc)

 # Monkeypatch datetime.now if needed, or just check relativedelta logic

 next_anniversary = reg_date + relativedelta(years=1)

 assert next_anniversary == datetime(2025, 2, 28, tzinfo=timezone.utc)

 ...

```

```

backend\tests\test_generators.py

```

```

```python
from datetime import datetime

from backend.app.utils.barangay_documents import (
    generate_barangay_clearance_pdf,
    generate_residency_certificate_pdf,
    generate_indigency_certificate_pdf,
    generate_good_moral_certificate_pdf,
    generate_business_clearance_pdf,
    generate_activity_permit_pdf,
    generate_blotter_report_pdf,
    generate_health_certificate_pdf,
    generate_barangay_id_pdf,
)

```

```
resident = {  
    "fullName": "Juan Dela Cruz",  
    "address": {  
        "houseNumber": "1243",  
        "street": "St. Paul St",  
        "purok": "1",  
        "city": "Parañaque",  
        "barangay": "Moonwalk",  
        "province": "Metro Manila",  
    },  
    "gender": "Male",  
    "birthDate": "1990-01-01",  
    "contactNumber": "09123456789",  
    "occupation": "Driver",  
    "voterStatus": "Registered",  
}
```

```
issued_by = "Sanya Lopez"
```

```
issued_at = datetime.now()
```

```
doc_id = "TEST-0001"
```

```
def quick_check(name, pdf_bytes):  
    print(f"{name}: {len(pdf_bytes)} bytes")  
    assert isinstance(pdf_bytes, (bytes, bytearray))  
    assert len(pdf_bytes) > 1000 # sanity check
```

```
if __name__ == "__main__":
```

```
    quick_check("Barangay Clearance", generate_barangay_clearance_pdf(resident,  
issued_by, issued_at, doc_id))
```

```
    quick_check("Residency Certificate", generate_residency_certificate_pdf(resident,  
issued_by, issued_at, doc_id))
```

```
    quick_check("Indigency Certificate", generate_indigency_certificate_pdf(resident,  
issued_by, issued_at, doc_id))
```

```
    quick_check("Good Moral Certificate", generate_good_moral_certificate_pdf(resident,  
issued_by, issued_at, doc_id))
```

```
    quick_check("Business Clearance", generate_business_clearance_pdf("Sample  
Business", "Juan Dela Cruz", resident["address"], issued_by, issued_at, doc_id))
```

```
    quick_check("Activity Permit", generate_activity_permit_pdf("Juan Dela Cruz",  
"Community Cleanup", resident["address"], issued_at, issued_by, issued_at, doc_id))
```

```
    quick_check("Blotter Report", generate_blotter_report_pdf("Complainant", "Respondent",  
"Noise Complaint", resident["address"], issued_at, issued_by, issued_at, doc_id))
```

```
    quick_check("Health Certificate", generate_health_certificate_pdf(resident,  
"Employment Requirement", issued_by, issued_at, doc_id))
```

```
    quick_check("Barangay ID", generate_barangay_id_pdf(resident, issued_by, issued_at,  
doc_id))
```

```
generators = {
```

```
    "barangay_clearance.pdf": generate_barangay_clearance_pdf(resident, issued_by,  
issued_at, doc_id),
```

```
    "residency_certificate.pdf": generate_residency_certificate_pdf(resident, issued_by,  
issued_at, doc_id),
```

```
    "indigency_certificate.pdf": generate_indigency_certificate_pdf(resident, issued_by,  
issued_at, doc_id),
```

```
    "good_moral_certificate.pdf": generate_good_moral_certificate_pdf(resident, issued_by,  
issued_at, doc_id),
```

```

    "business_clearance.pdf": generate_business_clearance_pdf("Sample Business", "Juan
Dela Cruz", resident["address"], issued_by, issued_at, doc_id),

    "activity_permit.pdf": generate_activity_permit_pdf("Juan Dela Cruz", "Community
Cleanup", resident["address"], issued_at, issued_by, issued_at, doc_id),

    "blotter_report.pdf": generate_blotter_report_pdf("Complainant", "Respondent", "Noise
Complaint", resident["address"], issued_at, issued_by, issued_at, doc_id),

    "health_certificate.pdf": generate_health_certificate_pdf(resident, "Employment
Requirement", issued_by, issued_at, doc_id),

    "barangay_id.pdf": generate_barangay_id_pdf(resident, issued_by, issued_at, doc_id),
}

```

```

for filename, pdf_bytes in generators.items():

```

```

    with open(filename, "wb") as f:

```

```

        f.write(pdf_bytes)

```

```

    print(f"Saved {filename}")

```

```

...

```

```

## backend\tests\test_residents.py

```

```

```python

```

```

from fastapi.testclient import TestClient

```

```

from app.main import app

```

```

client = TestClient(app)

```

```

def test_get_residents():

```



```
response = client.get("/residents/")
assert response.status_code == 200
assert isinstance(response.json(), list)
```

```
...
```

```
backend\tests\test_routes.py
```

```
```python
```

```
import unittest
```

```
from backend.app.main import app
```

```
class TestRoutes(unittest.TestCase):
```

```
    def test_health(self):
```

```
        # Example: use TestClient from FastAPI
```

```
        from fastapi.testclient import TestClient
```

```
        client = TestClient(app)
```

```
        response = client.get("/health")
```

```
        self.assertEqual(response.status_code, 200)
```

```
if __name__ == "__main__":
```

```
    unittest.main()
```

```
...
```

```
## config\role_permissions.json
```

```
```json
```

```
{
```

```
 "admin": [
```

```
 "viewDashboard",
```

```
 "createAccount",
```

```
 "deleteAccount",
```

```
 "updateRole",
```

```
 "manageResidents",
```

```
 "generateCertificates",
```

```
 "viewFinancialRecords",
```

```
 "manageFinancialRecords",
```

```
 "youthRegistryAccess",
```

```
 "auditBarangayData",
```

```
 "documentRequest",
```

```
 "manageDocuments",
```

```
 "issuedDocuments",
```

```
 "rejectedRequests",
```

```
 "viewDocuments",
```

```
 "viewBusinesses",
```

```
 "viewAllComplaints",
```

```
 "manageComplaints",
```

```
 "fileComplaintsForResidents",
```

```
 "viewIncidents",
```

```
 "manageSettings",
```

```
 "manageAnnouncements",
```

```
"manageEvents",
"viewUsers",
"manageUsers"
],
"staff": [
 "viewDashboard",
 "manageBusinesses",
 "registerResidentBusinesses",
 "manageComplaints",
 "manageResidents",
 "viewBusinesses",
 "viewIncidents",
 "viewAllComplaints",
 "fileComplaintsForResidents"
],
"resident": [
 "viewDashboard",
 "fileComplaints",
 "viewOwnComplaints",
 "viewOwnDocuments",
 "requestDocuments",
 "viewOwnBusinesses",
 "registerBusinesses",
 "reportIncidents",
 "viewOwnIncidents",
 "submitFeedback"
```

```
],
"secretary": [
 "viewDashboard",
 "documentRequest",
 "viewDocuments",
 "pendingRequests",
 "paidRequests",
 "issuedDocuments",
 "rejectedRequests",
 "manageDocuments"
],
"treasurer": [
 "viewDashboard",
 "incomingPayments",
 "barangayExpenses",
 "financialReports",
 "settings",
 "manageUsers"
],
"sk": [
 "viewDashboard",
 "youthRegistryAccess"
],
"dilg": [
 "viewDashboard",
 "auditBarangayData"
```

```
]
}
```

```
` ``
```

```
docker-compose.override.yml
```

```
` `` `yaml
```

```
version: "3.9"
```

```
services:
```

```
 backend:
```

```
 volumes:
```

- ./backend:/app/backend # bind-mount your source code
- ./config:/app/config # mount config folder
- ./backend/serviceAccountKey.json:/app/serviceAccountKey.json
- python\_deps:/usr/local/lib/python3.11/site-packages
- app\_data:/app/data

```
 command: uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload
```

```
 environment:
```

```
 GOOGLE_APPLICATION_CREDENTIALS: /app/serviceAccountKey.json
```

```
 env_file:
```

- .env

```
volumes:
```

```
 python_deps:
```

app\_data:

```

docker-compose.yml

```yaml

version: "3.9"

services:

backend:

build:

context: .

dockerfile: Dockerfile

container\_name: bis-backend

ports:

- "8000:8000"

env\_file:

- .env # inject all variables from your local .env

environment:

GOOGLE\_APPLICATION\_CREDENTIALS: /app/serviceAccountKey.json

volumes:

- ./backend/serviceAccountKey.json:/app/serviceAccountKey.json

- ./config:/app/config

restart: unless-stopped

```

Dockerfile

```text

# Use official Python image

FROM python:3.11-slim

# Set working directory

WORKDIR /app

# Copy requirements and install dependencies

COPY backend/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

# Copy backend code and config

COPY backend ./backend

COPY config ./config

# Environment variables

ENV PYTHONPATH=/app

#  Entrypoint for Cloud Run

CMD ["uvicorn", "backend.app.main:app", "--host", "0.0.0.0", "--port", "8080", "--proxy-headers", "--forwarded-allow-ips", "\*"]

```

firebase.json

```json

```
{
 "hosting": {
 "public": "public",
 "ignore": [
 "firebase.json",
 "**/.*",
 "**/node_modules/**"
],
 "rewrites": [
 {
 "source": "**",
 "destination": "/index.html"
 }
]
 },
 "functions": {
 "source": "functions",
 "codebase": "default",
 "disallowLegacyRuntimeConfig": true,
 "params": {
```



```
"gmail_client_id": "397499309217-
9cv9emp42rtcsfq0kjtali98j4v7ms2.apps.googleusercontent.com",

"gmail_client_secret": "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",

"gmail_refresh_token": "1//04hM59iLo_DF8CgYIARAAGAQSNowF-
L9IrxAO1ukPb1X9zts9RBYo_EqMV6qEUPLaVb3maHNMeNCjv4nwJNXLn6lt_rtt79k3PucE"

},

"ignore": [

 "node_modules",

 ".git",

 "firebase-debug.log",

 "firebase-debug.*.log",

 "*.local"

],

"predeploy": [

 "npm --prefix \"$RESOURCE_DIR\" run lint"

]

},

"emulators": {

 "functions": {

 "port": 5001

 },

 "hosting": {

 "port": 5000

 },

 "ui": {

 "enabled": true

 },

}
```

```
"singleProjectMode": true
}
}
```

```
```
```

```
## frontend\src\App.js
```

```
```javascript
```

```
import { BrowserRouter as Router } from "react-router-dom";
import { ToastContainer } from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import { NotificationProvider } from "../context/NotificationContext";
import { UserProvider, useUser } from "../context/UserContext";
import { ThemeProvider } from "../context/ThemeContext";
import AppRoutes from "../routes/AppRoutes";
import "../styles/app.css";
```

```
function AppContent() {
 const { token } = useUser(); //  safe here, inside UserProvider
```

```
 return (
 <ThemeProvider>
 <NotificationProvider token={token}>
 <Router>
 <div className="app-wrapper">
```

```
 <AppRoutes />

 <ToastContainer position="top-right" autoClose={3000} />

 </div>

</Router>

</NotificationProvider>

</ThemeProvider>

);
}
```

```
function App() {
 return (
 <UserProvider>
 <AppContent />
 </UserProvider>
);
}
```

```
export default App;
```

```
...
```

```
frontend\src\App.test.js
```

```
```javascript
```

```
import { render, screen } from '@testing-library/react';
```

```
import App from './App';
```

```
test('renders learn react link', () => {  
  render(<App />);  
  const linkElement = screen.getByText(/learn react/i);  
  expect(linkElement).toBeInTheDocument();  
});
```

```
````
```

```
frontend\src\components\admin\AnalyticsPanel.js
```

```
```javascript
```

```
import { useEffect, useMemo, useState } from "react";  
import { collection, getCountFromServer, query, where, getDocs } from "firebase/firestore";  
import { db } from "../../services/firebase";  
import DashboardCard from "../../dashboard/DashboardCard";  
import { roleCollections, metricConfig } from "../../config/metrics";  
import "../../styles/dashboard/analytics-panel.css";  
import { Bar } from "react-chartjs-2";  
import {  
  Chart as ChartJS,  
  CategoryScale,  
  LinearScale,  
  BarElement,  
  Title,  
  Tooltip,
```

```
Legend,  
} from "chart.js";
```

```
ChartJS.register(CategoryScale, LinearScale, BarElement, Title, Tooltip, Legend);
```

```
const PERIOD_OPTIONS = ["day", "week", "month", "year"];
```

```
const parseTimestamp = (value) => {  
  if (!value) return null;  
  if (value?.toDate && typeof value.toDate === "function") {  
    return value.toDate();  
  }  
  const date = new Date(value);  
  return Number.isNaN(date.getTime()) ? null : date;  
};
```

```
const loginEventWeight = (data = {}) => {  
  const count = Number(data.count);  
  return Number.isFinite(count) && count > 0 ? count : 1;  
};
```

```
const parseAmount = (value) => {  
  const num = Number(value);  
  if (Number.isFinite(num)) return num;
```

```
const cleaned = String(value ?? "").replace(/^[^d.-]/g, "");
```

```
const parsed = Number(cleaned);  
return Number.isFinite(parsed) ? parsed : 0;  
};
```

```
const startOfToday = () => {  
  const now = new Date();  
  return new Date(now.getFullYear(), now.getMonth(), now.getDate());  
};
```

```
const pad2 = (value) => String(value).padStart(2, "0");  
const startOfDay = (date) => new Date(date.getFullYear(), date.getMonth(), date.getDate());  
const startOfMonth = (date) => new Date(date.getFullYear(), date.getMonth(), 1);  
const addMonths = (date, months) => new Date(date.getFullYear(), date.getMonth() + months, 1);  
const addYears = (date, years) => new Date(date.getFullYear() + years, 0, 1);
```

```
const formatDateInput = (date) =>  
  `${date.getFullYear()}-${pad2(date.getMonth() + 1)}-${pad2(date.getDate())}`;
```

```
const formatMonthInput = (date) => `${date.getFullYear()}-${pad2(date.getMonth() + 1)}`;
```

```
const formatWeekInput = (date) => {  
  const utcDate = new Date(Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()));  
  const dayNumber = utcDate.getUTCDay() || 7;  
  utcDate.setUTCDate(utcDate.getUTCDate() + 4 - dayNumber);  
  const yearStart = new Date(Date.UTC(utcDate.getUTCFullYear(), 0, 1));
```

```
const weekNumber = Math.ceil((((utcDate - yearStart) / 86400000) + 1) / 7);  
return `${utcDate.getUTCFullYear()}-W${pad2(weekNumber)}`;  
};
```

```
const parseDateInput = (value) => {  
  if (!value) return null;  
  const [year, month, day] = value.split("-").map(Number);  
  if (!year || !month || !day) return null;  
  return new Date(year, month - 1, day);  
};
```

```
const parseMonthInput = (value) => {  
  if (!value) return null;  
  const [year, month] = value.split("-").map(Number);  
  if (!year || !month) return null;  
  return new Date(year, month - 1, 1);  
};
```

```
const parseWeekInput = (value) => {  
  if (!value || !value.includes("-W")) return null;  
  const [yearPart, weekPart] = value.split("-W");  
  const year = Number(yearPart);  
  const week = Number(weekPart);  
  if (!year || !week) return null;  
  
  const jan4 = new Date(year, 0, 4);
```

```
const jan4DayIndex = (jan4.getDay() + 6) % 7;
const week1Start = new Date(jan4);
week1Start.setDate(jan4.getDate() - jan4DayIndex);

const currentWeekStart = new Date(week1Start);
currentWeekStart.setDate(week1Start.getDate() + (week - 1) * 7);
return startOfDay(currentWeekStart);
};

const shiftDays = (date, days) => {
  const shifted = new Date(date);
  shifted.setDate(shifted.getDate() + days);
  return shifted;
};

const inRange = (date, start, endExclusive) => date >= start && date < endExclusive;

const getSinglePeriodRange = (period, picks, fallbackDate) => {
  const displayDate = fallbackDate || new Date();

  if (period === "day") {
    const start = startOfDay(parseDateInput(picks.day) || displayDate);
    const end = shiftDays(start, 1);
    return {
      start,
      end,
```



```
    label: start.toLocaleDateString(),  
  };  
}
```

```
if (period === "week") {  
  const start = parseWeekInput(picks.week) || startOfDay(displayDate);  
  const end = shiftDays(start, 7);  
  return {  
    start,  
    end,  
    label: `${start.toLocaleDateString()} - ${shiftDays(end, -1).toLocaleDateString()}`,  
  };  
}
```

```
if (period === "month") {  
  const start = parseMonthInput(picks.month) || startOfMonth(displayDate);  
  const end = addMonths(start, 1);  
  return {  
    start,  
    end,  
    label: start.toLocaleString("en-US", { month: "short", year: "numeric" }),  
  };  
}
```

```
const selectedYear = Number(picks.year) || displayDate.getFullYear();  
const start = new Date(selectedYear, 0, 1);
```

```
const end = addYears(start, 1);

return {
  start,
  end,
  label: `${selectedYear}`,
};

};
```

```
const sumEntriesInRange = (entries, start, endExclusive) =>
entries.reduce((total, item) => {
  if (!item?.date || !Number.isFinite(item.value)) return total;
  return inRange(item.date, start, endExclusive) ? total + item.value : total;
}, 0);
```

```
const getLoginMetrics = async (today, now) => {
  try {
    const loginQuery = query(collection(db, "logins"));
    const snapshot = await getDocs(loginQuery);

    let total = 0;

    const entries = [];

    const weekStart = shiftDays(now, -7);
    const monthStart = addMonths(now, -1);

    let weekly = 0;
    let monthly = 0;
```

```
snapshot.docs.forEach((item) => {  
  const data = item.data() || {};  
  const weight = loginEventWeight(data);  
  const timestamp = parseTimestamp(data.timestamp);  
  
  if (timestamp && timestamp >= today) {  
    total += weight;  
  }  
  if (timestamp) {  
    entries.push({ date: timestamp, value: weight });  
    if (timestamp >= weekStart) weekly += weight;  
    if (timestamp >= monthStart) monthly += weight;  
  }  
});  
  
return { total, weekly, monthly, entries };  
} catch (err) {  
  console.warn(" ⚠ Error fetching logins metrics:", err.message);  
  return {  
    total: "N/A",  
    weekly: "N/A",  
    monthly: "N/A",  
    entries: [],  
  };  
}  
};
```

```
const getCountMetrics = async (collectionName, dateFields, now) => {  
  try {  
    const snapshot = await getDocs(collection(db, collectionName));  
    const total = snapshot.size;  
    const entries = [];  
    const weekStart = shiftDays(now, -7);  
    const monthStart = addMonths(now, -1);  
    let weekly = 0;  
    let monthly = 0;  
  
    snapshot.docs.forEach((item) => {  
      const data = item.data() || {};  
      const eventDate = dateFields  
        .map((field) => parseTimestamp(data[field]))  
        .find(Boolean);  
  
      if (eventDate) {  
        entries.push({ date: eventDate, value: 1 });  
        if (eventDate >= weekStart) weekly += 1;  
        if (eventDate >= monthStart) monthly += 1;  
      }  
    });  
  
    return { total, weekly, monthly, entries };  
  } catch (err) {
```

```
console.warn(` ⚠ Cannot access ${collectionName}:`, err.message);

return {
  total: "N/A",
  weekly: "N/A",
  monthly: "N/A",
  entries: [],
};
}
};

const getCollectionsMetrics = async (today, now) => {
  try {
    const paidPaymentsQuery = query(collection(db, "payments"), where("status", "==",
"paid"));
    const snapshot = await getDocs(paidPaymentsQuery);

    let total = 0;
    const entries = [];
    const weekStart = shiftDays(now, -7);
    const monthStart = addMonths(now, -1);
    let weekly = 0;
    let monthly = 0;

    snapshot.docs.forEach((item) => {
      const data = item.data() || {};
      const amount = parseAmount(data.amount);
```

```
const paidDate = parseTimestamp(data.datePaid || data.paymentDate ||
data.timestamp || data.createdAt);
```

```
if (!paidDate) return;
```

```
if (paidDate >= today) {
```

```
    total += amount;
```

```
}
```

```
entries.push({ date: paidDate, value: amount });
```

```
if (paidDate >= weekStart) weekly += amount;
```

```
if (paidDate >= monthStart) monthly += amount;
```

```
});
```

```
return { total, weekly, monthly, entries };
```

```
} catch (err) {
```

```
    console.warn(" ⚠ Cannot access payments for collections analytics:", err.message);
```

```
    return {
```

```
        total: "N/A",
```

```
        weekly: "N/A",
```

```
        monthly: "N/A",
```

```
        entries: [],
```

```
    };
```

```
}
```

```
};
```

```
const formatCurrency = (amount) =>
```

```
new Intl.NumberFormat("en-PH", { style: "currency", currency: "PHP" }).format(amount || 0);
```

```
const AnalyticsPanel = ({ role }) => {  
  const nowAtLoad = new Date();  
  const [metrics, setMetrics] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [selectedMetricKey, setSelectedMetricKey] = useState("");  
  const [hasAutoSelectedMetric, setHasAutoSelectedMetric] = useState(false);  
  const [currentPeriodType, setCurrentPeriodType] = useState("month");  
  const [comparePeriodType, setComparePeriodType] = useState("month");  
  const [periodPicks, setPeriodPicks] = useState({  
    current: {  
      day: formatDateInput(nowAtLoad),  
      week: formatWeekInput(nowAtLoad),  
      month: formatMonthInput(nowAtLoad),  
      year: String(nowAtLoad.getFullYear()),  
    },  
    compare: {  
      day: formatDateInput(shiftDays(nowAtLoad, -1)),  
      week: formatWeekInput(shiftDays(nowAtLoad, -7)),  
      month: formatMonthInput(addMonths(nowAtLoad, -1)),  
      year: String(nowAtLoad.getFullYear() - 1),  
    },  
  });  
  const [error, setError] = useState(null);
```

```
const normalizedRole = role?.trim().toLowerCase();

const accessibleMetrics = useMemo(() => roleCollections[normalizedRole] || [],
[normalizedRole]);
```

```
useEffect(() => {

  setHasAutoSelectedMetric(false);

  setSelectedMetricKey("");

}, [normalizedRole]);
```

```
useEffect(() => {

  if (!normalizedRole) {

    setLoading(false);

    return;

  }

}
```

```
const fetchMetrics = async () => {

  try {

    const now = new Date();

    const today = startOfToday();
```

```
const results = await Promise.all(

  Object.keys(metricConfig).map(async (key) => {

    if (!accessibleMetrics.includes(key)) {

      return {

        key,
```



```
    ...metricConfig[key],  
    value: "N/A",  
    weekly: "N/A",  
    monthly: "N/A",  
    entries: [],  
  };  
}
```

```
if (key === "logins") {  
  const loginMetrics = await getLoginMetrics(today, now);  
  return {  
    key,  
    ...metricConfig[key],  
    value: loginMetrics.total,  
    weekly: loginMetrics.weekly,  
    monthly: loginMetrics.monthly,  
    entries: loginMetrics.entries,  
  };  
}
```

```
if (key === "collections") {  
  const collectionMetrics = await getCollectionsMetrics(today, now);  
  return {  
    key,  
    ...metricConfig[key],  
    value:
```

```
    typeof collectionMetrics.total === "number"
      ? formatCurrency(collectionMetrics.total)
      : collectionMetrics.total,
    weekly: collectionMetrics.weekly,
    monthly: collectionMetrics.monthly,
    entries: collectionMetrics.entries,
  };
}
```

```
if (key === "businesses") {
  const businessMetrics = await getCountMetrics(
    "businesses",
    ["submittedAt", "createdAt", "timestamp"],
    now
  );
  return {
    key,
    ...metricConfig[key],
    value: businessMetrics.total,
    weekly: businessMetrics.weekly,
    monthly: businessMetrics.monthly,
    entries: businessMetrics.entries,
  };
}
```

```
if (key === "complaints") {
```

```

const complaintMetrics = await getCountMetrics(
  "complaints",
  ["createdAt", "timestamp", "submittedAt"],
  now
);
return {
  key,
  ...metricConfig[key],
  value: complaintMetrics.total,
  weekly: complaintMetrics.weekly,
  monthly: complaintMetrics.monthly,
  entries: complaintMetrics.entries,
};
}

try {
  const snap = await getCountFromServer(collection(db, key));
  const total = snap.data().count;
  const metricCounts = await getCountMetrics(
    key,
    ["createdAt", "timestamp", "submittedAt"],
    now
  );
  const weekly = metricCounts.weekly;
  const monthly = metricCounts.monthly;
  return {

```

```

    key,
    ...metricConfig[key],
    value: total,
    weekly,
    monthly,
    entries: metricCounts.entries,
  };
} catch (err) {
  console.warn(` ⚠️ Error fetching ${key}:`, err.message);
  setError("Failed to load analytics data.");
  return {
    key,
    ...metricConfig[key],
    value: "N/A",
    weekly: "N/A",
    monthly: "N/A",
    entries: [],
  };
}
})
);

setMetrics(results);
} finally {
  setLoading(false);
}

```

```

};

fetchMetrics();
}, [normalizedRole, accessibleMetrics]);

useEffect(() => {
  if (!metrics.length) return;

  const hasSelected = metrics.some((metric) => metric.key === selectedMetricKey);
  if (!hasSelected && !selectedMetricKey && !hasAutoSelectedMetric) {
    const preferred = metrics.find((metric) => metric.key === "collections" && metric.value
    !== "N/A");

    const firstAvailable = preferred || metrics.find((metric) => metric.value !== "N/A") ||
    metrics[0];

    setSelectedMetricKey(firstAvailable?.key || "");

    setHasAutoSelectedMetric(true);

    return;
  }

  if (!hasSelected && selectedMetricKey) {
    const preferred = metrics.find((metric) => metric.key === "collections" && metric.value
    !== "N/A");

    const firstAvailable = preferred || metrics.find((metric) => metric.value !== "N/A") ||
    metrics[0];

    setSelectedMetricKey(firstAvailable?.key || "");
  }
}, [metrics, selectedMetricKey, hasAutoSelectedMetric]);

```

```
if (!normalizedRole) {  
  return (  
    <section className="analytics-panel">  
      <h3> 📊 Analytics Panel</h3>  
      <p>Analytics not available for your role.</p>  
    </section>  
  );  
}
```

```
const selectedMetric = metrics.find((metric) => metric.key === selectedMetricKey);  
  
const currentRange = getSinglePeriodRange(currentPeriodType, periodPicks.current, new  
Date());  
  
const compareRange = getSinglePeriodRange(comparePeriodType, periodPicks.compare,  
new Date());  
  
const selectedEntries = selectedMetric?.entries || [];  
  
const selectedComparison = {  
  previous: sumEntriesInRange(selectedEntries, compareRange.start,  
compareRange.end),  
  
  current: sumEntriesInRange(selectedEntries, currentRange.start, currentRange.end),  
};  
  
const periodLabels = { previous: compareRange.label, current: currentRange.label };  
  
const chartIsCurrency = selectedMetricKey === "collections";  
  
const chartOptions = {  
  responsive: true,  
  
  plugins: {  
    legend: { position: "top" },
```

```
title: {
  display: true,
  text: selectedMetric
  ? `${selectedMetric.label}: ${periodLabels.previous} vs ${periodLabels.current}`
  : "Analytics Trends",
},
tooltip: {
  callbacks: {
    label: (context) => {
      const value = Number(context.raw || 0);
      if (chartIsCurrency) {
        return formatCurrency(value);
      }
      return `${value}`;
    },
  },
},
},
scales: {
  x: { type: "category", title: { display: true, text: "Period" } },
  y: {
    beginAtZero: true,
    title: { display: true, text: chartIsCurrency ? "Amount" : "Count" },
  },
},
};
```

```

const chartData = {
  labels: [periodLabels.previous, periodLabels.current],
  datasets: [
    {
      label: selectedMetric?.label || "Metric",
      data: [
        Number(selectedComparison.previous) || 0,
        Number(selectedComparison.current) || 0,
      ],
      backgroundColor: ["#94a3b8", "#3b82f6"],
    },
  ],
};

return (
  <section className="analytics-panel" aria-labelledby="analytics-title">
    <header className="analytics-header">
      <h3 id="analytics-title">  {normalizedRole} Analytics</h3>
    </header>

    <div className="metrics-grid" aria-busy={loading} aria-live="polite">
      {loading ? (
        <p>Loading analytics...</p>
      ) : error ? (
        <p className="error">{error}</p>
      ) : (

```



```

):(
<>

{metrics.map(({ key, label, value, variant, icon }, index) => (
  <div
    key={index}
    className={`metric-card-wrap ${selectedMetricKey === key ? "selected" : ""}`}
    onClick={() => setSelectedMetricKey((prev) => (prev === key ? "" : key))}
    role="button"
    tabIndex={0}
    onKeyDown={(event) => {
      if (event.key === "Enter" || event.key === " ") {
        event.preventDefault();
        setSelectedMetricKey((prev) => (prev === key ? "" : key));
      }
    }}
  >
    <DashboardCard label={label} value={value} variant={variant} icon={icon} />
  </div>
)}}

{!!selectedMetric && (
  <div className="analytics-chart">
    <div className="chart-controls">
      <label htmlFor="analytics-category-select">Category</label>
      <select
        id="analytics-category-select"
        className="category-select"

```

```

value={selectedMetricKey}
onChange={(e) => setSelectedMetricKey(e.target.value)}
>
{metrics.map((metric) => (
  <option key={metric.key} value={metric.key}>
    {metric.label}
  </option>
))}
</select>

<div className="compare-controls">
  <div className="compare-block">
    <span className="compare-title">Current</span>
    <div className="period-toggle" aria-label="Current Period Toggle">
      {PERIOD_OPTIONS.map((period) => (
        <button
          key={`current-${period}`}
          className={currentPeriodType === period ? "active" : ""}
          type="button"
          onClick={() => setCurrentPeriodType(period)}
        >
          {period.charAt(0).toUpperCase() + period.slice(1)}
        </button>
      ))}
    </div>
  </div>

```

```
{currentPeriodType === "day" && (  
  <input  
    type="date"  
    className="period-input"  
    value={periodPicks.current.day}  
    onChange={(e) =>  
      setPeriodPicks((prev) => ({  
        ...prev,  
        current: { ...prev.current, day: e.target.value },  
      })))  
  }  
/>  
)}
```

```
{currentPeriodType === "week" && (  
  <input  
    type="week"  
    className="period-input"  
    value={periodPicks.current.week}  
    onChange={(e) =>  
      setPeriodPicks((prev) => ({  
        ...prev,  
        current: { ...prev.current, week: e.target.value },  
      })))  
  }  
/>  
)}
```

```
}}
```

```
{currentPeriodType === "month" && (  
  <input  
    type="month"  
    className="period-input"  
    value={periodPicks.current.month}  
    onChange={(e) =>  
      setPeriodPicks((prev) => ({  
        ...prev,  
        current: { ...prev.current, month: e.target.value },  
      })))  
  }  
/>  
)}
```

```
{currentPeriodType === "year" && (  
  <input  
    type="number"  
    className="period-input"  
    min="2000"  
    max="2100"  
    value={periodPicks.current.year}  
    onChange={(e) =>  
      setPeriodPicks((prev) => ({  
        ...prev,
```

```

        current: { ...prev.current, year: e.target.value },
      )))
    }
  />
)}
</div>

```

```

<div className="compare-block">
  <span className="compare-title">Compare</span>
  <div className="period-toggle" aria-label="Compare Period Toggle">
    {PERIOD_OPTIONS.map((period) => (
      <button
        key={`compare-${period}`}
        className={comparePeriodType === period ? "active" : ""}
        type="button"
        onClick={() => setComparePeriodType(period)}
      >
        {period.charAt(0).toUpperCase() + period.slice(1)}
      </button>
    ))}
  </div>

```

```

{comparePeriodType === "day" && (
  <input
    type="date"
    className="period-input"

```

```
value={periodPicks.compare.day}
onChange={(e) =>
  setPeriodPicks((prev) => ({
    ...prev,
    compare: { ...prev.compare, day: e.target.value },
  )))
}
/>
}}
```

```
{comparePeriodType === "week" && (
  <input
    type="week"
    className="period-input"
    value={periodPicks.compare.week}
    onChange={(e) =>
      setPeriodPicks((prev) => ({
        ...prev,
        compare: { ...prev.compare, week: e.target.value },
      )))
    }
  />
)}
```

```
{comparePeriodType === "month" && (
  <input
```

```

    type="month"
    className="period-input"
    value={periodPicks.compare.month}
    onChange={(e) =>
      setPeriodPicks((prev) => ({
        ...prev,
        compare: { ...prev.compare, month: e.target.value },
      )))
  }
/>
})

```

```

{comparePeriodType === "year" && (
  <input
    type="number"
    className="period-input"
    min="2000"
    max="2100"
    value={periodPicks.compare.year}
    onChange={(e) =>
      setPeriodPicks((prev) => ({
        ...prev,
        compare: { ...prev.compare, year: e.target.value },
      )))
  }
/>

```

```

        })
      </div>
    </div>
  </div>
  <Bar data={chartData} options={chartOptions} />
</div>
})
</>
})
</div>
</section>
);
};

```

```
export default AnalyticsPanel;
```

```
` ``
```

```
## frontend\src\components\admin\CreateAccountForm.js
```

```
` `` javascript
```

```

import { useState } from "react";
import { useUser } from "../../context/UserContext";
import { api } from "../../services/api";
import { ROLE_OPTIONS } from "../../config/roles";
import "../../styles/admin.css";

```



```
const defaultForm = {  
  full_name: "",  
  email: "",  
  password: "",  
  role: "staff",  
};
```

```
const CreateAccountForm = () => {  
  const { getToken } = useUser();  
  const [form, setForm] = useState(defaultForm);  
  const [loading, setLoading] = useState(false);  
  const [confirming, setConfirming] = useState(false);  
  const [feedback, setFeedback] = useState("");
```

```
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    setForm((prev) => ({ ...prev, [name]: value }));  
  };
```

```
  const resetForm = () => setForm(defaultForm);
```

```
  const validateForm = () => {  
    if (!form.full_name.trim()) return "Full name is required.";  
    if (!form.email.includes("@")) return "Invalid email address.";  
    if (form.password.length < 6) return "Password must be at least 6 characters.";
```

```
return null;
```

```
};
```

```
const handleSubmit = async (e) => {
```

```
  e.preventDefault();
```

```
  const validationError = validateForm();
```

```
  if (validationError) {
```

```
    setFeedback(` ❌ ${validationError} `);
```

```
    return;
```

```
  }
```

```
  setLoading(true);
```

```
  try {
```

```
    const token = await getToken();
```

```
    const { data } = await api.post("/api/admin/create-account", form, {
```

```
      headers: { Authorization: `Bearer ${token}` },
```

```
    });
```

```
    setFeedback(` ✅ Account created: ${data.uid} `);
```

```
    resetForm();
```

```
    setConfirming(false);
```

```
  } catch (error) {
```

```
    let errorMsg;
```

```
    if (error.response?.data) {
```

```
      const data = error.response.data;
```

```

// Case 1: API returns { detail: "message" }
if (typeof data.detail === "string") {
  errorMsg = data.detail;
}

// Case 2: API returns { detail: [{ msg: "error message", loc: [...] }] }
else if (Array.isArray(data.detail)) {
  errorMsg = data.detail.map(err => err.msg || JSON.stringify(err)).join("; ");
}

// Fallback: stringify the whole response
else {
  errorMsg = JSON.stringify(data);
}
} else {
  errorMsg = error.message;
}

console.error("❌ Account creation failed:", errorMsg);
setFeedback(`❌ Failed to create account: ${errorMsg}`);
setConfirming(false);
} finally {
  setLoading(false);
}
};

return (

```

```
<form
  className="create-account-form"
  onSubmit={handleSubmit}
  aria-busy={loading}
  aria-live="polite"
>

<h3>  Create Barangay Account</h3>
```

```
{feedback} && <p className="feedback">{feedback}</p>}
```

```
<label>
  Full Name
  <input
    name="full_name"
    placeholder="Juan Dela Cruz"
    value={form.full_name}
    onChange={handleChange}
    required
  />
</label>
```

```
<label>
  Email
  <input
    name="email"
    type="email"
```

```
placeholder="juan@example.com"
value={form.email}
onChange={handleChange}
required
/>
</label>
```

```
<label>
Password
<input
  name="password"
  type="password"
  placeholder="●●●●●●●●"
  value={form.password}
  onChange={handleChange}
  required
/>
</label>
```

```
<label>
Role
<select name="role" value={form.role} onChange={handleChange}>
  {ROLE_OPTIONS.filter((opt) => opt.value !== "admin").map((opt) => (
    <option key={opt.value} value={opt.value}>
      {opt.label}
    </option>
  )}
</select>
```

```

    )}}
  </select>
</label>

{!confirming ? (
  <button
    type="button"
    onClick={() => setConfirming(true)}
    disabled={loading}
  >
    {loading ? "Creating..." : "Confirm & Create Account"}
  </button>
): (
  <div className="confirm-actions">
    <p>
      Are you sure you want to create this account for{" " }
      <strong>{form.full_name}</strong>?
    </p>
    <button type="submit" className="btn-success" disabled={loading}>
       Yes, Create
    </button>
    <button
      type="button"
      className="btn-danger"
      onClick={() => setConfirming(false)}
      disabled={loading}

```

```

    >
     Cancel
  </button>
</div>

)}
</form>

);
};

```

```
export default CreateAccountForm;
```

```

` ` `

```

```
## frontend\src\components\admin\FeeTable.js
```

```
` ` ` javascript
```

```
import "../styles/fee-dashboard.css";
```

```
export default function FeeTable({ title, columns, data, onUpdate, onDelete }) {
```

```
  const handleChange = (item, col, e) => {
```

```
    let value;
```

```
    if (col.type === "checkbox") {
```

```
      value = e.target.checked;
```

```
    } else if (col.type === "number") {
```

```
      value = Number(e.target.value);
```

```
    } else {
```

```

    value = e.target.value;
  }
  // Update the specific field immediately
  onUpdate(item.id, col.key, value, item);
};

const handleSave = (item) => {
  // Save the entire item with its current values
  columns.forEach(col => {
    if (col.editable) {
      onUpdate(item.id, col.key, item[col.key], item);
    }
  });
};

```

```

return (
  <div className="fee-section">
    <h2>{title}</h2>
    <table className="fee-table">
      <thead>
        <tr>
          {columns.map(col => (
            <th key={col.key}>{col.label}</th>
          ))}
          <th>Actions</th>
        </tr>

```



```

</thead>

<tbody>

{data.map(item => (

  <tr key={item.id}>

    {columns.map(col => (

      <td key={col.key}>

        {col.editable ? (

          col.type === "checkbox" ? (

            <input

              type="checkbox"

              checked={!!item[col.key]}

              onChange={e => handleChange(item, col, e)}

            />

          ) : (

            <input

              type={col.type || "text"}

              value={item[col.key] ?? ""}

              onChange={e => handleChange(item, col, e)}

            />

          )

        ) : (

          item[col.key]?.toString() ?? "—"

        )}

      </td>

    )}

  <td>

```

```

        <button onClick={() => handleSave(item)}>Save</button>

        <button onClick={() => onDelete(item.id)}>Delete</button>

    </td>

</tr>

    )}

</tbody>

</table>

</div>

);

}

```

...

frontend\src\components\admin\RoleManager.js

```

```javascript

import { useEffect, useState, useCallback } from "react";
import { collection, getDocs } from "firebase/firestore";
import { auth, db } from "../../services/firebase";
import { AccountsAPI, RolesAPI, api } from "../../services/api";
import { useUser } from "../../context/UserContext";
import { ROLE_OPTIONS } from "../../config/roles";
import "../../styles/dashboard/role-manager.css";

const RoleManager = () => {

 const [users, setUsers] = useState([]);

```

```

const [loading, setLoading] = useState(true);
const [feedback, setFeedback] = useState("");
const [error, setError] = useState(null);
const [pendingUserId, setPendingUserId] = useState(null);
const [rolePresence, setRolePresence] = useState({});

const { userInfo, isAdmin, can } = useUser();
const hasManagePermission = isAdmin || can("manageUsers");

const fetchRolePresence = useCallback(async () => {
 if (!hasManagePermission) return;

 try {
 const { data } = await api.get("/api/ws/presence/roles");
 setRolePresence(data?.roles || {});
 } catch (err) {
 console.warn(" ⚠️ Failed to load role presence:", err?.message || err);
 setRolePresence({});
 }
}, [hasManagePermission]);

// 🔍 Fetch users (only if allowed)
const fetchUsers = useCallback(async () => {
 if (!hasManagePermission) {
 setError(" ❌ You do not have permission to view users.");
 setLoading(false);
 }

```

```

 return;
 }

 setLoading(true);

 try {
 // Force token refresh to ensure latest claims
 await auth.currentUser?.getIdToken(true);

 const snapshot = await getDocs(collection(db, "users"));
 const data = snapshot.docs.map((doc) => {
 const role = doc.data().role?.trim().toLowerCase();
 if (!ROLE_OPTIONS.some((opt) => opt.value === role)) {
 console.warn(
 ` ⚠️ Unknown role "${role}" for UID ${doc.id}, defaulting to resident `
);
 }
 });
 return {
 id: doc.id,
 ...doc.data(),
 role: ROLE_OPTIONS.some((opt) => opt.value === role)
 ? role
 : "resident", // normalize + validate
 };
 });

 setUsers(data);
 setError(null);

```

```

 await fetchRolePresence();
 } catch (err) {
 console.error(" ❌ Failed to load users:", err);
 setError("Failed to load users.");
 } finally {
 setLoading(false);
 }
}, [hasManagePermission, fetchRolePresence]);

```

```

useEffect(() => {
 fetchUsers();
}, [fetchUsers]);

```

```

useEffect(() => {
 if (!hasManagePermission) return;

```

```

 const interval = setInterval(() => {
 fetchRolePresence();
 }, 10000);

```

```

 return () => clearInterval(interval);
}, [hasManagePermission, fetchRolePresence]);

```

// 🛠️ Unified safe API call

```

const safeApiCall = useCallback(
 async (context, apiFunc, rollback, ...args) => {

```

```

try {
 setPendingUserId(args[0]); // assume first arg is userId

 const result = await apiFunc(...args);

 setFeedback(`✅ ${context} succeeded`);

 return result;
} catch (err) {
 console.error(`❌ ${context} error:`, err);

 if (rollback) rollback();

 setFeedback(`❌ Failed to ${context.toLowerCase()}`);

 return null;
} finally {
 setPendingUserId(null);
}
},
[]
);

```

// 🔧 Handle role change

```

const handleRoleChange = useCallback(
 async (userId, newRole) => {
 if (userId === userInfo?.uid && newRole !== "admin") {
 setFeedback("⚠️ You cannot downgrade your own role.");
 return;
 }
 }
);

```

```
}

if (!hasManagePermission) {
 setFeedback(" ❌ You do not have permission to change roles.");
 return;
}
```

```
const prevUsers = [...users];
setUsers((prev) =>
 prev.map((u) => (u.id === userId ? { ...u, role: newRole } : u))
);
```

```
const updatedAccount = await safeApiCall(
 "update role",
 AccountsAPI.updateRole,
 () => setUsers(prevUsers),
 userId,
 newRole
);
```

```
if (updatedAccount) {
 setUsers((prev) =>
 prev.map((u) => (u.id === userId ? { ...u, ...updatedAccount } : u))
);
}
},
[userInfo, hasManagePermission, users, safeApiCall]
```

```
);
```

```
// 🛠️ Handle delete
```

```
const handleDeleteUser = useCallback(
```

```
 async (userId) => {
```

```
 if (userId === userInfo?.uid) {
```

```
 setFeedback(" ⚠️ You cannot delete your own account.");
```

```
 return;
```

```
 }
```

```
 if (!hasManagePermission) {
```

```
 setFeedback(" ❌ You do not have permission to delete users.");
```

```
 return;
```

```
 }
```

```
 if (!window.confirm("Are you sure you want to delete this user?")) {
```

```
 return;
```

```
 }
```

```
 const prevUsers = [...users];
```

```
 setUsers((prev) => prev.filter((u) => u.id !== userId));
```

```
 await safeApiCall(
```

```
 "delete user",
```

```
 AccountsAPI.delete,
```

```
 () => setUsers(prevUsers),
```

```
 userId
```

```
);
```



```
},
[userInfo, hasManagePermission, users, safeApiCall]
);
```

```
// 🛠️ Auto-clear feedback
```

```
useEffect(() => {
 if (feedback) {
 const timer = setTimeout(() => setFeedback(""), 5000);
 return () => clearTimeout(timer);
 }
}, [feedback]);
```

```
const sortedUsers = [...users].sort(
 (a, b) =>
 a.role.localeCompare(b.role) ||
 (a.full_name || "").localeCompare(b.full_name || "")
);
```

```
const getRoleOnline = (role) => Boolean(rolePresence?.[role]?.online);
```

```
const bootstrapAdmin = async () => {
 try {
 const result = await RolesAPI.assignRole(userInfo.uid, "admin");
 setFeedback(`✅ Role '${result.role}' assigned to your account`);
 // Refresh token so claim is visible right away
 await auth.currentUser.getIdToken(true);
```

```

const tokenResult = await auth.currentUser.getIdTokenResult();

console.log("Updated role claim:", tokenResult.claims.role);
} catch (err) {

 console.error("❌ Failed to assign admin role:", err);

 setFeedback("❌ Failed to assign admin role");

}

};

return (
 <section className="role-manager" aria-busy={loading} aria-live="polite">

 <h3>👤 Role Manager</h3>

 {feedback} && <p className="feedback">{feedback}</p>

 <button onClick={fetchUsers} disabled={loading || !hasManagePermission}>

 ↶ Refresh Users

 </button>

 {loading ? (
 <p>Loading users...</p>
) : error ? (
 <p className="error">{error}</p>
) : users.length === 0 ? (
 <p>No users available for your role.</p>
) : (
 <table className="role-table" aria-label="User Role Table">

 <thead>

 <tr>

 <th>Name</th>

```

```

<th>Current Role</th>

<th>Status</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{sortedUsers.map((user) => (

 <tr

 key={user.id}

 className={pendingUserId === user.id ? "pending-row" : ""}

 >

 <td>{user.full_name || "Unnamed User"}</td>

 <td>

 {user.role}

 </td>

 <td>

 <span className="presence-indicator" title={getRoleOnline(user.role) ? "Online"
: "Offline"}>

 <span

 className={`presence-dot ${getRoleOnline(user.role) ? "online" : "offline"}` }

 aria-label={getRoleOnline(user.role) ? "Online" : "Offline"}

 />

 </td>

 <td>

 <select

```

```

 value={user.role}
 onChange={(e) => handleRoleChange(user.id, e.target.value)}
 aria-label={` Change role for ${user.full_name || "user"}`}
 disabled={!hasManagePermission || pendingUserId === user.id}
 >
 {ROLE_OPTIONS.map((opt) => (
 <option key={opt.value} value={opt.value}>
 {opt.label}
 </option>
))}
 </select>
 {user.id === userInfo?.uid && user.role !== "admin" && (
 <button onClick={bootstrapAdmin}>
 🚀 Bootstrap My Admin Role
 </button>
)}
 <button
 onClick={() => handleDeleteUser(user.id)}
 disabled={!hasManagePermission || pendingUserId === user.id}
 className="delete-btn danger"
 >
 🗑 Delete
 </button>
 </td>
</tr>
)))

```

```

 </tbody>
 </table>

 })
</section>

);

};

export default RoleManager;

` ``

frontend\src\components\audit\AuditExportPanel.js

` `` javascript
import "../../styles/admin.css";

const AuditExportPanel = () => {
 const handleExport = () => {
 console.log("Audit export triggered");
 // TODO: Generate and download audit report
 };

 return (
 <div className="audit-export-panel">
 <h3> 📦 Export Tools</h3>

```

<p>Download audit logs, registry snapshots, or compliance summaries for offline review.</p>

<button onClick={handleExport}>Export Audit Report</button>

</div>

);

};

export default AuditExportPanel;

...

## frontend\src\components\audit\ComplianceReport.js

```javascript

import "../styles/admin.css";

const mockCompliance = [

{ id: 1, category: "Resident Registry", status: "Compliant", lastAudit: "Oct 15, 2025" },

{ id: 2, category: "Document Logs", status: "Pending Review", lastAudit: "Oct 10, 2025" },

];

const ComplianceReport = () => (

<div className="compliance-report">

<h3>📄 Compliance Summary</h3>

{mockCompliance.map((item) => (

```

    <li key={item.id}>
      <strong>{item.category}</strong>: {item.status} <br />
      <small>Last audited: {item.lastAudit}</small>
    </li>
  )}
</ul>
</div>
);

```

```
export default ComplianceReport;
```

```

` ` `

```

```
## frontend\src\components\audit\RegistrySnapshot.js
```

```
` ` ` javascript
```

```
import "../styles/admin.css";
```

```

const mockSnapshot = [
  { registry: "Residents", entries: 1245 },
  { registry: "Businesses", entries: 312 },
  { registry: "Youth", entries: 198 },
];

```

```

const RegistrySnapshot = () => (
  <div className="registry-snapshot">

```

```
<h3> 📁 Registry Snapshot</h3>
```

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Registry</th>
```

```
      <th>Total Entries</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    {mockSnapshot.map((r, index) => (
```

```
      <tr key={index}>
```

```
        <td>{r.registry}</td>
```

```
        <td>{r.entries}</td>
```

```
      </tr>
```

```
    )))
```

```
  </tbody>
```

```
</table>
```

```
</div>
```

```
);
```

```
export default RegistrySnapshot;
```

```
...
```

```
## frontend\src\components\audit\SystemUsageStats.js
```



```
` `` javascript
```

```
import "../../styles/admin.css";
```

```
const mockUsage = [  
  { metric: "Logins (30 days)", value: 142 },  
  { metric: "Documents Generated", value: 87 },  
  { metric: "Complaints Filed", value: 23 },  
];
```

```
const SystemUsageStats = () => (  
  <div className="system-usage-stats">  
    <h3> 📊 System Usage</h3>  
    <ul>  
      {mockUsage.map((stat, index) => (  
        <li key={index}>  
          <strong>{stat.metric}</strong>: {stat.value}  
        </li>  
      ))}  
    </ul>  
  </div>  
);
```

```
export default SystemUsageStats;
```

```
` ``
```

```
## frontend\src\components\auth\ProtectedRoute.js
```

```
` `` javascript
```

```
import PropTypes from "prop-types";
```

```
import { Navigate } from "react-router-dom";
```

```
import { useUser } from "../../context/UserContext";
```

```
const ProtectedRoute = ({
```

```
  allowedRoles = [],
```

```
  allowAdminOverride = true,
```

```
  fallback = null,
```

```
  redirectPath = "/unauthorized",
```

```
  children,
```

```
}) => {
```

```
  const { isAuthenticated, loading, hasAccess, role } = useUser();
```

```
  // ⌚ Show fallback while loading
```

```
  if (loading) {
```

```
    return fallback || <div className="loading">  Verifying access...</div>;
```

```
  }
```

```
  // 🚫 Redirect if not logged in
```

```
  if (!isAuthenticated) {
```

```
    console.info("🚫 Redirecting unauthenticated user to login.");
```

```
    return <Navigate to="/login" replace />;
```

```
  }
```

```
//  Redirect if role is not allowed

if (!hasAccess(allowedRoles, allowAdminOverride)) {

  console.warn(

    `  Access denied for role "${role}". Allowed roles: [${allowedRoles.join(", ")}] `

  );

  return <Navigate to={redirectPath} replace />;
}
```

```
//  Authorized

return children;

};
```

```
ProtectedRoute.propTypes = {

  allowedRoles: PropTypes.arrayOf(PropTypes.string),

  allowAdminOverride: PropTypes.bool,

  fallback: PropTypes.node,

  redirectPath: PropTypes.string,

  children: PropTypes.node.isRequired,

};
```

```
export default ProtectedRoute;
```

```
` `` `
```

```
## frontend\src\components\auth\ResidentDashboardWrapper.js
```

```

```javascript
import { useUser } from "../../context/UserContext";
import ResidentDashboard from "../../pages/ResidentDashboard";

const ResidentDashboardWrapper = () => {
 const { userInfo } = useUser();
 return <ResidentDashboard residentId={userInfo?.uid} />;
};

export default ResidentDashboardWrapper;
```

```

frontend\src\components\auth\SecureNavLink.js

```

```javascript
import PropTypes from "prop-types";
import { NavLink } from "react-router-dom";
import { useUser } from "../../context/UserContext";

const SecureNavLink = ({ action, to, children, ...props }) => {
 const { can } = useUser();

 // 🚫 Hide link if user lacks permission
 if (!can(action)) return null;

```

```
return (

 <NavLink to={to} {...props}>
 {children}
 </NavLink>

);
};
```

```
SecureNavLink.propTypes = {
 action: PropTypes.string.isRequired,
 to: PropTypes.string.isRequired,
 children: PropTypes.node.isRequired,
};
```

```
export default SecureNavLink;
```

```
````
```

```
## frontend\src\components\dashboard\ComplaintList.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
import { ComplaintsAPI } from "../../services/api";
import { useUser } from "../../context/UserContext";
import { getAuth, onAuthStateChanged } from "firebase/auth";
```

```
import "../../styles/dashboard/complaint-list.css";

const ComplaintList = () => {
 const { role, can } = useUser();

 const [complaints, setComplaints] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);

 const canViewOwn = can("viewOwnComplaints");
 const canViewAll = can("viewAllComplaints");
 const canView = canViewOwn || canViewAll;

 useEffect(() => {
 const auth = getAuth();

 const unsubscribe = onAuthStateChanged(auth, async (user) => {
 if (!user) {
 setError("✖ Not logged in.");
 setLoading(false);
 return;
 }

 if (!canView) {
 setError("✖ You do not have permission to view complaints.");
 setLoading(false);
 }
 });
 });
}
```

```

 return;
 }

 try {
 // 🗝️ Force refresh token before API call
 await user.getIdToken(true);

 const data = canViewAll
 ? await ComplaintsAPI.listAll()
 : await ComplaintsAPI.listMine();

 setComplaints(data);
 setError(null);
 } catch (err) {
 console.error("❌ Failed to load complaints:", err);
 setError("Failed to load complaints.");
 setComplaints([]);
 } finally {
 setLoading(false);
 }
});

return () => unsubscribe();
}, [canView, canViewAll, canViewOwn, role]);

const renderContent = () => {

```

if (!canView) return <p>✖ You do not have access to view complaints.</p>;

if (loading) return <p>Loading complaints...</p>;

if (error) return <p className="error">{error}</p>;

if (complaints.length === 0) return <p>No complaints available.</p>;

return (

<table className="complaint-table">

<thead>

<tr>

{canViewAll && <th>Resident</th>}

<th>Category</th>

<th>Description</th>

<th>Location</th>

<th>Status</th>

<th>Timestamp</th>

<th>Resolution Notes</th>

</tr>

</thead>

<tbody>

{complaints.map((c) => (

<tr key={c.id}>

{canViewAll && <td>{c.filed\_for\_name || c.filed\_by\_name || "—"}</td>}

<td>{c.category}</td>

<td>{c.description}</td>

<td>{c.location}</td>

<td>{c.status}</td>



```

 <td>{c.timestamp ? new Date(c.timestamp).toLocaleString() : "—"}</td>

 <td>{c.resolution_notes || "—"}</td>

 </tr>

)}

</tbody>

</table>

);

};

```

```

return (
 <div className="complaint-list">

 <h3> 🚩 Complaints</h3>

 {renderContent()}

 </div>

);

};

```

```
export default ComplaintList;
```

```
```
```

```
## frontend\src\components\dashboard\DashboardCard.js
```

```
```javascript
```

```
import PropTypes from "prop-types";
```

```
import "../styles/dashboard/dashboard-card.css";
```

```

const DashboardCard = ({
 label,
 value = "...",
 variant = "accent",
 icon = null,
}) => {
 const cardId = `card-${label.replace(/\s+/g, "-").toLowerCase()}`;

 return (
 <div
 className={`dashboard-card ${variant}`}
 aria-labelledby={cardId}
 role="region"
 >
 <h4 id={cardId}>
 {icon && {icon}}{label}
 </h4>
 <p aria-live="polite">{value}</p>
 </div>
);
};

```

```

DashboardCard.propTypes = {
 label: PropTypes.string.isRequired,

```

```
value: PropTypes.oneOfType([PropTypes.string, PropTypes.number]),
variant: PropTypes.oneOf([
 "accent",
 "success",
 "info",
 "danger",
 "warning",
 "dilig",
 "neutral",
 "youth",
]),
icon: PropTypes.node,
};
```

```
export default DashboardCard;
```

```
` ``
```

```
frontend\src\components\dashboard\DocumentQueue.js
```

```
` `` javascript
```

```
import { useEffect, useState } from "react";
```

```
import ReactDOM from "react-dom";
```

```
import { collection, getDocs, doc, updateDoc } from "firebase/firestore";
```

```
import { db } from "../../services/firebase";
```

```
import { useUser } from "../../context/UserContext";
```

```
import "../../styles/dashboard/document-queue.css";
```

```
const DocumentQueue = () => {
 const { role } = useUser();
 const [requests, setRequests] = useState([]);
 const [selectedRequest, setSelectedRequest] = useState(null);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);
```

```
// 🔍 Fetch requests
```

```
useEffect(() => {
 const fetchRequests = async () => {
 try {
 const allowedRoles = ["admin", "secretary", "staff"];
 if (!allowedRoles.includes(role)) {
 setRequests([]);
 setLoading(false);
 return;
 }
 const snapshot = await getDocs(collection(db, "documents"));
 const data = snapshot.docs.map((doc) => ({
 id: doc.id,
 ...doc.data(),
 }));
 setRequests(data);
```

```
 } catch (err) {
 console.error(" ❌ Failed to load requests:", err);
 setError("Failed to load requests.");
 } finally {
 setLoading(false);
 }
 };
};
```

```
 fetchRequests();
}, [role]);
```

```
const handleReview = (req) => setSelectedRequest(req);
const closeModal = () => setSelectedRequest(null);
```

```
// 🛠 Approve/Reject actions
```

```
const updateStatus = async (newStatus) => {
 if (!selectedRequest) return;
 try {
 const ref = doc(db, "documents", selectedRequest.id);
 await updateDoc(ref, { status: newStatus });
 setRequests((prev) =>
 prev.map((r) =>
 r.id === selectedRequest.id ? { ...r, status: newStatus } : r
)
);
 closeModal();
 }
```

```

} catch (err) {

 console.error(" ❌ Failed to update status:", err);

 alert("Failed to update request.");

}

};

```

```

return (

 <div

 className={`document-queue ${selectedRequest ? "blurred" : ""}`}

 aria-busy={loading}

 aria-live="polite"

 >

 <h3> 📄 Document Requests</h3>

 {loading ? (

 <p>Loading requests...</p>

) : error ? (

 <p className="error">{error}</p>

) : requests.length === 0 ? (

 <p>No requests available.</p>

) : (

 <table className="queue-table" aria-label="Document Requests Table">

 <thead>

 <tr>

 <th>Resident</th>

 <th>Type</th>

 <th>Status</th>


```

```

 <th>Action</th>

 </tr>

</thead>

<tbody>

 {requests.map((req) => (

 <tr key={req.id}>

 <td>{req.residentName}</td>

 <td>{req.documentType}</td>

 <td>

 {req.status}

 </td>

 <td>

 <button

 className="review-btn"

 onClick={() => handleReview(req)}

 >

 Review

 </button>

 </td>

 </tr>

)})

</tbody>

</table>

)}

```

```
 {/* Portal for modal */}
 {selectedRequest &&
 ReactDOM.createPortal(
 <div className="modal-overlay" onClick={closeModal}>
 <div className="modal" onClick={(e) => e.stopPropagation()}>
 <header className="modal-header">
 <h4>Review Request</h4>
 <button className="close-btn" onClick={closeModal}>✕</button>
 </header>
 <div className="modal-body">
 <p>Resident: {selectedRequest.name}</p>
 <p>Type: {selectedRequest.type}</p>
 <p>Status: {selectedRequest.status}</p>
 </div>
 <footer className="modal-footer">
 <button
 className="approve-btn"
 onClick={() => updateStatus("Approved")}
 >
  Approve
 </button>
 <button
 className="reject-btn"
 onClick={() => updateStatus("Rejected")}
 >
```



✖ Reject

```
</button>
```

```
</footer>
```

```
</div>
```

```
</div>,
```

```
document.body
```

```
}}
```

```
</div>
```

```
);
```

```
};
```

```
export default DocumentQueue;
```

```
```
```

```
## frontend\src\components\dashboard\IncidentQueue.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
```

```
import { api, endpoints } from "../../services/api";
```

```
import { useUser } from "../../context/UserContext";
```

```
import "../../styles/dashboard/incident-queue.css";
```

```
const IncidentQueue = () => {
```

```
 const { can } = useUser();
```

```
 const [incidents, setIncidents] = useState([]);
```

```
const [loading, setLoading] = useState(true);

const [error, setError] = useState(null);

useEffect(() => {

 const fetchIncidents = async () => {

 if (!can("viewIncidents")) {

 setIncidents([]);

 setLoading(false);

 return;

 }

 try {

 const { data } = await api.get(endpoints.incidents, {

 params: { status: "pending" },

 });

 setIncidents(data || []);

 setError(null);

 } catch (err) {

 console.error(" ❌ Failed to load incidents:", err);

 setError("Failed to load incidents.");

 setIncidents([]);

 } finally {

 setLoading(false);

 }

 };

});
```

```
fetchIncidents();
}, [can]);

const renderContent = () => {
 if (loading) return <p>Loading incidents...</p>;
 if (error) return <p className="error">{error}</p>;
 if (incidents.length === 0) return <p>No incidents available.</p>;

 return (
 <table className="incident-table" aria-label="Incident Queue Table">
 <thead>
 <tr>
 <th>Type</th>
 <th>Description</th>
 <th>Location</th>
 <th>Reported By</th>
 <th>Status</th>
 <th>Timestamp</th>
 </tr>
 </thead>
 <tbody>
 {incidents.map(
 ({
 id,
 type,
 description,
```

```

location,
reported_by_name,
status,
timestamp,
}) => (
 <tr key={id}>
 <td>{type}</td>
 <td>{description}</td>
 <td>{location}</td>
 <td>{reported_by_name || "—"}</td>
 <td>
 <span
 className={`status-badge ${
 status?.toLowerCase() === "escalated"
 ? "escalated"
 : status?.toLowerCase()
 }`}
 >
 {status === "escalated" ? "Escalated" : status}

 </td>
 <td>
 {timestamp ? new Date(timestamp).toLocaleString() : "—"}
 </td>
 </tr>
)

```

```

 })
 </tbody>
</table>

);

};

return (
 <div className="incident-queue" aria-busy={loading} aria-live="polite">
 <h3> 🚨 Incident Queue ({incidents.length} pending)</h3>
 {renderContent()}
 </div>
);
};

```

```
export default IncidentQueue;
```

```
`,`,`,
```

```
frontend\src\components\dashboard\RegistryAudit.js
```

```
` `` javascript
```

```

import { useEffect, useState } from "react";
import { getCountFromServer, collection } from "firebase/firestore";
import { db } from "../../services/firebase";
import { useUser } from "../../context/UserContext";

```

```
import { CATEGORIES, CATEGORY_VARIANTS, COLLECTION_PERMISSIONS } from
"../../config/roles";
```

```
import "../../styles/dashboard/registry-audit.css";
```

```
const RegistryAudit = () => {
```

```
 const { can } = useUser();
```

```
 const [stats, setStats] = useState([]);
```

```
 const [loading, setLoading] = useState(true);
```

```
 useEffect(() => {
```

```
 let cancelled = false;
```

```
 const fetchRegistryStats = async () => {
```

```
 const now = new Date().toLocaleDateString("en-US", {
```

```
 month: "short",
```

```
 day: "numeric",
```

```
 year: "numeric",
```

```
 });
```

```
 const results = await Promise.all(
```

```
 Object.entries(CATEGORIES).map(async ([key, label]) => {
```

```
 const permissions = COLLECTION_PERMISSIONS[key];
```

```
 //  FIX: allow access if ANY permission matches
```

```
 const hasPermission =
```

```
 !permissions || permissions.some((perm) => can(perm));
```

```

 if (!hasPermission) {
 return { category: label, total: "N/A", lastUpdated: now };
 }

 try {
 const snap = await getCountFromServer(collection(db, key));
 return { category: label, total: snap.data().count, lastUpdated: now };
 } catch (err) {
 console.warn(` ⚠️ Error fetching ${key}:`, err.message);
 return { category: label, total: "N/A", lastUpdated: now };
 }
 })
);

if (!cancelled) {
 setStats(results);
 setLoading(false);
}

};

fetchRegistryStats();

return () => {
 cancelled = true;
};

}, [can]);

```

```

return (
 <section
 className="registry-audit"
 aria-labelledby="registry-audit-title"
 aria-busy={loading}
 aria-live="polite"
 >
 <h3 id="registry-audit-title"> 📋 Registry Audit</h3>
 {loading ? (
 <p>Loading registry stats...</p>
) : (

 {stats.map((entry, index) => {
 const variant = CATEGORY_VARIANTS[entry.category] || "neutral";
 return (
 <li key={index} className={`audit-entry ${variant}`}>
 {entry.category}: {entry.total} entries

 <small>Last updated: {entry.lastUpdated}</small>

);
 })}

)}
 </section>
);

```



```
};
```

```
export default RegistryAudit;
```

```
```\n
```

```
## frontend\\src\\components\\dashboard\\RegistryOverview.js
```

```
```javascript
```

```
import React, { useEffect, useState } from "react";
import { collection, getCountFromServer } from "firebase/firestore";
import { db } from "../../services/firebase";
import DashboardCard from "../DashboardCard";
import { useUser } from "../../context/UserContext";
import { COLLECTION_PERMISSIONS } from "../../config/roles";
import "../../styles/dashboard/registry-overview.css";
```

```
// 🗝 Static registry keys (avoid lint warning)
```

```
const REGISTRY_KEYS = ["residents", "businesses", "youth"];
```

```
const RegistryOverview = () => {
 const { can } = useUser();
 const [counts, setCounts] = useState({
 residents: "N/A",
 businesses: "N/A",
 youth: "N/A",
```

```

});

const [loading, setLoading] = useState(true);

useEffect(() => {

 const fetchCounts = async () => {

 const results = {};

 for (const key of REGISTRY_KEYS) {

 const permission = COLLECTION_PERMISSIONS[key];

 if (!permission || !can(permission)) {

 results[key] = "N/A";

 continue;

 }

 try {

 const snap = await getCountFromServer(collection(db, key));

 results[key] = snap.data().count;

 } catch (err) {

 console.warn(` ⚠ Cannot access ${key}:`, err.message);

 results[key] = "N/A";

 }

 }

 setCounts((prev) => ({ ...prev, ...results }));

 setLoading(false);

 };

```

```
fetchCounts();
}, [can]));
```

```
const registryData = [
 { label: "Resident Registry", value: counts.residents, variant: "accent", icon: "👥" },
 { label: "Business Registry", value: counts.businesses, variant: "success", icon: "💼" },
 { label: "Youth Registry", value: counts.youth, variant: "youth", icon: "👦" },
];
```

```
return (
 <section className="registry-overview" aria-busy={loading} aria-live="polite">
 <h3>📋 Registry Overview</h3>
 <div className="registry-grid">
 {loading ? (
 <p>Loading registry data...</p>
) : (
 registryData.map(({ label, value, variant, icon }, index) => (
 <DashboardCard
 key={index}
 label={label}
 value={value}
 variant={variant}
 icon={icon}
 />
))
)
 </div>
 </section>
```

```

 })
 </div>
</section>

);
};

export default RegistryOverview;

` ``

frontend\src\components\dashboard\SummaryCards.js

` `` javascript
import { useEffect, useState } from "react";
import { getCountFromServer, collection, getDocs, query, where } from "firebase/firestore";
import { db } from "../../services/firebase";
import DashboardCard from "../DashboardCard";
import { ALL_STATS, ROLE_COLLECTIONS } from "../../config/roles";
import { useUser } from "../../context/UserContext";
import "../../styles/dashboard/summary-card.css";

const SummaryCards = () => {
 const { role } = useUser();

 const [stats, setStats] = useState([]);
 const [loading, setLoading] = useState(true);

```

```
useEffect(() => {
 let cancelled = false;

 const parseTimestamp = (value) => {
 if (!value) return null;
 if (value?.toDate && typeof value.toDate === "function") {
 return value.toDate();
 }
 const date = new Date(value);
 return Number.isNaN(date.getTime()) ? null : date;
 };
```

```
 const startOfToday = () => {
 const now = new Date();
 return new Date(now.getFullYear(), now.getMonth(), now.getDate());
 };
```

```
 const getLoginRecordsCount = async () => {
 const loginQuery = query(collection(db, "logins"), where("timestamp", ">=",
startOfToday()));
 const snapshot = await getDocs(loginQuery);
 return snapshot.docs.reduce((total, item) => {
 const data = item.data() || {};
 const count = Number(data.count);
 const timestamp = parseTimestamp(data.timestamp);
 if (!timestamp) {
```

```
 return total;
 }
 return total + (Number.isFinite(count) && count > 0 ? count : 1);
}, 0);
};
```

```
const parseAmount = (value) => {
 const num = Number(value);
 if (Number.isFinite(num)) return num;
```

```
 const cleaned = String(value ?? "").replace(/^[^d.-]/g, "");
 const parsed = Number(cleaned);
 return Number.isFinite(parsed) ? parsed : 0;
};
```

```
const getCollectionsAmount = async () => {
 const paidPaymentsQuery = query(collection(db, "payments"), where("status", "==",
"paid"));

 const snapshot = await getDocs(paidPaymentsQuery);

 const today = startOfDayToday();

 const totalAmount = snapshot.docs.reduce((sum, item) => {
 const data = item.data() || {};

 const paidDate = parseTimestamp(data.datePaid || data.paymentDate ||
data.timestamp || data.createdAt);

 if (!paidDate || paidDate < today) {
 return sum;
```

```
}

const amount = parseAmount(data.amount);

return sum + amount;

, 0);
```

```

return new Intl.NumberFormat("en-PH", {

 style: "currency",

 currency: "PHP",

}).format(totalAmount);

};
```

```

const safeQuery = async (key) => {

 try {

 if (key === "logins") {

 const value = await getLoginRecordsCount();

 return { key, value };

 }

 }
```

```

 if (key === "collections") {

 const value = await getCollectionsAmount();

 return { key, value };

 }
```

```


const snap = await getCountFromServer(collection(db, key));

return { key, value: snap.data().count };

} catch (err) {
```

```
 console.warn(` ⚠️ ${role} cannot access ${key}:`, err.message);

 return { key, value: "N/A" };
 }
};
```

```
const start = async () => {

 // ✅ Only include collections allowed for this role

 const allowedKeys = ROLE_COLLECTIONS[role] || [];

 const results = await Promise.all(allowedKeys.map(safeQuery));

 if (!cancelled) {

 setStats(

 results.map(({ key, value }) => {

 const stat = ALL_STATS[key];

 if (!stat) {

 console.warn(` ⚠️ Missing ALL_STATS entry for key: ${key}`);

 return { label: key, value, variant: "neutral", icon: " ? " };

 }

 return {

 label: stat.label,

 value,

 variant: value === "N/A" ? "neutral" : stat.variant,

 icon: stat.icon,

 };

 })

)
```



```
);
 setLoading(false);
}
};
```

```
start();
return () => {
 cancelled = true;
};
}, [role]);
```

```
return (
 <section className="summary-cards" aria-busy={loading} aria-live="polite">
 {loading ? (
 <p>Loading summary...</p>
) : stats.length === 0 ? (
 <p>No accessible data for this role.</p>
) : (
 stats.map(({ label, value, variant, icon }, index) => (
 <DashboardCard
 key={index}
 label={label}
 value={value}
 variant={variant}
 icon={icon}
 />
)
)
)
);
```

```
))
 })
</section>

);
};
```

```
export default SummaryCards;
```

```
` ``
```

```
frontend\src\components\document\AuditTable.js
```

```
` `` ` javascript
```

```
import { useEffect, useState } from "react";
import { api } from "../../services/api";
import "../../styles/dashboard/audit-table.css";
```

```
const AuditTable = () => {
 const [logs, setLogs] = useState([]);
 const [loading, setLoading] = useState(true);
 const [status, setStatus] = useState("");
```

```
 const fetchLogs = async () => {
 setLoading(true);
 setStatus("Fetching audit logs...");
 try {
```

```

const response = await api.get("/api/document_audit");
setLogs(response.data || []);
setStatus("");
} catch (err) {
const errorMsg = err.response?.data?.detail || err.message;
console.error(" ❌ Error fetching audit logs:", errorMsg);
setStatus(" ❌ Failed to load audit logs.");
} finally {
setLoading(false);
}
};

```

```

useEffect(() => {
fetchLogs();
}, []);

```

```

return (
<div className="audit-table">
<h3> 📄 Document Audit Log</h3>
{status && <p className="status-message">{status}</p>}
{loading ? (
<p>Loading audit logs...</p>
) : logs.length === 0 ? (
<p>No audit records found.</p>
) : (
<table>

```

```

<thead>

 <tr>

 <th>Document</th>

 <th>Action</th>

 <th>Performed By</th>

 <th>Resident</th>

 <th>Date</th>

 </tr>

</thead>

<tbody>

 {logs.map((log) => (

 <tr key={log.id || `${log.doc_id}-${log.timestamp}`} >

 <td>{log.document_type || "N/A"}</td>

 <td>{log.action}</td>

 <td>{log.performed_by}</td>

 <td>{log.resident_name || log.resident_id}</td>

 <td>

 {log.timestamp

 ? new Date(log.timestamp).toLocaleString()

 : "N/A"}

 </td>

 </tr>

)

)}

</tbody>

</table>

)}

```

```
</div>
```

```
);
```

```
};
```

```
export default AuditTable;
```

```
````
```

```
## frontend\src\components\document\DocumentSummaryCards.js
```

```
````javascript
```

```
import { useEffect, useState } from "react";
```

```
import DashboardCard from "../dashboard/DashboardCard";
```

```
import { DashboardAPI } from "../../services/api";
```

```
import "../../styles/dashboard/summary-card.css";
```

```
const DOCUMENT_CATEGORIES = [
```

```
{ type: "Resident Certificate", label: "Residency", icon: "🏠", variant: "accent" },
```

```
{ type: "Barangay Clearance", label: "Barangay Clearance", icon: "✅", variant: "success" },
```

```
{ type: "Indigency Certificate", label: "Indigency", icon: "💛", variant: "info" },
```

```
{ type: "Good Moral Certificate", label: "Good Moral", icon: "☀️", variant: "neutral" },
```

```
{ type: "Business Clearance", label: "Business Clearance", icon: "👛", variant: "warning" },
```

```
{ type: "Activity Permit", label: "Activity Permit", icon: "🎨", variant: "accent" },
```

```
{ type: "Blotter Report", label: "Blotter Report", icon: "⚠️", variant: "danger" },
```

```
{ type: "Health Certificate", label: "Health Certificate", icon: "💊", variant: "success" },
```

```
{ type: "Barangay ID", label: "Barangay ID", icon: "🏠", variant: "info" },
];
```

```
const DocumentSummaryCards = () => {
 const [stats, setStats] = useState([]);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 let cancelled = false;

 const fetchStats = async () => {
 try {
 const results = await Promise.all(
 DOCUMENT_CATEGORIES.map(async (cat) => {
 try {
 const data = await DashboardAPI.issuedCount(cat.type);
 return { ...cat, value: data.issuedCount || 0 };
 } catch (err) {
 console.warn(` ⚠️ Error fetching issued count for ${cat.type}:`, err.message);
 return { ...cat, value: 0 };
 }
 })
);
 } catch (err) {
 console.warn(` ⚠️ Error fetching stats:`, err.message);
 }
 };

 if (!cancelled) {
 fetchStats();
 setLoading(false);
 }
 });
}
```

```

 }
 } catch (err) {
 if (!cancelled) {
 setStats([]);
 setLoading(false);
 }
 }
};

```

```

fetchStats();
return () => {
 cancelled = true;
};
}, []);

```

```

return (
 <section className="summary-cards" aria-busy={loading} aria-live="polite">
 <h2> 📄 Documents Issued per Category</h2>
 {loading ? (
 <p>Loading document stats...</p>
) : (
 stats.map(({ label, value, variant, icon }, index) => (
 <DashboardCard
 key={index}
 label={label}
 value={value}

```

```
 variant={variant}
 icon={icon}
 />
))
 }}
</section>

);
};
```

```
export default DocumentSummaryCards;
```

```
````
```

```
## frontend\src\components\document\SearchFilters.js
```

```
```javascript
```

```
import { useState } from "react";
```

```
import "../styles/dashboard/search-filters.css";
```

```
const SearchFilters = ({ onSearch }) => {
```

```
 const [documentType, setDocumentType] = useState("");
```

```
 const [issuedBy, setIssuedBy] = useState("");
```

```
 const [fromDate, setFromDate] = useState("");
```

```
 const [toDate, setToDate] = useState("");
```

```
 const handleSubmit = (e) => {
```



```
e.preventDefault();
```

```
// Build filters object only with non-empty values
```

```
const filters = {};
```

```
if (documentType) filters.documentType = documentType;
```

```
if (issuedBy) filters.issuedBy = issuedBy;
```

```
if (fromDate) filters.fromDate = fromDate;
```

```
if (toDate) filters.toDate = toDate;
```

```
onSearch(filters);
```

```
};
```

```
const handleReset = () => {
```

```
 setDocumentType("");
```

```
 setIssuedBy("");
```

```
 setFromDate("");
```

```
 setToDate("");
```

```
 onSearch({});
```

```
};
```

```
return (
```

```
<form className="search-filters" onSubmit={handleSubmit}>
```

```
<h3>  Filter Documents</h3>
```

```
<div className="filter-row">
```

```
<label htmlFor="type">Type:</label>
```

```
<select
 id="type"
 value={documentType}
 onChange={(e) => setDocumentType(e.target.value)}
>
 <option value="">All</option>
 <option value="Resident Certificate">Residency</option>
 <option value="Barangay Clearance">Clearance</option>
 <option value="Indigency Certificate">Indigency</option>
 <option value="Good Moral Certificate">Good Moral</option>
 <option value="Business Clearance">Business Clearance</option>
 <option value="Activity Permit">Activity Permit</option>
 <option value="Blotter Report">Blotter Report</option>
 <option value="Health Certificate">Health Certificate</option>
 <option value="Barangay ID">Barangay ID</option>
</select>
</div>
```

```
<div className="filter-row">
 <label htmlFor="staff">Issued By:</label>
 <input
 id="staff"
 type="text"
 value={issuedBy}
 onChange={(e) => setIssuedBy(e.target.value)}
 placeholder="Staff/Secretary name"
```

```
/>
```

```
</div>
```

```
<div className="filter-row">
```

```
 <label htmlFor="from-date">From:</label>
```

```
 <input
```

```
 id="from-date"
```

```
 type="date"
```

```
 value={fromDate}
```

```
 onChange={(e) => setFromDate(e.target.value)}
```

```
 />
```

```
</div>
```

```
<div className="filter-row">
```

```
 <label htmlFor="to-date">To:</label>
```

```
 <input
```

```
 id="to-date"
```

```
 type="date"
```

```
 value={toDate}
```

```
 onChange={(e) => setToDate(e.target.value)}
```

```
 />
```

```
</div>
```

```
<div className="filter-actions">
```

```
 <button type="submit">Apply Filters</button>
```

```
 <button type="button" onClick={handleReset} className="reset-btn">
```

```
 Reset
 </button>
 </div>
 </form>
);
};
```

```
export default SearchFilters;
```

```
` ``
```

```
frontend\src\components\forms\add-new-fee-form.css
```

```
` `` `css
```

```
.add-fee-form {
 margin-bottom: 2rem;
 padding: 1.5rem;
 border: 1px solid var(--color-border);
 border-radius: 8px;
 background: var(--color-card-bg);
 box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}
```

```
.add-fee-form h2 {
 font-size: 1.4rem;
 margin-bottom: 1rem;
```

```
color: var(--color-success);
}
```

```
/* Labels stack neatly above inputs */
```

```
.add-fee-form label {
 display: block;
 margin-bottom: 0.75rem;
 font-weight: 600;
 color: var(--color-text);
}
```

```
.add-fee-form select,
.add-fee-form input[type="text"],
.add-fee-form input[type="number"] {
 display: block;
 width: 100%;
 max-width: 320px;
 margin-top: 0.25rem;
 margin-bottom: 1rem;
 padding: 0.5rem;
 border: 1px solid #ccc;
 border-radius: 6px;
 font-size: 0.95rem;
 transition: border-color 0.2s ease;
}
```

```
.add-fee-form select:focus,
.add-fee-form input:focus {
 outline: none;
 border-color: #2980b9;
}
```

```
/* Save button */
```

```
.add-fee-form button {
 padding: 0.6rem 1.2rem;
 background: #27ae60;
 color: #fff;
 border: none;
 border-radius: 6px;
 font-size: 0.95rem;
 cursor: pointer;
 transition: background 0.2s ease;
}
```

```
.add-fee-form button:hover {
 background: #219150;
}
```

```
` ``
```

```
frontend\src\components\forms\AddNewFeeForm.js
```

```
` `` `javascript
```

```
import { useState } from "react";
```

```
import axios from "axios";
```

```
import { auth } from "../../services/firebase";
```

```
import "./add-new-fee-form.css";
```

```
import { validateFeePayload } from "../../utils/validation";
```

```
import modesConfig from "../../config/modesConfig";
```

```
export default function AddNewFeeForm({ onAdded, miscFees = [] }) {
```

```
 const [mode, setMode] = useState("document");
```

```
 const [formData, setFormData] = useState({ fee: 0, enabled: true });
```

```
 const handleChange = (field, value) => {
```

```
 setFormData(prev => ({ ...prev, [field]: value }));
```

```
 };
```

```
 const buildPayload = () => {
```

```
 const payload = { ...formData };
```

```
 // Attach miscType reference only if enabled
```

```
 if ((mode === "document" || mode === "business") && payload.enabled) {
```

```
 payload.miscType = payload.miscType ?? null;
```

```
 } else {
```

```
 delete payload.miscType; // ensure it's not sent when disabled
```

```
 }
```

```
 return payload;
 };
```

```
const handleSubmit = async () => {
 try {
 const payload = buildPayload();

 const { valid, message } = validateFeePayload(mode, payload);
 if (!valid) {
 alert(message);
 return;
 }
 }
```

```
 console.log("Submitting payload:", payload);
```

```
 const token = auth.currentUser ? await auth.currentUser.getIdToken() : null;
```

```
 await axios.post(modesConfig[mode].endpoint, payload, {
 headers: {
 "Content-Type": "application/json",
 ...(token && { Authorization: `Bearer ${token}` }),
 },
 });
```

```
 alert("✅ Fee saved!");
 onAdded?.();
```



```
 setFormData({});
 } catch (err) {
 console.error(" ❌ Fee submission failed:", err);
 alert(" ❌ Failed to save fee");
 }
};
```

```
const renderField = (field) => {
 const value =
 formData[field.name] ??
 (field.type === "checkbox" ? false : field.type === "number" ? 0 : "");
```

```
 switch (field.type) {
 case "checkbox":
 return (
 <label key={field.name}>
 {field.label}:
 <input
 type="checkbox"
 checked={!!value}
 onChange={e => handleChange(field.name, e.target.checked)}
 />
 </label>
);
```

```
 case "number":
```

```

return (
 <label key={field.name}>
 {field.label}:
 <input
 type="number"
 min="0"
 value={value}
 onChange={e => handleChange(field.name, Number(e.target.value))}
 />
 </label>
);

```

```

case "select":
 // Special handling for miscType dropdown
 if (field.name === "miscType") {
 // ✅ Only show miscType if enabled and mode is document/business
 if ((mode === "document" || mode === "business") && formData.enabled) {
 return (
 <label key={field.name}>
 {field.label}:
 <select
 value={value}
 onChange={e => handleChange(field.name, e.target.value)}
 >
 <option value="">-- choose --</option>
 {miscFees.map(m => (

```

```

 <option key={m.id} value={m.miscType}>
 {m.miscType} {m.fee}
 </option>
)}
 </select>
</label>

);
}

return null; // hide miscType when not enabled
}

```

```

// Fallback for other select fields
return (
 <label key={field.name}>
 {field.label}:
 <select
 value={value}
 onChange={e => handleChange(field.name, e.target.value)}
 >
 <option value="">-- choose --</option>
 {(field.options || []).map(opt => (
 <option key={opt.value} value={opt.value}>
 {opt.label}
 </option>
))}
 </select>
)

```

```

 </label>

);

 default:

 return (

 <label key={field.name}>

 {field.label}:

 <input

 type={field.type}

 value={value}

 onChange={e => handleChange(field.name, e.target.value)}

 />

 </label>

);

}

};

return (

 <div className="add-fee-form">

 <h2>Add New Fee</h2>

 <div>

 <label>

 Mode:

 <select value={mode} onChange={e => setMode(e.target.value)}>

 {Object.keys(modesConfig).map(m => (

 <option key={m} value={m}>{m}</option>

```

```
)}}
 </select>
 </label>
</div>
```

```
 { /* Dynamic fields */
 {modesConfig[mode].fields.map(renderField)}

 <button onClick={handleSubmit}>Save</button>
 </div>
);
}
```

```
...`
```

```
frontend\src\components\forms\business-form.css
```

```
```.css  
  
.business-form {  
  max-width: 600px;  
  margin: auto;  
  padding: 1rem;  
  background: #fff;  
  border-radius: 8px;  
  box-shadow: 0 0 6px rgba(0,0,0,0.1);  
}
```

```
.business-form label {  
  display: block;  
  margin-bottom: 1rem;  
  font-weight: bold;  
}
```

```
.business-form input,  
.business-form select {  
  width: 100%;  
  padding: 0.5rem;  
  margin-top: 0.25rem;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
}
```

```
.qr-wrapper {  
  margin-top: 1rem;  
  text-align: center;  
}
```

```
/* Wrapper for each uploaded file */
```

```
.file-preview-wrapper {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;
```

```
padding: 6px 10px;
margin: 4px 0;
border: 1px solid #ddd;
border-radius: 4px;
background-color: #fafafa;
}
```

```
/* File name text */
```

```
.file-name {
  font-size: 0.95rem;
  color: #333;
  font-weight: 500;
  margin: 0;
}
```

```
/* Status badge */
```

```
.status-badge {
  padding: 2px 8px;
  border-radius: 12px;
  font-size: 0.8rem;
  font-weight: 600;
  text-transform: uppercase;
}
```

```
/* Variants */
```

```
.status-required {
```

```
background-color: #e0f7e9;
color: #2e7d32; /* green */
border: 1px solid #2e7d32;
}
```

```
.status-optional {
background-color: #fff3e0;
color: #ef6c00; /* orange */
border: 1px solid #ef6c00;
}
```

```
.status-missing {
background-color: #ffebee;
color: #c62828; /* red */
border: 1px solid #c62828;
}
```

```
```
```

```
frontend\src\components\forms\BusinessForm.js
```

```
```javascript
```

```
// src/components/forms/BusinessForm.jsx
```

```
import { useState } from 'react';
```

```
import { useForm } from 'react-hook-form';
```

```
import { toast } from 'react-toastify';
```



```
import { useNavigate } from 'react-router-dom';
import { ref, uploadBytes, getDownloadURL } from 'firebase/storage';
import { collection, addDoc } from 'firebase/firestore';
import imageCompression from "browser-image-compression";
import { storage, db } from '../services/firebase';
import { useUser } from "../..context/UserContext";
import { PARANAQUE } from "../..data/locations";
import { usePublicFees } from "../..hooks/usePublicFees";
import { NotificationsAPI } from "../..services/api";
import './business-form.css';
```

```
const BusinessForm = ({ onBusinessAdded, onCancel }) => {
  const { register, handleSubmit, watch, reset, trigger, formState: { errors } } = useForm();
  const { userInfo: user } = useUser();
  const navigate = useNavigate();
```

```
  const [step, setStep] = useState(1);
  const [isSubmitting, setIsSubmitting] = useState(false);
```

```
//  use public fees
```

```
const { businessTypes: businessFees, loading, error } = usePublicFees();
```

```
const [documents, setDocuments] = useState({
  validId: null,
  proofOfAddress: null,
```

```

    dtiCert: null,
    businessLogo: null,
  });
const [previews, setPreviews] = useState({});

const sanitize = (str) => str.replace(/[^a-zA-Z0-9_.-]/g, "_");

if (!user) return <div className="loading-user">Loading your profile...</div>;
if (loading) return <div className="loading-user">Loading business types...</div>;
if (error) toast.error(" ⚠️ Could not load business types.");

// -----
// FILE HANDLING + COMPRESSION
// -----

const handleDocumentChange = (e, field) => {
  const selected = e.target.files[0];
  if (!selected) return;
  if (selected.size > 5 * 1024 * 1024) {
    toast.error(" ❌ File must be under 5MB.");
    return;
  }
  setDocuments(prev => ({ ...prev, [field]: selected }));
  setPreviews(prev => ({ ...prev, [field]: URL.createObjectURL(selected) }));
};

const compressIfNeeded = async (file) => {

```

```
if (!file.type.startsWith("image/")) return file;

try {

  return await imageCompression(file, {

    maxSizeMB: 1,

    maxWidthOrHeight: 1280,

    useWebWorker: true,

  });

} catch {

  return file;

}

};
```

```
const uploadAllDocuments = async (businessName) => {

  const uploaded = {};

  const safeBusinessName = sanitize(businessName);
```

```
  for (const key of Object.keys(documents)) {

    const file = documents[key];

    if (!file) continue;
```

```
    const compressed = await compressIfNeeded(file);

    const safeFileName = sanitize(file.name);

    const timestamp = Date.now();
```

```
    const path =

    `businesses/${safeBusinessName}/${key}_${timestamp}_${safeFileName}`;
```

```
const fileRef = ref(storage, path);
```

```
await uploadBytes(fileRef, compressed);
```

```
const url = await getDownloadURL(fileRef);
```

```
//  Save both URL and path
```

```
uploaded[key] = { url, path };
```

```
}
```

```
return uploaded;
```

```
};
```

```
// -----
```

```
// ID GENERATORS
```

```
// -----
```

```
const generateBusinessId = (barangay) => {
```

```
  const year = new Date().getFullYear();
```

```
  const random = Math.floor(1000 + Math.random() * 9000);
```

```
  return `BIZ-${barangay?.toUpperCase()}-${year}-${random}`;
```

```
};
```

```
const generatePermitNumber = (barangay) => {
```

```
  const year = new Date().getFullYear();
```

```
  const random = Math.floor(1000 + Math.random() * 9000);
```

```
  return `PERMIT-${year}-${barangay?.toUpperCase()}-${random}`;
```

```
};
```

```
// -----  
  
// SUBMIT HANDLER  
  
// -----  
  
const onSubmit = async (data) => {  
  setIsSubmitting(true);  
  
  try {  
    const uploadedDocs = await uploadAllDocuments(data.businessName);  
  
    const fullAddress = [data.street, data.barangay, data.city, data.province]  
      .filter(Boolean)  
      .join(", ");  
  
    // 🟡 Resolve fee for selected business type  
    const feeInfo = businessFees.find(f => f.businessType === data.businessType);  
    const amount = feeInfo ? feeInfo.registrationTotal : 0;  
    // or annualTotal depending on your workflow  
  
    const payload = {  
      ...data,  
      address: fullAddress,  
      ownerUid: user.uid,  
      ownerName: user.fullName,  
      contactNumber: user.contactNumber,  
      email: user.email,  
      documents: uploadedDocs,
```

```

status: "pending_evaluation",    // ✅ matches your workflow
amount,                        // ✅ include fee amount
businessId: generateBusinessId(data.barangay),
permitNumber: generatePermitNumber(data.barangay),
submittedAt: new Date().toISOString(),
};

await addDoc(collection(db, "businesses"), payload);
await NotificationsAPI.createBusinessSubmitted(
  user?.fullName || user?.name || user?.email || "Resident",
  data.businessName
);
toast.success("🔥 Application submitted for staff evaluation.");

reset();

setDocuments({ validId: null, proofOfAddress: null, dtiCert: null, businessLogo: null });
setPreviews({});
onBusinessAdded?.();
handleCancel();
} catch (error) {
  console.error("❌ Error registering business:", error);
  toast.error("❌ Failed to register business.");
} finally {
  setIsSubmitting(false);
}

```

```
};
```

```
const handleCancel = () => {  
  if (onCancel) onCancel();  
  else navigate("/businesses/my");  
};
```

```
// -----
```

```
// STEP 1 — BUSINESS DETAILS
```

```
// -----
```

```
const renderStep1 = () => {  
  const street = watch("street");  
  const barangay = watch("barangay");  
  const city = watch("city");  
  const province = watch("province");
```

```
  const fullAddress = [street, barangay, city, province].filter(Boolean).join(", ");
```

```
  return (  
    <div className="form-step">
```

```
      <h2>🏠 Business Details</h2>
```

```
      <label htmlFor="businessName">Business Name
```

```
        <input id="businessName" {...register("businessName", { required: "Business name is  
required" })} />
```

```
      {errors.businessName && <span className="error-  
text">{errors.businessName.message}</span>}  
  
    </label>
```

```
    <label htmlFor="businessType">Business Type  
  
      <select id="businessType" {...register("businessType", { required: "Business type is  
required" })}>  
  
        <option value="">Select Type</option>  
  
        {businessFees.map(bt => (  
  
          <option key={bt.id} value={bt.businessType}>  
  
            {bt.businessType} — ₱{bt.registrationTotal}  
  
          </option>  
  
        ))}  
  
      </select>  
  
      {errors.businessType && <span className="error-  
text">{errors.businessType.message}</span>}  
  
    </label>
```

```
<fieldset className="address-fieldset">  
  
  <legend>Business Address</legend>
```

```
<label htmlFor="street">Blk# / Lot#, Street  
  
  <input id="street" {...register("street", { required: "Street is required" })} />  
  
  {errors.street && <span className="error-text">{errors.street.message}</span>}  
  
</label>
```

```
<label htmlFor="barangay">Barangay
```



```

<select id="barangay" {...register("barangay", { required: "Barangay is required" })}>
  <option value="">Select Barangay</option>
  {PARANAQUE.barangays.map(brgy => (
    <option key={brgy} value={brgy}>{brgy}</option>
  ))}
</select>

{errors.barangay && <span className="error-
text">{errors.barangay.message}</span>}

</label>

<label htmlFor="city">City
  <input id="city" value={PARANAQUE.city} readOnly {...register("city")} />
</label>

<label htmlFor="province">Province
  <input id="province" value={PARANAQUE.province} readOnly {...register("province")}
/>
</label>
</fieldset>

{/* Hidden full address field */}
<input type="hidden" {...register("address")} value={fullAddress} />

<label htmlFor="registrationDate">Registration Date
  <input id="registrationDate" type="date" {...register("registrationDate", { required:
"Registration date is required" })} />

```

```
      {errors.registrationDate && <span className="error-  
text">{errors.registrationDate.message}</span>}
```

```
</label>
```

```
<button
```

```
  type="button"
```

```
  onClick={async () => {
```

```
    const valid = await trigger([
```

```
      "businessName",
```

```
      "businessType",
```

```
      "barangay",
```

```
      "street",
```

```
      "city",
```

```
      "province",
```

```
      "registrationDate",
```

```
    ]);
```

```
    if (valid) setStep(2);
```

```
    else toast.error(" ⚠ Please fill in all required fields.");
```

```
  }}  
>
```

```
  Next →
```

```
</button>
```

```
</div>
```

```
);
```

```
};
```

```

// -----
// STEP 2 — DOCUMENT SUBMISSION
// -----

const renderStep2 = () => {

  const allRequiredUploaded = documents.validId && documents.proofOfAddress;

  const renderPreview = (file, previewUrl, label) => {

    if (!file || !previewUrl) return null;

    const isPdf = file.name.toLowerCase().endsWith(".pdf");

    return (

      <div className="file-preview-wrapper">

        {isPdf ? (

          <a href={previewUrl} target="_blank" rel="noopener noreferrer">

             View {label} PDF ({file.name})

          </a>

        ) : (

          <img src={previewUrl} alt={`Preview of uploaded ${label}`} className="file-preview"
/>

        )}

        <p className="file-name">{file.name}</p>

      </div>

    );

  };

  return (

    <div className="form-step">

      <h2>  Document Submission</h2>

      <label>Valid ID (Required)

```

```

        <input type="file" accept="image/*,.pdf" onChange={(e) =>
handleDocumentChange(e, "validId")} />

    </label>

    {renderPreview(documents.validId, previews.validId, "Valid ID")}

    <label>Proof of Address (Required)

        <input type="file" accept="image/*,.pdf" onChange={(e) =>
handleDocumentChange(e, "proofOfAddress")} />

    </label>

    {renderPreview(documents.proofOfAddress, previews.proofOfAddress, "Proof of
Address")}

    <label>DTI Certificate (Optional)

        <input type="file" accept="image/*,.pdf" onChange={(e) =>
handleDocumentChange(e, "dtiCert")} />

    </label>

    {renderPreview(documents.dtiCert, previews.dtiCert, "DTI Certificate")}

    <label>Business Logo (Optional)

        <input type="file" accept="image/*,.pdf" onChange={(e) =>
handleDocumentChange(e, "businessLogo")} />

    </label>

    {renderPreview(documents.businessLogo, previews.businessLogo, "Business
Logo")}

<div className="step-buttons">

    <button type="button" onClick={() => setStep(1)}>< Back</button>

    <button

        type="button"

        disabled={!allRequiredUploaded}

        onClick={() => setStep(3)}

```

```
>
```

```
Next →
```

```
</button>
```

```
</div>
```

```
{!allRequiredUploaded && (
```

```
<p className="warning-text">
```

```
⚠ Please upload all required documents to continue.
```

```
</p>
```

```
})
```

```
</div>
```

```
);
```

```
};
```

```
// -----
```

```
// STEP 3 — REVIEW & SUBMIT
```

```
// -----
```

```
const renderStep3 = () => {
```

```
  const data = watch();
```

```
  const fullAddress = data.address ||
```

```
    [data.street, data.barangay, data.city, data.province].filter(Boolean).join(", ");
```

```
  const feeInfo = businessFees.find(f => f.businessType === data.businessType);
```

```
  const amount = feeInfo ? feeInfo.registrationTotal : 0;
```

```
return (  
  <div className="form-step">  
    <h2>✅ Review & Submit</h2>  
  
    <div className="review-box">  
      <h3>Business Details</h3>  
  
      <p><strong>Name:</strong> {data.businessName}</p>  
      <p><strong>Type:</strong> {data.businessType}</p>  
      <p><strong>Barangay:</strong> {data.barangay}</p>  
      <p><strong>Address:</strong> {fullAddress}</p>  
      <p><strong>Registration Date:</strong> {data.registrationDate}</p>  
      <p><strong>Fee Amount:</strong> ₱{amount}</p>  
  
      <hr />  
  
      <h3>Owner Details</h3>  
  
      <p><strong>Owner:</strong> {user?.fullName || "N/A"}</p>  
      <p><strong>Contact:</strong> {user?.contactNumber || "N/A"}</p>  
      <p><strong>Email:</strong> {user?.email || "N/A"}</p>  
    </div>  
  
    <div className="step-buttons">  
      <button type="button" onClick={() => setStep(2)}>← Back</button>  
      <button type="submit" disabled={isSubmitting}>  
        {isSubmitting ? "Submitting..." : "Submit Application"}  
      </button>  
    </div>  
  </div>  
)
```

```

        </div>

    </div>

    );

};

return (
    <>
        <form onSubmit={handleSubmit(onSubmit)} className="business-form">
            {step === 1 && renderStep1()}
            {step === 2 && renderStep2()}
            {step === 3 && renderStep3()}

            <button type="button" onClick={handleCancel} className="cancel-btn">
                Cancel
            </button>
        </form>
    </>

    );

};

export default BusinessForm;

` `` `

```

```

## frontend\src\components\forms\complaint-form.css

```

```
` `` `css

/* complaint-form.css */

.complaint-form {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  max-width: 500px;
  margin: auto;
  padding: 1.5rem;
  background-color: var(--color-bg);
  border-radius: 10px;
}

.complaint-form input,
.complaint-form textarea,
.complaint-form select {
  padding: 0.5rem;
  border-radius: 4px;
  border: 1px solid #ccc;
}

.complaint-form button {
  background-color: #28a745;
  color: white;
  padding: 0.5rem;
  border: none;
  border-radius: 4px;
}
```



```

## frontend\src\components\forms\ComplaintForm.js

```javascript

```
import { useState, useEffect } from "react";
import { api, endpoints } from "../../services/api";
import { db } from "../../services/firebase";
import { collection, addDoc, serverTimestamp } from "firebase/firestore";
import { useUser } from "../../context/UserContext";
import { toast } from "react-toastify";
import { validateComplaintPayload } from "../../helpers/validation";
import "../../complaint-form.css";
```

```
const ComplaintForm = ({ onSubmitSuccess }) => {
  const [residents, setResidents] = useState([]);
  const [formData, setFormData] = useState({
    category: "",
    description: "",
    location: "",
    filed_by: "",
  });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const { userInfo, can, role } = useUser();
```

```

const canFile =

(role === "resident" && can("fileComplaints")) ||

((role === "staff" || role === "admin") && can("fileComplaintsForResidents"));

// ✅ Pre-fill filed_by for residents, load resident list for staff

useEffect(() => {

if (role === "staff" && canFile) {

api.get(endpoints.residents, { params: { limit: 100 } })

.then((res) => {

const normalized = res.data?.results ?? res.data ?? [];

console.log("Residents loaded:", normalized); // 🔍 check if authUid is present

setResidents(Array.isArray(normalized) ? normalized : []);

})

.catch((err) => {

const errorMsg = err.response?.data?.detail || err.message;

console.error("❌ Failed to load residents:", errorMsg);

setError("Failed to load resident list.");

});

}

}, [role, userInfo, canFile]);

const handleChange = (e) => {

setFormData({ ...formData, [e.target.name]: e.target.value });

};

```

```
const handleSubmit = async (e) => {
```

```
  e.preventDefault();
```

```
  setLoading(true);
```

```
  setError(null);
```

```
  let filedByUid = formData.filed_by;
```

```
  if (role === "resident") {
```

```
    filedByUid = userInfo?.uid || "";
```

```
  }
```

```
  const payload = {
```

```
    category: formData.category.trim(),
```

```
    description: formData.description.trim(),
```

```
    location: formData.location.trim(),
```

```
    filed_by: filedByUid,
```

```
  };
```

```
  console.log("Submitting filed_by:", payload.filed_by, "length:", payload.filed_by?.length);
```

```
  //  Use helper
```

```
  const errors = validateComplaintPayload(payload);
```

```
  if (errors.length) {
```

```
    setError(errors);
```

```
    setLoading(false);
```

```
    return;
```

```
}
```

```
try {
```

```
  const { data } = await api.post(endpoints.complaints.base, payload);
```

```
  await addDoc(collection(db, "notifications"), {
```

```
    type: "complaint",
```

```
    refId: data.id,
```

```
    message: `New complaint filed: ${payload.category} at ${payload.location}`,
```

```
    notifyRoles: ["staff"],
```

```
    createdAt: serverTimestamp(),
```

```
    readBy: [],
```

```
  });
```

```
  toast.success("✅ Complaint filed successfully!");
```

```
  onSubmitSuccess?.();
```

```
  setFormData({
```

```
    category: "",
```

```
    description: "",
```

```
    location: "",
```

```
    filed_by: role === "resident" ? userInfo.uid : "",
```

```
  });
```

```
} catch (err) {
```

```
  const raw = err.response?.data;
```

```
  if (Array.isArray(raw)) {
```

```

      setError(raw.map((e) => `${e.loc?.join(".")}: ${e.msg}`));
    } else {
      setError(raw?.detail || err.message);
    }
  } finally {
    setLoading(false);
  }
};

```

```

if (!canFile) {
  return <p>✖ You do not have permission to file complaints.</p>;
}

```

```

return (
  <form className="complaint-form" onSubmit={handleSubmit}>
    <h2>{role === "staff" ? "Log Complaint (on behalf of resident)" : "File a Complaint"}</h2>

```

```

    {error && (
      <div className="error">
        {Array.isArray(error)
          ? error.map((e, i) => <p key={i}>{e}</p>)
          : <p>{error}</p>}
      </div>
    )}

```

```

    <label htmlFor="category">Category</label>

```

```
<select
  id="category"
  name="category"
  value={formData.category}
  onChange={handleChange}
  required
>
  <option value="">Select category</option>
  <option value="Noise">Noise</option>
  <option value="Service">Service</option>
  <option value="Neighbor">Neighbor</option>
  <option value="Other">Other</option>
</select>
```

```
<label htmlFor="description">Description</label>
<textarea
  id="description"
  name="description"
  value={formData.description}
  onChange={handleChange}
  required
/>
```

```
<label htmlFor="location">Location</label>
<input
  id="location"
```

```

name="location"
value={formData.location}
onChange={handleChange}
required
/>

{role === "resident" ? (
  // Resident auto-filing
  <input type="hidden" name="filed_by" value={formData.filed_by} />
) : (
  // Staff selects resident
  <>
    <label htmlFor="filed_by">Filed By (Resident)</label>
    <select
      id="filed_by"
      name="filed_by"
      value={formData.filed_by}
      onChange={handleChange}
      required
    >
      <option value="">Select a resident</option>
      {residents.map((r) => (
        <option key={r.id} value={r.id}>
          {r.fullName || "Unnamed"} — Barangay {r.address?.barangay || "N/A"}
        </option>
      ))}
    </select>
  </>
)
}

```

```
</select>
```

```
</>
```

```
}}
```

```
<button type="submit" disabled={loading}>
```

```
{loading ? "Submitting..." : "Submit"}
```

```
</button>
```

```
</form>
```

```
);
```

```
};
```

```
export default ComplaintForm;
```

```
...
```

```
## frontend\src\components\forms\DisbursementForm.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";
```

```
import { AccountsAPI, DisbursementsAPI } from "../../services/api";
```

```
import { getAuth } from "firebase/auth";
```

```
import "../../styles/treasurer/disbursement-form.css";
```

```
function DisbursementForm({ onClose }) {
```

```
 const auth = getAuth();
```

```
 const currentUser = auth.currentUser;
```



```
const initialState = {
 category: "",
 amount: "",
 date: "",
 recipient: "",
 processedBy: "",
 status: "pending",
};
```

```
const [form, setForm] = useState(initialFormState);
const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(false);
```

```
//  Load accounts for "Salaries" category
```

```
useEffect(() => {
 const fetchUsers = async () => {
 try {
 const accounts = await AccountsAPI.list({ limit: 50 });
 setUsers(accounts);

 if (currentUser) {
 const me = accounts.find(acc => acc.uid === currentUser.uid);
 if (me) {
 setForm(prev => ({ ...prev, processedBy: me.full_name }));
 } else {
```

```
 setForm(prev => ({ ...prev, processedBy: currentUser.email }));
 }
 }
} catch (err) {
 console.error("Failed to fetch accounts", err);
}
};

fetchUsers();
}, [currentUser]);
```

```
const handleChange = (e) => {
 const { name, value } = e.target;
 setForm((prev) => ({ ...prev, [name]: value }));
};
```

```
const handleSubmit = async (e) => {
 e.preventDefault();
 setLoading(true);
```

```
const payload = {
 ...form,
 amount: Number(form.amount),
 date: new Date(form.date),
 referenceNo: `DISB-${Date.now()}`,
 recipientId: form.category === "Salaries" ? form.recipient : null,
 recipientName: form.category === "Salaries"
```

```
 ? users.find(u => u.uid === form.recipient)?.full_name
 : form.recipient,
 processedById: currentUser?.uid,
 processedByName: users.find(u => u.uid === currentUser?.uid)?.full_name ||
currentUser?.email,
 };

```

```
try {
 await DisbursementsAPI.create(payload); //  persist to backend
 setForm({
 category: "",
 amount: "",
 date: "",
 recipient: "",
 processedBy: form.processedBy,
 status: "pending",
 });
 onClose();
} catch (err) {
 console.error("Failed to save disbursement", err);
} finally {
 setLoading(false);
}
};

```

```
return (
 <div className="modal-overlay">
 <div className="modal">
 <h2>New Disbursement</h2>
 <form onSubmit={handleSubmit}>
 <label>
 Category:
 <select name="category" value={form.category} onChange={handleChange}
required>
 <option value="">Select Category</option>
 {["Salaries","Supplies","Utilities","Infrastructure","Health Programs","Miscellaneous"]
 .map(cat => <option key={cat} value={cat}>{cat}</option>)}
 </select>
 </label>

 <label>
 Amount:
 <input type="number" name="amount" value={form.amount}
onChange={handleChange} required />
 </label>

 <label>
 Date:
 <input type="date" name="date" value={form.date} onChange={handleChange}
required />
 </label>
```

```

<label>

Recipient:

{form.category === "Salaries" ? (

 <select name="recipient" value={form.recipient} onChange={handleChange}
required>

 <option value="">Select User</option>

 {users.map(user => (

 <option key={user.uid} value={user.uid}>

 {user.full_name || user.email}

 </option>

))}

 </select>

) : (

 <input type="text" name="recipient" value={form.recipient}
onChange={handleChange} required />

)}

</label>

<div className="modal-actions">

 <button type="submit" disabled={loading}>

 {loading ? "Saving..." : "Save"}

 </button>

 <button type="button" onClick={onClose}>Cancel</button>

</div>

</form>

</div>

</div>

```

```
);
}
```

```
export default DisbursementForm;
```

```
`` `
```

```
frontend\src\components\forms\document-form.css
```

```
`` `css
```

```
.document-form {
 background-color: var(--bg-color, #ffffff);
 padding: 1.5rem;
 border-radius: 8px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
 margin-bottom: 2rem;
}
```

```
.document-form h2 {
 text-align: center;
 margin-bottom: 1rem;
 color: var(--text-color, #333);
}
```

```
.document-form form {
 display: flex;
```

```
flex-direction: column;
gap: 1rem;
}
```

```
.document-form label {
 font-weight: bold;
 color: var(--text-color, #444);
}
```

```
.document-form input,
.document-form select,
.document-form textarea {
 padding: 0.5rem;
 border: 1px solid #ccc;
 border-radius: 4px;
 font-size: 0.95rem;
 background-color: var(--input-bg, #f9f9f9);
 color: var(--text-color, #333);
}
```

```
.document-form textarea {
 resize: vertical;
 min-height: 80px;
}
```

```
.document-form button {
```

```
padding: 0.6rem 1rem;
background-color: #0077cc;
color: white;
border: none;
border-radius: 4px;
font-weight: bold;
cursor: pointer;
transition: background-color 0.2s ease;
}
```

```
.document-form button:hover {
 background-color: #005fa3;
}
```

```
.document-form p {
 margin-top: 1rem;
 text-align: center;
 font-size: 0.95rem;
}
```

```
.document-form a {
 color: #0077cc;
 text-decoration: none;
 font-weight: bold;
}
```



```
.document-form a:hover {
 text-decoration: underline;
}
```

```
```
```

```
## frontend\src\components\forms\incident-form.css
```

```
```css
```

```
/* incident-form.css */
```

```
.incident-form {
 display: flex;
 flex-direction: column;
 gap: 1rem;
 max-width: 500px;
 margin: auto;
 padding: 1.5rem;
 background-color: var(--color-bg);
 border-radius: 10px;
}
```

```
.incident-form input,
.incident-form textarea,
.incident-form select {
 padding: 0.5rem;
 border-radius: 4px;
 border: 1px solid #ccc;
```

```
}

.incident-form button {

 background-color: #007bff;

 color: white;

 padding: 0.5rem;

 border: none;

 border-radius: 4px;

}

` ` `
```

```
frontend\src\components\forms\IncidentForm.js
```

```
` ` ` javascript
```

```
import { useState, useEffect } from "react";
import { api, endpoints } from "../../services/api";
import { auth } from "../../services/firebase";
import { toast } from "react-toastify";
import "../../incident-form.css";

const IncidentForm = ({ role = "resident", userInfo, onSubmitSuccess }) => {
 const [residents, setResidents] = useState([]);
 const [loading, setLoading] = useState(false);
 const canLogForResident = role === "staff" || role === "admin";

 const initialForm = {
 type: "",
```

```
description: "",
location: "",
date: "",
time: "",
witness: "",
//  Always ensure residentId is set
residentId: role === "resident" ? userInfo?.uid || "" : "",
};
```

```
const [formData, setFormData] = useState(initialForm);
```

```
const handleChange = (e) => {
 const { name, value } = e.target;
 setFormData((prev) => ({ ...prev, [name]: value }));
};
```

```
const resetForm = () => {
 setFormData({
 ...initialForm,
 residentId: role === "resident" ? userInfo?.uid || "" : "",
 });
};
```

```
const handleSubmit = async (e) => {
 e.preventDefault();
 setLoading(true);
```

```
try {
 // ✅ Basic validation
 if (formData.description.trim().length < 5) {
 toast.error("Description must be at least 5 characters long.");
 setLoading(false);
 return;
 }

 if (role === "resident") {
 // 🔑 Ensure we have a logged-in Firebase user
 const currentUser = auth.currentUser;
 if (!currentUser) {
 toast.error("You must be logged in to report an incident.");
 setLoading(false);
 return;
 }

 // Resident self-report → backend API (triggers notification logic)
 await api.post(endpoints.incidents, {
 type: formData.type,
 description: formData.description,
 location: formData.location,
 authUid: currentUser.uid, // ✅ always from Firebase Auth
 residentId: currentUser.uid, // ✅ same for self-report
 });
 }
}
```

```
toast.success("✅ Incident report submitted!");
} else if (canLogForResident) {
 // Staff reporting → Firestore directly
 if (!formData.residentId) {
 toast.error("Please select a resident.");
 setLoading(false);
 return;
 }

 // 🔒 Ensure we have a logged-in Firebase user
 const currentUser = auth.currentUser;
 if (!currentUser) {
 toast.error("You must be logged in to log an incident.");
 setLoading(false);
 return;
 }

 // Staff logs incident on behalf of a resident via backend API
 await api.post(endpoints.incidents, {
 type: formData.type,
 description: formData.description,
 location: formData.location,
 authUid: currentUser.uid, // ✅ staff UID
 residentId: formData.residentId, // ✅ resident selected
 });
}
```

```

 toast.success("✅ Incident logged on behalf of resident!");
 }

 // ✅ Reset and refresh
 onSubmitSuccess?.();
 resetForm();
} catch (err) {
 console.error("❌ Incident submission failed:", err.response?.data || err.message);
 toast.error("❌ Failed to report incident. Please try again.");
} finally {
 setLoading(false);
}
};

// Staff/Admin: fetch resident list
useEffect(() => {
 if (canLogForResident) {
 api.get(endpoints.residents, { params: { limit: 100 } })
 .then((res) => {
 const raw = res.data;
 const normalized = Array.isArray(raw)
 ? raw
 : Array.isArray(raw?.results)
 ? raw.results
 : Array.isArray(raw?.items)

```

```

 ? raw.items
 : Array.isArray(raw?.data)
 ? raw.data
 : [];
 setResidents(normalized);
 })
 .catch((err) => {
 console.error(" ❌ Failed to load residents:", err.response?.data || err.message);
 });
}
}, [canLogForResident]);

```

```

return (
 <form className="incident-form" onSubmit={handleSubmit}>
 <h2>{canLogForResident ? "Log Incident (on behalf of resident)" : "Report Incident"}</h2>

```

```

 <label>Type</label>
 <select name="type" value={formData.type} onChange={handleChange} required>
 <option value="">Select type</option>
 <option value="Theft">Theft</option>
 <option value="Dispute">Dispute</option>
 <option value="Accident">Accident</option>
 <option value="Other">Other</option>
 </select>

```

```
<label>Description</label>
```

```
<textarea
```

```
 name="description"
```

```
 value={formData.description}
```

```
 onChange={handleChange}
```

```
 required
```

```
 minLength={5}
```

```
<label>Location</label>
```

```
<input name="location" value={formData.location} onChange={handleChange} required
>
```

```
{role === "resident" && (
```

```
 <>
```

```
 <label>Date</label>
```

```
 <input type="date" name="date" value={formData.date} onChange={handleChange}
required />
```

```
 <label>Time</label>
```

```
 <input type="time" name="time" value={formData.time} onChange={handleChange}
required />
```

```
 <label>Witness (optional)</label>
```

```
 <input name="witness" value={formData.witness} onChange={handleChange} />
```

```
 </>
```

```
)}
```



```

{canLogForResident && (
 <>
 <label>Reported Resident</label>
 <select
 name="residentId"
 value={formData.residentId}
 onChange={handleChange}
 required
 >
 <option value="">Select a resident</option>
 {residents.map((r) => (
 <option key={r.id} value={r.id}>
 {r.fullName || r.name || "Unnamed"} — Barangay {r.address?.barangay || "N/A"}
 </option>
))}
 </select>
</>
)}

<button type="submit" disabled={loading}>
 {loading ? "Submitting..." : "Submit Report"}
</button>
</form>

);
};

```

```
export default IncidentForm;
```

```
````
```

```
## frontend\src\components\forms\PaymentForm.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";
```

```
import { useForm } from "react-hook-form";
```

```
import { toast } from "react-toastify";
```

```
import { auth, db } from "../../services/firebase";
```

```
import { doc, getDoc } from "firebase/firestore";
```

```
import ReceiptPreview from "../../staff/ReceiptPreview";
```

```
import "../../styles/staff/payment_form.css";
```

```
const PaymentForm = ({
```

```
 entityId, // businessId or documentId
```

```
 entityType, // "business" or "document"
```

```
 resident,
```

```
 entityCategory,
```

```
 fee,
```

```
 onCancel,
```

```
 onPaymentCompleted,
```

```
 businessName,
```

```
 description,
```

```

customEntityId,
}) => {
 const { register, handleSubmit } = useForm();
 const [receiptData, setReceiptData] = useState(null);
 const [isSubmitting, setIsSubmitting] = useState(false);
 const [currentStaff, setCurrentStaff] = useState(null);
 const [generatePDF, setGeneratePDF] = useState(null);

 useEffect(() => {
 const fetchStaffProfile = async () => {
 try {
 if (!auth.currentUser) return;
 const ref = doc(db, "users", auth.currentUser.uid);
 const snap = await getDoc(ref);
 if (snap.exists()) {
 setCurrentStaff({ ...snap.data(), uid: auth.currentUser.uid });
 }
 } catch (err) {
 console.error(" ⚠️ Failed to fetch staff profile:", err);
 }
 };
 fetchStaffProfile();
 }, []);

 const onSubmit = async (data) => {

```

```
if (!resident || !currentStaff || !entityId) {
 toast.error(" ⚠ Missing required data.");
 return;
}
```

```
setIsSubmitting(true);
```

```
try {
```

```
 // 💡 Choose endpoint dynamically
```

```
 const url =
```

```
 entityType === "business"
```

```
 ? "/api/paymongo/payments/business"
```

```
 : "/api/paymongo/payments/document";
```

```
 const payload =
```

```
 entityType === "business"
```

```
 ? {
```

```
 businessId: entityId,
```

```
 businessName: data.businessName || businessName,
```

```
 amount: fee,
```

```
 method: data.method,
```

```
 processedBy: currentStaff.full_name || currentStaff.fullName,
```

```
 }
```

```
 : {
```

```
 documentId: entityId,
```

```
 documentName: customEntityId,
```

```
 docType: entityCategory,
```

```
 amount: fee,
 method: data.method,
 processedBy: currentStaff.full_name || currentStaff.fullName,
 residentName: resident.fullName,
 };
```

```
const response = await fetch(url, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(payload),
});
```

```
let result;

try {
 result = await response.json();
} catch {
 const text = await response.text();
 throw new Error(text);
}
```

```
console.log("Backend result:", result);
```

```
if (result.success) {
 toast.success(` 🇸🇬 Payment recorded. Receipt #: ${result.receiptNumber} `);
```

```
const receiptBase = {
```

```
receiptNumber: result.receiptNumber,
residentName: resident.fullName,
amount: fee,
method: data.method,
processedBy: currentStaff.full_name || currentStaff.fullName,
issuedAt: new Date().toISOString(),
entityType,
};
```

```
const receiptData =
entityType === "business"
? {
 ...receiptBase,
 entityId: result.businessId,
 customEntityId: result.businessId,
 businessId: result.businessId,
 businessName: result.businessName,
 ownerName: result.ownerName,
 businessType: result.businessType,
 barangay: result.barangay,
 description: result.businessType,
}
: {
 ...receiptBase,
 entityId: result.documentId, // Firestore ID
 customEntityId: customEntityId, // sequential ID
```

```

 documentType: result.documentType,
 barangay: result.barangay,
 description: result.documentType,
 };

 setReceiptData(receiptData);
} else {
 toast.error(` ❌ Failed to record payment: ${result.message || "Unknown error"} `);
}
} catch (error) {
 console.error(" ❌ Error recording payment:", error);
 toast.error(" ❌ Failed to record payment.");
} finally {
 setIsSubmitting(false);
}
};

return (
 <div className="form-step">
 <h2> 💰 Record Payment</h2>
 <p>Resident: {resident?.fullName}</p>
 <p>{entityType === "business" ? "Business Type" : "Document Type"}:
 {entityCategory}</p>
 <p>Fee: ₱{fee}</p>

 {!receiptData ? (

```

```
<>
```

```
<label>
```

```
 Payment Method
```

```
 <select {...register("method", { required: true })}>
```

```
 <option value="cash">Cash</option>
```

```
 <option value="gcash">GCash</option>
```

```
 <option value="card">Card</option>
```

```
 </select>
```

```
</label>
```

```
<button type="button" onClick={onCancel}>Cancel</button>
```

```
<button type="submit" disabled={isSubmitting} onClick={handleSubmit(onSubmit)}>
```

```
 {isSubmitting ? "Recording..." : "Record Payment"}
```

```
</button>
```

```
</>
```

```
): (
```

```
<>
```

```
<ReceiptPreview receiptData={receiptData} onGeneratePDF={setGeneratePDF} />
```

```
<div className="action-buttons">
```

```
 <button
```

```
 type="button"
```

```
 className="print-btn"
```

```
 onClick={() => {
```

```
 if (generatePDF) generatePDF();
```

```
 }}
```

```
>
```



 Print Receipt

</button>

{entityType === "document" ? (

<>

<button type="button" className="proceed-btn"  
onClick={onPaymentCompleted}>

 Proceed to Issue Document

</button>

<button type="button" className="close-btn" onClick={onCancel}>

 Close

</button>

</>

): (

<button type="button" className="close-btn" onClick={onCancel}>

 Close

</button>

)}

</div>

</>

)}

</div>

);

};

export default PaymentForm;

```

frontend\src\components\forms\resident-form.css

```css

/\* resident-form.css \*/

```
.resident-form {
 max-width: 600px;
 margin: 2rem auto;
 padding: 1.5rem;
 background: #f9f9f9;
 border-radius: 8px;
 border: 1px solid #ddd;
 font-family: system-ui, sans-serif;
}
```

```
.resident-form label {
 display: block;
 margin-top: 1rem;
 font-weight: 600;
 color: #333;
}
```

```
.resident-form input,
```

```
.resident-form select,
.resident-form textarea {
 width: 100%;
 padding: 0.5rem;
 margin-top: 0.25rem;
 border: 1px solid #ccc;
 border-radius: 4px;
 font-size: 0.95rem;
 box-sizing: border-box;
}
```

```
.resident-form input:focus,
.resident-form select:focus,
.resident-form textarea:focus {
 outline: none;
 border-color: #0078d4;
 box-shadow: 0 0 2px rgba(0, 120, 212, 0.2);
}
```

```
.resident-form fieldset {
 margin-top: 1.5rem;
 padding: 1rem;
 border: 1px solid #ccc;
 border-radius: 6px;
}
```

```
.resident-form legend {
 font-weight: 600;
 color: #444;
 padding: 0 0.5rem;
}
```

```
.input-error {
 border: 2px solid var(--color-danger);
 background-color: #ffe6e6;
}
```

```
.error {
 color: var(--color-danger);
 font-size: 0.85rem;
 margin-top: 4px;
 display: block;
}
```

```
.auth-warning {
 text-align: center;
 color: var(--color-warning);
 font-weight: 600;
 margin-top: 2rem;
}
```

```
.photo-preview-wrapper,
```

```
.fingerprint-preview-wrapper {
 margin-top: 0.5rem;
 display: flex;
 align-items: center;
 gap: 1rem;
}
```

```
.photo-preview,
.fingerprint-preview {
 max-width: 120px;
 max-height: 120px;
 border: 1px solid #ccc;
 border-radius: 6px;
 object-fit: cover;
}
```

```
.resident-form button {
 margin-top: 1.5rem;
 padding: 0.6rem 1.2rem;
 font-size: 0.95rem;
 font-weight: 600;
 border: none;
 border-radius: 4px;
 cursor: pointer;
}
```

```
.resident-form button[type="submit"] {
 background-color: #0078d4;
 color: #fff;
}
```

```
.resident-form button[type="submit"]:disabled {
 background-color: #ccc;
 cursor: not-allowed;
}
```

```
.resident-form button[type="button"] {
 background-color: #f3f3f3;
 color: #333;
}
```

```
.resident-form button[type="button"]:hover {
 background-color: #e1e1e1;
}
```

```
.signature-preview-wrapper {
 margin-top: 10px;
}
```

```
.signature-preview {
 max-width: 200px;
 height: 60px;
```

```
object-fit: contain;

border-bottom: 1px solid #000;
}
```

```
```
```

```
## frontend\src\components\forms\ResidentForm.js
```

```
```javascript
```

```
import { useState } from "react";
import { useOutletContext, useNavigate } from "react-router-dom";
import { useForm } from "react-hook-form";
import { ResidentsAPI } from "../../services/api";
import { toast } from "react-toastify";
import { auth } from "../../services/firebase";
import SignatureField from "./SignatureField";
import { uploadFile, uploadBase64Image, uploadThumbprint } from "../../utils/fileUtils";
import { PARANAQUE } from "../../data/locations";
import { cleanPayload } from "../../utils/cleanPayload";
import { sendEmail } from "../../services/email";
import "./resident-form.css";

const getAuthUid = () => {
 const uid = auth.currentUser?.uid;

 if (!uid) throw new Error(" ❌ Missing Firebase Auth UID. User may not be authenticated.");

 return uid;
}
```

```
};
```

```
const calculateAge = (birthDate) => {
 const today = new Date();
 const dob = new Date(birthDate);
 let age = today.getFullYear() - dob.getFullYear();
 const m = today.getMonth() - dob.getMonth();
 if (m < 0 || (m === 0 && today.getDate() < dob.getDate())) {
 age--;
 }
 return age;
};
```

```
const ResidentForm = ({ onResidentAdded, onCancel, user: userProp }) => {
 const outlet = useOutletContext();
 const user = userProp || outlet?.user;
 const navigate = useNavigate();
```

```
 const {
 register,
 handleSubmit,
 reset,
 watch,
 formState: { errors },
 } = useForm({ shouldFocusError: true });
```



```
const [isSubmitting, setIsSubmitting] = useState(false);
const [photo, setPhoto] = useState({ file: null, preview: null });
const [leftThumb, setLeftThumb] = useState({ file: null, preview: null });
const [rightThumb, setRightThumb] = useState({ file: null, preview: null });
const [signatureDataUrl, setSignatureDataUrl] = useState("");
const [isSignatureEmpty, setIsSignatureEmpty] = useState(true);
```

```
const birthDate = watch("birthDate");
const fullName = watch("fullName");
const age = birthDate ? calculateAge(birthDate) : "";
```

```
const handleFileChange = (e, setter, label) => {
 const file = e.target.files[0];
 if (!file) return;
 if (file.size > 2 * 1024 * 1024)
 return toast.error(` ❌ ${label} must be under 2MB.`);
 if (!file.type.startsWith("image/"))
 return toast.error(` ❌ Only image files allowed for ${label}.`);
 setter({ file, preview: URL.createObjectURL(file) });
};
```

```
const buildPayload = async (data) => {
 const uid = getAuthUid();
 await auth.currentUser.getIdToken(true);
 const tokenResult = await auth.currentUser.getIdTokenResult();
 console.log(" 🔑 Current role claim:", tokenResult.claims.role);
```

```
let photoUrl = null;

let leftThumbUrl = null;

let rightThumbUrl = null;

let signatureUrl = null;

try {
 if (photo.file) photoUrl = await uploadFile(uid, photo.file, "photos", true);
 if (leftThumb.file) leftThumbUrl = await uploadThumbprint(uid, leftThumb.file, "left");
 if (rightThumb.file) rightThumbUrl = await uploadThumbprint(uid, rightThumb.file,
"right");
 if (signatureDataUrl) signatureUrl = await uploadBase64Image(uid, signatureDataUrl,
"signatures");
} catch (err) {
 console.error("✖ Upload failed:", err);
 throw new Error("✖ Upload failed. Please retry.");
}

return cleanPayload(data, {
 photoUrl,
 fingerprints: { left: leftThumbUrl, right: rightThumbUrl },
 signatureUrl,
});
};

const clearForm = () => {
```

```
reset();

setPhoto({ file: null, preview: null });

setLeftThumb({ file: null, preview: null });

setRightThumb({ file: null, preview: null });

setSignatureDataURL("");

setIsSignatureEmpty(true);

};
```

```
const onSubmit = async (data) => {

 setIsSubmitting(true);

 try {

 if (isSignatureEmpty) {

 toast.error(" ❌ Please provide a signature before submitting.");

 return;

 }

 }
```

```
 const existing = await ResidentsAPI.findByNameAndBirthDate(data.fullName,
data.birthDate);

 if (existing?.length) {

 toast.error(" ❌ Duplicate resident detected!");

 return;

 }
```

```
const payload = await buildPayload(data);
```

```
console.log(" ✅ Payload email before sendWelcomeEmail:", payload.email);
```

```
if (
 (photo.file && !payload.photoUrl) ||
 (leftThumb.file && !payload.fingerprints.left) ||
 (rightThumb.file && !payload.fingerprints.right) ||
 (signatureDataURL && !payload.signatureUrl)
){
 toast.error("❌ One or more uploads failed. Please try again.");
 return;
}

console.log("🚀 Final payload:", payload);
const created = await ResidentsAPI.create(payload);
const residentId = created?.id || created?.uid;
if (!residentId) throw new Error("❌ Backend did not return a resident ID.");

// 📧 Send welcome email
try {
 await sendEmail({
 type: "welcome",
 fullName: payload.fullName,
 email: payload.email,
 barangay: payload.address?.barangay || payload.barangay,
 });
}
```

```
console.log("✉️ Function received payload:", payload);

console.log("✉️ Sending email to:", payload.email);

toast.success("✅ Resident added and email sent!");
} catch (emailErr) {

 console.error("❌ Failed to send welcome email:", emailErr);

 toast.warning("✅ Resident added but failed to send welcome email.");
}

clearForm();

if (onResidentAdded) onResidentAdded(residentId);

else navigate(`/residents`);

} catch (error) {

 console.error("❌ Error adding resident:", error);

 toast.error(error?.response?.data?.message || error.message || "❌ Failed to add resident.");

} finally {

 setIsSubmitting(false);

}

};

const handleCancel = () => {

 if (onCancel) onCancel();

 else navigate("/residents");

};
```

```
if (!user) return <p className="auth-warning"> ❌ You must be logged in to add a
resident.</p>;
```

```
return (
 <form onSubmit={handleSubmit(onSubmit)} className="resident-form">
 { /* Full Name */}
 <label htmlFor="fullName">Full Name</label>
 <input id="fullName" {...register("fullName", { required: "Full name is required" })}
className={errors.fullName ? "input-error" : ""} />
 {errors.fullName && {errors.fullName.message}}

 { /* Birth Date */}
 <label htmlFor="birthDate">Birth Date</label>
 <input id="birthDate" type="date" {...register("birthDate", { required: "Birth date is
required" })} className={errors.birthDate ? "input-error" : ""} />
 {errors.birthDate && {errors.birthDate.message}}
 {age && <p>Age: {age}</p>}

 { /* Gender */}
 <label htmlFor="gender">Gender</label>
 <select id="gender" {...register("gender", { required: "Gender is required" })}
className={errors.gender ? "input-error" : ""}>
 <option value="">Select Gender</option>
 <option value="Male">Male</option>
 <option value="Female">Female</option>
 <option value="Other">Other</option>
 </select>
```

```
{errors.gender && {errors.gender.message}}
```

```
{/* Civil Status */}
```

```
<label htmlFor="civilStatus">Civil Status</label>
```

```
<select id="civilStatus" {...register("civilStatus", { required: "Civil status is required" })}
className={errors.civilStatus ? "input-error" : ""}>
```

```
<option value="">Select Status</option>
```

```
<option value="Single">Single</option>
```

```
<option value="Married">Married</option>
```

```
<option value="Widowed">Widowed</option>
```

```
<option value="Separated">Separated</option>
```

```
</select>
```

```
{errors.civilStatus && {errors.civilStatus.message}}
```

```
{/* Contact Number */}
```

```
<label htmlFor="contactNumber">Contact Number</label>
```

```
<input id="contactNumber" {...register("contactNumber", { required: "Contact number is
required", pattern: { value: /^09\d{9}$/, message: "Must be a valid PH mobile number" }) }
className={errors.contactNumber ? "input-error" : ""} />
```

```
{errors.contactNumber && <span
className="error">{errors.contactNumber.message}}
```

```
{/* Email */}
```

```
<label htmlFor="email">Email</label>
```

```
<input id="email" type="email" {...register("email", { required: "Email is required" }) }
className={errors.email ? "input-error" : ""} />
```

```
{errors.email && {errors.email.message}}
```

```

 { /* Address */
<fieldset>

<legend>Address</legend>

<label htmlFor="houseNumber">House Number</label>

<input
 id="houseNumber"

 {...register("houseNumber", { required: "House number is required" })}

 className={errors.houseNumber ? "input-error" : ""}
/>

{errors.houseNumber && {errors.houseNumber.message}}

<label htmlFor="street">Street</label>

<input
 id="street"

 {...register("street", { required: "Street is required" })}

 className={errors.street ? "input-error" : ""}
/>

{errors.street && {errors.street.message}}

<label htmlFor="purok">Purok</label>

<input
 id="purok"

 {...register("purok", { required: "Purok is required" })}

```



```

 className={errors.purok ? "input-error" : ""}
 />
 {errors.purok && {errors.purok.message}}

 <label htmlFor="barangay">Barangay</label>

 <select
 id="barangay"
 {...register("barangay", { required: "Barangay is required" })}
 className={errors.barangay ? "input-error" : ""}
 >
 <option value="">Select Barangay</option>
 {PARANAQUE.barangays.map((b) => (
 <option key={b} value={b}>{b}</option>
))}
 </select>
 {errors.barangay && {errors.barangay.message}}

 <label htmlFor="city">City</label>

 <input id="city" value={PARANAQUE.city} readOnly {...register("city")} />

 <label htmlFor="province">Province</label>

 <input id="province" value={PARANAQUE.province} readOnly {...register("province")} />

 <label htmlFor="zipCode">Zip Code</label>

 <input id="zipCode" {...register("zipCode", { required: "Zip code is required" })}
 className={errors.zipCode ? "input-error" : ""} />

```

```
{errors.zipCode && {errors.zipCode.message}}
</fieldset>
```

```
{/* Head of Family */}
```

```
<label htmlFor="isHeadOfFamily">Head of Family</label>
```

```
<select
```

```
id="isHeadOfFamily"
```

```
{...register("isHeadOfFamily", { required: "Head of family selection is required" })}
```

```
className={errors.isHeadOfFamily ? "input-error" : ""}
```

```
>
```

```
<option value="">Select</option>
```

```
<option value="true">Yes</option>
```

```
<option value="false">No</option>
```

```
</select>
```

```
{errors.isHeadOfFamily && <span
className="error">{errors.isHeadOfFamily.message}}
```

```
{/* Voter Status */}
```

```
<label htmlFor="voterStatus">Voter Status</label>
```

```
<select
```

```
id="voterStatus"
```

```
{...register("voterStatus", { required: "Voter status is required" })}
```

```
className={errors.voterStatus ? "input-error" : ""}
```

```
>
```

```
<option value="">Select</option>
```

```
<option value="yes">Yes</option>
```

```

<option value="no">No</option>

<option value="unknown">Unknown</option>

</select>

{errors.voterStatus && {errors.voterStatus.message}}

{/* Occupation */}

<label htmlFor="occupation">Occupation</label>

<input id="occupation" {...register("occupation")} />

{/* Remarks */}

<label htmlFor="remarks">Remarks</label>

<textarea id="remarks" {...register("remarks")} />

{/* Photo */}

<label htmlFor="photo">Photo (2×2 inches)</label>

<input

 id="photo"

 type="file"

 accept="image/*"

 onChange={(e) => handleFileChange(e, setPhoto, "Photo")}

/>

{photo.preview && (

 <div className="photo-preview-wrapper">

 <img src={photo.preview} alt={` Preview of ${fullName}`} className="photo-preview"

 />

 <button type="button" onClick={() => setPhoto({ file: null, preview: null })}>

```

```
 Remove
 </button>
 </div>
)}
```

```
 { /* Left Thumb */
 <label htmlFor="leftThumb">Left Thumb</label>
 <input
 id="leftThumb"
 type="file"
 accept="image/*"
 onChange={(e) => handleFileChange(e, setLeftThumb, "Left Thumb")}
 />
 {leftThumb.preview && (
 <div className="fingerprint-preview-wrapper">
 <img src={leftThumb.preview} alt={` Left thumb of ${fullName}` }
 className="fingerprint-preview" />
 <button type="button" onClick={() => setLeftThumb({ file: null, preview: null })}>
 Remove
 </button>
 </div>
)}
 }
```

```
 { /* Right Thumb */
 <label htmlFor="rightThumb">Right Thumb</label>
 <input
```

```

 id="rightThumb"

 type="file"

 accept="image/*"

 onChange={(e) => handleFileChange(e, setRightThumb, "Right Thumb")}

 />

 {rightThumb.preview && (
 <div className="fingerprint-preview-wrapper">

 <img src={rightThumb.preview} alt={` Right thumb of ${fullName}`}
 className="fingerprint-preview" />

 <button type="button" onClick={() => setRightThumb({ file: null, preview: null })}>

 Remove

 </button>

 </div>

)}

 {/* Signature input */}

 <SignatureField onChange={setSignatureDataUrl}
 onEmptyCheck={setIsSignatureEmpty} />

 <button type="submit" disabled={isSubmitting}>

 {isSubmitting ? "Adding..." : "Add Resident"}

 </button>

 <button type="button" onClick={handleCancel} disabled={isSubmitting} style={{
 marginLeft: "1rem" }}>

 Cancel

 </button>

</form>

```

```
);
};
```

```
export default ResidentForm;
```

```
````
```

```
## frontend\src\components\forms\SecretaryDocumentForm.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";
import { usePublicFees } from "../../hooks/usePublicFees";
import { useResidents } from "../../hooks/useResidents";
import { auth, db } from "../../services/firebase";
import { doc, getDoc } from "firebase/firestore";
import PaymentForm from "../PaymentForm";
import documentConfig from "../../config/documentConfig";
import { resolveLocation } from "../../utils/resolveLocation";
import { API_BASE_URL, BusinessesAPI } from "../../services/api";
import "../../styles/secretary/secretary-document-form.css";
```

```
const SecretaryDocumentWorkflow = ({ onCompleted }) => {
 const { documentTypes, loading: feesLoading, error: feesError } = usePublicFees();
 const { residents, loading: residentsLoading } = useResidents();

 const [formData, setFormData] = useState({
```

```
resident_id: "",
document_type: "",
fee: 0,
});

const [attachments, setAttachments] = useState({});
const [currentSecretary, setCurrentSecretary] = useState(null);
const [status, setStatus] = useState({ message: null, type: null });
const [newDoc, setNewDoc] = useState(null);
const [issuedDocUrl, setIssuedDocUrl] = useState(null);
const [registeredBusinesses, setRegisteredBusinesses] = useState([]);
const [issueRemarks, setIssueRemarks] = useState("");
```

```
// Load secretary profile
```

```
useEffect(() => {
 const fetchSecretary = async () => {
 if (!auth.currentUser) return;
 const ref = doc(db, "users", auth.currentUser.uid);
 const snap = await getDoc(ref);
 if (snap.exists()) {
 setCurrentSecretary({ ...snap.data(), uid: auth.currentUser.uid });
 }
 };
 fetchSecretary();
}, []);
```

```

// Initialize default document type
useEffect(() => {
 if (documentTypes.length > 0) {
 const first = documentTypes[0];
 setFormData(prev => ({
 ...prev,
 document_type: first.documentType,
 fee: first.totalFee,
 }));
 }
}, [documentTypes]);

```

```

useEffect(() => {
 BusinessesAPI.listAll()
 .then(data => {
 // If backend returns { businesses: [...] }
 const list = Array.isArray(data.businesses)
 ? data.businesses
 : Array.isArray(data)
 ? data
 : [];
 setRegisteredBusinesses(list);
 })
 .catch(() => setRegisteredBusinesses([]));
}, []);

```



```

useEffect(() => {
 if (formData.document_type === "Blotter Report" && formData.resident_id) {
 const resident = residents.find(r => r.id === formData.resident_id);
 if (resident) {
 setFormData(prev => ({ ...prev, complainant: resident.fullName }));
 }
 }
}, [formData.document_type, formData.resident_id, residents]);

```

```

const handleChange = e => {
 const { name, value } = e.target;

 if (name === "document_type") {
 const selected = documentTypes.find(dt => dt.documentType === value);
 setFormData(prev => ({
 ...prev,
 document_type: value,
 fee: selected?.totalFee || 0,
 }));
 } else if (name === "businessName") {
 // ♦ Find the selected resident
 const resident = residents.find(r => r.id === formData.resident_id);

 // ♦ Find the business owned by that resident
 const business = registeredBusinesses.find(
 b => b.businessName === value && b.ownerName === resident?.fullName
);
 }
};

```

```
);
```

```
if (business) {
```

```
 setFormData(prev => ({
```

```
 ...prev,
```

```
 businessName: value,
```

```
 "location.street": business.street || "",
```

```
 "location.barangay": business.barangay || "",
```

```
 "location.city": business.city || "",
```

```
 "location.province": business.province || ""
```

```
 }));
```

```
 } else {
```

```
 setFormData(prev => ({ ...prev, businessName: value }));
```

```
 }
```

```
 } else {
```

```
 setFormData(prev => ({ ...prev, [name]: value }));
```

```
 }
```

```
};
```

```
const handleFileChange = (e, field) => {
```

```
 const file = e.target.files[0];
```

```
 setAttachments(prev => ({ ...prev, [field]: file }));
```

```
};
```

```
const validateForm = () => {
```

```
const config = documentConfig[formData.document_type] || {};
const { fields = [], attachments: attachmentRules = [] } = config;

if (formData.document_type === "Blotter Report" && !formData.resident_id) {
 return "Resident must be selected for Blotter Report.";
}

for (const field of fields) {
 if (formData.document_type === "Blotter Report" && field.name === "complainant") {
 continue;
 }
 const value = formData[field.name];
 if (field.required && !value) return `${field.label} is required.`;
 if (field.minLength && value?.length < field.minLength)
 return `${field.label} must be at least ${field.minLength} characters.`;
 if (field.min !== undefined && Number(value) < field.min)
 return `${field.label} must be at least ${field.min}.`;
}

for (const att of attachmentRules) {
 const file = attachments[att.name];
 if (att.required && !file) {
 return `${att.label} is required.`;
 }
}
```

```
if (formData.document_type === "Business Clearance") {
 const loc = resolveLocation(formData.document_type, formData, residents,
registeredBusinesses);
 if (!loc.address) return "Business address is required.";
}

return null;
};

const handleSubmit = async e => {
 e.preventDefault();

 const errorMsg = validateForm();
 if (errorMsg) {
 setStatus({ message: ` ❌ ${errorMsg}`, type: "error" });
 return;
 }
 if (!currentSecretary) {
 setStatus({ message: " ❌ Secretary profile not loaded yet.", type: "error" });
 return;
 }

 setStatus({ message: "Creating document...", type: "loading" });

 try {
 const payload = new FormData();
```

```
payload.append("resident_id", formData.resident_id);

payload.append("document_type", formData.document_type);

const config = documentConfig[formData.document_type] || {};
const { fields = [], attachments: attachmentRules = [] } = config;

const loc = resolveLocation(formData.document_type, formData, residents,
registeredBusinesses);

if (formData.document_type !== "Barangay Clearance") {
 if (loc.barangay) payload.append("locationBarangay", loc.barangay);
 if (loc.street) payload.append("locationStreet", loc.street);
 if (loc.city) payload.append("locationCity", loc.city);
 if (loc.province) payload.append("locationProvince", loc.province);
}

if (formData.document_type === "Business Clearance") {
 if (loc.barangay) payload.append("locationBarangay", loc.barangay);
 if (loc.street) payload.append("locationStreet", loc.street);
 if (loc.city) payload.append("locationCity", loc.city);
 if (loc.province) payload.append("locationProvince", loc.province);
 // ✅ Add normalized full address
 payload.append("address", loc.address);
}

fields.forEach(field => {
```

```

if (field.type === "group" && Array.isArray(field.fields)) {
 field.fields.forEach(sub => {
 if (!sub.name?.startsWith("location.")) {
 const value = formData[sub.name];
 if (value !== undefined && value !== null && value !== "") {
 payload.append(sub.name, value);
 }
 }
 });
} else if (field.name && !field.name.startsWith("location.")) {
 if (!(formData.document_type === "Blotter Report" && field.name === "complainant"))
 {
 const value = formData[field.name];
 if (value !== undefined && value !== null && value !== "") {
 payload.append(field.name, value);
 }
 }
}
});

```

```

if (formData.document_type === "Blotter Report" && formData.complainant) {
 payload.append("complainant", formData.complainant);
}

```

```

attachmentRules.forEach(att => {
 if (attachments[att.name]) {

```

```

 payload.append(att.name, attachments[att.name]);
 }
});

const response = await fetch(`${API_BASE_URL}/api/documents`, {
 method: "POST",
 body: payload,
});

if (!response.ok) throw new Error(` Backend error: ${response.statusText} `);
const createdDoc = await response.json();

setNewDoc(createdDoc);

setStatus({ message: "✅ Document created. Proceed to payment.", type: "success" });
} catch (err) {
 console.error("❌ Error creating document:", err);
 setStatus({ message: "❌ Failed to create document.", type: "error" });
}
};

const handlePaymentCompleted = async () => {
 try {
 // Re-fetch document to ensure payment status is updated
 const refreshedDoc = await fetch(`${API_BASE_URL}/api/documents/${newDoc.id}`);
 const docData = await refreshedDoc.json();

 if (docData.status !== "paid" && docData.status !== "payment_submitted") {

```

```
setStatus({ message: "❌ Payment not yet confirmed. Try again.", type: "error" });

return;

}
```

```
const issueResponse = await
fetch(`${API_BASE_URL}/api/documents/${newDoc.id}/issue`, {

 method: "PATCH",

 headers: { "Content-Type": "application/json" },

 body: JSON.stringify({

 issued_by: currentSecretary.full_name || currentSecretary.fullName,

 remarks: issueRemarks || `Issued by ${currentSecretary.full_name}` || `Issued by
${currentSecretary.fullName}`,

 }},

});
```

```
if (!issueResponse.ok) {

 const errData = await issueResponse.json();

 throw new Error(errData.detail || issueResponse.statusText);

}
```

```
const issuedDoc = await issueResponse.json();

setIssuedDocUrl(issuedDoc.fileUrl);

setStatus({ message: "✅ Document issued successfully.", type: "success" });

onCompleted?.(issuedDoc);

} catch (err) {

 console.error("❌ Error issuing document:", err);

}
```



```
setStatus({ message: ` ❌ Failed to issue document: ${err.message}`, type: "error" });

}
};
```

```
return (
 <div className="secretary-document-form">
 <h2> 📄 Walk-In Document Issuance </h2>

 {feesLoading && <p>Loading document types...</p>}
 {feesError && <p className="status-message error"> ❌ {feesError}</p>}

 {!newDoc && (
 <form onSubmit={handleSubmit}>
 { /* Resident selection */ }
 <div className="form-group">
 <label htmlFor="resident_id">Select Resident</label>
 {residentsLoading ? (
 <p>Loading residents...</p>
) : (
 <select
 name="resident_id"
 value={formData.resident_id}
 onChange={handleChange}
 required
 >
 <option value="">Select Resident</option>
 </select>
)
 }
 </div>
)
)
);
```

```

 {residents.map(r => (
 <option key={r.id} value={r.id}>
 {r.fullName} — {r.address.barangay}
 </option>
))}
 </select>
)}
</div>

{/* Document type */}
<div className="form-group">
 <label htmlFor="document_type">Document Type</label>
 <select
 name="document_type"
 value={formData.document_type}
 onChange={handleChange}
 required
 >
 {documentTypes.map(dt => (
 <option key={dt.id} value={dt.documentType}>
 {dt.documentType} — ₱{dt.totalFee}
 </option>
))}
 </select>
</div>

```

```

{ /* Dynamic fields */
{documentConfig[formData.document_type]?.fields?.map(field => {
 if (formData.document_type === "Blotter Report" && field.name === "complainant") {
 return (
 <div className="form-group" key={field.name}>
 <label htmlFor={field.name}>{field.label}</label>
 <input
 id={field.name}
 type="text"
 name={field.name}
 value={formData.complainant || ""}
 readOnly
 />
 </div>
);
 }
}

```

```

if (field.type === "group") {
 return (
 <fieldset key={field.label}>
 <legend>{field.label}</legend>
 {field.fields.map(sub => (
 <div className="form-group" key={sub.name}>
 <label htmlFor={sub.name}>{sub.label}</label>

 {sub.type === "select" ? (

```

```

 <select
 id={sub.name}
 name={sub.name}
 value={formData[sub.name] || ""}
 onChange={handleChange}
 required={sub.required}
 >
 <option value="">-- Select --</option>
 {sub.options?.map(opt => (
 <option key={opt} value={opt}>{opt}</option>
))}
 </select>
) : (
 <input
 id={sub.name}
 type={sub.type}
 name={sub.name}
 value={formData[sub.name] || sub.default || ""}
 onChange={handleChange}
 required={sub.required}
 />
)}
</div>

)}
</fieldset>

);

```

```
}
```

```
if (field.type === "select" && field.name === "businessName") {
 // ♦ Only show businesses for the selected resident
 const resident = residents.find(r => r.id === formData.resident_id);
 const residentBusinesses = resident
 ? registeredBusinesses.filter(b => b.ownerName === resident.fullName)
 : [];
```

```
return (
 <div className="form-group" key={field.name}>
 <label htmlFor={field.name}>{field.label}</label>
 <select
 id={field.name}
 name={field.name}
 value={formData[field.name] || ""}
 onChange={handleChange}
 required={field.required}
 >
 <option value="">
 {residentBusinesses.length === 0
 ? "No businesses available for this resident"
 : "Select Business"}
 </option>
 {residentBusinesses.map(b => (
 <option key={b.businessId} value={b.businessName}>
```

```

 {b.businessName} — {b.barangay}
 </option>
)}
</select>

```

```

{ /* ♦ Show auto-filled business address when selected */ }

```

```

{formData.document_type === "Business Clearance" && formData.businessName
&& (

```

```

 <fieldset>
 <legend>Business Address</legend>
 <div className="form-group">
 <label>Street</label>
 <input type="text" value={formData["location.street"]} readOnly />
 </div>
 <div className="form-group">
 <label>Barangay</label>
 <input type="text" value={formData["location.barangay"]} readOnly />
 </div>
 <div className="form-group">
 <label>City</label>
 <input type="text" value={formData["location.city"]} readOnly />
 </div>
 <div className="form-group">
 <label>Province</label>
 <input type="text" value={formData["location.province"]} readOnly />
 </div>
 </fieldset>

```

```

 { /* Hidden full address for backend */
 <input
 type="hidden"
 name="address"
 value={
 formData["location.street"],
 ` Brgy. ${formData["location.barangay"]} `,
 formData["location.city"],
 formData["location.province"]
 }.filter(Boolean).join(", ")
 />
 </fieldset>
)}
</div>
);
}

return (
 <div className="form-group" key={field.name}>
 <label htmlFor={field.name}>{field.label}</label>
 <input
 id={field.name}
 type={field.type}

```

```

 name={field.name}
 value={formData[field.name] || ""}
 onChange={handleChange}
 required={field.required}
 />
 </div>
);
}}

```

```

{documentConfig[formData.document_type]?.attachments?.map(att => (
 <div className="form-group" key={att.name}>
 <label htmlFor={att.name}>
 {att.label} {att.required && *}
 </label>
 <input
 id={att.name}
 type="file"
 accept=".jpg,.jpeg,.png,.pdf"
 required={att.required}
 onChange={e => handleFileChange(e, att.name)}
 />
 </div>
)}}

```

```

 <button type="submit" className="submit-btn" disabled={!currentSecretary}>
 Create Document (₹{formData.fee})
 </button>

```



```
</button>

</form>

}}
```

```
{status.message && <p className={` status-message
${status.type}` }>{status.message}</p>}
```

```
{/*  PaymentForm appears after document creation */}

{newDoc && !issuedDocUrl && (

 <PaymentForm

 docId={newDoc.id}

 entityId={newDoc.id}

 entityType="document"

 resident={residents.find(r => r.id === formData.resident_id)}

 entityCategory={formData.document_type}

 description={newDoc.documentType}

 fee={newDoc.amount}

 customEntityId={newDoc.documentId}

 onCancel={() => setNewDoc(null)}

 onPaymentCompleted={handlePaymentCompleted}

 />

)}
```

```
{/*  After payment, issue document */}

{issuedDocUrl && (

 <div className="download-section">
```

```
<a
 href={issuedDocUrl}
 target="_blank"
 rel="noopener noreferrer"
 className="download-btn"
>
  Download Issued Document (PDF)
```

```

```

```
<button
 type="button"
 className="print-btn"
 onClick={() => {
 const printWindow = window.open(issuedDocUrl, "_blank");
 if (printWindow) {
 printWindow.addEventListener("load", () => {
 printWindow.print();
 });
 }
 }}
>
```

```
  Print Document
```

```
</button>
```

```
<button
 type="button"
 className="back-btn"
```

```

onClick={() => {
 setNewDoc(null);
 setIssuedDocUrl(null);
 setStatus({ message: null, type: null });
 setFormData({
 resident_id: "",
 document_type: documentTypes.length > 0 ? documentTypes[0].documentType :
 "",
 fee: documentTypes.length > 0 ? documentTypes[0].totalFee : 0,
 });
 setAttachments({});
 setIssueRemarks("");
}}
>
  Back to Form
</button>
</div>
)}
</div>
);
};

```

```
export default SecretaryDocumentWorkflow;
```

```

` ` `

```

```
frontend\src\components\forms\signature-field.css
```

```
```.css
```

```
.signature-field {  
  margin-top: 20px;  
}
```

```
.signature-canvas {  
  border: 1px solid #ccc;  
  border-radius: 4px;  
  background: #fff;  
}
```

```
.signature-actions {  
  margin-top: 8px;  
  display: flex;  
  gap: 10px;  
}
```

```
.signature-actions button {  
  padding: 6px 12px;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.signature-actions button:first-child {  
  background: #6c757d;  
  color: #fff;  
}
```

```
.signature-actions button:last-child {  
  background: #007bff;  
  color: #fff;  
}
```

```
.signature-preview img {  
  margin-top: 10px;  
  max-width: 200px;  
  height: auto;  
  border-bottom: 1px solid #000;  
}
```

```
````
```

```
frontend\src\components\forms\SignatureField.js
```

```
```javascript
```

```
import React, { useRef, useState, useEffect } from 'react';  
import SignatureCanvas from 'react-signature-canvas';  
import './signature-field.css';
```

```
const SignatureField = ({ label = "Signature", onChange, onEmptyCheck }) => {  
  const sigCanvas = useRef(null);  
  const [signatureUrl, setSignatureUrl] = useState("");  
  
  // Check if pad is empty on mount  
  useEffect(() => {  
    if (onEmptyCheck) {  
      onEmptyCheck(sigCanvas.current?.isEmpty());  
    }  
  }, [onEmptyCheck]);  
  
  const clearSignature = () => {  
    sigCanvas.current.clear();  
    setSignatureUrl("");  
    onChange?.("");  
    onEmptyCheck?.(true); // pad is empty  
  };  
  
  const saveSignature = () => {  
    if (!sigCanvas.current.isEmpty()) {  
      const canvas = sigCanvas.current.getCanvas();  
      const dataUrl = canvas.toDataURL('image/png');  
      setSignatureUrl(dataUrl);  
      onChange?.(dataUrl);  
      onEmptyCheck?.(false); // pad has content  
    } else {
```

```
    setSignatureUrl("");
    onChange?.(null);
    onEmptyCheck?.(true);
  }
};
```

```
// Attach scroll prevention only for touch devices
```

```
useEffect(() => {
  const canvasEl = sigCanvas.current?.getCanvas();
  if (!canvasEl) return;

  const preventScroll = (e) => e.preventDefault();

  if ('ontouchstart' in window) {
    // Intentionally non-passive: prevent page scroll while signing on touch devices
    canvasEl.addEventListener('touchstart', preventScroll, { passive: false });
    canvasEl.addEventListener('touchmove', preventScroll, { passive: false });

    return () => {
      canvasEl.removeEventListener('touchstart', preventScroll);
      canvasEl.removeEventListener('touchmove', preventScroll);
    };
  }
}, []);

return (
```

```

<div className="signature-field">
  <label>{label}</label>
  <SignatureCanvas
    ref={sigCanvas}
    penColor="black"
    canvasProps={{ width: 400, height: 150, className: 'signature-canvas' }}
  />
  <div className="signature-actions">
    <button type="button" onClick={clearSignature}>Clear</button>
    <button type="button" onClick={saveSignature}>Save</button>
  </div>

  {signatureUrl && (
    <div className="signature-preview">
      <img src={signatureUrl} alt="Signature preview" />
    </div>
  )}
</div>

);
};

export default SignatureField;

` ``

## frontend\src\components\forms\StaffBusinessForm.js

```



```
` `` javascript
```

```
import { useEffect, useState } from "react";  
import { useForm } from "react-hook-form";  
import { toast } from "react-toastify";  
import { useNavigate } from "react-router-dom";  
import { doc, getDoc, collection, addDoc } from "firebase/firestore";  
import { auth, db } from "../services/firebase";  
import { PARANAQUE } from "../data/locations";  
import { usePublicFees } from "../hooks/usePublicFees";  
import { useResidents } from "../hooks/useResidents";  
import { NotificationsAPI } from "../services/api";  
import PaymentForm from "./PaymentForm";  
import "../business-form.css";
```

```
const getDisplayName = (profile = {}, fallbackEmail = "") => {  
  const firstLast = [profile.firstName || profile.first_name, profile.lastName ||  
    profile.last_name]  
    .filter(Boolean)  
    .join(" ")  
    .trim();  
  
  return (  
    profile.fullName ||  
    profile.full_name ||  
    profile.name ||
```

```
    firstLast ||  
    fallbackEmail  
  );  
};
```

```
const StaffBusinessForm = ({ onBusinessAdded, onCancel }) => {  
  const { register, handleSubmit, trigger, reset } = useForm();  
  const navigate = useNavigate();
```

```
  const [step, setStep] = useState(1);  
  const [isSubmitting, setIsSubmitting] = useState(false);  
  const [docId, setDocId] = useState(null);  
  const [businessId, setBusinessId] = useState(null);
```

```
  const { businessTypes: businessFees } = usePublicFees();  
  const { residents, loading: residentsLoading } = useResidents();  
  const [currentStaff, setCurrentStaff] = useState(null);
```

```
  const [selectedResident, setSelectedResident] = useState(null);  
  const [selectedFee, setSelectedFee] = useState(0);  
  const [selectedBusinessType, setSelectedBusinessType] = useState("");  
  const [submittedBusinessName, setSubmittedBusinessName] = useState("");
```

```
  useEffect(() => {  
    const fetchStaffProfile = async () => {  
      if (!auth.currentUser) return;
```

```

const ref = doc(db, "users", auth.currentUser.uid);

const snap = await getDoc(ref);

if (snap.exists()) {

  setCurrentStaff({ ...snap.data(), uid: auth.currentUser.uid });

}

};

fetchStaffProfile();

}, []);

```

// 🛠 Custom ID Generators

```

const generateBusinessId = (barangay) => {

  const year = new Date().getFullYear();

  const random = Math.floor(1000 + Math.random() * 9000);

  return `BIZ-${barangay?.toUpperCase()}-${year}-${random}`;

};

```

```

const generatePermitNumber = (barangay) => {

  const year = new Date().getFullYear();

  const random = Math.floor(1000 + Math.random() * 9000);

  return `PERMIT-${year}-${barangay?.toUpperCase()}-${random}`;

};

```

```

const onSubmit = async (data) => {

  const feeObj = businessFees.find(bt => bt.businessType === data.businessType);

  if (!selectedResident) {

```

```
toast.error(" ⚠ Please select a resident.");

return;
}

if (!currentStaff) {

  toast.error(" ⚠ Staff profile not loaded yet. Please wait.");

  return;
}


setIsSubmitting(true);

try {

  const customBusinessId = generateBusinessId(data.barangay);

  const staffEmail = currentStaff.email || auth.currentUser?.email || "";

  const staffName = getDisplayName(currentStaff, staffEmail) || "Unknown Staff";


  const payload = {

    ...data,

    ownerName: selectedResident.fullName,

    contactNumber: selectedResident.contactNumber,

    email: selectedResident.email || "",

    status: "approved",

    businessId: customBusinessId,

    permitNumber: generatePermitNumber(data.barangay),

    submittedAt: new Date().toISOString(),

    createdBy: {

      uid: currentStaff.uid || auth.currentUser?.uid || "",

      name: staffName,
```

```
    email: staffEmail,  
  },  
};
```

```
const docRef = await addDoc(collection(db, "businesses"), payload);
```

```
await NotificationsAPI.createBusinessSubmitted(  
  selectedResident.fullName || selectedResident.name || selectedResident.email ||  
  "Resident",  
  data.businessName  
)  
.catch((notifyError) => {  
  console.warn(" ⚠️ Business registration notification failed:", notifyError);  
});
```

```
await NotificationsAPI.createBusinessStatusUpdate(  
  "approved",  
  null,  
  data.businessName,  
  customBusinessId,  
  docRef.id  
)  
.catch((notifyError) => {  
  console.warn(" ⚠️ Business owner notification failed:", notifyError);  
});
```

```
setDocId(docRef.id);  
setBusinessId(customBusinessId);
```

```

toast.success(`  Business registered for ${selectedResident.fullName}`);

reset();

onBusinessAdded?.();

setSubmittedBusinessName(data.businessName);

setStep(4);

setSelectedFee(feeObj?.registrationTotal || 0);

setSelectedBusinessType(data.businessType);
} catch (error) {

  console.error("  Error registering business:", error);

  toast.error("  Failed to register business.");

} finally {

  setIsSubmitting(false);

}

};

const handleCancel = () => {

  if (onCancel) onCancel();

  else navigate("/businesses");

};

return (

  <form className="business-form" onSubmit={handleSubmit(onSubmit)}>

    {step === 1 && (

      <div className="form-step">

```

<h2>  Select Resident</h2>

{residentsLoading ? (

<p>Loading residents...</p>

): (

<select

value={selectedResident?.id || ""}

onChange={(e) =>

setSelectedResident(residents.find((r) => r.id === e.target.value))

}

>

<option value="">Select Resident</option>

{residents.map((r) => (

<option key={r.id} value={r.id}>

{r.fullName} — {r.address.barangay}

</option>

))}

</select>

)}

<h2>  Business Details</h2>

<label>Business Name

<input {...register("businessName", { required: true })} />

</label>

<label>Business Type

```

<select {...register("businessType", { required: true })}>
  <option value="">Select Type</option>
  {businessFees.map((bt) => (
    <option key={bt.id} value={bt.businessType}>
      {bt.businessType} — ₱{bt.registrationTotal}
    </option>
  ))}
</select>
</label>

```

```

{/* Grouped Business Address */}
<fieldset className="address-fieldset">
  <legend>Business Address</legend>

```

```

<label>Street
  <input {...register("street", { required: true })} />
</label>

```

```

<label>Barangay
  <select {...register("barangay", { required: true })}>
    <option value="">Select Barangay</option>
    {PARANAQUE.barangays.map((brgy) => (
      <option key={brgy} value={brgy}>{brgy}</option>
    ))}
  </select>
</label>

```



```
<label>City
```

```
  <input value={PARANAQUE.city} readOnly {...register("city")} />
```

```
</label>
```

```
<label>Province
```

```
  <input value={PARANAQUE.province} readOnly {...register("province")} />
```

```
</label>
```

```
</fieldset>
```

```
<label>Registration Date
```

```
  <input type="date" {...register("registrationDate", { required: true })} />
```

```
</label>
```

```
<button
```

```
  type="button"
```

```
  disabled={isSubmitting}
```

```
  onClick={async () => {
```

```
    const valid = await trigger([
```

```
      "businessName",
```

```
      "businessType",
```

```
      "street",
```

```
      "barangay",
```

```
      "city",
```

```
      "province",
```

```
      "registrationDate",
```

```

    });

    if (valid && selectedResident) handleSubmit(onSubmit());

    else toast.error(" ⚠️ Please select a resident and fill in all required fields.");

  }}

  >

  {isSubmitting ? "Registering..." : "Register Business →"}

</button>

</div>

)}}

{step === 4 && docId && businessId && (
  <PaymentForm
    docId={docId}          // Firestore UID
    entityId={businessId}   // use generated businessId for backend payment lookup
    entityType="business"
    resident={selectedResident}
    entityCategory={selectedBusinessType}
    fee={selectedFee}
    businessId={businessId} // ✅ use generated custom ID
    businessName={submittedBusinessName} // ✅ pass business name for receipt
    onCancel={handleCancel}
  />
)}}

</form>

);

```

```
};
```

```
export default StaffBusinessForm;
```

```
```\n
```

```
frontend\\src\\components\\layout\\DashboardSection.js
```

```
```javascript
```

```
// components/layout/DashboardSection.js
```

```
const DashboardSection = ({
```

```
  title,
```

```
  icon,
```

```
  accent = "accent",
```

```
  layout = "grid-auto",
```

```
  ariaLabel,
```

```
  children,
```

```
}) => {
```

```
  const sectionClass = `dashboard-section ${layout} ${accent}`;
```

```
  return (
```

```
    <section className={sectionClass} aria-label={ariaLabel}>
```

```
      <h3>{icon} {title}</h3>
```

```
      {children}
```

```
    </section>
```

```
  );
```

```
};
```

```
export default DashboardSection;
```

```
```\n
```

```
frontend\\src\\components\\lists\\resident-list.css
```

```
```css
```

```
/* 🚧 General layout */
```

```
.resident-list {  
  max-width: 1000px;  
  margin: 0 auto;  
  padding: 20px;  
  font-family: "Segoe UI", Arial, sans-serif;  
  color: var(--color-text);  
}
```

```
.resident-list h2 {  
  text-align: center;  
  margin-bottom: 20px;  
}
```

```
/* 🔍 Filters */
```

```
.filters {  
  display: flex;
```

```
flex-wrap: wrap;
gap: 10px;
margin-bottom: 20px;
justify-content: center;
}
```

```
.filters input,
.filters select,
.filters button {
padding: 8px 12px;
border-radius: 4px;
border: 1px solid #ccc;
font-size: 14px;
}
```

```
.filters button {
background: #007bff;
color: #fff;
cursor: pointer;
border: none;
}
```

```
.filters button:hover {
background: #0056b3;
}
```

```
/* 🇵🇭 Barangay summary */
```

```
.barangay-summary {  
  margin: 20px 0;  
}
```

```
.barangay-summary h3 {  
  margin-bottom: 10px;  
}
```

```
.barangay-summary ul {  
  list-style: none;  
  padding: 0;  
}
```

```
.barangay-summary li {  
  margin: 4px 0;  
}
```

```
/* 🇵🇭 Pie chart container */
```

```
.recharts-wrapper {  
  margin: 0 auto;  
  display: block;  
}
```

```
/* 🏠 Resident grid */
```

```
.resident-grid {
```

```
display: grid;
grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
gap: 16px;
margin-top: 20px;
}
```

```
.resident-card {
background: #f9f9f9;
border-radius: 8px;
padding: 12px;
text-align: center;
box-shadow: 0 2px 6px rgba(0,0,0,0.1);
transition: transform 0.2s ease;
}
```

```
.resident-card:hover {
transform: translateY(-4px);
}
```

```
.resident-photo {
width: 80px;
height: 80px;
border-radius: 50%;
object-fit: cover;
margin-bottom: 10px;
}
```

```
.resident-name {  
  display: block;  
  margin: 8px 0;  
  font-weight: bold;  
  background: none;  
  border: none;  
  color: #007bff;  
  cursor: pointer;  
}
```

```
.resident-name:hover {  
  text-decoration: underline;  
}
```

```
.delete-btn {  
  background: #dc3545;  
  color: #fff;  
  border: none;  
  padding: 6px 10px;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.delete-btn:hover {  
  background: #a71d2a;
```



```
}
```

```
/* 🗑 Pop-up overlay */
```

```
.popup-overlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background: rgba(0,0,0,0.6);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  z-index: 999;  
}
```

```
/* 🗑 Pop-up window */
```

```
.popup-window {  
  background: #fff;  
  padding: 20px;  
  border-radius: 8px;  
  width: 400px;  
  max-height: 85vh;  
  overflow-y: auto;  
  text-align: center;  
  box-shadow: 0 4px 12px rgba(0,0,0,0.3);
```

```
    position: relative;
}
```

```
.popup-window h3 {
    margin-top: 0;
}
```

```
.popup-actions {
    margin-top: 20px;
    display: flex;
    justify-content: space-around;
}
```

```
.confirm-btn {
    background: #d9534f;
    color: #fff;
    padding: 8px 16px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
```

```
.confirm-btn:hover {
    background: #a71d2a;
}
```

```
.cancel-btn {  
  background: #6c757d;  
  color: #fff;  
  padding: 8px 16px;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.cancel-btn:hover {  
  background: #565e64;  
}
```

```
/* 👤 Resident details pop-up */
```

```
.resident-details-popup {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

```
.close-btn {  
  position: absolute;  
  top: 10px;  
  right: 12px;  
  background: none;  
  border: none;
```

```
font-size: 20px;  
cursor: pointer;  
color: #333;  
}
```

```
.close-btn:hover {  
  color: #d9534f;  
}
```

```
.photo-section {  
  text-align: center;  
  margin-bottom: 15px;  
}
```

```
.resident-photo-large {  
  width: 120px;  
  height: 120px;  
  border-radius: 50%;  
  object-fit: cover;  
}
```

```
.details-section {  
  align-self: flex-start;  
  text-align: left;  
  width: 100%;  
  margin-bottom: 20px;
```

```
}
```

```
.details-section p {  
  margin: 6px 0;  
  font-size: 14px;  
}
```

```
/* Fingerprint boxes */  
.fingerprint-section {  
  display: flex;  
  justify-content: space-around;  
  width: 100%;  
  margin: 20px 0;  
}
```

```
.finger-box {  
  border: 1px solid #ccc;  
  border-radius: 6px;  
  padding: 10px;  
  width: 45%;  
  text-align: center;  
}
```

```
.finger-box img {  
  max-width: 100%;  
  height: 80px;
```

```
    object-fit: contain;
}
```

```
/* Signature section */
```

```
.signature-section {
    margin-top: 20px;
    text-align: left;
    width: 100%;
}
```

```
.signature-section img {
    max-width: 200px;
    height: 60px;
    object-fit: contain;
    border-bottom: 1px solid #000;
}
```

```
````
```

```
frontend\src\components\lists\ResidentList.js
```

```
```javascript
```

```
import { useState, useMemo, useEffect } from 'react';
import { PieChart, Pie, Cell, Tooltip, Legend } from 'recharts';
import { ResidentsAPI } from '../services/api';
import { toast } from 'react-toastify';
```

```
import './resident-list.css';
```

```
const COLORS = ['#0088FE', '#00C49F', '#FFBB28', '#FF8042'];
```

```
const DEFAULT_AVATAR = 'https://cdn-icons-png.flaticon.com/512/149/149071.png';
```

```
const ResidentList = ({ residents, loading, onResidentDeleted, fetchResidents }) => {
```

```
  const [searchTerm, setSearchTerm] = useState("");
```

```
  const [barangayFilter, setBarangayFilter] = useState("");
```

```
  const [voterFilter, setVoterFilter] = useState("");
```

```
  const [selectedResident, setSelectedResident] = useState(null);
```

```
  const [confirmDeleteId, setConfirmDeleteId] = useState(null);
```

```
//  Auto refresh every 30s
```

```
useEffect(() => {
```

```
  const interval = setInterval(() => {
```

```
    fetchResidents?();
```

```
  }, 30000);
```

```
  return () => clearInterval(interval);
```

```
}, [fetchResidents]);
```

```
//  Normalize resident input
```

```
const residentArray = useMemo(() => {
```

```
  if (Array.isArray(residents)) return residents;
```

```
  if (Array.isArray(residents?.results)) return residents.results;
```

```
  return [];
```

```
}, [residents]);
```

// 🗺️ Barangay summary

```
const barangayCounts = useMemo(() => {  
  return residentArray.reduce((acc, r) => {  
    const barangay = r.address?.barangay;  
    if (barangay) acc[barangay] = (acc[barangay] || 0) + 1;  
    return acc;  
  }, {});  
}, [residentArray]);
```

```
const chartData = useMemo(  
  () => Object.entries(barangayCounts).map(([barangay, count]) => ({ name: barangay,  
value: count })),  
  [barangayCounts]  
);
```

```
const barangayOptions = useMemo(  
  () => Array.from(new Set(residentArray.map((r) =>  
r.address?.barangay).filter(Boolean))).sort(),  
  [residentArray]  
);
```

// 🔍 Filtered residents

```
const filteredResidents = useMemo(() => {  
  return residentArray.filter((r) => {  
    const name = r.fullName?.toLowerCase() || "";
```



```

    const matchesSearch = name.includes(searchTerm.toLowerCase());

    const matchesBarangay = barangayFilter ? r.address?.barangay === barangayFilter :
true;

    const matchesVoter = voterFilter ? r.voterStatus === voterFilter : true;

    return matchesSearch && matchesBarangay && matchesVoter;

  });
}, [residentArray, searchTerm, barangayFilter, voterFilter]);

```

// 📄 CSV Export

```

const exportToCSV = () => {

  const headers = ['Full Name', 'Birth Date', 'Gender', 'Contact', 'Address', 'Occupation'];

  const rows = filteredResidents.map((r) => [

    r.fullName || "",

    r.birthDate || "",

    r.gender || "",

    r.contactNumber || "",

    `${r.address?.houseNumber || ""} ${r.address?.street || ""}, ${r.address?.barangay || ""},
${r.address?.city || ""}`,

    r.occupation || "",

  ]);

```

```

const csvContent = [headers, ...rows]

  .map((row) => row.map((cell) => `${cell}`).join(','))

  .join('\n');

```

```

const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });

```

```

const url = URL.createObjectURL(blob);
const link = document.createElement('a');
link.href = url;
link.setAttribute('download', 'resident_registry.csv');
document.body.appendChild(link);
link.click();
document.body.removeChild(link);
};

```

```

// 🗑 Delete resident

```

```

const confirmDelete = async () => {
  if (!confirmDeleteld) return;
  try {
    await ResidentsAPI.delete(confirmDeleteld);
    toast.success('Resident deleted successfully');
    setSelectedResident(null);
    onResidentDeleted?.(confirmDeleteld);
    fetchResidents?.(); // refresh immediately after delete
  } catch (err) {
    console.error('❌ Error deleting resident:', err);
    toast.error('❌ Failed to delete resident');
  } finally {
    setConfirmDeleteld(null);
  }
};

```

```
if (loading) return <p>Loading residents...</p>;
```

```
if (residentArray.length === 0) return <p>No residents found.</p>;
```

```
return (
```

```
  <div className="resident-list">
```

```
    <h2>Resident Directory</h2>
```

```
    { /*  Filters */ }
```

```
    <div className="filters">
```

```
      <input
```

```
        type="text"
```

```
        placeholder="Search by name"
```

```
        value={searchTerm}
```

```
        onChange={(e) => setSearchTerm(e.target.value)}
```

```
      />
```

```
      <select value={barangayFilter} onChange={(e) => setBarangayFilter(e.target.value)}>
```

```
        <option value="">All Barangays</option>
```

```
        {barangayOptions.map((b) => (
```

```
          <option key={b} value={b}>{b}</option>
```

```
        )}}
```

```
      </select>
```

```
      <select value={voterFilter} onChange={(e) => setVoterFilter(e.target.value)}>
```

```
        <option value="">All Voter Status</option>
```

```
        <option value="yes">Registered</option>
```

```
        <option value="no">Unregistered</option>
```

```
        <option value="unknown">Unknown</option>
```

```
</select>
```

```
<button onClick={exportToCSV}> 📄 Export CSV</button>
```

```
</div>
```

```
{/* 📊 Barangay Summary */}
```

```
<div className="barangay-summary">
```

```
<h3> 📊 Residents per Barangay</h3>
```

```
<ul>
```

```
{Object.entries(barangayCounts).map(([barangay, count]) => (
```

```
<li key={barangay}>
```

```
<strong>{barangay}</strong> {count} resident{count > 1 ? 's' : ''}
```

```
</li>
```

```
)))
```

```
</ul>
```

```
</div>
```

```
{/* 📊 Pie Chart */}
```

```
<PieChart width={400} height={300}>
```

```
<Pie data={chartData} dataKey="value" nameKey="name" cx="50%" cy="50%"  
outerRadius={100} label>
```

```
{chartData.map((entry, index) => (
```

```
<Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
```

```
)))
```

```
</Pie>
```

```
<Tooltip />
```

```
<Legend />
```

```
</PieChart>
```

```
{/* 🧑 Resident Cards */}
```

```
<div className="resident-grid">
```

```
{filteredResidents.map((r) => (
```

```
<div key={r.id} className="resident-card">
```

```
<img
```

```
src={r.photoUrl || DEFAULT_AVATAR}
```

```
alt={` ${r.fullName || 'Unnamed'}'s ID` }
```

```
className="resident-photo"
```

```
/>
```

```
<button className="resident-name" onClick={() => setSelectedResident(r)}>
```

```
{r.fullName || 'Unnamed'}
```

```
</button>
```

```
<button className="delete-btn" onClick={() => setConfirmDeleteId(r.id)}> 🗑️
```

```
Delete</button>
```

```
</div>
```

```
)))
```

```
</div>
```

```
{/* 🗑️ Confirmation Pop-Up */}
```

```
{confirmDeleteId && (
```

```
<div className="popup-overlay">
```

```
<div className="popup-window">
```

```
<h3>Confirm Deletion</h3>
```

```
<p>Are you sure you want to delete this resident?</p>
```

```
<div className="popup-actions">

  <button className="confirm-btn" onClick={confirmDelete}>Yes, Delete</button>

  <button className="cancel-btn" onClick={() =>
setConfirmDeleteId(null)}>Cancel</button>

</div>

</div>

</div>

)}
```

```
{/* 🧑 Resident Details Pop-Up */}

{selectedResident && (

  <div className="popup-overlay">

    <div className="popup-window resident-details-popup">

      <button className="close-btn" onClick={() =>
setSelectedResident(null)}>✕</button>


```

```
<div className="photo-section">

  <img

    src={selectedResident.photoUrl || DEFAULT_AVATAR}

    alt="Resident ID"

    className="resident-photo-large"

  />

</div>


```

```
<div className="details-section">

  <p><strong>Full Name:</strong> {selectedResident.fullName}</p>


```

```

<p><strong>Birth Date:</strong> {selectedResident.birthDate}</p>
<p><strong>Gender:</strong> {selectedResident.gender}</p>
<p><strong>Contact:</strong> {selectedResident.contactNumber}</p>
<p><strong>Address:</strong> `{ ${selectedResident.address?.houseNumber || ""}
${selectedResident.address?.street || ""}, ${selectedResident.address?.barangay || ""},
${selectedResident.address?.city || ""} `</p>
<p><strong>Occupation:</strong> {selectedResident.occupation}</p>
<p><strong>Voter Status:</strong> {selectedResident.voterStatus}</p>
</div>

```

```

{/* Fingerprint boxes */}
<div className="fingerprint-section">
  <div className="finger-box">
    <p>Left Thumb</p>
    {selectedResident.fingerprints?.left ? (
      <img
        src={selectedResident.fingerprints.left}
        alt="Left Thumb"
        className="fingerprint-img"
      />
    ) : (
      <p>No left thumb uploaded</p>
    )}
  </div>
  <div className="finger-box">
    <p>Right Thumb</p>
    {selectedResident.fingerprints?.right ? (

```

```
<img
  src={selectedResident.fingerprints.right}
  alt="Right Thumb"
  className="fingerprint-img"
/>
):(
  <p>No right thumb uploaded</p>
)
</div>
</div>
```

```
{/* Signature */}
<div className="signature-section">
  <p>Signature:</p>
  {selectedResident.signatureUrl ? (
    <img
      src={selectedResident.signatureUrl}
      alt="Signature"
      className="signature-img"
    />
  ) : (
    <p>No signature uploaded</p>
  )}
</div>
</div>
</div>
```



```
    })  
  </div>  
);  
};
```

```
export default ResidentList;
```

```
```
```

```
frontend\src\components\main-layout.css
```

```
```css
```

```
/* =====
```

```
 Dashboard Layout (Grid)
```

```
===== */
```

```
body.dashboard-scroll-lock {  
  overflow: hidden;  
}
```

```
.dashboard-layout {  
  display: grid;  
  grid-template-columns: 250px 1fr 0px; /* sidebar, content, hidden notifications */  
  height: 100vh;  
  min-height: 0;  
  width: 100%;  
  max-width: 100vw;
```

```

overflow-y: hidden;

overflow-x: hidden;

background: var(--color-bg);

color: var(--color-text);

font-family: var(--font-family-base);

transition: grid-template-columns 0.3s ease;
}

/* When notifications are active, expand to 3 columns */
.dashboard-layout.with-notifications {
  grid-template-columns: 250px 1fr 300px;
}

/* =====

📌 Header Actions (in header flow)
===== */

.header-actions {
  display: flex;
  align-items: center;
  gap: 1rem;
}

/* Style tweaks for buttons */
.dark-mode-toggle {
  background: var(--color-card);

```

```
border: 1px solid var(--color-border);
padding: 0.5rem 0.75rem;
border-radius: var(--radius-sm);
cursor: pointer;
box-shadow: var(--shadow-sm);
transition: background 0.2s ease;
}
```

```
.header-actions button:hover {
  background: var(--color-hover);
}
```

```
/* =====
```

 Sidebar (Left)

```
===== */
```

```
.sidebar {
  background: var(--gradient-card);
  border-right: 1px solid var(--color-border);
  padding: var(--space-md);
  box-shadow: var(--shadow-sm);
  display: flex;
  flex-direction: column;
  gap: 2rem;
  height: 100vh;
  overflow-y: hidden;
  overflow-x: hidden;
```

```
}
```

```
.sidebar-title {  
  margin: 0 0 var(--space-sm) 0;  
}
```

```
.sidebar ul {  
  list-style: none;  
  margin: 0;  
  padding: 0;  
  display: flex;  
  flex-direction: column;  
  gap: 2rem;  
}
```

```
.sidebar li {  
  margin: 0;  
}
```

```
/* =====
```

```
 Dashboard Content (Center)
```

```
===== */
```

```
.dashboard-content {  
  display: flex;  
  flex-direction: column;  
  min-width: 0;
```

```
height: 100vh;
min-height: 0;
padding: var(--space-lg);
transition: padding 0.3s ease;
overflow-y: auto;
overflow-x: hidden;
}
```

```
.main-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: var(--space-lg);
}
```

```
.main-header h1 {
margin: 0;
}
```

```
/* =====
```

 Notification Panel (Right Sidebar)

```
===== */
```

```
.notification-panel {
width: 300px;
min-width: 0;
opacity: 0;
```

```
transform: translateX(16px);  
pointer-events: none;  
overflow-y: hidden;  
overflow-x: hidden;  
transition: opacity 0.22s ease, transform 0.22s ease;  
background: var(--color-card-bg);  
border-left: 1px solid var(--color-border);  
box-shadow: var(--shadow-md);  
height: 100vh;  
display: grid;  
grid-template-rows: auto minmax(0, 1fr) auto;  
}
```

```
.dashboard-layout.with-notifications .notification-panel {  
  opacity: 1;  
  transform: translateX(0);  
  pointer-events: auto;  
}
```

```
.notification-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: var(--space-md);  
  border-bottom: 1px solid var(--color-border);  
  background: var(--color-card-bg);
```

```
}
```

```
.notification-close-btn {  
  background: none;  
  border: none;  
  font-size: 1.1rem;  
  line-height: 1;  
  cursor: pointer;  
  color: var(--color-text);  
  padding: 0.25rem;  
  border-radius: var(--radius-sm);  
  transition: background 0.2s ease, opacity 0.2s ease;  
}
```

```
.notification-close-btn:hover {  
  background: var(--color-hover);  
  opacity: 0.9;  
}
```

```
.notification-body {  
  flex: 1;  
  min-height: 0;  
  overflow-y: auto;  
  padding: var(--space-md);  
}
```

```
.notification-item {  
  padding: var(--space-sm);  
  margin-bottom: var(--space-sm);  
  background: var(--color-card-muted);  
  color: var(--color-text);  
  border: 1px solid var(--color-border);  
  border-radius: var(--radius-sm);  
  display: flex;  
  flex-direction: column;  
  gap: 0.5rem;  
}
```

```
.notification-item.unread {  
  background: var(--color-table-row-hover);  
  border-color: var(--color-accent);  
  font-weight: 500;  
}
```

```
.notification-item .actions {  
  display: flex;  
  gap: 0.5rem;  
}
```

```
.notification-item button {  
  padding: 0.25rem 0.5rem;  
  font-size: 0.85rem;
```



```
cursor: pointer;

border: 1px solid var(--color-border);

background: var(--color-card-bg);

color: var(--color-text);

border-radius: var(--radius-sm);

transition: background 0.2s ease, color 0.2s ease;
}
```

```
.notification-item button:hover {

  background: var(--color-accent-hover);

  color: var(--color-card-bg);
}
```

```
.notification-footer {

  display: flex;

  gap: 0.5rem;

  padding: var(--space-md);

  border-top: 1px solid var(--color-border);

  background: var(--color-card-bg);
}
```

```
.notification-footer button {

  flex: 1;

  padding: 0.5rem;

  cursor: pointer;

  background: var(--color-accent);
}
```

```
color: var(--color-card-bg);  
border: 1px solid var(--color-border-strong);  
border-radius: var(--radius-sm);  
box-shadow: var(--shadow-sm);  
font-weight: var(--font-weight-bold);  
transition: background 0.2s ease, transform 0.15s ease;  
}
```

```
.notification-footer button:hover {  
  background: var(--color-accent-hover);  
  transform: translateY(-1px);  
}
```

```
.notification-footer button:last-child {  
  background: var(--color-danger);  
}
```

```
.notification-footer button:last-child:hover {  
  background: var(--color-danger-hover);  
}
```

```
@media (max-width: 1024px) {  
  .dashboard-layout,  
  .dashboard-layout.with-notifications {  
    grid-template-columns: 220px 1fr;  
    height: auto;
```

```
    min-height: 100dvh;  
}
```

```
.sidebar,  
.dashboard-content {  
    height: auto;  
    min-height: 100dvh;  
}
```

```
.notification-panel {  
    position: fixed;  
    right: 0;  
    top: 0;  
    height: 100dvh;  
    width: min(92vw, 360px);  
    z-index: 1200;  
    transform: translateX(100%);  
    opacity: 1;  
    pointer-events: none;  
}
```

```
.dashboard-layout.with-notifications .notification-panel {  
    transform: translateX(0);  
    pointer-events: auto;  
}  
}
```

```
@media (max-width: 768px) {  
  .dashboard-layout,  
  .dashboard-layout.with-notifications {  
    grid-template-columns: 1fr;  
    height: 100dvh;  
    min-height: 100dvh;  
    overflow-y: hidden;  
  }
```

```
.sidebar {  
  height: auto;  
  min-height: 0;  
  overflow: visible;  
  gap: 1rem;  
  padding: 0.75rem;  
}
```

```
.sidebar ul {  
  gap: 0.5rem;  
}
```

```
.main-header {  
  flex-direction: column;  
  align-items: flex-start;  
  gap: 0.75rem;
```

```
}
```

```
.main-header h1 {  
  font-size: 1.2rem;  
  line-height: 1.3;  
}
```

```
.header-actions {  
  width: 100%;  
  flex-wrap: wrap;  
  gap: 0.5rem;  
}
```

```
.header-actions > * {  
  width: 100%;  
}
```

```
.dark-mode-toggle {  
  width: 100%;  
}
```

```
.dashboard-content {  
  height: 100dvh;  
  min-height: 0;  
  padding: 0.75rem;  
  overflow-y: auto;
```

```
}  
}  
` ``
```

```
## frontend\src\components\MainLayout.js
```

```
` `` javascript
```

```
import { Outlet, useNavigate } from "react-router-dom";  
import { useUser } from "../context/UserContext";  
import { useTheme } from "../context/ThemeContext";  
import SecureNavLink from "../auth/SecureNavLink";  
import NotificationBell from "../components/NotificationBell";  
import { useNotifications } from "../context/NotificationContext";  
import { NotificationsAPI } from "../services/api";  
import { useEffect, useState } from "react";  
import sidebarLinks from "../config/navigation";  
import "../main-layout.css";
```

```
const getDisplayName = (profile = {}, fallbackEmail = "") => {  
  const firstLast = [profile.firstName || profile.first_name, profile.lastName ||  
    profile.last_name]  
    .filter(Boolean)  
    .join(" ")  
    .trim();  
  
  return (  

```

```
    profile.fullName ||  
    profile.full_name ||  
    profile.name ||  
    firstLast ||  
    fallbackEmail  
  );  
};
```

```
const MainLayout = () => {  
  const navigate = useNavigate();  
  const { userInfo, role, logout } = useUser();  
  const { isDarkMode, toggleDarkMode } = useTheme();  
  const {  
    notifications,  
    unreadCount,  
    markAsRead,  
    deleteNotification,  
    bulkDeleteNotifications,  
  } = useNotifications();  
  
  const [showNotifications, setShowNotifications] = useState(false);  
  
  useEffect(() => {  
    document.body.classList.add("dashboard-scroll-lock");  
    return () => {  
      document.body.classList.remove("dashboard-scroll-lock");  
    };  
  });  
};
```

```
};
```

```
, []);
```

```
const linksToRender = sidebarLinks[role] || sidebarLinks.default || [];
```

```
const handleBellClick = () => {
```

```
  setShowNotifications((prev) => !prev);
```

```
};
```

```
const handleLogout = async () => {
```

```
  try {
```

```
    // 🛎 Trigger logout notification
```

```
    if (role === "staff" || role === "admin") {
```

```
      await NotificationsAPI.createOfficerLogOut(
```

```
        getDisplayName(userInfo, userInfo?.email || ""),
```

```
        role
```

```
      );
```

```
    } else if (role === "resident") {
```

```
      await NotificationsAPI.createResidentLogOut(1);
```

```
    }
```

```
    await logout();
```

```
    navigate("/login");
```

```
  } catch (error) {
```

```
    console.error("❌ Logout failed:", error);
```

```
  }
```



```
};
```

```
return (
```

```
<div className={`dashboard-layout ${showNotifications ? "with-notifications" : ""}`}>
```

```
  {/* Sidebar */}
```

```
  <nav className="sidebar" aria-label="Sidebar Navigation">
```

```
    <h3 className="sidebar-title">Navigation</h3>
```

```
    <ul>
```

```
      {linksToRender.map(({ path, label, action }) => (
```

```
        <SecureNavLink key={path} action={action} to={path}>
```

```
          {label}
```

```
        </SecureNavLink>
```

```
      )))
```

```
      {role === "admin" && (
```

```
        <SecureNavLink action="manageSettings" to="/settings">
```

```
           Settings
```

```
        </SecureNavLink>
```

```
      )}
```

```
    <li>
```

```
      <button onClick={handleLogout} className="logout-btn" aria-label="Logout">
```

```
         Logout
```

```
      </button>
```

```
    </li>
```

```
</ul>
```

```
</nav>
```

```
{/* Main Content */}
```

```
<div className="dashboard-content">
```

```
<header className="main-header" aria-label="Main Header">
```

```
<h1>Barangay Information System</h1>
```

```
<div className="header-actions">
```

```
<button
```

```
  onClick={toggleDarkMode}
```

```
  className="dark-mode-toggle"
```

```
  aria-label="Toggle Dark Mode"
```

```
>
```

```
  {isDarkMode ? "☀️ Light Mode" : "🌙 Dark Mode"}
```

```
</button>
```

```
<NotificationBell
```

```
  onClick={handleBellClick}
```

```
  count={unreadCount}
```

```
/>
```

```
</div>
```

```
</header>
```

```
<Outlet context={{ user: userInfo }} />
```

```
</div>
```

```
{/* Notification Panel - third column that slides in */}
```

```
<aside className="notification-panel">
```

```
<header className="notification-header">
```

```

<h2>Notifications</h2>

<button
  type="button"
  className="notification-close-btn"
  onClick={() => setShowNotifications(false)}
  aria-label="Close notifications"
>
  ✕
</button>
</header>

<div className="notification-body">
  {notifications.length === 0 ? (
    <p>No notifications</p>
  ) : (
    notifications.map((n) => (
      <div
        key={n.id}
        className={`notification-item ${n.read ? "read" : "unread"}`}
      >
        <span>{n.message}</span>
        <small>{new Date(n.timestamp).toLocaleString()}</small>
        <div className="actions">
          {!n.read && (
            <button onClick={() => markAsRead(n.id)}>Mark read</button>
          )}
          <button onClick={() => deleteNotification(n.id)}>Delete</button>
        </div>
      </div>
    )
  )}

```

```

        </div>

    </div>

    ))

    })
</div>

<footer className="notification-footer">

    <button onClick={() => bulkDeleteNotifications(true)}>🗑 Clear Read</button>

    <button onClick={() => bulkDeleteNotifications(false)}>🗑 Delete All</button>

</footer>

</aside>

</div>

);

};

```

```
export default MainLayout;
```

```
...
```

```
## frontend\src\components\NotificationBell.js
```

```
```javascript
```

```
// components/NotificationBell.js
```

```
import "../styles/notification-bell.css";
```

```
const NotificationBell = ({ onClick, count = 0 }) => {
```

```
 return (
```

```

<div className="notification-bell">
 <button className="bell-button" onClick={onClick}>
 🛎️
 {count > 0 && {count}}
 </button>
</div>
);
};

```

```
export default NotificationBell;
```

```
...`
```

```
frontend\src\components\PublicLayout.js
```

```
```javascript
```

```
import { Outlet } from "react-router-dom";
```

```

const PublicLayout = () => (
  <div className="public-layout">
    <Outlet />
  </div>
);

```

```
export default PublicLayout;
```

```

## frontend\src\components\resident\FeedbackForm.txt

```text

```
import { useState } from "react";
```

```
import "../styles/admin.css";
```

```
const FeedbackForm = () => {
```

```
  const [message, setMessage] = useState("");
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
```

```
    console.log("Feedback submitted:", message);
```

```
  };
```

```
  return (
```

```
    <div className="feedback-form">
```

```
      <h3>💬 Submit Feedback</h3>
```

```
      <form onSubmit={handleSubmit}>
```

```
        <textarea
```

```
          rows="4"
```

```
          placeholder="Your feedback..."
```

```
          value={message}
```

```
          onChange={(e) => setMessage(e.target.value)}
```

```
        />
```

```
        <button type="submit">Send</button>

    </form>

</div>

);

};
```

```
export default FeedbackForm;
```

```
```
```

```
frontend\src\components\resident\MyComplaints.txt
```

```
```text
```

```
import { useEffect, useState } from "react";
import { collection, query, where, getDocs } from "firebase/firestore";
import { db, auth } from "../services/firebase";
import "../styles/admin.css";
```

```
const MyComplaints = () => {
  const [complaints, setComplaints] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    const fetchComplaints = async () => {
      try {
        const user = auth.currentUser;
```

```
if (!user) {  
  setComplaints([]);  
  setLoading(false);  
  return;  
}  
  
const q = query(  
  collection(db, "complaints"),  
  where("filed_for", "=", user.uid)  
);  
  
const snapshot = await getDocs(q);  
  
const data = snapshot.docs.map((doc) => ({  
  id: doc.id,  
  ...doc.data(),  
}));  
  
setComplaints(data);  
} catch (err) {  
  console.error("Error fetching complaints:", err);  
} finally {  
  setLoading(false);  
}  
};
```



```

    fetchComplaints();
  }, []);

  if (loading) return <p>Loading complaints...</p>;

  return (
    <div className="my-complaints">
      <h3> 📌 My Complaints</h3>

      {complaints.length === 0 ? (
        <p>No complaints filed yet.</p>
      ) : (
        <ul>
          {complaints.map((c) => (
            <li key={c.id}>
              <strong>{c.category}</strong>: {c.description} — {c.status}
              <br />
              <small>{c.timestamp?.toDate().toLocaleString()}</small>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
};

```

```
export default MyComplaints;
```

```
`,`,
```

```
## frontend\src\components\resident\MyDocuments.js
```

```
` `` `javascript
```

```
import { useState } from "react";
```

```
import { useMyDocuments } from "../../hooks/useMyDocuments";
```

```
import "../../styles/resident/my-documents.css";
```

```
const MyDocuments = ({ residentId }) => {
```

```
  const [tab, setTab] = useState("active");
```

```
  const { docs, loading } = useMyDocuments(residentId);
```

```
  const [payingMap, setPayingMap] = useState({});
```

```
  // -----
```

```
  // Helpers
```

```
  // -----
```

```
  const renderStatusBadge = (status) => (
```

```
    <span className={` status-badge status-${status}`} >{status}</span>
```

```
  );
```

```
  const handleResubmit = (docId) => {
```

```
    window.location.href = `/resubmit/${docId}`;
```

```
  };
```

```

const createDocumentPaymentLink = async (doc) => {
  const res = await fetch(`/api/paymongo/create-document-link`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      documentId: doc.document_id,
      documentType: doc.document_type,
      remarks: doc.remarks || "",
    }),
  });
  if (!res.ok) throw new Error(`Payment API error: ${await res.text()}`);
  return res.json();
};

```

```

const attachPaymentMethod = async (paymentIntentId, paymongoClientKey, method,
doc) => {
  const res = await fetch(`/api/paymongo/attach-payment-method`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      type: "payment_method",
      paymentIntentId,
      paymongoClientKey,
      method,
      billing: {

```

```

    name: doc.resident_name || "Resident",
    email: doc.resident_email || "resident@example.com",
  },
  return_url: "http://localhost:3000/payment-success?type=document",
}),
});

if (!res.ok) throw new Error(` Attach method error: ${await res.text()} `);

const data = await res.json();

return data?.redirectUrl;

};

// -----
// Payment Flow
// -----

const handlePaymentIntent = async (doc, method = "gcash") => {
  const docId = doc.document_id;

  if (!docId) {
    alert("Invalid document ID.");

    return;
  }

  setPayingMap((prev) => ({ ...prev, [docId]: true }));

  try {
    const data = await createDocumentPaymentLink(doc);

    const { paymentIntentId, checkoutUrl, paymongoClientKey } = data;

```

```

    if (checkoutUrl) {
        window.open(checkoutUrl, "_blank", "noopener,noreferrer");
        return;
    }
    if (!paymentIntentId) {
        alert("No payment intent ID returned.");
        return;
    }

    const redirectUrl = await attachPaymentMethod(paymentIntentId, paymongoClientKey,
method, doc);

    if (redirectUrl) {
        window.open(redirectUrl, "_blank", "noopener,noreferrer");
    } else {
        alert("Payment method attached but no redirect URL returned.");
    }
} catch (err) {
    console.error("❌ Payment flow error:", err);
    alert("Failed to initiate payment. Please try again.");
} finally {
    setPayingMap((prev) => ({ ...prev, [docId]: false }));
}
};

// -----
// Rendering

```

```
// -----

const renderPaymentButtons = (doc, isPaying) => (
  <div className="payment-options">
    [{"gcash", "grab_pay"].map((method) => (
      <button
        key={method}
        className="btn-pay"
        disabled={isPaying}
        onClick={() => handlePaymentIntent(doc, method)}
      >
         Pay with {method === "gcash" ? "GCash" : "GrabPay"}
      </button>
    )})
  </div>
);

const renderDocumentCard = (doc) => {
  const docId = doc.document_id;
  const isPaying = payingMap[docId] || false;

  return (
    <li key={docId} className="doc-card">
      <div className="doc-info">
        <strong>{doc.document_type || "Untitled Document"}</strong>
        <span>Status: {renderStatusBadge(doc.status)}</span>
        {doc.issuedAt && (
```

```

    <span>
      Issued At: {new Date(doc.issuedAt).toLocaleString()}
    </span>
  )}
  {doc.transactionId && (
    <span>
      Transaction ID: {doc.transactionId}
    </span>
  )}
</div>

```

```

{doc.status === "approved" && doc.remarks ? (
  <p className="doc-remarks">Issuance Remarks: {doc.remarks}</p>
) : (
  doc.remarks && <p className="doc-remarks">Remarks: {doc.remarks}</p>
)}

```

```

{doc.status === "rejected" && tab === "needsAttention" && (
  <button className="btn-resubmit" onClick={() => handleResubmit(docId)}>
     Resubmit
  </button>
)}

```

```

{doc.status === "for_payment" && tab === "active" && renderPaymentButtons(doc,
isPaying)}

```

```

    { /* ✅ Show download link if document is approved and fileUrl exists */
    {doc.status === "approved" && doc.fileUrl && (
      <div className="download-section">
        <a
          href={doc.fileUrl}
          target="_blank"
          rel="noopener noreferrer"
          className="btn-download"
        >
           Download Issued Document
        </a>
      </div>
    )}
  </li>
);
};

```

```

const filterDocs = (docs) => {
  switch (tab) {
    case "active":
      return docs.filter((doc) => ["pending", "for_payment", "awaiting_payment",
"paid"].includes(doc.status));
    case "needsAttention":
      return docs.filter((doc) => doc.status === "rejected" && !doc.resubmitted);
    case "history":

```



```

return docs.filter(
  (doc) =>
    ["approved", "paid"].includes(doc.status) ||
    (doc.status === "rejected" && doc.resubmitted)
);
default:
  return [];
}
};

```

```

const renderContent = () => {
  const filteredDocs = filterDocs(docs);
  if (loading) return <p>Loading...</p>;
  if (filteredDocs.length === 0) return <p>No documents found.</p>;
  return <ul className="doc-list">{filteredDocs.map(renderDocumentCard)}</ul>;
};

```

```

const renderTabs = () => (
  <div className="tabs">
    {["active", "needsAttention", "history"].map((key) => (
      <button key={key} onClick={() => setTab(key)} className={tab === key ? "active" : ""}>
        {key === "active" && "Active Requests"}
        {key === "needsAttention" && "Needs Attention"}
        {key === "history" && "History and Issued Documents"}
      </button>
    ))}
  </div>
);

```

```

    </div>

    );

    return (
      <div className="my-documents">
        <h3> 📄 My Documents</h3>
        {renderTabs()}
        {renderContent()}
      </div>
    );
  };

```

```
export default MyDocuments;
```

```

` ` `

```

```
## frontend\src\components\resident\MyIncidents.js
```

```
` ` ` javascript
```

```

import { useEffect, useState } from "react";
import { collection, query, where, getDocs } from "firebase/firestore";
import { db, auth } from "../../services/firebase"; // adjust path as needed
import "../../styles/admin.css";

```

```

const MyIncidents = () => {
  const [incidents, setIncidents] = useState([]);

```

```
const [loading, setLoading] = useState(true);
```

```
useEffect(() => {
```

```
  const fetchIncidents = async () => {
```

```
    try {
```

```
      const user = auth.currentUser;
```

```
      if (!user) {
```

```
        setIncidents([]);
```

```
        setLoading(false);
```

```
        return;
```

```
      }
```

```
      const q1 = query(collection(db, "incidents"), where("residentId", "==", user.uid));
```

```
      const q2 = query(collection(db, "incidents"), where("authUid", "==", user.uid));
```

```
      const [snap1, snap2] = await Promise.all([getDocs(q1), getDocs(q2)]);
```

```
      const data = [...snap1.docs, ...snap2.docs].map((doc) => ({
```

```
        id: doc.id,
```

```
        ...doc.data(),
```

```
      }));
```

```
      // remove duplicates if any
```

```
      const unique = data.reduce((acc, curr) => {
```

```
        acc[curr.id] = curr;
```

```
        return acc;
```

```
    }, {});  
    setIncidents(Object.values(unique));  
  } catch (err) {  
    console.error("Error fetching incidents:", err);  
  } finally {  
    setLoading(false);  
  }  
};
```

```
fetchIncidents();  
}, []);
```

```
if (loading) return <p>Loading incidents...</p>;
```

```
return (  
  <div className="my-incidents">  
    <h3> 🚨 My Incidents</h3>  
  
    {incidents.length === 0 ? (  
      <p>No incidents reported yet.</p>  
    ) : (  
      <table className="incident-table">  
        <thead>  
          <tr>  
            <th>Type</th>  
            <th>Description</th>
```

```

<th>Status</th>

<th>Assigned To</th>

<th>Location</th>

<th>Reported Date/Time</th>

<th>Last Updated</th>

</tr>

</thead>

<tbody>

{incidents.map((i) => (

  <tr key={i.id}>

    <td>{i.type}</td>

    <td>{i.description}</td>

    <td>{i.status}</td>

    <td>{i.assigned_to_name || "—"}</td>

    <td>{i.location}</td>

    <td>

      {i.date && i.time

        ? ` ${i.date} ${i.time}`

        : i.createdAt

          ? new Date(i.createdAt.seconds * 1000).toLocaleString()

          : "—"}

    </td>

    <td>

      {i.updatedAt

        ? new Date(i.updatedAt.seconds * 1000).toLocaleString()

        : "—"}


```

```

        </td>
      </tr>
    )}
  </tbody>
</table>

)}
</div>

);
};

```

```
export default MyIncidents;
```

```

` ` `

```

```
## frontend\src\components\resident\ResidentBusinessDashboard.js
```

```
` ` ` javascript
```

```

import { useEffect, useState } from "react";
import { collection, query, where, onSnapshot } from "firebase/firestore";
import { db } from "../../services/firebase";
import { useUser } from "../../context/UserContext";
import { QRCodeCanvas } from "qrcode.react";
import ResidentBusinessPayment from "../ResidentBusinessPayment";
import "../../styles/resident/resident-business-dashboard.css";

```

```
const ResidentBusinessDashboard = () => {
```

```
const { userInfo: user } = useUser();  
const [businesses, setBusinesses] = useState([]);  
const [loading, setLoading] = useState(true);  
const [activeTab, setActiveTab] = useState("InProgress");
```

```
useEffect(() => {  
  if (!user?.email) {  
    setBusinesses([]);  
    setLoading(false);  
    return;  
  }  
}
```

```
  setLoading(true);  
  const q = query(collection(db, "businesses"), where("email", "=", user.email));
```

```
  const unsubscribe = onSnapshot(  
    q,  
    (snapshot) => {  
      const data = snapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));  
      setBusinesses(data);  
      setLoading(false);  
    },  
    (error) => {  
      console.error(" ❌ Firebase fetch error:", error);  
      setLoading(false);  
    }  
  );
```

```
);
```

```
return () => unsubscribe();
```

```
}, [user]);
```

```
const renderStatus = (b) => {
```

```
  const status = b.status || b.paymentStatus || "pending_evaluation";
```

```
  if (status === "approved") {
```

```
    const approvedDate = b.submittedAt ? new Date(b.submittedAt) : new Date();
```

```
    const validUntil = new Date(approvedDate);
```

```
    validUntil.setFullYear(validUntil.getFullYear() + 1);
```

```
    return (
```

```
      <div className="qr-wrapper">
```

```
        <p><strong>Permit Number:</strong> {b.permitNumber}</p>
```

```
        <QRCodeCanvas value={b.businessId || b.id} size={96} />
```

```
        <p><strong>Valid Until:</strong> {validUntil.toLocaleDateString()}</p>
```

```
      </div>
```

```
    );
```

```
  }
```

```
  switch (status) {
```

```
    case "pending_evaluation":
```

```
      return <p className="pending-text"> ⌚ Application submitted. Awaiting staff  
evaluation.</p>;
```



```

case "for_payment":
case "awaiting_payment":
return (
  <div className="payment-section">
    <p>🏠 Your application is ready for payment.</p>
    <ResidentBusinessPayment business={b} />
  </div>
);
case "payment_submitted":
  return <p className="info-text">⌚ Payment submitted. Awaiting staff
verification.</p>;
case "rejected":
  return (
    <p className="warning-text">
      ❌ Application was rejected. Notes: {b.notes || "No reason provided."}
    </p>
  );
default:
  return <p className="info-text">ℹ️ Status: {status}</p>;
}
};

```

```

const renderDocuments = (b) => {
  const docs = [
    { key: "validId", label: "Valid ID" },
    { key: "proofOfAddress", label: "Proof of Address" },

```

```
{ key: "dtiCert", label: "DTI Certificate" },
{ key: "businessLogo", label: "Business Logo" },
];
```

```
return (
  <div className="documents-list">
    {docs.map(
      ({ key, label }) =>
        b[key] && (
          <p key={key}>
            <a href={b[key]} target="_blank" rel="noopener noreferrer">
              📄 View {label}
            </a>
          </p>
        )
      )}
  </div>
);
};
```

```
const approvedBusinesses = businesses.filter((b) => b.status === "approved");
const inProgressBusinesses = businesses.filter((b) => b.status !== "approved");
const listToRender = activeTab === "inProgress" ? inProgressBusinesses :
approvedBusinesses;

return (
```

```
<div className="resident-business-dashboard">
```

```
<h2> 🏢 My Business Applications</h2>
```

```
<div className="tabs">
```

```
<button
```

```
  className={activeTab === "InProgress" ? "active" : ""}
```

```
  onClick={() => setActiveTab("InProgress")}
```

```
>
```

```
  ⌚ In Progress
```

```
</button>
```

```
<button
```

```
  className={activeTab === "approved" ? "active" : ""}
```

```
  onClick={() => setActiveTab("approved")}
```

```
>
```

```
  ✅ Approved
```

```
</button>
```

```
</div>
```

```
{loading ? (
```

```
<p>Loading your businesses...</p>
```

```
) : listToRender.length === 0 ? (
```

```
<p>No businesses found in this tab.</p>
```

```
): (
```

```
<div className="business-grid">
```

```
  {listToRender.map((b) => (
```

```
    <div
```

```
key={b.id}

className={`business-card ${b.status === "approved" ? "approved-card" : ""}`}

>

{b.status === "approved" && (

  <div className="approved-badge">APPROVED  </div>

)}
```

```
<h3>{b.businessName}</h3>
```

```
<div className="card-section">

  <p><strong>Type:</strong> {b.businessType}</p>

  <p><strong>Barangay:</strong> {b.barangay}</p>

  <p>

    <strong>Address:</strong>{" "}

    {[b.street, b.barangay, b.city, b.province].filter(Boolean).join(", ")}

  </p>

</div>
```

```
<div className="card-section">

  {renderDocuments(b)}

</div>
```

```
<div className="card-section">

  {renderStatus(b)}

</div>

</div>
```

```

    )}
  </div>

  )}
</div>

);

};

export default ResidentBusinessDashboard;

` ``

## frontend\src\components\resident\ResidentBusinessPayment.js

` `` `javascript
import { useUser } from "../../context/UserContext";

const ResidentBusinessPayment = ({ business }) => {
  const { userInfo: user } = useUser();

  const handlePayment = async (method = "gcash") => {
    try {
      const identifier = business.businessId || business.id;

      // Step 1: Request backend to create a PayMongo Payment Link or Intent
      const res = await fetch(`/api/paymongo/create-business-link`, {
        method: "POST",

```

```
headers: { "Content-Type": "application/json" },
body: JSON.stringify({
  businessId: identifier,
  businessType: business.businessType,
  feeType: "registrationFee",
  remarks: "Business registration fee",
}),
});
```

```
if (!res.ok) {
  const errText = await res.text();
  console.error(" ❌ Payment API error:", errText);
  alert("Failed to initiate payment.");
  return;
}
```

```
const data = await res.json();
```

```
// ✅ Payment Link flow
```

```
if (data.checkoutUrl) {
  window.open(data.checkoutUrl, "_blank", "noopener,noreferrer");
  return;
}
```

```
// ✅ Payment Intent flow
```

```
if (data.paymentIntentId && data.paymongoClientKey) {
```

```
const attachRes = await fetch(`/api/paymongo/attach-payment-method`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    paymentIntentId: data.paymentIntentId,
    paymongoClientKey: data.paymongoClientKey,
    method,
    billing: {
      name: user?.name || "Resident",
      email: user?.email || "resident@example.com",
    },
    type: "business"
  }),
});
```

```
if (!attachRes.ok) {
  const errText = await attachRes.text();
  console.error("✖ Attach method error:", errText);
  alert("Failed to attach payment method.");
  return;
}
```

```
const attachData = await attachRes.json();
const redirectUrl = attachData?.redirectUrl;
```

```
if (redirectUrl) {
```

```

        window.open(redirectUrl, "_blank", "noopener,noreferrer");
    } else {
        alert("Payment method attached but no redirect URL returned.");
    }
    return;
}

// If neither flow returned usable data
alert("No valid payment link or intent returned.");
} catch (err) {
    console.error("❌ Payment flow error:", err);
    alert("Unexpected error during payment.");
}
};

return (
    <div className="resident-business-payment">
        <p>🇹🇼 Choose a payment method:</p>
        <button className="pay-btn" onClick={() => handlePayment("gcash")}>
            Pay with GCash
        </button>
        <button className="pay-btn" onClick={() => handlePayment("grab_pay")}>
            Pay with GrabPay
        </button>
    </div>
);

```



```
};
```

```
export default ResidentBusinessPayment;
```

```
````
```

```
frontend\src\components\resident\ResidentDocumentRequestForm.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";
```

```
import { api, endpoints, BusinessesAPI } from "../../services/api";
```

```
import { usePublicFees } from "../../hooks/usePublicFees";
```

```
import documentConfig from "../../config/documentConfig";
```

```
import { resolveLocation } from "../../utils/resolveLocation";
```

```
import { useNavigate } from "react-router-dom";
```

```
import "../../styles/resident/resident-document-form.css";
```

```
import { PARANAQUE } from "../../data/locations";
```

```
const ResidentDocumentRequestForm = ({ residentId = "", residentName = "",  
onRequestSubmitted }) => {
```

```
  const { documentTypes, loading, error } = usePublicFees();
```

```
  const navigate = useNavigate();
```

```
  const [residentBusinesses, setResidentBusinesses] = useState([]);
```

```
  const [formData, setFormData] = useState({
```

```
    resident_id: residentId,
```

```
    document_type: "",
```

```
purpose: "",
remarks: "",
fee: 0,
complainant: residentName || "", // for Blotter Report autoFill
businessName: "", // for Business Clearance select
"location.barangay": "",
"location.street": "",
"location.city": PARANAQUE.city,
"location.province": PARANAQUE.province
});
```

```
const [attachments, setAttachments] = useState({});
const [status, setStatus] = useState({ message: null, type: null });
```

```
useEffect(() => {
  if (residentName) {
    BusinessesAPI.listByOwner(residentName)
      .then(data => setResidentBusinesses(data))
      .catch(err => console.error(" ❌ Failed to fetch resident businesses:", err));
  }
}, [residentName]);
```

```
useEffect(() => {
  if (documentTypes.length > 0) {
```

```
const first = documentTypes[0];

setFormData(prev => ({
  ...prev,
  document_type: first.documentType,
  fee: first.totalFee,
}));
}, [documentTypes]);
```

```
const handleChange = e => {
  const { name, value } = e.target;

  if (name === "document_type") {
    const selected = documentTypes.find(dt => dt.documentType === value);
    setFormData(prev => ({
      ...prev,
      document_type: value,
      fee: selected?.totalFee || 0,
      complainant: value === "Blotter Report" ? residentName || "" : prev.complainant
    }));
  } else if (name === "businessName") {
    const business = residentBusinesses.find(b => b.businessName === value);
    if (business) {
      setFormData(prev => ({
        ...prev,
        businessName: value,
```

```
    "location.street": business.street || "",
    "location.barangay": business.barangay || "",
    "location.city": business.city || "",
    "location.province": business.province || ""
  }));
} else {
  setFormData(prev => ({ ...prev, businessName: value }));
}
} else {
  setFormData(prev => ({ ...prev, [name]: value }));
}
};
```

```
const handleFileChange = (e, field) => {
  const file = e.target.files[0];
  setAttachments(prev => ({ ...prev, [field]: file }));
};
```

```
const validateForm = () => {
  const config = documentConfig[formData.document_type] || {};
  const rules = config.fields || [];
  const attachmentRules = config.attachments || [];

  for (const field of rules) {
    const value = formData[field.name];
```

```
if (field.required && !value) return `${field.label} is required.`;

if (field.minLength && value?.length < field.minLength)

  return `${field.label} must be at least ${field.minLength} characters.`;

if (field.min !== undefined && Number(value) < field.min)

  return `${field.label} must be at least ${field.min}.`;

}
```

```
for (const att of attachmentRules) {

  const file = attachments[att.name];

  if (att.required && !file) return `${att.label} is required.`;

}
```

```
if (formData.document_type === "Business Clearance") {

  const loc = resolveLocation(formData.document_type, formData, [],
residentBusinesses);

  if (!loc.address) {

    return "Business address is required.";

  }

}
```

```
return null;

};
```

```
const handleSubmit = async e => {

  e.preventDefault();

  const errorMsg = validateForm();
```

```
if (errorMsg) {  
  setStatus({ message: ` ❌ ${errorMsg}`, type: "error" });  
  return;  
}
```

```
setStatus({ message: "Submitting request...", type: "loading" });
```

```
try {  
  const payload = new FormData();  
  Object.entries(formData).forEach(([key, value]) => payload.append(key, value));
```

```
// ♦ Normalize location fields
```

```
const loc = resolveLocation(formData.document_type, formData, [],  
residentBusinesses);
```

```
payload.append("locationBarangay", loc.barangay);  
payload.append("locationStreet", loc.street);  
payload.append("locationCity", loc.city);  
payload.append("locationProvince", loc.province);  
payload.append("address", loc.address); // ✅ backend requires this
```

```
if (formData.document_type !== "Barangay Clearance") {  
  if (loc.barangay) payload.append("locationBarangay", loc.barangay);  
  if (loc.street) payload.append("locationStreet", loc.street);  
  if (loc.city) payload.append("locationCity", loc.city);  
  if (loc.province) payload.append("locationProvince", loc.province);
```

```
const fullAddress = [loc.street, `Brgy. ${loc.barangay}`, loc.city, loc.province]
    .filter(Boolean)
    .join(", "); payload.append("address", fullAddress);
}
```

```
// ♦ Attach files
```

```
const attachmentRules = documentConfig[formData.document_type]?.attachments ||
[];
attachmentRules.forEach(att => {
    if (attachments[att.name]) payload.append(att.name, attachments[att.name]);
});
```

```
await api.post(endpoints.documents || "/documents", payload, {
    headers: { "Content-Type": "multipart/form-data" },
});
```

```
setStatus({ message: "✅ Request submitted. Awaiting secretary validation.", type:
"success" });
```

```
onRequestSubmitted?.();
```

```
// Reset form
```

```
setFormData({
    resident_id: residentId,
    document_type: documentTypes.length > 0 ? documentTypes[0].documentType : "",
    purpose: "",
```

```

    remarks: "",
    fee: documentTypes.length > 0 ? documentTypes[0].totalFee : 0,
    complainant: residentName || "",
    businessName: "",
    "location.barangay": "",
    "location.street": "",
    "location.city": "",
    "location.province": ""
  });
  setAttachments({});

  setTimeout(() => { navigate("/ownDocuments"); }, 1500); // 1500ms = 1.5 seconds
} catch (err) {
  const msg = err.response?.data?.detail || err.message;
  console.error("✖ Error submitting request:", msg);
  setStatus({ message: "✖ Failed to submit request.", type: "error" });
}
};

return (
  <div className="resident-document-form">
    <h2> 📄 Request a Barangay Document</h2>

    {loading && <p>Loading document types...</p>}

    {error && <p className="status-message error"> ✖ {error}</p>}}
  </div>
);

```



```

<form onSubmit={handleSubmit}>

  {/* Document type */}

  <div className="form-group">

    <label htmlFor="document_type">Document Type</label>

    <select

      name="document_type"

      value={formData.document_type}

      onChange={handleChange}

      required

    >

      {documentTypes.length > 0 ? (

        documentTypes.map(dt => (

          <option key={dt.id} value={dt.documentType}>

            {dt.documentType} — ₪{dt.totalFee}

          </option>

        ))

      ) : (

        <option disabled>Loading types...</option>

      )}

    </select>

  </div>

  {/* Dynamic fields */}

  {documentConfig[formData.document_type]?.fields?.map(field => {

    if (field.type === "group") {

      return (

```

```

<fieldset key={field.label}>

  <legend>{field.label}</legend>

  {field.fields.map(sub => (

    <div className="form-group" key={sub.name}>

      <label htmlFor={sub.name}>{sub.label}</label>

      {sub.type === "select" ? (

        <select id={sub.name} name={sub.name} value={formData[sub.name] || ""}
onChange={handleChange} required={sub.required} >

          <option value="">-- Select --</option>

          {sub.options?.map(opt => (

            <option key={opt} value={opt}>{opt}</option>

          ))}

        </select>

      ) : (

        <input

          id={sub.name}

          type={sub.type}

          name={sub.name}

          value={formData[sub.name] || sub.default || ""}

          onChange={handleChange}

          required={sub.required}

          readOnly={sub.readOnly}

        />

      )}

    </div>

  ))}

```

```
</fieldset>
```

```
);
```

```
}
```

```
if (field.autoFill) {
```

```
  return (
```

```
    <div className="form-group" key={field.name}>
```

```
      <label htmlFor={field.name}>{field.label}</label>
```

```
      <input
```

```
        id={field.name}
```

```
        type={field.type}
```

```
        name={field.name}
```

```
        value={formData.complainant}
```

```
        readOnly
```

```
      />
```

```
    </div>
```

```
  );
```

```
}
```

```
if (field.type === "select" && field.name === "businessName") {
```

```
  return (
```

```
    <div className="form-group" key={field.name}>
```

```
      <label htmlFor={field.name}>{field.label}</label>
```

```
      <select
```

```
        id={field.name}
```

```
        name={field.name}
```

```

value={formData[field.name] || ""}
onChange={handleChange}
required={field.required}
>
<option value="">Select your business</option>
{residentBusinesses.map(biz => (
  <option key={biz.businessId} value={biz.businessName}>
    {biz.businessName} — {biz.businessType}
  </option>
))}
</select>

```

```

{/* ♦ Render auto-filled address fields right after business select */}

```

```

{formData.document_type === "Business Clearance" && formData.businessName
&& (

```

```

<fieldset>
  <legend>Business Address</legend>
  <div className="form-group">
    <label>Street</label>
    <input type="text" value={formData["location.street"]} readOnly />
  </div>
  <div className="form-group">
    <label>Barangay</label>
    <input type="text" value={formData["location.barangay"]} readOnly />
  </div>
  <div className="form-group">

```

```

        <label>City</label>

        <input type="text" value={formData["location.city"]} readOnly />
    </div>

    <div className="form-group">

        <label>Province</label>

        <input type="text" value={formData["location.province"]} readOnly />
    </div>

    { /* Hidden full address field for submission */}

    <input
        type="hidden"
        name="address"
        value={[
            formData["location.street"],
            ` Brgy. ${formData["location.barangay"]}`,
            formData["location.city"],
            formData["location.province"]
        ].filter(Boolean).join(", ")}
    />

    </fieldset>

    )}

</div>

);

}

```

```
return (  
  <div className="form-group" key={field.name}>  
    <label htmlFor={field.name}>{field.label}</label>  
    {field.type === "textarea" ? (  
      <textarea  
        id={field.name}  
        name={field.name}  
        value={formData[field.name] || ""}  
        onChange={handleChange}  
        required={field.required}  
      />  
    ) : (  
      <input  
        id={field.name}  
        type={field.type}  
        name={field.name}  
        value={formData[field.name] || ""}  
        onChange={handleChange}  
        required={field.required}  
      />  
    )}  
  </div>  
);  
}}}
```

```

    { /* Dynamic attachments */
    {documentConfig[formData.document_type]?.attachments?.map(att => (
      <div className="form-group" key={att.name}>
        <label htmlFor={att.name}>
          {att.label} {att.required && <span className="required">*</span>}
        </label>
        <input
          id={att.name}
          type="file"
          accept=".jpg,.jpeg,.png,.pdf"
          required={att.required}
          onChange={e => handleFileChange(e, att.name)}
        />
      </div>
    ))}

    <button type="submit" className="submit-btn">
      Submit Request ({formData.fee})
    </button>
  </form>

  {status.message && (
    <p className={`status-message ${status.type}`}>{status.message}</p>
  )}
</div>

);

```

```
};
```

```
export default ResidentDocumentRequestForm;
```

```
````
```

```
frontend\src\components\resident\ResubmissionForm.js
```

```
```javascript
```

```
import { useState } from "react";
```

```
import { api } from "../../services/api";
```

```
import "../../styles/resident/resubmission-form.css";
```

```
const ResubmissionForm = ({ doc, onSuccess }) => {  
  const [purpose, setPurpose] = useState(doc.purpose || "");  
  const [documentType] = useState(doc.document_type);  
  const [idAttachment, setIdAttachment] = useState(null);  
  const [residencyAttachment, setResidencyAttachment] = useState(null);  
  const [remarks, setRemarks] = useState("");  
  const [loading, setLoading] = useState(false);  
  const [success, setSuccess] = useState(false);  
  const [error, setError] = useState("");
```

```
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    setLoading(true);
```



```
setError("");

try {
  const formData = new FormData();
  formData.append("resident_id", doc.resident_id);
  formData.append("document_type", documentType);
  formData.append("purpose", purpose);
  formData.append("remarks", remarks);
  if (idAttachment) formData.append("idAttachment", idAttachment);
  if (residencyAttachment) formData.append("residencyAttachment",
residencyAttachment);

  // Create new document
  await api.post("/api/documents", formData, {
    headers: { "Content-Type": "multipart/form-data" },
  });

  // Mark old rejected document as handled
  if (doc.status === "rejected") {
    await api.patch(`/api/documents/${doc.id}/resubmission`, { resubmitted: true });
  }

  setSuccess(true);

  // Redirect after short delay
  if (onSuccess) {
    setTimeout(() => onSuccess(), 1500);
  }
}
```

```

    }
  } catch (err) {
    console.error(" ❌ Error resubmitting document:", err.response?.data?.detail ||
err.message);

    setError(err.response?.data?.detail || "Failed to resubmit document.");
  } finally {
    setLoading(false);
  }
};

return (
  <div className="resubmission-form">

    <h3> ⬅️ Resubmit Document</h3>

    {success ? (
      <p className="success-message"> ✅ Document resubmitted successfully!
Redirecting...</p>
    ) : (
      <form onSubmit={handleSubmit}>

        <div className="form-group">

          <label>Document Type</label>

          <input type="text" value={documentType} disabled />

        </div>

        <div className="form-group">

          <label>Purpose</label>

```

```
<input
  type="text"
  value={purpose}
  onChange={(e) => setPurpose(e.target.value)}
  required
/>
</div>
```

```
<div className="form-group">
  <label>Remarks (optional)</label>
  <textarea
    value={remarks}
    onChange={(e) => setRemarks(e.target.value)}
    placeholder="Add any notes for secretary"
  />
</div>
```

```
<div className="form-group">
  <label>Upload Valid ID</label>
  <input
    type="file"
    accept="image/*,.pdf"
    onChange={(e) => setIdAttachment(e.target.files[0])}
    required
  />
</div>
```

```
<div className="form-group">
  <label>Upload Proof of Residency</label>
  <input
    type="file"
    accept="image/*,.pdf"
    onChange={(e) => setResidencyAttachment(e.target.files[0])}
    required
  />
</div>
```

```
{error && <p className="error-message"> ❌ {error}</p>}
```

```
<button type="submit" className="btn-submit" disabled={loading}>
  {loading ? "Submitting..." : "Resubmit"}
</button>
```

```
</form>
```

```
  )}
```

```
</div>
```

```
);
```

```
};
```

```
export default ResubmissionForm;
```

```
...
```

```
## frontend\src\components\resident\ResubmissionPage.js
```

```
` `` `javascript
```

```
import { useEffect, useState } from "react";
```

```
import { useParams, useNavigate } from "react-router-dom";
```

```
import { api } from "../../services/api";
```

```
import ResubmissionForm from "../ResubmissionForm";
```

```
const ResubmissionPage = () => {
```

```
  const { docId } = useParams();
```

```
  const navigate = useNavigate();
```

```
  const [doc, setDoc] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState("");
```

```
  useEffect(() => {
```

```
    const fetchDoc = async () => {
```

```
      try {
```

```
        const res = await api.get(`/api/documents/${docId}`);
```

```
        setDoc(res.data);
```

```
      } catch (err) {
```

```
        console.error(" ❌ Error fetching document:", err.response?.data?.detail ||  
err.message);
```

```
        setError(err.response?.data?.detail || "Failed to load document.");
```

```
      } finally {
```

```
        setLoading(false);
```

```
    }  
  };  
  fetchDoc();  
}, [docId]);
```

```
if (loading) return <p className="loading-message"> ⌚ Loading document...</p>;
```

```
if (error) return <p className="error-message"> ❌ {error}</p>;
```

```
if (!doc) return <p className="error-message"> ❌ Document not found.</p>;
```

```
return (  
  <div className="resubmission-page">  
    <header className="resubmission-header">  
      <h2> ⬅️ END Resubmission for <span>{doc.document_type}</span></h2>  
      <div className="doc-meta">  
        <p><strong>Purpose:</strong> {doc.purpose || "—"}</p>  
        <p><strong>Status:</strong> {doc.status}</p>  
        {doc.remarks && <p className="doc-remarks">Remarks: {doc.remarks}</p>}  
      </div>  
    </header>  
  
    <ResubmissionForm  
      doc={doc}  
      onSuccess={() => navigate("/my-documents")}  
    />  
  </div>  
>);
```

```
};
```

```
export default ResubmissionPage;
```

```
```\n
```

```
frontend\\src\\components\\resident\\StatusTracker.txt
```

```
```text
```

```
import "../styles/admin.css";
```

```
const mockStatuses = [
```

```
  { id: 1, type: "Document", name: "Barangay Clearance", status: "Ready for pickup" },
```

```
  { id: 2, type: "Complaint", name: "Flooding", status: "Under review" },
```

```
];
```

```
const StatusTracker = () => (
```

```
  <div className="status-tracker">
```

```
    <h3> 📌 Status Tracker</h3>
```

```
    <ul>
```

```
      {mockStatuses.map((item) => (
```

```
        <li key={item.id}>
```

```
          <strong>{item.name}</strong> ({item.type}) — {item.status}
```

```
        </li>
```

```
      )}}
```

```
    </ul>
```

```

</div>

);

export default StatusTracker;

` ``

## frontend\src\components\ResidentRegistry.js

` `` javascript

import React, { useState } from "react";
import ResidentForm from "../forms/ResidentForm";
import ResidentList from "../lists/ResidentList";
import { useResidents } from "../hooks/useResidents";
import { useOutletContext } from "react-router-dom";
import "../components/forms/resident-form.css";

const ResidentRegistry = () => {
  const [showForm, setShowForm] = useState(false);
  const { user } = useOutletContext();

  //  use custom hook for residents

  const { residents, loading, fetchResidents, updateResident, deleteResident } =
    useResidents();

  //  Handle new resident added

```



```
const handleResidentAdded = async () => {  
  await fetchResidents();  
  setShowForm(false);  
};
```

```
const handleCancel = () => setShowForm(false);
```

```
// 🗝️ Role flags
```

```
const canAdd = user?.role === "staff" || user?.role === "admin";  
const canEdit = user?.role === "staff" || user?.role === "admin";  
const canDelete = user?.role === "admin";
```

```
return (  
  <div className="resident-registry">  
    {showForm ? (  
      <ResidentForm  
        user={user}  
        onResidentAdded={handleResidentAdded}  
        onCancel={handleCancel}  
      />  
    ) : (  
      <>  
        <div className="registry-header">  
          {canAdd && (  
            <>  
              <h2>Resident Registry</h2>
```

```

        <button onClick={() => setShowForm(true)}> + Add Resident</button>

    </>

  )}

</div>

<ResidentList
  residents={residents}
  loading={loading}
  canEdit={canEdit}
  canDelete={canDelete}
  onUpdate={updateResident}
  onDelete={deleteResident}
  fetchResidents={fetchResidents}
/>

{!loading && residents.length === 0 && (
  <p className="empty-state">No residents found. Add the first one + </p>
)}

</>

)}

</div>

);

};

export default ResidentRegistry;

```

```

## frontend\src\components\secretary\DocumentTypeChart.js

```javascript

import { useMemo } from "react";

import { Bar } from "react-chartjs-2";

import {

Chart as ChartJS,

CategoryScale,

LinearScale,

BarElement,

Title,

Tooltip,

Legend,

} from "chart.js";

ChartJS.register(CategoryScale, LinearScale, BarElement, Title, Tooltip, Legend);

const DocumentTypeChart = ({ documents = [], counters = [] }) => {

const { labels, counts } = useMemo(() => {

// Case 1: Use documents if available

if (documents && documents.length > 0) {

const map = {};

documents.forEach((doc) => {

const type = doc.document_type || doc.documentType;

```
    if (!type) return;
    map[type] = (map[type] || 0) + 1;
  });
  return { labels: Object.keys(map), counts: Object.values(map) };
}
```

```
// Case 2: Fallback to counters if documents are empty
if (counters && counters.length > 0) {
  const labels = counters.map((c) => c.id || c.documentType);
  const counts = counters.map((c) => c.last_number || 0);
  return { labels, counts };
}
```

```
// Case 3: No data
return { labels: [], counts: [] };
}, [documents, counters]);
```

```
const data = {
  labels,
  datasets: [
    {
      label: "Requests",
      data: counts,
      backgroundColor: "#3498db",
    },
  ],
},
```

```
};
```

```
const options = {  
  responsive: true,  
  plugins: {  
    legend: { position: "top" },  
    title: { display: true, text: "Document Types Requested" },  
  },  
};
```

```
return <Bar data={data} options={options} />;  
};
```

```
export default DocumentTypeChart;
```

```
````
```

```
frontend\src\components\secretary\IssuedDocuments.js
```

```
```javascript
```

```
import { useState } from "react";  
import { useEnrichedRequests } from "../../hooks/useEnrichedRequests";  
import "../../styles/secretary/issued-documents.css";
```

```
const IssuedDocuments = () => {  
  const { approved, loading } = useEnrichedRequests();
```

```
const [selectedDoc, setSelectedDoc] = useState(null);
```

```
const renderDocumentCard = (doc) => (
```

```
<li key={doc.id} className="request-card">
```

```
<div className="request-info">
```

```
<strong>{doc.residentName || doc.residentId}</strong>
```

```
<span>{doc.documentType}</span>
```

```
<small className="doc-id">{doc.documentId}</small>
```

```
</div>
```

```
<button className="btn-view" onClick={() => setSelectedDoc(doc)}>👁 View</button>
```

```
</li>
```

```
);
```

```
return (
```

```
<div className="issued-documents">
```

```
<h3>📄 Issued Documents</h3>
```

```
{loading && <p className="status-message">Loading...</p>}
```

```
{approved.length === 0 ? (
```

```
<p className="status-message">No issued documents yet.</p>
```

```
): (
```

```
<ul className="request-list">{approved.map(renderDocumentCard)}</ul>
```

```
))
```

```
{selectedDoc && (
```

```
<div className="modal-overlay" onClick={() => setSelectedDoc(null)}>
```

```
<div className="modal-content" onClick={(e) => e.stopPropagation()}>

  <header>

    <h4>{selectedDoc.residentName || selectedDoc.residentId}</h4>

    <p className="doc-type">{selectedDoc.documentType}</p>

    <p className="doc-id">Document ID: {selectedDoc.documentId}</p>

  </header>
```

```
<section className="doc-meta">

  <p><strong>Purpose:</strong> {selectedDoc.purpose || "—"}</p>

  <p><strong>Issued By:</strong> {selectedDoc.issuedBy || "—"}</p>

  <p><strong>Issued At:</strong> {selectedDoc.issuedAt || "—"}</p>

  <p><strong>Status:</strong> {selectedDoc.status}</p>

</section>
```

```
<section className="doc-file">

  <h5>Issued File</h5>

  {selectedDoc.fileUrl ? (

    <a

      href={selectedDoc.fileUrl}

      target="_blank"

      rel="noopener noreferrer"

      className="btn-download"

    >

       Download Document

    </a>

  ) : (
```

```
<p>No file uploaded yet.</p>
```

```
  )}
```

```
</section>
```

```
    <button className="btn-close" onClick={() => setSelectedDoc(null)}>
```

```
      Close</button>
```

```
  </div>
```

```
</div>
```

```
  )}
```

```
</div>
```

```
);
```

```
};
```

```
export default IssuedDocuments;
```

```
```
```

```
frontend\src\components\secretary\MonthlyTrendChart.js
```

```
```javascript
```

```
import { useMemo } from "react";
```

```
import { Line } from "react-chartjs-2";
```

```
import {
```

```
  Chart as ChartJS,
```

```
  CategoryScale,
```

```
  LinearScale,
```



```
PointElement,  
LineElement,  
Title,  
Tooltip,  
Legend,  
} from "chart.js";
```

```
ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip,  
Legend);
```

```
const MonthlyTrendChart = ({ documents = [] }) => {  
  // Aggregate counts by month  
  const { labels, counts } = useMemo(() => {  
    const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov",  
"Dec"];  
    const map = Array(12).fill(0);  
  
    documents.forEach((doc) => {  
      const createdAt = doc.created_at || doc.createdAt;  
      if (!createdAt) return;  
  
      const date = new Date(createdAt);  
      const monthIndex = date.getMonth();  
      map[monthIndex] += 1;  
    });  
  
    return { labels: months, counts: map };  
  });
```

```
}, [documents]));
```

```
const data = {  
  labels,  
  datasets: [  
    {  
      label: "Requests",  
      data: counts,  
      borderColor: "#9b59b6",  
      backgroundColor: "rgba(155, 89, 182, 0.2)",  
      tension: 0.3,  
    },  
  ],  
};
```

```
const options = {  
  responsive: true,  
  plugins: {  
    legend: { position: "top" },  
    title: { display: true, text: "Monthly Request Trends" },  
  },  
};
```

```
return <Line data={data} options={options} />;  
};
```

```
export default MonthlyTrendChart;
```

```
`, `
```

```
## frontend\src\components\secretary\PaidRequests.js
```

```
` `` javascript
```

```
import { useState } from "react";
```

```
import { DocumentsAPI } from "../../services/api";
```

```
import { useEnrichedRequests } from "../../hooks/useEnrichedRequests";
```

```
import "../../styles/secretary/paid-requests.css";
```

```
const PaidRequests = () => {
```

```
  const { toVerify, readyToIssue, loading, fetchRequests } = useEnrichedRequests();
```

```
  const [selectedDoc, setSelectedDoc] = useState(null);
```

```
  const [rejectionReason, setRejectionReason] = useState("");
```

```
  const [issueRemarks, setIssueRemarks] = useState("");
```

```
  const userInfo = JSON.parse(sessionStorage.getItem("userInfo") || "{}");
```

```
  const issueDocument = async (firestoreId) => {
```

```
    try {
```

```
      await DocumentsAPI.issue(firestoreId, {
```

```
        issued_by: userInfo?.full_name || userInfo?.uid || "Unknown User",
```

```
        remarks: issueRemarks.trim() || undefined,
```

```
      });
```

```
    setSelectedDoc(null);
    setIssueRemarks("");
    fetchRequests();
  } catch (err) {
    console.error("❌ Error issuing document:", err.message);
  }
};
```

```
const verifyPayment = async (firestoreId) => {
  try {
    await DocumentsAPI.patchStatus(firestoreId, {
      newStatus: "paid",
      remarks: `Payment verified by ${userInfo?.full_name}`,
    });
    setSelectedDoc(null);
    setIssueRemarks("");
    fetchRequests();
  } catch (err) {
    console.error("❌ Error verifying payment:", err.message);
  }
};
```

```
const rejectDocument = async (firestoreId, reason) => {
  try {
    await DocumentsAPI.patchStatus(firestoreId, {
      newStatus: "rejected",
```

```

    remarks: reason,
  });
  setSelectedDoc(null);
  setRejectionReason("");
  setIssueRemarks("");
  fetchRequests();
} catch (err) {
  console.error(" ❌ Error rejecting document:", err.message);
}
};

return (
  <div className="paid-requests">
    {loading && <p className="status-message">Loading...</p>}

    {/* Section 1: Awaiting Verification */}

    <h3> 🇳🇮 Requests Awaiting Verification</h3>
    {toVerify.length === 0 ? (
      <p className="status-message">No requests awaiting verification.</p>
    ) : (
      <ul className="request-list">
        {toVerify.map((doc) => (
          <li key={doc.id} className="request-card">
            <div className="request-info">
              <strong>{doc.resident_name || doc.resident_id}</strong>
              <span>{doc.document_type}</span>

```

```

      <small className="doc-id">{{doc.document_id}}</small>

      <span className="badge-pending">Payment Submitted</span>

    </div>

    <button className="btn-view" onClick={() => setSelectedDoc(doc)}>👁️
View</button>

  </li>

  )}

</ul>

)}

{ /* Section 2: Ready to Issue */ }

<h3>📄 Ready to Issue</h3>

{readyToIssue.length === 0 ? (

  <p className="status-message">No documents ready to issue.</p>

) : (

  <ul className="request-list">

    {readyToIssue.map((doc) => (

      <li key={doc.id} className="request-card">

        <div className="request-info">

          <strong>{{doc.resident_name || doc.resident_id}}</strong>

          <span>{{doc.document_type}}</span>

          <small className="doc-id">{{doc.document_id}}</small>

          {doc.amount === 0 ? (

            <span className="badge-free">No Payment Required</span>

          ) : (

            <span className="badge-paid">Payment Verified</span>

```

```

    })
  </div>

  <button className="btn-view" onClick={() => setSelectedDoc(doc)}>👁️
View</button>

</li>

  )}
</ul>

})

{/* Modal */}

{selectedDoc && (

  <div className="modal-overlay" onClick={() => { setSelectedDoc(null);
setIssueRemarks(""); }}>

    <div className="modal-content" onClick={(e) => e.stopPropagation()}>

      <header>

        <h4>{selectedDoc.resident_name || selectedDoc.resident_id}</h4>

        <p className="doc-type">{selectedDoc.document_type}</p>

        <p className="doc-id">Document ID: {selectedDoc.document_id}</p>

        {selectedDoc.amount === 0 && (

          <p className="doc-free">This is a free document (no payment required).</p>

        )}

      </header>

      <section className="doc-meta">

        <p><strong>Purpose:</strong> {selectedDoc.purpose || "—"}</p>

        <p><strong>Remarks:</strong> {selectedDoc.remarks || "—"}</p>

```

```

<p>
  <strong>Payment Ref:</strong>{" "}
  {selectedDoc.amount > 0
    ? selectedDoc.transactionId || selectedDoc.paymentIntentId || "—"
    : selectedDoc.referenceNumber || selectedDoc.transactionId || "—"}
</p>

<p><strong>Amount:</strong> {selectedDoc.amount === 0 ? "Free" :
`P$${selectedDoc.amount}`}</p>

<p><strong>Status:</strong> {selectedDoc.status}</p>
</section>

<section className="doc-actions">
  {selectedDoc.status === "payment_submitted" && (
    <button className="btn-verify" onClick={() => verifyPayment(selectedDoc.id)}>
       Verify Payment
    </button>
  )}

  {(selectedDoc.status === "paid" || selectedDoc.amount === 0) && (
    <div className="reject-section">
      <textarea
        className="reject-textarea"
        placeholder="Enter issuance remarks (optional)..."
        value={issueRemarks}
        onChange={(e) => setIssueRemarks(e.target.value)}
      />

```



```
      <button className="btn-approve" onClick={() =>
issueDocument(selectedDoc.id)}>
```

 Issue Document

```
    </button>
```

```
  </div>
```

```
  })
```

```
<div className="reject-section">
```

```
  <textarea
```

```
    className="reject-textarea"
```

```
    placeholder="Enter rejection reason..."
```

```
    value={rejectionReason}
```

```
    onChange={(e) => setRejectionReason(e.target.value)}
```

```
  />
```

```
  <button
```

```
    className="btn-reject"
```

```
    onClick={() => {
```

```
      if (!rejectionReason.trim()) {
```

```
        alert("Rejection reason is required.");
```

```
        return;
```

```
      }
```

```
      rejectDocument(selectedDoc.id, rejectionReason.trim());
```

```
    }}
```

```
  >
```

 Reject

```
</button>
```

```
</div>
```

```
      <button className="btn-close" onClick={() => { setSelectedDoc(null);  
setIssueRemarks(""); }}>  Close</button>
```

```
    </section>
```

```
  </div>
```

```
</div>
```

```
  )}
```

```
</div>
```

```
);
```

```
};
```

```
export default PaidRequests;
```

```
...
```

```
## frontend\src\components\secretary\PaymentStatusChart.js
```

```
```javascript
```

```
import { useMemo } from "react";
```

```
import { Pie } from "react-chartjs-2";
```

```
import { Chart as ChartJS, ArcElement, Tooltip, Legend } from "chart.js";
```

```
ChartJS.register(ArcElement, Tooltip, Legend);
```

```
const PaymentStatusChart = ({ documents = [] }) => {
```

```
const { free, paid, awaiting_payment } = useMemo(() => {

 let free = 0;

 let paid = 0;

 let awaiting_payment = 0;

 documents.forEach((doc) => {

 const amount = doc.amount || 0;

 const status = doc.status || doc.documentStatus;

 if (amount === 0) {

 free += 1;

 } else if (status === "paid" || status === "approved") {

 paid += 1;

 } else if (status === "awaiting_payment" || status === "pending") {

 awaiting_payment += 1;

 }

 });

 return { free, paid, awaiting_payment };

}, [documents]);

const data = {

 labels: ["Free", "Paid", "Awaiting Payment"],

 datasets: [

 {

 data: [free, paid, awaiting_payment],
```

```
 backgroundColor: ["#3498db", "#2ecc71", "#f39c12"],
 },
],
};
```

```
const options = {
 responsive: true,
 plugins: {
 legend: { position: "bottom" },
 title: { display: true, text: "Payment Status" },
 },
};
```

```
return <Pie data={data} options={options} />;
};
```

```
export default PaymentStatusChart;
```

```
```
```

```
## frontend\src\components\secretary\PendingRequests.js
```

```
```javascript
```

```
import { useState } from "react";
```

```
import { DocumentsAPI } from "../../services/api";
```

```
import { useEnrichedRequests } from "../../hooks/useEnrichedRequests";
```

```

import RequestModal from "../RequestModal";

import "../../styles/secretary/pending-requests.css";

// ♦ Main Component

const PendingRequests = () => {
 const { pending, loading, fetchRequests } = useEnrichedRequests();
 const [selectedDoc, setSelectedDoc] = useState(null);

 const updateStatus = async (firestoreId, newStatus, remarks = null) => {
 try {
 await DocumentsAPI.patchStatus(firestoreId, { newStatus, remarks });
 setSelectedDoc(null);
 fetchRequests();
 } catch (err) {
 console.error("❌ Error updating document:", err.message);
 alert("Failed to update status. Please try again.");
 }
 };

 return (
 <div className="pending-requests">
 <h3> 📋 Requests to Verify</h3>
 {loading && <p className="status-message">Loading requests...</p>}

 {pending.length === 0 && !loading ? (
 <p className="status-message">No requests awaiting verification.</p>

```

```

): (
 <ul className="request-list">
 {pending.map((doc) => (
 <li key={doc.id} className="request-card">
 <div className="request-info">
 {doc.residentName || doc.residentId}
 {doc.documentType}
 <small className="doc-id">{{doc.documentId}}</small>
 </div>
 <button className="btn-view" onClick={() => setSelectedDoc(doc)}>
  View
 </button>

))}

)}

```

```

{selectedDoc && (
 <RequestModal
 doc={selectedDoc}
 onClose={() => setSelectedDoc(null)}
 onUpdateStatus={updateStatus}
 />
)}
</div>
);

```

```
};
```

```
export default PendingRequests;
```

```
```\n
```

```
## frontend\\src\\components\\secretary\\RejectedRequests.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
```

```
import { api } from "../../services/api";
```

```
const RejectedRequests = () => {
```

```
 const [rejected, setRejected] = useState([]);
```

```
 const [loading, setLoading] = useState(true);
```

```
 useEffect(() => {
```

```
 const fetchRejected = async () => {
```

```
 try {
```

```
 const { data } = await api.get("/api/documents", {
```

```
 params: { status: "rejected" },
```

```
 });
```

```
 setRejected(data);
```

```
 } catch (err) {
```

```
 console.error(" ❌ Error fetching rejected requests:", err.message);
```

```
 } finally {
```

```
 setLoading(false);
 }
};
fetchRejected();
, []);
```

```
if (loading) {
 return (
 <div className="sidebar-section">
 <h3> ❌ Rejected Requests</h3>
 <p>Loading rejected requests...</p>
 </div>
);
}
```

```
return (
 <div className="sidebar-section">
 <h3> ❌ Rejected Requests</h3>
 {rejected.length === 0 ? (
 <p>No rejected requests.</p>
) : (
 <ul className="rejected-list">
 {rejected.map((doc) => (
 <li key={doc.id} className="rejected-card">
 <div className="rejected-info">
 {doc.resident_name || doc.resident_id}
```



```

 {doc.document_type}
 </div>
 <p className="rejection-reason">
 Reason: {doc.rejection_reason || "Not specified"}
 </p>

)}

)}
</div>
);
};

```

```
export default RejectedRequests;
```

```

` ` `

```

```
frontend\src\components\secretary\RequestModal.js
```

```
` ` ` javascript
```

```
import { useState } from "react";
```

```
import documentConfig from "../../config/documentConfig";
```

```
const RequestModal = ({ doc, onClose, onUpdateStatus }) => {
```

```
 const [rejectionReason, setRejectionReason] = useState("");
```

```
const handleReject = () => {
 if (!rejectionReason.trim()) {
 alert("Rejection reason is required.");
 return;
 }
 onUpdateStatus(doc.id, "rejected", rejectionReason.trim());
};
```

```
// ♦ Get config for this document type
```

```
const config = documentConfig[doc.document_type] || {};
```

```
return (
 <div className="modal-overlay" onClick={onClose}>
 <div className="modal-content" onClick={(e) => e.stopPropagation()}>
 <header>
 <h4>{doc.resident_name || doc.resident_id}</h4>
 <p className="doc-type">{doc.document_type}</p>
 <p className="doc-id">Document ID: {doc.document_id}</p>
 </header>
```

```
 { /* ♦ Details Section */ }
```

```
 {config.fields && (
 <section className="doc-details">
 <h5>Details</h5>

 {config.fields.map((field) => {
```

```

if (field.type === "group" && field.fields) {
 return (
 <li key={field.label}>
 {field.label}

 {field.fields.map((subField) => {
 let value;
 if (typeof subField.name === "string" && subField.name.includes(".")) {
 value = subField.name.split(".").reduce(
 (acc, key) => acc?.[key],
 { ...doc, ...doc.extraFields }
);
 } else {
 value = doc[subField.name] ?? doc.extraFields?.[subField.name];
 }
 return value ? (
 <li key={subField.name}>
 {subField.label} {value}

) : null;
 })}

);
}

```

```

// normal field
if (!field?.name) return null;
let value;
if (typeof field.name === "string" && field.name.includes(".")) {
 value = field.name.split(".").reduce(
 (acc, key) => acc?.[key],
 { ...doc, ...doc.extraFields }
);
} else {
 value = doc[field.name] ?? doc.extraFields?.[field.name];
}

return value ? (
 <li key={field.name}>
 {field.label} {value}

) : null;
}}

</section>
)}

```

```

{/* ♦ Attachments Section */}
{config.attachments && doc.attachments && (
 <section className="attachments">
 <h5>Attachments</h5>

```

```

```

```
{config.attachments.map((att) => {
```

```
 const file = doc.attachments[att.name];
```

```
 if (!file) return null;
```

```
 return (
```

```
 <li key={att.name}>
```

```
 {att.label}{" "
```

```
 {file.url.match(/\.(jpg|jpeg|png|gif)$/i) ? (
```

```
 <div>
```

```
 <img
```

```
 src={file.url}
```

```
 alt={att.label}
```

```
 style={{ maxWidth: "200px", display: "block", marginTop: "5px" }}
```

```
 />
```

```
 <small style={{ wordBreak: "break-all" }}>
```

```
 URL: {file.url}
```

```
 </small>
```

```
 </div>
```

```
) : (
```

```
 <div>
```

```

```

```
 View File
```

```

```

```
 <small style={{ wordBreak: "break-all" }}>
```

```
 URL: {file.url}
```

```
 </small>
 </div>
 })

);
}}

</section>
})
```

```
{/* ♦ Actions Section */}
```

```
<section className="doc-actions">
 <button
 className="btn-awaiting"
 onClick={() => onUpdateStatus(doc.id, "for_payment")}
 >
  For Payment
 </button>
```

```
<div className="reject-section">
 <textarea
 className="reject-textarea"
 placeholder="Enter rejection reason..."
 value={rejectionReason}
 onChange={(e) => setRejectionReason(e.target.value)}
 />
```

```

 <button className="btn-reject" onClick={handleReject}>
 ✖ Reject
 </button>
 </div>

 <button className="btn-close" onClick={onClose}>
 ⬅ Close
 </button>
</section>
</div>
</div>

);
};

```

```
export default RequestModal;
```

```

` ` `

```

```
frontend\src\components\secretary\SummaryCards.js
```

```
` ` ` javascript
```

```
import "../styles/secretary/summary-cards.css";
```

```
const SummaryCards = ({ stats }) => {
```

```
 const cards = [
```

```
 { key: "total", label: "Total Requests" },
```

```
{ key: "pending", label: "Pending" },
{ key: "for_payment", label: "For Payment" },
{ key: "approved", label: "Approved" },
{ key: "rejected", label: "Rejected" },
];
```

```
return (
 <div className="summary-cards">
 {cards.map(({ key, label }) => (
 <div className="card" key={key}>
 <h3>{stats[key] ?? 0}</h3>
 <p>{label}</p>
 </div>
))}
 </div>
);
};
```

```
export default SummaryCards;
```

```
`, ``,
```

```
frontend\src\components\staff\business-eval-modal.css
```

```
` `` `css
```

```
/* Modal overlay */
```



```
.modal-overlay {
 position: fixed;
 top: 0;
 left: 0;
 width: 100%;
 height: 100%;
 background-color: rgba(0,0,0,0.5);
 display: flex;
 align-items: center;
 justify-content: center;
 z-index: 999;
}
```

```
/* Modal container */
.modal-container {
 background-color: #fff;
 border-radius: 8px;
 width: 500px;
 max-width: 90%;
 padding: 1.5rem;
 box-shadow: 0 4px 12px rgba(0,0,0,0.2);
 animation: fadeIn 0.3s ease;
}
```

```
@keyframes fadeIn {
 from { opacity: 0; transform: translateY(-10px); }
```

```
to { opacity: 1; transform: translateY(0); }
}
```

```
/* Modal heading */
.modal-container h2 {
 margin-top: 0;
 margin-bottom: 1rem;
 font-size: 1.4rem;
 color: #0078d4;
}
```

```
/* Section with details */
.modal-section p {
 margin: 0.4rem 0;
 font-size: 0.95rem;
 color: #333;
}
```

```
/* Form styling */
.modal-form {
 margin-top: 1rem;
}
```

```
.modal-form label {
 display: block;
 margin-top: 0.8rem;
```

```
margin-bottom: 0.3rem;
font-weight: 500;
}
```

```
.modal-form select,
.modal-form textarea {
width: 100%;
padding: 0.5rem;
border: 1px solid #ccc;
border-radius: 4px;
}
```

```
.modal-form textarea {
min-height: 80px;
resize: vertical;
}
```

```
/* Approved section */
.approved-section {
margin-top: 1rem;
padding: 1rem;
border: 1px solid #28a745;
border-radius: 6px;
background-color: #e6f9ec;
}
```

```
.approved-section p {
 margin: 0.5rem 0;
 color: #155724;
 font-weight: 500;
}
```

```
.qr-wrapper {
 margin-top: 1rem;
 text-align: center;
}
```

*/\* Actions \*/*

```
.modal-actions {
 display: flex;
 justify-content: flex-end;
 gap: 0.8rem;
 margin-top: 1.2rem;
}
```

```
.cancel-btn,
.submit-btn {
 padding: 0.5rem 1rem;
 border-radius: 4px;
 font-weight: 500;
 cursor: pointer;
 border: none;
```

```
}
```

```
.cancel-btn {
 background-color: #ccc;
 color: #333;
}
```

```
.cancel-btn:hover {
 background-color: #bbb;
}
```

```
.submit-btn {
 background-color: #0078d4;
 color: #fff;
}
```

```
.submit-btn:hover {
 background-color: #005a9e;
}
```

```
...
```

```
frontend\src\components\staff\BusinessEvaluationModal.js
```

```
```javascript
```

```
import { useState } from "react";
```

```
import { doc, updateDoc } from "firebase/firestore";
```

```
import { db } from "../../services/firebase";
```

```
import { toast } from "react-toastify";
```

```
import { QRCodeCanvas } from "qrcode.react";
```

```
import "./business-eval-modal.css";
```

```
const BusinessEvaluationModal = ({ business, onClose, onUpdated, onSubmit }) => {
```

```
  const isApproved = business.status === "approved";
```

```
  const [status, setStatus] = useState(  
    business.status === "payment_submitted" ? "approved" : (business.status ||  
    "pending_evaluation")  
  );
```

```
  const [notes, setNotes] = useState(business.notes || "");
```

```
  const handleSubmit = async (e) => {
```

```
    e.preventDefault();
```

```
    try {
```

```
      if (typeof onSubmit === "function") {
```

```
        await onSubmit({ status, notes });
```

```
        toast.success("✅ Status updated!");
```

```
        return;
```

```
      }
```

```
      const businessRef = doc(db, "businesses", business.id);
```

```
      await updateDoc(businessRef, {
```

```
        status,
```

```
    notes,  
    evaluatedAt: new Date().toISOString(),  
  });  
  toast.success("✅ Status updated!");  
  if (onUpdated) onUpdated();  
  onClose();  
} catch (err) {  
  console.error("❌ Error updating status:", err);  
  toast.error("❌ Failed to update status.");  
}  
};
```

```
return (  
  <div className="modal-overlay">  
    <div className="modal-container">  
      <h2>Evaluate Business Application</h2>  
  
      <div className="modal-section">  
        <p><strong>Business Name:</strong> {business.businessName}</p>  
        <p><strong>Owner:</strong> {business.ownerName}</p>  
        <p><strong>Type:</strong> {business.businessType}</p>  
        <p><strong>Barangay:</strong> {business.barangay}</p>  
        <p><strong>Address:</strong> {business.address}</p>  
        <p><strong>Contact:</strong> {business.contactNumber}</p>  
        <p><strong>Submitted:</strong> {business.registrationDate}</p>  
      </div>  
    </div>  
  </div>
```

```
<div className="modal-section">

  <h4>Submitted Documents</h4>

  {business.documents?.validId && (

    <p>

      <a href={business.documents.validId} target="_blank" rel="noopener noreferrer">

         View Valid ID

      </a>

    </p>

  )}

  {business.documents?.proofOfAddress && (

    <p>

      <a href={business.documents.proofOfAddress} target="_blank" rel="noopener
noreferrer">

         View Proof of Address

      </a>

    </p>

  )}

  {business.documents?.dtiCert && (

    <p>

      <a href={business.documents.dtiCert} target="_blank" rel="noopener noreferrer">

         View DTI Certificate

      </a>

    </p>

  )}

  {business.documents?.businessLogo && (
```



```
<p>
  <a href={business.documents.businessLogo} target="_blank" rel="noopener
norereferrer">
```

```
    <img alt="Business Logo" data-bbox={business.documents.businessLogo} /> View Business Logo
```

```
  </a>
</p>
})
</div>
```

```
{isApproved ? (
```

```
  <div className="approved-section">
```

```
    <p><strong>Status:</strong> <span style={status === 'Approved' ? 'color: green; font-weight: bold;' : ''}>✔</span> Approved</p>
```

```
    <p><strong>Permit Number:</strong> {business.permitNumber || "—"}</p>
```

```
    <div className="qr-wrapper">
```

```
      <QRCodeCanvas value={business.businessId || business.id} size={96} />
```

```
    </div>
```

```
    <div className="modal-actions">
```

```
      <button type="button" className="submit-btn" onClick={onClose}>
```

```
        Close
```

```
      </button>
```

```
    </div>
```

```
  </div>
```

```
): (
```

```
  <form onSubmit={handleSubmit} className="modal-form">
```

```
    <label>Status</label>
```

```
    <select value={status} onChange={(e) => setStatus(e.target.value)}>
```

```
{business.status === "payment_submitted" ? (  
  <>  
    <option value="approved">Approved</option>  
    <option value="rejected">Rejected</option>  
  </>  
) : (  
  <>  
    <option value="pending_evaluation">Pending Evaluation</option>  
    <option value="for_payment">For Payment</option>  
    <option value="rejected">Rejected</option>  
  </>  
  )}  
</select>
```

```
<label>Notes</label>
```

```
<textarea  
  value={notes}  
  onChange={(e) => setNotes(e.target.value)}  
  placeholder="Add evaluation notes..."  
>
```

```
<div className="modal-actions">  
  <button type="button" className="cancel-btn" onClick={onClose}>  
    Cancel  
  </button>  
  <button type="submit" className="submit-btn">
```

```

        Save Changes
      </button>
    </div>
  </form>
)}
</div>
</div>
);
};

export default BusinessEvaluationModal;

```

```

` ` `

```

```

## frontend\src\components\staff\ComplaintEvaluationModal.js

```

```

` ` ` javascript

```

```

import { useState } from "react";

```

```

import "./modal.css";

```

```

const ComplaintEvaluationModal = ({ complaint, onClose, onSubmit }) => {

```

```

  const [status, setStatus] = useState(complaint.status || "open");

```

```

  const [notes, setNotes] = useState(complaint.resolution_notes || "");

```

```

  const [saving, setSaving] = useState(false);

```

```

  const handleSubmit = async (e) => {

```

```
e.preventDefault();
```

```
// Basic validation
```

```
if (status === "resolved" && notes.trim().length === 0) {  
    alert("Please add resolution notes before marking as resolved.");  
    return;  
}
```

```
setSaving(true);
```

```
try {  
    await onSubmit({  
        status,  
        notes,  
        resolved_at: status === "resolved" ? new Date().toISOString() : null,  
    });  
} finally {  
    setSaving(false);  
}  
};
```

```
return (  
    <div className="modal-overlay" role="dialog" aria-modal="true">  
        <div className="modal-container">  
            <h2>Evaluate Complaint</h2>
```

```

{ /* Resident-submitted details (read-only) */ }

<div className="modal-section">

  <p><strong>ID:</strong> {complaint.id}</p>

  <p><strong>Filed By:</strong> {complaint.filed_by_name}</p>

  <p><strong>Category:</strong> {complaint.category}</p>

  <p><strong>Description:</strong> {complaint.description}</p>

  <p><strong>Location:</strong> {complaint.location}</p>

</div>

```

```

{ /* Staff evaluation form */ }

<form onSubmit={handleSubmit} className="modal-form">

  <label>Status</label>

  <select

    value={status}

    onChange={(e) => setStatus(e.target.value)}

    >

    <option value="open">Open</option>

    <option value="in_review">In Review</option>

    <option value="resolved">Resolved</option>

  </select>

```

```

<label>Resolution Notes</label>

<textarea

  value={notes}

  onChange={(e) => setNotes(e.target.value)}

  placeholder="Add resolution notes here..."

```

```
        rows={5}
      />

      <div className="modal-actions">
        <button
          type="button"
          className="cancel-btn"
          onClick={onClose}
          disabled={saving}
        >
          Cancel
        </button>

        <button
          type="submit"
          className="submit-btn"
          disabled={saving}
        >
          {saving ? "Saving..." : "Save Changes"}
        </button>
      </div>
    </form>
  </div>
</div>
);
};
```

```
export default ComplaintEvaluationModal;
```

```
`,`,
```

```
## frontend\src\components\staff\IncidentEvaluation.js
```

```
` `` javascript
```

```
import { useState } from "react";
```

```
import { IncidentsAPI } from "../../services/api";
```

```
const normalizeStatus = (value) => {
```

```
  const normalized = String(value || "").trim().toLowerCase();
```

```
  if (normalized === "in-progress" || normalized === "in_progress" || normalized ===  
  "inreview") {
```

```
    return "pending";
```

```
  }
```

```
  if (normalized === "pending" || normalized === "resolved" || normalized === "escalated") {
```

```
    return normalized;
```

```
  }
```

```
  return "pending";
```

```
};
```

```
const formatDateTime = (value) => {
```

```
  if (!value) return "—";
```

```
  if (value instanceof Date) return Number.isNaN(value.getTime()) ? "—" :  
  value.toLocaleString();
```

```

if (typeof value === "string" || typeof value === "number") {
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? "—" : parsed.toLocaleString();
}
if (typeof value === "object") {
  const seconds = value.seconds ?? value._seconds;
  if (typeof seconds === "number") {
    const parsed = new Date(seconds * 1000);
    return Number.isNaN(parsed.getTime()) ? "—" : parsed.toLocaleString();
  }
}
return "—";
};

```

```

const IncidentEvaluation = ({ incident, role, onClose, onUpdate }) => {
  const [status, setStatus] = useState(normalizeStatus(incident.status));
  const [assignedTo, setAssignedTo] = useState(incident.assigned_to_name || "");

  const handleUpdate = async () => {
    try {

      const payload = {
        status: normalizeStatus(status),
        assigned_to: assignedTo || undefined,
      };
    }
  };

```



```
await IncidentsAPI.patchStatus(incident.id, payload);

onUpdate();
onClose();
} catch (err) {
  console.error(
    " ❌ Failed to update incident:",
    err.response?.data?.detail || err.message
  );
}
};

return (
  <div className="incident-evaluation">
    <h3>Evaluate Incident</h3>

    <div className="incident-details">
      <p><strong>Type:</strong> {incident.type}</p>
      <p><strong>Description:</strong> {incident.description}</p>
      <p><strong>Location:</strong> {incident.location}</p>
      <p><strong>Reported By:</strong> {incident.reported_by_name}</p>
      <p><strong>Current Status:</strong> {incident.status}</p>
      <p><strong>Created At:</strong> {formatDateTime(incident.timestamp ||
incident.createdAt)}</p>
    </div>
```

```
<div className="evaluation-actions">
```

```
<label>
```

Status:

```
<select value={status} onChange={(e) => setStatus(normalizeStatus(e.target.value))}>
```

```
<option value="pending">Pending</option>
```

```
<option value="resolved">Resolved</option>
```

```
<option value="escalated">Escalated</option>
```

```
</select>
```

```
</label>
```

```
{/*  Allow staff to assign too */}
```

```
<label>
```

Assign To:

```
<input
```

```
  type="text"
```

```
  value={assignedTo}
```

```
  onChange={(e) => setAssignedTo(e.target.value)}
```

```
  placeholder="Staff name or ID"
```

```
</label>
```

```
<div className="buttons">
```

```
<button onClick={handleUpdate}>  Save Changes</button>
```

```
<button onClick={onClose}>  Cancel</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default IncidentEvaluation;
```

```
```
```

```
frontend\src\components\staff\modal.css
```

```
```css
```

```
.modal-overlay {
```

```
  position: fixed;
```

```
  top: 0;
```

```
  right: 0;
```

```
  bottom: 0;
```

```
  left: 0;
```

```
  background: rgba(0,0,0,0.45);
```

```
  display: flex;
```

```
  align-items: center;
```

```
  justify-content: center;
```

```
  z-index: 2000;
```

```
}
```

```
.modal-container {
```

```
  background: white;
```

```
padding: 24px;
border-radius: 8px;
width: 420px;
max-width: 90%;
box-shadow: 0 4px 20px rgba(0,0,0,0.2);
}
```

```
.modal-section p {
  margin: 4px 0;
}
```

```
.modal-form {
  margin-top: 16px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

```
.modal-actions {
  display: flex;
  justify-content: flex-end;
  gap: 12px;
  margin-top: 12px;
}
```

```
.cancel-btn {
```

```
background: #ccc;
padding: 6px 12px;
border-radius: 4px;
}
```

```
.submit-btn {
background: #007bff;
color: white;
padding: 6px 12px;
border-radius: 4px;
}
```

```
```
```

```
frontend\src\components\staff\ReceiptPreview.js
```

```
```javascript
```

```
import { useEffect, useCallbaek } from "react";
import { QRCodeCanvas } from "qrcode.react";
import jsPDF from "jspdf";
import QRCode from "qrcode";
import logo from "../../assets/barangay_logo.png";
```

```
const ReceiptPreview = ({ receiptData, onGeneratePDF }) => {
```

```
const generatePDF = useCallbaek(async () => {
```

```
if (!receiptData) return;
```

```
const doc = new jsPDF({  
  orientation: "portrait",  
  unit: "mm",  
  format: [80, 200],  
});
```

```
const margin = 5;  
const pageWidth = doc.internal.pageSize.getWidth();  
const pageHeight = doc.internal.pageSize.getHeight();  
const lineHeight = 10;
```

```
// Logo
```

```
let headerY = margin + 10;
```

```
try {
```

```
  const img = new Image();
```

```
  img.src = logo;
```

```
  await new Promise((resolve) => { img.onload = resolve; });
```

```
  doc.addImage(img, "PNG", margin, headerY - 6, 12, 12);
```

```
} catch {
```

```
  console.warn("Logo not found, skipping image.");
```

```
}
```

```
// Header text
```

```
doc.setFont("helvetica", "bold");
```

```
doc.setFontSize(12);

const headerText =
  receiptData.entityType === "document"
    ? "Barangay Document Payment Receipt"
    : "Barangay Business Payment Receipt";

const headerLines = doc.splitTextToSize(headerText, pageWidth - (margin + 20));
doc.text(headerLines, margin + 18, headerY);


// Divider

doc.setLineWidth(0.3);
doc.line(margin, headerY + 8, pageWidth - margin, headerY + 8);


// Details

doc.setFontSize(10);
doc.setFont("helvetica", "normal");


let y = headerY + 16;
const labelX = margin;

const drawField = (label, value) => {
  doc.text(label, labelX, y);
  doc.text(value || "", labelX + doc.getTextWidth(label) + 2, y);
  y += lineHeight;
};

drawField("Receipt #:", receiptData.receiptNumber);
```

```
drawField(
  receiptData.entityType === "document" ? "Document ID:" : "Business ID:",
  receiptData.customEntityId || receiptData.entityId // ✅ use custom ID if available
);
drawField("Resident:", receiptData.residentName);
drawField("Type:", receiptData.description);
drawField("Amount Paid:", "PHP " + getCleanAmount(receiptData.amount).toFixed(2));
drawField("Payment Method:", receiptData.method);
drawField("Date Issued:", new Date(receiptData.issuedAt).toLocaleString());
drawField("Processed By:", receiptData.processedBy);

// Footer
y += lineHeight;
doc.setFontSize(9);
doc.text("This receipt serves as proof of payment.", pageWidth / 2, y, { align: "center" });
y += lineHeight;
doc.text("Thank you.", pageWidth / 2, y, { align: "center" });

// Signature line centered
y += lineHeight * 2;
const lineWidth = 40; // length of signature line
const lineX = (pageWidth - lineWidth) / 2; // center horizontally
doc.line(lineX, y, lineX + lineWidth, y);
doc.text("Authorized Signature", pageWidth / 2, y + 4, { align: "center" });

// QR code
```



```

const qrDataUrl = await QRCode.toDataURL(JSON.stringify(receiptData));

const qrSize = 25;

doc.addImage(qrDataUrl, "PNG", pageWidth / 2 - qrSize / 2, pageHeight - qrSize - margin,
qrSize, qrSize);

// doc.save(` ${receiptData.receiptNumber}.pdf` );

doc.autoPrint();

doc.output("dataurlnewwindow");

}, [receiptData]);

useEffect(() => {
  if (onGeneratePDF) {
    onGeneratePDF(() => generatePDF);
  }
}, [onGeneratePDF, generatePDF]);

if (!receiptData) return null;

const getCleanAmount = (amount) => {
  const cleaned = String(amount || "0").replace(/[^\\d.-]/g, "");
  const parsed = parseFloat(cleaned);
  return isNaN(parsed) ? 0 : parsed;
};

return (

```

```
<div className="receipt-preview">
```

```
<h3>✅ Receipt Preview</h3>
```

```
<p><strong>Receipt #:</strong> {receiptData.receiptNumber}</p>
```

```
{receiptData.entityType === "document" && (
```

```
<p>
```

```
<strong>Document ID:</strong> {receiptData.customEntityId || receiptData.entityId}
```

```
</p>
```

```
)}
```

```
{receiptData.entityType === "business" && (
```

```
<p>
```

```
<strong>Business ID:</strong> {receiptData.entityId}
```

```
</p>
```

```
)}
```

```
<p><strong>Type:</strong> {receiptData.description}</p>
```

```
<p><strong>Amount:</strong> ₱{getCleanAmount(receiptData.amount).toFixed(2)}</p>
```

```
<p><strong>Method:</strong> {receiptData.method}</p>
```

```
<p><strong>Date:</strong> {new Date(receiptData.issuedAt).toLocaleString()}</p>
```

```
<p><strong>Processed By:</strong> {receiptData.processedBy}</p>
```

```
<div className="qr-preview">
```

```
<QRCodeCanvas value={JSON.stringify(receiptData)} size={150} />
```

```
<p>Scan QR to verify receipt</p>
```

```
</div>
```

```
{/* Conditional footer line in preview */}
```

```
<p className="footer-line">
  {receiptData.entityType === "document"
    ? ` Issued by ${receiptData.barangay || "Barangay"} Secretary`
    : ` Approved by ${receiptData.barangay || "Barangay"} Staff` }
</p>
```

```
<button type="button" onClick={generatePDF}>Download PDF Receipt</button>
</div>
```

```
);
};
```

```
export default ReceiptPreview;
```

```
```
```

```
frontend\src\components\staff\StaffBusinessDashboard.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
import { collection, getDocs } from "firebase/firestore";
import { db } from "../../services/firebase";
import BusinessEvaluationModal from "../BusinessEvaluationModal";
import "../../styles/staff/staff-business-dashboard.css";
```

```
const StaffBusinessDashboard = () => {
  const [businesses, setBusinesses] = useState([]);
```

```
const [loading, setLoading] = useState(true);

const [activeTab, setActiveTab] = useState("needsEvaluation");

const [selectedBusiness, setSelectedBusiness] = useState(null);
```

```
//  Fetch all businesses
```

```
useEffect(() => {

  const fetchBusinesses = async () => {

    try {

      const snapshot = await getDocs(collection(db, "businesses"));

      const data = snapshot.docs.map(doc => ({

        id: doc.id,      // Firestore doc ID

        ...doc.data()

      }));

      setBusinesses(data);

    } catch (error) {

      console.error("  Firebase fetch error:", error);

    } finally {

      setLoading(false);

    }

  };

  fetchBusinesses();

}, []);
```

```
//  Separate businesses by status
```

```
const needsEvaluation = businesses.filter(b =>
```

```
["pending_evaluation", "for_payment", "payment_submitted"].includes(b.status)
);
```

```
const evaluated = businesses.filter(b =>
  ["approved", "rejected"].includes(b.status)
);
```

```
// 🌸 Render business cards
```

```
const renderBusinessCard = (b) => (
  <div key={b.id} className="business-card">
    <h3>{b.businessName}</h3>
    <p><strong>Owner:</strong> {b.ownerName}</p>
    <p><strong>Type:</strong> {b.businessType}</p>
    <p><strong>Barangay:</strong> {b.barangay}</p>
    <p><strong>Status:</strong> {b.status}</p>
    <button onClick={() => setSelectedBusiness(b)}>Evaluate</button>
  </div>
);
```

```
return (
  <div className="staff-business-dashboard">
    <h2> 📄 Business Applications</h2>

    { /* Tabs */ }

    <div className="tabs">
      <button
```

```

        className={activeTab === "needsEvaluation" ? "active" : ""}
        onClick={() => setActiveTab("needsEvaluation")}
      >
        Needs Evaluation
      </button>
      <button
        className={activeTab === "evaluated" ? "active" : ""}
        onClick={() => setActiveTab("evaluated")}
      >
        Evaluated
      </button>
    </div>

```

```

    { /* Business lists */ }
    { loading ? (
      <p>Loading businesses...</p>
    ) : (
      <div className="business-grid">
        { activeTab === "needsEvaluation"
          ? needsEvaluation.map(renderBusinessCard)
          : evaluated.map(renderBusinessCard) }
      </div>
    ) }

```

```

    { /* Evaluation Modal */ }
    { selectedBusiness && (

```

```

    <BusinessEvaluationModal
      business={selectedBusiness}
      onClose={() => setSelectedBusiness(null)}
    />
  )}
</div>

);
};

```

```
export default StaffBusinessDashboard;
```

```

` ` `

```

```
## frontend\src\components\treasurer\CategoryList.js
```

```
` ` ` javascript
```

```
import "../../styles/treasurer/category-list.css";
```

```

function CategoryList({ revenueByCategory }) {
  return (
    <section className="categories">
      <h2>Revenue by Category</h2>
      {Object.keys(revenueByCategory).length === 0 ? (
        <p>No collections recorded yet.</p>
      ) : (
        <table className="category-table">

```

```

<thead>

  <tr>

    <th>Category</th>

    <th>Paid</th>

    <th>Unpaid</th>

  </tr>

</thead>

<tbody>

  {Object.entries(revenueByCategory).map(([category, { paid, unpaid }]) => (

    <tr key={category}>

      <td><strong>{category}</strong></td>

      <td className="paid">P{paid.toLocaleString()}</td>

      <td className="unpaid">P{unpaid.toLocaleString()}</td>

    </tr>

  )))

</tbody>

</table>

  })

</section>

);

}

```

```

export default CategoryList;

```

```

...

```



```
## frontend\src\components\treasurer\Collections.js
```

```
` `` javascript
```

```
import { usePayments } from "../../hooks/usePayments";
```

```
import SummaryCards from "./SummaryCards";
```

```
import CategoryList from "./CategoryList";
```

```
import TransactionsTable from "./TransactionsTable";
```

```
function Collections() {
```

```
  const { transactions = [], totals, revenueByCategory } = usePayments();
```

```
  return (
```

```
    <div className="treasurer-main">
```

```
      <header>
```

```
        <h1>Barangay Collections</h1>
```

```
      </header>
```

```
      { /* Summary Section */ }
```

```
      <SummaryCards totals={totals} />
```

```
      { /* Revenue by Category */ }
```

```
      <CategoryList revenueByCategory={revenueByCategory} />
```

```
      { /* Transactions Table */ }
```

```
      <TransactionsTable transactions={transactions} />
```

```
    </div>
```

```
);  
}
```

```
export default Collections;
```

```
````
```

```
frontend\src\components\treasurer\Disbursements.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";  
import { useDisbursements } from "../../hooks/useDisbursements";  
import { AccountsAPI, DisbursementsAPI } from "../../services/api";  
import DisbursementForm from "../forms/DisbursementForm";  
import StatusModal from "../StatusModal";
```

```
function Disbursements() {  
  const { disbursements = [], totals = {}, byCategory = {} } = useDisbursements();  
  const [showForm, setShowForm] = useState(false);  
  const [selectedDisbursement, setSelectedDisbursement] = useState(null);  
  const [users, setUsers] = useState([]);  
  
  useEffect(() => {  
    const fetchUsers = async () => {  
      try {  
        const accounts = await AccountsAPI.list({ limit: 50 });
```

```
    setUsers(accounts);
  } catch (err) {
    console.error("Failed to fetch accounts", err);
  }
};

fetchUsers();
}, []);
```

```
const getUserUserName = (uid) => {
  const user = users.find(u => u.uid === uid);
  return user ? (user.full_name || user.email) : uid;
};
```

```
const handleDelete = async (id) => {
  if (!window.confirm("Are you sure you want to delete this disbursement?")) return;
  try {
    await DisbursementsAPI.delete(id);
  } catch (err) {
    console.error("Failed to delete disbursement", err);
  }
};
```

```
return (
  <div className="treasurer-main">
    <header className="header">
      <h1>Disbursements</h1>
```

```
<button className="add-btn" onClick={() => setShowForm(true)}>
  + Add Disbursement
</button>

</header>
```

```
{showForm && <DisbursementForm onClose={() => setShowForm(false)} />}
{selectedDisbursement && (
  <StatusModal
    disbursement={selectedDisbursement}
    onClose={() => setSelectedDisbursement(null)}
  />
)}
```

```
{/* Summary Section */}

<section className="summary">
  <h2>Summary</h2>
  <ul>
    <li><strong>Total Approved Disbursed:</strong>
      ₪{totals.spentApproved?.toLocaleString() || 0}</li>
    <li><strong>Total Pending Disbursement:</strong>
      ₪{totals.spentPending?.toLocaleString() || 0}</li>
    <li><strong>Approved Transactions:</strong> {totals.countApproved}</li>
    <li><strong>Pending Transactions:</strong> {totals.countPending}</li>
    <li><strong>All Transactions:</strong> {totals.totalCount}</li>
  </ul>
</section>
```

```

{ /* Category Breakdown */}

<section className="categories">

  <h2>By Category</h2>

  {Object.keys(byCategory).length === 0 ? (

    <p>No disbursements recorded yet.</p>

  ) : (

    <ul>

      {Object.entries(byCategory).map(([category, amount]) => (

        <li key={category}>{category}: ₱{amount.toLocaleString()}</li>

      ))}

    </ul>

  )}

</section>

```

```

{ /* Detailed Transactions */}

<section className="transactions">

  <h2>Transaction Details</h2>

  {disbursements.length === 0 ? (

    <p>No transactions available.</p>

  ) : (

    <table className="transactions-table">

      <thead>

        <tr>

          <th>Reference ID</th>

          <th>Category</th>


```

```

<th>Amount</th>

<th>Date</th>

<th>Recipient</th>

<th>Processed By</th>

<th>Status</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{disbursements.map(d => (

<tr

  key={d.id}

  onClick={() => setSelectedDisbursement(d)}

  style={{ cursor: "pointer" }}

>

  <td>{d.referenceNo || d.id}</td>

  <td>{d.category || "Miscellaneous"}</td>

  <td>P{(d.amount || 0).toLocaleString()}</td>

  <td>{d.date ? new Date(d.date).toLocaleDateString() : "—"}</td>

  <td>

    {d.category === "Salaries"

      ? getUsername(d.recipientId || d.recipient)

      : d.recipient || "—"}

  </td>

  <td>{getUsername(d.processedByName)}</td>

  <td>{d.status || "—"}</td>

```

```

        <td>

        <button
            className="delete-btn"
            onClick={(e) => {
                e.stopPropagation();
                handleDelete(d.id)
            }}
        >

            Delete

        </button>

    </td>

</tr>

    )}

</tbody>

</table>

)}

</section>

</div>

);

}

```

```

export default Disbursements;

```

```

` ` `

```

```

## frontend\src\components\treasurer\DisbursementTable.js

```

```
````javascript
```

```
import { useDisbursements } from "../../hooks/useDisbursements";
```

```
function DisbursementTable() {
```

```
 const { disbursements } = useDisbursements();
```

```
 return (
```

```
 <div className="disbursement-table">
```

```
 <h2>Barangay Disbursements</h2>
```

```
 <table>
```

```
 <thead>
```

```
 <tr>
```

```
 <th>Category</th>
```

```
 <th>Amount</th>
```

```
 <th>Date</th>
```

```
 </tr>
```

```
 </thead>
```

```
 <tbody>
```

```
 {disbursements.map(d => (
```

```
 <tr key={d.id}>
```

```
 <td>{d.category}</td>
```

```
 <td>₱{d.amount}</td>
```

```
 <td>{d.date}</td>
```

```
 </tr>
```

```
)))
```



```
 </tbody>
 </table>
 </div>
);
}
```

```
export default DisbursementTable;
```

```
```
```

```
## frontend\src\components\treasurer\Reports.js
```

```
```javascript
import { useState } from "react";
import { useReports } from "../../hooks/useReports";
import { Line } from "react-chartjs-2";
import {
 Chart as ChartJS,
 LineElement,
 CategoryScale,
 LinearScale,
 PointElement,
 Title,
 Tooltip,
 Legend
} from "chart.js";
```

```
ChartJS.register(LineElement, CategoryScale, LinearScale, PointElement, Title, Tooltip, Legend);
```

```
function Reports() {
 const { generateMonthlyReport } = useReports();
 const [currentReport, setCurrentReport] = useState(null);
 const [archive, setArchive] = useState([]);

 const handleGenerate = () => {
 const result = generateMonthlyReport();
 setCurrentReport(result);

 setArchive(prev => [
 ...prev,
 {
 ...result,
 month: new Date().toLocaleString("default", { month: "long", year: "numeric" })
 }
]);
 };

 const handleClearSelected = (index) => {
 setArchive(prev => prev.filter((_, i) => i !== index));
 };
```

```
// Chart data

const chartData = {
 labels: archive.map(r => r.month),
 datasets: [
 {
 label: "Collections",
 data: archive.map(r => r.collections || 0),
 borderColor: "#2d8fdd",
 backgroundColor: "rgba(45,143,221,0.2)",
 tension: 0.3
 },
 {
 label: "Approved Disbursements",
 data: archive.map(r =>
 r.disbursements
 ?.filter(d => d.status === "approved")
 .reduce((sum, d) => sum + (d.amount || 0), 0) || 0
),
 borderColor: "#28a745",
 backgroundColor: "rgba(40,167,69,0.2)",
 tension: 0.3
 },
 {
 label: "Pending Disbursements",
 data: archive.map(r =>
 r.disbursements
```

```
 ?.filter(d => d.status === "pending")
 .reduce((sum, d) => sum + (d.amount || 0), 0) || 0
),
 borderColor: "#ffc107",
 backgroundColor: "rgba(255,193,7,0.2)",
 tension: 0.3
}
]
};
```

```
const chartOptions = {
 responsive: true,
 plugins: {
 legend: { position: "top" },
 title: { display: true, text: "Collections vs Disbursements Over Time" }
 }
};
```

```
return (
 <div className="treasurer-main">
 <header className="header">
 <h1>Reports</h1>
 <button className="generate-btn" onClick={handleGenerate}>
 Generate Monthly Report (PDF)
 </button>
 </header>
```

```

 { /* Current Report */ }

 {currentReport ? (

 <section className="summary">

 <h2>Current Report Summary</h2>

 Total Collections:
 ₪{currentReport.collections?.toLocaleString() || 0}

 Approved Disbursements:
 ₪{currentReport.disbursements?.filter(d => d.status === "approved").reduce((sum, d) =>
 sum + (d.amount || 0), 0).toLocaleString()}

 Pending Disbursements:
 ₪{currentReport.disbursements?.filter(d => d.status === "pending").reduce((sum, d) =>
 sum + (d.amount || 0), 0).toLocaleString()}

 Net Balance: ₪{(currentReport.collections -
 currentReport.disbursements?.filter(d => d.status === "approved").reduce((sum, d) => sum
 + (d.amount || 0), 0)).toLocaleString()}

 </section>

) : (

 <section className="report-output">

 <h2>Report Output</h2>

 <p>No report generated yet.</p>

 </section>

)}

 { /* Archive Table */ }

 {archive.length > 0 && (

```

```

<section className="archive">

 <h2>Previous Reports</h2>

 <table className="archive-table">

 <thead>

 <tr>

 <th>Month</th>

 <th>Total Collections</th>

 <th>Approved Disbursements</th>

 <th>Pending Disbursements</th>

 <th>Net Balance</th>

 <th>Action</th>

 </tr>

 </thead>

 <tbody>

 {archive.map((r, idx) => {

 const approved = r.disbursements?.filter(d => d.status ===
"approved").reduce((sum, d) => sum + (d.amount || 0), 0) || 0;

 const pending = r.disbursements?.filter(d => d.status === "pending").reduce((sum,
d) => sum + (d.amount || 0), 0) || 0;

 const netBalance = (r.collections || 0) - approved;

 return (

 <tr key={idx}>

 <td>{r.month}</td>

 <td>₹{r.collections?.toLocaleString() || 0}</td>

 <td>₹{approved.toLocaleString()}</td>

 <td>₹{pending.toLocaleString()}</td>

 <td>₹{netBalance.toLocaleString()}</td>

```

```

 <td>

 <button className="clear-btn" onClick={() => handleClearSelected(idx)}>

 Clear

 </button>

 </td>

 </tr>

);

 }}

</tbody>

</table>

</section>

)}

```

```

 { /* Chart Visualization */
 {archive.length > 0 && (
 <section className="chart">

 <Line data={chartData} options={chartOptions} />

 </section>

)}

 </div>

);

}

```

```

export default Reports;

```

```

...

```

```
frontend\src\components\treasurer\ReportsSection.js
```

```
```javascript
```

```
import { useReports } from "../../hooks/useReports";
```

```
function ReportsSection() {
```

```
  const { generateMonthlyReport } = useReports();
```

```
  return (
```

```
    <div className="reports-section">
```

```
      <h2>Reports</h2>
```

```
      <button onClick={generateMonthlyReport}>Export Monthly Report</button>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default ReportsSection;
```

```
```
```

```
frontend\src\components\treasurer\RevenueChart.js
```

```
```javascript
```

```
import { usePayments } from "../../hooks/usePayments";
```

```
import { Bar } from "react-chartjs-2";
```



```
import { useTheme } from "../../context/ThemeContext";
```

```
function RevenueChart() {
```

```
  const { revenueByCategory } = usePayments();
```

```
  const { isDarkMode } = useTheme();
```

```
  if (!revenueByCategory || Object.keys(revenueByCategory).length === 0) {
```

```
    return <p>No revenue data available</p>;
```

```
  }
```

```
// 🎨 Define colors based on theme
```

```
const textColor = isDarkMode ? "#ffffff" : "#000000";
```

```
const bgColorPaid = isDarkMode ? "rgba(75,192,192,0.8)" : "rgba(75,192,192,0.6)";
```

```
const bgColorUnpaid = isDarkMode ? "rgba(255,99,132,0.8)" : "rgba(255,99,132,0.6)";
```

```
const containerBg = isDarkMode ? "#222" : "#fff";
```

```
const data = {
```

```
  labels: Object.keys(revenueByCategory),
```

```
  datasets: [
```

```
    {
```

```
      label: "Collected Revenue",
```

```
      data: Object.values(revenueByCategory).map(c => c.paid),
```

```
      backgroundColor: bgColorPaid
```

```
    },
```

```
    {
```

```
      label: "Outstanding Balance",
```

```
    data: Object.values(revenueByCategory).map(c => c.unpaid),
    backgroundColor: bgColorUnpaid
  }
]
};
```

```
const options = {
  maintainAspectRatio: false,
  plugins: {
    tooltip: {
      callbacks: {
        label: (context) => `₹${context.raw.toLocaleString()}`
      }
    },
    legend: {
      labels: {
        color: textColor
      }
    }
  },
  scales: {
    y: {
      beginAtZero: true,
      title: { display: true, text: "Amount (₹)", color: textColor },
      ticks: { color: textColor }
    },
  },
}
```

```
x: {  
  title: { display: true, text: "Category", color: textColor },  
  ticks: { color: textColor }  
}  
}  
};
```

```
return (  
  <div  
    className="revenue-chart"  
    style={{  
      maxWidth: "600px",  
      height: "300px",  
      backgroundColor: containerBg,  
      color: textColor,  
      padding: "1rem",  
      borderRadius: "8px"  
    }}  
  >  
    <h2>Revenue Breakdown</h2>  
    <Bar data={data} options={options} />  
  </div>  
);  
}
```

```
export default RevenueChart;
```

```

## frontend\src\components\treasurer\Settings.js

```javascript

import { useState } from "react";

function Settings() {

const [preferences, setPreferences] = useState({

theme: "light",

language: "en",

autoArchive: true,

notifications: true,

});

const handleChange = e => {

const { name, type, checked, value } = e.target;

setPreferences(prev => ({

...prev,

[name]: type === "checkbox" ? checked : value

}));

};

return (

<div className="treasurer-main">

```
<h1>Settings</h1>
```

```
<p>Manage Treasurer account preferences here.</p>
```

```
<section className="settings-section">
```

```
  <h2>System Preferences</h2>
```

```
  <label>
```

```
    Theme:
```

```
    <select name="theme" value={preferences.theme} onChange={handleChange}>
```

```
      <option value="light">Light</option>
```

```
      <option value="dark">Dark</option>
```

```
    </select>
```

```
  </label>
```

```
  <label>
```

```
    Language:
```

```
    <select name="language" value={preferences.language} onChange={handleChange}>
```

```
      <option value="en">English</option>
```

```
      <option value="fil">Filipino</option>
```

```
    </select>
```

```
  </label>
```

```
  <label>
```

```
    <input
```

```
      type="checkbox"
```

```
      name="autoArchive"
```

```
      checked={preferences.autoArchive}
```

```

        onChange={handleChange}
      />
      Auto-archive monthly reports
    </label>

    <label>
      <input
        type="checkbox"
        name="notifications"
        checked={preferences.notifications}
        onChange={handleChange}
      />
      Enable email notifications
    </label>
  </section>
</div>
);
}

export default Settings;

` ``

## frontend\src\components\treasurer\StatusModal.js

` `` javascript

```

```

import { useState } from "react";

import { DisbursementsAPI } from "../../services/api";

function StatusModal({ disbursement, onClose }) {

  const [status, setStatus] = useState(disbursement.status || "pending");

  const [loading, setLoading] = useState(false);

  const handleSave = async () => {

    setLoading(true);

    try {

      await DisbursementsAPI.patchStatus(disbursement.id, { status });

      onClose();

    } catch (err) {

      console.error("Failed to update status", err);

    } finally {

      setLoading(false);

    }

  };

  return (

    <div className="modal-overlay">

      <div className="modal">

        <h2>Update Status</h2>

        <p><strong>Reference:</strong> {disbursement.referenceNo || disbursement.id}</p>

        <p><strong>Category:</strong> {disbursement.category}</p>

        <p><strong>Amount:</strong> ₪{disbursement.amount?.toLocaleString()}</p>

```

```

<label>
  Status:
  <select value={status} onChange={(e) => setStatus(e.target.value)}>
    <option value="pending">Pending</option>
    <option value="approved">Approved</option>
    <option value="rejected">Rejected</option>
  </select>
</label>

<div className="modal-actions">
  <button onClick={handleSave} disabled={loading}>
    {loading ? "Saving..." : "Save"}
  </button>
  <button onClick={onClose}>Cancel</button>
</div>
</div>
</div>
);
}

export default StatusModal;

...

## frontend\src\components\treasurer\SummaryCards.js

```



```
` `` `javascript
```

```
import { usePayments } from "../../hooks/usePayments";
```

```
import "../../styles/treasurer/summary-cards.css";
```

```
function SummaryCard({ title, value, icon }) {
```

```
  return (
```

```
    <div className="card">
```

```
      <span className="card-icon">{icon}</span>
```

```
      <div className="card-content">
```

```
        <h3>{title}</h3>
```

```
        <p>{value}</p>
```

```
      </div>
```

```
    </div>
```

```
  );
```

```
}
```

```
function SummaryCards() {
```

```
  const { totals } = usePayments();
```

```
  // Provide safe fallbacks
```

```
  const collections = totals?.collections ?? 0;
```

```
  const pendingCount = totals?.pendingCount ?? 0;
```

```
  const completedCount = totals?.completedCount ?? 0;
```

```
  const outstandingAmount = totals?.outstandingAmount ?? 0;
```

```
return (  
  <div className="summary-cards">  
    <SummaryCard  
      title="Total Collections"  
      value={`₱${collections.toLocaleString()}`} `}  
      icon="💰 "  
    />  
    <SummaryCard  
      title="Completed Payments"  
      value={completedCount}  
      icon="✅ "  
    />  
    <SummaryCard  
      title="Pending Payments"  
      value={pendingCount}  
      icon="⌚ "  
    />  
    <SummaryCard  
      title="Outstanding Balances"  
      value={`₱${outstandingAmount.toLocaleString()}`} `}  
      icon="⚠️ "  
    />  
  </div>  
);  
}
```

```
export default SummaryCards;
```

```
`, `
```

```
## frontend\src\components\treasurer\TransactionsTable.js
```

```
` `` javascript
```

```
import { usePayments } from "../../hooks/usePayments";
```

```
import "../../styles/treasurer/transactions-table.css";
```

```
function TransactionsTable() {
```

```
  const { transactions = [] } = usePayments();
```

```
  const formatDate = (dateValue) => {
```

```
    if (!dateValue) return "—";
```

```
    // Firestore Timestamp object
```

```
    if (dateValue?.toDate) {
```

```
      return new Date(dateValue.toDate()).toLocaleDateString();
```

```
    }
```

```
    // JS Date object
```

```
    if (dateValue instanceof Date) {
```

```
      return dateValue.toLocaleDateString();
```

```
    }
```

```
// ISO string or millis

try {

    return new Date(dateValue).toLocaleDateString();

} catch {

    return "—";

}

};
```

```
return (

<div className="transactions-table">

    <h2>Recent Transactions</h2>

    {transactions.length === 0 ? (

        <p>No transactions available.</p>

    ) : (

        <table>

            <thead>

                <tr>

                    <th>Business Name / Document</th>

                    <th>Owner Name</th>

                    <th>Type</th>

                    <th>Amount</th>

                    <th>Status</th>

                    <th>Channel</th>

                    <th>Receipt #</th>

                    <th>Date Paid</th>
```

```

</tr>

</thead>

<tbody>

{transactions.map((tx) => (

  <tr

    key={tx.customPaymentId || tx.id}

    className={` status-${tx.paymentStatus || tx.status}`}

    >

      {/* Business Name */}

      <td>{tx.businessName || "Barangay document"}</td>


      {/* Owner Name */}

      <td>{tx.ownerName || tx.residentName || "Unknown"}</td>


      {/* Entity Type/Category */}

      <td>{tx.entityCategory || tx.businessType || tx.documentType || tx.description || "—"
">"}</td>


      {/* Amount */}

      <td>₱{(tx.amount || 0).toLocaleString()}</td>


      {/* Status */}

      <td>{tx.paymentStatus || tx.status || "—"}</td>


      {/* Channel */}

      <td>{tx.method || tx.channel || "Cash / PayMongo"}</td>

```

```

        {/* Receipt Number (from receipts collection) */}
        <td>{tx.receiptNumber || "—"}</td>

        {/* Date Paid */}
        <td>

            {(tx.paymentStatus === "paid" || tx.status === "paid")
              ? formatDate(tx.datePaid || tx.createdAt)
              : "—"}

        </td>

    </tr>

    )}}
</tbody>
</table>

)}
</div>

);
}

```

```
export default TransactionsTable;
```

```

` ` `

```

```
## frontend\src\components\youth\EventCalendar.js
```

```
` `` javascript
```

```
import "../styles/admin.css";
```

```
const mockEvents = [
```

```
  { id: 1, title: "Tree Planting", date: "Oct 28, 2025", location: "Barangay Plaza" },
```

```
  { id: 2, title: "SK General Assembly", date: "Nov 5, 2025", location: "Covered Court" },
```

```
];
```

```
const EventCalendar = () => (
```

```
  <div className="event-calendar">
```

```
    <h3>  Upcoming Events</h3>
```

```
    <ul>
```

```
      {mockEvents.map((event) => (
```

```
        <li key={event.id}>
```

```
          <strong>{event.title}</strong><br />
```

```
          {event.date} — {event.location}
```

```
        </li>
```

```
      )})
```

```
    </ul>
```

```
  </div>
```

```
);
```

```
export default EventCalendar;
```

```
` ``
```

```
## frontend\src\components\youth\ProgramList.js
```

```
` `` `javascript
```

```
import "../styles/admin.css";
```

```
const mockPrograms = [
```

```
  { id: 1, title: "Clean & Green Drive", date: "Oct 25, 2025" },
```

```
  { id: 2, title: "Youth Leadership Seminar", date: "Nov 10, 2025" },
```

```
];
```

```
const ProgramList = () => (
```

```
  <div className="program-list">
```

```
    <h3> 🎯 Youth Programs</h3>
```

```
    <ul>
```

```
      {mockPrograms.map((program) => (
```

```
        <li key={program.id}>
```

```
          <strong>{program.title}</strong> — {program.date}
```

```
        </li>
```

```
      )})
```

```
    </ul>
```

```
  </div>
```

```
);
```

```
export default ProgramList;
```

```
` `` `
```



```
## frontend\src\components\youth\YouthFeedbackForm.js
```

```
` `` javascript
```

```
import { useState } from "react";
```

```
import "../styles/admin.css";
```

```
const YouthFeedbackForm = () => {
```

```
  const [message, setMessage] = useState("");
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault();
```

```
    console.log("Youth feedback submitted:", message);
```

```
    // TODO: Send to Firestore
```

```
    setMessage("");
```

```
  };
```

```
  return (
```

```
    <div className="youth-feedback-form">
```

```
      <h3>💬 Youth Feedback</h3>
```

```
      <form onSubmit={handleSubmit}>
```

```
        <textarea
```

```
          rows="4"
```

```
          placeholder="Share your thoughts or suggestions..."
```

```
          value={message}
```

```
          onChange={(e) => setMessage(e.target.value)}
```

```
    />

    <button type="submit">Send</button>

  </form>

</div>

);

};
```

```
export default YouthFeedbackForm;
```

```
` ``
```

```
## frontend\src\components\youth\YouthRegistry.js
```

```
` `` javascript
```

```
import "../styles/admin.css";
```

```
const mockYouth = [

  { id: 1, name: "Ana Reyes", age: 17, status: "Active" },

  { id: 2, name: "Mark Cruz", age: 19, status: "Inactive" },

];
```

```
const YouthRegistry = () => (

  <div className="youth-registry">

    <h3> 📄 Youth Registry</h3>

    <table>

      <thead>
```

```
<tr>
  <th>Name</th>
  <th>Age</th>
  <th>Status</th>
</tr>
</thead>
<tbody>
  {mockYouth.map((youth) => (
    <tr key={youth.id}>
      <td>{youth.name}</td>
      <td>{youth.age}</td>
      <td>{youth.status}</td>
    </tr>
  ))}
</tbody>
</table>
</div>
);
```

```
export default YouthRegistry;
```

```
`,`,`
```

```
## frontend\src\config\documentConfig.js
```

```
` `` javascript
```

```
// src/data/documentConfig.js

import { PARANAQUE } from "../data/locations";

const documentConfig = {
  "Barangay Clearance": {
    fields: [
      { name: "purpose", label: "Purpose", type: "text", required: true, minLength: 5 },
    ],
    attachments: [
      { name: "idAttachment", label: "Upload Valid ID", required: true },
      { name: "residencyAttachment", label: "Upload Proof of Residency", required: true },
    ],
  },
  "Resident Certificate": {
    fields: [
      { name: "yearsOfStay", label: "Years of Residency", type: "number", required: true, min: 1 },
    ],
    attachments: [
      { name: "idAttachment", label: "Upload Valid ID", required: true },
      { name: "residencyAttachment", label: "Upload Proof of Residency", required: true },
    ],
  },
  "Indigency Certificate": {
    fields: [
      { name: "remarks", label: "Reason / Remarks", type: "textarea", required: true, minLength: 5 },
    ],
  },
}
```

```
],
attachments: [
  { name: "idAttachment", label: "Upload Valid ID", required: true },
  { name: "residencyAttachment", label: "Upload Proof of Residency", required: true },
],
},
"Good Moral Certificate": {
  fields: [
    { name: "purpose", label: "Purpose (e.g. school/employment)", type: "text", required: true },
  ],
  attachments: [
    { name: "idAttachment", label: "Upload Valid ID", required: true },
  ],
},
"Business Clearance": {
  fields: [
    { name: "businessName", label: "Business Name", type: "select", required: true, options:
[] }
  ],
  attachments: [
    { name: "businessPermit", label: "Upload Business Permit", required: true },
    { name: "idAttachment", label: "Upload Valid ID", required: true },
  ],
},
"Activity Permit": {
  fields: [
```

```
{ name: "activityName", label: "Activity Name", type: "text", required: true },
{
  label: "Activity Location",
  type: "group",
  fields: [
    { name: "location.barangay", label: "Barangay", type: "select", required: true, options:
PARANAQUE.barangays },
    { name: "location.street", label: "(Blk# / Lot#), Street", type: "text" },
    { name: "location.city", label: "City", type: "text", default: PARANAQUE.city, readOnly:
true },
    { name: "location.province", label: "Province", type: "text", default:
PARANAQUE.province, readOnly: true },
  ]
},
{ name: "activityDate", label: "Activity Date", type: "date", required: true },
],
attachments: [
  { name: "idAttachment", label: "Upload Valid ID", required: true },
  { name: "activityPlan", label: "Upload Activity Plan", required: true },
],
},
"Blotter Report": {
  fields: [
    { name: "complainant", label: "Complainant Name", type: "text", required: true, autoFill:
true },
    { name: "respondent", label: "Respondent Name", type: "text", required: true },
```

```
{ name: "incident", label: "Incident Details", type: "textarea", required: true, minLength:
10 },
{
  label: "Incident Location",
  type: "group",
  fields: [
    { name: "location.barangay", label: "Barangay", type: "select", required: true, options:
PARANAQUE.barangays },
    { name: "location.street", label: "(Blk# / Lot#), Street", type: "text" },
    { name: "location.city", label: "City", type: "text", default: PARANAQUE.city , readOnly:
true},
    { name: "location.province", label: "Province", type: "text", default:
PARANAQUE.province , readOnly: true},
  ]
}
],
attachments: [
  { name: "idAttachment", label: "Upload Valid ID", required: true },
  { name: "residencyAttachment", label: "Upload Proof of Residency", required: true },
],
},
"Health Certificate": {
  fields: [
    { name: "purpose", label: "Purpose of Certificate", type: "text", required: true },
  ],
  attachments: [
    { name: "medicalAttachment", label: "Upload Medical Result", required: true },
```

```
    { name: "idAttachment", label: "Upload Valid ID", required: true },
  ],
},
"Barangay ID": {
  fields: [
    { name: "occupation", label: "Occupation", type: "text", required: true },
    { name: "voterStatus", label: "Voter Status", type: "text", required: true },
  ],
  attachments: [
    { name: "photoAttachment", label: "Upload 1x1 Photo", required: true },
    { name: "idAttachment", label: "Upload Valid ID", required: true },
  ],
},
};
```

```
export default documentConfig;
```

```
```
```

```
frontend\src\config\metrics.js
```

```
```javascript
```

```
export const roleCollections = {
  admin: ["logins", "collections", "businesses", "complaints", "documents", "incidents"],
  secretary: ["documents", "incidents"],
  staff: ["incidents"],
```



```
resident: ["documents"],  
};
```

```
export const metricConfig = {  
  logins: { label: "Logins", variant: "success", icon: "🔑" },  
  collections: { label: "Collections", variant: "info", icon: "💰" },  
  businesses: { label: "Registered Businesses", variant: "success", icon: "🏢" },  
  complaints: { label: "Complaints", variant: "danger", icon: "🗣️" },  
  documents: { label: "Certificates Issued", variant: "accent", icon: "📄" },  
  incidents: { label: "Incidents Logged", variant: "danger", icon: "⚠️" },  
};
```

```
```\n
```

```
frontend\\src\\config\\modesConfig.js
```

```
```javascript
```

```
const modesConfig = {  
  document: {  
    endpoint: "/api/fees/documents",  
    fields: [  
      { name: "documentType", label: "Document Type", type: "text" },  
      { name: "fee", label: "Fee", type: "number" },  
      { name: "enabled", label: "Enabled", type: "checkbox" },  
      { name: "miscType", label: "Misc Fee Type", type: "select" }, // ✅ new
```

```

    ],
  },
  business: {
    endpoint: "/api/fees/businesses",
    fields: [
      { name: "businessType", label: "Business Type", type: "text" },
      { name: "fee", label: "Base Fee (₱)", type: "number" },
      { name: "registrationFee", label: "Registration Fee", type: "number" },
      { name: "annualFee", label: "Annual Fee", type: "number" },
      { name: "enabled", label: "Enabled", type: "checkbox" },
      { name: "miscType", label: "Misc Fee Type", type: "select" }, //  new
    ],
  },
  misc: {
    endpoint: "/api/fees/misc",
    fields: [
      { name: "miscType", label: "Misc Type", type: "text" },
      { name: "fee", label: "Fee", type: "number" },
      { name: "enabled", label: "Enabled", type: "checkbox" },
    ],
  },
};

```

```
export default modesConfig;
```

```

...

```

```
## frontend\src\config\navigation.js
```

```
` `` javascript
```

```
// 🛠 Common link definitions
```

```
const links = {
```

```
  dashboard: (path) => ({ path, label: " 🏠 Dashboard", action: "viewDashboard" }),
```

```
  residents: { path: "/residents", label: " 👤 Residents", action: "manageResidents" },
```

```
  addResident: { path: "/residents/new", label: " ➕ Add Resident", action: "manageResidents" },
```

```
  businesses: { path: "/businesses", label: " 📁 Businesses", action: "viewBusinesses" },
```

```
  ownBusinesses: { path: "/businesses/my", label: " 📁 My Businesses", action: "viewOwnBusinesses" },
```

```
  registerBusiness: { path: "/businesses/new", label: " ➕ Register Business", action: "registerBusinesses" },
```

```
  registerResidentBusiness: { path: "/residentBusinesses", label: " ➕ Register Resident Businesses", action: "registerResidentBusinesses" },
```

```
  documents: { path: "/documents", label: " 📄 Documents", action: "viewDocuments" },
```

```
  ownDocuments: { path: "/ownDocuments", label: " 📄 My Documents", action: "viewOwnDocuments" },
```

```
  requestDocument: { path: "/documents/request", label: " 📄 Request Document", action: "requestDocuments" },
```

```
  incidents: { path: "/incidents", label: " ⚠ Incidents", action: "viewIncidents" },
```

```
  reportIncident: { path: "/incidents/new", label: " ⚠ Report Incident", action: "reportIncidents" },
```

```
myIncidents: { path: "/myIncidents", label: " ⚠️ My Incidents", action: "viewOwnIncidents" },
```

```
fileComplaint: { path: "/complaints/new", label: " 🗨️ File Complaint", action: "fileComplaints" },
```

```
evaluateComplaints: { path: "/complaints/evaluate", label: " 🗨️ Evaluate Complaints", action: "manageComplaints" },
```

```
viewOwnComplaints: { path: "/myComplaints", label: " 🗨️ My Complaints", action: "viewOwnComplaints" },
```

```
viewAllComplaints: { path: "/allComplaints", label: " 🗨️ Complaints", action: "viewAllComplaints" },
```

```
createAccount: { path: "/accounts/new", label: " + Create Account", action: "createAccount" },
```

```
treasurer: { path: "/treasurer", label: " 💰 Finance", action: "viewFinancialRecords" },
```

```
audit: { path: "/audit", label: " 📊 Audit", action: "auditBarangayData" },
```

```
youth: { path: "/youth", label: " 🏠 Dashboard", action: "viewDashboard" },
```

```
home: { path: "/", label: " 🏠 Home", action: "viewDashboard" },
```

// Secretary-specific links

```
requestForDocuments: { path: "/secretary/documents", label: " 📄 Document Request", action: "documentRequest" },
```

```
pendingRequests: { path: "/secretary/pending", label: " 📋 Pending Requests", action: "pendingRequests" },
```

```
paidRequests: { path: "/secretary/payments", label: " 💵 Paid Requests", action: "paidRequests" },
```

```
issuedDocuments: { path: "/secretary/issued", label: " ✅ Issued Documents", action: "issuedDocuments" },
```

```
rejectedRequests: { path: "/secretary/rejected", label: " ❌ Rejected Requests", action: "rejectedRequests" },
```

```
incomingPayments: { path: "/treasurer/incoming", label: "💵 Incoming Payments", action:
"incomingPayments" },

expenses: { path: "/treasurer/expenses", label: "💰 Expenses", action:
"barangayExpenses" },

reports: { path: "/treasurer/reports", label: "📊 Financial Reports", action:
"financialReports" },

settings: { path: "/treasurer/settings", label: "⚙️ Settings", action: "settings" },
};
```

```
const sidebarLinks = {

  admin: [

    links.dashboard("/admin"),

    links.residents,

    links.businesses,

    links.documents,

    links.incidents,

    links.viewAllComplaints,

    links.fileComplaint,

    links.createAccount,

    links.treasurer,

    links.audit,

  ],

  staff: [

    links.dashboard("/staff"),

    links.residents,

    links.addResident,
```

```
links.businesses,  
links.registerResidentBusiness,  
links.incidents,  
links.viewAllComplaints,  
links.fileComplaint,  
],
```

```
secretary: [  
  links.dashboard("/secretary"),  
  links.requestForDocuments,  
  links.pendingRequests,  
  links.paidRequests,  
  links.issuedDocuments,  
  links.documents,  
  links.rejectedRequests,  
],
```

```
treasurer: [  
  links.dashboard("/treasurer"),  
  links.incomingPayments,  
  links.expenses,  
  links.reports,  
  links.settings,  
],
```

```
sk: [  
  links.youth,  
],
```

```
dilg: [  
  links.youth,
```

```
    links.audit,
  ],
  resident: [
    links.dashboard("/resident"),
    links.fileComplaint,
    links.viewOwnComplaints,
    links.ownDocuments,
    links.requestDocument,
    links.registerBusiness,
    links.ownBusinesses,
    links.reportIncident,
    links.myIncidents,
  ],
  default: [
    links.home,
  ],
};
```

```
export default sidebarLinks;
```

```
```
```

```
frontend\src\config\role_permissions.json
```

```
```json
```

```
{
```

```
"admin": [  
  "viewDashboard",  
  "createAccount",  
  "deleteAccount",  
  "updateRole",  
  "manageResidents",  
  "generateCertificates",  
  "viewFinancialRecords",  
  "manageFinancialRecords",  
  "youthRegistryAccess",  
  "auditBarangayData",  
  "documentRequest",  
  "manageDocuments",  
  "issuedDocuments",  
  "rejectedRequests",  
  "viewDocuments",  
  "viewBusinesses",  
  "viewAllComplaints",  
  "manageComplaints",  
  "fileComplaintsForResidents",  
  "viewIncidents",  
  "manageSettings",  
  "manageAnnouncements",  
  "manageEvents",  
  "viewUsers",  
  "manageUsers"
```



```
],  
"staff": [  
  "viewDashboard",  
  "manageBusinesses",  
  "registerResidentBusinesses",  
  "manageComplaints",  
  "manageResidents",  
  "viewBusinesses",  
  "viewIncidents",  
  "viewAllComplaints",  
  "fileComplaintsForResidents"  
],
```

```
"resident": [  
  "viewDashboard",  
  "fileComplaints",  
  "viewOwnComplaints",  
  "viewOwnDocuments",  
  "requestDocuments",  
  "viewOwnBusinesses",  
  "registerBusinesses",  
  "reportIncidents",  
  "viewOwnIncidents",  
  "submitFeedback"  
],
```

```
"secretary": [  
  "viewDashboard",
```

```
"documentRequest",
"viewDocuments",
"pendingRequests",
"paidRequests",
"issuedDocuments",
"rejectedRequests",
"manageDocuments"
],
"treasurer": [
"viewDashboard",
"incomingPayments",
"barangayExpenses",
"financialReports",
"settings",
"manageUsers"
],
"sk": [
"viewDashboard",
"youthRegistryAccess"
],
"dilg": [
"viewDashboard",
"auditBarangayData"
]
}
```

```

## frontend\src\config\roles.js

```javascript

import roleOverrides from "../role_permissions.json"; // shared JSON

// 🔑 Valid roles (shared across frontend + Firestore)

export const VALID_ROLES = Object.freeze(Object.keys(roleOverrides));

// 📊 Unified stats config (keys must match Firestore collection names)

export const ALL_STATS = Object.freeze({

 residents: { label: "Residents", variant: "accent", icon: "👥" },

 businesses: { label: "Businesses", variant: "success", icon: "👛" },

 complaints: { label: "Complaints", variant: "danger", icon: "🗣️" },

 incidents: { label: "Incidents", variant: "warning", icon: "⚠️" },

 documents: { label: "Documents", variant: "dilig", icon: "📄" },

 logins: { label: "Login Records", variant: "neutral", icon: "📊" },

 youth: { label: "Youth Registry", variant: "youth", icon: "👤" },

 collections: { label: "Collections", variant: "info", icon: "💰" },

});

// 🛠️ Derived maps

export const CATEGORIES = Object.freeze(

 Object.fromEntries(Object.entries(ALL_STATS).map(([key, { label }]) => [key, label]))

```
);
```

```
export const CATEGORY_VARIANTS = Object.freeze(  
  Object.fromEntries(Object.entries(ALL_STATS).map(([_, { label, variant }]) => [label,  
    variant])))  
);
```

```
// 🗨️ Role options for dropdowns
```

```
export const ROLE_OPTIONS = Object.freeze(  
  VALID_ROLES.map((role) => ({  
    value: role,  
    label: role.charAt(0).toUpperCase() + role.slice(1),  
  })))  
);
```

```
// 🚦 Base permissions template (all possible actions)
```

```
export const BASE_PERMISSIONS = Object.freeze({  
  viewDashboard: true,  
  manageResidents: false,  
  fileComplaints: false,  
  fileComplaintsForResidents: false,  
  viewOwnComplaints: false,  
  viewAllComplaints: false,  
  manageComplaints: false,  
  generateCertificates: false,  
  viewFinancialRecords: false,
```

manageFinancialRecords: false,
incomingPayments: false,
barangayExpenses: false,
financialReports: false,
youthRegistryAccess: false,
auditBarangayData: false,
manageSettings: false,
settings: false,
viewDocuments: false,
viewOwnDocuments: false,
documentRequest: false,
requestDocuments: false,
manageDocuments: false,
issuedDocuments: false,
pendingRequests: false,
paidRequests: false,
rejectedRequests: false,
viewBusinesses: false,
viewOwnBusinesses: false,
registerBusinesses: false,
registerResidentBusinesses: false,
manageBusinesses: false,
viewIncidents: false,
viewOwnIncidents: false,
reportIncidents: false,
manageIncidents: false,

```
manageAnnouncements: false,  
manageEvents: false,  
createAccounts: false,  
viewUsers: false,  
manageUsers: false,  
viewStatus: false,  
submitFeedback: false,  
});
```

```
// 🧠 Utility to apply overrides
```

```
const applyOverrides = (keys = []) =>  
  Object.fromEntries(keys.map((perm) => [perm, true]));
```

```
// 🚦 Final role-permission map (built from shared JSON)
```

```
export const ROLE_PERMISSIONS = Object.freeze(  
  Object.fromEntries(  
    VALID_ROLES.map((role) => [role, applyOverrides(roleOverrides[role])])  
  )  
);
```

```
// 🗝️ Collection → permission mapping
```

```
// Each collection can map to one or more permissions
```

```
export const COLLECTION_PERMISSIONS = Object.freeze({  
  residents: ["manageResidents"],  
  businesses: ["viewBusinesses"],
```

```
complaints: ["viewAllComplaints", "fileComplaints", "viewOwnComplaints",
"manageComplaints"],
incidents: ["viewIncidents", "reportIncidents"],
documents: ["viewDocuments", "viewOwnDocuments", "requestDocuments"],
logins: ["auditBarangayData"],
youth: ["youthRegistryAccess"],
collections: ["viewFinancialRecords"],
});
```

```
//  END Auto-derived ROLE_COLLECTIONS (handles arrays of permissions)
```

```
export const ROLE_COLLECTIONS = Object.freeze(
  Object.fromEntries(
    VALID_ROLES.map((role) => [
      role,
      Object.entries(COLLECTION_PERMISSIONS)
        .filter(([_, perms]) =>
          perms.some((perm) => ROLE_PERMISSIONS[role][perm])
        )
        .map(([collection]) => collection),
    ])
  )
);
```

```
...
```

```
## frontend\src\config\stats.js
```

```
` `` javascript
```

```
// 🇮🇹 Unified stats + role access config
```

```
export const statsConfig = {
```

```
  residents: {
```

```
    label: "Residents",
```

```
    variant: "accent",
```

```
    icon: "👥",
```

```
    roles: ["admin", "staff"],
```

```
  },
```

```
  businesses: {
```

```
    label: "Businesses",
```

```
    variant: "success",
```

```
    icon: "👛",
```

```
    roles: ["admin", "staff"],
```

```
  },
```

```
  complaints: {
```

```
    label: "Complaints Filed",
```

```
    variant: "danger",
```

```
    icon: "🗣️",
```

```
    roles: ["admin", "staff", "resident"],
```

```
  },
```

```
  incidents: {
```

```
    label: "Incidents Reported",
```

```
    variant: "warning",
```



```
    icon: " ⚠️ ",
    roles: ["admin", "staff"],
  },
  documents: {
    label: "Documents Issued",
    variant: "info",
    icon: " 📄 ",
    roles: ["admin", "secretary", "resident"],
  },
  logins: {
    label: "Login Records",
    variant: "neutral",
    icon: " 🗂️ ",
    roles: ["admin", "dilg"],
  },
  youth: {
    label: "Youth Registry",
    variant: "youth",
    icon: " 🧑 ",
    roles: ["admin", "sk"],
  },
  fees: {
    label: "Fees Collected",
    variant: "treasurer",
    icon: " 💰 ",
```

```
    roles: ["admin", "treasurer"],  
  },  
};
```

```
````
```

```
frontend\src\config\validationConfig.js
```

```
```javascript
```

```
// src/config/validationConfig.js
```

```
const validationConfig = {  
  document: {  
    rules: [  
      {  
        field: "documentType",  
        validate: (val) => val && val.trim().length > 0,  
        message: " ⚠ Document Type is required",  
      },  
      {  
        field: "fee",  
        validate: (val) => val !== null && val !== undefined && Number.isFinite(val) && val >= 0,  
        message: " ⚠ Fee must be a non-negative number",  
      },  
    ],  
  },  
};
```

```
business: {
  rules: [
    {
      field: "businessType",
      validate: (val) => val && val.trim().length > 0,
      message: " ⚠ Business Type is required",
    },
    {
      field: "registrationFee",
      validate: (val) => typeof val === "number" && val >= 0,
      message: " ⚠ Registration Fee must be non-negative",
    },
    {
      field: "annualFee",
      validate: (val) => typeof val === "number" && val >= 0,
      message: " ⚠ Annual Fee must be non-negative",
    },
  ],
},
misc: {
  rules: [
    {
      field: "miscType",
      validate: (val) => val && val.trim().length > 0,
      message: " ⚠ Misc Type is required",
    },
  ],
}
```

```

    {
      field: "fee",
      validate: (val) => typeof val === "number" && val >= 0,
      message: " ⚠ Misc Fee must be non-negative",
    },
    {
      field: "enabled",
      validate: (val) => typeof val === "boolean",
      message: " ⚠ Enabled must be true or false",
    },
  ],
},
};

```

```
export default validationConfig;
```

```

` ` `

```

```
## frontend\src\context\NotificationContext.js
```

```
` ` ` javascript
```

```
// context/NotificationContext.js
```

```
import { createContext, useContext, useEffect, useState, useRef } from "react";
```

```
import { NotificationsAPI } from "../services/api";
```

```
import { getAuth } from "firebase/auth";
```

```
const NotificationContext = createContext();  
export const useNotifications = () => useContext(NotificationContext);
```

```
// 🗝 Token utilities
```

```
async function refreshToken() {  
  const auth = getAuth();  
  const user = auth.currentUser;  
  if (!user) return null;  
  return user.getIdToken(true); // force refresh  
}
```

```
function decodeToken(jwt) {  
  try {  
    return JSON.parse(atob(jwt.split(".")[1]));  
  } catch {  
    return null;  
  }  
}
```

```
function normalizeWsBase(url) {  
  return url  
    .replace(/^http:/i, "ws:")  
    .replace(/^https:/i, "wss:")  
    .replace("ws://localhost:", "ws://127.0.0.1:")  
    .replace("wss://localhost:", "wss://127.0.0.1:");  
}
```

```

function toMillis(value) {
  if (!value) return 0;

  if (value instanceof Date) return value.getTime();

  if (typeof value === "number") return value;

  if (typeof value === "string") {
    const parsed = Date.parse(value);
    return Number.isNaN(parsed) ? 0 : parsed;
  }

  if (typeof value === "object") {
    const seconds = value.seconds ?? value._seconds;

    if (typeof seconds === "number") return seconds * 1000;

    if (value.toDate && typeof value.toDate === "function") {
      const date = value.toDate();
      return date instanceof Date ? date.getTime() : 0;
    }
  }

  return 0;
}

```

```

function sortByNewest(list) {
  return [...list].sort((a, b) => {
    const left = toMillis(a.timestamp);
    const right = toMillis(b.timestamp);
    return right - left;
  });
}

```

```
}
```

```
export const NotificationProvider = ({ children, token }) => {  
  const [notifications, setNotifications] = useState([]);  
  const [unreadCount, setUnreadCount] = useState(0);  
  
  const wsRef = useRef(null);  
  const reconnectTimerRef = useRef(null);  
  const retryCountRef = useRef(0);  
  const authRetryRef = useRef(0);  
  const shouldReconnectRef = useRef(true);  
  
  const countUnread = (list) => list.filter((n) => !n.read).length;  
  
  // 📖 Load initial history  
  useEffect(() => {  
    if (!token) return;  
  
    const loadHistory = async () => {  
      try {  
        const history = await NotificationsAPI.list();  
        setNotifications(sortByNewest(history));  
        setUnreadCount(countUnread(history));  
      } catch (err) {  
        console.error("⚠️ Failed to load notifications", err);  
      }  
    }  
  });  
}
```

```
};
```

```
loadHistory();
```

```
}, [token]);
```

```
// 🚦 WebSocket connection + reconnection
```

```
useEffect(() => {
```

```
  if (!token) return;
```

```
  shouldReconnectRef.current = true;
```

```
  const connect = async (authToken = token) => {
```

```
    const payload = decodeToken(authToken);
```

```
    // Refresh if expired
```

```
    if (!payload || Date.now() / 1000 > payload.exp) {
```

```
      console.warn("🔙 Token expired, attempting refresh...");
```

```
      const newToken = await refreshToken();
```

```
      if (!newToken) {
```

```
        console.error("❌ Failed to refresh token, not reconnecting");
```

```
        return;
```

```
      }
```

```
      return connect(newToken);
```

```
    }
```

```
const apiBase = process.env.REACT_APP_API_BASE_URL || "http://127.0.0.1:8000";
```

```
const wsBase = normalizeWsBase(process.env.REACT_APP_WS_BASE_URL || apiBase);
```



```
const ws = new WebSocket(`${wsBase}/ws/notifications`);
```

```
wsRef.current = ws;
```

```
ws.onopen = () => {  
  ws.send(JSON.stringify({ token: authToken }));  
  console.log("✅ WebSocket connected");  
  retryCountRef.current = 0;  
  authRetryRef.current = 0;  
};
```

```
ws.onmessage = (event) => {  
  try {  
    const incoming = JSON.parse(event.data);  
    setNotifications((prev) => {  
      const deduped = prev.filter((n) => n.id !== incoming.id);  
      const updated = sortByNewest([incoming, ...deduped]);  
      setUnreadCount(countUnread(updated));  
      return updated;  
    });  
  } catch (err) {  
    console.error("⚠️ Failed to parse notification payload", err);  
  }  
};
```

```
ws.onerror = (err) => {
```

```
console.error(" ⚠ WebSocket error", err);  
};  
  
ws.onclose = async (event) => {  
  console.log(" ❌ WebSocket disconnected", event.code, event.reason);  
  
  if (!shouldReconnectRef.current || ws !== wsRef.current) {  
    return;  
  }  
  
  if (event.code === 4001) {  
    if (authRetryRef.current >= 2) {  
      console.error(" ❌ WebSocket auth failed repeatedly — stopping reconnect attempts");  
      return;  
    }  
    authRetryRef.current += 1;  
    console.error("Forbidden — trying token refresh before reconnect");  
    const newToken = await refreshToken();  
    if (newToken) connect(newToken);  
    return;  
  }  
  
  if (!navigator.onLine) {  
    console.error("Offline — not retrying until back online");  
    return;  
  }  
}
```

```

    }

    if (retryCountRef.current < 5) {
      retryCountRef.current++;

      const delay = Math.min(5000 * retryCountRef.current, 30000);
      reconnectTimerRef.current = setTimeout(() => connect(authToken), delay);
    } else {
      console.error("Max retries reached — giving up");
    }
  };
};

connect();

return () => {
  shouldReconnectRef.current = false;

  if (reconnectTimerRef.current) clearTimeout(reconnectTimerRef.current);

  if (
    wsRef.current &&
    (wsRef.current.readyState === WebSocket.OPEN ||
      wsRef.current.readyState === WebSocket.CONNECTING)
  ) {
    wsRef.current.close(1000, "Notification context cleanup");
  }

  wsRef.current = null;
};

```

```
}, [token]);
```

```
// 🛠 Notification actions
```

```
const updateState = (updater) => {
```

```
  setNotifications((prev) => {
```

```
    const updated = updater(prev);
```

```
    setUnreadCount(countUnread(updated));
```

```
    return updated;
```

```
  });
```

```
};
```

```
const resetUnread = () => setUnreadCount(0);
```

```
const markAsRead = async (id) => {
```

```
  updateState((prev) => prev.map((n) => (n.id === id ? { ...n, read: true } : n)));
```

```
  try {
```

```
    await NotificationsAPI.markAsRead(id);
```

```
  } catch (err) {
```

```
    console.error(" ⚠ Failed to mark notification as read", err);
```

```
    refreshNotifications();
```

```
  }
```

```
};
```

```
const deleteNotification = async (id) => {
```

```
  updateState((prev) => prev.filter((n) => n.id !== id));
```

```
  try {
```

```
    await NotificationsAPI.delete(id);
  } catch (err) {
    console.error(" ⚠️ Failed to delete notification", err);
  }
};
```

```
const bulkDeleteNotifications = async (onlyRead = true) => {
  updateState((prev) => (onlyRead ? prev.filter((n) => !n.read) : []));
  try {
    if (onlyRead) {
      await NotificationsAPI.bulkDelete(true);
    } else {
      await NotificationsAPI.deleteAll();
    }
  } catch (err) {
    console.error(" ⚠️ Failed to bulk delete notifications", err);
    refreshNotifications();
  }
};
```

```
const markAllAsRead = async () => {
  updateState((prev) => prev.map((n) => ({ ...n, read: true })));
  try {
    await NotificationsAPI.markAllAsRead();
  } catch (err) {
    console.error(" ⚠️ Failed to mark all notifications as read", err);
  }
};
```

```
    refreshNotifications();  
  }  
};
```

```
const refreshNotifications = async () => {  
  try {  
    const history = await NotificationsAPI.list();  
    setNotifications(sortByNewest(history));  
    setUnreadCount(countUnread(history));  
  } catch (err) {  
    console.error(" ⚠️ Failed to refresh notifications", err);  
  }  
};
```

```
return (  
  <NotificationContext.Provider  
    value={{  
      notifications,  
      unreadCount,  
      resetUnread,  
      markAsRead,  
      deleteNotification,  
      bulkDeleteNotifications,  
      markAllAsRead,  
      refreshNotifications,  
    }}  
  )
```

```

    >
    {children}
  </NotificationContext.Provider>
);
};

```

```

` ` `

```

```

## frontend\src\context\ThemeContext.js

```

```

` ` ` javascript

```

```

import React, { createContext, useState, useEffect, useContext } from "react";

```

```

const ThemeContext = createContext();

```

```

export const ThemeProvider = ({ children }) => {
  const [isDarkMode, setIsDarkMode] = useState(() => {
    const stored = localStorage.getItem("darkMode");
    return stored
      ? JSON.parse(stored)
      : window.matchMedia("(prefers-color-scheme: dark)").matches;
  });

```

```

// Effect 1: Sync theme attribute whenever isDarkMode changes

```

```

useEffect(() => {
  document.documentElement.setAttribute("data-theme", isDarkMode ? "dark" : "light");

```

```
localStorage.setItem("darkMode", JSON.stringify(isDarkMode));  
}, [isDarkMode]);
```

```
// Effect 2: Listen for system preference changes (only once)
```

```
useEffect(() => {  
  const mediaQuery = window.matchMedia("(prefers-color-scheme: dark)");  
  const handler = (e) => setIsDarkMode(e.matches);  
  mediaQuery.addEventListener("change", handler);  
  return () => mediaQuery.removeEventListener("change", handler);  
}, []);
```

```
const toggleDarkMode = () => setIsDarkMode((prev) => !prev);
```

```
return (  
  <ThemeContext.Provider value={{ isDarkMode, toggleDarkMode }}>  
    {children}  
  </ThemeContext.Provider>  
);  
};
```

```
export const useTheme = () => useContext(ThemeContext);
```

```
...
```

```
## frontend\src\context\UserContext.js
```



```
` `` `javascript
// context/UserContext.js

import {
  createContext,
  useState,
  useEffect,
  useContext,
  useCallback,
} from "react";

import {
  onAuthStateChanged,
  signOut,
  getIdToken,
  getIdTokenResult,
} from "firebase/auth";

import { doc, getDoc } from "firebase/firestore";

import { auth, db } from "../services/firebase";

import { ROLE_PERMISSIONS, VALID_ROLES } from "../config/roles";

const UserContext = createContext();

const DEBUG = process.env.NODE_ENV !== "production";

const CACHE_TTL = 1000 * 60 * 30; // 30 minutes

// ✅ Normalize role safely

const normalizeRole = (role) => {
  const normalized = role?.trim().toLowerCase();
```

```

if (!normalized || !VALID_ROLES.includes(normalized)) {
  if (DEBUG) console.warn(` ⚠️ Unknown or missing role "${role}"`);
  return null; // invalid role → unauthorized
}
return normalized;
};

```

```

export const UserProvider = ({ children }) => {
  const [userInfo, setUserInfo] = useState(null);
  const [role, setRole] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [token, setToken] = useState(null);

```

```

const isAdmin = role === "admin";

```

```

// ✅ Permission checks

```

```

const can = useCallback(
  (action) => {
    const rolePerms = ROLE_PERMISSIONS[role] || {};
    const allowed = Boolean(rolePerms[action]);
    const actionExists = Object.prototype.hasOwnProperty.call(rolePerms, action);

    if (DEBUG && !allowed && actionExists && role !== "resident") {
      console.warn(` 🚫 Role "${role}" lacks permission for "${action}"`);
    }
  }
);

```

```
    }  
    return allowed;  
  },  
  [role]  
);
```

```
const hasAccess = useCallback(  
  (allowedRoles = [], allowAdminOverride = true) =>  
    allowedRoles.includes(role) || (allowAdminOverride && isAdmin),  
  [role, isAdmin]  
);
```

```
const updateUserInfo = useCallback((info) => {  
  const enriched = { ...info, cachedAt: Date.now() };  
  setUserInfo(enriched);  
  sessionStorage.setItem("userInfo", JSON.stringify(enriched));  
}, []);
```

```
const updateRole = useCallback((newRole) => {  
  setRole(normalizeRole(newRole));  
}, []);
```

```
const clearUserState = useCallback(() => {  
  setUserInfo(null);  
  setRole(null);  
  setIsAuthenticated(false);
```

```
setError(null);

sessionStorage.removeItem("userInfo");

sessionStorage.removeItem("authToken");

}, []);
```

```
//  Error helper
```

```
const safeSetError = (msg) => {

  setError(msg);

  setTimeout(() => setError(null), 5000);

};
```

```
//  Token helpers
```

```
const getToken = useCallback(async (forceRefresh = true) => {

  if (!auth.currentUser) return null;

  try {

    const freshToken = await getIdToken(auth.currentUser, forceRefresh);

    sessionStorage.setItem("authToken", freshToken);

    setToken(freshToken);

    return freshToken;

  } catch (err) {

    console.error("  Failed to get ID token:", err);

    safeSetError("Token retrieval failed");

    const cached = sessionStorage.getItem("authToken");

    return cached || null;

  }

}, []);
```

```

const getTokenResult = useCallback(async (forceRefresh = false) => {
  if (!auth.currentUser) return null;

  try {
    return await getIdTokenResult(auth.currentUser, forceRefresh);
  } catch (err) {
    console.error("❌ Failed to get ID token result:", err);
    safeSetError("Token result retrieval failed");
    return null;
  }
}, []);

```

```

const logout = useCallback(async () => {
  clearUserState();

  try {
    await signOut(auth);
  } catch (err) {
    console.error("❌ Sign-out failed:", err);
    safeSetError("Sign-out failed");
  }
}, [clearUserState]);

```

```

// ✅ Fetch profile from users/{uid} or residents/{uid}

const fetchUserProfile = useCallback(async (uid) => {
  try {
    let ref = doc(db, "users", uid);

```

```
let snapshot = await getDoc(ref);
```

```
if (!snapshot.exists()) {
```

```
  ref = doc(db, "residents", uid);
```

```
  snapshot = await getDoc(ref);
```

```
}
```

```
if (!snapshot.exists()) {
```

```
  console.warn(" ⚠️ No Firestore profile found for UID:", uid);
```

```
  return null;
```

```
}
```

```
return { ...snapshot.data(), uid };
```

```
} catch (err) {
```

```
  console.error(" ❌ Error fetching user profile:", err);
```

```
  safeSetError("Failed to load user profile");
```

```
  return null;
```

```
}
```

```
}, []);
```

```
// ✅ Restore session from cache with expiry
```

```
useEffect(() => {
```

```
  try {
```

```
    const cached = sessionStorage.getItem("userInfo");
```

```
    if (cached) {
```

```
      const parsed = JSON.parse(cached);
```

```

    if (parsed?.cachedAt && Date.now() - parsed.cachedAt < CACHE_TTL) {
      setUserInfo(parsed);
      setRole(normalizeRole(parsed.role));
      setIsAuthenticated(true);
    }
  }
} catch (err) {
  console.warn(" ⚠️ Failed to hydrate session cache:", err);
} finally {
  setLoading(false);
}
}, []);

```

// ✅ Main auth listener

```

useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, async (user) => {
    if (!user) {
      if (!loading) clearUserState();
      setLoading(false);
      return;
    }

    try {
      const tokenResult = await getIdTokenResult(user, true);
      const claimRole = tokenResult.claims.role
        ? normalizeRole(tokenResult.claims.role)
    }
  }

```

```
: null;
```

```
const profile = await fetchUserProfile(user.uid);
```

```
const effectiveRole = claimRole || normalizeRole(profile?.role);
```

```
if (!effectiveRole) {
```

```
  clearUserState();
```

```
  safeSetError("Unauthorized role");
```

```
  setLoading(false);
```

```
  return;
```

```
}
```

```
const enriched = { ...(profile || {}), uid: user.uid, role: effectiveRole };
```

```
updateRole(effectiveRole);
```

```
updateUserInfo(enriched);
```

```
setIsAuthenticated(true);
```

```
setError(null);
```

```
const freshToken = await getToken(true);
```

```
setToken(freshToken);
```

```
} catch (err) {
```

```
  console.error("✖ Error during auth state handling:", err);
```

```
  safeSetError("Authentication error");
```

```
}
```

```
setLoading(false);
```



```
});
```

```
return () => unsubscribe();
```

```
}, [fetchUserProfile, clearUserState, updateRole, updateUserInfo, getToken, loading]);
```

```
//  Token auto-refresh every 55 minutes
```

```
useEffect(() => {
```

```
  if (!auth.currentUser) return;
```

```
  const interval = setInterval(() => {
```

```
    getToken(true);
```

```
  }, 55 * 60 * 1000);
```

```
  return () => clearInterval(interval);
```

```
}, [getToken]);
```

```
//  Safety timeout
```

```
useEffect(() => {
```

```
  const timeout = setTimeout(() => {
```

```
    if (loading) {
```

```
      console.warn("⌚ Timeout: forcing loading to false");
```

```
      setLoading(false);
```

```
    }
```

```
  }, 5000);
```

```
  return () => clearTimeout(timeout);
```

```
}, [loading]);
```

```
//  Manual refresh
```

```
const refreshProfile = useCallback(async () => {  
  if (!auth.currentUser) return;  
  const profile = await fetchUserProfile(auth.currentUser.uid);  
  if (profile) {  
    updateUserInfo(profile);  
    updateRole(profile.role);  
  }  
}, [fetchUserProfile, updateUserInfo, updateRole]);
```

```
return (  
  <UserContext.Provider  
    value={{  
      userInfo,  
      role,  
      isAuthenticated,  
      isAdmin,  
      loading,  
      error,  
      logout,  
      getToken,  
      getTokenResult,  
      hasAccess,  
      can,  
      updateUserInfo,  
      updateRole,  
      refreshProfile,
```

```

        token,
      }}
    >
    {children}
  </UserContext.Provider>
);
};

```

```

export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) throw new Error("useUser must be used within a UserProvider");
  return context;
};

```

```

` ` `

```

```

## frontend\src\data\locations.js

```

```

` ` ` javascript

```

```

// src/data/locations.js

```

```

export const PARANAQUE = {
  code: "137607000", // PSGC code
  city: "Parañaque",
  province: "Metro Manila",
  barangays: [
    "Baclaran",

```

```
"BF Homes",  
"Don Bosco",  
"Moonwalk",  
"San Dionisio",  
"San Isidro",  
"San Antonio",  
"Sto. Niño",  
"Sun Valley",  
"Vitalez"  
]  
};
```

```
```
```

```
frontend\src\helpers\validation.js
```

```
```javascript
```

```
// helpers/validation.js
```

```
// Individual validators
```

```
const validateCategory = (category) => {  
  if (!category) return "Category is required.";   
  const allowed = ["Noise", "Service", "Neighbor", "Other"];  
  if (!allowed.includes(category)) return "Invalid category.";   
  return null;  
};
```

```
const validateDescription = (description) => {  
  if (!description || description.trim().length < 5) {  
    return "Description must be at least 5 characters.";  
  }  
  return null;  
};
```

```
const validateLocation = (location) => {  
  if (!location) return "Location is required.";  
  return null;  
};
```

```
const validateResidentUid = (uid) => {  
  if (!uid) return "Resident UID is required.";  
  if (typeof uid !== "string") return "Resident UID must be a string.";  
  if (!/^[A-Za-z0-9]+$/.test(uid)) return "Resident UID must be alphanumeric.";  
  if (uid.length < 20 || uid.length > 40) {  
    return "Resident UID length looks invalid.";  
  }  
  return null;  
};
```

// Main payload validator

```
export const validateComplaintPayload = (payload) => {  
  const errors = [];
```

```
const categoryError = validateCategory(payload.category);
```

```
if (categoryError) errors.push(categoryError);
```

```
const descriptionError = validateDescription(payload.description);
```

```
if (descriptionError) errors.push(descriptionError);
```

```
const locationError = validateLocation(payload.location);
```

```
if (locationError) errors.push(locationError);
```

```
const uidError = validateResidentUid(payload.filed_by);
```

```
if (uidError) errors.push(uidError);
```

```
return errors;
```

```
};
```

```
```\n
```

```
frontend\\src\\hooks\\useAllDocuments.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
```

```
import { api, endpoints } from "../services/api";
```

```
import { getAuth } from "firebase/auth";
```

```
export const useAllDocuments = () => {
```

```
const [docs, setDocs] = useState([]);

const [loading, setLoading] = useState(true);

const [error, setError] = useState(null);

useEffect(() => {

  const timer = setTimeout(async () => {

    try {

      const auth = getAuth();

      const user = auth.currentUser;

      if (!user) throw new Error("No logged-in user");

      // 🔑 get custom claims or role from your backend

      const token = await user.getIdTokenResult();

      const role = token.claims.role;

      let url = endpoints.documents;

      let params = {};

      if (role === "resident") {

        url = `${endpoints.documents}/my`;

        params = { resident_id: user.uid };

      }

      const res = await api.get(url, { params });

      setDocs(res.data);

    } catch (err) {
```

```

        console.error(" ❌ Error fetching documents:", err);
        setError(err.message);
    } finally {
        setLoading(false);
    }
}, 1000);

return () => clearTimeout(timer);
}, []);

return { docs, loading, error };
};

```

```

` ` `

```

```

## frontend\src\hooks\useCounters.js

```

```

` ` ` javascript

```

```

import { useEffect, useState } from "react";

```

```

import { api } from "../services/api";

```

```

export const useCounters = () => {

```

```

    const [counters, setCounters] = useState([]);

```

```

    const [loading, setLoading] = useState(false);

```

```

    useEffect(() => {

```



```

const fetchCounters = async () => {
  setLoading(true);
  try {
    const { data } = await api.get("/api/counters");
    setCounters(data);
  } catch (err) {
    console.error(" ❌ Error fetching counters:", err);
  } finally {
    setLoading(false);
  }
};

fetchCounters();
}, []);

return { counters, loading };
};

...

```

frontend\src\hooks\useDisbursements.js

```

```javascript
import { useEffect, useState, useCallback } from "react";
import { db } from "../services/firebase";
import { collection, onSnapshot, getDocs } from "firebase/firestore";

```

```

export function useDisbursements() {
 const [disbursements, setDisbursements] = useState([]);
 const [totals, setTotals] = useState({});
 const [byCategory, setByCategory] = useState({});
 const [dailySummary, setDailySummary] = useState({});

 //  Shared calculation logic
 const recalc = useCallback((data) => {
 setDisbursements(data);
 setTotals(calculateTotals(data));
 setByCategory(calculateByCategory(data));
 setDailySummary(calculateDailySummary(data));
 }, []);

 //  Real-time subscription
 useEffect(() => {
 const disbursementsRef = collection(db, "disbursements");
 const unsubscribe = onSnapshot(disbursementsRef, snapshot => {
 const data = snapshot.docs.map(doc => ({
 id: doc.id,
 ...doc.data(),
 }));
 recalc(data);
 });
 return unsubscribe;
 });

```

```
}, [recalc]);
```

```
//  Manual refresh (e.g. after API mutation)
```

```
const refresh = useCallback(async () => {
 try {
 const snapshot = await getDocs(collection(db, "disbursements"));
 const data = snapshot.docs.map(doc => ({
 id: doc.id,
 ...doc.data(),
 }));
 recalc(data);
 } catch (err) {
 console.error("Failed to refresh disbursements", err);
 }
}, [recalc]);
```

```
return { disbursements, totals, byCategory, dailySummary, refresh };
}
```

```
/* ----- Helper Functions ----- */
```

```
function calculateTotals(disbursements) {
 const spentApproved = disbursements
 .filter(d => d.status === "approved")
 .reduce((sum, d) => sum + (d.amount || 0), 0);
```

```
const spentPending = disbursements
 .filter(d => d.status === "pending")
 .reduce((sum, d) => sum + (d.amount || 0), 0);
```

```
const countApproved = disbursements.filter(d => d.status === "approved").length;
const countPending = disbursements.filter(d => d.status === "pending").length;
```

```
return {
 spentApproved,
 spentPending,
 countApproved,
 countPending,
 totalCount: disbursements.length,
};
}
```

```
function calculateByCategory(disbursements) {
 return disbursements.reduce((acc, d) => {
 const category = d.category || "Miscellaneous";
 acc[category] = (acc[category] || 0) + (d.amount || 0);
 return acc;
 }, {});
}
```

```
function calculateDailySummary(disbursements) {
 return disbursements.reduce((acc, d) => {
```

```

const date = normalizeDate(d.date);
if (date) {
 const dateKey = date.toLocaleDateString();
 acc[dateKey] = (acc[dateKey] || 0) + (d.amount || 0);
}
return acc;
}, {});
}

```

```

function normalizeDate(dateValue) {
 if (!dateValue) return null;
 if (dateValue instanceof Date) return dateValue;
 if (typeof dateValue?.toDate === "function") return dateValue.toDate();
 const parsed = new Date(dateValue);
 return isNaN(parsed) ? null : parsed;
}

```

```

frontend\src\hooks\useDocumentCounters.js

```javascript

```

import { useEffect, useState } from "react";
import { collection, onSnapshot } from "firebase/firestore";
import { db } from "../services/firebase";

```

```

export const useDocumentCounters = () => {
 const [counters, setCounters] = useState([]);

 useEffect(() => {
 const ref = collection(db, "counters");

 const unsubscribe = onSnapshot(ref, (snapshot) => {
 const docs = snapshot.docs.map((doc) => ({
 id: doc.id,
 ...doc.data(),
 }));
 setCounters(docs);
 });

 return () => unsubscribe();
 }, []);

 return counters;
};

```

```

frontend\src\hooks\useEmail.js

```javascript

```
import { useState, useCallack } from "react";
```

```
import { sendEmail } from "../services/email";

export function useEmail() {
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState(null);
 const [result, setResult] = useState(null);

 const triggerEmail = useCallback(async (payload) => {
 setLoading(true);
 setError(null);
 setResult(null);

 try {
 const res = await sendEmail(payload);
 setResult(res);
 return res;
 } catch (err) {
 setError(err);
 throw err;
 } finally {
 setLoading(false);
 }
 }, []);

 return { triggerEmail, loading, error, result };
}
```

```

frontend\src\hooks\useEnrichedRequests.js

```javascript

import { useEffect, useState } from "react";

import { DocumentsAPI } from "../services/api";

import { getAuth, onAuthStateChanged } from "firebase/auth";

import { getDoc, doc } from "firebase/firestore";

import { db } from "../services/firebase";

const normalizeDoc = (doc) => ({

...doc,

document\_id: doc.documentId || doc.document\_id || doc.id,

resident\_id: doc.residentId || doc.resident\_id,

document\_type: doc.documentType || doc.document\_type,

created\_at: doc.createdAt || doc.created\_at,

updated\_at: doc.updatedAt || doc.updated\_at,

paymentStatus: doc.paymentStatus ?? doc.payment\_status,

transactionId: doc.transactionId ?? doc.transaction\_id,

paymentIntentId: doc.paymentIntentId ?? doc.payment\_intent\_id,

referenceNumber: doc.referenceNumber ?? doc.reference\_number,

attachments: doc.attachments ?? {},

});



```
const getResidentName = async (uid) => {
 if (!uid) return null;
 try {
 const snap = await getDoc(doc(db, "residents", uid));
 return snap.exists() ? snap.data().fullName : null;
 } catch (err) {
 console.error(` ❌ Failed to fetch resident name for ${uid}:`, err.message);
 return null;
 }
};
```

```
export const useEnrichedRequests = () => {
 const [pending, setPending] = useState([]);
 const [toVerify, setToVerify] = useState([]);
 const [readyToIssue, setReadyToIssue] = useState([]);
 const [approved, setApproved] = useState([]);
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState(null);
 const [authReady, setAuthReady] = useState(false);
```

```
const fetchRequests = async () => {
 setLoading(true);
 setError(null);
 try {
 const data = await DocumentsAPI.list();
 const normalized = data.map(normalizeDoc);
```

```

const enrich = async (docs) =>
 Promise.all(
 docs.map(async (doc) => {
 const name = await getResidentName(doc.resident_id);
 return { ...doc, resident_name: name };
 })
);

setPending(await enrich(normalized.filter((doc) => doc.status === "pending")));
setToVerify(await enrich(normalized.filter((doc) => doc.status ===
"payment_submitted")));
setReadyToIssue(await enrich(
 normalized.filter((doc) => doc.status === "paid")
));
setApproved(await enrich(normalized.filter((doc) => doc.status === "approved")));
} catch (err) {
 console.error("❌ Error fetching requests:", err.message);
 setError(err.message);
} finally {
 setLoading(false);
}
};

useEffect(() => {
 const unsubscribe = onAuthStateChanged(getAuth(), async (user) => {

```

```
 if (user) {
 await user.getIdToken(true);
 setAuthReady(true);
 }
 });
 return () => unsubscribe();
}, []);
```

```
useEffect(() => {
 if (authReady) {
 fetchRequests();
 }
}, [authReady]);
```

```
return {
 pending,
 toVerify,
 readyToIssue,
 approved,
 loading,
 error,
 fetchRequests,
};
};
```

```

frontend\src\hooks\useFees.js

```javascript

```
import { useState, useEffect, useCallback } from "react";
import { FeesAPI } from "../services/api";
import {
 buildDocumentPayload,
 buildBusinessPayload,
 buildMiscPayload,
} from "../utils/payloadBuilders";

export function useFees(delayMs = 500) {
 const [documentFees, setDocumentFees] = useState([]);
 const [businessFees, setBusinessFees] = useState([]);
 const [miscFees, setMiscFees] = useState([]);
 const [loading, setLoading] = useState(false);
 const [error, setError] = useState(null);

 const handleError = (err, fallbackMsg) => {
 console.error(err);
 setError(err?.message || fallbackMsg);
 };

 const refreshData = useCallback(async () => {
```

```

setLoading(true);
setError(null);
try {
 const [docs, businesses, misc] = await Promise.all([
 FeesAPI.listDocuments(),
 FeesAPI.listBusinesses(),
 FeesAPI.listMisc(),
]);
 setDocumentFees(docs || []);
 setBusinessFees(businesses || []);
 setMiscFees(misc || []);
} catch (err) {
 handleError(err, "Failed to fetch fees");
} finally {
 setLoading(false);
}
}, []);

```

// 🚀 Fetch on mount with delay

```

useEffect(() => {
 const timer = setTimeout(() => {
 refreshData();
 }, delayMs);

 return () => clearTimeout(timer);
}, [refreshData, delayMs]);

```

```
return {
 documentFees,
 businessFees,
 miscFees,
 loading,
 error,
 refreshData,
 updateDocumentFee: async (id, item, key, value) => {
 try {
 const payload = buildDocumentPayload(item, key, value);
 await FeesAPI.updateDocument(id, payload);
 setDocumentFees(prev =>
 prev.map(d => (d.id === id ? { ...d, ...payload } : d))
);
 } catch (err) {
 handleError(err, "Failed to update document fee");
 }
 },
 updateBusinessFee: async (id, item, key, value) => {
 try {
 const payload = buildBusinessPayload(item, key, value);
 await FeesAPI.updateBusiness(id, payload);
 setBusinessFees(prev =>
 prev.map(b => (b.id === id ? { ...b, ...payload } : b))
);
 }
 }
}
```

```

 } catch (err) {
 handleError(err, "Failed to update business fee");
 }
 },
 updateMiscFee: async (id, item, key, value) => {
 try {
 const payload = buildMiscPayload(item, key, value);
 await FeesAPI.updateMisc(id, payload);
 setMiscFees(prev =>
 prev.map(m => (m.id === id ? { ...m, ...payload } : m))
);
 } catch (err) {
 handleError(err, "Failed to update miscellaneous fee");
 }
 },
 deleteDocumentFee: async (id) => {
 try {
 await FeesAPI.deleteDocument(id);
 setDocumentFees(prev => prev.filter(d => d.id !== id));
 } catch (err) {
 handleError(err, "Failed to delete document fee");
 }
 },
 deleteBusinessFee: async (id) => {
 try {
 await FeesAPI.deleteBusiness(id);

```

```

 setBusinessFees(prev => prev.filter(b => b.id !== id));
 } catch (err) {
 handleError(err, "Failed to delete business fee");
 }
},
deleteMiscFee: async (id) => {
 try {
 await FeesAPI.deleteMisc(id);
 setMiscFees(prev => prev.filter(m => m.id !== id));
 } catch (err) {
 handleError(err, "Failed to delete miscellaneous fee");
 }
},
};
}

```

```

frontend\src\hooks\useFirestoreCollection.js

```javascript

```

import { useEffect, useState } from "react";
import { collection, onSnapshot } from "firebase/firestore";
import { db } from "../services/firebase"; // adjust path if needed

```

/\*\*



```

* Generic Firestore collection listener
* @param {string} collectionName - Firestore collection to listen to
* @returns {Array} documents - Array of { id, ...data }
*/

export const useFirestoreCollection = (collectionName) => {
 const [documents, setDocuments] = useState([]);

 useEffect(() => {
 const ref = collection(db, collectionName);

 const unsubscribe = onSnapshot(ref, (snapshot) => {
 const docs = snapshot.docs.map((doc) => ({
 id: doc.id,
 ...doc.data(),
 }));
 setDocuments(docs);
 });

 return () => unsubscribe();
 }, [collectionName]);

 return documents;
};

```

```
frontend\src\hooks\useFirestoreStats.js
```

```
` `` `javascript
```

```
import { useEffect, useState } from "react";
```

```
import { collection, onSnapshot } from "firebase/firestore";
```

```
import { db } from "../services/firebase";
```

```
/**
```

```
 * Subscribe to a Firestore collection and return aggregated stats by status
```

```
 * @param {string} collectionName - Firestore collection name
```

```
 * @param {string} statusField - Field name in documents that represents status
```

```
 * @param {Array<string>} statuses - List of statuses to track
```

```
 */
```

```
export const useFirestoreStats = (
```

```
 collectionName,
```

```
 statusField = "status",
```

```
 statuses = []
```

```
) => {
```

```
 const [stats, setStats] = useState({ total: 0 });
```

```
 useEffect(() => {
```

```
 const ref = collection(db, collectionName);
```

```
 const unsubscribe = onSnapshot(ref, (snapshot) => {
```

```
 const counts = { total: snapshot.size };
```

```

// Initialize counts
statuses.forEach((s) => (counts[s] = 0));

snapshot.forEach((doc) => {
 const data = doc.data();
 const status = data[statusField];
 if (status && counts[status] !== undefined) {
 counts[status]++;
 }
});

setStats(counts);

});

return () => unsubscribe();
}, [collectionName, statusField, statuses]);

return stats;
};

...

frontend\src\hooks\useMyDocuments.js

```javascript
import { useEffect, useState, useCallback } from "react";

```

```
import { api } from "../services/api";
```

```
const cleanText = (value) => {  
  if (value === undefined || value === null) return null;  
  const text = String(value).trim();  
  if (!text) return null;  
  if (["undefined", "null", "none", "nan"].includes(text.toLowerCase())) return null;  
  return text;  
};
```

```
// 🔧 Normalize Firestore/API field names to consistent snake_case
```

```
const normalizeDoc = (doc) => ({  
  ...doc,  
  document_id: doc.documentId || doc.document_id || doc.id,  
  resident_id: doc.residentId || doc.resident_id,  
  document_type: doc.documentType || doc.document_type,  
  created_at: doc.createdAt || doc.created_at,  
  updated_at: doc.updatedAt || doc.updated_at,  
  reference_number: doc.referenceNumber ?? doc.reference_number,  
  paymentStatus: doc.paymentStatus ?? doc.payment_status,  
  issuedAt: doc.issuedAt ?? doc.issued_at,  
  issuedBy: doc.issuedBy ?? doc.issued_by,  
  fileUrl: doc.fileUrl ?? doc.file_url,  
  transactionId: doc.transactionId ?? doc.transaction_id,  
  paymentIntentId: doc.paymentIntentId ?? doc.payment_intent_id,  
  remarks: cleanText(  

```

```
    doc.remarks ??
    doc.remark ??
    doc.issueRemarks ??
    doc.issue_remarks ??
    doc.extraFields?.remarks ??
    null
  ),
});
```

```
export const useMyDocuments = (residentId) => {
  const [docs, setDocs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchDocs = useCallback(async () => {
    if (!residentId) return; // guard against missing ID
    try {
      setLoading(true);
      setError(null);
      const res = await api.get(`/api/documents/my`, {
        params: { resident_id: residentId },
      });
      // ✅ normalize before setting
      setDocs(res.data.map(normalizeDoc));
    } catch (err) {
      const detail = err.response?.data?.detail || err.message;
    }
  }, [residentId]);
```

```
    console.error(" ❌ Error fetching documents:", detail);
    setError(detail);
  } finally {
    setLoading(false);
  }
}, [residentId]);
```

```
useEffect(() => {
  fetchDocs();
}, [fetchDocs]);
```

```
return { docs, loading, error, refresh: fetchDocs };
};
```

```
````
```

```
frontend\src\hooks\usePasswordReset.js
```

```
```javascript
```

```
import { useState, useCallback } from "react";
import { toast } from "react-toastify";
import { PasswordAPI } from "../services/api";
```

```
export function usePasswordReset() {
  const [loading, setLoading] = useState(false);
  const [email, setEmail] = useState("");
```

```
const [error, setError] = useState(null);
const [success, setSuccess] = useState(false);

const requestReset = useCallback(async (targetEmail) => {
  setLoading(true);
  setError(null);
  setSuccess(false);
  try {
    await PasswordAPI.requestReset(targetEmail);
    setEmail(targetEmail);
    toast.success("📧 Password reset email sent. Check your inbox.");
    setSuccess(true);
  } catch (err) {
    console.error("❌ Reset request failed:", err);
    setError(err);
    toast.error(err?.message || "❌ Failed to send reset email.");
  } finally {
    setLoading(false);
  }
}, []);
```

```
const verifyToken = useCallback(async (token) => {
  setLoading(true);
  setError(null);
  setSuccess(false);
  try {
```

```
const data = await PasswordAPI.verifyToken(token);

setEmail(data.email);

toast.info("🔑 Reset link verified. Please enter a new password.");

setSuccess(true);

return data;
} catch (err) {

  console.error("❌ Token verification failed:", err);

  setError(err);

  toast.error("❌ Reset link is invalid or expired.");

  return null;

} finally {

  setLoading(false);

}

}, []);
```

```
const applyReset = useCallback(async (token, newPassword, confirmPassword) => {

  setLoading(true);

  setError(null);

  setSuccess(false);

  try {

    await PasswordAPI.applyReset(token, newPassword, confirmPassword);

    toast.success("✅ Password reset successful. You can now log in.");

    setSuccess(true);

    setEmail("");

    return true;

  } catch (err) {
```



```
    console.error(" ❌ Reset apply failed:", err);

    setError(err);

    toast.error(err?.message || " ❌ Failed to reset password.");

    return false;

  } finally {

    setLoading(false);

  }

}, []);
```

```
return {
  loading,
  email,
  error,
  success,
  requestReset,
  verifyToken,
  applyReset,
};
}
```

```
```
```

```
frontend\src\hooks\usePayments.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
```

```

import { db } from "../services/firebase";

import { collection, onSnapshot, query, where, getDocs } from "firebase/firestore";

export function usePayments() {

  const [transactions, setTransactions] = useState([]);

  const [totals, setTotals] = useState({ collections: 0, pendingCount: 0, completedCount: 0,
  outstandingAmount: 0 });

  const [revenueByCategory, setRevenueByCategory] = useState({});

  const [dailySummary, setDailySummary] = useState({});

  useEffect(() => {

    const paymentsRef = collection(db, "payments");

    const businessesRef = collection(db, "businesses");

    const documentsRef = collection(db, "documents");

    const unsubPayments = onSnapshot(paymentsRef, async snapshot => {

      const rawPayments = snapshot.docs.map(docSnap => ({

        id: docSnap.id,

        entityType: "payment",

        ...docSnap.data()

      }));

      // Enrich with receipt numbers

      const enrichedPayments = await Promise.all(

        rawPayments.map(async tx => {

          if (tx.transactionId) {

```

```

const receiptQuery = query(
  collection(db, "receipts"),
  where("transactionId", "==", tx.transactionId)
);

const receiptSnap = await getDocs(receiptQuery);
if (!receiptSnap.empty) {
  tx.receiptNumber = receiptSnap.docs[0].data().receiptNumber;
}
}
return tx;
})
);

updateTransactions(enrichedPayments, "payment");
});

const unsubBusinesses = onSnapshot(businessesRef, snapshot => {
  const pendingBiz = snapshot.docs
    .map(docSnap => ({ id: docSnap.id, ...docSnap.data() }))
    .filter(b => b.paymentStatus === "unpaid" || b.status === "for_payment")
    .map(b => ({
      ...b,
      entityType: "business",
      amount: b.amount,
      paymentStatus: "unpaid",
    }));
});

```

```
updateTransactions(pendingBiz, "business");  
});
```

```
const unsubDocuments = onSnapshot(documentsRef, async snapshot => {  
  const docs = await Promise.all(  
    snapshot.docs.map(async docSnap => {  
      const data = docSnap.data();  
      let receiptNumber = "—";  
      let method = data.method || "—";  
  
      if (data.transactionId) {  
        const receiptQuery = query(  
          collection(db, "receipts"),  
          where("transactionId", "=", data.transactionId)  
        );  
        const receiptSnap = await getDocs(receiptQuery);  
        if (!receiptSnap.empty) {  
          const receiptData = receiptSnap.docs[0].data();  
          receiptNumber = receiptData.receiptNumber || receiptNumber;  
          method = receiptData.method || method; // ✅ pull method from receipt if available  
        }  
      }  
    })  
  );  
  
  return {  
    id: docSnap.id,
```

```
    ...data,  
    entityType: "document",  
    entityCategory: data.documentType,  
    ownerName: data.resident?.fullName,  
    amount: data.amount,  
    paymentStatus: data.paymentStatus || data.status || "unpaid",  
    receiptNumber,  
    method,  
  };  
})  
);
```

```
    updateTransactions(docs, "document");  
  });
```

```
return () => {  
  unsubPayments();  
  unsubBusinesses();  
  unsubDocuments();  
};  
, []);
```

```
const updateTransactions = (newItems, type) => {  
  setTransactions(prev => {
```

```
const merged = [...prev.filter(t => t.entityType !== type), ...newItems];

const map = new Map();

for (const tx of merged) {
  const key = tx.transactionId || tx.id;

  // If a payment exists, prefer it over document/business
  if (!map.has(key)) {
    map.set(key, tx);
  } else {
    const existing = map.get(key);
    if (existing.entityType !== "payment" && tx.entityType === "payment") {
      map.set(key, tx);
    }
  }
}

const unique = Array.from(map.values());

setTotals(calculateTotals(unique));
setRevenueByCategory(calculateRevenueByCategory(unique));
setDailySummary(calculateDailySummary(unique));

return unique;
});
};
```

```

    return { transactions, totals, revenueByCategory, dailySummary };
}

/* ----- Helper Functions ----- */

function calculateTotals(data) {
    const collections = data
        .filter(d => d.status === "paid" || d.paymentStatus === "paid")
        .reduce((sum, d) => sum + (d.amount || 0), 0);

    const pendingCount = data.filter(
        d => d.paymentStatus === "unpaid" || d.status === "for_payment" || d.status ===
        "pending"
    ).length;

    const completedCount = data.filter(
        d => d.status === "paid" || d.paymentStatus === "paid"
    ).length;

    const outstandingAmount = data
        .filter(d => d.paymentStatus === "unpaid" || d.status === "for_payment" || d.status ===
        "pending")
        .reduce((sum, d) => sum + (d.amount || 0), 0);

    return { collections, pendingCount, completedCount, outstandingAmount };
}

```

```

function calculateRevenueByCategory(data) {
  return data.reduce((acc, d) => {
    // Prefer entityCategory (e.g., "Activity Permit", "Business Clearance")
    const category = d.entityCategory || d.businessType || d.documentType || d.entityType ||
    "Miscellaneous";

    if (!acc[category]) {
      acc[category] = { paid: 0, unpaid: 0 };
    }

    if (d.paymentStatus === "paid" || d.status === "paid") {
      acc[category].paid += d.amount || 0;
    } else if (d.paymentStatus === "unpaid" || d.status === "for_payment" || d.status ===
    "pending") {
      acc[category].unpaid += d.amount || 0;
    }

    return acc;
  }, {});
}

function calculateDailySummary(data) {
  return data.reduce((acc, d) => {
    // ✅ If not paid, mark as "-"

    if (d.status === "pending" || d.paymentStatus === "unpaid" || d.status === "for_payment")
    {

```



```

const dateKey = "-";
acc[dateKey] = (acc[dateKey] || 0) + (d.amount || 0);
return acc;
}

```

//  Otherwise, use actual datePaid

```
const date = d.datePaid instanceof Date
```

```
? d.datePaid
```

```
: d.datePaid?.toDate?.() || (d.datePaid ? new Date(d.datePaid) : null);
```

```
if (date && !isNaN(date)) {
```

```
const dateKey = date.toLocaleDateString();
```

```
acc[dateKey] = (acc[dateKey] || 0) + (d.amount || 0);
```

```
}
```

```
return acc;
```

```
}, {});
```

```
}
```

```
```
```

```
frontend\src\hooks\usePublicFees.js
```

```
```javascript
```

```
import { useState, useEffect } from "react";
```

```
import { API_BASE_URL } from "../services/api";
```

```

/**
 * Hook for residents: fetches public business/document fees
 * from /api/fees/public/... endpoints and computes totals.
 */

export function usePublicFees() {
  const [businessTypes, setBusinessTypes] = useState([]);
  const [documentTypes, setDocumentTypes] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // 🛠️ Helper to compute totals for businesses
  const computeBusinessTotals = (bt) => {
    const baseFee = bt.fee || 0;
    const registrationFee = bt.registrationFee || 0;
    const annualFee = bt.annualFee || 0;
    const miscFee = bt.miscFeeResolved || 0;
    const enabled = bt.enabled ?? false;

    return {
      ...bt,
      registrationTotal: baseFee + registrationFee + (enabled ? miscFee : 0),
      annualTotal: baseFee + annualFee + (enabled ? miscFee : 0),
    };
  };

  // 🛠️ Helper to compute totals for documents

```

```
const computeDocumentTotals = (doc) => {  
  const baseFee = doc.fee || 0;  
  const miscFee = doc.miscFeeResolved || 0;  
  const enabled = doc.enabled ?? false;  
  
  return {  
    ...doc,  
    totalFee: baseFee + (enabled ? miscFee : 0),  
  };  
};
```

```
useEffect(() => {  
  const fetchPublicFees = async () => {  
    setLoading(true);  
    setError(null);  
  
    try {  
      const [bizRes, docRes] = await Promise.allSettled([  
        fetch(`${API_BASE_URL}/api/fees/public/businesses`),  
        fetch(`${API_BASE_URL}/api/fees/public/documents`),  
      ]);  
  
      //  Handle businesses  
      if (bizRes.status === "fulfilled" && bizRes.value.ok) {  
        const bizData = await bizRes.value.json();  
        const rawBusinesses = Array.isArray(bizData?.data)
```

```

    ? bizData.data
    : Array.isArray(bizData)
    ? bizData
    : [];
    setBusinessTypes(rawBusinesses.map(computeBusinessTotals));
  } else {
    console.warn(" ⚠️ Failed to fetch business fees");
    setBusinessTypes([]);
  }

  // ✅ Handle documents
  if (docRes.status === "fulfilled" && docRes.value.ok) {
    const docData = await docRes.value.json();
    const rawDocuments = Array.isArray(docData?.data)
    ? docData.data
    : Array.isArray(docData)
    ? docData
    : [];
    setDocumentTypes(rawDocuments.map(computeDocumentTotals));
  } else {
    console.warn(" ⚠️ Failed to fetch document fees");
    setDocumentTypes([]);
  }
} catch (err) {
  console.error(" ❌ Error fetching public fees:", err);
  setError(err.message || "Failed to load public fees");
}

```

```
    setBusinessTypes([]);

    setDocumentTypes([]);

  } finally {

    setLoading(false);

  }

};
```

```
    fetchPublicFees();

  }, []);
```

```
// 🔍 Lookup helpers
```

```
const getBusinessFee = (type) =>

  businessTypes.find((b) => b.type === type) || null;

const getDocumentFee = (type) =>

  documentTypes.find((d) => d.type === type) || null;
```

```
return {

  businessTypes,

  documentTypes,

  getBusinessFee,

  getDocumentFee,

  loading,

  error,

};

}
```

```

## frontend\src\hooks\useReports.js

```javascript

import { usePayments } from "./usePayments";

import { useDisbursements } from "./useDisbursements";

import jsPDF from "jspdf";

export function useReports() {

const { transactions, totals } = usePayments();

const { disbursements } = useDisbursements();

function generateMonthlyReport() {

const report = {

collections: totals.collections,

pending: totals.pending,

completed: totals.completed,

outstanding: totals.outstanding,

transactions,

disbursements

};

// Create PDF

const doc = new jsPDF();

doc.setFontSize(14);

```

doc.text("Barangay Monthly Report", 20, 20);

doc.setFontSize(12);

doc.text(` Collections: ₱${report.collections || 0}` , 20, 40);
doc.text(` Pending: ₱${report.pending || 0}` , 20, 50);
doc.text(` Completed: ₱${report.completed || 0}` , 20, 60);
doc.text(` Outstanding: ₱${report.outstanding || 0}` , 20, 70);


doc.text("Disbursements:", 20, 90);
report.disbursements.forEach((d, i) => {
  doc.text(
    `${i + 1}. ${d.category} - ₱${d.amount} (${d.recipient})` ,
    25,
    100 + i * 10
  );
});

// Save PDF
doc.save("Monthly_Report.pdf");

return report;
}

return { generateMonthlyReport };
}

```

```

## frontend\src\hooks\useResidents.js

```javascript

import { useState, useCallback, useEffect } from "react";

import { ResidentsAPI } from "../services/api";

import { toast } from "react-toastify";

export const useResidents = () => {

const [residents, setResidents] = useState([]);

const [loading, setLoading] = useState(true);

const fetchResidents = useCallback(async () => {

setLoading(true);

try {

const data = await ResidentsAPI.list();

const items = Array.isArray(data) ? data : data?.items ?? [];

setResidents(items);

} catch (err) {

console.error(" ❌ Failed to fetch residents:", err.message || err);

toast.error(" ❌ Failed to fetch residents");

setResidents([]);

} finally {

setLoading(false);

}


```
}, []);
```

```
useEffect(() => {  
  fetchResidents();  
}, [fetchResidents]);
```

```
const addResident = async (payload) => {  
  try {  
    await ResidentsAPI.create(payload);  
    toast.success("✅ Resident added");  
    fetchResidents();  
  } catch (err) {  
    console.error("❌ Failed to add resident:", err.message || err);  
    toast.error("❌ Failed to add resident");  
  }  
};
```

```
const updateResident = async (id, updatedData) => {  
  try {  
    await ResidentsAPI.update(id, updatedData);  
    toast.success("✅ Resident updated");  
    fetchResidents();  
  } catch (err) {  
    console.error("❌ Failed to update resident:", err.message || err);  
    toast.error("❌ Failed to update resident");  
  }  
};
```

```
}  
};
```

```
const deleteResident = async (id) => {  
  if (!window.confirm("Are you sure you want to delete this resident?")) return;
```

```
  setResidents((prev) => prev.filter((r) => r.id !== id));
```

```
  try {
```

```
    await ResidentsAPI.delete(id);
```

```
    toast.success("🗑 Resident deleted");
```

```
    fetchResidents();
```

```
  } catch (err) {
```

```
    console.error("❌ Failed to delete resident:", err.message || err);
```

```
    toast.error("❌ Failed to delete resident");
```

```
    fetchResidents();
```

```
  }
```

```
};
```

```
return { residents, loading, fetchResidents, addResident, updateResident, deleteResident  
};
```

```
};
```

```
...
```

```
## frontend\src\hooks\useResolvedFees.js
```

```
` `` `javascript

import { useFees } from "../useFees";

import { resolveMiscFees } from "../utils/fees";

/**
 * Wraps useFees and resolves misc fees for documents and businesses.
 * Provides registrationTotal and annualTotal for businesses.
 */
export function useResolvedFees() {
  const {
    documentFees,
    businessFees,
    miscFees,
    loading,
    error,
    refreshData,
    updateDocumentFee,
    updateBusinessFee,
    updateMiscFee,
    deleteDocumentFee,
    deleteBusinessFee,
    deleteMiscFee,
  } = useFees();

  const resolvedDocuments = resolveMiscFees(documentFees, miscFees, true);
```

```
const resolvedBusinesses = resolveMiscFees(businessFees, miscFees, true);
```

```
/**
```

```
 * Compute totals for a given item.
```

```
 * - Document: base fee + misc (if enabled)
```

```
 * - Business: registrationTotal and annualTotal
```

```
 */
```

```
const getRegistrationTotal = (item) => {
```

```
  let total = (item.fee || 0) + (item.registrationFee || 0);
```

```
  if (item.enabled && item.miscFeeResolved) {
```

```
    total += item.miscFeeResolved;
```

```
  }
```

```
  return total;
```

```
};
```

```
const getAnnualTotal = (item) => {
```

```
  let total = (item.fee || 0) + (item.annualFee || 0);
```

```
  if (item.enabled && item.miscFeeResolved) {
```

```
    total += item.miscFeeResolved;
```

```
  }
```

```
  return total;
```

```
};
```

```
const getDocumentTotal = (item) => {
```

```
  let total = item.fee || 0;
```

```
  if (item.enabled && item.miscFeeResolved) {
```

```
    total += item.miscFeeResolved;
  }
  return total;
};

return {
  documentFees: resolvedDocuments.map(doc => ({
    ...doc,
    totalFee: getDocumentTotal(doc),
  })),
  businessFees: resolvedBusinesses.map(biz => ({
    ...biz,
    registrationTotal: getRegistrationTotal(biz),
    annualTotal: getAnnualTotal(biz),
  })),
  miscFees,
  loading,
  error,
  refreshData,
  updateDocumentFee,
  updateBusinessFee,
  updateMiscFee,
  deleteDocumentFee,
  deleteBusinessFee,
  deleteMiscFee,
  getRegistrationTotal,
```

```
    getAnnualTotal,  
    getDocumentTotal,  
};  
}
```

```
```\n
```

```
frontend\\src\\index.css
```

```
```\ncss
```

```
body{  
    margin: 0;  
    overflow-y: auto;  
    overflow-x: hidden;  
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',  
        'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',  
        sans-serif;  
    -webkit-font-smoothing: antialiased;  
    -moz-osx-font-smoothing: grayscale;  
}
```

```
html {  
    min-height: 100%;  
    overflow-y: auto;  
    overflow-x: hidden;  
}
```

```
#root {  
  min-height: 100%;  
  overflow-x: hidden;  
  overflow-y: auto;  
}
```

```
.app-wrapper {  
  min-height: 100%;  
  overflow-x: hidden;  
  overflow-y: auto;  
}
```

```
code {  
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',  
    monospace;  
}
```

```
```
```

```
frontend\src\index.js
```

```
```javascript
```

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import './index.css';
```

```

import './styles/mobile-responsive.css';

import App from './App';

import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();

```

```

` ``

```

```

## frontend\src\pages\adminDashboard.css

```

```

` `` `css

```

```

/* =====

```

```

👤 Admin Dashboard Styles

```

```

===== */

```

```

.dashboard.admin-dashboard {

```

```

  display: flex;

```



```
flex-direction: column;

gap: var(--space-lg);

padding: var(--space-lg);

background: var(--color-bg);

color: var(--color-text);

}
```

```
/* Header */
```

```
.dashboard-header {

  margin-bottom: var(--space-md);

  border-bottom: 1px solid var(--color-border);

  padding-bottom: var(--space-sm);

}
```

```
.dashboard-header h2 {

  font-size: var(--font-size-xl);

  font-weight: 700;

  color: var(--color-accent);

  margin-bottom: var(--space-xs);

}
```

```
.dashboard-subtitle {

  font-size: var(--font-size-md);

  color: var(--color-text-strong);

}
```

```
/* Section stack */
```

```
.section-stack {  
  display: flex;  
  flex-direction: column;  
  gap: var(--space-lg);  
}
```

```
/* DashboardSection wrapper */
```

```
.dashboard-section {  
  background: var(--gradient-card);  
  border-radius: var(--radius-md);  
  border: 1px solid var(--color-border);  
  box-shadow: var(--shadow-sm);  
  padding: var(--space-md);  
  transition: transform 0.25s ease, box-shadow 0.25s ease;  
}
```

```
.dashboard-section:hover {  
  transform: translateY(-4px);  
  box-shadow: var(--shadow-card-hover);  
}
```

```
.dashboard-section h3 {  
  font-size: var(--font-size-lg);  
  font-weight: 600;  
  margin-bottom: var(--space-sm);  
}
```

```
display: flex;

align-items: center;

gap: var(--space-sm);

}
```

```
/* Accent borders */
```

```
.dashboard-section.accent { border-left: 6px solid var(--color-accent); }

.dashboard-section.success { border-left: 6px solid var(--color-success); }

.dashboard-section.danger { border-left: 6px solid var(--color-danger); }
```

```
/* Layout variants */
```

```
.dashboard-section.grid-auto {

display: grid;

grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));

gap: var(--space-md);

}
```

```
.dashboard-section.grid-2 {

display: grid;

grid-template-columns: repeat(2, 1fr);

gap: var(--space-md);

}
```

```
.dashboard-section.flex-wrap {

display: flex;

flex-wrap: wrap;
```

```
gap: var(--space-md);  
}
```

```
.dashboard-section.flex-wrap > * {  
  flex: 1 1 320px;  
  min-width: 0;  
}
```

```
/* Responsive adjustments */  
@media (max-width: var(--bp-md)) {  
  .dashboard-section.grid-2 {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.dashboard-section.flex-wrap > * {  
  flex-basis: 100%;  
}
```

```
table {  
  display: block;  
  width: 100%;  
  overflow-x: auto;  
  border-collapse: collapse;  
}
```

```
table tr {
```

```
display: flex;
flex-direction: column;
margin-bottom: var(--space-md);
border: 1px solid var(--color-border);
border-radius: var(--radius-sm);
padding: var(--space-sm);
background: var(--color-bg-alt);
box-shadow: var(--shadow-sm);
transition: box-shadow 0.2s ease;
}
```

```
table tr:hover {
  box-shadow: var(--shadow-card-hover);
}
```

```
table td, table th {
  display: block;
  text-align: left;
  padding: var(--space-xs) 0;
}
```

```
.dashboard.admin-dashboard {
  padding: var(--space-md);
}
}
```

```

## frontend\src\pages\AdminDashboard.js

```javascript

```
import SummaryCards from "../components/dashboard/SummaryCards";
import RegistryOverview from "../components/dashboard/RegistryOverview";
import RoleManager from "../components/admin/RoleManager";
import AnalyticsPanel from "../components/admin/AnalyticsPanel";
import DocumentQueue from "../components/dashboard/DocumentQueue";
import ComplaintList from "../components/dashboard/ComplaintList";
import IncidentQueue from "../components/dashboard/IncidentQueue";
import RegistryAudit from "../components/dashboard/RegistryAudit";
import { useUser } from "../context/UserContext";
import { Navigate } from "react-router-dom";
import DashboardSection from "../components/layout/DashboardSection";

import "../adminDashboard.css";

const AdminDashboard = () => {
  const { isAdmin, role } = useUser();
```

```
if (!isAdmin) {  
  return <Navigate to="/unauthorized" replace />;  
}
```

```
return (  
  <section aria-label="Admin Dashboard">  
    <header className="dashboard-header">  
      <h2> 🧑 Barangay Captain Dashboard </h2>  
      <p className="dashboard-subtitle">  
        Manage registries, roles, complaints, and analytics.  
      </p>  
    </header>
```

```
<div className="section-stack">  
  <DashboardSection  
    title="Registry Management"  
    icon=" 📦 "  
    accent="accent"  
    layout="stack"  
    ariaLabel="Registry Management"  
  >  
    <SummaryCards role={role} />  
    <RegistryOverview />  
    <RegistryAudit />
```

```
</DashboardSection>
```

```
<DashboardSection
```

```
  title="Administrative Tools"
```

```
  icon=" ⚙️ "
```

```
  accent="success"
```

```
  layout="stack"
```

```
  ariaLabel="Administrative Tools"
```

```
>
```

```
  <RoleManager />
```

```
  {role === "admin" && <AnalyticsPanel role={role} />}
```

```
</DashboardSection>
```

```
<DashboardSection
```

```
  title="Document, Complaints & Incident Queue"
```

```
  icon=" 📄 "
```

```
  accent="danger"
```

```
  layout="flex-wrap"
```

```
  ariaLabel="Document, Complaints & Incidents"
```

```
>
```

```
  <DocumentQueue />
```

```
  <ComplaintList />
```

```
  <IncidentQueue />
```

```
</DashboardSection>
```

```
</div>
```



```

    </section>

    );
};

export default AdminDashboard;

...

## frontend\src\pages\AuditView.js

```javascript
import ComplianceReport from "../components/audit/ComplianceReport";
import SystemUsageStats from "../components/audit/SystemUsageStats";
import RegistrySnapshot from "../components/audit/RegistrySnapshot";
import AuditExportPanel from "../components/audit/AuditExportPanel";
import "../styles/admin.css";

const AuditView = () => {
 return (
 <section className="dashboard audit-dashboard">
 <header>
 <h2> 🕵️ DILG Audit View</h2>
 <p>Review compliance, monitor system usage, and access registry snapshots.</p>
 </header>

 <div className="audit-tools">

```

```
<section className="tool-section">
```

```
<h3>  Compliance Report</h3>
```

```
<ComplianceReport />
```

```
</section>
```

```
<section className="tool-section">
```

```
<h3>  System Usage Stats</h3>
```

```
<SystemUsageStats />
```

```
</section>
```

```
<section className="tool-section">
```

```
<h3>  Registry Snapshot</h3>
```

```
<RegistrySnapshot />
```

```
</section>
```

```
<section className="tool-section">
```

```
<h3>  Export Tools</h3>
```

```
<AuditExportPanel />
```

```
</section>
```

```
</div>
```

```
</section>
```

```
);
```

```
};
```

```
export default AuditView;
```

```

frontend\src\pages\business-dashboard.css

```css

/\* Overall dashboard container \*/

```
.business-dashboard {
 padding: 2rem;
 font-family: "Segoe UI", Arial, sans-serif;
 background-color: #f9fafb;
 min-height: 100vh;
}
```

/\* Heading \*/

```
.business-dashboard-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 1rem;
 margin-bottom: 1.5rem;
}
```

```
.business-dashboard h2 {
 margin-bottom: 0;
 font-size: 1.8rem;
 color: #333;
```

```
}
```

```
.register-business-btn {
 padding: 0.6rem 1rem;
 border: 1px solid #0078d4;
 background-color: #0078d4;
 color: #fff;
 border-radius: 4px;
 cursor: pointer;
 font-weight: 500;
}
```

```
.register-business-btn:hover {
 background-color: #005fa3;
 border-color: #005fa3;
}
```

```
/* Filters */
```

```
.filters {
 margin-bottom: 1.5rem;
}
```

```
.filters label {
 margin-right: 0.5rem;
 font-weight: 500;
}
```

```
.filters select {
 padding: 0.4rem 0.8rem;
 border: 1px solid #ccc;
 border-radius: 4px;
}
```

```
/* Tabs */
```

```
.tabs {
 display: flex;
 gap: 1rem;
 margin-bottom: 1.5rem;
}
```

```
.tabs button {
 padding: 0.6rem 1.2rem;
 border: 1px solid #ccc;
 background-color: #fff;
 cursor: pointer;
 border-radius: 4px;
 font-weight: 500;
 transition: all 0.2s ease;
}
```

```
.tabs button:hover {
 background-color: #f0f0f0;
```

```
}
```

```
.tabs button.active {
 background-color: #0078d4;
 color: #fff;
 border-color: #0078d4;
}
```

```
/* Table view */
```

```
.business-table {
 width: 100%;
 border-collapse: collapse;
 margin-top: 1rem;
}
```

```
.business-table th,
.business-table td {
 border: 1px solid #ddd;
 padding: 0.75rem;
 text-align: left;
}
```

```
.business-table th {
 background-color: #0078d4;
 color: #fff;
}
```

```
.clickable-row {
 cursor: pointer;
 transition: background-color 0.2s ease;
}
```

```
.clickable-row:hover {
 background-color: #f0f8ff;
}
```

```
```
```

```
## frontend\src\pages\BusinessDashboard.js
```

```
```javascript  
import { useEffect, useState } from 'react';
import './business-dashboard.css';
import { collection, query, where, getDocs, doc, updateDoc, deleteDoc } from
'firebase/firestore';
import { getStorage, ref, deleteObject } from "firebase/storage";
import { db } from '../services/firebase';
import { useNavigate } from 'react-router-dom';
import BusinessEvaluationModal from '../components/staff/BusinessEvaluationModal';
import { NotificationsAPI } from '../services/api';

const BusinessDashboard = () => {
```

```
const navigate = useNavigate();

const [businesses, setBusinesses] = useState([]);

const [filter, setFilter] = useState("");

const [loading, setLoading] = useState(true);

const [selectedBusiness, setSelectedBusiness] = useState(null);

const [activeTab, setActiveTab] = useState("needsEvaluation");

const fetchBusinesses = async () => {

 try {

 const snapshot = await getDocs(collection(db, 'businesses'));

 const data = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));

 setBusinesses(data);

 } catch (error) {

 console.error(' ❌ Firebase fetch error:', error);

 } finally {

 setLoading(false);

 }

};

useEffect(() => {

 fetchBusinesses();

}, []);

const handleEvaluationSubmit = async ({ status, notes }) => {

 try {
```



```

const refDoc = doc(db, "businesses", selectedBusiness.id);

await updateDoc(refDoc, {
 status,
 notes,
 evaluatedAt: new Date().toISOString(),
});

await NotificationsAPI.createBusinessStatusUpdate(
 status,
 selectedBusiness?.ownerUid || null,
 selectedBusiness?.businessName,
 selectedBusiness?.businessId,
 selectedBusiness?.id
);

setSelectedBusiness(null);
fetchBusinesses();
} catch (err) {
 console.error("❌ Failed to update business:", err);
 alert("Failed to update business.");
}
};

const approvedBusinesses = businesses.filter(b => b.status === "approved");
const otherBusinesses = businesses.filter(b => b.status !== "approved");

```

```
const applyFilter = (list) =>
 filter ? list.filter(b => b.barangay === filter) : list;
```

```
const barangayOptions = Array.from(
 new Set(businesses.map(b => b.barangay).filter(Boolean))
).sort();
```

```
const listToRender =
 activeTab === "needsEvaluation"
 ? applyFilter(otherBusinesses)
 : applyFilter(approvedBusinesses);
```

```
const storage = getStorage();
```

```
const deleteCollectionDocs = async (colName, field, value) => {
 const q = query(collection(db, colName), where(field, "==", value));
 const snapshot = await getDocs(q);
 for (const d of snapshot.docs) {
 await deleteDoc(doc(db, colName, d.id));
 }
};
```

```
const deleteStorageFiles = async (docs = {}) => {
 for (const [key, docInfo] of Object.entries(docs)) {
 if (docInfo?.path) {
 try {
```

```

const fileRef = ref(storage, docInfo.path);

await deleteObject(fileRef);

console.log(` 🗑 Deleted file: ${docInfo.path}`);
} catch (err) {

 console.error(` ⚠ Failed to delete file: ${docInfo.path}`, err);

}
} else {

 console.warn(` ⚠ Document ${key} has no path, skipping storage deletion.`);

}
}
};

```

```

const handleDeleteBusiness = async (business) => {
 if (window.confirm(` Delete ${business.businessName}?`)) {
 try {
 await deleteDoc(doc(db, "businesses", business.id));
 await deleteCollectionDocs("payments", "businessId", business.businessId);
 await deleteCollectionDocs("receipts", "businessId", business.businessId);

 if (business.documents) {
 await deleteStorageFiles(business.documents);
 }

 fetchBusinesses();
 } catch (err) {

 console.error(" ❌ Failed to delete:", err);
 }
 }
}

```

```
 alert("Failed to delete business.");
 }
}
};
```

```
return (
 <div className="business-dashboard">
 <div className="business-dashboard-header">
 <h2> 🇵🇭 Business Registry</h2>
 <button
 type="button"
 className="register-business-btn"
 onClick={() => navigate('/residentBusinesses')}
 >
 Register Business
 </button>
 </div>

 <div className="filters">
 <label htmlFor="barangay-filter">Filter by Barangay:</label>
 <select
 id="barangay-filter"
 value={filter}
 onChange={(e) => setFilter(e.target.value)}
 >
 <option value="">All Barangays</option>
```

```
{barangayOptions.map(b => (
 <option key={b} value={b}>{b}</option>
))}
</select>
</div>
```

```
<div className="tabs">
 <button
 className={activeTab === "needsEvaluation" ? "active" : ""}
 onClick={() => setActiveTab("needsEvaluation")}
 >
 🕒 Needs Evaluation
 </button>
 <button
 className={activeTab === "approved" ? "active" : ""}
 onClick={() => setActiveTab("approved")}
 >
 ✅ Approved Businesses
 </button>
</div>
```

```
{loading ? (
 <p>Loading businesses...</p>
) : listToRender.length === 0 ? (
 <p>No businesses found in this tab.</p>
) : (
```

```

<table className="business-table">

 <thead>

 <tr>

 <th>Business Owner</th>

 <th>Business Name</th>

 <th>Status</th>

 <th>Actions</th>

 </tr>

 </thead>

 <tbody>

 {listToRender.map(b => (

 <tr key={b.id} className="clickable-row">

 <td onClick={() => setSelectedBusiness(b)}>{b.ownerName}</td>

 <td onClick={() => setSelectedBusiness(b)}>{b.businessName}</td>

 <td>{b.status}</td>

 <td>

 <button

 className="delete-btn"

 onClick={(e) => {

 e.stopPropagation();

 handleDeleteBusiness(b);

 }}

 >

  Delete

 </button>

 </td>

 </tr>

))}

 </tbody>

</table>

```

```

 </tr>

)}

</tbody>

</table>

)}

```

```

{selectedBusiness && (
 <BusinessEvaluationModal
 business={selectedBusiness}
 onClose={() => setSelectedBusiness(null)}
 onSubmit={handleEvaluationSubmit}
 />
)}
</div>

);

};

```

```

export default BusinessDashboard;

```

```

` ` `

```

```

frontend\src\pages\complaints.css

```

```

` ` ` css

```

```

.complaints-page {
 padding: 2rem;
}

```

```
}
```

```
.header {
 display: flex;
 justify-content: space-between;
 align-items: center;
 margin-bottom: 1rem;
}
```

```
button {
 background-color: #007bff;
 color: #fff;
 padding: 0.5rem 1rem;
 border: none;
 cursor: pointer;
 border-radius: 4px;
 font-size: 0.9rem;
 transition: background-color 0.2s ease;
}
```

```
button:hover {
 background-color: #0056b3;
}
```

```
.complaint-table {
 width: 100%;
```



```
border-collapse: collapse;
margin-top: 1rem;
}
```

```
.complaint-table th,
.complaint-table td {
border: 1px solid #ddd;
padding: 0.75rem;
text-align: left;
}
```

```
.complaint-table th {
background-color: var(--color-table-header);
font-weight: 600;
}
```

```
.actions-cell {
display: flex;
gap: 0.5rem;
}
```

```
.evaluate-btn {
background-color: #28a745;
}
```

```
.evaluate-btn:hover {
```

```
background-color: #218838;
}
```

```
.delete-btn {
 background-color: #dc3545;
}
```

```
.delete-btn:hover {
 background-color: #c82333;
}
```

```
`` `
```

```
frontend\src\pages\Complaints.js
```

```
`` ` javascript
```

```
import { useEffect, useState, useCallback } from "react";
import { api, endpoints, ComplaintsAPI } from "../services/api";
import ComplaintForm from "../components/forms/ComplaintForm";
import ComplaintEvaluationModal from "../components/staff/ComplaintEvaluationModal";
import { useUser } from "../context/UserContext";
import "../complaints.css";
```

```
const Complaints = () => {
 const [complaints, setComplaints] = useState([]);
 const [mode, setMode] = useState("dashboard");
```

```
const [error, setError] = useState(null);
```

```
const [loading, setLoading] = useState(true);
```

```
const { getToken, role, can, isAuthenticated, loading: authLoading } = useUser();
```

```
const [selectedComplaint, setSelectedComplaint] = useState(null);
```

```
const [showModal, setShowModal] = useState(false);
```

```
const canViewAll = can("viewAllComplaints");
```

```
const canViewOwn = can("viewOwnComplaints");
```

```
const canView = canViewAll || canViewOwn;
```

```
const canFile =
```

```
 (role === "resident" && can("fileComplaints")) ||
```

```
 ((role === "staff" || role === "admin") && can("fileComplaintsForResidents"));
```

```
const canEvaluate = can("manageComplaints");
```

```
const canDelete = can("manageComplaints");
```

```
const fetchComplaints = useCallback(async () => {
```

```
 if (!canView) {
```

```
 setError("❌ You do not have permission to view complaints.");
```

```
 setLoading(false);
```

```
 return;
```

```
 }
```

```

try {
 let token = await getToken();
 if (!token) {
 console.warn(" ⚠️ No token yet, retrying in 500ms...");
 setTimeout(fetchComplaints, 500);
 return;
 }

 const endpoint = canViewAll ? endpoints.complaints.all : endpoints.complaints.mine;
 const { data } = await api.get(endpoint, {
 headers: { Authorization: `Bearer ${token}` },
 });

 setComplaints(data);
 setError(null);
} catch (err) {
 const errorMsg = err.response?.data?.detail || err.message;
 console.error(" ❌ Failed to fetch complaints:", errorMsg);
 setError("Failed to load complaints.");
 setComplaints([]);
} finally {
 setLoading(false);
}
}, [canView, canViewAll, getToken]);

```

```
useEffect(() => {
 if (!authLoading && isAuthenticated) {
 fetchComplaints();
 }
}, [authLoading, isAuthenticated, fetchComplaints]);
```

```
const openEvaluationModal = (complaint) => {
 setSelectedComplaint(complaint);
 setShowModal(true);
};
```

```
const handleEvaluationSubmit = async ({ status, notes, resolved_at }) => {
 try {
 await ComplaintsAPI.patchStatus(selectedComplaint.id, {
 status,
 resolution_notes: notes,
 resolved_at: status === "resolved" ? new Date().toISOString() : null,
 });
 setShowModal(false);
 setSelectedComplaint(null);
 fetchComplaints();
 } catch (err) {
 console.error(" ❌ Failed to update complaint:", err);
 alert("Failed to update complaint.");
 }
};
```

```

const handleDeleteComplaint = async (id) => {
 if (!window.confirm("Are you sure you want to delete this complaint?")) return;
 try {
 await ComplaintsAPI.deleteComplaint(id);
 fetchComplaints();
 } catch (err) {
 console.error("❌ Failed to delete complaint:", err);
 alert("Failed to delete complaint.");
 }
};

```

```

if (!isAuthenticated || (!canView && !canFile)) {
 return (
 <div className="complaints-page">
 <h2>👤 Complaints</h2>
 <p>❌ You do not have access to this module.</p>
 </div>
);
}

```

```

const renderDashboard = () => (
 <>
 <div className="header">
 <h2>{canViewAll ? "Complaints Dashboard" : "My Complaints"}</h2>
 {canFile && <button onClick={() => setMode("form")}>+ Report Complaint</button>}
 </div>
 </>
)

```

</div>

{loading || authLoading ? (

<p>Loading complaints...</p>

): error ? (

<p className="error">{error}</p>

): complaints.length === 0 ? (

<p>No complaints found.</p>

): (

<table className="complaint-table">

<thead>

<tr>

{canViewAll && <th>{role === "resident" ? "Logged By Officer" : "Filed By"}</th>}

<th>Category</th>

<th>Description</th>

<th>Location</th>

<th>Status</th>

<th>Timestamp</th>

<th>Resolution Notes</th>

{canEvaluate && <th>Actions</th>}

</tr>

</thead>

<tbody>

{complaints.map((complaint) => (

<tr key={complaint.id}>

{canViewAll && (

```

<td>

 {role === "resident"
 ? complaint.logged_by_officer || "—"
 : complaint.filed_by_name}

</td>

)}

<td>{complaint.category}</td>
<td>{complaint.description}</td>
<td>{complaint.location}</td>
<td>{complaint.status}</td>
<td>

 {complaint.timestamp
 ? new Date(complaint.timestamp).toLocaleString()
 : "—"}

</td>

<td>{complaint.resolution_notes || "—"}</td>

{(canEvaluate || canDelete) && (
 <td className="actions-cell">

 <button
 className="evaluate-btn"
 onClick={() => openEvaluationModal(complaint)}
 >

 Evaluate

 </button>

 {canDelete && (
 <button

```



```

 className="delete-btn"
 onClick={() => handleDeleteComplaint(complaint.id)}
 >
 Delete
 </button>
)}
 </td>
)}
</tr>
)))
</tbody>
</table>
)}
</>
);

```

```

const renderForm = () => (
 <>
 <div className="header">
 <h2>
 {role === "staff" || role === "admin"
 ? "Log / Evaluate Complaint"
 : "Report Complaint"}
 </h2>
 <button onClick={() => setMode("dashboard")}>← Back to Dashboard</button>
 </div>
 </>
);

```

```

 <ComplaintForm
 onSubmitSuccess={() => {
 fetchComplaints();
 setMode("dashboard");
 }}
 />
 </>
);

return (
 <div className="complaints-page" aria-busy={loading} aria-live="polite">
 {mode === "form" ? renderForm() : renderDashboard()}

 {showModal && selectedComplaint && (
 <ComplaintEvaluationModal
 complaint={selectedComplaint}
 onClose={() => setShowModal(false)}
 onSubmit={handleEvaluationSubmit}
 />
)}
 </div>
);
};

export default Complaints;

```

```

frontend\src\pages\documents.css

```css

```
.documents-page {
 max-width: 800px;
 margin: 2rem auto;
 padding: 1rem;
 background-color: var(--bg-color, #f9f9f9);
 border-radius: 8px;
 box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```

```
.documents-page h1,
.documents-page h2 {
 text-align: center;
 margin-bottom: 1rem;
 color: var(--text-color, #333);
}
```

```
.resident-filter {
 display: flex;
 gap: 0.5rem;
 align-items: center;
 margin-bottom: 1.5rem;
```

```
justify-content: center;
}
```

```
.resident-filter input {
padding: 0.5rem;
border: 1px solid #ccc;
border-radius: 4px;
width: 200px;
}
```

```
.resident-filter button {
padding: 0.5rem 1rem;
background-color: #0077cc;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
}
```

```
.resident-filter button:hover {
background-color: #005fa3;
}
```

```
.document-list {
list-style: none;
padding: 0;
```

```
}
```

```
.document-item {
 background-color: white;
 border: 1px solid #ddd;
 border-radius: 6px;
 padding: 1rem;
 margin-bottom: 1rem;
 transition: box-shadow 0.2s ease;
}
```

```
.document-item:hover {
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}
```

```
.document-item a {
 color: #0077cc;
 text-decoration: none;
 font-weight: bold;
}
```

```
.document-item a:hover {
 text-decoration: underline;
}
```

```
...
```

```
frontend\src\pages\DocumentsAdmin.js
```

```
` `` javascript
```

```
import { useState, useEffect } from "react";
```

```
import { useNavigate } from "react-router-dom";
```

```
import { api, DocumentsAPI } from "../services/api";
```

```
import DocumentSummaryCards from
"../components/document/DocumentSummaryCards";
```

```
import AuditTable from "../components/document/AuditTable";
```

```
import SearchFilters from "../components/document/SearchFilters";
```

```
import RequestModal from "../components/secretary/RequestModal";
```

```
import "../styles/dashboard/documents-admin.css";
```

```
import "../styles/secretary/pending-requests.css";
```

```
import "../styles/secretary/paid-requests.css";
```

```
import "../styles/secretary/issued-documents.css";
```

```
const DocumentsAdmin = () => {
```

```
 const [documents, setDocuments] = useState([]);
```

```
 const [loading, setLoading] = useState(false);
```

```
 const [status, setStatus] = useState("");
```

```
 const [refreshKey, setRefreshKey] = useState(0);
```

```
 const [selectedDoc, setSelectedDoc] = useState(null);
```

```
 const [rejectionReason, setRejectionReason] = useState("");
```

```
 const [issueRemarks, setIssueRemarks] = useState("");
```

```
 const navigate = useNavigate();
```

```
const normalizeForModal = (doc) => ({
 ...doc,
 document_id: doc.documentId || doc.document_id || doc.id,
 resident_id: doc.residentId || doc.resident_id,
 resident_name: doc.residentName || doc.resident_name || doc.residentId,
 document_type: doc.documentType || doc.document_type,
 created_at: doc.createdAt || doc.created_at,
 updated_at: doc.updatedAt || doc.updated_at,
 extraFields: doc.extraFields || {},
 attachments: doc.attachments || {},
});
```

```
const closeModal = () => {
 setSelectedDoc(null);
 setRejectionReason("");
 setIssueRemarks("");
};
```

```
const fetchDocuments = async (filters = {}) => {
 setLoading(true);
 setStatus("Fetching all documents...");
 const cleanFilters = Object.fromEntries(
 Object.entries(filters).filter(([_, v]) => v !== "" && v !== null)
);
 try {
```

```

const { data } = await api.get("/api/documents", { params: cleanFilters });

const normalized = data.map((doc) => ({
 ...doc,
 residentName: doc.residentName || doc.residentId,
}));

setDocuments(normalized);

setStatus("");

} catch (err) {

 console.error(" ❌ Error fetching documents:", err);

 setStatus(" ❌ Failed to load documents.");

} finally {

 setLoading(false);

}

};

useEffect(() => {

 const timer = setTimeout(() => {

 fetchDocuments({});

 }, 5000);

 return () => clearTimeout(timer);

}, []);

const handleDelete = async (docId) => {

 if (!window.confirm("Are you sure you want to delete this document?")) return;

 try {

 setStatus("Deleting document...");

 }

```



```

await api.delete(`/api/documents/${docId}`);

setDocuments((prev) => prev.filter((d) => d.id !== docId));

setStatus("✅ Document deleted successfully.");

setRefreshKey((prev) => prev + 1);

} catch (err) {

 const errorMsg = err.response?.data?.detail || err.message;

 console.error("❌ Error deleting document:", errorMsg);

 setStatus("❌ Failed to delete document.");

}

};

```

```

const updateStatus = async (storeId, newStatus, remarks = null) => {

 try {

 setStatus("Updating status...");

 await DocumentsAPI.patchStatus(storeId, { newStatus, remarks });

 setDocuments((prev) =>

 prev.map((doc) =>

 doc.id === storeId ? { ...doc, status: newStatus, remarks: remarks || doc.remarks } :

doc

)

);

 setSelectedDoc((prev) =>

 prev && prev.id === storeId

 ? { ...prev, status: newStatus, remarks: remarks || prev.remarks }

 : prev

);

 }

```

```
setStatus("✅ Status updated successfully.");
setRefreshKey((prev) => prev + 1);
fetchDocuments({});
closeModal();
} catch (err) {
 const errorMsg = err.response?.data?.detail || err.message;
 console.error("❌ Error updating status:", errorMsg);
 setStatus("❌ Failed to update status.");
}
};
```

```
const issueDocument = async (firestoreId) => {
 try {
 await DocumentsAPI.issue(firestoreId, {
 issued_by: "System Admin",
 remarks: issueRemarks.trim() || undefined,
 });
 setStatus("✅ Document issued successfully.");
 setRefreshKey((prev) => prev + 1);
 fetchDocuments({});
 closeModal();
 } catch (err) {
 const errorMsg = err.response?.data?.detail || err.message;
 console.error("❌ Error issuing document:", errorMsg);
 setStatus("❌ Failed to issue document.");
 }
};
```

```
}
};
```

```
const verifyPayment = async (firestoreId) => {
 try {
 const userInfo = JSON.parse(sessionStorage.getItem("userInfo") || "{}");
 await DocumentsAPI.patchStatus(firestoreId, {
 newStatus: "paid",
 remarks: `Payment verified by ${userInfo?.full_name || userInfo?.uid || "Admin"}`,
 });
 setStatus("✅ Payment verified successfully.");
 setRefreshKey((prev) => prev + 1);
 fetchDocuments({});
 closeModal();
 } catch (err) {
 const errorMsg = err.response?.data?.detail || err.message;
 console.error("❌ Error verifying payment:", errorMsg);
 setStatus("❌ Failed to verify payment.");
 }
};
```

```
const renderStatusModal = () => {
 if (!selectedDoc) return null;

 if (selectedDoc.status === "pending") {
 return (

```

```

<RequestModal
 doc={selectedDoc}
 onClose={closeModal}
 onUpdateStatus={updateStatus}
/>
);
}

```

```

if (selectedDoc.status === "payment_submitted" || selectedDoc.status === "paid" ||
selectedDoc.status === "for_payment") {

```

```

 return (

```

```

 <div className="modal-overlay" onClick={closeModal}>

```

```

 <div className="modal-content" onClick={(e) => e.stopPropagation()}>

```

```

 <header>

```

```

 <h4>{selectedDoc.resident_name || selectedDoc.resident_id}</h4>

```

```

 <p className="doc-type">{selectedDoc.document_type}</p>

```

```

 <p className="doc-id">Document ID: {selectedDoc.document_id}</p>

```

```

 {selectedDoc.amount === 0 && (

```

```

 <p className="doc-free">This is a free document (no payment required).</p>

```

```

)}

```

```

 </header>

```

```

 <section className="doc-meta">

```

```

 <p>Purpose: {selectedDoc.purpose || "—"}</p>

```

```

 <p>Remarks: {selectedDoc.remarks || "—"}</p>

```

```

 <p>

```

```

Payment Ref:{" "}
{selectedDoc.amount > 0
? selectedDoc.transactionId || selectedDoc.paymentIntentId || "—"
: selectedDoc.referenceNumber || selectedDoc.transactionId || "—"}
</p>

<p>Amount: {selectedDoc.amount === 0 ? "Free" :
`P$${selectedDoc.amount}`}</p>

<p>Status: {selectedDoc.status}</p>
</section>

<section className="doc-actions">
{selectedDoc.status === "payment_submitted" && (
<button className="btn-verify" onClick={() => verifyPayment(selectedDoc.id)}>
 Verify Payment
</button>
)}

{(selectedDoc.status === "paid" || selectedDoc.amount === 0) && (
<div className="reject-section">
<textarea
className="reject-textarea"
placeholder="Enter issuance remarks (optional)..."
value={issueRemarks}
onChange={(e) => setIssueRemarks(e.target.value)}
/>

```

```
 <button className="btn-approve" onClick={() =>
issueDocument(selectedDoc.id)}>
```

```
  Issue Document
```

```
 </button>
```

```
 </div>
```

```
}}
```

```
<div className="reject-section">
```

```
 <textarea
```

```
 className="reject-textarea"
```

```
 placeholder="Enter rejection reason..."
```

```
 value={rejectionReason}
```

```
 onChange={(e) => setRejectionReason(e.target.value)}
```

```
 />
```

```
 <button
```

```
 className="btn-reject"
```

```
 onClick={() => {
```

```
 if (!rejectionReason.trim()) {
```

```
 alert("Rejection reason is required.");
```

```
 return;
```

```
 }
```

```
 updateStatus(selectedDoc.id, "rejected", rejectionReason.trim());
```

```
 }}
```

```
 >
```

```
  Reject
```

```
 </button>
```

```

</div>

<button className="btn-close" onClick={closeModal}> ⬅ Close</button>

</section>

</div>

</div>

);
}

```

```

if (selectedDoc.status === "approved") {
 return (
 <div className="modal-overlay" onClick={closeModal}>
 <div className="modal-content" onClick={(e) => e.stopPropagation()}>
 <header>
 <h4>{selectedDoc.resident_name || selectedDoc.resident_id}</h4>
 <p className="doc-type">{selectedDoc.document_type}</p>
 <p className="doc-id">Document ID: {selectedDoc.document_id}</p>
 </header>

 <section className="doc-meta">
 <p>Purpose: {selectedDoc.purpose || "—"}</p>
 <p>Issued By: {selectedDoc.issuedBy || "—"}</p>
 <p>Issued At: {selectedDoc.issuedAt || "—"}</p>
 <p>Remarks: {selectedDoc.remarks || "—"}</p>
 <p>Status: {selectedDoc.status}</p>
 </section>
 </div>
 </div>
);
}

```

```

<section className="doc-file">
 <h5>Issued File</h5>
 {selectedDoc.fileUrl ? (
 <a
 href={selectedDoc.fileUrl}
 target="_blank"
 rel="noopener noreferrer"
 className="btn-download"
 >
  Download Document

) : (
 <p>No file uploaded yet.</p>
)}
</section>

 <button className="btn-close" onClick={closeModal}>  Close</button>
</div>
</div>
);
}

```

```

if (selectedDoc.status === "rejected") {
 return (
 <div className="modal-overlay" onClick={closeModal}>

```



```

<div className="modal-content" onClick={(e) => e.stopPropagation()}>

 <header>

 <h4>{selectedDoc.resident_name || selectedDoc.resident_id}</h4>

 <p className="doc-type">{selectedDoc.document_type}</p>

 <p className="doc-id">Document ID: {selectedDoc.document_id}</p>

 </header>

 <section className="doc-meta">

 <p>Purpose: {selectedDoc.purpose || "—"}</p>

 <p>Status: {selectedDoc.status}</p>

 <p>Rejection Reason: {selectedDoc.remarks || "No reason
provided."}</p>

 <p>Updated At: {selectedDoc.updatedAt ||
selectedDoc.updated_at || "—"}</p>

 </section>

 <button className="btn-close" onClick={closeModal}> ⬅ Close</button>

</div>

</div>

);

}

return (

 <RequestModal

 doc={selectedDoc}

 onClose={closeModal}

```

```

 onUpdateStatus={updateStatus}

 />

);

};

return (
 <div className="documents-admin-page">
 <div className="header-row">
 <h1> 📁 Document Oversight</h1>
 <button className="issue-document-btn" onClick={() =>
navigate("/secretary/documents")}
 >
 + Issue Document
 </button>
 </div>
 <DocumentSummaryCards key={refreshKey} role="admin" />
 <SearchFilters onSearch={fetchDocuments} />
 <h2>Recent Document Activity</h2>
 <AuditTable />
 <h2>All Documents</h2>
 {status && <p className="status-message">{status}</p>}
 {loading ? (
 <p>Loading documents...</p>
) : documents.length === 0 ? (
 <p>No documents found.</p>
) : (

```

```
<table className="document-table">

 <thead>

 <tr>

 <th>Document Type</th>

 <th>Resident</th>

 <th>Status</th>

 <th>Created At</th>

 <th>Actions</th>

 </tr>

 </thead>

 <tbody>

 {documents.map((doc) => (

 <tr

 key={doc.id}

 onClick={() => setSelectedDoc(normalizeForModal(doc))}

 style={{ cursor: "pointer" }}

 >

 {/* Document Type */}

 <td>{doc.documentType}</td>

 {/* Resident */}

 <td>{doc.residentName}</td>

 {/* Status */}

 <td>{doc.status || "N/A"}</td>
```

```

 { /* Created At */ }

 <td>

 { doc.createdAt

 ? new Date(doc.createdAt).toLocaleDateString()

 : "N/A" }

 </td>

```

```

 { /* Actions */ }

 <td>

 { doc.status === "approved" && doc.fileUrl && (

 <>

 <a

 href={doc.fileUrl}

 target="_blank"

 rel="noopener noreferrer"

 onClick={(e) => e.stopPropagation()}

 >

 📄 View

 {" "}

 |{" "}

 </>

)}

```

```

 <button

 onClick={(e) => {

 e.stopPropagation();

 handleDelete(doc.id);

```

```
 }}
 >
  Delete
 </button>
</td>
</tr>
)}
</tbody>
</table>
```

```
)}
```

```
 {renderStatusModal()}
</div>
);
};
```

```
export default DocumentsAdmin;
```

```
`,`,
```

```
frontend\src\pages\FeeDashboard.js
```

```
````javascript
```

```
import "../styles/fee-dashboard.css";
```

```
import AddNewFeeForm from "../components/forms/AddNewFeeForm";
```

```
import FeeTable from "../components/admin/FeeTable";
import { useResolvedFees } from "../hooks/useResolvedFees";
import {
  buildDocumentPayload,
  buildBusinessPayload,
  buildMiscPayload,
} from "../utils/payloadBuilders";
```

```
export default function FeeDashboard() {
  const {
    documentFees,
    businessFees,
    miscFees,
    loading,
    error,
    refreshData,
    updateDocumentFee,
    updateBusinessFee,
    updateMiscFee,
    deleteDocumentFee,
    deleteBusinessFee,
    deleteMiscFee,
    getRegistrationTotal,
    getAnnualTotal,
    getDocumentTotal,
```

```
} = useResolvedFees();
```

```
const documentColumns = [  
  { key: "documentType", label: "Document Type", editable: false },  
  { key: "fee", label: "Fee (₱)", editable: true },  
  { key: "enabled", label: "Enabled", editable: true, type: "checkbox" },  
  { key: "miscType", label: "Misc Type", editable: false },  
  { key: "miscFeeResolved", label: "Misc Fee (₱)", editable: false },  
  { key: "totalFee", label: "Total Fee (₱)", editable: false },  
];
```

```
const businessColumns = [  
  { key: "businessType", label: "Business Type", editable: false },  
  { key: "fee", label: "Base Fee (₱)", editable: true },  
  { key: "registrationFee", label: "Registration Fee (₱)", editable: true },  
  { key: "annualFee", label: "Annual Fee (₱)", editable: true },  
  { key: "enabled", label: "Enabled", editable: true, type: "checkbox" },  
  { key: "miscType", label: "Misc Type", editable: false },  
  { key: "miscFeeResolved", label: "Misc Fee (₱)", editable: false },  
  { key: "registrationTotal", label: "Registration Total (₱)", editable: false },  
  { key: "annualTotal", label: "Annual Total (₱)", editable: false },  
];
```

```
const miscColumns = [  
  { key: "miscType", label: "Misc Type", editable: false },  
  { key: "fee", label: "Fee (₱)", editable: true },
```

```
{ key: "enabled", label: "Enabled", editable: true, type: "checkbox" },  
];
```

```
const renderFeeTable = (title, columns, data, onUpdate, onDelete) => (  
  <FeeTable  
    title={title}  
    columns={columns}  
    data={data}  
    onUpdate={onUpdate}  
    onDelete={onDelete}  
  />  
);
```

```
return (  
  <div className="fee-dashboard">  
    <h1> ⚙️ Admin Fee Dashboard</h1>  
  
    {loading && <p className="loading">Loading fees...</p>}  
    {error && <p className="error">{error}</p>}  
  
    <AddNewFeeForm onAdded={refreshData} miscFees={miscFees} />  
  
    { /* 📄 Document Fees */}  
    {renderFeeTable(  
      " 📄 Document Fees",  
      documentColumns,
```



```

documentFees.map(doc => ({
  ...doc,
  totalFee: getDocumentTotal(doc, "document"),
})),
(id, key, value, item) => updateDocumentFee(id, buildDocumentPayload(item, key,
value)),
deleteDocumentFee
})

```

```

{/* 🏢 Business Fees */}

```

```

{renderFeeTable(
  "🏢 Business Fees",
  businessColumns,
  businessFees.map(biz => ({
    ...biz,
    registrationTotal: getRegistrationTotal(biz, "business"),
    annualTotal: getAnnualTotal(biz, "business"),
  })),
  (id, key, value, item) => updateBusinessFee(id, buildBusinessPayload(item, key, value)),
  deleteBusinessFee
)}

```

```

{/* 🆕 Miscellaneous Fees */}

```

```

{renderFeeTable(
  "🆕 Miscellaneous Fees",
  miscColumns,

```

```
        miscFees,
        (id, key, value, item) => updateMiscFee(id, buildMiscPayload(item, key, value)),
        deleteMiscFee
    )}
</div>

);
}

` ``
```

```
## frontend\src\pages\incidents.css
```

```
` `` `css
```

```
.incidents-page {
    padding: 2rem;
}
```

```
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1rem;
}
```

```
button {
    background-color: #007bff;
```

```
color: white;
padding: 0.5rem 1rem;
border: none;
cursor: pointer;
}
```

```
.incident-table {
width: 100%;
border-collapse: collapse;
}
```

```
.incident-table th,
.incident-table td {
border: 1px solid #ccc;
padding: 0.75rem;
text-align: left;
}
```

```
.incident-table th {
background-color: var(--color-table-header);
}
```

```
...
```

```
## frontend\src\pages\Incidents.js
```

```
` `` javascript
```

```
import { useEffect, useState, useCallback } from "react";  
import { api, endpoints } from "../services/api";  
import IncidentForm from "../components/forms/IncidentForm";  
import IncidentEvaluation from "../components/staff/IncidentEvaluation";  
import "../incidents.css";
```

```
const formatDate = (value) => {  
  if (!value) return "—";
```

```
  if (value instanceof Date) {  
    return Number.isNaN(value.getTime()) ? "—" : value.toLocaleString();  
  }
```

```
  if (typeof value === "string" || typeof value === "number") {  
    const parsed = new Date(value);  
    return Number.isNaN(parsed.getTime()) ? "—" : parsed.toLocaleString();  
  }
```

```
  if (typeof value === "object") {  
    const seconds = value.seconds ?? value._seconds;  
    if (typeof seconds === "number") {  
      const parsed = new Date(seconds * 1000);  
      return Number.isNaN(parsed.getTime()) ? "—" : parsed.toLocaleString();  
    }  
  }  
}
```

```
return "—";  
};
```

```
const Incidents = () => {  
  const [incidents, setIncidents] = useState([]);  
  const [mode, setMode] = useState("dashboard");  
  const [selectedIncident, setSelectedIncident] = useState(null);  
  const userInfo = JSON.parse(sessionStorage.getItem("userInfo"));  
  const role = userInfo?.role || "resident";  
  const canDelete = role === "staff" || role === "admin";  
  const canLogForResident = role === "staff" || role === "admin";
```

```
  const fetchIncidents = useCallback(async () => {  
    try {  
  
      const url = endpoints.incidents;  
      const { data } = await api.get(url);  
      setIncidents(data);  
    } catch (err) {  
      const errorMsg = err.response?.data?.detail || err.message;  
      console.error(" ❌ Failed to fetch incidents:", errorMsg);  
    }  
  }, []);
```

```
  useEffect(() => {
```

```

    fetchIncidents();
  }, [fetchIncidents]);

const handleDeleteIncident = async (incidentId, event) => {
  event.stopPropagation();
  if (!window.confirm("Are you sure you want to delete this incident?")) return;

  try {
    await api.delete(`${endpoints.incidents}/${incidentId}`);
    fetchIncidents();
  } catch (err) {
    const errorMsg = err.response?.data?.detail || err.message;
    console.error("❌ Failed to delete incident:", errorMsg);
    alert("Failed to delete incident.");
  }
};

return (
  <div className="incidents-page">
    {selectedIncident ? (
      <IncidentEvaluation
        incident={selectedIncident}
        role={role}
        onClose={() => setSelectedIncident(null)}
        onUpdate={fetchIncidents}
      />

```

```

): mode === "form" ? (
  <>
    <div className="header">
      <h2>
        {canLogForResident
          ? "Report / Log Incident"
          : "Report Incident"}
      </h2>
      <button onClick={() => setMode("dashboard")}>← Back to Dashboard</button>
    </div>
    <IncidentForm
      role={role}
      userInfo={userInfo}
      onSubmitSuccess={() => {
        fetchIncidents();
        setMode("dashboard");
      }}
    />
  </>
): (
  <>
    <div className="header">
      <h2>
        {canLogForResident ? "Incident Dashboard" : "Incidents Dashboard"}
      </h2>
      <button onClick={() => setMode("form")}>+ Report Incident</button>

```

```
</div>
```

```
<table className="incident-table">
```

```
<thead>
```

```
<tr>
```

```
<th>Type</th>
```

```
<th>Description</th>
```

```
<th>Location</th>
```

```
<th>Reported By</th>
```

```
{role === "staff" && <th>Assigned To</th>}
```

```
{role === "resident" && <th>Logged By Officer</th>}
```

```
<th>Status</th>
```

```
<th>Timestamp</th>
```

```
{canDelete && <th>Actions</th>}
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{incidents.map((incident) => (
```

```
<tr key={incident.id} onClick={() => setSelectedIncident(incident)}>
```

```
<td>{incident.type}</td>
```

```
<td>{incident.description}</td>
```

```
<td>{incident.location}</td>
```

```
<td>{incident.reported_by_name}</td>
```

```
{role === "staff" && (
```

```
<td>{incident.assigned_to_name || "—"}</td>
```



```

    })
    {role === "resident" && (
      <td>{incident.logged_by_officer || "—"}</td>
    })

    <td>{incident.status}</td>
    <td>{formatDateTime(incident.timestamp || incident.createdAt)}</td>
    {canDelete && (
      <td>
        <button
          type="button"
          onClick={(e) => handleDeleteIncident(incident.id, e)}
        >
          Delete
        </button>
      </td>
    )}
  </tr>
)}}
</tbody>
</table>
</>
)}
</div>
);
};

```

```
export default Incidents;
```

```
` `` `
```

```
## frontend\src\pages\login.css
```

```
` `` `css
```

```
/* =====
```

```
 Login Page Styles
```

```
===== */
```

```
.login-container {
```

```
  display: flex;
```

```
  justify-content: center;
```

```
  align-items: center;
```

```
  min-height: 100vh;
```

```
  background: var(--color-neutral);
```

```
  padding: var(--space-lg);
```

```
}
```

```
.login-form {
```

```
  background: var(--gradient-card);
```

```
  border-radius: var(--radius-md);
```

```
  box-shadow: var(--shadow-card);
```

```
  padding: var(--space-lg);
```

```
  width: 100%;
```

```
max-width: 400px;
color: var(--color-text);
display: flex;
flex-direction: column;
gap: var(--space-md);
}
```

```
.login-form h2 {
font-size: var(--font-size-xl);
font-weight: 700;
color: var(--color-accent);
margin-bottom: var(--space-md);
text-align: center;
}
```

```
/* =====
```

Form Groups

```
===== */
```

```
.form-group {
display: flex;
flex-direction: column;
gap: var(--space-xs);
position: relative; /* allow absolute positioning for eye icon */
}
```

```
.form-group label {
```

```
font-size: var(--font-size-sm);  
font-weight: 600;  
color: var(--color-text-strong);  
}
```

```
.form-group input {  
width: 100%;  
padding: var(--space-sm) var(--space-md);  
border-radius: var(--radius-sm);  
border: 1px solid var(--color-border);  
background-color: var(--color-input-bg);  
font-size: var(--font-size-md);  
transition: border-color 0.2s ease, box-shadow 0.2s ease;  
box-sizing: border-box;  
}
```

```
.form-group input:focus {  
outline: none;  
border-color: var(--color-accent);  
box-shadow: var(--shadow-focus);  
}
```

```
/* =====
```

 Password Toggle (stable)

```
===== */
```

```
.password-wrapper {
```

```
position: relative;

display: flex;

align-items: center;

}
```

```
.password-wrapper input {

width: 100%;

padding-right: 2.5rem; /* reserve space for eye icon */

box-sizing: border-box;

}
```

```
.toggle-password {

position: absolute;

right: 0.75rem;

top: 50%;

transform: translateY(-50%); /* lock vertical centering */

background: none;

border: none;

cursor: pointer;

color: var(--color-text-strong);

}
```

```
/* lock dimensions so hover doesn't shift */
```

```
width: 24px;

height: 24px;

padding: 0;

margin: 0;
```

```
display: flex;
align-items: center;
justify-content: center;
```

```
line-height: 1; /* prevent baseline wiggle */
transition: color 0.2s ease;
}
```

```
.toggle-password:hover {
  color: var(--color-accent);
  transform: translateY(-50%); /* keep same transform on hover */
}
```

```
.toggle-password:focus-visible {
  outline: 2px solid var(--color-accent);
  outline-offset: 2px;
}
```

```
.toggle-password svg {
  width: 20px;
  height: 20px;
  display: block; /* removes inline baseline jump */
  pointer-events: none;
}
```

```
/* =====
```

```
⚙ Options
```

```
===== */
```

```
.form-options {
```

```
  display: flex;
```

```
  justify-content: space-between;
```

```
  align-items: center;
```

```
  font-size: var(--font-size-sm);
```

```
}
```

```
.remember-me {
```

```
  display: flex;
```

```
  align-items: center;
```

```
  gap: var(--space-xs);
```

```
}
```

```
.forgot-password a {
```

```
  color: var(--color-link);
```

```
  text-decoration: none;
```

```
  transition: color 0.2s ease;
```

```
}
```

```
.forgot-password a:hover {
```

```
  color: var(--color-link-hover);
```

```
}
```

```
/* =====
```

```
🔴 Login Button
```

```
===== */
```

```
.login-button {  
  padding: var(--space-sm) var(--space-md);  
  border-radius: var(--radius-sm);  
  border: none;  
  font-weight: 600;  
  font-size: var(--font-size-md);  
  cursor: pointer;  
  background: var(--color-accent);  
  color: #fff;  
  transition: background-color 0.25s ease, transform 0.15s ease;  
}
```

```
.login-button:hover:not(:disabled) {  
  background: var(--color-accent-hover);  
  transform: translateY(-2px);  
}
```

```
.login-button:disabled {  
  opacity: 0.6;  
  cursor: not-allowed;  
}
```



```

/* =====
📱 Responsive
===== */

@media (max-width: var(--bp-sm)) {
  .login-form {
    padding: var(--space-md);
  }

  .form-options {
    flex-direction: column;
    gap: var(--space-sm);
    align-items: flex-start;
  }

  .login-button {
    width: 100%;
  }
}

...

```

```

## frontend\src\pages\Login.js

```

```

```javascript
import { useState, useEffect } from "react";
import {

```

```
 signInWithEmailAndPassword,
 setPersistence,
 browserSessionPersistence,
 browserLocalPersistence,
 } from "firebase/auth";
import { auth, db } from "../services/firebase";
import { doc, getDoc } from "firebase/firestore";
import { useNavigate, Link } from "react-router-dom";
import { toast } from "react-toastify";
import { NotificationsAPI } from "../services/api"; // 🖱️ import
import "../login.css";
```

```
const roleRedirects = {
 admin: "/admin",
 staff: "/staff",
 resident: "/resident",
 secretary: "/secretary",
 treasurer: "/treasurer",
 sk: "/youth",
 dilg: "/audit",
};
```

```
const normalizeRole = (role) => (role?.trim().toLowerCase() || "resident");
```

```
const getDisplayName = (profile = {}, fallbackEmail = "") => {
```

```
const firstLast = [profile.firstName || profile.first_name, profile.lastName ||
profile.last_name]

.filter(Boolean)

.join(" ")

.trim();

return (

profile.fullName ||
profile.full_name ||
profile.name ||
firstLast ||
fallbackEmail

);

};
```

```
const Login = () => {

const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [rememberMe, setRememberMe] = useState(false);
const [loading, setLoading] = useState(false);
const [showPassword, setShowPassword] = useState(false);
const navigate = useNavigate();

// Load remembered email
useEffect(() => {

const savedEmail = localStorage.getItem("rememberedEmail");
```

```
if (savedEmail) {
 setEmail(savedEmail);
 setRememberMe(true);
}
}, []);
```

```
const validateInputs = () => {
 if (!email.includes("@") || password.length < 6) {
 toast.error("❌ Invalid email or password format");
 return false;
 }
 return true;
};
```

```
const cacheUser = (userData, uid, role) => {
 const enriched = { ...userData, uid, role, cachedAt: Date.now() };
 sessionStorage.setItem("userInfo", JSON.stringify(enriched));
 return enriched;
};
```

```
const redirectByRole = (role) => {
 const target = roleRedirects[normalizeRole(role)] || "/unauthorized";
 navigate(target, { replace: true });
};
```

```
const handleLogin = async (e) => {
 e.preventDefault();
 if (!validateInputs()) return;

 setLoading(true);

 try {
 await setPersistence(auth, rememberMe ? browserLocalPersistence :
browserSessionPersistence);

 const { user } = await signInWithEmailAndPassword(auth, email, password);

 await user.getIdToken(true);
 const tokenResult = await user.getIdTokenResult();
 let role = normalizeRole(tokenResult.claims.role);

 let userData = null;
 const userDoc = await getDoc(doc(db, "users", user.uid));
 if (userDoc.exists()) {
 userData = userDoc.data();
 role = normalizeRole(userData.role || role);
 } else {
 const residentDoc = await getDoc(doc(db, "residents", user.uid));
 if (residentDoc.exists()) {
 userData = residentDoc.data();
 role = "resident";
 }
 }
 }
}
```

```
}
```

```
if (!userData) {
```

```
 toast.error("❌ Unauthorized account. Contact admin.");
```

```
 return;
```

```
}
```

```
cacheUser(userData, user.uid, role);
```

```
if (rememberMe) {
```

```
 localStorage.setItem("rememberedEmail", email);
```

```
} else {
```

```
 localStorage.removeItem("rememberedEmail");
```

```
}
```

```
// ✅ Trigger notification
```

```
if (role === "resident") {
```

```
 await NotificationsAPI.createResidentLogin(1);
```

```
} else {
```

```
 const officerName = getDisplayName(userData, email);
```

```
 await NotificationsAPI.createOfficerLogin(officerName, role);
```

```
}
```

```
toast.success(`✅ Welcome, ${getDisplayName(userData, email)}`);
```

```
redirectByRole(role);
```

```
} catch (error) {
```

```
console.error(" ❌ Login error:", error);

switch (error.code) {

 case "auth/wrong-password":

 case "auth/user-not-found":

 toast.error(" ❌ Invalid credentials");

 break;

 case "auth/too-many-requests":

 toast.error(" ❌ Too many attempts. Try again later.");

 break;

 case "permission-denied":

 toast.error(" ❌ Access denied. Contact admin.");

 break;

 default:

 toast.error(" ❌ Login failed. Please try again.");

}

} finally {

 setLoading(false);

}

};
```

```
return (

 <div className="login-container">

 <form

 onSubmit={handleLogin}
```

```
className="login-form"
aria-label="Login Form"
aria-busy={loading}
>
<h2>🔒 Barangay Login</h2>
```

```
{/* Email */}
<div className="form-group">
 <label htmlFor="email">Email Address</label>
 <input
 id="email"
 type="email"
 value={email}
 onChange={(e) => setEmail(e.target.value)}
 required
 autoComplete="username"
 placeholder="admin@barangay.gov.ph"
 disabled={loading}
 />
</div>
```

```
{/* Password */}
<div className="form-group password-group">
 <label htmlFor="password">Password</label>
 <div className="password-wrapper">
 <input
```



```

 id="password"
 type={showPassword ? "text" : "password"}
 value={password}
 onChange={(e) => setPassword(e.target.value)}
 required
 autoComplete="current-password"
 placeholder="●●●●●●●●"
 disabled={loading}
 />

 <button
 type="button"
 className="toggle-password"
 onClick={() => setShowPassword(!showPassword)}
 disabled={loading}
 aria-label={showPassword ? "Hide password" : "Show password"}
 >

 {showPassword ? "🙈" : "👁️"}

 </button>

</div>

</div>

```

```

{/* Options */}

<div className="form-options">

 <label className="remember-me">

 <input
 type="checkbox"

```

```

 checked={rememberMe}
 onChange={() => setRememberMe(!rememberMe)}
 disabled={loading}
 />

 Remember Me

</label>

<div className="forgot-password">
 <Link to="/reset-password">Forgot Password?</Link>
</div>
</div>

 { /* Submit */ }

 <button type="submit" className="login-button" disabled={loading}>
 {loading ? "Logging in..." : "Login"}
 </button>

</form>

</div>

);

};

export default Login;

` ``

frontend\src\pages\NotFound.js

```

```

` `` `javascript
const NotFound = () => {
 return (
 <div style={{ padding: '2rem', textAlign: 'center' }}>
 <h2>404 - Page Not Found</h2>
 <p>The page you're looking for doesn't exist.</p>
 </div>
);
};

export default NotFound;

```

`` `

## frontend\src\pages\PaymentCancelPage.js

```

` `` `javascript
import { useEffect } from "react";
import { useNavigate, useSearchParams } from "react-router-dom";

const PaymentCancelPage = () => {
 const navigate = useNavigate();
 const [params] = useSearchParams();

 useEffect(() => {
 const timer = setTimeout(() => {

```

```
if (window.opener) {

 window.close();

} else {

 const type = params.get("type");

 if (type === "business") {

 navigate("/businesses/my");

 } else {

 navigate("/ownDocuments");

 }

}

}, 3000);

return () => clearTimeout(timer);

}, [navigate, params]);

return (

 <div className="payment-cancel">

 <h2> ❌ Payment Cancelled</h2>

 <p>Your payment was cancelled or failed. No charges were made.</p>

 <p>You will be redirected or the window will close shortly...</p>

 </div>

);

};
```

```
export default PaymentCancelPage;
```

```
`, `
```

```
frontend\src\pages\PaymentSuccess.js
```

```
` `` javascript
```

```
import { useEffect } from "react";
```

```
import { Link, useNavigate, useLocation } from "react-router-dom";
```

```
import { useAuth } from "../hooks/useAuth";
```

```
const PaymentSuccess = () => {
```

```
 const { user } = useAuth();
```

```
 const navigate = useNavigate();
```

```
 const location = useLocation();
```

```
 useEffect(() => {
```

```
 if (user) {
```

```
 const timer = setTimeout(() => {
```

```
 navigate("/resident/business-dashboard");
```

```
 }, 4000);
```

```
 return () => clearTimeout(timer);
```

```
 }
```

```
 }, [user, navigate]);
```

```
 const params = new URLSearchParams(location.search);
```

```
const type = params.get("type");
```

```
return (
```

```
<div className="payment-success">
```

```
<h2>  Payment submitted</h2>
```

```
<p>
```

```
 Your {type || "application"} payment has been submitted successfully.
```

```
 We'll notify you once staff verifies.
```

```
</p>
```

```
{user ? (
```

```
 <Link to="/resident/business-dashboard" className="btn">
```

```
 Go to My Dashboard
```

```
 </Link>
```

```
): (
```

```
 <Link to="/login" className="btn">
```

```
 Login to view your dashboard
```

```
 </Link>
```

```
)}
```

```
</div>
```

```
);
```

```
};
```

```
export default PaymentSuccess;
```

```
...
```

```
frontend\src\pages\PaymentSuccessPage.js
```

```
` `` javascript
```

```
// pages/PaymentSuccessPage.js
```

```
import React, { useEffect } from "react";
```

```
import { useNavigate, useSearchParams } from "react-router-dom";
```

```
import "../styles/payment-success-page.css";
```

```
const PaymentSuccessPage = () => {
```

```
 const navigate = useNavigate();
```

```
 const [params] = useSearchParams();
```

```
 useEffect(() => {
```

```
 const timer = setTimeout(() => {
```

```
 if (window.opener) {
```

```
 window.close();
```

```
 } else {
```

```
 const type = params.get("type");
```

```
 if (type === "business") {
```

```
 navigate("/businesses/my");
```

```
 } else {
```

```
 navigate("/ownDocuments");
```

```
 }
```

```
 }
```

```
 }, 3000);
```

```
 return () => clearTimeout(timer);
 }, [navigate, params]);
```

```
 return (
 <div className="payment-success-container">
 <div className="payment-success-card">
 <div className="payment-success-icon">  </div>
 <h2 className="payment-success-title">GCash Payment successfully
received.</h2>
 <p className="payment-success-subtitle">You may now close this window.</p>
 </div>
 </div>
);
};
```

```
export default PaymentSuccessPage;
```

```
`,`,
```

```
frontend\src\pages\ResetPage.js
```

```
` `` javascript
```

```
import React, { useEffect, useState } from "react";
import { useSearchParams } from "react-router-dom";
import { toast } from "react-toastify";
```



```
import { usePasswordReset } from "../hooks/usePasswordReset"; //  use the hook
```

```
const ResetPage = () => {
 const [searchParams] = useSearchParams();
 const token = searchParams.get("token");

 const [newPassword, setNewPassword] = useState("");
 const [confirmPassword, setConfirmPassword] = useState("");

 const { verifyToken, applyReset, loading, email } = usePasswordReset();
 const [isValidToken, setIsValidToken] = useState(false);

 // Verify token on mount
 useEffect(() => {
 const checkToken = async () => {
 if (!token) return;
 const data = await verifyToken(token);
 if (data) {
 setIsValidToken(true);
 }
 };
 checkToken();
 }, [token, verifyToken]);

 const handleSubmit = async (e) => {
 e.preventDefault();
```

```
if (newPassword !== confirmPassword) {
 toast.warn(" ⚠ Passwords do not match.");
 return;
}
```

```
const success = await applyReset(token, newPassword, confirmPassword);
if (success) {
 setNewPassword("");
 setConfirmPassword("");
}
};
```

```
if (!token) {
 return <p> ❌ No reset token provided.</p>;
}
```

```
if (!isValidToken) {
 return <p> ❌ Reset link is invalid or expired.</p>;
}
```

```
return (
 <form onSubmit={handleSubmit} className="reset-form" aria-label="Set New
Password Form">
 <h2> 🔒 Set New Password</h2>
 <p>Resetting password for: {email}</p>
```

```
<label htmlFor="newPassword">New Password</label>
```

```
<input
```

```
 id="newPassword"
```

```
 type="password"
```

```
 value={newPassword}
```

```
 onChange={(e) => setNewPassword(e.target.value)}
```

```
 required
```

```
 disabled={loading}
```

```
<label htmlFor="confirmPassword">Confirm Password</label>
```

```
<input
```

```
 id="confirmPassword"
```

```
 type="password"
```

```
 value={confirmPassword}
```

```
 onChange={(e) => setConfirmPassword(e.target.value)}
```

```
 required
```

```
 disabled={loading}
```

```
<button type="submit" disabled={loading}>
```

```
 {loading ? "Resetting..." : "Reset Password"}
```

```
</button>
```

```
</form>
```

```
);
```

```
};
```

```
export default ResetPage;
```

```
````
```

```
## frontend\src\pages\ResetPassword.js
```

```
```javascript
```

```
import { useEffect, useState } from "react";
```

```
import { useSearchParams, useNavigate } from "react-router-dom";
```

```
import { toast } from "react-toastify";
```

```
import { usePasswordReset } from "../hooks/usePasswordReset";
```

```
import "../styles/reset-password.css"
```

```
const ResetPassword = () => {
```

```
 const [searchParams] = useSearchParams();
```

```
 const navigate = useNavigate();
```

```
 const token = searchParams.get("token");
```

```
 const [email, setEmail] = useState("");
```

```
 const [newPassword, setNewPassword] = useState("");
```

```
 const [confirmPassword, setConfirmPassword] = useState("");
```

```
 const [isValidToken, setIsValidToken] = useState(false);
```

```
const { requestReset, verifyToken, applyReset, loading, email: verifiedEmail } =
usePasswordReset();
```

```
//  Verify token if present
```

```
useEffect(() => {
 const checkToken = async () => {
 if (!token) return;
 const data = await verifyToken(token);
 if (data) setIsValidToken(true);
 };
 checkToken();
}, [token, verifyToken]);
```

```
const handleRequest = async (e) => {
 e.preventDefault();
 if (!email.trim()) {
 toast.warn("📧 Please enter your email.");
 return;
 }
 await requestReset(email);
 setEmail("");
};
```

```
const handleApply = async (e) => {
 e.preventDefault();
 if (newPassword !== confirmPassword) {
```

```

toast.warn(" ⚠ Passwords do not match.");

return;
}

const success = await applyReset(token, newPassword, confirmPassword);

if (success) {

 setNewPassword("");

 setConfirmPassword("");

 toast.success(" ✅ Password reset successful. Redirecting to login...");

 setTimeout(() => navigate("/login"), 2000); // redirect after 2s

}

};

```

```

// ✅ Apply mode

```

```

if (token) {

 if (!isValidToken) {

 return <p> ❌ Reset link is invalid or expired.</p>;

 }

}

```

```

return (

 <form onSubmit={handleApply} className="reset-form" aria-label="Set New Password Form">

```

```

 <h2> 🔒 Set New Password</h2>

```

```

 <p>Resetting password for: {verifiedEmail}</p>

```

```

 <label htmlFor="newPassword">New Password</label>

```

```

 <input

```

```

 id="newPassword"

```

```
 type="password"
 value={newPassword}
 onChange={(e) => setNewPassword(e.target.value)}
 required
 disabled={loading}
 />
```

```
<label htmlFor="confirmPassword">Confirm Password</label>
<input
 id="confirmPassword"
 type="password"
 value={confirmPassword}
 onChange={(e) => setConfirmPassword(e.target.value)}
 required
 disabled={loading}
/>
```

```
<button type="submit" disabled={loading}>
 {loading ? "Resetting..." : "Reset Password"}
</button>
</form>
```

```
);
```

```
}
```

```
//  Request mode
```

```
return (
```

```
<form onSubmit={handleRequest} className="reset-form" aria-label="Request Reset Form">
```

```
<h2>🔒 Reset Password</h2>
```

```
<label htmlFor="email">Email</label>
```

```
<input
```

```
 id="email"
```

```
 type="email"
```

```
 value={email}
```

```
 onChange={(e) => setEmail(e.target.value)}
```

```
 required
```

```
 autoComplete="username"
```

```
 disabled={loading}
```

```
<button type="submit" disabled={loading}>
```

```
 {loading ? "Sending..." : "Send Reset Email"}
```

```
</button>
```

```
</form>
```

```
);
```

```
};
```

```
export default ResetPassword;
```

```
`, `
```

```
frontend\src\pages\resident.css
```



```
` `` `css
```

```
.resident-resume {
 max-width: 100%;
 margin: 0;
 padding: 2rem;
 background: var(--color-bg);
 border-radius: 12px;
 box-shadow: 0 4px 20px rgba(0,0,0,0.08);
 font-family: system-ui, sans-serif;
}
```

```
.resume-header {
 display: flex;
 align-items: center;
 gap: 1.5rem;
 margin-bottom: 2rem;
}
```

```
.resume-photo {
 width: 140px;
 height: 140px;
 object-fit: cover;
 border-radius: 8px;
 border: 2px solid #ddd;
}
```

```
.resume-occupation {
 color: var(--color-text);
 margin-top: -8px;
}
```

```
section {
 margin-bottom: 2rem;
}
```

```
h2 {
 margin-bottom: 0.8rem;
 border-bottom: 2px solid #eee;
 padding-bottom: 4px;
}
```

```
.resume-grid {
 display: grid;
 grid-template-columns: repeat(2, 1fr);
 gap: 0.6rem 1rem;
}
```

```
.biometric-row {
 display: flex;
 gap: 2rem;
}
```

```
.fingerprint-img {
 width: 120px;
 height: 120px;
 object-fit: contain;
 border: 1px solid #ccc;
 padding: 4px;
 border-radius: 6px;
}
```

```
.signature-img {
 width: 200px;
 height: auto;
 border: 1px solid #ccc;
 padding: 4px;
 border-radius: 6px;
}
```

```
.remarks-box {
 background: #fafafa;
 padding: 1rem;
 border-radius: 6px;
 border: 1px solid #eee;
}
```

```
.resident-dashboard-page {
 max-width: 1300px;
```

```
margin: 0 auto;
padding: 1.5rem;
padding-bottom: 2.5rem;
}
```

```
.resident-dashboard-actions {
display: flex;
justify-content: flex-end;
margin-bottom: 1rem;
}
```

```
.profile-toggle-btn {
border: 1px solid var(--color-accent, #0078d4);
background: var(--color-accent, #0078d4);
color: #fff;
border-radius: 6px;
padding: 0.5rem 0.9rem;
cursor: pointer;
font-weight: 600;
}
```

```
.resident-records {
margin-top: 1rem;
display: grid;
grid-template-columns: repeat(2, minmax(0, 1fr));
gap: 3rem;
```

```
align-items: start;
}
```

```
.resident-records section {
 margin-bottom: 0;
 background: var(--color-bg);
 border: 1px solid var(--color-border, #e5e7eb);
 border-radius: 10px;
 padding: 1rem;
}
```

```
.resident-records section h2 {
 border-bottom-width: 2px;
}
```

```
.resident-records section:nth-child(1) {
 border-top: 4px solid var(--color-success, #16a34a);
}
```

```
.resident-records section:nth-child(1) h2 {
 color: var(--color-success, #16a34a);
}
```

```
.resident-records section:nth-child(2) {
 border-top: 4px solid var(--color-info, #0ea5e9);
}
```

```
.resident-records section:nth-child(2) h2 {
 color: var(--color-info, #0ea5e9);
}
```

```
.resident-records section:nth-child(3) {
 border-top: 4px solid var(--color-warning, #f59e0b);
}
```

```
.resident-records section:nth-child(3) h2 {
 color: var(--color-warning, #f59e0b);
}
```

```
.resident-records section:nth-child(4) {
 border-top: 4px solid var(--color-danger, #ef4444);
}
```

```
.resident-records section:nth-child(4) h2 {
 color: var(--color-danger, #ef4444);
}
```

```
.resident-table-wrap {
 overflow-x: auto;
}
```

```
.resident-table {
```

```
width: 100%;
border-collapse: collapse;
}
```

```
.resident-table th,
.resident-table td {
 border: 1px solid #e5e7eb;
 padding: 0.6rem;
 text-align: left;
 vertical-align: top;
}
```

```
.resident-table th {
 background: #f8fafc;
}
```

```
@media (max-width: 1024px) {
 .resident-records {
 grid-template-columns: 1fr;
 }
}
```

```
.resident-resume {
 padding: 1.2rem;
}
}
```

```

frontend\src\pages\ResidentDashboard.js

```javascript

import { useEffect, useState } from "react";

import { ResidentsAPI, ComplaintsAPI, api } from "../services/api";

import { collection, getDocs, query, where } from "firebase/firestore";

import { db } from "../services/firebase";

import "../resident.css";

const DEFAULT\_AVATAR = "/assets/default-avatar.png";

const toDate = (value) => {

if (!value) return null;

if (value?.toDate && typeof value.toDate === "function") return value.toDate();

const date = new Date(value);

return Number.isNaN(date.getTime()) ? null : date;

};

const formatDateTime = (value) => {

const date = toDate(value);

return date ? date.toLocaleString() : "—";

};

//  Age calculator



```
const calculateAge = (birthDate) => {
 if (!birthDate) return "—";
 const dob = new Date(birthDate);
 const today = new Date();
 let age = today.getFullYear() - dob.getFullYear();
 const m = today.getMonth() - dob.getMonth();
 if (m < 0 || (m === 0 && today.getDate() < dob.getDate())) age--;
 return age;
};
```

//  Normalize resident object (handles nested address + missing fields)

```
const normalizeResident = (resident) => {
```

```
 if (!resident) return null;
```

```
 const {
```

```
 address = {},
```

```
 fingerprints = {},
```

```
 signatureUrl,
```

```
 photoUrl,
```

```
 birthDate,
```

```
 ...rest
```

```
 } = resident;
```

```
 return {
```

```
 ...rest,
```

```
 birthDate,
```

```
 age: calculateAge(birthDate),
 photoUrl,
 signatureUrl,
 fingerprints,
 address: {
 houseNumber: address.house || "—",
 street: address.street || "—",
 purok: address.purok || "—",
 barangay: address.barangay || "—",
 city: address.city || "—",
 province: address.province || "—",
 zipCode: address.zipCode || "—",
 },
 };
};
```

```
const ResidentDashboard = ({ residentId }) => {
 const [resident, setResident] = useState(null);
 const [loading, setLoading] = useState(true);
 const [showProfile, setShowProfile] = useState(false);
 const [recordsLoading, setRecordsLoading] = useState(true);
 const [businesses, setBusinesses] = useState([]);
 const [documents, setDocuments] = useState([]);
 const [incidents, setIncidents] = useState([]);
 const [complaints, setComplaints] = useState([]);
```

//  Always run hooks — no conditional before this

```
useEffect(() => {
 const fetchResident = async () => {
 if (!residentId) {
 setLoading(false);
 return;
 }

 try {
 const data = await ResidentsAPI.getById(residentId);
 setResident(normalizeResident(data));
 } catch (err) {
 console.error("  Failed to load resident:", err);
 } finally {
 setLoading(false);
 }
 };
}
```

```
 fetchResident();
}, [residentId]);
```

```
useEffect(() => {
 const fetchRecords = async () => {
 if (!residentId) {
 setRecordsLoading(false);
 return;
 }
 }
}
```

```
}
```

```
try {
```

```
 setRecordsLoading(true);
```

```
 const [byUidBusinesses, byEmailBusinesses, docsRes, complaintsRes,
 incidentsByResident, incidentsByAuth] = await Promise.all([
```

```
 getDocs(query(collection(db, "businesses"), where("ownerUid", "==",
 residentId))).catch(() => ({ docs: [] })),
```

```
 resident?.email
```

```
 ? getDocs(query(collection(db, "businesses"), where("email", "==",
 resident.email))).catch(() => ({ docs: [] })),
```

```
 : Promise.resolve({ docs: [] })),
```

```
 api.get("/api/documents/my", { params: { resident_id: residentId } }).catch(() => ({ data:
 [] })),
```

```
 ComplaintsAPI.listMine().catch(() => []),
```

```
 getDocs(query(collection(db, "incidents"), where("residentId", "==",
 residentId))).catch(() => ({ docs: [] })),
```

```
 getDocs(query(collection(db, "incidents"), where("authUid", "==", residentId))).catch(()
=> ({ docs: [] })),
```

```
]);
```

```
 const businessMap = {};
```

```
 [...(byUidBusinesses.docs || []), ...(byEmailBusinesses.docs || [])].forEach((docSnap) =>
 {
```

```
 businessMap[docSnap.id] = { id: docSnap.id, ...docSnap.data() };
```

```
 });
```

```
 setBusinesses(Object.values(businessMap));
```

```

 const normalizedDocuments = (Array.isArray(docsRes.data) ? docsRes.data :
[])].map((docItem) => {
 id: docItem.id || docItem.documentId || docItem.document_id,
 documentType: docItem.documentType || docItem.document_type || "—",
 status: docItem.status || "—",
 createdAt: docItem.createdAt || docItem.created_at || null,
 });
 setDocuments(normalizedDocuments);

 const incidentMap = {};
 [...(incidentsByResident.docs || []), ...(incidentsByAuth.docs || [])].forEach((docSnap) =>
{
 incidentMap[docSnap.id] = { id: docSnap.id, ...docSnap.data() };
 });
 setIncidents(Object.values(incidentMap));

 setComplaints(Array.isArray(complaintsRes) ? complaintsRes : []);
} catch (err) {
 console.error(" ❌ Failed to load resident records:", err);
 setBusinesses([]);
 setDocuments([]);
 setIncidents([]);
 setComplaints([]);
} finally {
 setRecordsLoading(false);

```

```
}
};
```

```
fetchRecords(
 }, [residentId, resident?.email]);
```

```
//  Safe conditional rendering AFTER hooks
if (!residentId) return <p>Invalid resident ID.</p>;
if (loading) return <p>Loading resident...</p>;
if (!resident) return <p>Resident not found.</p>;
```

```
const {
 fullName,
 occupation,
 photoUrl,
 birthDate,
 age,
 gender,
 civilStatus,
 contactNumber,
 email,
 isHeadOfFamily,
 voterStatus,
 fingerprints,
 signatureUrl,
 remarks,
```

```
 address,
 } = resident;
```

```
return (
 <div className="resident-dashboard-page">
 <div className="resident-dashboard-actions">
 <button
 type="button"
 className="profile-toggle-btn"
 onClick={() => setShowProfile((prev) => !prev)}
 >
 {showProfile ? "Hide Profile" : "View Profile"}
 </button>
 </div>
 </div>
);
```

```
{showProfile && (
 <div className="resident-resume">
 <div className="resume-header">
 <img
 src={photoUrl || DEFAULT_AVATAR}
 alt={fullName || "Resident photo"}
 className="resume-photo"
 onError={(e) => {
 e.target.onerror = null;
 e.target.src = DEFAULT_AVATAR;
 }}
 />
 </div>
 </div>
);
```

```
/>

<div>

 <h1>{fullName}</h1>

 <p className="resume-occupation">{occupation || "—"}</p>

</div>

</div>
```

```
<Section title="Personal Information">

 <Grid>

 <Item label="Birth Date" value={birthDate} />

 <Item label="Age" value={age} />

 <Item label="Gender" value={gender} />

 <Item label="Civil Status" value={civilStatus} />

 <Item label="Contact" value={contactNumber} />

 <Item label="Email" value={email || "—"} />

 </Grid>

</Section>
```

```
<Section title="Address">

 <Grid>

 <Item label="House No." value={address.houseNumber} />

 <Item label="Street" value={address.street} />

 <Item label="Purok" value={address.purok} />

 <Item label="Barangay" value={address.barangay} />

 <Item label="City" value={address.city} />

 <Item label="Province" value={address.province} />
```



```

 <Item label="Zip Code" value={address.zipCode} />

 </Grid>

</Section>

<Section title="Residency Details">

 <Grid>

 <Item label="Head of Family" value={isHeadOfFamily ? "Yes" : "No"} />

 <Item label="Voter Status" value={voterStatus} />

 </Grid>

</Section>

<Section title="Biometrics">

 <div className="biometric-row">

 <BioItem label="Left Thumb" src={fingerprints.left} alt="Left thumbprint" />

 <BioItem label="Right Thumb" src={fingerprints.right} alt="Right thumbprint" />

 <BioItem label="Signature" src={signatureUrl} alt="Resident signature"
className="signature-img" />

 </div>

</Section>

<Section title="Remarks">

 <p className="remarks-box">{remarks || "No remarks"}</p>

</Section>

</div>

)}

```

```

<div className="resident-records">

 <Section title="My Businesses">

 {recordsLoading ? (

 <p>Loading records...</p>

) : businesses.length === 0 ? (

 <p>No businesses found.</p>

) : (

 <SimpleTable

 headers={["Business Name", "Type", "Status", "Submitted"]}

 rows={businesses.map((item) => ([

 item.businessName || "—",

 item.businessType || "—",

 item.status || item.paymentStatus || "—",

 formatDateTime(item.submittedAt),

]))}

 />

)}

 </Section>

```

```

<Section title="My Documents">

 {recordsLoading ? (

 <p>Loading records...</p>

) : documents.length === 0 ? (

 <p>No documents found.</p>

) : (

 <SimpleTable

```

```

 headers=["Document Type", "Status", "Requested"]}
 rows={documents.map((item) => ([
 item.documentType || "—",
 item.status || "—",
 formatDateTime(item.createdAt),
]))}
 />
})
</Section>

<Section title="My Incident Reports">
 {recordsLoading ? (
 <p>Loading records...</p>
) : incidents.length === 0 ? (
 <p>No incidents found.</p>
) : (
 <SimpleTable
 headers=["Type", "Description", "Status", "Reported"]}
 rows={incidents.map((item) => ([
 item.type || "—",
 item.description || "—",
 item.status || "—",
 item.date && item.time ? `${item.date} ${item.time}` :
formatDateTime(item.createdAt),
]))}
 />
)
}

```

```

 })
 </Section>

 <Section title="My Complaints">
 {recordsLoading ? (
 <p>Loading records...</p>
) : complaints.length === 0 ? (
 <p>No complaints found.</p>
) : (
 <SimpleTable
 headers={["Category", "Description", "Location", "Status", "Filed"]}
 rows={complaints.map((item) => ([
 item.category || "—",
 item.description || "—",
 item.location || "—",
 item.status || "—",
 formatDateTime(item.timestamp),
]))}
 />
)}
 </Section>
</div>
</div>

);
};

```

```
export default ResidentDashboard;
```

```
/* -----
```

```
  Reusable Components
```

```
----- */
```

```
const Section = ({ title, children }) => (
```

```
 <section>
```

```
 <h2>{title}</h2>
```

```
 {children}
```

```
 </section>
```

```
);
```

```
const Grid = ({ children }) => (
```

```
 <div className="resume-grid">{children}</div>
```

```
);
```

```
const Item = ({ label, value }) => (
```

```
 <div>
```

```
 {label} {value || "—"}
```

```
 </div>
```

```
);
```

```
const BioItem = ({ label, src, alt, className = "fingerprint-img" }) => (
```

```
 <div>
```

```
 <p>{label}</p>
```

```
 {src ? : "No image"}
 </div>

);
```

```
const SimpleTable = ({ headers, rows }) => (
 <div className="resident-table-wrap">
 <table className="resident-table">
 <thead>
 <tr>
 {headers.map((head) => (
 <th key={head}>{head}</th>
))}
 </tr>
 </thead>
 <tbody>
 {rows.map((row, index) => (
 <tr key={index}>
 {row.map((cell, cellIndex) => (
 <td key={` ${index}-${cellIndex}`}>{cell}</td>
))}
 </tr>
))}
 </tbody>
 </table>
 </div>
);
```

```

frontend\src\pages\SecretaryDashboard.js

```javascript

```
import { useAllDocuments } from "../hooks/useAllDocuments";
import { useDocumentCounters } from "../hooks/useDocumentCounters";
import { useFirestoreStats } from "../hooks/useFirestoreStats";
import SummaryCards from "../components/secretary/SummaryCards";
import DocumentTypeChart from "../components/secretary/DocumentTypeChart";
import PaymentStatusChart from "../components/secretary/PaymentStatusChart";
import MonthlyTrendChart from "../components/secretary/MonthlyTrendChart";
import "../styles/secretary.css"
```

```
const SecretaryDashboard = () => {
 const { docs: documents } = useAllDocuments();
 const counters = useDocumentCounters();
 const statusStats = useFirestoreStats("documents", "status", [
 "pending",
 "for_payment",
 "approved",
 "rejected",
]);

 const stats = {
```

```
total: counters.reduce((sum, c) => sum + (c.last_number || 0), 0),
...statusStats,
};
```

```
return (
 <section className="dashboard secretary-dashboard">
 <header>
 <h2>📄 Secretary Dashboard</h2>
 <p>Overview of resident requests and payment facilitation.</p>
 </header>

 <SummaryCards stats={stats} />

 <div className="charts-container">
 <section className="chart-section">
 <h3>📁 Document Types Requested</h3>
 <div style={{ width: "100%", height: "400px" }}>
 <DocumentTypeChart documents={documents} counters={counters} />
 </div>
 </section>

 <section className="chart-section">
 <h3>💰 Payment Status</h3>
 <div style={{ width: "100%", height: "400px" }}>
 <PaymentStatusChart documents={documents} />
 </div>
 </section>
 </div>
 </section>
)
```



```
</section>
```

```
<section className="chart-section">
```

```
<h3>  Monthly Request Trends</h3>
```

```
<div style={{ width: "100%", height: "450px" }}>
```

```
<MonthlyTrendChart documents={documents} />
```

```
</div>
```

```
</section>
```

```
</div>
```

```
</section>
```

```
);
```

```
};
```

```
export default SecretaryDashboard;
```

```
```
```

```
## frontend\src\pages\SKDashboard.js
```

```
```javascript
```

```
import React from "react";
```

```
import YouthRegistry from "../components/youth/YouthRegistry";
```

```
import ProgramList from "../components/youth/ProgramList";
```

```
import EventCalendar from "../components/youth/EventCalendar";
```

```
import YouthFeedbackForm from "../components/youth/YouthFeedbackForm";
```

```
import "../styles/admin.css";
```

```
const SKDashboard = () => {
 return (
 <section className="dashboard sk-dashboard">
 <header>
 <h2> 🧑 SK Dashboard</h2>
 <p>Manage youth registry, programs, events, and feedback.</p>
 </header>

 <div className="sk-tools">
 <section className="tool-section">
 <h3> 📋 Youth Registry</h3>
 <YouthRegistry />
 </section>

 <section className="tool-section">
 <h3> 🎯 Program List</h3>
 <ProgramList />
 </section>

 <section className="tool-section">
 <h3> 📅 Event Calendar</h3>
 <EventCalendar />
 </section>

 <section className="tool-section">
```

```
<h3>💬 Youth Feedback</h3>
```

```
<YouthFeedbackForm />
```

```
</section>
```

```
</div>
```

```
</section>
```

```
);
```

```
};
```

```
export default SKDashboard;
```

```
```
```

```
## frontend\src\pages\staff-dashboard.css
```

```
```css
```

```
.staff-dashboard-content {
```

```
 transition: filter 0.3s ease;
```

```
}
```

```
.staff-dashboard-content.blurred {
```

```
 filter: blur(4px);
```

```
 pointer-events: none;
```

```
 -webkit-user-select: none;
```

```
 user-select: none;
```

```
}
```

```
.form-overlay {
 position: fixed;
 top: 0;
 right: 0;
 bottom: 0;
 left: 0;
 background: rgba(0,0,0,0.4);
 display: flex;
 justify-content: center;
 align-items: center;
 z-index: 1000;
}
```

```
.form-container {
 background: var(--color-card-bg);
 padding: var(--space-lg);
 border-radius: var(--radius-md);
 box-shadow: var(--shadow-lg);
 min-width: 400px;
 max-height: 80vh;
 overflow-y: auto;
 position: relative;
}
```

```
.close-btn {
 position: absolute;
 top: var(--space-sm);
 right: var(--space-sm);
 background: transparent;
 border: none;
 font-size: var(--font-size-lg);
 cursor: pointer;
 color: var(--color-text);
}
```

```
.dashboard-header {
 /* display: flex;
 justify-content: space-between;
 align-items: center; */
 padding: var(--space-md);
}
```

```
.header-buttons {
 display: flex;
 gap: var(--space-sm);
}
```

```
.header-buttons button {
 padding: var(--space-sm) var(--space-md);
 border-radius: var(--radius-sm);
```

```
border: none;
cursor: pointer;
font-weight: var(--font-weight-bold);
}
```

```
/* Role-colored buttons using your tokens */
```

```
.btn-staff {
 background-color: var(--color-staff);
 color: #fff;
}
.btn-staff:hover {
 background-color: var(--color-accent-hover);
}
```

```
.btn-admin {
 background-color: var(--color-admin);
 color: #fff;
}
.btn-admin:hover {
 background-color: var(--color-danger-hover);
}
```

```
...
```

```
frontend\src\pages\StaffDashboard.js
```

```
` `` javascript
```

```
import React, { useState } from "react";
import SummaryCards from "../components/dashboard/SummaryCards";
import IncidentQueue from "../components/dashboard/IncidentQueue";
import ComplaintList from "../components/dashboard/ComplaintList";
import ResidentForm from "../components/forms/ResidentForm";
import { useUser } from "../context/UserContext";
import { Navigate } from "react-router-dom";
import "../staff-dashboard.css";
```

```
const StaffDashboard = ({ residents, loading }) => {
 const { role } = useUser();
 const [activeForm, setActiveForm] = useState(null);
```

```
//  Redirect if not staff
```

```
if (role !== "staff") {
 return <Navigate to="/unauthorized" replace />;
}
```

```
const openForm = (formType) => setActiveForm(formType);
const closeForm = () => setActiveForm(null);
```

```
const isOverlayActive = Boolean(activeForm);
```

```
return (
 <section className="dashboard staff-dashboard">
```

```

<header className="dashboard-header flex-between">

 <div className="header-info">

 <h2> 🧑 Staff Dashboard</h2>

 <p>Manage incidents, complaints, and resident documentation.</p>

 </div>

 <div className="header-buttons">

 <button className="btn-staff" onClick={() => openForm("resident")}>

 + Resident

 </button>

 </div>

</header>

```

```

{/* Content area blurs when overlay is active */}

```

```

<div

 className={` staff-dashboard-content ${isOverlayActive ? "blurred" : ""}`}

 aria-hidden={isOverlayActive}

>

 <SummaryCards role={role} />

 <IncidentQueue />

 <ComplaintList />

</div>

```

```

{isOverlayActive && (

 <div className="form-overlay" role="dialog" aria-modal="true">

 <div className="form-container">

 <button className="close-btn" onClick={closeForm}>✕</button>

```



```

 {activeForm === "resident" && <ResidentForm />}
 </div>
</div>

})
</section>

);
};

```

```
export default StaffDashboard;
```

```

` ` `

```

```
frontend\src\pages\TreasurerDashboard.js
```

```
` ` ` javascript
```

```

import SummaryCards from "../components/treasurer/SummaryCards";
import RevenueChart from "../components/treasurer/RevenueChart";
import TransactionsTable from "../components/treasurer/TransactionsTable";
import DisbursementTable from "../components/treasurer/DisbursementTable";
import ReportsSection from "../components/treasurer/ReportsSection";
import "../styles/treasurer.css"

```

```

function TreasurerDashboard() {
 return (
 <div className="treasurer-dashboard">
 <h1>Treasurer Dashboard</h1>

```

```

 <SummaryCards />
 <RevenueChart />
 <TransactionsTable />
 <DisbursementTable />
 <ReportsSection />
 </div>
);
}

export default TreasurerDashboard;

```

```

` ` `

```

```

frontend\src\pages\Unauthorized.js

```

```

` ` ` javascript

```

```

import { Link } from "react-router-dom";

```

```

const Unauthorized = () => (
 <section className="unauthorized-page" aria-labelledby="unauthorized-title">
 <div className="unauthorized-content">
 <h2 id="unauthorized-title" className="unauthorized-heading">  Access Denied</h2>
 <p className="unauthorized-message">You don't have permission to view this page.</p>
 <Link to="/" className="unauthorized-link">< Back to Dashboard</Link>
 </div>
 </section>
);

```

```

 </div>

</section>

);

export default Unauthorized;

...

frontend\src\reportWebVitals.js

```javascript
const reportWebVitals = onPerfEntry => {
  if (onPerfEntry && onPerfEntry instanceof Function) {
    import('web-vitals').then(({ getCLS, getFID, getFCP, getLCP, getTTFB }) => {
      getCLS(onPerfEntry);
      getFID(onPerfEntry);
      getFCP(onPerfEntry);
      getLCP(onPerfEntry);
      getTTFB(onPerfEntry);
    });
  }
};

export default reportWebVitals;

...

```

```
## frontend\src\routes\AppRoutes.js
```

```
` `` javascript
```

```
import { Routes, Route, Navigate, useLocation } from "react-router-dom";
```

```
import { useUser } from "../context/UserContext";
```

```
import ProtectedRoute from "../components/auth/ProtectedRoute";
```

```
import MainLayout from "../components/MainLayout";
```

```
import PublicLayout from "../components/PublicLayout";
```

```
import PaymentSuccessPage from "../pages/PaymentSuccessPage";
```

```
import PaymentCancelPage from "../pages/PaymentCancelPage";
```

```
import Login from "../pages/Login";
```

```
import ResetPassword from "../pages/ResetPassword";
```

```
import Unauthorized from "../pages/Unauthorized";
```

```
import NotFound from "../pages/NotFound";
```

```
import ResidentRegistry from "../components/ResidentRegistry";
```

```
import ResidentForm from "../components/forms/ResidentForm";
```

```
import BusinessDashboard from "../pages/BusinessDashboard";
```

```
import BusinessForm from "../components/forms/BusinessForm";
```

```
import StaffBusinessForm from "../components/forms/StaffBusinessForm";
```

```
import DocumentsAdmin from "../pages/DocumentsAdmin";
```

```
import ComplaintForm from "../components/forms/ComplaintForm";
```

```
import IncidentForm from "../components/forms/IncidentForm";
```

```
import Incidents from "../pages/Incidents";
```

```
import Complaints from "../pages/Complaints";
```

```
import ResidentBusinessDashboard from  
"../components/resident/ResidentBusinessDashboard";
```

```
import MyIncidents from "../components/resident/MyIncidents";
```

```
import FeeDashboard from "../pages/FeeDashboard";
```

```
import CreateAccountForm from "../components/admin/CreateAccountForm";
```

```
// Dashboard pages
```

```
import AdminDashboard from "../pages/AdminDashboard";
```

```
import StaffDashboard from "../pages/StaffDashboard";
```

```
import ResidentDashboard from "../pages/ResidentDashboard";
```

```
import SecretaryDashboard from "../pages/SecretaryDashboard";
```

```
import TreasurerDashboard from "../pages/TreasurerDashboard";
```

```
import SKDashboard from "../pages/SKDashboard";
```

```
import AuditView from "../pages/AuditView";
```

```
import ComplaintList from "../components/dashboard/ComplaintList";
```

```
// Resident sidebar components
```

```
import ResidentDocumentRequestForm from  
"../components/resident/ResidentDocumentRequestForm";
```

```
import MyDocuments from "../components/resident/MyDocuments";
```

```
import ResubmissionPage from "../components/resident/ResubmissionPage";
```

```
// Secretary sidebar components

import SecretaryDocumentForm from "../components/forms/SecretaryDocumentForm";
import PendingRequests from "../components/secretary/PendingRequests";
import PaidRequests from "../components/secretary/PaidRequests";
import IssuedDocuments from "../components/secretary/IssuedDocuments";
import RejectedRequests from "../components/secretary/RejectedRequests";


import Collections from "../components/treasurer/Collections";
import Disbursements from "../components/treasurer/Disbursements";
import Reports from "../components/treasurer/Reports";
import Settings from "../components/treasurer/Settings";


const AppRoutes = ({ isDarkMode, toggleDarkMode }) => {
  const { userInfo, role, loading } = useUser();
  const location = useLocation();
  const normalizedRole = role?.trim().toLowerCase();

  const roleRedirects = {
    admin: "/admin",
    staff: "/staff",
    resident: "/resident",
    secretary: "/secretary",
    treasurer: "/treasurer",
    sk: "/youth",
    dilg: "/audit",
```

```
};
```

```
const getRedirectPath = () => {  
  if (!normalizedRole) return "/unauthorized";  
  return roleRedirects[normalizedRole] || "/unauthorized";  
};
```

```
const isPublicRoute = ["/login", "/reset-password", "/unauthorized", "/payment-success",  
"/payment-cancel"].includes(location.pathname);
```

```
if (loading && !isPublicRoute) {  
  return <div className="loading">  Loading user data...</div>;  
}
```

```
return (  
  <div className={`App ${isDarkMode ? "dark-mode" : "light-mode"}`} >  
    <main>  
      <Routes>  
        { /* Public Routes */}  
        <Route element={<PublicLayout />} >  
          <Route path="/login" element={<Login />} />  
          <Route path="/reset-password" element={<ResetPassword />} />  
          <Route path="/payment-success" element={<PaymentSuccessPage />} />  
          <Route path="/payment-cancel" element={<PaymentCancelPage />} />  
        </Route>  
      </Routes>  
    </main>  
  </div>  
)
```

```

{ /* Protected Routes */}

<Route
  element={
    <MainLayout
      user={userInfo}
      toggleDarkMode={toggleDarkMode}
      isDarkMode={isDarkMode}
    />
  }
>

{ /* Root redirect */}

<Route
  path="/"
  element={
    loading ? (
      <div className="loading">  Restoring session...</div>
    ) : userInfo && normalizedRole ? (
      <Navigate to={getRedirectPath()} replace />
    ) : userInfo ? (
      <Navigate to="/unauthorized" replace />
    ) : (
      <Navigate to="/login" replace />
    )
  }
/>

```



```
<Route path="/unauthorized" element={<Unauthorized />} />
```

```
{/* Admin */}
```

```
<Route path="/admin" element={<ProtectedRoute  
allowedRoles={["admin"]} ><AdminDashboard /></ProtectedRoute>} />
```

```
<Route path="/accounts/new" element={<ProtectedRoute  
allowedRoles={["admin"]} ><CreateAccountForm /></ProtectedRoute>} />
```

```
<Route path="/settings" element={<ProtectedRoute  
allowedRoles={["admin"]} ><FeeDashboard /></ProtectedRoute>} />
```

```
{/* Staff */}
```

```
<Route path="/staff" element={<ProtectedRoute  
allowedRoles={["staff"]} ><StaffDashboard /></ProtectedRoute>} />
```

```
<Route path="/complaints/evaluate" element={<ProtectedRoute  
allowedRoles={["staff"]} ><Complaints /></ProtectedRoute>} />
```

```
{/* Resident */}
```

```
<Route path="/resident" element={<ProtectedRoute allowedRoles={["resident"]}  
allowAdminOverride={false} ><ResidentDashboard residentId={userInfo?.uid}  
/></ProtectedRoute>} />
```

```
<Route path="/myComplaints" element={<ProtectedRoute  
allowedRoles={["resident"]} allowAdminOverride={false} ><ComplaintList  
residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
<Route path="/businesses/new" element={<ProtectedRoute  
allowedRoles={["resident"]} allowAdminOverride={false} ><BusinessForm  
residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
<Route path="/businesses/my" element={<ProtectedRoute  
allowedRoles={["resident"]} allowAdminOverride={false} ><ResidentBusinessDashboard  
residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
<Route path="/incidents/new" element={<ProtectedRoute
allowedRoles=[["resident"]] allowAdminOverride={false}><IncidentForm
isReportMode={true} residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
<Route path="/myIncidents" element={<ProtectedRoute allowedRoles=[["resident"]]
allowAdminOverride={false}><MyIncidents residentId={userInfo?.uid}
/></ProtectedRoute>} />
```

```
<Route path="/documents/request" element={<ProtectedRoute
allowedRoles=[["resident"]]
allowAdminOverride={false}><ResidentDocumentRequestForm isRequestMode={true}
residentId={userInfo?.uid} residentName={userInfo?.fullName} /></ProtectedRoute>} />
```

```
<Route path="/ownDocuments" element={<ProtectedRoute
allowedRoles=[["resident"]] allowAdminOverride={false}><MyDocuments
residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
<Route path="/resubmit/:docId" element={<ProtectedRoute
allowedRoles=[["resident"]] allowAdminOverride={false}><ResubmissionPage
/></ProtectedRoute>} />
```

```
{/* Secretary */}
```

```
<Route path="/secretary" element={<ProtectedRoute
allowedRoles=[["secretary"]]><SecretaryDashboard /></ProtectedRoute>} />
```

```
<Route path="/secretary/documents" element={<ProtectedRoute
allowedRoles=[["secretary"]]><SecretaryDocumentForm /></ProtectedRoute>} />
```

```
<Route path="/secretary/pending" element={<ProtectedRoute
allowedRoles=[["secretary"]]><PendingRequests /></ProtectedRoute>} />
```

```
<Route path="/secretary/payments" element={<ProtectedRoute
allowedRoles=[["secretary"]]><PaidRequests /></ProtectedRoute>} />
```

```
<Route path="/secretary/issued" element={<ProtectedRoute
allowedRoles=[["secretary"]]><IssuedDocuments /></ProtectedRoute>} />
```

```
<Route path="/secretary/rejected" element={<ProtectedRoute
allowedRoles=[["secretary"]]><RejectedRequests /></ProtectedRoute>} />
```

```
{/* Treasurer */}
```

```
<Route path="/treasurer" element={<ProtectedRoute  
allowedRoles=[["admin","treasurer"]]><TreasurerDashboard /></ProtectedRoute>} />
```

```
<Route path="/treasurer/incoming" element={<ProtectedRoute  
allowedRoles=[["treasurer"]]><Collections /></ProtectedRoute>} />
```

```
<Route path="/treasurer/expenses" element={<ProtectedRoute  
allowedRoles=[["treasurer"]]><Disbursements /></ProtectedRoute>} />
```

```
<Route path="/treasurer/reports" element={<ProtectedRoute  
allowedRoles=[["treasurer"]]><Reports /></ProtectedRoute>} />
```

```
<Route path="/treasurer/settings" element={<ProtectedRoute  
allowedRoles=[["treasurer"]]><Settings /></ProtectedRoute>} />
```

```
{/* SK */}
```

```
<Route path="/youth" element={<ProtectedRoute  
allowedRoles=[["admin","sk"]]><SKDashboard /></ProtectedRoute>} />
```

```
{/* DILG Auditor */}
```

```
<Route path="/audit" element={<ProtectedRoute allowedRoles=[["dilg"]]><AuditView  
/></ProtectedRoute>} />
```

```
{/* Admin + Staff shared */}
```

```
<Route path="/residents" element={<ProtectedRoute  
allowedRoles=[["admin","staff"]]><ResidentRegistry /></ProtectedRoute>} />
```

```
<Route path="/residents/new" element={<ProtectedRoute  
allowedRoles=[["admin","staff"]]><ResidentForm user={userInfo} /></ProtectedRoute>} />
```

```
<Route path="/businesses" element={<ProtectedRoute  
allowedRoles=[["admin","staff"]]><BusinessDashboard /></ProtectedRoute>} />
```

```
<Route path="/residentBusinesses" element={<ProtectedRoute  
allowedRoles=[["admin","staff"]]><StaffBusinessForm /></ProtectedRoute>} />
```

```
    <Route path="/allComplaints" element={<ProtectedRoute
allowedRoles={["admin","staff"]} ><Complaints /></ProtectedRoute>} />
```

```
    <Route path="/incidents" element={<ProtectedRoute
allowedRoles={["admin","staff"]} ><Incidents /></ProtectedRoute>} />
```

```
    { /* Secretary + Admin shared */ }
```

```
    <Route path="/documents" element={<ProtectedRoute
allowedRoles={["admin","secretary"]} ><DocumentsAdmin /></ProtectedRoute>} />
```

```
    { /* Resident + Admin + Staff shared */ }
```

```
    <Route path="/complaints/new" element={<ProtectedRoute
allowedRoles={["resident","staff","admin"]} allowAdminOverride={false}><ComplaintForm
residentId={userInfo?.uid} /></ProtectedRoute>} />
```

```
  </Route>
```

```
    { /* Catch-all */ }
```

```
    <Route path="*" element={<NotFound />} />
```

```
  </Routes>
```

```
</main>
```

```
</div>
```

```
);
```

```
};
```

```
export default AppRoutes;
```

```
...
```

```
## frontend\src\services\api.js
```

```
` `` `javascript
```

```
import axios from "axios";
```

```
import { getAuth, onAuthStateChanged } from "firebase/auth";
```

```
const isDevelopment = process.env.NODE_ENV !== "production";
```

```
const rawEnvApiBaseUrl = process.env.REACT_APP_API_BASE_URL || "";
```

```
const normalizeDevBaseUrl = (url) => {
```

```
  if (!url) return "";
```

```
  return url.replace("http://localhost:", "http://127.0.0.1:");
```

```
};
```

```
// 🌐 API base URL strategy:
```

```
// - If env var exists, use it (normalized in dev)
```

```
// - In development without env var, use local backend
```

```
// - In production without env var, use deployed API
```

```
export const API_BASE_URL =
```

```
  (isDevelopment ? normalizeDevBaseUrl(rawEnvApiBaseUrl) : rawEnvApiBaseUrl) ||
```

```
  (isDevelopment ? "http://127.0.0.1:8000" : "https://asia-southeast1-barangay-  
1721d.cloudfunctions.net/sendEmailAsia");
```

```
console.log(" 🌐 API Base URL:", API_BASE_URL);
```

```
if (process.env.NODE_ENV === "production" && API_BASE_URL.startsWith("http://")) {
```

```
    throw new Error("❌ Production API_BASE_URL must use HTTPS");
  }
```

```
// 📦 Centralized endpoint registry
```

```
export const endpoints = {
  residents: "/api/residents",
  incidents: "/api/incidents",
  complaints: {
    base: "/api/complaints",
    mine: "/api/complaints/mine",
    all: "/api/complaints/all",
    delete: (id) => `/api/complaints/${id}`,
  },
  documents: "/api/documents",
  businesses: "/api/businesses",
  audit: "/api/document_audit",
  dashboard: "/dashboard-summary",
  disbursements: "/api/disbursements",
  accounts: {
    create: "/api/admin/create-account",
    updateRole: (uid) => `/api/admin/update-role/${uid}`,
    delete: (uid) => `/api/admin/delete-account/${uid}`,
    list: "/api/admin/accounts",
  },
  settings: {
    permissions: "/api/settings/permissions",
```

```
  },
  fees: {
    documents: "/api/fees/documents",
    businesses: "/api/fees/businesses",
    misc: "/api/fees/misc",
  },
  password: {
    request: "/api/password/request",
    verify: (token) => `/api/password/verify/${token}`,
    apply: "/api/password/apply",
  },
};
```

```
//  Axios instance
```

```
export const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { "Content-Type": "application/json" },
  timeout: 30000,
});
```

```
const waitForAuthUser = (auth, timeoutMs = 1200) =>
  new Promise((resolve) => {
    if (auth.currentUser) {
      resolve(auth.currentUser);
      return;
    }
  })
```

```
let settled = false;

const unsubscribe = onAuthStateChanged(auth, (user) => {
  if (settled) return;

  settled = true;

  unsubscribe();

  resolve(user || null);
});
```

```
setTimeout(() => {
  if (settled) return;

  settled = true;

  unsubscribe();

  resolve(auth.currentUser || null);
}, timeoutMs);
});
```

// 🛡️ Always inject a fresh token

```
api.interceptors.request.use(async (config) => {
  const auth = getAuth();

  const cachedToken = sessionStorage.getItem("authToken");

  if (config.url?.includes("/api/password/")) { return config; }

  let tokenToUse = cachedToken;

  const user = auth.currentUser || await waitForAuthUser(auth);
```



```
if (user) {  
  try {  
    const freshToken = await user.getIdToken(false);  
    tokenToUse = freshToken || tokenToUse;  
    if (freshToken) {  
      sessionStorage.setItem("authToken", freshToken);  
    }  
  } catch (err) {  
    console.warn(" ⚠️ Failed to refresh Firebase token", err);  
  }  
}
```

```
if (tokenToUse) {  
  config.headers.Authorization = `Bearer ${tokenToUse}`;  
}
```

```
return config;  
});
```

```
// ⚠️ Unified error handler
```

```
class APIError extends Error {  
  constructor(message, status, context) {  
    super(message);  
    this.name = "APIError";  
    this.status = status;
```

```
    this.context = context;
  }
}
```

```
const extractErrorDetail = (data) => {
  if (!data) return "Unknown error";
  if (Array.isArray(data)) {
    return data.map(d => `${d.loc?.join(".")}: ${d.msg}`).join("; ");
  }
  if (typeof data === "object") {
    if (Array.isArray(data.detail)) {
      return data.detail.map(d => `${d.loc?.join(".")}: ${d.msg}`).join("; ");
    }
    return data.detail || data.message || data.error || JSON.stringify(data);
  }
  if (typeof data === "string") return data;
  return "Unexpected error format";
};
```

```
const handleError = (error, context) => {
  const status = error.response?.status;
  const detail = extractErrorDetail(error.response?.data) || error.message;
  if (process.env.NODE_ENV !== "production") {
    console.error(` ❌ ${context} failed [${status ?? "no status"}]:`, detail);
  }
  throw new APIError(detail, status, context);
}
```

```
};
```

```
//  Base API class
```

```
class BaseAPI {
```

```
  constructor(endpoint) {
```

```
    this.endpoint = endpoint;
```

```
  }
```

```
  list(params = {}) {
```

```
    return api.get(this.endpoint, { params })
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, `GET ${this.endpoint}`));
```

```
  }
```

```
  getById(id) {
```

```
    return api.get(`${this.endpoint}/${id}`)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, `GET ${this.endpoint}/${id}`));
```

```
  }
```

```
  create(data) {
```

```
    return api.post(this.endpoint, data)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, `POST ${this.endpoint}`));
```

```
  }
```

```
update(id, data) {  
  return api.put(`${this.endpoint}/${id}`, data)  
    .then((res) => res.data)  
    .catch((err) => handleError(err, `PUT ${this.endpoint}/${id}`));  
}
```

```
patch(id, data) {  
  return api.patch(`${this.endpoint}/${id}`, data)  
    .then((res) => res.data)  
    .catch((err) => handleError(err, `PATCH ${this.endpoint}/${id}`));  
}
```

```
delete(id) {  
  return api.delete(`${this.endpoint}/${id}`)  
    .then((res) => res.data)  
    .catch((err) => handleError(err, `DELETE ${this.endpoint}/${id}`));  
}  
}
```

```
// 🧑 Resident APIs
```

```
export const ResidentsAPI = new BaseAPI(endpoints.residents);  
ResidentsAPI.findByNameAndBirthDate = (fullName, birthDate) =>  
  api.get(endpoints.residents, { params: { fullName, birthDate } })  
    .then((res) => res.data)  
    .catch((err) => handleError(err, "Resident duplicate check"));
```

```
// 🚨 Incident APIs
```

```
export const IncidentsAPI = new BaseAPI(endpoints.incidents);
```

```
IncidentsAPI.patchStatus = (id, payload) =>
```

```
  api.patch(`${endpoints.incidents}/${id}/status`, payload)
```

```
    .then((res) => res.data)
```

```
    .catch((err) => handleError(err, "Incident status update"));
```

```
// 🗨️ Complaint APIs
```

```
export const ComplaintsAPI = {
```

```
  listMine: () =>
```

```
    api.get(endpoints.complaints.mine)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "GET /complaints/mine")),
```

```
  listAll: () =>
```

```
    api.get(endpoints.complaints.all)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "GET /complaints/all")),
```

```
  create: (data) =>
```

```
    api.post(endpoints.complaints.base, data)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "POST /complaints")),
```

```
  patchStatus: (id, payload) =>
```

```
    api.patch(`${endpoints.complaints.base}/${id}/status`, payload)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "PATCH complaint status")),
```

```
  deleteComplaint: (id) =>
```

```
api.delete(` ${endpoints.complaints.base}/${id}` )  
  .then((res) => res.data)  
  .catch((err) => handleError(err, "DELETE complaint")),  
};
```

```
// 📄 Document APIs
```

```
export const DocumentsAPI = new BaseAPI(endpoints.documents);
```

```
// 🔄 Update document status
```

```
DocumentsAPI.patchStatus = (id, payload) =>  
  api.patch(` ${endpoints.documents}/${id}/status`, payload)  
    .then((res) => res.data)  
    .catch((err) => handleError(err, "PATCH document status"));
```

```
// 🏧 Confirm payment
```

```
DocumentsAPI.confirmPayment = (id, payload = {}) =>  
  api.patch(` ${endpoints.documents}/${id}/payment`, payload)  
    .then((res) => res.data)  
    .catch((err) => handleError(err, "Confirm payment"));
```

```
// 📄 Issue document
```

```
DocumentsAPI.issue = (id, payload = {}) => {  
  const normalizedPayload = {  
    ...payload,  
    issued_by: payload.issued_by || payload.issuedBy,  
  };  
};
```

```

delete normalizedPayload.issuedBy;

return api.patch(` ${endpoints.documents}/${id}/issue`, normalizedPayload)
  .then((res) => res.data)
  .catch((err) => handleError(err, "Issue document"));
};

// 🏠 Mark resubmitted
DocumentsAPI.markResubmitted = (id) =>
  api.patch(` ${endpoints.documents}/${id}/resubmission`, { resubmitted: true })
    .then((res) => res.data)
    .catch((err) => handleError(err, "Mark resubmitted"));

// 🏢 Business APIs
export const BusinessesAPI = new BaseAPI(endpoints.businesses);

// 🔍 List all businesses
BusinessesAPI.listAll = () =>
  api.get(endpoints.businesses)
    .then((res) => res.data)
    .catch((err) => handleError(err, "List all businesses"));

// 🔍 List businesses owned by a specific resident
BusinessesAPI.listByOwner = (ownerName) =>
  api.get(endpoints.businesses, { params: { ownerName } })
    .then(res => res.data)

```

```
.catch(err => handleError(err, "List resident businesses"));
```

```
// 📄 Audit APIs
```

```
export const AuditAPI = {
```

```
  list: () =>
```

```
    api.get(endpoints.audit)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Audit log fetch")),
```

```
  listByDoc: (docId) =>
```

```
    api.get(`${endpoints.audit}?documentId=${docId}`)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Audit log fetch by doc")),
```

```
};
```

```
// 👤 Account APIs
```

```
export const AccountsAPI = {
```

```
  create: (data) =>
```

```
    api.post(endpoints.accounts.create, data)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Account creation")),
```

```
  delete: (uid) =>
```

```
    api.delete(endpoints.accounts.delete(uid))
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Account deletion")),
```

```
  updateRole: (uid, role) =>
```

```
    api.put(endpoints.accounts.updateRole(uid), { role })
```



```
.then((res) => res.data)

.catch((err) => handleError(err, "Account role update")),

list: (params = {}) =>

  api.get("/api/admin/accounts", { params })

  .then((res) => res.data)

  .catch((err) => handleError(err, "List accounts")),

};
```

// 🇮🇹 Dashboard APIs

```
export const DashboardAPI = {

  summary: () =>

    api.get(endpoints.dashboard)

    .then((res) => res.data)

    .catch((err) => handleError(err, "Dashboard summary fetch")),

  issuedCount: (documentType) =>

    api.get("/api/documents/count/issued", { params: { documentType } })

    .then((res) => res.data)

    .catch((err) => handleError(err, "Issued count fetch")),

};
```

// ⚙️ Settings APIs

```
export const SettingsAPI = {

  getPermissions: () =>

    api.get(endpoints.settings.permissions)

    .then((res) => res.data)
```

```
    .catch((err) => handleError(err, "Permission fetch")),  
  updatePermissions: (role, permissions) =>  
    api.put(endpoints.settings.permissions, { role, permissions })  
      .then((res) => res.data)  
      .catch((err) => handleError(err, "Permission update")),  
};
```

```
// 💰 Fees APIs (patched)
```

```
export const FeesAPI = {
```

```
  // 📄 Document Fees
```

```
  listDocuments: () =>
```

```
    api.get(endpoints.fees.documents)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "List document fees")),
```

```
  updateDocument: (id, payload) =>
```

```
    api.put(`${endpoints.fees.documents}/${id}`, payload)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Update document fee")),
```

```
  deleteDocument: (id) =>
```

```
    api.delete(`${endpoints.fees.documents}/${id}`)
```

```
      .then((res) => res.data)
```

```
      .catch((err) => handleError(err, "Delete document fee")),
```

```
// 🏢 Business Fees
```

```
listBusinesses: () =>
```

```
  api.get(endpoints.fees.businesses)
    .then((res) => res.data)
    .catch((err) => handleError(err, "List business fees")),
```

```
updateBusiness: (id, payload) =>
```

```
  api.put(`${endpoints.fees.businesses}/${id}`, payload)
    .then(res => res.data)
    .catch(err => handleError(err, "Update business fee")),
```

```
deleteBusiness: (id) =>
```

```
  api.delete(`${endpoints.fees.businesses}/${id}`)
    .then((res) => res.data)
    .catch((err) => handleError(err, "Delete business fee")),
```

```
//  Miscellaneous Fees
```

```
listMisc: () =>
```

```
  api.get(endpoints.fees.misc)
    .then((res) => res.data)
    .catch((err) => handleError(err, "List miscellaneous fees")),
```

```
updateMisc: (id, payload) =>
```

```
  api.put(`${endpoints.fees.misc}/${id}`, payload)
    .then((res) => res.data)
    .catch((err) => handleError(err, "Update miscellaneous fee")),
```

```
deleteMisc: (id) =>
  api.delete(` ${endpoints.fees.misc}/${id}` )
    .then((res) => res.data)
    .catch((err) => handleError(err, "Delete miscellaneous fee")),
};
```

```
// 💰 Disbursement APIs
```

```
export const DisbursementsAPI = new BaseAPI(endpoints.disbursements);
```

```
// 🏠 Extra helpers
```

```
DisbursementsAPI.patchStatus = (id, payload) =>
  api.patch(` ${endpoints.disbursements}/${id}/status` , payload)
    .then((res) => res.data)
    .catch((err) => handleError(err, "PATCH disbursement status"));
```

```
DisbursementsAPI.listByCategory = (category) =>
  api.get(endpoints.disbursements, { params: { category } })
    .then((res) => res.data)
    .catch((err) => handleError(err, "List disbursements by category"));
```

```
DisbursementsAPI.listByRecipient = (recipientId) =>
  api.get(endpoints.disbursements, { params: { recipientId } })
    .then((res) => res.data)
    .catch((err) => handleError(err, "List disbursements by recipient"));
```

```
// 👤 Role APIs
```

```
export const RolesAPI = {  
  assignRole: (uid, role) =>  
    api.post(`/api/users/${uid}/role`, { role })  
      .then((res) => res.data)  
      .catch((err) => handleError(err, "Assign role")),  
};
```

```
export const PasswordAPI = {  
  requestReset: (email) =>  
    api.post(endpoints.password.request, { email })  
      .then((res) => res.data)  
      .catch((err) => handleError(err, "Password reset request")),
```

```
  verifyToken: (token) =>  
    api.get(endpoints.password.verify(token))  
      .then((res) => res.data)  
      .catch((err) => handleError(err, "Password token verification")),
```

```
  applyReset: (token, newPassword, confirmPassword) =>  
    api.post(endpoints.password.apply, { token, new_password: newPassword,  
      confirm_password: confirmPassword })  
      .then((res) => res.data)  
      .catch((err) => handleError(err, "Password reset apply")),  
};
```

```
// 📢 Notification APIs
```

```
export const NotificationsAPI = {  
  
  list: () =>  
  
    api.get("/api/notifications/")  
  
      .then((res) => res.data)  
  
      .catch((err) => handleError(err, "List notifications")),  
  
  markAsRead: (id) =>  
  
    api.patch(`/api/notifications/${id}/read`)  
  
      .then((res) => res.data)  
  
      .catch((err) => handleError(err, "Mark notification read")),  
  
  delete: (id) =>  
  
    api.delete(`/api/notifications/${id}`)  
  
      .then((res) => res.data)  
  
      .catch((err) => handleError(err, "Delete notification")),  
  
  bulkDelete: (onlyRead = true) =>  
  
    api.delete(`/api/notifications/actions/bulk-delete?only_read=${onlyRead}`)  
  
      .then((res) => res.data)  
  
      .catch((err) => handleError(err, "Bulk delete notifications")),  
  
  deleteAll: () =>  
  
    api.delete(`/api/notifications/actions/delete-all`)  
  
      .then((res) => res.data)  
  
      .catch((err) => handleError(err, "Delete all notifications")),  
}
```

```
markAllAsRead: () =>
  api.patch(`/api/notifications/actions/mark-all-read`)
    .then((res) => res.data)
    .catch((err) => handleError(err, "Mark all notifications read")),
```

```
createResidentLogin: (count) =>
  api.post("/api/notifications/resident-login", { count })
    .then((res) => res.data),
```

```
createResidentLogOut: (count) =>
  api.post("/api/notifications/resident-logout", { count })
    .then((res) => res.data),
```

```
createOfficerLogin: (name, role) =>
  api.post("/api/notifications/officer-login", { name, role })
    .then((res) => res.data),
```

```
createOfficerLogOut: (name, role) =>
  api.post("/api/notifications/officer-logout", { name, role })
    .then((res) => res.data),
```

```
createBusinessSubmitted: (residentName, businessName) =>
  api.post("/api/notifications/business-submitted", {
    resident_name: residentName,
    business_name: businessName,
  }).then((res) => res.data),
```

```

    createBusinessStatusUpdate: (status, residentUid, businessName, businessId,
    firestoreId) => {

        const payload = {

            status,

            business_name: businessName,

        };

        if (residentUid) payload.resident_uid = residentUid;
        if (businessId) payload.business_id = businessId;
        if (firestoreId) payload.firestore_id = firestoreId;

        return api.post("/api/notifications/business-status-update", payload)

            .then((res) => res.data);

    },

};

```

```

## frontend\src\services\email.js

```javascript

```

export async function sendEmail(payload) {

    const url = process.env.REACT_APP_MAIL_URL;

    if (!payload?.email) {

```



```
    throw new Error("❌ Missing recipient email in payload");
  }
```

```
const body = JSON.stringify({
  type: payload.type || "welcome",
  ...payload,
});
```

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 10000); // 10s timeout
```

```
try {
  console.debug("✉️ Sending email request:", { url, payload });
```

```
  const response = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body,
    signal: controller.signal,
  });
```

```
  clearTimeout(timeoutId);
```

```
  if (!response.ok) {
    const errorBody = await response.text();
    console.error("❌ Email API error:", {
```

```

    status: response.status,
    statusText: response.statusText,
    errorBody,
  });
  throw new Error(
    ` ❌ Failed to send email: ${response.status} ${response.statusText} - ${errorBody}`
  );
}

```

```

const result = await response.json();
console.debug(" ✅ Email API response:", result);
return result;
} catch (err) {
  clearTimeout(timeoutId);
  console.error(" ❌ Email request failed:", err);
  throw err;
}
}

```

...

frontend\src\services\firebase.js

```javascript

```

import { initializeApp, getApps, getApp } from "firebase/app";
import { getAuth } from "firebase/auth";

```

```
import { getFirestore } from "firebase/firestore";
import { getStorage, ref } from "firebase/storage";

const STORAGE_BUCKET =
 process.env.REACT_APP_FIREBASE_STORAGE_BUCKET ||
 "barangay-1721d.firebaseiostorage.app";

// 🗝️ Firebase configuration
const firebaseConfig = {
 apiKey: "AlzaSyDX41U2aTmvhl7Fs4QbDRzCRHuExcKFF8g",
 authDomain: "barangay-1721d.firebaseio.com",
 projectId: "barangay-1721d",
 storageBucket: STORAGE_BUCKET,
 messagingSenderId: "397499309217",
 appId: "1:397499309217:web:38d393fea54dca6d6964e0",
 measurementId: "G-3G0PT7F4N5"
};

// 🚀 Safe app initialization
export const app = getApps().length ? getApp() : initializeApp(firebaseConfig);

// 🔧 Firebase services
export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app, `gs://${STORAGE_BUCKET}`);
```

```
// 🚦 Runtime validation

try{

 ref(storage, "healthcheck.txt");

} catch (err) {

 console.error(" ❌ Firebase Storage failed to initialize:", err);

 throw new Error(" ❌ Firebase Storage bucket is undefined. Check
firebaseConfig.storageBucket and SDK version.");

}
```

```
...
```

```
frontend\src\setupTests.js
```

```
```javascript
```

```
// jest-dom adds custom jest matchers for asserting on DOM nodes.
```

```
// allows you to do things like:
```

```
// expect(element).toHaveTextContent(/react/i)
```

```
// learn more: https://github.com/testing-library/jest-dom
```

```
import '@testing-library/jest-dom';
```

```
...
```

```
## frontend\src\styles\admin.css
```

```
```css
```

```
/* =====
```

## Admin Dashboard Layout

```
===== */
```

```
.dashboard {
 padding: 2rem;
 background-color: var(--bg);
 color: var(--text);
}
```

```
.dashboard-header h2 {
 font-size: 2rem;
 margin-bottom: 0.5rem;
 color: var(--accent);
}
```

```
.dashboard-subtitle {
 font-size: 1rem;
 color: var(--text);
 opacity: 0.75;
 margin-bottom: 2rem;
}
```

```
.dashboard-section {
 display: flex;
 flex-direction: column;
 margin-bottom: 3rem;
 padding: 1.5rem;
```

```
background: linear-gradient(145deg, var(--color-card-bg), var(--color-card-muted));
border-radius: 12px;
box-shadow:
 0 6px 12px rgba(0,0,0,0.15),
 inset 0 2px 4px rgba(255,255,255,0.1);
transition: transform 0.25s ease, box-shadow 0.25s ease;
}
```

```
.dashboard-section:hover {
 transform: translateY(-4px);
 box-shadow:
 0 10px 18px rgba(0,0,0,0.25),
 inset 0 3px 6px rgba(255,255,255,0.15);
}
```

```
.dashboard-section h3 {
 font-size: 1.25rem;
 margin-bottom: 1rem;
 color: var(--color-accent);
 font-weight: 600;
}
```

```
.registry-section { border-left: 6px solid var(--accent); }
.tools-section { border-left: 6px solid var(--success); }
.complaints-section { border-left: 6px solid var(--danger); }
```

```
/* =====
```

## Registry Overview

```
===== */
```

```
.registry-overview { margin-top: 2rem; }

.registry-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
 gap: 1rem;
 margin-top: 1rem;
}
```

```
/* =====
```

## Summary & Metric Cards

```
===== */
```

```
.summary-cards,
.metrics-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
 gap: 1rem;
 margin-top: 2rem;
}

.dashboard-card {
 background: linear-gradient(145deg, var(--card-bg), var(--card-muted));
 color: var(--text);
 border-radius: 12px;
```

```
padding: 1.5rem;

box-shadow:

 0 8px 16px rgba(0,0,0,0.25),

 0 4px 6px rgba(0,0,0,0.15),

 inset 0 2px 4px rgba(255,255,255,0.1);

transition: transform 0.25s ease, box-shadow 0.25s ease;

}
```

```
.dashboard-card:hover {

 transform: translateY(-6px) scale(1.02);

 box-shadow:

 0 12px 20px rgba(0,0,0,0.35),

 0 6px 10px rgba(0,0,0,0.2),

 inset 0 3px 6px rgba(255,255,255,0.15);

}
```

```
.dashboard-card h4 {

 font-size: 1.25rem;

 margin-bottom: 0.75rem;

 color: var(--accent);

 font-weight: 600;

}
```

```
.dashboard-card p {

 font-size: 1.75rem;

 font-weight: bold;

}
```



```
color: var(--text-strong);
}
```

```
/* Icon styling */
```

```
.card-icon { margin-right: 0.5rem; }
```

```
/* =====
```

```
 Role Manager
```

```
===== */
```

```
.role-manager { margin-top: 2rem; }
```

```
.role-table {
 width: 100%;
 border-collapse: collapse;
 margin-top: 1rem;
 background-color: var(--card-bg);
 color: var(--text);
}
```

```
.role-table th {
 background-color: var(--table-header);
 color: var(--text-strong);
 padding: 0.75rem;
 text-align: left;
}
```

```
.role-table td {
 padding: 0.75rem;
 border-bottom: 1px solid var(--border);
}
```

```
.role-badge {
 display: inline-block;
 padding: 0.25rem 0.5rem;
 border-radius: 4px;
 color: #fff;
 font-size: 0.875rem;
 text-transform: capitalize;
 transition: transform 0.2s ease, box-shadow 0.2s ease;
}
```

```
.role-badge:hover {
 transform: scale(1.05);
 box-shadow: 0 0 8px rgba(0,0,0,0.3);
}
```

```
/* Role badge variants */
```

```
.role-badge.resident { background-color: #7f8c8d; }
.role-badge.staff { background-color: #2980b9; }
.role-badge.secretary { background-color: #27ae60; }
.role-badge.admin { background-color: #c0392b; }
```

```
[data-theme="dark"] .role-badge.resident { background-color: #95a5a6; }
[data-theme="dark"] .role-badge.staff { background-color: #3498db; }
[data-theme="dark"] .role-badge.secretary { background-color: #2ecc71; }
[data-theme="dark"] .role-badge.admin { background-color: #e74c3c; }
```

```
/* =====
```

 Create Account Form

```
===== */
```

```
.create-account-form {
 display: flex;
 flex-direction: column;
 gap: 1rem;
 background-color: var(--card-bg);
 padding: 1.5rem;
 border-radius: 12px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.1);
 max-width: 400px;
 width: 100%;
 color: var(--text);
}

.create-account-form h3 { color: var(--accent); }
.create-account-form label { color: var(--text); }

.create-account-form input,
.create-account-form select {
```

```
background-color: var(--input-bg);
color: var(--text);
border: 1px solid var(--border);
}
```

```
/* Confirmation block */
```

```
.confirm-actions {
background-color: var(--confirm-bg);
color: var(--confirm-text);
border: 1px solid var(--confirm-border);
}
```

```
/* =====
```

⚙ Settings Panel

```
===== */
```

```
.settings-panel {
padding: 2rem;
font-family: system-ui, sans-serif;
color: var(--text);
background-color: var(--bg);
}
```

```
.settings-panel table {
width: 100%;
border-collapse: collapse;
margin-bottom: 2rem;
```

```
background-color: var(--card-bg);
box-shadow: 0 0 4px rgba(0,0,0,0.05);
color: var(--text);
}
```

```
.settings-panel th, .settings-panel td {
 border: 1px solid var(--border);
 padding: 0.75rem;
 text-align: center;
}
```

```
@media (max-width: 768px) {
 .dashboard {
 padding: 0.75rem;
 }
}
```

```
.dashboard-section {
 padding: 0.9rem;
 margin-bottom: 1rem;
}
```

```
.dashboard-section.flex-wrap {
 flex-direction: column;
}
```

```
.dashboard-section.flex-wrap > * {
```

```
 flex: 1 1 100%;
 width: 100%;
 }
}
```

```
.settings-panel th {
 background-color: var(--table-header);
 color: var(--text-strong);
}
```

```
.settings-panel tr:nth-child(even) {
 background-color: var(--table-row-even);
}
```

```
/* Inputs */
@supports (accent-color: auto) {
 input[type="checkbox"],
 input[type="radio"] {
 accent-color: var(--accent);
 }
}
```

```
/* Error message */
.error-message {
 color: var(--danger);
}
```

```
background-color: #fdecea;
border-radius: 4px;
}
```

```
/* =====
```

 Delete Button Styling

```
===== */
```

```
.delete-btn {
 background-color: var(--danger);
 color: var(--color-danger);
 border: none;
 border-radius: 4px;
 font-weight: bold;
 cursor: pointer;
 transition: background-color 0.2s ease, transform 0.1s ease;
}
```

```
.delete-btn:hover { background-color: var(--danger-hover); transform: scale(1.05); }
```

```
.delete-btn:disabled { background-color: var(--disabled); opacity: 0.7; }
```

```
/* =====
```

 Theme Toggle Button

```
===== */
```

```
.theme-toggle {
 background-color: var(--accent);
 color: #fff;
```

```
border: none;

border-radius: 6px;

padding: 0.5rem 1rem;

cursor: pointer;

transition: background-color 0.3s ease, transform 0.2s ease;
}
```

```
.theme-toggle:hover{

background-color: var(--accent-hover);

transform: scale(1.05);
}
```

```
/* =====
```

 Card Icon Styling

```
===== */
```

```
.card-icon {

margin-right: 0.5rem;
}
```

```
```
```

```
## frontend\src\styles\app.css
```

```
```css
```

```
/* app.css */
```

```
@import "../core/variables.css";
```



```
@import "../core/base.css";
@import "../components/buttons.css";
@import "../components/cards.css";
@import "../components/tables.css";
@import "../components/forms.css";
@import "../components/badges.css";
@import "../components/sections.css";
@import "../core/utilities.css";
```

```
...
```

```
frontend\src\styles\audit.css
```

```
```css
```

```
/* =====
```

```
 Audit Dashboard Layout
```

```
===== */
```

```
.audit-dashboard {
  padding: 2rem;
}
```

```
.audit-dashboard .tool-section {
  background-color: var(--card-bg);
  padding: 1rem;
  border-radius: 6px;
  border: 1px solid var(--border);
```

```
}
```

```
.audit-dashboard .tool-section h3 {
```

```
  margin-bottom: 1rem;
```

```
  font-size: 1.2rem;
```

```
  color: var(--accent);
```

```
}
```

```
.audit-dashboard .audit-tools {
```

```
  display: grid;
```

```
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
```

```
  gap: 2rem;
```

```
  margin-top: 2rem;
```

```
}
```

```
/* =====
```

```
 Compliance & Usage Stats
```

```
===== */
```

```
.compliance-report ul,
```

```
.system-usage-stats ul {
```

```
  list-style: none;
```

```
  padding: 0;
```

```
  margin-top: 1rem;
```

```
}
```

```
.compliance-report li,
```

```
.system-usage-stats li {  
  padding: 0.5rem 0;  
  border-bottom: 1px solid var(--border);  
}
```

```
/* =====
```

 Registry Snapshot

```
===== */
```

```
.registry-snapshot table {  
  width: 100%;  
  margin-top: 1rem;  
  border-collapse: collapse;  
}
```

```
.registry-snapshot th,  
.registry-snapshot td {  
  padding: 0.75rem;  
  border: 1px solid var(--border);  
  text-align: left;  
}
```

```
/* =====
```

 Audit Export Panel

```
===== */
```

```
.audit-export-panel button {  
  background-color: var(--accent);
```

```
color: #fff;

border: none;

padding: 0.5rem 1rem;

border-radius: 4px;

cursor: pointer;

margin-top: 1rem;

}

.audit-export-panel button:hover {

  background-color: var(--accent-hover);

}
```

```
` ``
```

```
## frontend\src\styles\components\badges.css
```

```
` `` `css

/* Base Badge */

.badge {

  display: inline-block;

  padding: var(--space-xxs) var(--space-xs);

  border-radius: var(--radius-sm);

  font-size: 0.75rem;

  font-weight: 600;

  line-height: 1;

  text-transform: uppercase;
```

```
letter-spacing: 0.05em;

cursor: pointer;

transition: background-color 0.2s ease, color 0.2s ease,
            box-shadow 0.2s ease, transform 0.15s ease;

color: var(--color-on-badge, #fff);
}
```

```
/* Interactive States */
```

```
.badge:hover { transform: scale(1.05); box-shadow: var(--shadow-sm); }

.badge:focus-visible { outline: var(--focus-ring); outline-offset: 2px; }

.badge:active { transform: scale(0.95); box-shadow: var(--shadow-inset, inset 0 2px 4px
rgb(0,0,0,0.2)); }

.badge:disabled,

.badge[aria-disabled="true"] { opacity: 0.6; cursor: not-allowed; pointer-events: none; }
```

```
/* Role Variants */
```

```
.badge.resident { background-color: var(--color-resident); }

.badge.staff    { background-color: var(--color-staff); }

.badge.secretary { background-color: var(--color-secretary); }

.badge.admin     { background-color: var(--color-admin); }
```

```
/* Dark Theme Overrides */
```

```
[data-theme="dark"] .badge.resident { background-color: var(--color-resident-dark); }

[data-theme="dark"] .badge.staff    { background-color: var(--color-staff-dark); }

[data-theme="dark"] .badge.secretary { background-color: var(--color-secretary-dark); }

[data-theme="dark"] .badge.admin     { background-color: var(--color-admin-dark); }
```

```
/* Status Variants */
```

```
.badge-success { background-color: var(--color-success-bg); color: var(--color-success-text); }
```

```
.badge-danger { background-color: var(--color-danger-bg); color: var(--color-danger-text); }
```

```
.badge-warning { background-color: var(--color-warning-bg); color: var(--color-warning-text); }
```

```
.badge-info { background-color: var(--color-info-bg, var(--color-accent-bg)); color: var(--color-info-text, var(--color-accent-text)); }
```

```
.badge-muted { background-color: var(--color-card-muted); color: var(--color-text); }
```

```
.badge-review { background-color: var(--color-review); color: #fff; }
```

```
.badge-outline { background-color: transparent; border: 1px solid var(--color-border); color: var(--color-text-strong); }
```

```
/* Selection Variants */
```

```
.badge-selected {  
  background-color: var(--color-selected-bg, var(--color-accent));  
  color: var(--color-selected-text, #fff);  
  box-shadow: var(--shadow-sm);  
  transform: scale(1.05);  
}
```

```
.badge-unselected {  
  background-color: var(--color-unselected-bg, var(--color-card-muted));  
  color: var(--color-unselected-text, var(--color-text));  
  border: 1px solid var(--color-border);  
  opacity: 0.8;
```

```

}

.badge-unselected:hover {
  background-color: var(--color-border-strong);
  color: var(--color-text-strong);
  opacity: 1;
}

.badge-xs { font-size: clamp(0.65rem, 1vw + 0.5rem, 0.7rem); padding: clamp(0.1rem,
0.5vw + 0.1rem, 0.25rem) clamp(0.25rem, 1vw + 0.25rem, 0.5rem); }

.badge-sm { font-size: clamp(0.75rem, 1vw + 0.6rem, 0.8rem); padding: clamp(0.2rem,
0.75vw + 0.2rem, 0.4rem) clamp(0.4rem, 1vw + 0.3rem, 0.75rem); }

.badge-md { font-size: clamp(0.85rem, 1vw + 0.7rem, 0.9rem); padding: clamp(0.25rem,
1vw + 0.25rem, 0.5rem) clamp(0.5rem, 1.5vw + 0.5rem, 1rem); }

.badge-lg { font-size: clamp(1rem, 1.5vw + 0.8rem, 1.125rem); padding: clamp(0.4rem, 1vw
+ 0.3rem, 0.75rem) clamp(0.75rem, 2vw + 0.5rem, 1.25rem); }

.badge-xl { font-size: clamp(1.125rem, 2vw + 1rem, 1.25rem); padding: clamp(0.5rem,
1.5vw + 0.5rem, 1rem) clamp(1rem, 2.5vw + 1rem, 1.5rem); }

.badge-group { display: inline-flex; flex-wrap: wrap; gap: var(--space-xs); align-items:
center; }

.badge-group.compact { gap: var(--space-xs); }

.badge-group.vertical { flex-direction: column; align-items: flex-start; }

/* Pill Group Base */

.badge-pill-group {
  display: inline-flex;
  flex-wrap: nowrap;
  border: 1px solid var(--color-border);

```

```
border-radius: var(--radius-lg);

overflow: hidden;

background-color: var(--color-card-bg);
}

.badge-pill-group .badge { border-radius: 0; flex: 1; text-align: center; }

.badge-pill-group .badge:first-child { border-top-left-radius: var(--radius-lg); border-
bottom-left-radius: var(--radius-lg); }

.badge-pill-group .badge:last-child { border-top-right-radius: var(--radius-lg); border-
bottom-right-radius: var(--radius-lg); }

/* Justified */

.badge-pill-group.justified { width: 100%; }

/* Scrollable */

.badge-pill-group.scrollable {

  overflow-x: auto;

  gap: var(--space-xs);

  padding: var(--space-xxs);
}

/* Firefox + Chrome ≥121 */

@supports (scrollbar-width: none) {

  .badge-pill-group.scrollable {

    scrollbar-width: none;

  }

}
```



```
/* Safari/WebKit */
```

```
.badge-pill-group.scrollable::-webkit-scrollbar {  
  display: none;  
}
```

```
/* Snap Variants */
```

```
.badge-pill-group.scrollable.snap { scroll-snap-type: x mandatory; }  
.badge-pill-group.scrollable.snap .badge { scroll-snap-align: center; }  
.badge-pill-group.scrollable.snap-start .badge { scroll-snap-align: start; }  
.badge-pill-group.scrollable.snap-end .badge { scroll-snap-align: end; }  
.badge-pill-group.scrollable.snap-proximity { scroll-snap-type: x proximity; }
```

```
` `` `
```

```
## frontend\src\styles\components\buttons.css
```

```
` `` `css
```

```
/* =====
```

```
 Buttons (Base)
```

```
===== */
```

```
button,
```

```
.btn {
```

```
  font-family: inherit;
```

```
  font-size: 1rem;
```

```
  padding: var(--space-sm) var(--space-md);
```

```
border-radius: var(--radius-md);  
border: none;  
cursor: pointer;  
background-color: var(--color-accent);  
color: var(--color-text);  
transition: background-color 0.3s ease, transform 0.2s ease;  
}
```

```
button:hover,  
.btn:hover {  
  background-color: var(--color-accent-hover);  
  transform: scale(1.05);  
}
```

```
button:focus-visible,  
.btn:focus-visible {  
  outline: var(--focus-ring);  
  outline-offset: 2px;  
}
```

```
button:disabled,  
.btn:disabled {  
  background-color: var(--color-disabled);  
  cursor: not-allowed;  
  opacity: 0.7;  
  pointer-events: none;
```

```
}
```

```
/* =====
```

```
 Variants (Role / State)
```

```
===== */
```

```
.btn-success { background-color: var(--color-success); }
```

```
.btn-success:hover { background-color: var(--color-success-hover); }
```

```
.btn-danger { background-color: var(--color-danger); }
```

```
.btn-danger:hover { background-color: var(--color-danger-hover); }
```

```
.btn-muted { background-color: var(--color-card-muted); color: var(--color-text); }
```

```
.btn-muted:hover { background-color: var(--color-border-strong); }
```

```
/* Role-based buttons (ties into your role tokens) */
```

```
.btn-staff { background-color: var(--color-staff); color: #fff; }
```

```
.btn-admin { background-color: var(--color-admin); color: #fff; }
```

```
.btn-resident { background-color: var(--color-resident); color: #fff; }
```

```
.btn-secretary { background-color: var(--color-secretary); color: #fff; }
```

```
.btn-youth { background-color: var(--color-youth); color: #fff; }
```

```
/* =====
```

```
 Fluid Button Sizes
```

```
===== */
```

```
.btn-xs { font-size: var(--font-size-sm); padding: var(--space-xs) var(--space-sm); border-radius: var(--radius-sm); }
```

```
.btn-sm { font-size: var(--font-size-sm); padding: var(--space-sm) var(--space-md); border-radius: var(--radius-sm); }
```

```
.btn-md { font-size: var(--font-size-md); padding: var(--space-md) var(--space-lg); border-radius: var(--radius-md); }
```

```
.btn-lg { font-size: var(--font-size-lg); padding: var(--space-lg) var(--space-xl); border-radius: var(--radius-lg); }
```

```
.btn-xl { font-size: var(--font-size-xl); padding: var(--space-xl) var(--space-xxl); border-radius: var(--radius-lg); }
```

```
/* =====
```

 Responsive Size Variants

```
===== */
```

```
@media (min-width: var(--bp-sm)) {
```

```
  .btn-sm-sm { font-size: var(--font-size-sm); padding: var(--space-xs) var(--space-md); }
```

```
  .btn-lg-sm { font-size: var(--font-size-lg); padding: var(--space-md) var(--space-lg); }
```

```
}
```

```
@media (min-width: var(--bp-md)) {
```

```
  .btn-sm-md { font-size: var(--font-size-sm); padding: var(--space-xs) var(--space-md); }
```

```
  .btn-lg-md { font-size: var(--font-size-lg); padding: var(--space-md) var(--space-lg); }
```

```
  .btn-xl-md { font-size: var(--font-size-xl); padding: var(--space-lg) var(--space-xl); }
```

```
}
```

```
@media (min-width: var(--bp-lg)) {
```

```
  .btn-sm-lg { font-size: var(--font-size-sm); padding: var(--space-xs) var(--space-md); }
```

```
  .btn-lg-lg { font-size: var(--font-size-lg); padding: var(--space-md) var(--space-lg); }
```

```
  .btn-xl-lg { font-size: var(--font-size-xl); padding: var(--space-lg) var(--space-xl); }
```

```
}
```

```
@media (min-width: var(--bp-xl)) {  
  .btn-sm-xl { font-size: var(--font-size-sm); padding: var(--space-xs) var(--space-md); }  
  .btn-lg-xl { font-size: var(--font-size-lg); padding: var(--space-md) var(--space-lg); }  
  .btn-xl-xl { font-size: var(--font-size-xl); padding: var(--space-lg) var(--space-xl); }  
}
```

```
` ``
```

```
## frontend\src\styles\components\cards.css
```

```
` `` `css
```

```
/* =====
```

```
 Base Card
```

```
===== */
```

```
.dashboard-card {  
  background: linear-gradient(145deg, var(--card-bg), var(--card-muted));  
  color: var(--text);  
  border-radius: 12px;  
  padding: 1.5rem;  
  box-shadow:  
    0 8px 16px rgba(0,0,0,0.25),  
    0 4px 6px rgba(0,0,0,0.15),  
    inset 0 2px 4px rgba(255,255,255,0.1);  
  transition: transform 0.25s ease, box-shadow 0.25s ease;
```

```
}
```

```
.dashboard-card:hover {  
  transform: translateY(-6px) scale(1.02);  
  box-shadow:  
    0 12px 20px rgba(0,0,0,0.35),  
    0 6px 10px rgba(0,0,0,0.2),  
    inset 0 3px 6px rgba(255,255,255,0.15);  
}
```

```
/* =====
```

 Typography & Icons

```
===== */
```

```
.dashboard-card h3,  
.dashboard-card h4 {  
  font-size: 1.25rem;  
  margin-bottom: 0.75rem;  
  color: var(--accent);  
  font-weight: 600;  
}
```

```
.dashboard-card p {  
  font-size: 1.75rem;  
  font-weight: bold;  
  color: var(--text-strong);  
}
```

```
.card-icon { margin-right: 0.5rem; }
```

```
/* =====
```

Card Group Utilities

```
===== */
```

```
.card-group { display: grid; gap: var(--space-md); }
```

```
.card-group.responsive {  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
}
```

```
.card-group.row {  
  display: flex;  
  flex-wrap: wrap;  
  gap: var(--space-md);  
}
```

```
.card-group.compact { gap: var(--space-sm); }
```

```
.card-group.vertical {  
  display: flex;  
  flex-direction: column;  
  gap: var(--space-md);  
}
```

```
/* =====
```

Deck Variants

```
===== */
```

```
.card-group.deck {
```

```
  display: flex;
```

```
  flex-wrap: wrap;
```

```
  gap: var(--space-md);
```

```
  align-items: stretch;
```

```
}
```

```
.card-group.deck .dashboard-card {
```

```
  flex: 1 1 250px;
```

```
  display: flex;
```

```
  flex-direction: column;
```

```
  justify-content: space-between;
```

```
}
```

```
.card-group.deck-grid {
```

```
  display: grid;
```

```
  gap: var(--space-md);
```

```
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

```
  align-items: stretch;
```

```
}
```

```
.card-group.deck-grid .dashboard-card {
```

```
  display: flex;
```

```
  flex-direction: column;
```

```
  justify-content: space-between;
```



```
}
```

```
/* =====
```



Masonry Variants

```
===== */
```

```
.card-group.masonry {
```

```
  column-count: 3;
```

```
  column-gap: var(--space-md);
```

```
}
```

```
.card-group.masonry .dashboard-card {
```

```
  display: inline-block;
```

```
  width: 100%;
```

```
  margin-bottom: var(--space-md);
```

```
  break-inside: avoid;
```

```
}
```

```
.card-group.masonry-responsive {
```

```
  column-count: 3;
```

```
  column-gap: var(--space-md);
```

```
}
```

```
.card-group.masonry-responsive .dashboard-card {
```

```
  display: inline-block;
```

```
  width: 100%;
```

```
  margin-bottom: var(--space-md);
```

```
  break-inside: avoid;
```

```
}
```

```
@media (max-width: 1024px) {  
  .card-group.masonry-responsive { column-count: 2; }  
}
```

```
@media (max-width: 600px) {  
  .card-group.masonry-responsive { column-count: 1; }  
}
```

```
.card-group.masonry-dynamic {  
  column-width: 250px;  
  column-gap: var(--space-md);  
}
```

```
.card-group.masonry-dynamic .dashboard-card {  
  display: inline-block;  
  width: 100%;  
  margin-bottom: var(--space-md);  
  break-inside: avoid;  
}
```

```
/* =====
```

Hybrid Masonry Variants

```
===== */
```

```
.card-group.hybrid-masonry,  
.card-group.hybrid-masonry-responsive {  
  display: grid;  
  grid-auto-flow: dense;  
  gap: var(--space-md);
```

```
align-items: start;
}
```

```
.card-group.hybrid-masonry {
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
}
```

```
.card-group.hybrid-masonry-responsive {
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
}
```

```
@media (max-width: 1024px) {
  .card-group.hybrid-masonry-responsive {
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  }
}
```

```
@media (max-width: 600px) {
  .card-group.hybrid-masonry-responsive {
    grid-template-columns: 1fr;
  }
}
```

```
.card-group.hybrid-masonry .dashboard-card,
.card-group.hybrid-masonry-responsive .dashboard-card {
  display: block;
  width: 100%;
}
```

```

## frontend\src\styles\components\forms.css

```css

/* =====

 Form Wrapper

===== */

```
.create-account-form {  
  display: flex;  
  flex-direction: column;  
  gap: var(--space-md);  
  background: var(--gradient-card);  
  padding: var(--space-lg);  
  border-radius: var(--radius-md);  
  box-shadow: var(--shadow-card);  
  max-width: 500px;  
  width: 100%;  
  margin: 0 auto;  
  color: var(--color-text);  
}
```

```
.create-account-form h3 {  
  font-size: var(--font-size-lg);  
  font-weight: var(--font-weight-bold);
```

```
color: var(--color-accent);
margin-bottom: var(--space-sm);
}
```

```
.create-account-form label {
  display: flex;
  flex-direction: column;
  font-size: var(--font-size-md);
  font-weight: var(--font-weight-normal);
  color: var(--color-text-strong);
}
```

```
/* =====
```

Inputs & Selects

```
===== */
```

```
.create-account-form input,
.create-account-form select {
  margin-top: var(--space-xs);
  padding: var(--space-sm) var(--space-md);
  border-radius: var(--radius-sm);
  border: 1px solid var(--color-border);
  background-color: var(--color-input-bg);
  color: var(--color-text);
  font-size: var(--font-size-md);
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}
```

```
.create-account-form input:focus,  
.create-account-form select:focus {  
  outline: none;  
  border-color: var(--color-accent);  
  box-shadow: var(--shadow-focus);  
}
```

```
/* =====
```

Buttons

```
===== */
```

```
.create-account-form button,  
.create-account-form .btn {  
  padding: var(--space-sm) var(--space-md);  
  border-radius: var(--radius-sm);  
  border: none;  
  font-weight: var(--font-weight-bold);  
  cursor: pointer;  
  transition: background-color 0.25s ease, transform 0.15s ease;  
}
```

```
.create-account-form button:hover:not(:disabled),  
.create-account-form .btn:hover:not(:disabled) {  
  transform: translateY(-2px);  
}
```

```
.create-account-form button:disabled,  
.create-account-form .btn:disabled {  
  opacity: 0.6;  
  cursor: not-allowed;  
}
```

```
/* Button Variants */
```

```
.btn-success { background-color: var(--color-success); color: #fff; }  
.btn-success:hover { background-color: var(--color-success-hover); }
```

```
.btn-danger { background-color: var(--color-danger); color: #fff; }  
.btn-danger:hover { background-color: var(--color-danger-hover); }
```

```
.btn-accent { background-color: var(--color-accent); color: #fff; }  
.btn-accent:hover { background-color: var(--color-accent-hover); }
```

```
/* =====
```

✅ Confirmation Block

```
===== */
```

```
.confirm-actions {  
  background-color: var(--color-confirm-bg);  
  color: var(--color-confirm-text);  
  border: 1px solid var(--color-confirm-border);  
  border-radius: var(--radius-sm);  
  padding: var(--space-md);  
  display: flex;
```

```
flex-direction: column;

gap: var(--space-sm);

}
```

```
.confirm-actions p {

margin: 0;

font-size: var(--font-size-md);

}
```

```
/* =====
```

 Responsive Form Variant

```
===== */
```

```
.create-account-form.responsive {

max-width: 700px;

}
```

```
@media (min-width: var(--bp-md)) {

.create-account-form.responsive label {

flex-direction: row;

align-items: center;

justify-content: space-between;

}
```

```
.create-account-form.responsive label span {

flex: 0 0 30%;

margin-right: var(--space-sm);
```



```
}
```

```
.create-account-form.responsive input,  
.create-account-form.responsive select {  
  flex: 1;  
  margin-top: 0;  
}
```

```
.create-account-form.responsive .form-actions {  
  display: flex;  
  justify-content: flex-end;  
  gap: var(--space-sm);  
}  
}
```

```
` ``
```

```
## frontend\src\styles\components\sections.css
```

```
` `` `css
```

```
/* =====
```

```
 Section Wrapper (Base)
```

```
===== */
```

```
.section {  
  margin-top: var(--space-lg);  
  padding: var(--space-md);
```

```
background: linear-gradient(145deg, var(--card-bg), var(--card-muted));
border-radius: var(--radius-md);
border: 1px solid var(--border);
box-shadow:
  0 6px 12px rgba(0,0,0,0.15),
  inset 0 2px 4px rgba(255,255,255,0.1);
transition: transform 0.25s ease, box-shadow 0.25s ease;
}
```

```
.section:hover {
  transform: translateY(-4px);
  box-shadow:
    0 10px 18px rgba(0,0,0,0.25),
    inset 0 3px 6px rgba(255,255,255,0.15);
}
```

```
.section h3 {
  font-size: 1.25rem;
  margin-bottom: var(--space-sm);
  color: var(--accent);
  font-weight: 600;
}
```

```
/* =====
```

 Accent Variants

```
===== */
```

```
.section.accent { border-left: 6px solid var(--accent); }  
.section.success { border-left: 6px solid var(--success); }  
.section.danger { border-left: 6px solid var(--danger); }  
.section.warning { border-left: 6px solid var(--warning); }  
.section.info { border-left: 6px solid var(--info); }
```

```
/* =====
```

 Responsive Variant

```
===== */
```

```
.section.responsive {  
  margin-top: var(--space-md);  
  padding: var(--space-md);  
}
```

```
/* Tablet breakpoint */
```

```
@media (min-width: 768px) {
```

```
  .section.responsive {  
    margin-top: var(--space-lg);  
    padding: var(--space-lg);  
  }  
}
```

```
}
```

```
/* Desktop breakpoint */
```

```
@media (min-width: 1200px) {
```

```
  .section.responsive {  
    margin-top: var(--space-xl);
```

```
padding: var(--space-xl);
max-width: 1200px;
margin-left: auto;
margin-right: auto;
}
}
```

```
` ``
```

```
## frontend\src\styles\components\tables.css
```

```
` `` `css
```

```
/* =====
```

```
 Tables (Base)
```

```
===== */
```

```
table {
width: 100%;
border-collapse: collapse;
background-color: var(--color-card-bg);
color: var(--color-text);
font-size: clamp(0.875rem, 1vw + 0.75rem, 1rem); /* responsive text */
border-radius: var(--radius-md);
overflow: hidden; /* keeps rounded corners */
}
```

```
th, td {
```

```
padding: var(--space-sm) var(--space-md);
text-align: left;
vertical-align: middle;
}

th {
  background-color: var(--color-table-header);
  color: var(--color-text-strong);
  font-weight: 600;
  border-bottom: 2px solid var(--color-border-strong);
}
```

```
td {
  border-bottom: 1px solid var(--color-border);
}
```

```
/* Row Styling */
```

```
tr:nth-child(even) { background-color: var(--color-table-row-even); }
tr:hover {
  background-color: var(--color-accent-hover);
  color: #fff;
  transition: background-color 0.2s ease, color 0.2s ease;
}
```

```
/* =====
```

 Responsive Wrapper

```
===== */
```

```
.table-responsive {  
  display: block;  
  width: 100%;  
  overflow-x: auto;  
}
```

```
/* =====
```

Accessibility

```
===== */
```

```
th[scope="col"] {  
  text-transform: uppercase;  
  letter-spacing: 0.05em;  
}
```

```
tr:focus-within {  
  outline: var(--focus-ring);  
  outline-offset: -2px;  
}
```

```
/* =====
```

Table Variants

```
===== */
```

```
.table-striped tr:nth-child(even) { background-color: var(--color-table-row-even); }
```

```
.table-bordered {  
  border: 1px solid var(--color-border-strong);
```

```
border-radius: var(--radius-sm);
overflow: hidden;
}

.table-bordered th,
.table-bordered td { border: 1px solid var(--color-border); }

.table-hover tr:hover {
background-color: var(--color-accent-hover);
color: #fff;
}

.table-compact th,
.table-compact td {
padding: var(--space-xs) var(--space-sm);
font-size: 0.875rem;
}

.table-dark {
background-color: var(--color-card-muted);
color: var(--color-text-strong);
}

.table-dark th {
background-color: var(--color-border-strong);
color: var(--color-text);
}

.table-dark tr:nth-child(even) { background-color: var(--color-bg); }
```

```
/* =====
```

Table Row Status Variants

```
===== */
```

```
tr.row-success { background-color: var(--color-success); color: #fff; }
```

```
tr.row-success:hover { background-color: var(--color-success-hover); }
```

```
tr.row-danger { background-color: var(--color-danger); color: #fff; }
```

```
tr.row-danger:hover { background-color: var(--color-danger-hover); }
```

```
tr.row-warning { background-color: var(--color-confirm-bg); color: var(--color-confirm-text);  
}
```

```
tr.row-warning:hover { background-color: var(--color-confirm-border); }
```

```
tr.row-info { background-color: var(--color-accent); color: #fff; }
```

```
tr.row-info:hover { background-color: var(--color-accent-hover); }
```

```
tr.row-inreview { background-color: var(--color-review); color: #fff; }
```

```
tr.row-inreview:hover { background-color: var(--color-review-hover); }
```

```
````
```

```
frontend\src\styles\core\base.css
```

```
````css
```



```
/* =====
```

Base Layout & Typography

```
===== */
```

```
html {
```

```
  font-size: 100%; /* baseline: 16px */
```

```
}
```

```
body {
```

```
  background-color: var(--color-bg);
```

```
  color: var(--color-text);
```

```
  font-family: var(--font-family-base);
```

```
  margin: 0; padding: 0;
```

```
  line-height: clamp(1.5, 2vw + 1.2, 1.8);
```

```
  transition: background-color 0.3s ease, color 0.3s ease;
```

```
}
```

```
/* Progressive enhancement: smooth scrolling */
```

```
@supports (scroll-behavior: smooth) {
```

```
  body {
```

```
    scroll-behavior: smooth;
```

```
  }
```

```
}
```

```
/* Headings with responsive clamp() for font-size + line-height */
```

```
h1 {
```

```
  font-size: clamp(2rem, 4vw + 1rem, 3rem);
```

```
line-height: clamp(1.2, 2vw + 1, 1.3); /* tighter on large screens */  
font-weight: 700;  
color: var(--color-text-strong);  
}
```

```
h2 {  
  font-size: clamp(1.5rem, 3vw + 0.5rem, 2.25rem);  
  line-height: clamp(1.25, 2vw + 1, 1.4);  
  font-weight: 600;  
  color: var(--color-text-strong);  
}
```

```
h3 {  
  font-size: clamp(1.25rem, 2vw + 0.5rem, 1.75rem);  
  line-height: clamp(1.3, 2vw + 1, 1.5);  
  font-weight: 600;  
  color: var(--color-text-strong);  
}
```

```
h4 {  
  font-size: clamp(1rem, 1.5vw + 0.25rem, 1.25rem);  
  line-height: clamp(1.4, 2vw + 1, 1.6);  
  font-weight: 500;  
  color: var(--color-text-strong);  
}
```

```

h5, h6 {
  font-size: clamp(0.875rem, 1vw + 0.25rem, 1rem);
  line-height: clamp(1.4, 2vw + 1, 1.6);
  font-weight: 500;
  color: var(--color-text-strong);
}

/* Paragraphs */
p {
  font-size: clamp(1rem, 1.25vw + 0.25rem, 1.125rem);
  line-height: clamp(1.5, 2vw + 1.2, 1.8); /* looser for readability */
  margin: 0 0 var(--space-md);
}

/* =====

🔗 Links
===== */

a {
  color: var(--color-accent);
  text-decoration: none;
  font-weight: 500;
  transition: color 0.2s ease;
}

a:hover { text-decoration: underline; }

```

```
a:focus { outline: var(--focus-ring); }
a:active { color: var(--color-accent-hover); }
```

```
` ``
```

```
## frontend\src\styles\core\utilities.css
```

```
` `` `css
```

```
/* =====
```

```
 Grid Utilities
```

```
===== */
```

```
.grid {
  display: grid;
  gap: var(--space-md);
}
```

```
.grid-1 { grid-template-columns: repeat(1, 1fr); }
```

```
.grid-2 { grid-template-columns: repeat(2, 1fr); }
```

```
.grid-3 { grid-template-columns: repeat(3, 1fr); gap: var(--space-lg); }
```

```
.grid-4 { grid-template-columns: repeat(4, 1fr); gap: var(--space-lg); }
```

```
.grid-auto { grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); gap: var(--space-
lg); }
```

```
/* =====
```

```
 Flex Utilities
```

```
===== */
```

```
.flex {  
  display: flex;  
  gap: var(--space-md);  
}
```

```
.flex-start { justify-content: flex-start; align-items: flex-start; }  
.flex-end   { justify-content: flex-end; align-items: flex-end; }  
.flex-center { justify-content: center; align-items: center; }  
.flex-between { justify-content: space-between; align-items: center; }  
.flex-around { justify-content: space-around; align-items: center; }  
.flex-wrap   { flex-wrap: wrap; }
```

```
/* =====
```

Spacing Utilities

```
===== */
```

```
/* Margin */
```

```
.m-0 { margin: 0; }  
.m-xs { margin: var(--space-xs); }  
.m-sm { margin: var(--space-sm); }  
.m-md { margin: var(--space-md); }  
.m-lg { margin: var(--space-lg); }  
.m-xl { margin: var(--space-xl); }
```

```
/* Padding */
```

```
.p-0 { padding: 0; }  
.p-xs { padding: var(--space-xs); }
```

```
.p-sm { padding: var(--space-sm); }  
.p-md { padding: var(--space-md); }  
.p-lg { padding: var(--space-lg); }  
.p-xl { padding: var(--space-xl); }
```

/* Directional Margin */

```
.mt-xs { margin-top: var(--space-xs); }  
.mt-sm { margin-top: var(--space-sm); }  
.mt-md { margin-top: var(--space-md); }  
.mt-lg { margin-top: var(--space-lg); }  
.mt-xl { margin-top: var(--space-xl); }
```

```
.mb-xs { margin-bottom: var(--space-xs); }  
.mb-sm { margin-bottom: var(--space-sm); }  
.mb-md { margin-bottom: var(--space-md); }  
.mb-lg { margin-bottom: var(--space-lg); }  
.mb-xl { margin-bottom: var(--space-xl); }
```

```
.ml-xs { margin-left: var(--space-xs); }  
.ml-sm { margin-left: var(--space-sm); }  
.ml-md { margin-left: var(--space-md); }  
.ml-lg { margin-left: var(--space-lg); }  
.ml-xl { margin-left: var(--space-xl); }
```

```
.mr-xs { margin-right: var(--space-xs); }  
.mr-sm { margin-right: var(--space-sm); }
```

```
.mr-md { margin-right: var(--space-md); }  
.mr-lg { margin-right: var(--space-lg); }  
.mr-xl { margin-right: var(--space-xl); }
```

/* Directional Padding */

```
.pt-xs { padding-top: var(--space-xs); }  
.pt-sm { padding-top: var(--space-sm); }  
.pt-md { padding-top: var(--space-md); }  
.pt-lg { padding-top: var(--space-lg); }  
.pt-xl { padding-top: var(--space-xl); }
```

```
.pb-xs { padding-bottom: var(--space-xs); }  
.pb-sm { padding-bottom: var(--space-sm); }  
.pb-md { padding-bottom: var(--space-md); }  
.pb-lg { padding-bottom: var(--space-lg); }  
.pb-xl { padding-bottom: var(--space-xl); }
```

```
.pl-xs { padding-left: var(--space-xs); }  
.pl-sm { padding-left: var(--space-sm); }  
.pl-md { padding-left: var(--space-md); }  
.pl-lg { padding-left: var(--space-lg); }  
.pl-xl { padding-left: var(--space-xl); }
```

```
.pr-xs { padding-right: var(--space-xs); }  
.pr-sm { padding-right: var(--space-sm); }  
.pr-md { padding-right: var(--space-md); }
```

```
.pr-lg { padding-right: var(--space-lg); }
```

```
.pr-xl { padding-right: var(--space-xl); }
```

```
/* Auto Helpers */
```

```
.mx-auto { margin-left: auto; margin-right: auto; }
```

```
.my-auto { margin-top: auto; margin-bottom: auto; }
```

```
/* =====
```

```
 List Reset
```

```
===== */
```

```
.list-reset { list-style: none; padding: 0; margin: 0; }
```

```
` ``
```

```
## frontend\src\styles\core\variables.css
```

```
` `` `css
```

```
/* =====
```

```
 Base Theme Tokens
```

```
===== */
```

```
:root {
```

```
/*  Colors */
```

```
--color-bg: #f9f9f9;
```

```
--color-text: #333;
```

```
--color-text-strong: #111;
```


--color-card-bg: #ffffff;

--color-card-muted: #f5f5f5;

--color-accent: #3498db;

--color-accent-hover: #2980b9;

--color-success: #28a745;

--color-success-hover: #218838;

--color-danger: #dc3545;

--color-danger-hover: #c82333;

--color-warning: #ffc40f;

--color-warning-hover: #d4ac0d;

--color-info: #9b59b6;

--color-info-hover: #8e44ad;

--color-review: #3498db;

--color-review-hover: #2980b9;

--color-neutral: #bdc3c7;

--color-link: #0066cc;

--color-link-hover: #004999;

--color-disabled: #ccc;

--color-border: #ccc;

--color-border-strong: #ddd;

--color-table-header: #f0f0f0;

--color-table-row-even: #f9f9f9;

--color-table-row-hover: #eef;

--color-input-bg: #fff;

--color-confirm-bg: #fff3cd;

--color-confirm-text: #212529;

--color-confirm-border: #ffeeba;

/* 👤 Role Colors */

--color-resident: #7f8c8d;

--color-staff: #2980b9;

--color-secretary: #27ae60;

--color-admin: #c0392b;

--color-youth: #f63bed;

--color-treasurer: #f39c12;

--color-dilg: #8b4513;

/* ✎ Typography */

--font-family-base: system-ui, sans-serif;

--font-size-sm: 0.875rem;

--font-size-md: 1rem;

--font-size-lg: 1.25rem;

--font-size-xl: 2rem;

--line-height-sm: 1.2;

--line-height-md: 1.5;

--line-height-lg: 1.8;

--font-weight-light: 300;

--font-weight-normal: 400;

--font-weight-bold: 700;

--letter-spacing-sm: 0.5px;

--letter-spacing-md: 1px;

/*  Spacing */

--space-xs: 0.25rem;

--space-sm: 0.5rem;

--space-md: 1rem;

--space-lg: 2rem;

--space-xl: 3rem;

--space-xxl: 4rem;

/*  Radius */

--radius-sm: 4px;

```
--radius-md: 8px;  
--radius-lg: 12px;  
--radius-xl: 16px;  
--radius-round: 50%;
```

```
/* 🟣 Shadows */
```

```
--shadow-sm: 0 2px 4px rgba(0,0,0,0.1);  
--shadow-card: 0 8px 16px rgba(0,0,0,0.25);  
--shadow-card-hover: 0 12px 20px rgba(0,0,0,0.35);  
--shadow-lg: 0 16px 32px rgba(0,0,0,0.25);  
--shadow-inset: inset 0 2px 4px rgba(255,255,255,0.1);  
--shadow-focus: 0 0 0 3px rgba(52,152,219,0.4);
```

```
/* 🎨 Gradients */
```

```
--gradient-card: linear-gradient(145deg, var(--color-card-bg), var(--color-card-muted));  
--gradient-accent: linear-gradient(145deg, #3498db, #2980b9);  
--gradient-success: linear-gradient(145deg, #28a745, #218838);  
--gradient-danger: linear-gradient(145deg, #dc3545, #c82333);
```

```
/* 🍷 Focus Ring */
```

```
--focus-ring: 0 0 0 3px rgba(52, 152, 219, 0.25);  
--focus-ring-color: #ffcc00;
```

```
/* 🏠 Breakpoints */
```

```
--bp-xs: 400px;  
--bp-sm: 576px;
```

```
--bp-md: 768px;
--bp-lg: 992px;
--bp-xl: 1200px;
--bp-xxl: 1400px;
}
```

```
/* =====
```

Dark Theme Overrides

```
===== */
```

```
[data-theme="dark"] {
```

```
--color-bg: #1e1e2f;
```

```
--color-text: #e0e0e0;
```

```
--color-text-strong: #ffffff;
```

```
--color-card-bg: #2c2c3c;
```

```
--color-card-muted: #3a3a4a;
```

```
--color-accent: #4aa8ff;
```

```
--color-accent-hover: #007acc;
```

```
--color-success: #3ddc84;
```

```
--color-success-hover: #2cbf6e;
```

```
--color-danger: #ff6b6b;
```

```
--color-danger-hover: #e63946;
```

--color-warning: #f39c12;

--color-warning-hover: #d35400;

--color-info: #8e44ad;

--color-info-hover: #6c3483;

--color-review: #4aa8ff;

--color-review-hover: #007acc;

--color-neutral: #95a5a6;

--color-link: #4aa8ff;

--color-link-hover: #007acc;

--color-disabled: #666;

--color-border: #666;

--color-border-strong: #555;

--color-table-header: #2f2f3f;

--color-table-row-even: #252535;

--color-table-row-hover: #3a3a4a;

--color-input-bg: #1e1e2f;

--color-confirm-bg: #2c2c2c;

```
--color-confirm-text: #f8f9fa;
--color-confirm-border: #444;
```

```
/* 👥 Role Colors */
```

```
--color-resident: #95a5a6;
--color-staff: #3498db;
--color-secretary: #2ecc71;
--color-admin: #e74c3c;
--color-treasurer: #f39c12;
--color-dilg: #8b4513;
--color-youth: #f63bed;
```

```
}
```

```
` ``
```

```
## frontend\src\styles\dashboard\analytics-panel.css
```

```
` `` `css
```

```
/* =====
```

```
 Analytics Panel
```

```
===== */
```

```
.analytics-panel {
  background: var(--gradient-card);
  border-radius: var(--radius-md);
  padding: var(--space-md);
  box-shadow: var(--shadow-sm);
```

```
border: 1px solid var(--color-border);  
margin-top: var(--space-lg);  
}
```

```
/* Header with toggle */
```

```
.analytics-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: var(--space-md);  
}
```

```
.analytics-header h3 {  
  font-size: var(--font-size-lg);  
  font-weight: 600;  
  color: var(--color-accent);  
}
```

```
/* Toggle buttons */
```

```
.trend-toggle {  
  display: flex;  
  gap: var(--space-sm);  
}
```

```
.chart-controls {  
  display: flex;
```



```
align-items: center;

gap: var(--space-sm);

margin-bottom: var(--space-md);
}
```

```
.chart-controls label {

font-weight: 600;

color: var(--color-text-strong);
}
```

```
.category-select {

border-radius: var(--radius-sm);

border: 1px solid var(--color-border);

background: var(--color-card-muted);

padding: var(--space-xs) var(--space-sm);

color: var(--color-text-strong);
}
```

```
.period-input {

border-radius: var(--radius-sm);

border: 1px solid var(--color-border);

background: var(--color-card-muted);

padding: var(--space-xs) var(--space-sm);

color: var(--color-text-strong);
}
```

```
.period-toggle {  
  display: flex;  
  gap: var(--space-xs);  
}
```

```
.compare-controls {  
  display: flex;  
  gap: var(--space-md);  
  flex-wrap: wrap;  
  width: 100%;  
}
```

```
.compare-block {  
  display: flex;  
  flex-direction: column;  
  gap: var(--space-xs);  
  min-width: 260px;  
}
```

```
.compare-title {  
  font-weight: 600;  
  color: var(--color-text-strong);  
}
```

```
.trend-toggle button {  
  padding: var(--space-xs) var(--space-sm);
```

```
border-radius: var(--radius-sm);  
border: 1px solid var(--color-border);  
background: var(--color-card-muted);  
cursor: pointer;  
font-weight: 600;  
transition: background-color 0.2s ease, color 0.2s ease;  
color: var(--color-text-strong); /* ✅ ensures visibility in light mode */  
}
```

```
.period-toggle button {  
padding: var(--space-xs) var(--space-sm);  
border-radius: var(--radius-sm);  
border: 1px solid var(--color-border);  
background: var(--color-card-muted);  
cursor: pointer;  
font-weight: 600;  
transition: background-color 0.2s ease, color 0.2s ease;  
color: var(--color-text-strong);  
}
```

```
.period-toggle button:hover {  
background: var(--color-table-row-hover);  
}
```

```
.period-toggle button.active {  
background: var(--color-accent);
```

```
color: #fff;
border-color: var(--color-accent);
}
```

```
.trend-toggle button:hover {
  background: var(--color-table-row-hover);
}
```

```
.trend-toggle button.active {
  background: var(--color-accent);
  color: #fff; /* ✅ white text for contrast */
  border-color: var(--color-accent);
}
```

```
/* Metrics grid */
```

```
.metrics-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: var(--space-md);
}
```

```
/* Trend indicators inside cards */
```

```
.trend-indicator {
  margin-left: var(--space-xs);
  font-size: var(--font-size-sm);
  font-weight: 600;
```

```
}
```

```
.trend-indicator.up {  
  color: var(--color-success);  
}
```

```
.trend-indicator.down {  
  color: var(--color-danger);  
}
```

```
.trend-indicator.neutral {  
  color: var(--color-text-muted);  
}
```

```
/* Chart container */
```

```
.analytics-chart {  
  grid-column: 1 / -1; /* span full width */  
  margin-top: var(--space-lg);  
  background: var(--color-card-muted);  
  padding: var(--space-md);  
  border-radius: var(--radius-md);  
  box-shadow: var(--shadow-sm);  
}
```

```
.metric-card-wrap {  
  border-radius: var(--radius-md);
```

```
outline: 2px solid transparent;
transition: outline-color 0.2s ease;
cursor: pointer;
}
```

```
.metric-card-wrap.selected {
  outline-color: var(--color-accent);
}
```

```
.metric-card-wrap:focus-visible {
  outline-color: var(--color-accent);
}
```

```
/* Responsive adjustments */
@media (max-width: 768px) {
  .analytics-header {
    flex-direction: column;
    align-items: flex-start;
    gap: var(--space-sm);
  }
}
```

```
.trend-toggle {
  width: 100%;
  justify-content: flex-start;
}
```

```
.period-toggle {  
  width: 100%;  
  flex-wrap: wrap;  
}
```

```
.compare-block {  
  min-width: 100%;  
}
```

```
.analytics-chart {  
  margin-top: var(--space-md);  
}  
}
```

```
...
```

```
## frontend\src\styles\dashboard\audit-table.css
```

```
```css
```

```
/* audit-table.css */
```

```
/* Container */
```

```
.audit-table {
 margin-top: 1.5rem;
 background-color: #fff;
 border-radius: 6px;
```

```
overflow: hidden;

box-shadow: 0 1px 3px rgba(0,0,0,0.1);

padding: 1rem;
}
```

```
/* Heading */
```

```
.audit-table h3 {

font-size: 1.2rem;

margin-bottom: 1rem;

color: var(--color-dilg);
}
```

```
/* Status messages */
```

```
.audit-table .status-message {

margin: 0.5rem 0;

padding: 0.75rem 1rem;

border-radius: 4px;

background-color: #fff3cd;

color: #856404;

font-size: 0.9rem;
}
```

```
/* Table */
```

```
.audit-table table {

width: 100%;

border-collapse: collapse;
```



```
font-size: 0.9rem;
}
```

```
.audit-table th,
.audit-table td {
padding: 0.75rem 1rem;
text-align: left;
border-bottom: 1px solid #e5e7eb;
}
```

```
.audit-table th {
background-color: #f3f4f6;
font-weight: 600;
color: #111827;
}
```

```
.audit-table tr:hover {
background-color: #f9fafb;
}
```

```
/* Action column (if you add buttons later) */
```

```
.audit-table td button {
background: none;
border: none;
color: #2563eb;
font-weight: 500;
```

```
 cursor: pointer;
}
```

```
.audit-table td button:hover {
 text-decoration: underline;
}
```

```
/* Empty state */
.audit-table p {
 margin: 1rem 0;
 color: #6b7280;
 font-style: italic;
}
```

```
...
```

```
frontend\src\styles\dashboard\complaint-list.css
```

```
```css
```

```
/* =====
```

```
 Complaint List
```

```
===== */
```

```
.complaint-list {
    margin-top: var(--space-md);
    background: var(--gradient-card);
    padding: var(--space-md);
```

```
border-radius: var(--radius-md);  
box-shadow: var(--shadow-sm);  
min-width: 0;  
overflow-x: auto;  
overflow-y: hidden;  
}
```

```
.complaint-list h3 {  
  font-size: var(--font-size-lg);  
  font-weight: 600;  
  color: var(--color-accent);  
  margin-bottom: var(--space-md);  
}
```

/ Table */*

```
.complaint-table {  
  width: 100%;  
  border-collapse: collapse;  
}
```

```
.complaint-table th,  
.complaint-table td {  
  padding: var(--space-sm) var(--space-md);  
  border-bottom: 1px solid var(--color-border);  
  text-align: left;  
}
```

```
.complaint-table th {  
  font-weight: 600;  
  color: var(--color-text-strong);  
}
```

```
.complaint-table tr:hover {  
  background: var(--color-table-row-hover);  
  color: var(--color-text-strong);  
}
```

```
/* Status badges */
```

```
.status-badge {  
  display: inline-block;  
  padding: 0.25rem 0.5rem;  
  border-radius: var(--radius-sm);  
  font-size: var(--font-size-sm);  
  font-weight: 600;  
  text-transform: capitalize;  
  color: #fff;  
}
```

```
/* Open = amber */
```

```
.status-badge.open {  
  background: var(--color-warning);  
}
```

```
/* In Review = blue */
```

```
.status-badge.inreview {  
  background: var(--color-review);  
}
```

```
/* Resolved = green */
```

```
.status-badge.resolved {  
  background: var(--color-success);  
}
```

```
/* Rejected = red */
```

```
.status-badge.rejected {  
  background: var(--color-danger);  
}
```

```
/* Escalated = purple/accent */
```

```
.status-badge.escalated {  
  background: var(--color-accent);  
}
```

```
@media (max-width: 768px) {
```

```
  .complaint-list {  
    padding: 0.75rem;  
  }
```

```
.complaint-list h3 {  
    font-size: 1rem;  
}
```

```
.complaint-table {  
    display: table;  
    min-width: 720px;  
    white-space: nowrap;  
}
```

```
.complaint-table th,  
.complaint-table td {  
    padding: 0.5rem 0.625rem;  
    font-size: 0.875rem;  
}  
}
```

```
```
```

```
frontend\src\styles\dashboard\dashboard-card.css
```

```
```css
```

```
/* =====
```

```
 Dashboard Card
```

```
===== */
```

```
.dashboard-card {
```

```
background: var(--gradient-card);  
  
border-radius: var(--radius-md);  
  
padding: var(--space-md);  
  
box-shadow: var(--shadow-card);  
  
transition: transform 0.25s ease, box-shadow 0.25s ease;  
}
```

```
.dashboard-card:hover {  
  
  transform: translateY(-4px);  
  
  box-shadow: var(--shadow-card-hover);  
}
```

```
.dashboard-card h4 {  
  
  font-size: var(--font-size-md);  
  
  font-weight: 600;  
  
  margin-bottom: var(--space-xs);  
  
  color: var(--color-text-strong);  
  
  display: flex;  
  
  align-items: center;  
  
  gap: var(--space-xs);  
}
```

```
.dashboard-card p {  
  
  font-size: var(--font-size-lg);  
  
  font-weight: bold;  
  
  color: var(--color-text);  
}
```

```
}
```

```
/* Icon spacing */
```

```
.card-icon {  
  margin-right: var(--space-xs);  
}
```

```
/* Variants */
```

```
.dashboard-card.accent { border-left: 6px solid var(--color-accent); }  
.dashboard-card.success { border-left: 6px solid var(--color-success); }  
.dashboard-card.info { border-left: 6px solid var(--color-info); }  
.dashboard-card.warning { border-left: 6px solid var(--color-warning); }  
.dashboard-card.neutral { border-left: 6px solid var(--color-neutral); }  
.dashboard-card.danger { border-left: 6px solid var(--color-danger); }  
.dashboard-card.youth { border-left: 6px solid var(--color-youth); }  
.dashboard-card.dilg { border-left: 6px solid var(--color-dilg); }  
` ``
```

```
## frontend\src\styles\dashboard\document-queue.css
```

```
` `` `css
```

```
/* =====
```

```
 Document Queue
```

```
===== */
```

```
.document-queue {  
  margin-top: var(--space-md);
```



```
background: var(--gradient-card);
padding: var(--space-md);
border-radius: var(--radius-md);
box-shadow: var(--shadow-sm);
min-width: 0;
overflow-x: auto;
overflow-y: hidden;
}
```

```
.document-queue h3 {
  font-size: var(--font-size-lg);
  font-weight: 600;
  color: var(--color-accent);
  margin-bottom: var(--space-md);
}
```

```
/* =====
```

 Table

```
===== */
```

```
.queue-table {
  width: 100%;
  border-collapse: collapse;
}
```

```
.queue-table th,
```

```
.queue-table td {
```

```
padding: var(--space-sm) var(--space-md);
border-bottom: 1px solid var(--color-border);
text-align: left;
}
```

```
.queue-table th {
  font-weight: 600;
  color: var(--color-text-strong);
}
```

```
.queue-table tr:hover {
  background: var(--color-table-row-hover);
  color: var(--color-text-strong); /* ✅ ensures text stays visible */
}
```

```
/* =====
```

Status Badges

```
===== */
```

```
.status-badge {
  display: inline-block;
  padding: 0.25rem 0.5rem;
  border-radius: var(--radius-sm);
  font-size: var(--font-size-sm);
  font-weight: 600;
  text-transform: capitalize;
  color: #fff;
```

```
}
```

```
.status-badge.pending { background: var(--color-warning); }
```

```
.status-badge.approved { background: var(--color-success); }
```

```
.status-badge.rejected { background: var(--color-danger); }
```

```
/* =====
```

Action Buttons

```
===== */
```

```
.review-btn,
```

```
.approve-btn,
```

```
.reject-btn {
```

```
padding: var(--space-xs) var(--space-sm);
```

```
border-radius: var(--radius-sm);
```

```
border: none;
```

```
font-weight: 600;
```

```
cursor: pointer;
```

```
transition: background-color 0.2s ease;
```

```
color: #fff;
```

```
}
```

```
.review-btn {
```

```
background: var(--color-accent);
```

```
}
```

```
.review-btn:hover {
```

```
background: var(--color-accent-hover);
```

```
}
```

```
.approve-btn {
```

```
  background: var(--color-success);
```

```
}
```

```
.approve-btn:hover {
```

```
  background: var(--color-success-hover);
```

```
}
```

```
.reject-btn {
```

```
  background: var(--color-danger);
```

```
}
```

```
.reject-btn:hover {
```

```
  background: var(--color-danger-hover);
```

```
}
```

```
/* =====
```

```
 Modal
```

```
===== */
```

```
.modal-overlay {
```

```
  position: fixed;
```

```
  inset: 0;
```

```
  background: rgba(0,0,0,0.5);
```

```
  display: flex;
```

```
  justify-content: center;
```

```
  align-items: center;
```

```
z-index: 999;  
}
```

```
.modal {  
  background: var(--color-card-bg, #fff);  
  border-radius: var(--radius-md);  
  padding: var(--space-md);  
  width: 400px;  
  max-width: 90%;  
  box-shadow: var(--shadow-lg);  
  animation: fadeIn 0.3s ease;  
}
```

```
.modal-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: var(--space-sm);  
}
```

```
.modal-header h4 {  
  font-size: var(--font-size-md);  
  font-weight: 600;  
  color: var(--color-accent);  
}
```

```
.close-btn {  
  background: none;  
  border: none;  
  font-size: var(--font-size-lg);  
  cursor: pointer;  
  color: var(--color-text-strong); /* ✅ visible in light mode */  
  transition: color 0.2s ease;  
}  
  
.close-btn:hover {  
  color: var(--color-danger); /* red highlight on hover */  
}
```

```
.modal-body p {  
  margin: var(--space-xs) 0;  
  font-size: var(--font-size-sm);  
}
```

```
.modal-footer {  
  display: flex;  
  justify-content: flex-end;  
  gap: var(--space-sm);  
  margin-top: var(--space-md);  
}
```

```
/* =====
```

💡 Effects

```
===== */  
  
.document-queue.blurred {  
  filter: blur(4px);  
  pointer-events: none; /* prevent clicks behind modal */  
}
```

```
@keyframes fadeIn {  
  from { opacity: 0; transform: scale(0.95); }  
  to { opacity: 1; transform: scale(1); }  
}
```

```
@media (max-width: 768px) {  
  .document-queue {  
    padding: 0.75rem;  
  }  
}
```

```
.document-queue h3 {  
  font-size: 1rem;  
}
```

```
.queue-table {  
  display: table;  
  min-width: 640px;  
  white-space: nowrap;  
}
```

```
.queue-table th,  
.queue-table td {  
    padding: 0.5rem 0.625rem;  
    font-size: 0.875rem;  
}
```

```
.modal {  
    width: 100%;  
    max-width: 100%;  
}  
}
```

```
```
```

```
frontend\src\styles\dashboard\documents-admin.css
```

```
```css
```

```
/* documents-admin.css */
```

```
/* Page container */
```

```
.documents-admin-page {  
    padding: 2rem;  
    background-color: var(--color-bg);  
    font-family: "Segoe UI", Roboto, sans-serif;  
    color: var(--color-text);  
    margin-bottom: 3rem;
```



```
}
```

```
.header-row {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 1rem;  
}
```

```
.issue-document-btn {  
  background-color: #007bff;  
  color: white;  
  border: none;  
  padding: 0.5rem 1rem;  
  font-size: 0.9rem;  
  border-radius: 4px;  
  cursor: pointer;  
  transition: background-color 0.2s ease;  
}
```

```
.issue-document-btn:hover {  
  background-color: #0056b3;  
}
```

```
/* Headings */
```

```
.documents-admin-page h1 {
```

```
font-size: 1.8rem;  
margin-bottom: 1.5rem;  
color: var(--color-accent);  
}
```

```
.documents-admin-page h2 {  
font-size: 1.4rem;  
margin-top: 2rem;  
margin-bottom: 1rem;  
color: var(--color-success);  
}
```

```
/* Status messages */  
.status-message {  
margin: 0.5rem 0;  
padding: 0.75rem 1rem;  
border-radius: 4px;  
background-color: var(--color-neutral);  
color: var(--color-text);  
font-size: 0.9rem;  
}
```

```
/* Document table */  
.document-table {  
width: 100%;  
border-collapse: collapse;
```

```
margin-top: 1rem;

background-color: var(--color-bg);

border-radius: 6px;

overflow: hidden;

box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

```
.document-table th,
.document-table td {

padding: 0.75rem 1rem;

text-align: left;

border-bottom: 1px solid var(--color-border);
}
```

```
.document-table th {

background-color: var(--color-bg);

font-weight: 600;

color: var(--color-text-strong);
}
```

```
.document-table tr:hover {

background-color: var(--color-neutral);
}
```

```
/* Action links/buttons */
```

```
.document-table a {
```

```
color: #2563eb;
text-decoration: none;
font-weight: 500;
}
```

```
.document-table a:hover {
  text-decoration: underline;
}
```

```
.document-table button {
  background: none;
  border: none;
  color: #dc2626;
  font-weight: 500;
  cursor: pointer;
}
```

```
.document-table button:hover {
  text-decoration: underline;
}
```

```
/* Audit table styling (shared look) */
```

```
.audit-table {
  margin-top: 1rem;
  background-color: var(--color-bg);
  border-radius: 6px;
```

```
overflow: hidden;
box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

```
.audit-table table {
width: 100%;
border-collapse: collapse;
}
```

```
.audit-table th,
.audit-table td {
padding: 0.75rem 1rem;
border-bottom: 1px solid #e5e7eb;
}
```

```
.audit-table th {
background-color: #f3f4f6;
font-weight: 600;
color: #111827;
}
```

```
.audit-table tr:hover {
background-color: #f9fafb;
}
```

```
...
```

```
## frontend\src\styles\dashboard\incident-queue.css
```

```
` `` `css
```

```
/* =====
```

```
 Incident Queue
```

```
===== */
```

```
.incident-queue {  
  margin-top: var(--space-md);  
  background: var(--gradient-card);  
  padding: var(--space-md);  
  border-radius: var(--radius-md);  
  box-shadow: var(--shadow-sm);  
  min-width: 0;  
  overflow-x: auto;  
  overflow-y: hidden;  
}
```

```
.incident-queue h3 {  
  font-size: var(--font-size-lg);  
  font-weight: 600;  
  color: var(--color-accent);  
  margin-bottom: var(--space-md);  
}
```

```
/* Table */
```

```
.incident-table {  
  width: 100%;  
  border-collapse: collapse;  
}
```

```
.incident-table th,  
.incident-table td {  
  padding: var(--space-sm) var(--space-md);  
  border-bottom: 1px solid var(--color-border);  
  text-align: left;  
}
```

```
.incident-table th {  
  font-weight: 600;  
  color: var(--color-text-strong);  
}
```

```
.incident-table tr:hover {  
  background: var(--color-table-row-hover);  
  color: var(--color-text-strong);  
}
```

/ Status badges */*

```
.status-badge {  
  display: inline-block;  
  padding: 0.25rem 0.5rem;
```

```
border-radius: var(--radius-sm);  
font-size: var(--font-size-sm);  
font-weight: 600;  
text-transform: capitalize;  
color: #fff;  
}
```

```
/* Pending = amber */  
.status-badge.pending {  
  background: var(--color-warning);  
}
```

```
/* Escalated = red, stands out */  
.status-badge.escalated {  
  background: var(--color-danger);  
}
```

```
/* Resolved = green */  
.status-badge.resolved {  
  background: var(--color-success);  
}
```

```
/* Dismissed = gray */  
.status-badge.dismissed {  
  background: var(--color-muted);  
}
```



```
@media (max-width: 768px) {
```

```
  .incident-queue {
```

```
    padding: 0.75rem;
```

```
  }
```

```
  .incident-queue h3 {
```

```
    font-size: 1rem;
```

```
  }
```

```
  .incident-table {
```

```
    display: table;
```

```
    min-width: 700px;
```

```
    white-space: nowrap;
```

```
  }
```

```
  .incident-table th,
```

```
  .incident-table td {
```

```
    padding: 0.5rem 0.625rem;
```

```
    font-size: 0.875rem;
```

```
  }
```

```
}
```

```
...
```

```
## frontend\src\styles\dashboard\registry-audit.css
```

```
```css
```

```
.registry-audit {
 background: var(--gradient-card);
 border-radius: var(--radius-md);
 padding: var(--space-md);
 box-shadow: var(--shadow-sm);
 border: 1px solid var(--color-border);
}
```

```
.registry-audit h3 {
 font-size: var(--font-size-lg);
 font-weight: 600;
 color: var(--color-accent);
 margin-bottom: var(--space-md);
}
```

```
.registry-audit ul {
 list-style: none;
 padding: 0;
 margin: 0;
 display: flex;
 flex-direction: column;
 gap: var(--space-sm);
}
```

```
.audit-entry {
 padding: var(--space-sm);
 background-color: var(--color-card-muted);
 border-radius: var(--radius-sm);
 transition: background-color 0.2s ease;
}
```

```
.audit-entry:hover {
 background-color: var(--color-table-row-hover);
}
```

```
.audit-entry strong {
 color: var(--color-text-strong);
}
```

```
.audit-entry small {
 color: var(--color-text);
 font-size: var(--font-size-sm);
}
```

```
/* Role-based accent borders */
```

```
.audit-entry.accent { border-left: 4px solid var(--color-accent); }
.audit-entry.success { border-left: 4px solid var(--color-success); }
.audit-entry.info { border-left: 4px solid var(--color-info); }
.audit-entry.danger { border-left: 4px solid var(--color-danger); }
.audit-entry.warning { border-left: 4px solid var(--color-warning); }
```

```
.audit-entry.neutral { border-left: 4px solid var(--color-neutral); }
.audit-entry.youth { border-left: 4px solid var(--color-youth); }
.audit-entry.dilg { border-left: 4px solid var(--color-dilg); }
```

```
` `` `
```

```
frontend\src\styles\dashboard\registry-overview.css
```

```
` `` `css
```

```
/* =====
```

```
 Registry Overview
```

```
===== */
```

```
.registry-overview {
 margin-top: var(--space-md);
}
```

```
.registry-overview h3 {
 font-size: var(--font-size-lg);
 font-weight: 600;
 color: var(--color-accent);
 margin-bottom: var(--space-md);
}
```

```
.registry-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
```

```
gap: var(--space-md);
}
```

```
` ``
```

```
frontend\src\styles\dashboard\role-manager.css
```

```
` `` `css
```

```
/* =====
```

```
 Role Manager
```

```
===== */
```

```
.role-manager {
 background: var(--gradient-card);
 border-radius: var(--radius-md);
 padding: var(--space-md);
 box-shadow: var(--shadow-sm);
 border: 1px solid var(--color-border);
}
```

```
.role-manager h3 {
 font-size: var(--font-size-lg);
 font-weight: 600;
 color: var(--color-accent);
 margin-bottom: var(--space-md);
}
```

```
/* Table */

.role-table {
 width: 100%;
 border-collapse: collapse;
}

.role-table th,
.role-table td {
 padding: var(--space-sm) var(--space-md);
 border-bottom: 1px solid var(--color-border);
 text-align: left;
}

.role-table th {
 font-weight: 600;
 color: var(--color-text-strong);
}

/* Role badges */

.role-badge {
 display: inline-block;
 padding: 0.25rem 0.5rem;
 border-radius: var(--radius-sm);
 font-size: var(--font-size-sm);
 font-weight: 600;
 text-transform: capitalize;
```

```
}
```

```
.role-badge.admin { background: var(--color-admin); color: #fff; }
```

```
.role-badge.staff { background: var(--color-staff); color: #fff; }
```

```
.role-badge.secretary { background: var(--color-secretary); color: #fff; }
```

```
.role-badge.treasurer { background: var(--color-treasurer); color: #fff; }
```

```
.role-badge.sk { background: var(--color-youth); color: #fff; }
```

```
.role-badge.dilg { background: var(--color-dilg); color: #fff; }
```

```
.role-badge.resident { background: var(--color-resident); color: #fff; }
```

```
/* Role presence indicator */
```

```
.presence-indicator {
```

```
 display: inline-flex;
```

```
 align-items: center;
```

```
}
```

```
.presence-dot {
```

```
 width: 12px;
```

```
 height: 12px;
```

```
 border-radius: 50%;
```

```
 display: inline-block;
```

```
 border: 1px solid var(--color-border);
```

```
}
```

```
.presence-dot.online {
```

```
 background: #22c55e;
```

```
}
```

```
.presence-dot.offline {
 background: #9ca3af;
}
```

```
/* Actions */
```

```
select {
 padding: var(--space-xs) var(--space-sm);
 border-radius: var(--radius-sm);
 border: 1px solid var(--color-border);
 background: var(--color-input-bg);
 color: var(--color-text);
}
```

```
.delete-btn {
 padding: var(--space-xs) var(--space-sm);
 border-radius: var(--radius-sm);
 border: none;
 cursor: pointer;
 font-weight: 600;
}
```

```
.delete-btn.danger {
 background: var(--color-danger);
 color: #fff;
```



```
}
```

```
.delete-btn.danger:hover{
 background: var(--color-danger-hover);
}
```

```
`` `
```

```
frontend\src\styles\dashboard\search-filters.css
```

```
`` `css
```

```
/* search-filters.css */
```

```
.search-filters {
 background-color: var(--color-bg);
 padding: 1.5rem;
 margin: 1.5rem 0;
 border-radius: 6px;
 box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

```
.search-filters h3 {
 margin-bottom: 1rem;
 font-size: 1.2rem;
 color: var(--color-success);
}
```

```
.filter-row {
 display: flex;
 align-items: center;
 margin-bottom: 0.75rem;
}
```

```
.filter-row label {
 width: 100px;
 font-weight: 500;
 color: var(--color-text);
}
```

```
.filter-row input,
.filter-row select {
 flex: 1;
 padding: 0.5rem;
 border: 1px solid #d1d5db;
 border-radius: 4px;
 font-size: 0.9rem;
 background: var(--color-bg);
 color: var(--color-text);
}
```

```
.filter-actions {
 display: flex;
```

```
gap: 0.75rem;
margin-top: 1rem;
}
```

```
.filter-actions button {
padding: 0.5rem 1rem;
border: none;
border-radius: 4px;
font-weight: 600;
cursor: pointer;
}
```

```
.filter-actions button[type="submit"] {
background-color: #2563eb;
color: #fff;
}
```

```
.filter-actions .reset-btn {
background-color: #f3f4f6;
color: #374151;
}
```

```
...
```

```
frontend\src\styles\dashboard\summary-card.css
```

```
` `` `css
```

```
/* =====
```

```
 Summary Cards Layout
```

```
===== */
```

```
.summary-cards {
```

```
 display: grid;
```

```
 grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
```

```
 gap: var(--space-md);
```

```
 margin-top: var(--space-md);
```

```
}
```

```
.card-icon {
```

```
 font-size: 1.25rem;
```

```
 margin-right: var(--space-xs);
```

```
 display: inline-block;
```

```
}
```

```
` `` `
```

```
frontend\src\styles\fee-dashboard.css
```

```
` `` `css
```

```
/* Container */
```

```
.fee-dashboard {
```

```
 padding: 2rem;
```

```
font-family: "Segoe UI", Roboto, sans-serif;
color: #333;
}
```

```
/* Heading */
```

```
.fee-dashboard h1 {
 font-size: 1.8rem;
 margin-bottom: 1.5rem;
 color: var(--color-accent);
}
```

```
/* Status messages */
```

```
.fee-dashboard p {
 margin: 0.5rem 0;
}
```

```
.fee-dashboard p.error {
 color: #e74c3c;
 font-weight: bold;
}
```

```
.fee-dashboard p.loading {
 color: #2980b9;
}
```

```
/* Tables */
```

```
.fee-table {
 width: 100%;
 border-collapse: collapse;
 margin-bottom: 2rem;
 background: var(--color-neutral);
 box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

```
.fee-table th,
.fee-table td {
 border: 1px solid #ddd;
 padding: 0.75rem;
 text-align: left;
}
```

```
.fee-table th {
 background: var(--color-table-header);
 font-weight: 600;
}
```

```
.fee-table tr:nth-child(even) {
 background: var(--color-table-row-even);
}
```

```
/* Inputs */
```

```
.fee-table input[type="number"] {
```

```
width: 100px;

padding: 0.25rem;

border: 1px solid var(--color-border);

border-radius: 4px;

}
```

```
/* Buttons */
```

```
.fee-table button {

margin-right: 0.5rem;

padding: 0.4rem 0.8rem;

border: none;

border-radius: 4px;

cursor: pointer;

font-size: 0.9rem;

}
```

```
.fee-table button:first-of-type {

background: #27ae60;

color: #fff;

}
```

```
.fee-table button:last-of-type {

background: #e74c3c;

color: #fff;

}
```

```
.fee-table button:hover {
 opacity: 0.9;
}
```

```
```\
```

```
## frontend\src\styles\mobile-responsive.css
```

```
```css
```

```
/* Global mobile responsiveness overrides */
```

```
* {
 box-sizing: border-box;
}
```

```
html,
body {
 touch-action: manipulation;
}
```

```
img,
svg,
video,
canvas,
iframe {
 max-width: 100%;
```



```
height: auto;
}
```

```
@media (max-width: 1024px) {
 .dashboard-grid,
 .cards-grid,
 .summary-grid,
 .stats-grid,
 .chart-grid,
 .charts-container {
 grid-template-columns: 1fr !important;
 }
}
```

```
.header-row,
.top-row,
.actions-row,
.toolbar,
.filter-row {
 flex-wrap: wrap;
 gap: 0.5rem;
}
}
```

```
@media (max-width: 768px) {
 html,
 body,
```

```
#root,
.app-wrapper,
.main-content,
.dashboard-content,
.content,
main {
 min-height: 100%;
 width: 100%;
 overflow-x: hidden;
}
```

```
[class*="-page"],
[class*="-dashboard"],
.container,
.content-wrapper,
.documents-admin-page,
.dashboard,
.section,
.card,
.panel,
.form-card {
 width: 100%;
 max-width: 100%;
}
```

```
[class*="-page"],
```

```
[class*="-dashboard"],
.container,
.content-wrapper,
.documents-admin-page,
.dashboard,
.section,
.panel,
.form-card {
 padding-left: 0.75rem !important;
 padding-right: 0.75rem !important;
}
```

```
h1 {
 font-size: 1.4rem !important;
 line-height: 1.3;
}
```

```
h2 {
 font-size: 1.2rem !important;
 line-height: 1.3;
}
```

```
h3 {
 font-size: 1.05rem !important;
 line-height: 1.3;
}
```

```
p,
li,
label,
input,
select,
textarea,
button,
a {
 font-size: 0.95rem;
}
```

```
input,
select,
textarea {
 font-size: 16px !important;
}
```

```
input,
select,
textarea,
.form-control,
.search-input {
 width: 100% !important;
 max-width: 100%;
}
```

```
button,
.btn,
[class*="btn-"]{
 min-height: 40px;
 width: 100%;
}
```

```
.header-row .issue-document-btn,
.header-row button,
.request-card .btn-view,
.doc-actions button,
.payment-options .btn-pay {
 width: 100% !important;
}
```

```
.tabs {
 display: grid;
 grid-template-columns: 1fr;
 gap: 0.5rem;
}
```

```
.tabs button {
 width: 100%;
}
```

```
.resident-filter,
.search-filters,
.filter-group,
.payment-options,
.doc-actions,
.request-info,
.request-card {
 flex-direction: column !important;
 align-items: stretch !important;
}
```

```
.request-card,
.doc-card,
.document-item,
.audit-card,
.notification-item {
 padding: 0.75rem !important;
}
```

```
table,
.document-table,
.audit-table table,
.table {
 display: block;
 width: 100%;
 overflow-x: auto;
```

```
white-space: nowrap;
}
```

```
table thead,
.document-table thead,
.audit-table table thead {
 display: table-header-group;
}
```

```
table tbody,
.document-table tbody,
.audit-table table tbody {
 display: table-row-group;
}
```

```
.modal-overlay {
 padding: 0.75rem;
 align-items: flex-start;
}
```

```
.modal-content {
 width: 100% !important;
 max-width: 100% !important;
 max-height: 90vh !important;
 margin-top: 0.5rem;
 border-radius: 10px;
```

```
}
}
```

```
@media (max-width: 480px) {
 [class*="-page"],
 [class*="-dashboard"],
 .container,
 .content-wrapper,
 .documents-admin-page,
 .dashboard,
 .section,
 .panel,
 .form-card {
 padding-left: 0.5rem !important;
 padding-right: 0.5rem !important;
 }
```

```
 .status-message,
 .doc-meta p,
 .request-info span,
 .request-info small {
 font-size: 0.85rem !important;
 }
```

```
 .doc-id {
 word-break: break-word;
```



```
}
}
```

```
```
```

```
## frontend\src\styles\notification-bell.css
```

```
```css
```

```
/* notificationBell.css */
```

```
/* .notification-bell {
```

```
 position: relative;
```

```
 display: inline-block;
```

```
} */
```

```
.bell-button {
```

```
 background: none;
```

```
 border: none;
```

```
 font-size: 1.5rem;
```

```
 position: relative;
```

```
 cursor: pointer;
```

```
}
```

```
.badge {
```

```
 position: absolute;
```

```
 top: -5px;
```

```
 right: -5px;
```

```
background: red;
color: white;
border-radius: 50%;
padding: 2px 6px;
font-size: 0.8rem;
}
```

```
.dropdown {
position: absolute;
right: 0;
margin-top: 10px;
background: white;
border: 1px solid #ddd;
box-shadow: 0px 2px 6px rgba(0,0,0,0.2);
width: 250px;
overflow-y: auto;
}
```

```
.notification-item {
padding: 8px;
/* border-bottom: 1px solid #ddd; */
}

.notification-item.unread {
font-weight: bold;
background-color: #f9f9f9;
```

```
}
.notification-item.read {
 color: #666;
}
.actions {
 margin-top: 4px;
}
.actions button {
 margin-right: 6px;
 font-size: 0.8rem;
}
```

```
`` `
```

```
frontend\src\styles\payment-success-page.css
```

```
`` `css
```

```
.payment-success-container {
 height: 100vh;
 display: flex;
 justify-content: center;
 align-items: center;
 background-color: #f5f5f5;
}
```

```
.payment-success-card {
 background-color: #ffffff;
 padding: 40px;
 border-radius: 12px;
 box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
 text-align: center;
 max-width: 400px;
}
```

```
.payment-success-icon {
 font-size: 48px;
 color: #28a745;
 margin-bottom: 20px;
}
```

```
.payment-success-title {
 font-size: 20px;
 font-weight: 600;
 margin-bottom: 10px;
 color: #333;
}
```

```
.payment-success-subtitle {
 font-size: 16px;
 color: #666;
}
```

```

frontend\src\styles\reset-password.css

```css

```
.reset-form {
 max-width: 400px;
 margin: 3rem auto;
 padding: 2rem;
 background: #ffffff;
 border-radius: 8px;
 box-shadow: 0 4px 12px rgba(0,0,0,0.1);
 font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}
```

```
.reset-form h2 {
 margin-bottom: 1rem;
 font-size: 1.5rem;
 text-align: center;
 color: #333;
}
```

```
.reset-form p {
 margin-bottom: 1rem;
 text-align: center;
```

```
color: #555;
}
```

```
.reset-form label {
 display: block;
 margin-top: 1rem;
 margin-bottom: 0.5rem;
 font-weight: 600;
 color: #444;
}
```

```
.reset-form input {
 width: 100%;
 padding: 0.75rem;
 border: 1px solid #ccc;
 border-radius: 6px;
 font-size: 1rem;
 transition: border-color 0.2s ease, box-shadow 0.2s ease;
}
```

```
.reset-form input:focus {
 border-color: #007bff;
 box-shadow: 0 0 0 3px rgba(0,123,255,0.2);
 outline: none;
}
```

```
.reset-form button {
 margin-top: 1.5rem;
 width: 100%;
 padding: 0.75rem;
 background: #007bff;
 color: #fff;
 font-size: 1rem;
 font-weight: 600;
 border: none;
 border-radius: 6px;
 cursor: pointer;
 transition: background 0.2s ease;
}
```

```
.reset-form button:hover:not(:disabled) {
 background: #0056b3;
}
```

```
.reset-form button:disabled {
 background: #ccc;
 cursor: not-allowed;
}
```

```
.reset-form strong {
 color: #007bff;
}
```

```

frontend\src\styles\resident.css

```css

/\* =====

 Resident Dashboard Layout

===== \*/

.resident-dashboard {

padding: 2rem;

}

.resident-dashboard .resident-actions {

display: grid;

grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));

gap: 2rem;

margin-top: 2rem;

}

/\* Add more resident-specific tools or forms here \*/

```

frontend\src\styles\resident\my-documents.css


```
` `` `css
```

```
/* Container */
```

```
.my-documents {
```

```
padding: 24px;
```

```
background-color: var(--color-bg);
```

```
border-radius: 12px;
```

```
font-family: 'Segoe UI', sans-serif;
```

```
}
```

```
/* Header */
```

```
.my-documents h3 {
```

```
font-size: 1.4rem;
```

```
margin-bottom: 20px;
```

```
color: var(--color-accent);
```

```
font-weight: 600;
```

```
}
```

```
/* Tabs */
```

```
.tabs {
```

```
display: flex;
```

```
gap: 12px;
```

```
margin-bottom: 16px;
```

```
flex-wrap: wrap;
```

```
}
```

```
.tabs button {
```

```
padding: 10px 18px;
border: 2px solid #dce3ea;
border-radius: 8px;
background: #ffffff;
color: #34495e;
font-size: 1rem;
font-weight: 500;
cursor: pointer;
transition: all 0.2s ease;
box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}
```

```
.tabs button:hover {
  background-color: #ecf0f1;
  border-color: #bfcabd9;
}
```

```
.tabs button.active {
  background-color: #3498db;
  color: #fff;
  border-color: #3498db;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
}
```

```
/* Document list */
```

```
.doc-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.doc-card {  
  background: #ffffff;  
  border: 1px solid #dce3ea;  
  border-radius: 10px;  
  padding: 16px 20px;  
  margin-bottom: 14px;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);  
  transition: box-shadow 0.2s ease;  
}
```

```
.doc-card:hover {  
  box-shadow: 0 4px 10px rgba(0,0,0,0.08);  
}
```

```
/* Document info */
```

```
.doc-info {  
  display: flex;  
  flex-direction: column;  
  gap: 4px;  
}
```

```
.doc-info strong {  
  font-size: 1.05rem;  
  color: #2c3e50;  
  font-weight: 500;  
}
```

```
.doc-info span {  
  font-size: 0.9rem;  
  color: #7f8c8d;  
}
```

```
/* Remarks */
```

```
.doc-remarks {  
  margin-top: 8px;  
  font-size: 0.9rem;  
  color: #c0392b; /* red tone for emphasis */  
  font-style: italic;  
}
```

```
/* Status badge styles */
```

```
.status-badge {  
  display: inline-block;  
  padding: 4px 10px;  
  border-radius: 12px;  
  font-size: 0.8rem;
```

```
font-weight: 600;
margin-top: 6px;
width: fit-content;
}
```

```
/* Status colors */
```

```
.status-pending{
  background-color: #f1c40f;
  color: #fff;
}
```

```
.status-awaiting_payment {
  background-color: #e67e22;
  color: #fff;
}
```

```
.status-paid {
  background-color: var(--color-success);
  color: #fff;
}
```

```
.status-approved {
  background-color: #2ecc71;
  color: #fff;
}
```

```
.status-rejected {  
  background-color: #e74c3c;  
  color: #fff;  
}
```

```
/* Resubmit button */
```

```
.btn-resubmit {  
  margin-top: 10px;  
  padding: 8px 14px;  
  background-color: #3498db;  
  color: #fff;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  font-size: 0.9rem;  
  transition: background 0.2s ease;  
}
```

```
.btn-resubmit:hover {  
  background-color: #2980b9;  
}
```

```
.btn-pay {  
  background-color: #27ae60;  
  color: #fff;  
  border: none;
```

```
padding: 8px 14px;
border-radius: 6px;
cursor: pointer;
transition: background 0.2s ease;
}
```

```
.btn-pay:hover {
  background-color: #219150;
}
```

```
/* Responsive tweaks */
```

```
@media (max-width: 600px) {
  .doc-card {
    padding: 14px;
  }
}
```

```
.doc-info strong {
  font-size: 1rem;
}
```

```
.doc-info span {
  font-size: 0.85rem;
}
```

```
.tabs {
  flex-direction: column;
  gap: 8px;
}
```

```
}  
}
```

```
`` `
```

```
## frontend\src\styles\resident\resident-business-dashboard.css
```

```
`` `css
```

```
/* Dashboard container */
```

```
.resident-business-dashboard {
```

```
  max-width: 1200px;
```

```
  margin: 0 auto;
```

```
  padding: 2rem;
```

```
  font-family: "Segoe UI", Arial, sans-serif;
```

```
  /* color: var(--color-text); */
```

```
}
```

```
/* Tabs */
```

```
.tabs {
```

```
  display: flex;
```

```
  gap: 1rem;
```

```
  margin-bottom: 1.5rem;
```

```
}
```

```
.tabs button {
```

```
  padding: 0.6rem 1.2rem;
```



```
border: none;

border-radius: 6px;

background: #f0f0f0;

cursor: pointer;

font-weight: 500;

transition: background 0.2s ease;

}
```

```
.tabs button:hover {

    background: #e0e0e0;

}
```

```
.tabs button.active {

    background: #4caf50;

    color: #fff;

}
```

```
/* Business grid */

.business-grid {

    display: grid;

    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));

    gap: 1.5rem;

}
```

```
/* Base card */

.business-card {
```

```
background: #fff;
border: 1px solid #ddd;
border-radius: 10px;
padding: 1rem;
box-shadow: 0 2px 6px rgba(0,0,0,0.1);
transition: transform 0.2s ease, box-shadow 0.2s ease;
position: relative;
}
```

```
.business-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
}
```

```
.business-card h3 {
  margin-top: 0;
  margin-bottom: 0.5rem;
}
```

/ Approved card variant */*

```
.approved-card {
  border: 2px solid #4caf50;
  background: var(--color-bg);
  box-shadow: 0 4px 12px rgba(76, 175, 80, 0.3);
}
```

```
.approved-card h3 {  
  color: #2e7d32;  
}
```

```
/* Approved badge */
```

```
.approved-badge {  
  position: absolute;  
  top: -10px;  
  right: -10px;  
  background: #4caf50;  
  color: #fff;  
  font-size: 0.8rem;  
  font-weight: bold;  
  padding: 4px 8px;  
  border-radius: 4px;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.2);  
}
```

```
/* Documents list */
```

```
.documents-list {  
  margin-top: 0.5rem;  
}
```

```
.documents-list a {  
  color: #1976d2;  
  text-decoration: none;
```

```
}
```

```
.documents-list a:hover {  
  text-decoration: underline;  
}
```

```
/* Status text styles */  
.pending-text {  
  color: #ff9800;  
  font-weight: 500;  
}
```

```
.info-text {  
  color: #2196f3;  
}
```

```
.warning-text {  
  color: #f44336;  
  font-weight: 500;  
}
```

```
.payment-section {  
  margin-top: 1rem;  
}
```

```
.card-section {
```

```
margin-top: 0.75rem;
padding-top: 0.75rem;
border-top: 1px solid #eee;
}
```

```
.qr-wrapper {
margin-top: 1rem;
text-align: center;
}
```

```
.qr-wrapper p {
margin: 0.25rem 0;
}
```

```
.qr-wrapper strong {
color: #2e7d32;
}
```

```
` ``
```

```
## frontend\src\styles\resident\resident-document-form.css
```

```
` `` `css
```

```
.resident-document-form {
max-width: 600px;
margin: 2rem auto;
```

```
padding: 2rem;
background: var(--color-neutral);
border-radius: 8px;
box-shadow: 0 4px 12px rgba(0,0,0,0.1);
font-family: "Segoe UI", Arial, sans-serif;
}
```

```
.resident-document-form h2 {
margin-bottom: 1.5rem;
font-size: 1.5rem;
color: #333;
text-align: center;
}
```

```
.form-group {
margin-bottom: 1.2rem;
}
```

```
.form-group label {
display: block;
font-weight: 600;
margin-bottom: 0.5rem;
color: #444;
}
```

```
.form-group input,
```

```
.form-group select,  
.form-group textarea {  
  width: 100%;  
  padding: 0.6rem;  
  border: 1px solid #ccc;  
  border-radius: 6px;  
  font-size: 0.95rem;  
  transition: border-color 0.2s ease;  
}
```

```
.form-group input:focus,  
.form-group select:focus,  
.form-group textarea:focus {  
  border-color: #0078d7;  
  outline: none;  
}
```

```
fieldset {  
  border: 1px solid #ddd;  
  border-radius: 6px;  
  padding: 1rem;  
  margin-bottom: 1.2rem;  
}
```

```
legend {  
  font-weight: 600;
```

```
color: #555;  
padding: 0 0.5rem;  
}
```

```
.submit-btn {  
  display: block;  
  width: 100%;  
  padding: 0.8rem;  
  background: #0078d7;  
  color: #fff;  
  font-size: 1rem;  
  font-weight: 600;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  transition: background 0.2s ease;  
}
```

```
.submit-btn:hover {  
  background: #005fa3;  
}
```

```
.status-message {  
  margin-top: 1rem;  
  padding: 0.8rem;  
  border-radius: 6px;
```



```
font-size: 0.95rem;  
text-align: center;  
}
```

```
.status-message.success {  
  background: #e6f4ea;  
  color: #2e7d32;  
  border: 1px solid #c8e6c9;  
}
```

```
.status-message.error {  
  background: #fdecea;  
  color: #c62828;  
  border: 1px solid #f5c6cb;  
}
```

```
.status-message.loading {  
  background: #fff3cd;  
  color: #856404;  
  border: 1px solid #ffeeba;  
}
```

```
.required {  
  color: #c62828;  
  font-weight: bold;  
}
```

```

## frontend\src\styles\resident\resubmission-form.css

```css

```
.resubmission-form {  
  padding: 24px;  
  background: #ffffff;  
  border: 1px solid #dce3ea;  
  border-radius: 12px;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);  
}
```

```
.resubmission-form h3 {  
  font-size: 1.3rem;  
  margin-bottom: 20px;  
  color: #2c3e50;  
}
```

```
.form-group {  
  margin-bottom: 16px;  
  display: flex;  
  flex-direction: column;  
}
```

```
.form-group label {  
  font-size: 0.95rem;  
  font-weight: 500;  
  margin-bottom: 6px;  
  color: #34495e;  
}
```

```
.form-group input,  
.form-group textarea {  
  padding: 10px;  
  border: 1px solid #ccd6dd;  
  border-radius: 6px;  
  font-size: 0.9rem;  
}
```

```
.form-group textarea {  
  min-height: 80px;  
  resize: vertical;  
}
```

```
.btn-submit {  
  padding: 10px 16px;  
  background-color: #3498db;  
  color: #fff;  
  border: none;  
  border-radius: 6px;
```

```
    cursor: pointer;

    font-size: 0.95rem;

    transition: background 0.2s ease;
}

```

```
.btn-submit:hover {

    background-color: #2980b9;
}

```

```
.success-message {

    color: #27ae60;

    font-weight: 600;
}

```

```
...

```

```
## frontend\src\styles\secretary.css

```

```
` `` `css

```

```
/* ===== Dashboard Layout ===== */

```

```
.dashboard.secretary-dashboard {

    padding: 1.5rem;

    background-color: #f7f9fc;

    min-height: 100vh;

    font-family: "Segoe UI", Roboto, sans-serif;
}

```

```
.dashboard.secretary-dashboard header {  
  margin-bottom: 1.5rem;  
}
```

```
.dashboard.secretary-dashboard header h2 {  
  font-size: 1.6rem;  
  margin: 0;  
  color: #2c3e50;  
}
```

```
.dashboard.secretary-dashboard header p {  
  font-size: 0.95rem;  
  color: #7f8c8d;  
  margin-top: 0.25rem;  
}
```

```
/* ===== Summary Cards ===== */
```

```
.summary-cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));  
  gap: 1rem;  
  margin-bottom: 2rem;  
}
```

```
.summary-card {
```

```
background: #fff;
border-radius: 8px;
padding: 1rem;
box-shadow: 0 2px 6px rgba(0,0,0,0.08);
text-align: center;
}
```

```
.summary-card h4 {
margin: 0;
font-size: 1rem;
color: #34495e;
}
```

```
.summary-card p {
margin: 0.25rem 0 0;
font-size: 1.2rem;
font-weight: bold;
color: #2980b9;
}
```

```
/* ===== Charts Container ===== */
.charts-container {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
gap: 1.5rem;
margin-top: 1.5rem;
```

```
}
```

```
/* ===== Chart Section ===== */
```

```
.chart-section {  
  background: #fff;  
  border-radius: 8px;  
  padding: 1rem;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

```
.chart-section h3 {  
  font-size: 1.1rem;  
  margin-bottom: 0.75rem;  
  color: #2c3e50;  
}
```

```
/* ===== Chart Canvas Sizing ===== */
```

```
.chart-section canvas {  
  width: 100% !important;  
  height: auto !important;  
  min-width: 0; /* prevent mobile horizontal clipping */  
  min-height: 300px; /* bigger minimum height */  
  max-width: 800px; /* allow larger charts on desktop */
```

```
    max-height: 500px; /* more room for labels and legends */  
}
```

```
/* Large screens */
```

```
@media (min-width: 1024px) {  
    .chart-section canvas {  
        max-width: 900px;  
        max-height: 550px;  
    }  
}
```

```
/* Medium screens (tablets) */
```

```
@media (max-width: 900px) {  
    .chart-section canvas {  
        max-width: 700px;  
        max-height: 450px;  
    }  
}
```

```
/* Small screens (phones) */
```

```
@media (max-width: 600px) {  
    .dashboard.secretary-dashboard {  
        padding: 1rem;  
    }  
}
```

```
.chart-section canvas {
```



```
    max-width: 100%;  
    max-height: 320px; /* still bigger than before */  
  }  
}
```

```
```
```

```
frontend\src\styles\secretary\issued-documents.css
```

```
```css
```

```
/* Container */
```

```
.issued-documents {  
  padding: 2rem;  
  font-family: "Segoe UI", Arial, sans-serif;  
  background-color: var(--color-bg);  
  min-height: 100vh;  
}
```

```
/* Heading */
```

```
.issued-documents h3 {  
  margin-bottom: 1.5rem;  
  font-size: 1.6rem;  
  color: var(--color-accent);  
}
```

```
/* Status message */
```

```
.status-message {  
  font-size: 0.95rem;  
  color: #555;  
  margin: 1rem 0;  
}
```

```
/* Request list */
```

```
.request-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.request-card {  
  background-color: var(--color-card-bg);  
  border: 1px solid var(--color-border);  
  border-radius: 8px;  
  padding: 1rem;  
  margin-bottom: 1rem;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

```
.request-info strong {
```

```
display: block;

font-size: 1.1rem;

font-weight: bold;

color: var(--color-success);
}
```

```
.request-info span {

display: block;

font-size: 0.9rem;

color: var(--color-text);
}
```

/* View button */

```
.btn-view {

padding: 0.4rem 0.8rem;

background-color: #0078d4;

color: #fff;

border: none;

border-radius: 4px;

font-size: 0.85rem;

cursor: pointer;

transition: background-color 0.2s ease;
}
```

```
.btn-view:hover {

background-color: #005a9e;
}
```

```
}
```

```
/* Modal overlay */
```

```
.modal-overlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  right: 0;  
  bottom: 0;  
  background-color: rgba(0,0,0,0.5);  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  z-index: 999;  
}
```

```
/* Modal content */
```

```
.modal-content {  
  background-color: #fff;  
  border-radius: 8px;  
  padding: 1.5rem;  
  width: 500px;  
  max-width: 90%;  
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);  
}
```

```
.modal-content header h4 {  
  margin: 0;  
  font-size: 1.2rem;  
  color: #333;  
}
```

```
.modal-content .doc-type {  
  font-size: 0.95rem;  
  color: #555;  
  margin-top: 0.25rem;  
}
```

```
/* Meta info */
```

```
.doc-meta p {  
  margin: 0.5rem 0;  
  font-size: 0.9rem;  
  color: #444;  
}
```

```
/* File section */
```

```
.doc-file h5 {  
  margin-top: 1rem;  
  font-size: 1rem;  
  color: #333;  
}
```

```
.doc-file a {  
  display: inline-block;  
  margin-top: 0.5rem;  
  padding: 0.4rem 0.8rem;  
  background-color: #28a745;  
  color: #fff;  
  border-radius: 4px;  
  font-size: 0.85rem;  
  text-decoration: none;  
  transition: background-color 0.2s ease;  
}
```

```
.doc-file a:hover {  
  background-color: #218838;  
}
```

/* Close button */

```
.btn-close {  
  margin-top: 1rem;  
  padding: 0.4rem 0.8rem;  
  background-color: #6c757d;  
  color: #fff;  
  border: none;  
  border-radius: 4px;  
  font-size: 0.85rem;  
  cursor: pointer;
```

```
    transition: background-color 0.2s ease;
}
```

```
.btn-close:hover {
    background-color: #5a6268;
}
```

```
/* Responsive */
```

```
@media (max-width: 600px) {
    .modal-content {
        width: 95%;
        padding: 1rem;
    }
}
```

```
`` `
```

```
## frontend\src\styles\secretary\paid-requests.css
```

```
`` `css
```

```
/* Container */
```

```
.paid-requests {
    padding: 2rem;
    font-family: "Segoe UI", Arial, sans-serif;
    background-color: var(--color-bg);
    min-height: 100vh;
```

```
}
```

```
/* Heading */
```

```
.paid-requests h3 {  
  margin-bottom: 1.5rem;  
  font-size: 1.6rem;  
  color: var(--color-accent);  
}
```

```
/* Status message */
```

```
.status-message {  
  font-size: 0.95rem;  
  color: var(--color-text);  
  margin: 1rem 0;  
}
```

```
/* Request list */
```

```
.request-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.request-card {  
  background-color: #fff;  
  border: 1px solid #e5e7eb;
```



```
border-radius: 8px;
padding: 1rem;
margin-bottom: 1rem;
box-shadow: 0 2px 4px rgba(0,0,0,0.05);
display: flex;
justify-content: space-between;
align-items: center;
}
```

```
.request-info strong {
display: block;
font-size: 1.1rem;
font-weight: bold;
color: #0078d4;
}
```

```
.request-info span {
display: block;
font-size: 0.9rem;
color: #555;
}
```

```
/* View button */
```

```
.btn-view {
padding: 0.4rem 0.8rem;
background-color: #0078d4;
```

```
color: #fff;

border: none;

border-radius: 4px;

font-size: 0.85rem;

cursor: pointer;

transition: background-color 0.2s ease;

}
```

```
.btn-view:hover{

background-color: #005a9e;

}
```

```
/* Modal overlay */

.modal-overlay{

position: fixed;

top: 0;

left: 0;

right: 0;

bottom: 0;

background-color: rgba(0,0,0,0.5);

display: flex;

justify-content: center;

align-items: center;

z-index: 999;

}
```

```
/* Modal content */  
  
.modal-content {  
  background-color: #fff;  
  border-radius: 8px;  
  padding: 1.5rem;  
  width: 500px;  
  max-width: 90%;  
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);  
}
```

```
.modal-content header h4 {  
  margin: 0;  
  font-size: 1.2rem;  
  color: #333;  
}
```

```
.modal-content .doc-type {  
  font-size: 0.95rem;  
  color: #555;  
  margin-top: 0.25rem;  
}
```

```
/* Meta info */  
  
.doc-meta p {  
  margin: 0.5rem 0;  
  font-size: 0.9rem;
```

```
color: #444;  
}
```

```
/* Attachments */
```

```
.doc-attachments h5 {  
  margin-top: 1rem;  
  font-size: 1rem;  
  color: #333;  
}
```

```
.doc-attachments ul {  
  list-style: none;  
  padding: 0;  
}
```

```
.doc-attachments li {  
  font-size: 0.85rem;  
  margin: 0.3rem 0;  
}
```

```
/* Actions */
```

```
.doc-actions {  
  margin-top: 1.5rem;  
  display: flex;  
  flex-direction: column;  
  gap: 0.8rem;
```

```
}
```

```
.btn-approve {  
  padding: 0.5rem 1rem;  
  background-color: #28a745;  
  color: #fff;  
  border: none;  
  border-radius: 4px;  
  font-size: 0.9rem;  
  cursor: pointer;  
  transition: background-color 0.2s ease;  
}
```

```
.btn-approve:hover {  
  background-color: #218838;  
}
```

```
.reject-section {  
  display: flex;  
  flex-direction: column;  
  gap: 0.5rem;  
}
```

```
.reject-textarea {  
  width: 100%;  
  min-height: 60px;
```

```
padding: 0.5rem;  
font-size: 0.85rem;  
border: 1px solid #ccc;  
border-radius: 4px;  
}
```

```
.btn-reject {  
padding: 0.5rem 1rem;  
background-color: #dc3545;  
color: #fff;  
border: none;  
border-radius: 4px;  
font-size: 0.9rem;  
cursor: pointer;  
transition: background-color 0.2s ease;  
}
```

```
.btn-reject:hover {  
background-color: #c82333;  
}
```

```
.btn-close {  
padding: 0.4rem 0.8rem;  
background-color: #6c757d;  
color: #fff;  
border: none;
```

```
border-radius: 4px;
font-size: 0.85rem;
cursor: pointer;
transition: background-color 0.2s ease;
}
```

```
.btn-close:hover {
background-color: #5a6268;
}
```

```
/* Responsive */
@media (max-width: 600px) {
.modal-content {
width: 95%;
padding: 1rem;
}
}
```

```
```
```

```
frontend\src\styles\secretary\pending-requests.css
```

```
```css
```

```
/* =====
Pending Requests Styles
===== */
```

```
.pending-requests {  
  padding: 1.5rem;  
  font-family: Arial, sans-serif;  
  color: #333;  
}
```

```
.pending-requests h3 {  
  margin-bottom: 1rem;  
  font-size: 1.4rem;  
  font-weight: bold;  
  display: flex;  
  align-items: center;  
  gap: 0.5rem;  
}
```

```
.status-message {  
  font-style: italic;  
  color: #666;  
  margin: 0.5rem 0;  
}
```

```
/* =====  
  
Request List  
  
===== */
```



```
.request-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.request-card {  
  background: #f9f9f9;  
  border: 1px solid #ddd;  
  border-radius: 6px;  
  padding: 0.75rem 1rem;  
  margin-bottom: 0.75rem;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

```
.request-info strong {  
  display: block;  
  font-size: 1rem;  
  margin-bottom: 0.25rem;  
}
```

```
.request-info span {  
  font-size: 0.9rem;  
  color: #555;
```

```
}
```

```
.btn-view {  
  background: #007bff;  
  color: #fff;  
  border: none;  
  padding: 0.4rem 0.8rem;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.btn-view:hover {  
  background: #0056b3;  
}
```

```
/* =====  
Document Panel  
===== */
```

```
.doc-panel {  
  background: #fff;  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  margin-top: 1rem;  
  padding: 1rem;  
}
```

```
.doc-panel header {  
  border-bottom: 1px solid #eee;  
  margin-bottom: 0.75rem;  
}
```

```
.doc-panel h4 {  
  margin: 0;  
  font-size: 1.2rem;  
}
```

```
.doc-type {  
  font-size: 0.95rem;  
  color: #777;  
}
```

```
/* =====  
Document Meta & Attachments  
===== */
```

```
.doc-meta p {  
  margin: 0.25rem 0;  
}
```

```
.doc-attachments h5 {  
  margin-top: 0.75rem;
```

```
margin-bottom: 0.5rem;
}
```

```
.doc-attachments ul {
  list-style: none;
  padding: 0;
}
```

```
.doc-attachments li {
  margin-bottom: 0.25rem;
}
```

```
.doc-attachments a {
  color: #007bff;
  text-decoration: none;
}
```

```
.doc-attachments a:hover {
  text-decoration: underline;
}
```

```
/* =====
Actions
===== */
```

```
.doc-actions {
```

```
margin-top: 1rem;  
display: flex;  
flex-direction: column;  
gap: 0.75rem;  
}
```

```
.btn-awaiting {  
  background: #ffc107;  
  color: #000;  
  border: none;  
  padding: 0.5rem 0.9rem;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.btn-awaiting:hover {  
  background: #e0a800;  
}
```

```
.reject-section {  
  display: flex;  
  flex-direction: column;  
  gap: 0.5rem;  
}
```

```
.reject-textarea {
```

```
width: 100%;  
min-height: 60px;  
padding: 0.5rem;  
border: 1px solid #ccc;  
border-radius: 4px;  
}
```

```
.btn-reject {  
  background: #dc3545;  
  color: #fff;  
  border: none;  
  padding: 0.5rem 0.9rem;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.btn-reject:hover {  
  background: #a71d2a;  
}
```

```
.btn-close {  
  background: #6c757d;  
  color: #fff;  
  border: none;  
  padding: 0.5rem 0.9rem;  
  border-radius: 4px;
```

```
    cursor: pointer;
}
```

```
.btn-close:hover {
    background: #5a6268;
}
```

```
.modal-overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0,0,0,0.6);
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 999;
}
```

```
.modal-content {
    background: #fff;
    border-radius: 8px;
    padding: 1rem;
    width: 90%;
    max-width: 500px;
```

```
max-height: 80vh;
overflow-y: auto;
box-shadow: 0 4px 12px rgba(0,0,0,0.3);
}
```

```
` ``
```

```
## frontend\src\styles\secretary\secretary-document-form.css
```

```
` `` `css
```

```
/* secretary-document-form.css */
```

```
.secretary-document-form {
max-width: 600px;
margin: 2rem auto;
padding: 1.5rem;
background: var(--color-bg);
border-radius: 8px;
box-shadow: 0 2px 8px rgba(0,0,0,0.1);
font-family: "Segoe UI", Arial, sans-serif;
}
```

```
.secretary-document-form h2 {
font-size: 1.4rem;
margin-bottom: 1rem;
color: #333;
```



```
text-align: center;
}
```

```
.form-group {
  margin-bottom: 1rem;
}
```

```
.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.4rem;
  color: var(--color-text);
}
```

```
.form-group input,
.form-group select,
.form-group textarea {
  width: 100%;
  padding: 0.6rem;
  border: 1px solid var(--color-success);
  border-radius: 4px;
  font-size: 0.95rem;
  transition: border-color 0.2s ease;
  color: var(--color-text);
}
```

```
.form-group input:focus,  
.form-group select:focus,  
.form-group textarea:focus {  
  border-color: #0078d4;  
  outline: none;  
}
```

```
textarea {  
  min-height: 80px;  
  resize: vertical;  
}
```

```
.submit-btn {  
  display: inline-block;  
  width: 100%;  
  padding: 0.8rem;  
  background-color: #0078d4;  
  color: #fff;  
  font-size: 1rem;  
  font-weight: 600;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
  transition: background 0.2s ease;  
}
```

```
.submit-btn:hover {  
  background-color: #005a9e;  
}
```

```
.status-message {  
  margin-top: 1rem;  
  padding: 0.6rem;  
  border-radius: 4px;  
  font-size: 0.9rem;  
  text-align: center;  
}
```

```
.status-message.success {  
  background-color: #e6f4ea;  
  color: #2e7d32;  
}
```

```
.status-message.error {  
  background-color: #fdecea;  
  color: #c62828;  
}
```

```
.status-message.loading {  
  background-color: #e3f2fd;  
  color: #1565c0;  
}
```

```
.download-section {  
    margin-top: 1rem;  
    text-align: center;  
}
```

```
.download-btn,  
.print-btn,  
.back-btn {  
    display: inline-block;  
    margin: 0 8px;  
    color: white;  
    padding: 12px 20px;  
    font-size: 16px;  
    font-weight: bold;  
    text-decoration: none;  
    border-radius: 6px;  
    transition: background-color 0.3s ease;  
    cursor: pointer;  
}
```

```
/* Download button */
```

```
.download-btn {  
    background-color: #4CAF50; /* green */  
    color: white;  
}
```

```
.download-btn:hover {  
    background-color: #45a049;  
}
```

```
.download-btn:active {  
    background-color: #3e8e41;  
}
```

```
.print-btn {  
    background-color: #2196F3; /* blue for print */  
    color: white;  
    border: none;  
}
```

```
.print-btn:hover {  
    background-color: #1976D2;  
}
```

/* Back button */

```
.back-btn {  
    background-color: #f44336; /* red */  
    color: white;  
    border: none;  
}
```

```
.back-btn:hover {  
  background-color: #d32f2f;  
}
```

```
.back-btn:active {  
  background-color: #b71c1c;  
}
```

```
...
```

```
## frontend\src\styles\secretary\summary-cards.css
```

```
```css
```

```
.summary-cards {
 display: flex;
 gap: 20px;
 margin: 20px 0;
}
```

```
.card {
 flex: 1;
 background-color: #fff;
 border-radius: 8px;
 padding: 20px;
 text-align: center;
 box-shadow: 0 2px 6px rgba(0,0,0,0.1);
```

```
}
```

```
.card h3 {
 font-size: 2rem;
 margin: 0;
 color: #2c3e50;
}
```

```
.card p {
 margin-top: 8px;
 font-size: 0.9rem;
 color: var(--color-text-muted);
}
```

```
...
```

```
frontend\src\styles\sk.css
```

```
` `` `css
```

```
/* =====
```

```
 SK Dashboard Layout
```

```
===== */
```

```
.sk-dashboard {
 padding: 2rem;
}
```

```
.sk-dashboard .tool-section {
 background-color: var(--card-bg);
 padding: 1rem;
 border-radius: 6px;
 border: 1px solid var(--border);
}
```

```
.sk-dashboard .tool-section h3 {
 margin-bottom: 1rem;
 font-size: 1.2rem;
 color: var(--accent);
}
```

```
.sk-dashboard .sk-tools {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
 gap: 2rem;
 margin-top: 2rem;
}
```

```
/* =====
```

```
 Youth Registry & Programs
```

```
===== */
```

```
.youth-registry table {
 width: 100%;
 margin-top: 1rem;
```



```
border-collapse: collapse;
}
```

```
.youth-registry th,
.youth-registry td {
padding: 0.75rem;
border: 1px solid var(--border);
text-align: left;
}
```

```
.program-list ul,
.event-calendar ul {
list-style: none;
padding: 0;
margin-top: 1rem;
}
```

```
.program-list li,
.event-calendar li {
padding: 0.5rem 0;
border-bottom: 1px solid var(--border);
}
```

```
/* =====
```

 Youth Feedback Form

```
===== */
```

```
.youth-feedback-form textarea {
 width: 100%;
 padding: 0.75rem;
 margin-bottom: 1rem;
 border-radius: 4px;
 border: 1px solid var(--border);
 resize: vertical;
}
```

```
.youth-feedback-form button {
 background-color: var(--accent);
 color: #fff;
 border: none;
 padding: 0.5rem 1rem;
 border-radius: 4px;
 cursor: pointer;
}
```

```
.youth-feedback-form button:hover {
 background-color: var(--accent-hover);
}
```

```
...
```

```
frontend\src\styles\staff.css
```

```

` `` `css

/* =====

👤 Staff Dashboard Layout

===== */

.staff-dashboard {
 padding: 2rem;
}

.staff-dashboard .staff-tools {
 display: flex;
 flex-wrap: wrap;
 gap: 2rem;
 margin-top: 2rem;
}

/* Add staff-specific cards, tables, or forms here */

` `` `

frontend\src\styles\staff\payment_form.css

` `` `css

/* payment_form.css */

.form-step {
 background: #fff;

```

```
border: 1px solid #ddd;
border-radius: 6px;
padding: 20px;
margin-top: 20px;
font-family: "Segoe UI", Arial, sans-serif;
}
```

```
.form-step h2 {
margin-bottom: 15px;
font-size: 1.4rem;
color: #333;
}
```

```
.form-step p {
margin: 5px 0;
font-size: 0.95rem;
color: #555;
}
```

```
.form-step label {
display: block;
margin: 12px 0;
font-weight: 500;
color: #444;
}
```

```
.form-step select {
 width: 100%;
 padding: 8px;
 margin-top: 6px;
 border: 1px solid #ccc;
 border-radius: 4px;
}
```

```
button {
 padding: 8px 14px;
 margin: 8px 6px 0 0;
 border: none;
 border-radius: 4px;
 font-size: 0.95rem;
 cursor: pointer;
 transition: background 0.2s ease;
}
```

```
button:disabled {
 opacity: 0.6;
 cursor: not-allowed;
}
```

```
/* Specific button styles */
button.cancel-btn,
button.close-btn {
```

```
background: #f5f5f5;
color: #333;
}
```

```
button.cancel-btn:hover,
button.close-btn:hover {
 background: #e0e0e0;
}
```

```
button.proceed-btn {
 background: #4caf50;
 color: #fff;
}
```

```
button.proceed-btn:hover {
 background: #43a047;
}
```

```
button.print-btn {
 background: #2196f3;
 color: #fff;
}
```

```
button.print-btn:hover {
 background: #1976d2;
}
```

```
button[type="submit"] {
 background: #ff9800;
 color: #fff;
}
```

```
button[type="submit"]:hover {
 background: #fb8c00;
}
```

```
/* Align Proceed and Close buttons side by side */
```

```
.action-buttons {
 display: flex;
 justify-content: flex-start;
 gap: 10px;
 margin-top: 15px;
}
```

```
```
```

```
## frontend\src\styles\staff\registered-persons.css
```

```
```css
```

```
.registered-persons {
 background: var(--color-card-bg);
 padding: var(--space-md);
```

```
border-radius: var(--radius-md);
box-shadow: var(--shadow-sm);
margin-top: var(--space-lg);
}
```

```
.rp-header h3 {
 margin: 0;
 font-size: var(--font-size-lg);
 color: var(--color-text-strong);
}
```

```
.rp-header p {
 margin: var(--space-xs) 0 var(--space-md);
 font-size: var(--font-size-sm);
 color: var(--color-text);
}
```

```
.rp-table {
 width: 100%;
 border-collapse: collapse;
}
```

```
.rp-table th,
.rp-table td {
 padding: var(--space-sm);
 border-bottom: 1px solid var(--color-border);
}
```



```
text-align: left;
}
```

```
.rp-table tr:hover {
 background-color: var(--color-table-row-hover);
}
```

```
/* Role colors */
```

```
.role-resident { color: var(--color-resident); font-weight: var(--font-weight-bold); }
```

```
.role-staff { color: var(--color-staff); font-weight: var(--font-weight-bold); }
```

```
.role-admin { color: var(--color-admin); font-weight: var(--font-weight-bold); }
```

```
````
```

```
## frontend\src\styles\staff\staff-business-dashboard.css
```

```
````css
```

```
/* staff-business-dashboard.css */
```

```
/* Overall container */
```

```
.staff-business-dashboard {
```

```
 padding: 2rem;
```

```
 font-family: "Segoe UI", Arial, sans-serif;
```

```
 background-color: #f9fafb;
```

```
 min-height: 100vh;
```

```
}
```

```
/* Heading */

.staff-business-dashboard h2 {
 margin-bottom: 1.5rem;
 font-size: 1.6rem;
 color: #333;
}
```

```
/* Tabs */

.tabs {
 display: flex;
 gap: 1rem;
 margin-bottom: 2rem;
}
```

```
.tabs button {
 padding: 0.6rem 1.2rem;
 border: 1px solid #ccc;
 background-color: #fff;
 cursor: pointer;
 border-radius: 4px;
 font-weight: 500;
 transition: all 0.2s ease;
}
```

```
.tabs button:hover {
```

```
background-color: #f0f0f0;
}
```

```
.tabs button.active {
background-color: #0078d4;
color: #fff;
border-color: #0078d4;
}
```

```
/* Business grid */
.business-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
gap: 1.5rem;
}
```

```
/* Business card */
.business-card {
background-color: #fff;
border: 1px solid #e5e7eb;
border-radius: 8px;
padding: 1rem;
box-shadow: 0 2px 4px rgba(0,0,0,0.05);
transition: transform 0.2s ease;
}
```

```
.business-card:hover {
 transform: translateY(-2px);
}
```

```
.business-card h3 {
 margin-top: 0;
 margin-bottom: 0.5rem;
 font-size: 1.2rem;
 color: #0078d4;
}
```

```
.business-card p {
 margin: 0.25rem 0;
 font-size: 0.9rem;
 color: #555;
}
```

/\* Evaluate button \*/

```
.business-card button {
 margin-top: 1rem;
 padding: 0.5rem 1rem;
 background-color: #0078d4;
 color: #fff;
 border: none;
 border-radius: 4px;
 cursor: pointer;
```

```
font-weight: 500;
transition: background-color 0.2s ease;
}
```

```
.business-card button:hover{
 background-color: #005a9e;
}
```

```
```
```

```
## frontend\src\styles\treasurer.css
```

```
```css
```

```
.treasurer-dashboard {
 padding: 2rem;
 font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
 background-color: var(--color-bg);
 min-height: 100vh;
}
```

```
.treasurer-dashboard h1 {
 font-size: 2rem;
 font-weight: 600;
 margin-bottom: 1.5rem;
 color: var(--color-accent);
}
```

```
/* Summary Cards */
```

```
.summary-cards {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
 gap: 1rem;
 margin-bottom: 2rem;
}
```

```
.summary-cards .card {
 background: var(--color-card-bg);
 border-radius: 8px;
 padding: 1rem;
 box-shadow: 0 2px 6px rgba(0,0,0,0.08);
 font-size: 1rem;
 font-weight: 500;
 color: var(--color-text);
}
```

```
/* Revenue Chart */
```

```
.revenue-chart {
 background: var(--color-card-bg);
 border-radius: 8px;
 padding: 1rem;
 margin-bottom: 2rem;
 box-shadow: 0 2px 6px rgba(0,0,0,0.08);
```

```
/* 📌 Add these */
```

```
max-width: 600px; /* limit width */
```

```
height: 300px; /* fix height */
```

```
}
```

```
.revenue-chart canvas {
```

```
max-height: 250px; /* shrink chart inside */
```

```
}
```

```
.revenue-chart h2 {
```

```
font-size: 1.25rem;
```

```
margin-bottom: 1rem;
```

```
color: var(--color-success);
```

```
}
```

```
/* Transactions Table */
```

```
.transactions-table {
```

```
background: var(--color-card-bg);
```

```
border-radius: 8px;
```

```
padding: 1rem;
```

```
margin-bottom: 2rem;
```

```
box-shadow: 0 2px 6px rgba(0,0,0,0.08);
```

```
}
```

```
.transactions-table h2 {
 font-size: 1.25rem;
 margin-bottom: 1rem;
}
```

```
.transactions-table table {
 width: 100%;
 border-collapse: collapse;
}
```

```
.transactions-table th,
.transactions-table td {
 padding: 0.75rem;
 text-align: left;
 border-bottom: 1px solid var(--color-border);
}
```

```
.transactions-table th {
 background-color: var(--color-table-header);
 font-weight: 600;
 color: var(--color-text-strong);
}
```

```
.transactions-table tr:hover {
 background-color: var(--color-accent-hover);
 color: var(--color-text);
}
```



```
 transition: background-color 0.2s ease, color 0.2s ease;
}
```

```
/* Disbursement Table */
```

```
.disbursement-table {
 background: var(--color-card-bg);
 border-radius: 8px;
 padding: 1rem;
 margin-bottom: 2rem;
 box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}
```

```
.disbursement-table h2 {
 font-size: 1.25rem;
 margin-bottom: 1rem;
 color: var(--color-danger);
}
```

```
.disbursement-table table {
 width: 100%;
 border-collapse: collapse;
}
```

```
.disbursement-table th,
.disbursement-table td {
 padding: 0.75rem;
```

```
text-align: left;
border-bottom: 1px solid #e5e7eb;
}
```

```
.disbursement-table th {
 background-color: var(--color-table-header);
 font-weight: 600;
 color: var(--color-text-strong);
}
```

```
.disbursement-table tr:hover {
 background-color: #f9fafb;
}
```

```
/* Reports Section */
```

```
.reports-section {
 background: var(--color-card-bg);
 border-radius: 8px;
 padding: 1rem;
 box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}
```

```
.reports-section h2 {
 font-size: 1.25rem;
 margin-bottom: 1rem;
}
```

```
.reports-section button {
 background-color: #2563eb; /* blue */
 color: #ffffff;
 padding: 0.6rem 1.2rem;
 border: none;
 border-radius: 6px;
 font-size: 0.95rem;
 cursor: pointer;
 transition: background-color 0.2s ease;
}
```

```
.reports-section button:hover {
 background-color: #1e40af;
}
```

```
` ``
```

```
frontend\src\styles\treasurer\category-list.css
```

```
` `` `css
```

```
.category-table {
 width: 100%;
 border-collapse: collapse;
 margin-top: 1rem;
}
```

```
.category-table th,
.category-table td {
 border: 1px solid #ddd;
 padding: 0.75rem;
 text-align: left;
}
```

```
.category-table th {
 background-color: var(--color-table-header);
 font-weight: bold;
}
```

```
.category-table tr:nth-child(even) {
 background-color: var(--color-table-row-even);
}
```

```
.category-table .paid {
 color: var(--color-success); /* green */
 font-weight: 500;
}
```

```
.category-table .unpaid {
 color: var(--color-danger); /* red */
 font-weight: 500;
}
```

```
.category-table tr:hover {
 background-color: var(--color-accent-hover);
 color: #fff;
 transition: background-color 0.2s ease, color 0.2s ease;
}
` ``
```

```
frontend\src\styles\treasurer\disbursement-form.css
```

```
` `` `css
```

```
/* Overlay background */
```

```
.modal-overlay {
 position: fixed;
 top: 0;
 left: 0;
 width: 100%;
 height: 100%;
 background: rgba(0,0,0,0.5); /* semi-transparent black */
 display: flex;
 align-items: center;
 justify-content: center;
 z-index: 999; /* ensure it sits above other content */
}
```

```
/* Modal box */
```

```
.modal {
 background: #fff;
 padding: 2rem;
 border-radius: 8px;
 width: 400px;
 max-width: 90%;
 box-shadow: 0 4px 12px rgba(0,0,0,0.3);
 animation: fadeIn 0.3s ease-in-out;
}
```

```
/* Modal title */
```

```
.modal h2 {
 margin-top: 0;
 margin-bottom: 1rem;
 font-size: 1.4rem;
 color: #333;
}
```

```
/* Form labels */
```

```
.modal form label {
 display: block;
 margin-bottom: 1rem;
 font-weight: 500;
 color: #444;
}
```

```
/* Inputs & select */
```

```
.modal form input,
```

```
.modal form select {
```

```
width: 100%;
```

```
padding: 0.5rem;
```

```
margin-top: 0.3rem;
```

```
border: 1px solid #ccc;
```

```
border-radius: 4px;
```

```
}
```

```
/* Actions (buttons) */
```

```
.modal-actions {
```

```
display: flex;
```

```
justify-content: flex-end;
```

```
gap: 0.5rem;
```

```
margin-top: 1rem;
```

```
}
```

```
.modal-actions button {
```

```
padding: 0.5rem 1rem;
```

```
border: none;
```

```
border-radius: 4px;
```

```
cursor: pointer;
```

```
}
```

```
.modal-actions button[type="submit"] {
```

```
background: #2d8fdd;
color: #fff;
}
```

```
.modal-actions button[type="button"] {
background: #ccc;
color: #333;
}
```

```
/* Animation */
@keyframes fadeIn {
from { opacity: 0; transform: scale(0.95); }
to { opacity: 1; transform: scale(1); }
}
```

```
```
```

```
## frontend\src\styles\treasurer\summary-cards.css
```

```
```css
```

```
.summary-cards {
display: flex;
gap: 1rem;
margin: 1rem 0;
}
```



```
.card {
 flex: 1;
 background: #f9f9f9;
 border-radius: 8px;
 padding: 1rem;
 box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

```
.card-icon {
 font-size: 1.5rem;
}
```

```
.card-content h3 {
 margin: 0;
 font-size: 1rem;
 color: var(--color-text-muted);
}
```

```
.card-content p {
 margin: 0.5rem 0 0;
 font-size: 1.2rem;
 font-weight: bold;
}
```

```
...
```

```
frontend\src\styles\treasurer\transactions-table.css
```

```
` `` `css
```

```
.transactions-table h2 {
 margin-bottom: 12px;
 font-size: 1.2rem;
 color: var(--color-success);
}
```

```
.transactions-table table {
 width: 100%;
 border-collapse: collapse;
 font-size: 0.95rem;
}
```

```
.transactions-table th, .transactions-table td {
 padding: 10px 12px;
 text-align: left;
 border-bottom: 1px solid #ddd;
}
```

```
.transactions-table th {
 background-color: #f5f5f5;
 font-weight: 600;
 color: #222;
}
```

```
.transactions-table tr:nth-child(even) {
 background-color: var(--color-neutral);
}
```

```
.transactions-table tr:hover {
 background-color: var(--color-accent-hover);
 color: var(--color-text);
}
```

```
.transactions-table td {
 color: var(--color-text);
}
```

```
`, `
```

```
frontend\src\utils\cleanPayload.js
```

```
` `` `javascript
```

```
import { PARANAQUE } from "../data/locations";
```

```
/**
```

```
 * Normalize and sanitize resident form data before submission.
```

```
 * - Removes forbidden top-level fields (city, province, etc.)
```

```
 * - Converts empty strings to null
```

```
 * - Trims remarks
```

```
 * - Ensures booleans/enums are clean
```

\* - Normalizes fingerprint structure

\*/

```
export const cleanPayload = (data, uploads = {}) => {
```

```
 const {
```

```
 houseNumber,
```

```
 street,
```

```
 purok,
```

```
 barangay,
```

```
 city, // stripped out
```

```
 province, // stripped out
```

```
 zipCode,
```

```
 ...rest
```

```
 } = data;
```

```
 return {
```

```
 ...rest,
```

```
 photoUrl: uploads.photoUrl || null,
```

```
 signatureUrl: uploads.signatureUrl || null,
```

```
 fingerprints: {
```

```
 left: uploads.fingerprints?.left || null,
```

```
 right: uploads.fingerprints?.right || null,
```

```
 },
```

```
 address: {
```

```
 houseNumber: houseNumber || null,
```

```
 street: street || null,
```

```
 purok: purok || null,
```

```

 barangay: barangay || null,
 city: PARANAQUE.city,
 province: PARANAQUE.province,
 zipCode: zipCode || null,
 },
 isHeadOfFamily: data.isHeadOfFamily === "true",
 remarks: data.remarks?.trim() || null,
 email: data.email || null,
 occupation: data.occupation || null,
};
};

```

```

` ` `

```

```

frontend\src\utils\fees.js

```

```

` ` ` javascript

```

```

// src/utils/fees.js

```

```

export function resolveMiscFees(items, miscFees, shouldResolve = true) {
 const miscMap = Object.fromEntries(
 miscFees.map(m => [m.miscType.toLowerCase(), m])
);

```

```

 return items.map(item => {
 if (!shouldResolve) return { ...item, miscFeeResolved: null };

```

```

const misc = item.miscType

? miscMap[item.miscType.toLowerCase()]

: null;

return {

...item,

miscFeeResolved:

misc && misc.enabled && item.enabled ? misc.fee : null,

};

});

}

```

```

` ` `

```

```

frontend\src\utils\fileUtils.js

```

```

` ` ` javascript

```

```

import { ref, uploadBytes, getDownloadURL } from "firebase/storage";

import { storage } from "../services/firebase";

```

```

/**

```

```

*  Generate safe file reference

```

```

*/

```

```

export const generateSafeFileRef = (uid, path, fileName) => {

const safeFileName = encodeURIComponent(fileName);

return ref(storage, ` residents/${uid}/${path}/${safeFileName}`);

```

```
};
```

```
/**
```

```
 *  Resize image to square thumbnail (default 192px)
```

```
 */
```

```
export const resizeImage = (file, size = 192) =>
```

```
 new Promise((resolve, reject) => {
```

```
 const img = new Image();
```

```
 const reader = new FileReader();
```

```
 reader.onload = (e) => (img.src = e.target.result);
```

```
 reader.onerror = () => reject(new Error("FileReader failed"));
```

```
 reader.readAsDataURL(file);
```

```
 img.onload = () => {
```

```
 const canvas = document.createElement("canvas");
```

```
 canvas.width = size;
```

```
 canvas.height = size;
```

```
 const ctx = canvas.getContext("2d");
```

```
 ctx.drawImage(img, 0, 0, size, size);
```

```
 canvas.toBlob(
```

```
 (blob) => (blob ? resolve(blob) : reject(new Error("Resize failed"))),
```

```
 "image/jpeg",
```

```
 0.9
```

```
);
```

```
};
```

```
img.onerror = () => reject(new Error("Invalid image file"));
```

```
});
```

```
/**
```

```
 *  Upload blob with type-safe extension
```

```
 */
```

```
export const uploadBlobToStorage = async (uid, blob, path, fileName) => {
```

```
 const fileRef = generateSafeFileRef(uid, path, fileName);
```

```
 const contentType = blob.type || "image/png";
```

```
 console.log("📁 Uploading to:", fileRef.fullPath);
```

```
 console.log("📦 Blob type:", contentType);
```

```
 console.log("📦 Blob size:", blob.size);
```

```
 await uploadBytes(fileRef, blob, { contentType });
```

```
 return await getDownloadURL(fileRef);
```

```
};
```

```
/**
```

```
 *  Normalize extension based on MIME type
```

```
 */
```

```
const getExtensionFromMime = (mime) => {
```

```
 switch (mime) {
```



```

 case "image/jpeg":
 return "jpg";
 case "image/webp":
 return "webp";
 case "image/png":
 return "png";
 default:
 return "png";
 }
};

/**
 *  Upload file (with optional resize)
 */
export const uploadFile = async (uid, file, path, resize = false) => {
 try {
 const timestamp = Date.now();
 const blob = resize ? await resizeImage(file) : file;

 if (!blob || blob.size === 0) throw new Error(` ${path} blob is empty `);

 const ext = getExtensionFromMime(blob.type);
 const fileName = ` ${path}_ ${timestamp}. ${ext} `;

 return await uploadBlobToStorage(uid, blob, path, fileName);
 } catch (err) {

```

```

 console.error(` ❌ uploadFile error for ${path}:`, err);
 throw err;
 }
};

/**
 * ✅ Upload base64 image (e.g. signature pad)
 */
export const uploadBase64Image = async (uid, dataUrl, path) => {
 if (!dataUrl?.startsWith("data:image"))
 throw new Error("Malformed signature data URL");

 const response = await fetch(dataUrl);
 const blob = await response.blob();

 if (!blob || blob.size === 0) throw new Error("Signature blob is empty");

 const timestamp = Date.now();
 const ext = getExtensionFromMime(blob.type);
 const fileName = `${path}_${timestamp}.${ext}`;

 return await uploadBlobToStorage(uid, blob, path, fileName);
};

/**
 * ✅ Upload thumbprint (left/right)

```

```

*/

export const uploadThumbprint = async (uid, file, hand) => {
 if (!["left", "right"].includes(hand)) {
 throw new Error("Invalid thumb orientation. Must be 'left' or 'right!');
 }
 return await uploadFile(uid, file, `fingerprints/${hand}`, true);
};

```

```

` ` `

```

```

frontend\src\utils\locationService.js

```

```

` ` ` javascript

```

```

export const fetchBarangays = async (cityCode = "137607000") => {
 const res = await fetch(`https://psgc.cloud/api/barangays/city-
municipality/${cityCode}`);
 if (!res.ok) throw new Error("Failed to fetch barangays");
 const barangays = await res.json();
 return barangays.map(b => b.name);
};

```

```

` ` `

```

```

frontend\src\utils\payloadBuilders.js

```

```

` ` ` javascript

```

```
// src/utils/payloadBuilders.js
```

```
// 🛠️ Helper to safely coerce numbers
```

```
const safeNumber = (val) => {
 if (val === null || val === undefined || val === "") {
 return null; // explicitly missing
 }
 const num = Number(val);
 return Number.isFinite(num) ? num : null;
};
```

```
// 📄 Document payload builder
```

```
export const buildDocumentPayload = (item, key, value) => ({
 fee: safeNumber(key === "fee" ? value : item.fee),
 miscType: item.miscType || null,
 enabled: key === "enabled" ? !value : !item.enabled,
 documentType: item.documentType || item.id || null, // ✅ always include
});
```

```
// 🏢 Business payload builder
```

```
export const buildBusinessPayload = (item, key, value) => ({
 fee: safeNumber(key === "fee" ? value : item.fee),
 registrationFee: safeNumber(
 key === "registrationFee" ? value : item.registrationFee
),
});
```

```
annualFee: safeNumber(
 key === "annualFee" ? value : item.annualFee
),
miscType: item.miscType || null,
enabled: key === "enabled" ? !!value : !!item.enabled,
businessType: item.businessType || item.id || null, // ✅ always include
});
```

```
// 🆕 Misc payload builder
```

```
export const buildMiscPayload = (item, key, value) => ({
 fee: safeNumber(key === "fee" ? value : item.fee),
 enabled: key === "enabled" ? !!value : !!item.enabled,
 miscType: item.miscType || item.id || null, // ✅ always include
});
```

```
...
```

```
frontend\src\utils\resolveLocation.js
```

```
```javascript
```

```
// resolveLocation.js
```

```
export function resolveLocation(documentType, formData, residents = [], businesses = []) {
  let location = {
    barangay: "",
    street: "",
    city: "",
```

```
    province: "",  
    address: ""  
};
```

```
switch (documentType) {  
  case "Barangay Clearance": {  
    const resident = residents.find(r => r.id === formData.resident_id);  
    if (resident?.address) {  
      location = {  
        barangay: resident.address.barangay || "",  
        street: resident.address.street || "",  
        city: resident.address.city || "",  
        province: resident.address.province || ""  
      };  
    }  
    break;  
  }  
}
```

```
  case "Business Clearance": {  
    const business = businesses.find(b => b.businessName === formData.businessName);  
    if (business) {  
      location = {  
        barangay: business.barangay || "",  
        street: business.street || "",  
        city: business.city || "",  
        province: business.province || ""  
      }  
    }  
  }  
}
```

```
};  
}  
break;  
}
```

```
default: {  
  // Activity Permit, Blotter Report, etc.  
  location = {  
    barangay: formData["location.barangay"] || "",  
    street: formData["location.street"] || "",  
    city: formData["location.city"] || "",  
    province: formData["location.province"] || ""  
  };  
}  
}
```

```
//  Always build a clean address string  
const parts = [  
  location.street,  
  location.barangay ? `Brgy. ${location.barangay}` : "",  
  location.city,  
  location.province  
].filter(Boolean);  
  
location.address = parts.join(", ");
```

```
    return location;
}
```

```
````
```

```
frontend\src\utils\stats.js
```

```
```javascript
```

```
// utils/stats.js
```

```
export const computePaymentStats = (documents = []) => {
```

```
    let paid = 0;
```

```
    let awaiting_payment = 0;
```

```
    documents.forEach((doc) => {
```

```
        const amount = doc.amount || 0;
```

```
        const status = doc.status || doc.documentStatus;
```

```
        // Free documents (amount = 0) are considered paid
```

```
        if (amount === 0) {
```

```
            paid += 1;
```

```
            return;
```

```
        }
```

```
        // Otherwise, check status
```

```
        if (status === "paid" || status === "approved") {
```

```
            paid += 1;
```



```
    } else if (status === "awaiting_payment" || status === "pending") {  
        awaiting_payment += 1;  
    }  
});
```

```
    return { paid, awaiting_payment };  
};
```

```
...
```

```
## frontend\src\utils\validation.js
```

```
```javascript
```

```
// src/utils/validation.js
```

```
import validationConfig from "../config/validationConfig";
```

```
export function validateFeePayload(mode, payload) {
 const config = validationConfig[mode];
 if (!config) {
 return { valid: false, message: `Unknown mode: ${mode}` };
 }
```

```
 for (const rule of config.rules) {
 const value = payload[rule.field];
 if (!rule.validate(value)) {
 return { valid: false, message: rule.message };
 }
```

```
 }
 }

 return { valid: true, message: "" };
}
```

```
```
```

```
## functions\eslinttrc.js
```

```
```javascript  
module.exports = {
 env: {
 es6: true,
 node: true,
 },
 parserOptions: {
 "ecmaVersion": 2018,
 },
 extends: [
 "eslint:recommended",
 "google",
],
 rules: {
 "no-restricted-globals": ["error", "name", "length"],
 "prefer-arrow-callback": "error",
 },
}
```

```
"quotes": ["error", "double", {"allowTemplateLiterals": true}],
},
overrides: [
 {
 files: ["**/*.spec.*"],
 env: {
 mocha: true,
 },
 rules: {},
 },
],
globals: {},
};
```

```
...`
```

```
functions\emailTemplates.js
```

```
` `` `javascript
```

```
// functions/emailTemplates.js
```

```
function welcomeTemplate({ fullName, email, barangay }) {
 return {
 subject: `Welcome to ${barangay}`,
 text: `Dear ${fullName},
```

Welcome to the Barangay \${barangay} Registry system.

Login details:

Username: \${email}

Password: 123456 (please change after first login)

Login here: <https://barangay-1721d.web.app>

Best regards,

Barangay \${barangay} Registry Team ` ,

};

}

```
function passwordResetTemplate({ fullName, resetLink }) {
```

```
 return {
```

```
 subject: "Password Reset Request",
```

```
 text: ` Hello ${fullName},
```

We received a request to reset your password.

Click the link below to set a new password:

[\\${resetLink}](${resetLink})

If you did not request this, please ignore this email. ` ,

html: `

```
<p>Hello ${fullName},</p>
```

```
<p>We received a request to reset your password.</p>
```

```
<p>
```

```

```

```
👉 Click here to set a new password
```

```

```

```
</p>
```

```
<p>If you did not request this, please ignore this email.</p>
```

```
` ,
```

```
};
```

```
}
```

```
module.exports = {
```

```
 welcomeTemplate,
```

```
 passwordResetTemplate,
```

```
};
```

```
` ``
```

```
functions\index.js
```

```
` `` ` javascript
```

```
require("dotenv").config();
```

```
const { google } = require("googleapis");
```

```
const nodemailer = require("nodemailer");
```

```
const { https } = require("firebase-functions/v2"); // ✅ v2 import
```

```
const express = require("express");

const cors = require("cors");

const { welcomeTemplate, passwordResetTemplate } = require("./emailTemplates");

const CLIENT_ID = process.env.GMAIL_CLIENT_ID;

const CLIENT_SECRET = process.env.GMAIL_CLIENT_SECRET;

const REFRESH_TOKEN = process.env.GMAIL_REFRESH_TOKEN;

const REDIRECT_URI = "https://developers.google.com/oauthplayground";

const oAuth2Client = new google.auth.OAuth2(CLIENT_ID, CLIENT_SECRET,
REDIRECT_URI);

oAuth2Client.setCredentials({ refresh_token: REFRESH_TOKEN });

const app = express();

app.use(cors({ origin: true }));

app.use(express.json());

// 🛠 Template selector
function getTemplate(type, payload) {
 switch (type) {
 case "welcome":
 return welcomeTemplate(payload);
 case "reset":
 return passwordResetTemplate(payload);
 default:
 throw new Error("Invalid email type");
 }
}
```

```
}
}
```

```
app.post("/", async (req, res) => {
```

```
 console.log("📧 Backend received body:", req.body);
```

```
 const { type, fullName, email, barangay, resetLink } = req.body;
```

```
 if (!email) return res.status(400).json({ error: "Recipient email is required" });
```

```
 try {
```

```
 const accessToken = await oAuth2Client.getAccessToken();
```

```
 const tokenValue = typeof accessToken === "string" ? accessToken : accessToken?.token;
```

```
 if (!tokenValue) throw new Error("❌ Failed to retrieve Gmail access token");
```

```
 const transporter = nodemailer.createTransport({
```

```
 service: "gmail",
```

```
 auth: {
```

```
 type: "OAuth2",
```

```
 user: "jonladyong@gmail.com",
```

```
 clientId: CLIENT_ID,
```

```
 clientSecret: CLIENT_SECRET,
```

```
 refreshToken: REFRESH_TOKEN,
```

```
 accessToken: tokenValue,
```

```
 },
```

```
 });
```

```

const { subject, text, html } = getTemplate(type, { fullName, email, barangay, resetLink });

const mailOptions = {
 from: ` "Barangay System" <jonladyong@gmail.com> `,
 to: email,
 subject,
 text,
 html,
};

console.log(` 📧 Sending ${type} email to: ` , email);

const result = await transporter.sendMail(mailOptions);

console.log(" ✅ Email sent:", result);
res.json({ success: true });
} catch (err) {
 console.error(" ❌ Email error:", err.message);
 res.status(500).json({ error: "Failed to send email" });
}
});

// ✅ v2 region placement
exports.sendEmailAsia = https.onRequest({ region: "asia-southeast1" }, app);

...

```



```
functions\package.json
```

```
` `` `json
{
 "name": "functions",
 "description": "Cloud Functions for Firebase",
 "scripts": {
 "lint": "echo \"No linting configured\"",
 "serve": "firebase emulators:start --only functions",
 "shell": "firebase functions:shell",
 "start": "npm run shell",
 "deploy": "firebase deploy --only functions",
 "logs": "firebase functions:log"
 },
 "engines": {
 "node": "24"
 },
 "main": "index.js",
 "dependencies": {
 "dotenv": "^17.3.1",
 "firebase-admin": "^13.6.0",
 "firebase-functions": "^7.0.5",
 "googleapis": "^171.4.0",
 "nodemailer": "^8.0.1"
 },

```

```
"devDependencies": {
 "eslint": "^10.0.2",
 "eslint-config-google": "^0.14.0",
 "firebase-functions-test": "^2.1.0"
},
"private": true
}
```

...

## functions\package-lock.json

```
```.json
{
  "name": "functions",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "functions",
      "dependencies": {
        "dotenv": "^17.3.1",
        "firebase-admin": "^13.6.0",
        "firebase-functions": "^7.0.5",
        "googleapis": "^171.4.0",
        "nodemailer": "^8.0.1"
      }
    }
  }
}
```

```
  },
  "devDependencies": {
    "eslint": "^10.0.2",
    "eslint-config-google": "^0.14.0",
    "firebase-functions-test": "^2.1.0"
  },
  "engines": {
    "node": "24"
  }
},
"node_modules/@eslint-community/eslint-utils": {
  "version": "4.9.1",
  "resolved": "https://registry.npmjs.org/@eslint-community/eslint-utils/-/eslint-utils-4.9.1.tgz",
  "integrity": "sha512-phrYmNiYppR7znFEdqgFWHXR6NCkZEK7hwWDHZUjit/2/U0r6XvkDI0SYnoM51Hq7FhCGdLDT6zxCCOY1hexsQ==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "eslint-visitor-keys": "^3.4.3"
  },
  "engines": {
    "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
  },
  "funding": {
    "url": "https://opencollective.com/eslint"
  }
}
```

```
  },
  "peerDependencies": {
    "eslint": "^6.0.0 || ^7.0.0 || >=8.0.0"
  },
  "node_modules/@eslint-community/regexpp": {
    "version": "4.12.2",
    "resolved": "https://registry.npmjs.org/@eslint-community/regexpp/-/regexpp-4.12.2.tgz",
    "integrity": "sha512-EriSTlt5OC9/7SXkRSCAhfSxxoSUGBm33OH+lkwbdpgoqsSsUg7y3uh+IICI/Qg4BBWr3U2i39RpmYcbxMq4ew==",
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": "^12.0.0 || ^14.0.0 || >=16.0.0"
    }
  },
  "node_modules/@eslint/config-array": {
    "version": "0.23.2",
    "resolved": "https://registry.npmjs.org/@eslint/config-array/-/config-array-0.23.2.tgz",
    "integrity": "sha512-YF+fE6LV4v5MGWRGj7G404/OZzGNepVF8fxk7jqmqo3lrza7a0uUcDnROGRBG1WFC1omYUS/Wp1f42i0M+3Q3A==",
    "dev": true,
    "license": "Apache-2.0",
    "dependencies": {
```

```
"@eslint/object-schema": "^3.0.2",
"debug": "^4.3.1",
"minimatch": "^10.2.1"
},
"engines": {
  "node": "^20.19.0 || ^22.13.0 || >=24"
}
},
"node_modules/@eslint/config-helpers": {
  "version": "0.5.2",
  "resolved": "https://registry.npmjs.org/@eslint/config-helpers/-/config-helpers-0.5.2.tgz",
  "integrity": "sha512-a5MxrdDXEvqnlq+LisyCX6tQMPF/dSJpCfBgBauY+pNZ28yCtSsTvyTYrMhal+LK26bVvCJfJkTOu8Klj2i1dQ==",
  "dev": true,
  "license": "Apache-2.0",
  "dependencies": {
    "@eslint/core": "^1.1.0"
  },
  "engines": {
    "node": "^20.19.0 || ^22.13.0 || >=24"
  }
},
"node_modules/@eslint/core": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/@eslint/core/-/core-1.1.0.tgz",
```

```
"integrity": "sha512-
/nr9K9wkr3P1EzFTdFdMoLuo1PmIxjmwvPozwoSodjNBdefGujXQUF93u1DDZpEaTuDvMslQ
ddsd35BwtrW9Xw==",

"dev": true,

"license": "Apache-2.0",

"dependencies": {

  "@types/json-schema": "^7.0.15"

},

"engines": {

  "node": "^20.19.0 || ^22.13.0 || >=24"

}

},

"node_modules/@eslint/object-schema": {

  "version": "3.0.2",

  "resolved": "https://registry.npmjs.org/@eslint/object-schema/-/object-schema-
3.0.2.tgz",

  "integrity": "sha512-
HOy56KJt48Bx8KmJ+XGQNSUMT/6dZee/M54XyUyuvTvPXJmsERRvBchsUVx1UMe1WwIH4
9XLAczNC7V2INsuUw==",

  "dev": true,

  "license": "Apache-2.0",

  "engines": {

    "node": "^20.19.0 || ^22.13.0 || >=24"

  }

},

"node_modules/@eslint/plugin-kit": {

  "version": "0.6.0",
```

```
"resolved": "https://registry.npmjs.org/@eslint/plugin-kit/-/plugin-kit-0.6.0.tgz",
"integrity": "sha512-b1ZEUzOI1jkhviX2cp5vNyXQc6olz2ohewQubuYlMXZ2Q/XjBO0x0XhGPvc9fjSliUN0vw+0hq53BJ4eQSJKQ==",
"dev": true,
"license": "Apache-2.0",
"dependencies": {
"@eslint/core": "^1.1.0",
"levn": "^0.4.1"
},
"engines": {
"node": "^20.19.0 || ^22.13.0 || >=24"
},
"node_modules/@fastify/busboy": {
"version": "3.2.0",
"resolved": "https://registry.npmjs.org/@fastify/busboy/-/busboy-3.2.0.tgz",
"integrity": "sha512-b1WZ588pG1U1H8R1E2Z1H389UgT1iN7ZS0WoZvWCVcpv6m4T51vN7Z1jWZ4tdv3l2T9W0T86nU1pX2w==",
"license": "MIT"
},
"node_modules/@firebase/app-check-interop-types": {
"version": "0.3.3",
"resolved": "https://registry.npmjs.org/@firebase/app-check-interop-types/-/app-check-interop-types-0.3.3.tgz",
```

```
  "integrity": "sha512-  
gAlxfPLT2j8bTl/qfe3ahl2l2YcBQ8cFIBdhAQA4l2f3TndcO+22YizyGYuttLHPQEpWkhmpFW60  
VCfEPg4g5A==",
```

```
  "license": "Apache-2.0"
```

```
},
```

```
"node_modules/@firebase/app-types": {
```

```
  "version": "0.9.3",
```

```
  "resolved": "https://registry.npmjs.org/@firebase/app-types/-/app-types-0.9.3.tgz",
```

```
  "integrity": "sha512-  
kRVpIl4vVGJ4baogMDINbyrIOtOxqhkZQg4jTq3l8Lw6WSk0xfpEYzezFu+Kl4ve4fbPl79dvwRta  
FqAC/ucCw==",
```

```
  "license": "Apache-2.0"
```

```
},
```

```
"node_modules/@firebase/auth-interop-types": {
```

```
  "version": "0.2.4",
```

```
  "resolved": "https://registry.npmjs.org/@firebase/auth-interop-types/-/auth-interop-  
types-0.2.4.tgz",
```

```
  "integrity": "sha512-  
JPgcXKCuO+CWqGDnigBtvo09HeBs5u/Ktc2GaFj2m01hLarbxthLNm7Fk8iOP1aqAtXV+fnn  
Gj7U28xmk7lwVA==",
```

```
  "license": "Apache-2.0"
```

```
},
```

```
"node_modules/@firebase/component": {
```

```
  "version": "0.7.0",
```

```
  "resolved": "https://registry.npmjs.org/@firebase/component/-/component-0.7.0.tgz",
```

```
  "integrity": "sha512-  
wR9En2A+WESUHexjmRHkqtaVH94WLNKt6rmeqZhSLBybg4Wyf0Umk04SZsS6sBq4102Zs  
DBFwoqMqJYj2loDSg==",
```

```
  "license": "Apache-2.0",
```



```
"dependencies": {
  "@firebase/util": "1.13.0",
  "tslib": "^2.1.0"
},
"engines": {
  "node": ">=20.0.0"
}
},
"node_modules/@firebase/database": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/@firebase/database/-/database-1.1.0.tgz",
  "integrity": "sha512-gM6MJFae3pTyNLoc9VcJNuaUDej0ctdjn3cVtLo3D5lpp0dmUHHLFN/pUKe7ImyeB1KAvRIEYxvIHNF04Filg==",
  "license": "Apache-2.0",
  "dependencies": {
    "@firebase/app-check-interop-types": "0.3.3",
    "@firebase/auth-interop-types": "0.2.4",
    "@firebase/component": "0.7.0",
    "@firebase/logger": "0.5.0",
    "@firebase/util": "1.13.0",
    "faye-websocket": "0.11.4",
    "tslib": "^2.1.0"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}
```

```
  },
  "node_modules/@firebase/database-compat": {
    "version": "2.1.0",
    "resolved": "https://registry.npmjs.org/@firebase/database-compat/-/database-compat-2.1.0.tgz",
    "integrity": "sha512-8nYc43RqxScsePVd1qe1xxvWNf0OBnbwHxmXJ7MHSuuTVYFO3eLyLW3PiCKJ9fHnmlz4p4LbieXwz+qtr9PZDg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@firebase/component": "0.7.0",
      "@firebase/database": "1.1.0",
      "@firebase/database-types": "1.0.16",
      "@firebase/logger": "0.5.0",
      "@firebase/util": "1.13.0",
      "tslib": "^2.1.0"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@firebase/database-types": {
    "version": "1.0.16",
    "resolved": "https://registry.npmjs.org/@firebase/database-types/-/database-types-1.0.16.tgz",
    "integrity": "sha512-xkQLQfU5De7+SPhEGAXFBnDryUWhhlFXelEg2YeZOQMCdoe7dL64DDAd77SQsR+6uoXIZY5MB4y/inCs4GTfcw==",
```

```
"license": "Apache-2.0",
"dependencies": {
  "@firebase/app-types": "0.9.3",
  "@firebase/util": "1.13.0"
},
"node_modules/@firebase/logger": {
  "version": "0.5.0",
  "resolved": "https://registry.npmjs.org/@firebase/logger/-/logger-0.5.0.tgz",
  "integrity": "sha512-cGskaAvkrnh42b3BA3doDWeBmuHFO/Mx5A83rbRDYakPjO9bJtRL3dX7javzc2Rr/JHZf4Hlte
rTW2lUkfeN4g==",
  "license": "Apache-2.0",
  "dependencies": {
    "tslib": "^2.1.0"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@firebase/util": {
  "version": "1.13.0",
  "resolved": "https://registry.npmjs.org/@firebase/util/-/util-1.13.0.tgz",
  "integrity": "sha512-0AZUyYUfpMNcztR5l09izHwXkZpghLgCUaAGjtMwXnCg3bj4ml5VgiwqOMOxJ+Nw4qN/zJAa
OQBcJ7KGkWStqQ==",
  "hasInstallScript": true,
```

```
"license": "Apache-2.0",
"dependencies": {
  "tslib": "^2.1.0"
},
"engines": {
  "node": ">=20.0.0"
}
},
"node_modules/@google-cloud/firestore": {
  "version": "7.11.6",
  "resolved": "https://registry.npmjs.org/@google-cloud/firestore/-/firestore-7.11.6.tgz",
  "integrity": "sha512-EW/O8ktzwLfYWBOsNuhRoMi8lrC3clHM5LVFhGvO1HCsLozCOOXRAIHrYBoE6HL42Sc8yYMuCb2XqcnJ4OOEpw==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "@opentelemetry/api": "^1.3.0",
    "fast-deep-equal": "^3.1.1",
    "functional-red-black-tree": "^1.0.1",
    "google-gax": "^4.3.3",
    "protobufjs": "^7.2.6"
  },
  "engines": {
    "node": ">=14.0.0"
  }
},
```

```
"node_modules/@google-cloud/paginator": {
  "version": "5.0.2",
  "resolved": "https://registry.npmjs.org/@google-cloud/paginator/-/paginator-5.0.2.tgz",
  "integrity": "sha512-DJS3s0OVH4zFDB1PzjxAsHqJT6sKVbRwwML0ZBP9PbU7Yebtu/7SWMRzvO2J3nUi9pRNITCfu4LJeoM2w4pJg==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "arrify": "^2.0.0",
    "extend": "^3.0.2"
  },
  "engines": {
    "node": ">=14.0.0"
  }
},
"node_modules/@google-cloud/projectify": {
  "version": "4.0.0",
  "resolved": "https://registry.npmjs.org/@google-cloud/projectify/-/projectify-4.0.0.tgz",
  "integrity": "sha512-MmaX6HeSvyPbWGwFq7mXdo0uQZLGBYCwziiLIGq5JVX+/bdI3SAq6bP98trV5eTWfLuvsMcIC1YJOF2vftLFA==",
  "license": "Apache-2.0",
  "optional": true,
  "engines": {
    "node": ">=14.0.0"
  }
}
```

```
  },
  "node_modules/@google-cloud/promisify": {
    "version": "4.0.0",
    "resolved": "https://registry.npmjs.org/@google-cloud/promisify/-/promisify-4.0.0.tgz",
    "integrity": "sha512-Orxzlf9c67A15cq2JQEYVc7wEsmFBmHjZWZYQMUYJ1qivXyMwdyNOs9odi79hze+2zqdTtu1E19IM/FtqZ10g==",
    "license": "Apache-2.0",
    "optional": true,
    "engines": {
      "node": ">=14"
    }
  },
  "node_modules/@google-cloud/storage": {
    "version": "7.19.0",
    "resolved": "https://registry.npmjs.org/@google-cloud/storage/-/storage-7.19.0.tgz",
    "integrity": "sha512-n2FjE7NAOYyshogdc7KQOI/VZb4sneqPjWouSyia9CMDdMhRX5+RibqalNmC7LOLzuLAN89VIF2HvG8na9G+zQ==",
    "license": "Apache-2.0",
    "optional": true,
    "dependencies": {
      "@google-cloud/paginator": "^5.0.0",
      "@google-cloud/projectify": "^4.0.0",
      "@google-cloud/promisify": "<4.1.0",
      "abort-controller": "^3.0.0",
      "async-retry": "^1.3.3",
    }
  }
}
```

```
"duplexify": "^4.1.3",
"fast-xml-parser": "^5.3.4",
"gaxios": "^6.0.2",
"google-auth-library": "^9.6.3",
"html-entities": "^2.5.2",
"mime": "^3.0.0",
"p-limit": "^3.0.1",
"retry-request": "^7.0.0",
"teeny-request": "^9.0.0",
"uuid": "^8.0.0"
},
"engines": {
  "node": ">=14"
}
},
"node_modules/@google-cloud/storage/node_modules/uuid": {
  "version": "8.3.2",
  "resolved": "https://registry.npmjs.org/uuid/-/uuid-8.3.2.tgz",
  "integrity": "sha512-+NYs2QeMWy+GWFOEm9xnn6HCDp0l7QBD7ml8zLUmJ+93Q5NF0NocErnwkTkXVFNiX3/fpC6afS8Dhb/gz7R7eg==",
  "license": "MIT",
  "optional": true,
  "bin": {
    "uuid": "dist/bin/uuid"
  }
},
```

```
"node_modules/@grpc/grpc-js": {
  "version": "1.14.3",
  "resolved": "https://registry.npmjs.org/@grpc/grpc-js/-/grpc-js-1.14.3.tgz",
  "integrity": "sha512-lq8QQQ/7X3Sac15oB6p0FmUg/klxQvXLeileoqrTRGJYLV+/9tubbr9ipz0GKHjmXVsgFPo/+W+2cA8eNcR+XA==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "@grpc/proto-loader": "^0.8.0",
    "@js-sdsl/ordered-map": "^4.4.2"
  },
  "engines": {
    "node": ">=12.10.0"
  }
},
"node_modules/@grpc/grpc-js/node_modules/@grpc/proto-loader": {
  "version": "0.8.0",
  "resolved": "https://registry.npmjs.org/@grpc/proto-loader/-/proto-loader-0.8.0.tgz",
  "integrity": "sha512-rc1hOQtjIWGxcxpb9aHAfLpIctjEnsDehj0DAiVfBlmT84uvR0uUtN2hEi/ecvWVjXUGf5qPF4qEgiLOx1YIMQ==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "lodash.camelcase": "^4.3.0",
    "long": "^5.0.0",

```



```
"protobufjs": "^7.5.3",
"yargs": "^17.7.2"
},
"bin": {
  "proto-loader-gen-types": "build/bin/proto-loader-gen-types.js"
},
"engines": {
  "node": ">=6"
}
},
"node_modules/@grpc/proto-loader": {
  "version": "0.7.15",
  "resolved": "https://registry.npmjs.org/@grpc/proto-loader/-/proto-loader-0.7.15.tgz",
  "integrity": "sha512-tMXdRCfYVixjuFK+Hk0Q1s38gV9zDiDJfWL3h1rv4Qc39oILCu1TRTDt7+fGUI8K4G1Fj125Hx/ru3azECWTyQ==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "lodash.camelcase": "^4.3.0",
    "long": "^5.0.0",
    "protobufjs": "^7.2.5",
    "yargs": "^17.7.2"
  },
  "bin": {
    "proto-loader-gen-types": "build/bin/proto-loader-gen-types.js"
  },
}
```

```
"engines": {
  "node": ">=6"
},
"node_modules/@humanfs/core": {
  "version": "0.19.1",
  "resolved": "https://registry.npmjs.org/@humanfs/core/-/core-0.19.1.tgz",
  "integrity": "sha512-5DyQ4+1JEUzejeK1JGICcideyfUbGixgS9jNgex5nqkW+cY7WZhxBigmieN5Qnw9ZosSNVC9KQKyb+GUaGyKUA==",
  "dev": true,
  "license": "Apache-2.0",
  "engines": {
    "node": ">=18.18.0"
  },
  "node_modules/@humanfs/node": {
    "version": "0.16.7",
    "resolved": "https://registry.npmjs.org/@humanfs/node/-/node-0.16.7.tgz",
    "integrity": "sha512-/zUx+yOslrG4Y43Eh2peDeKCxlRt/gET6aHfaKpuq267qXdYDFViVHfMaLyygZOnl0kGWxFlgsBy8QFuTLUXEQ==",
    "dev": true,
    "license": "Apache-2.0",
    "dependencies": {
      "@humanfs/core": "^0.19.1",
      "@humanwhocodes/retry": "^0.4.0"
    }
  }
}
```

```
,
  "engines": {
    "node": ">=18.18.0"
  }
},
"node_modules/@humanwhocodes/module-importer": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/@humanwhocodes/module-importer/-/module-importer-1.0.1.tgz",
  "integrity": "sha512-bxveV4V8v5Yb4ncFTT3rPSgZBOpCkjfK0y4oVVVJwluDVBRMDXrPyXRL988i5ap9m9bnyEEjWfm5WkBmtffLfA==",
  "dev": true,
  "license": "Apache-2.0",
  "engines": {
    "node": ">=12.22"
  },
  "funding": {
    "type": "github",
    "url": "https://github.com/sponsors/nzakas"
  }
},
"node_modules/@humanwhocodes/retry": {
  "version": "0.4.3",
  "resolved": "https://registry.npmjs.org/@humanwhocodes/retry/-/retry-0.4.3.tgz",
```

```
"integrity": "sha512-
bV0Tgo9K4hfPCek+aMAn81RppFKv2ySDQeMoSZuvTASywNTnVJCArCZE2FWqpvlAtKu7VMR
LWlR1EazvVhDyhQ==",

"dev": true,

"license": "Apache-2.0",

"engines": {

  "node": ">=18.18"

},

"funding": {

  "type": "github",

  "url": "https://github.com/sponsors/nzakas"

},

"node_modules/@isaacs/cliui": {

  "version": "8.0.2",

  "resolved": "https://registry.npmjs.org/@isaacs/cliui/-/cliui-8.0.2.tgz",

  "integrity": "sha512-
O8jcbabXaleOG9DQ0+ARXWZBTfnP4WNAqzuiJK7ll44AmxGKv/J2M4TPjxjY3znBCfvBXFzuc
m1twdyFybFqEA==",

  "license": "ISC",

  "dependencies": {

    "string-width": "^5.1.2",

    "string-width-cjs": "npm:string-width@^4.2.0",

    "strip-ansi": "^7.0.1",

    "strip-ansi-cjs": "npm:strip-ansi@^6.0.1",

    "wrap-ansi": "^8.1.0",

    "wrap-ansi-cjs": "npm:wrap-ansi@^7.0.0"
```

```
,
  "engines": {
    "node": ">=12"
  }
},
"node_modules/@isaacs/cliui/node_modules/ansi-regex": {
  "version": "6.2.2",
  "resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-6.2.2.tgz",
  "integrity": "sha512-Bq3SmSpYFHAWjPk8lf9yc6svM8c56dB5BAW4Qbw5jHTwwXXcTLoRMkpDJp6VL0XzIWaCHTXrkFURMYmD0sLqg==",
  "license": "MIT",
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "https://github.com/chalk/ansi-regex?sponsor=1"
  }
},
"node_modules/@isaacs/cliui/node_modules/strip-ansi": {
  "version": "7.1.2",
  "resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-7.1.2.tgz",
  "integrity": "sha512-Gq4a8I00qoKqL2Ue6O9WlH23yVtYUeZWD9y8a2EqP6+4B8Vbf1+o62lP2K2jLPMKdjQ6Qec4YiK2ABl2Iw==",
  "license": "MIT",
  "dependencies": {
```

```
"ansi-regex": "^6.0.1"
},
"engines": {
  "node": ">=12"
},
"funding": {
  "url": "https://github.com/chalk/strip-ansi?sponsor=1"
}
},
"node_modules/@js-sdsl/ordered-map": {
  "version": "4.4.2",
  "resolved": "https://registry.npmjs.org/@js-sdsl/ordered-map/-/ordered-map-4.4.2.tgz",
  "integrity": "sha512-iUKgm52T8HOE/makSxjqoWhe95ZJA1/G1sYsGev2JDKUSS14KAgg1LHb+Ba+IPow0xflbnSkOsZcO08C7w1gYw==",
  "license": "MIT",
  "optional": true,
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/js-sdsl"
  }
},
"node_modules/@opentelemetry/api": {
  "version": "1.9.0",
  "resolved": "https://registry.npmjs.org/@opentelemetry/api/-/api-1.9.0.tgz",
```

```
"integrity": "sha512-3giAOQvZiH5F9bMlMiv8+GSPMeqg0dbaeo58/0SlA9sxSqZhnUtxzX9/2FzyhS9sWQf5S0GJE0AKBrFqjpeYcg==",
  "license": "Apache-2.0",
  "optional": true,
  "engines": {
    "node": ">=8.0.0"
  }
},
"node_modules/@pkgjs/parseargs": {
  "version": "0.11.0",
  "resolved": "https://registry.npmjs.org/@pkgjs/parseargs/-/parseargs-0.11.0.tgz",
  "integrity": "sha512-+1VkjdD0QBLPodGrJUeqarH8VAIvQODIbwh9XpP5Syisf7YoQgsJKPNFoqqLQlu+VQ/tVSshMR6loPMn8U+dPg==",
  "license": "MIT",
  "optional": true,
  "engines": {
    "node": ">=14"
  }
},
"node_modules/@protobufjs/aspromise": {
  "version": "1.1.2",
  "resolved": "https://registry.npmjs.org/@protobufjs/aspromise/-/aspromise-1.1.2.tgz",
  "integrity": "sha512-+gKExEuLmKwvz3OgROXtrJ2UG2x8Ch2YZUxahh+s1F2HZ+wAceUNLkvy6zKCPVRkU++ZWQrdxsUeQXmcg4uoQ==",
  "license": "BSD-3-Clause"
```

```
  },
  "node_modules/@protobufjs/base64": {
    "version": "1.1.2",
    "resolved": "https://registry.npmjs.org/@protobufjs/base64/-/base64-1.1.2.tgz",
    "integrity": "sha512-4Z9Zg0AqUnb1s/e2D+H4dOgTcUCihqcrzuHAY4Rl3PmfzIQdNcKtc0AMzJdIUaVXpD7Lh0wFqP7fzUyRkQ==",
    "license": "BSD-3-Clause"
  },
  "node_modules/@protobufjs/codegen": {
    "version": "2.0.4",
    "resolved": "https://registry.npmjs.org/@protobufjs/codegen/-/codegen-2.0.4.tgz",
    "integrity": "sha512-6xR1Bx7lS8d7F76W44QZGUWHDv1wNt8I2zD01aIDS0W9H6/4i7FwK955rCS8iMfJpZD1MzH8y8UgQ==",
    "license": "BSD-3-Clause"
  },
  "node_modules/@protobufjs/eventemitter": {
    "version": "1.1.0",
    "resolved": "https://registry.npmjs.org/@protobufjs/eventemitter/-/eventemitter-1.1.0.tgz",
    "integrity": "sha512-9oqHnBj8H4OxHogQzXwQUDvT/SifK1MuQbnj75Q5zI5hUeMKGHDqZKwH0ag2r7KsrB7713XU9UQ0SzPjg==",
    "license": "BSD-3-Clause"
  },
  "node_modules/@protobufjs/fetch": {
    "version": "1.1.0",
```



```
"resolved": "https://registry.npmjs.org/@protobufjs/fetch/-/fetch-1.1.0.tgz",

"integrity": "sha512-lljVXpqXebpsijW71PZaCYelcE5on1w5DIQy5WH6GLbFryLUrBD4932W/E2BSpfRJWseIL4v/KPgBFxD0IdKpQ==",

"license": "BSD-3-Clause",

"dependencies": {

  "@protobufjs/aspromise": "^1.1.1",

  "@protobufjs/inquire": "^1.1.0"

}

},

"node_modules/@protobufjs/float": {

  "version": "1.0.2",

  "resolved": "https://registry.npmjs.org/@protobufjs/float/-/float-1.0.2.tgz",

  "integrity": "sha512-Ddb+kVXISt9d+R9PfTlxh1EdNkgoRe5tOX6t01f1lYW0vJnSPDBlG241QLzcyPdoNTsblLUdujGSE4RzrTZGQ==",

  "license": "BSD-3-Clause"

},

"node_modules/@protobufjs/inquire": {

  "version": "1.1.0",

  "resolved": "https://registry.npmjs.org/@protobufjs/inquire/-/inquire-1.1.0.tgz",

  "integrity": "sha512-kdSefcPdruJiFMVSbn801t4vFK7KB/5gd2fYvrXhuJYg8ILrmn9SKSX2tZdV6V+ksulWqS7aXjBcRXL3wHoD9Q==",

  "license": "BSD-3-Clause"

},

"node_modules/@protobufjs/path": {

  "version": "1.1.2",
```

```
"resolved": "https://registry.npmjs.org/@protobufjs/path/-/path-1.1.2.tgz",
  "integrity": "sha512-6JOcJ5Tm08dOHAbdR3GrvP+yUUfkiG5ePsHYczMFLq3ZmMkAD98cDgcT2iA1U9NVwFd4tH/iSSoe44YWkltEA==",
  "license": "BSD-3-Clause"
},
"node_modules/@protobufjs/pool": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/@protobufjs/pool/-/pool-1.1.0.tgz",
  "integrity": "sha512-0kELaGSIDBKvcgS4zkjz1PeddatrjYcmMWOLAuAPWAeccUrPHdUqo/J6LiymHHEiJT5NrF1UVwxY14f+fy4WQw==",
  "license": "BSD-3-Clause"
},
"node_modules/@protobufjs/utf8": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/@protobufjs/utf8/-/utf8-1.1.0.tgz",
  "integrity": "sha512-Vvn3zZrhQZkkBE8LSuW3em98c0FwgO4nxzv6OdSxPKJIEKY2bGbHn+mhGIPerzl4twdxap8/0+06HBpwf345Lw==",
  "license": "BSD-3-Clause"
},
"node_modules/@tootallnate/once": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/@tootallnate/once/-/once-2.0.0.tgz",
  "integrity": "sha512-XCuKFP5PS55gnMVu3dty8KPatLqUoy/ZYzDzAGCQ8JNFCkLXzmi7vNHCR+XpbZaMWQK/vQubr7PkYq8g470J/A==",

```

```
"license": "MIT",

"optional": true,

"engines": {

  "node": ">= 10"

}

},

"node_modules/@types/body-parser": {

  "version": "1.19.6",

  "resolved": "https://registry.npmjs.org/@types/body-parser/-/body-parser-1.19.6.tgz",

  "integrity": "sha512-HLFeCYgz89uk22N5Qg3dvGvsv46B8GLvKKo1zKG4NybA8U2DiEO3w9lqGg29t/tfLRJpJ6iQxnVw4OnB7MoM9g==",

  "license": "MIT",

  "dependencies": {

    "@types/connect": "*",

    "@types/node": "*"

  }

},

"node_modules/@types/caseless": {

  "version": "0.12.5",

  "resolved": "https://registry.npmjs.org/@types/caseless/-/caseless-0.12.5.tgz",

  "integrity": "sha512-hWtVTC2q7hc7xZ/RLbxapMvDMgUnDvKvMOpKal4DrMyfGBUfB1oKaZlIRr6mJL+lf3bAP6sV/QneGzF6tJjZDg==",

  "license": "MIT",

  "optional": true

},
```

```
"node_modules/@types/connect": {
  "version": "3.4.38",
  "resolved": "https://registry.npmjs.org/@types/connect/-/connect-3.4.38.tgz",
  "integrity": "sha512-K6uROf1LD88uDQqJCktA4yzL1YYAK6NgfsI0v/mTgyPKWsX1CnJ0XPSDhViejru1GcRkLWb8RlzFYJRqGUbaug==",
  "license": "MIT",
  "dependencies": {
    "@types/node": "*"
  }
},
"node_modules/@types/cors": {
  "version": "2.8.19",
  "resolved": "https://registry.npmjs.org/@types/cors/-/cors-2.8.19.tgz",
  "integrity": "sha512-mFNylyeyqN93lfe/9CSxOGREz8cpzAhH+E93xJ4xWQf62V8sQ/24reV2nyzUWM6H6Xji+GGHpkbLe7pVoUEskg==",
  "license": "MIT",
  "dependencies": {
    "@types/node": "*"
  }
},
"node_modules/@types/esrecurse": {
  "version": "4.3.1",
  "resolved": "https://registry.npmjs.org/@types/esrecurse/-/esrecurse-4.3.1.tgz",
  "integrity": "sha512-xJBAbDifo5hpffDBuHI0Y8ywswbiAp/Wi7Y/GtAgSlZylABpppyurxVueOPE8LUQOxdlgi6Zqce7uoEpqNteiUw==",
```

```
"dev": true,

"license": "MIT"

},

"node_modules/@types/estree": {

  "version": "1.0.8",

  "resolved": "https://registry.npmjs.org/@types/estree/-/estree-1.0.8.tgz",

  "integrity": "sha512-dWHzHa2WqEXI/O1E9OjrocMTKJl2mSrEolh1lomrv6U+JuNwaHXsXx9bLu5gG7BUWFIN0skIQJQ/L1rlex4X6w==",

  "dev": true,

  "license": "MIT"

},

"node_modules/@types/express": {

  "version": "4.17.25",

  "resolved": "https://registry.npmjs.org/@types/express/-/express-4.17.25.tgz",

  "integrity": "sha512-dVd04UKsfpINUnK0yBoYHDF3xu7xVH4BuDotC/xGuycx4CgbP48X/KF/586bcObxT0HENHXEU8Nqtu6NR+eKhW==",

  "license": "MIT",

  "dependencies": {

    "@types/body-parser": "*",

    "@types/express-serve-static-core": "^4.17.33",

    "@types/qs": "*",

    "@types/serve-static": "^1"

  }

},

"node_modules/@types/express-serve-static-core": {
```

```
"version": "4.19.8",

"resolved": "https://registry.npmjs.org/@types/express-serve-static-core/-/express-serve-static-core-4.19.8.tgz",

"integrity": "sha512-02S5fmqeoKzVZCHPZid4b8JH2eM5HzQLZWN2FohQEY/0eXTq8VXZfSN6Pcr3F6N9R/vNrj7cpgbhjie6m/1tCA==",

"license": "MIT",

"dependencies": {

  "@types/node": "*",

  "@types/qs": "*",

  "@types/range-parser": "*",

  "@types/send": "*"

},

"node_modules/@types/http-errors": {

  "version": "2.0.5",

  "resolved": "https://registry.npmjs.org/@types/http-errors/-/http-errors-2.0.5.tgz",

  "integrity": "sha512-r8Tayk8HJnX0FztbZN7oVqGccWgw98T/0neJphO91KkmOzug1KkofZURD4UaD5uH8AqcFLf dPErnBod0u71/qg==",

  "license": "MIT"

},

"node_modules/@types/json-schema": {

  "version": "7.0.15",

  "resolved": "https://registry.npmjs.org/@types/json-schema/-/json-schema-7.0.15.tgz",

  "integrity": "sha512-5+fP8P8MFNC+AyZCDxrB2pkZFPgzqQWUzpSeuuVLvm8VMcorNYavBqoFcxK8bQz4Qsbn4oUEEem4wDLfcysGHA==",
```

```
"dev": true,

"license": "MIT"

},

"node_modules/@types/jsonwebtoken": {

  "version": "9.0.10",

  "resolved": "https://registry.npmjs.org/@types/jsonwebtoken/-/jsonwebtoken-9.0.10.tgz",

  "integrity": "sha512-asx5hIG9Qmf/1oStypjanR7iKTv0gXQ1Ov/jfrX6kS/EO0OFni8orbmgCn0672NHR3kXHwpAwR+B368ZGN/2rA==",

  "license": "MIT",

  "dependencies": {

    "@types/ms": "*",

    "@types/node": "*"

  }

},

"node_modules/@types/lodash": {

  "version": "4.17.24",

  "resolved": "https://registry.npmjs.org/@types/lodash/-/lodash-4.17.24.tgz",

  "integrity": "sha512-glW7lQLZbue7lRSWEFql49QJJWThrTFFeIMJdp3eH4tKoxm1OvEPg02rm4wCCSHS0cL3/Fizi mb35b7k8atwsQ==",

  "dev": true,

  "license": "MIT"

},

"node_modules/@types/long": {

  "version": "4.0.2",
```

```
"resolved": "https://registry.npmjs.org/@types/long/-/long-4.0.2.tgz",

"integrity": "sha512-MqTGEo5bj5t157U6fA/BiDynNkn0YknVdh48CMPkTSpFTVmvao5UQmm7uEF6xBEo7qIMAlY/JSlEYaE6VOdpaA==",

"license": "MIT",

"optional": true

},

"node_modules/@types/mime": {

"version": "1.3.5",

"resolved": "https://registry.npmjs.org/@types/mime/-/mime-1.3.5.tgz",

"integrity": "sha512-/pyBZWSLD2n0dcHE3hq8s8ZvcETHtEuF+3E7XVt0lg2nvsVQXdghHVcEkIWjy9A0wKfTn97a/PSDYohKIlnP/w==",

"license": "MIT"

},

"node_modules/@types/ms": {

"version": "2.1.0",

"resolved": "https://registry.npmjs.org/@types/ms/-/ms-2.1.0.tgz",

"integrity": "sha512-GsCCIZDE/p3i96vtEqx+7dBUGXrc7zeSK3wwPHIaRThS+9OhWIXRqzs4d6k1SVU8g91DrNRWxWUGhp5KXQb2VA==",

"license": "MIT"

},

"node_modules/@types/node": {

"version": "22.19.11",

"resolved": "https://registry.npmjs.org/@types/node/-/node-22.19.11.tgz",
```



```
  "integrity": "sha512-
BH7YwL6rA93ReqeQS1c4bsPpcfOmJasG+Fkr6Y59q83f9M1WcBRHR2vM+P9eOisYRcN3uj
QoiZY8uk5W+1WL8w==",

  "license": "MIT",

  "dependencies": {

    "undici-types": "~6.21.0"

  },
},

"node_modules/@types/qs": {

  "version": "6.14.0",

  "resolved": "https://registry.npmjs.org/@types/qs/-/qs-6.14.0.tgz",

  "integrity": "sha512-
eOunJqu0K1923aExK6y8p6fsihYEn/BYuQ4g0CxAAgFc4b/ZLN4CrsRZ55srTdqoiLzU2B2evC
+apElxprEzkQ==",

  "license": "MIT"

},

"node_modules/@types/range-parser": {

  "version": "1.2.7",

  "resolved": "https://registry.npmjs.org/@types/range-parser/-/range-parser-1.2.7.tgz",

  "integrity": "sha512-
hKormJbkJqzQGhziAx5PltDUTMAM9uE2XXQmM37dyd4hVM+5aVl7oVxMVUiVQn2oCQFN/L
KCZdvSM0pFRqbSmQ==",

  "license": "MIT"

},

"node_modules/@types/request": {

  "version": "2.48.13",

  "resolved": "https://registry.npmjs.org/@types/request/-/request-2.48.13.tgz",
```

```
  "integrity": "sha512-
FGJ6udDNUCjd19pp0Q3iTidkwhYup7J8hpMW9c4k53NrccQFFWKRho6hvtPPEhnXWKvukf
wAlB6DbDz4yhH5Gg==",

  "license": "MIT",

  "optional": true,

  "dependencies": {

    "@types/caseless": "*",

    "@types/node": "*",

    "@types/tough-cookie": "*",

    "form-data": "^2.5.5"

  }

},

"node_modules/@types/send": {

  "version": "1.2.1",

  "resolved": "https://registry.npmjs.org/@types/send/-/send-1.2.1.tgz",

  "integrity": "sha512-
arsCikDvlU99zl1g69TcAB3mzZPpxgw0UQnaHeC1Nwb015xp8bknZv5rlfri9xTOcMuaVgvabfl
RA7PSZVuZIQ==",

  "license": "MIT",

  "dependencies": {

    "@types/node": "*"

  }

},

"node_modules/@types/serve-static": {

  "version": "1.15.10",

  "resolved": "https://registry.npmjs.org/@types/serve-static/-/serve-static-1.15.10.tgz",
```

```
"integrity": "sha512-  
tRs1dB+g8ltk72rlSI2ZrW6vZg0YrLI81iQSTkMmOqnqCaNr/8Ek4VwWcN5vZgCYWbg/JJSGBI  
UaYGAOP73qBw==",
```

```
"license": "MIT",
```

```
"dependencies": {
```

```
"@types/http-errors": "*",
```

```
"@types/node": "*",
```

```
"@types/send": "<1"
```

```
}
```

```
},
```

```
"node_modules/@types/serve-static/node_modules/@types/send": {
```

```
"version": "0.17.6",
```

```
"resolved": "https://registry.npmjs.org/@types/send/-/send-0.17.6.tgz",
```

```
"integrity": "sha512-  
Uqt8rPBE8SY0RK8JB1EzVOIZ32uqy8HwdxCnoCOsYrvnswqmFZ/k+9lkidlk/ImhsdvBsloHbA  
lewb2IEBV/Og==",
```

```
"license": "MIT",
```

```
"dependencies": {
```

```
"@types/mime": "^1",
```

```
"@types/node": "*"
```

```
}
```

```
},
```

```
"node_modules/@types/tough-cookie": {
```

```
"version": "4.0.5",
```

```
"resolved": "https://registry.npmjs.org/@types/tough-cookie/-/tough-cookie-4.0.5.tgz",
```

```
"integrity": "sha512-  
/Ad8+nIOV7Rl++6f1BdKxFSMgmoqEoYbHRpPcx3JEfv8VRsQe9Z4mCXeJBzxs7mbHY/XOZZu  
XlRNfhpVPbs6ZA==",
```

```
"license": "MIT",
"optional": true
},
"node_modules/abort-controller": {
  "version": "3.0.0",
  "resolved": "https://registry.npmjs.org/abort-controller/-/abort-controller-3.0.0.tgz",
  "integrity": "sha512-l99XudHd/6+yFLvOj1w4uIZsppHGT3GgEKhYW66GdhmI0cBbZ+VCon1WjRhhSfG4Xo2YnZTUhspk1DwnpVVvw==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "event-target-shim": "^5.0.0"
  },
  "engines": {
    "node": ">=6.5"
  }
},
"node_modules/accepts": {
  "version": "1.3.8",
  "resolved": "https://registry.npmjs.org/accepts/-/accepts-1.3.8.tgz",
  "integrity": "sha512-NDAYCfW+ZdFIA0E8H5B1ZnWY0w80wYuzCQH5IcP4QPMa0tN5UeyVrP4gU6Njv2H6VwOX1gZvOh0WLlV9w==",
  "license": "MIT",
  "dependencies": {
    "mime-types": "~2.1.34",

```

```
    "negotiator": "0.6.3"
  },
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/acorn": {
  "version": "8.16.0",
  "resolved": "https://registry.npmjs.org/acorn/-/acorn-8.16.0.tgz",
  "integrity": "sha512-UVJyE9MttOsBQIDKw1skb9nAwQuR5wuGD3+82K6JgJlm/Y+KI92oNsMNGZCYdDsVtRHSak0pcV5Dno5+4jh9sw==",
  "dev": true,
  "license": "MIT",
  "bin": {
    "acorn": "bin/acorn"
  },
  "engines": {
    "node": ">=0.4.0"
  }
},
"node_modules/acorn-jsx": {
  "version": "5.3.2",
  "resolved": "https://registry.npmjs.org/acorn-jsx/-/acorn-jsx-5.3.2.tgz",
  "integrity": "sha512-1q1130303308213e39d1968f2024582303e536136378e00b92984d71c32400c6",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "acorn": "^8.0.0"
  }
}
```

```
"dev": true,

"license": "MIT",

"peerDependencies": {

  "acorn": "^6.0.0 || ^7.0.0 || ^8.0.0"

},

"node_modules/agent-base": {

  "version": "7.1.4",

  "resolved": "https://registry.npmjs.org/agent-base/-/agent-base-7.1.4.tgz",

  "integrity": "sha512-MnA+YT8fwfJPgBx3m60MNqakm30XOkvloH1y6huTQvC0PwZG7ki8NacLBcrPbNoo8vEZy7Jpuk7+jMO+CUovTQ==",

  "license": "MIT",

  "engines": {

    "node": ">= 14"

  }

},

"node_modules/ajv": {

  "version": "6.14.0",

  "resolved": "https://registry.npmjs.org/ajv/-/ajv-6.14.0.tgz",

  "integrity": "sha512-IWrosm/yrn43eiKqkfkhHis7QioDleaXQHdDVpKg0FSwwd/DuvyX79TZnFOnYpB7dcsFAMmtFztZuXPdVSePkFw==",

  "dev": true,

  "license": "MIT",

  "dependencies": {

    "fast-deep-equal": "^3.1.1",
```

```
"fast-json-stable-stringify": "^2.0.0",
"json-schema-traverse": "^0.4.1",
"uri-js": "^4.2.2"
},
"funding": {
  "type": "github",
  "url": "https://github.com/sponsors/epoberezkin"
}
},
"node_modules/ansi-regex": {
  "version": "5.0.1",
  "resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-5.0.1.tgz",
  "integrity": "sha512-UrgsinAqurLdYhmFhIaR9t130Zp4H4UzYtdM0N3sRy4G4JtN7P5boeY5qWdU7XNwM1K6lE6r9+N5YSJ1KQ==",
  "license": "MIT",
  "engines": {
    "node": ">=8"
  }
},
"node_modules/ansi-styles": {
  "version": "4.3.0",
  "resolved": "https://registry.npmjs.org/ansi-styles/-/ansi-styles-4.3.0.tgz",
  "integrity": "sha512-Bb3U17u70Fz76UL31QVFyT5zVW9G4KfEln0lLLDp44Hj0UDUbnk1Q7oF33DnG693cl0zPaMbovsgVySP0A==",
  "license": "MIT",
```

```
"dependencies": {
  "color-convert": "^2.0.1"
},
"engines": {
  "node": ">=8"
},
"funding": {
  "url": "https://github.com/chalk/ansi-styles?sponsor=1"
}
},
"node_modules/array-flatten": {
  "version": "1.1.1",
  "resolved": "https://registry.npmjs.org/array-flatten/-/array-flatten-1.1.1.tgz",
  "integrity": "sha512-S38UwT83ZM00N/gHkoOOSB8td7Lg9e2kDqV82fZ1X1s68YR4YvDANiAjy+rCUHaEy7Yr/TYmP58nKwupZA==",
  "license": "MIT"
},
"node_modules/arrify": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/arrify/-/arrify-2.0.1.tgz",
  "integrity": "sha512-31uK+Lsr5vC5sFRX3r3nRzHqqe0Jsq+nfXSQn3Ew9J913N7iJDNqsr4ZhIdK97ziICXrqLWscF52UeHF7lYvZg==",
  "license": "MIT",
  "optional": true,
  "engines": {
```



```
    "node": ">=8"
  }
},
"node_modules/async-retry": {
  "version": "1.3.3",
  "resolved": "https://registry.npmjs.org/async-retry/-/async-retry-1.3.3.tgz",
  "integrity": "sha512-wfr/jstw9xNi/0teMHRW7dsz3Lt5ARhYNZ2ewpadnhalp5mbALhOAP+EAdsC7t4Z6wqsDVv9+W6gm1Dk9mEyw==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "retry": "0.13.1"
  }
},
"node_modules/asynckit": {
  "version": "0.4.0",
  "resolved": "https://registry.npmjs.org/asynckit/-/asynckit-0.4.0.tgz",
  "integrity": "sha512-Oei9OH4tRh0YqU3GxhX79dM/mwVgvbZJaSNaRk+bshkj0S5cfHcgYakreBjrHwatXKbz+IoldYLxrKim2MjW0Q==",
  "license": "MIT",
  "optional": true
},
"node_modules/balanced-match": {
  "version": "4.0.4",
  "resolved": "https://registry.npmjs.org/balanced-match/-/balanced-match-4.0.4.tgz",
```

```
  "integrity": "sha512-
BLrgEcRTwX2o6gGxGOCNyMvGSp35YofuYzw9h1IMTRmKqttAZZVU67bdb9Pr2vUHA8+j3i2t
JfjO6C6+4myGTA==",

  "license": "MIT",

  "engines": {

    "node": "18 || 20 || >=22"

  },

  "node_modules/base64-js": {

    "version": "1.5.1",

    "resolved": "https://registry.npmjs.org/base64-js/-/base64-js-1.5.1.tgz",

    "integrity": "sha512-
AKpaYlHn8t4SVbOHCy+b5+KKgvR4vrsD8vbvrbiQJps7fKDTkjkDry6ji0rUJjC0kzbNePLwzxq8i
ypo41qeWA==",

    "funding": [

      {

        "type": "github",

        "url": "https://github.com/sponsors/feross"

      },

      {

        "type": "patreon",

        "url": "https://www.patreon.com/feross"

      },

      {

        "type": "consulting",

        "url": "https://feross.org/support"

      }

    ]

  }

}
```

```
  ],
  "license": "MIT"
},
"node_modules/bignumber.js": {
  "version": "9.3.1",
  "resolved": "https://registry.npmjs.org/bignumber.js/-/bignumber.js-9.3.1.tgz",
  "integrity": "sha512-Ko0uX15oIU7wJ3Rb30Fs6SkVbLmPBAKdlm7q9+ak9bbbleFf0MwuBsQV6z7+X768/cHsfg+WlysDWJcmthjsjQ==",
  "license": "MIT",
  "engines": {
    "node": "*"
  }
},
"node_modules/body-parser": {
  "version": "1.20.4",
  "resolved": "https://registry.npmjs.org/body-parser/-/body-parser-1.20.4.tgz",
  "integrity": "sha512-SRo03U+Jd/sKm/Sdi/3WkjZVAh49p8UgJPd4aknQV1JiL8I+/1U0tkq0xF8cDy0WhewoBDI5nI5JehhZ87w==",
  "license": "MIT",
  "dependencies": {
    "bytes": "~3.1.2",
    "content-type": "~1.0.5",
    "debug": "2.6.9",
    "depd": "2.0.0",
    "destroy": "~1.2.0",

```

```
"http-errors": "~2.0.1",
"iconv-lite": "~0.4.24",
"on-finished": "~2.4.1",
"qs": "~6.14.0",
"raw-body": "~2.5.3",
"type-is": "~1.6.18",
"unpipe": "~1.0.0"
},
"engines": {
  "node": ">= 0.8",
  "npm": "1.2.8000 || >= 1.4.16"
}
},
"node_modules/body-parser/node_modules/debug": {
  "version": "2.6.9",
  "resolved": "https://registry.npmjs.org/debug/-/debug-2.6.9.tgz",
  "integrity": "sha512-bC7ElrdJaJnPbAP+1EotYvqZsb3ecl5wi6Bfi6BJTUcNowp6cvspg0jXznRTKDjm/E7AdgFBVeAPVMNcKGsHMA==",
  "license": "MIT",
  "dependencies": {
    "ms": "2.0.0"
  }
},
"node_modules/body-parser/node_modules/ms": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/ms/-/ms-2.0.0.tgz",
```

```
    "integrity": "sha512-
Tpp60P6IUJDTuOq/5Z8cdskzJujfwqfOTkrwLwj7IRISpnkJnT6SyJ4PCPnGMoFjC9ddhal5KVIYt
At97ix05A==",

    "license": "MIT"

  },

  "node_modules/brace-expansion": {

    "version": "5.0.3",

    "resolved": "https://registry.npmjs.org/brace-expansion/-/brace-expansion-5.0.3.tgz",

    "integrity": "sha512-
fy6KJm2RawA5RcHkLa1z/ScpBeA762UF9KmZQxwlbDtRJgLzM10depAiEQ+CXycoiqW1/m
96OAAoke2nE9EeA==",

    "license": "MIT",

    "dependencies": {

      "balanced-match": "^4.0.2"

    },

    "engines": {

      "node": "18 || 20 || >=22"

    }

  },

  "node_modules/buffer-equal-constant-time": {

    "version": "1.0.1",

    "resolved": "https://registry.npmjs.org/buffer-equal-constant-time/-/buffer-equal-
constant-time-1.0.1.tgz",

    "integrity": "sha512-
zRpUiDwd/xk6ADqPMATG8vc9VPrkck7T07OlX0gnjmJAnHnTVXNQG3vfvWNuiZlkwu9KrKdA
1iJKfsfTVxE6NA==",

    "license": "BSD-3-Clause"

  },
```

```
"node_modules/bytes": {
  "version": "3.1.2",
  "resolved": "https://registry.npmjs.org/bytes/-/bytes-3.1.2.tgz",
  "integrity": "sha512-1f8p06Y+D45WV8g2i2M4JYr062T4JZu3D8J41W6TwyD4IZuTOz3Z5XWV7EX6Hr6zQp95fDQ8gIwIc0Q==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  }
},
"node_modules/call-bind-apply-helpers": {
  "version": "1.0.2",
  "resolved": "https://registry.npmjs.org/call-bind-apply-helpers/-/call-bind-apply-helpers-1.0.2.tgz",
  "integrity": "sha512-+hWZ/clwV84LgFmz3L3uB4G8iS5Y0uB1d8+K2881HLL9aOa2Q8W7S2CwJ8z0Ea6qCv18z06dUxO36TA==",
  "license": "MIT",
  "dependencies": {
    "es-errors": "^1.3.0",
    "function-bind": "^1.1.2"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
```

```
"node_modules/call-bound": {
  "version": "1.0.4",
  "resolved": "https://registry.npmjs.org/call-bound/-/call-bound-1.0.4.tgz",
  "integrity": "sha512-+ys997U96po4Kx/ABpBCqhA9EuxJaQWDQg7295H4hBphv3lZg0boBKuwYpt4YXp6MZ5AmZQnU/tyMTlRpaSejg==",
  "license": "MIT",
  "dependencies": {
    "call-bind-apply-helpers": "^1.0.2",
    "get-intrinsic": "^1.3.0"
  },
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/cliui": {
  "version": "8.0.1",
  "resolved": "https://registry.npmjs.org/cliui/-/cliui-8.0.1.tgz",
  "integrity": "sha512-BSeNnyus75C4//NQ9gQt1/csTXyo/8Sb+afLakzAptFuMsod9HFokGNudZpi/oQV73hnVK+sR+5PVRMd+Dr7YQ==",
  "license": "ISC",
  "optional": true,
  "dependencies": {
```

```
"string-width": "^4.2.0",
"strip-ansi": "^6.0.1",
"wrap-ansi": "^7.0.0"
},
"engines": {
  "node": ">=12"
}
},
"node_modules/cliui/node_modules/emoji-regex": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-8.0.0.tgz",
  "integrity": "sha512-MSjYzcWNOA0ewAHpz0MxpYFvwg6yjy1NG3xteoqz644VCo/RPgnr1/GGt+ic3iJTzQ8Eu3TdM14SawnVUmGE6A==",
  "license": "MIT",
  "optional": true
},
"node_modules/cliui/node_modules/string-width": {
  "version": "4.2.3",
  "resolved": "https://registry.npmjs.org/string-width/-/string-width-4.2.3.tgz",
  "integrity": "sha512-wKyQRQpjJ0sIp62ErSZdGsjMJWsap5oRNihHhu6G7JVO/9jIB6UyevL+tXuOqrng8j/cxKTWwWUwvSTrriZz/g==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "emoji-regex": "^8.0.0",

```



```
"is-fullwidth-code-point": "^3.0.0",
"strip-ansi": "^6.0.1"
},
"engines": {
  "node": ">=8"
}
},
"node_modules/cliui/node_modules/wrap-ansi": {
  "version": "7.0.0",
  "resolved": "https://registry.npmjs.org/wrap-ansi/-/wrap-ansi-7.0.0.tgz",
  "integrity": "sha512-YVGIj2kamLSTxw6NsZjoBxfSwsn0ycdesmc4p+Q21c5zPuZ1pl+NfxVdxPtdHvmNVOQ6XSYG4AUtyt/Fi7D16Q==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "ansi-styles": "^4.0.0",
    "string-width": "^4.1.0",
    "strip-ansi": "^6.0.0"
  },
  "engines": {
    "node": ">=10"
  },
  "funding": {
    "url": "https://github.com/chalk/wrap-ansi?sponsor=1"
  }
},
```

```
"node_modules/color-convert": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/color-convert/-/color-convert-2.0.1.tgz",
  "integrity": "sha512-RRECPsj7iu/xb5oKYcsFHSppFNnsj/52OVTRKb4zP5onXwVF3zVmmToNcOfGC+CRDpfK/U584fMg38ZHCaElKQ==",
  "license": "MIT",
  "dependencies": {
    "color-name": "~1.1.4"
  },
  "engines": {
    "node": ">=7.0.0"
  }
},
"node_modules/color-name": {
  "version": "1.1.4",
  "resolved": "https://registry.npmjs.org/color-name/-/color-name-1.1.4.tgz",
  "integrity": "sha512-dOy+3AuW3a2wNbZHIuMZpTcgjGuLU/uBL/ubcZF9OXbDo8ff4O8yVp5Bf0efS8uEoYo5q4Fx7dY9OgQGXgAsQA==",
  "license": "MIT"
},
"node_modules/combined-stream": {
  "version": "1.0.8",
  "resolved": "https://registry.npmjs.org/combined-stream/-/combined-stream-1.0.8.tgz",
  "integrity": "sha512-jQkuOcb7b4AIPbds9EB/7DAjS2kvcBSzFekPbOwvx8QxfhUO+K/dNSzb7JRy6CQymAMqOw+ohRDHSqLwF4yq2w==",
  "license": "MIT",
  "dependencies": {
    "delayed-stream": "~1.0.0"
  }
}
```

```
"license": "MIT",
"optional": true,
"dependencies": {
  "delayed-stream": "~1.0.0"
},
"engines": {
  "node": ">= 0.8"
}
},
"node_modules/content-disposition": {
  "version": "0.5.4",
  "resolved": "https://registry.npmjs.org/content-disposition/-/content-disposition-0.5.4.tgz",
  "integrity": "sha512-FveZTNuGw04cxIAiWbzi6zTAL/lhehaWbTtgluJh4/E95DqMwTmha3KZN1aAWA8cFIhHzMZUvLevkw5Rqk+tSQ==",
  "license": "MIT",
  "dependencies": {
    "safe-buffer": "5.2.1"
  },
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/content-type": {
  "version": "1.0.5",
  "resolved": "https://registry.npmjs.org/content-type/-/content-type-1.0.5.tgz",
```

```
  "integrity": "sha512-nTjqfcBFEipKdXCv4YDQWCfmcLZKm81ldF0pAopTvyrFGVbcR6P/VAAd5G7N+0tTr8QqiU0tFadD6FK4NtJwOA==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 0.6"
```

```
  }
```

```
},
```

```
"node_modules/cookie": {
```

```
  "version": "0.7.2",
```

```
  "resolved": "https://registry.npmjs.org/cookie/-/cookie-0.7.2.tgz",
```

```
  "integrity": "sha512-yki5XnKuf750l50uGTllt6kKILY4nQ1eNIQatoXEBByZ5dWgnKqbnqmTrBE5B4N7lrMJKQ2ytWMiTO2o0v6Ew/w==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 0.6"
```

```
  }
```

```
},
```

```
"node_modules/cookie-signature": {
```

```
  "version": "1.0.7",
```

```
  "resolved": "https://registry.npmjs.org/cookie-signature/-/cookie-signature-1.0.7.tgz",
```

```
  "integrity": "sha512-NXdYc3dLr47pBkpUCHtKSwlOQXLVn8dZEuywboCOJY/osA0wFSLlSawr3KN8qXJEyX66FcONTH8EIlVuK0yyFA==",
```

```
  "license": "MIT"
```

```
},
```

```
"node_modules/cors": {
```

```
"version": "2.8.6",

"resolved": "https://registry.npmjs.org/cors/-/cors-2.8.6.tgz",

"integrity": "sha512-tJtZBBHA6vjIAaF6Enlaq6laBBP9aq/Y3ouVJjEfoHbRBcHBAHYcMh/w8LDrk2PvIMMq8gmopa
5D4V8RmbrxGw==",

"license": "MIT",

"dependencies": {

  "object-assign": "^4",

  "vary": "^1"

},

"engines": {

  "node": ">= 0.10"

},

"funding": {

  "type": "opencollective",

  "url": "https://opencollective.com/express"

}

},

"node_modules/cross-spawn": {

  "version": "7.0.6",

  "resolved": "https://registry.npmjs.org/cross-spawn/-/cross-spawn-7.0.6.tgz",

  "integrity": "sha512-uV2QOWP2nWzsy2aMp8aRibhi9dlzF5Hgh5SHaB9OiTGEyDTiJJyx0uy51QXdyWbtAHNua4XJ
zUKca3OzKUd3vA==",

  "license": "MIT",

  "dependencies": {

    "path-key": "^3.1.0",
```

```
"shebang-command": "^2.0.0",
"which": "^2.0.1"
},
"engines": {
  "node": ">= 8"
}
},
"node_modules/data-uri-to-buffer": {
  "version": "4.0.1",
  "resolved": "https://registry.npmjs.org/data-uri-to-buffer/-/data-uri-to-buffer-4.0.1.tgz",
  "integrity": "sha512-0R9ikRb668HB7QDxT1vkpuUBtqc53YyAwMwGeUfKRojY/NWKvdZ+9UYtRfGmhqNbRkTSVpMbmyhXipFFv2cb/A==",
  "license": "MIT",
  "engines": {
    "node": ">= 12"
  }
},
"node_modules/debug": {
  "version": "4.4.3",
  "resolved": "https://registry.npmjs.org/debug/-/debug-4.4.3.tgz",
  "integrity": "sha512-RGwwWnwQvkVfavKVt22FGLw+xYSdzARwm0ru6DhTVA3umU5hZc28V3kO4stgYryrTILpuvG19GiiJltAjNbcqA==",
  "license": "MIT",
  "dependencies": {
    "ms": "^2.1.3"
```

```
,
"engines": {
  "node": ">=6.0"
},
"peerDependenciesMeta": {
  "supports-color": {
    "optional": true
  }
}
},
"node_modules/deep-is": {
  "version": "0.1.4",
  "resolved": "https://registry.npmjs.org/deep-is/-/deep-is-0.1.4.tgz",
  "integrity": "sha512-oIPzksmTg4/MriiaYGO+okXDT7ztn/w3Eptv/+gSIldMdKsJo0u4CfYNFJPy+4SKMuCqGw2wxnA+URMg3t8a/bQ==",
  "dev": true,
  "license": "MIT"
},
"node_modules/delayed-stream": {
  "version": "1.0.0",
  "resolved": "https://registry.npmjs.org/delayed-stream/-/delayed-stream-1.0.0.tgz",
  "integrity": "sha512-ZySD7Nf91aLB0RxL4KGrKHBXl7Eds1DAmEdcoVawXnLD7SDhpNgtuII2aAkg7a7QS41jxPSZ17p4VdGnMHk3MQ==",
  "license": "MIT",
  "optional": true,
```

```
"engines": {
  "node": ">=0.4.0"
},
"node_modules/depd": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/depd/-/depd-2.0.0.tgz",
  "integrity": "sha512-g7nH6P6dyDioJogAAGprGpCtVImJhpPk/roCzdb3flh61/s/nPsfR6onyMwkCAR/OlC3yBC0lESvUoQEAsslrw==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  },
"node_modules/destroy": {
  "version": "1.2.0",
  "resolved": "https://registry.npmjs.org/destroy/-/destroy-1.2.0.tgz",
  "integrity": "sha512-2sJGJTtXIIlaR1w4iJSNoN0hnMY7Gpc/n8D4qSCJw8QqFWXf7cuAgnEHxBpweaVcPevC2l3KpjYCx3NypQQgaJg==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8",
    "npm": "1.2.8000 || >= 1.4.16"
  },
}
```



```
"node_modules/dotenv": {
  "version": "17.3.1",
  "resolved": "https://registry.npmjs.org/dotenv/-/dotenv-17.3.1.tgz",
  "integrity": "sha512-IO8C/dzEb6O3F9/twg6ZLXz164a2fhTnEWb95H23Dm4OuN+92NmEAlTrupP9VW6Jm3sO26tQlqyvvi4CsnY9GA==",
  "license": "BSD-2-Clause",
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "https://dotenvx.com"
  }
},
"node_modules/dunder-proto": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/dunder-proto/-/dunder-proto-1.0.1.tgz",
  "integrity": "sha512-KIN/nDJBQRcXw0MLVhZE9iQHmG68qAVIBg9CqmUYjmQlhgij9U5MFvrqkUL5FbtyyzZuOeOt0zdeRe4UY7ct+A==",
  "license": "MIT",
  "dependencies": {
    "call-bind-apply-helpers": "^1.0.1",
    "es-errors": "^1.3.0",
    "gopd": "^1.2.0"
  },
  "engines": {
```

```
"node": ">= 0.4"
}
},
"node_modules/duplexify": {
  "version": "4.1.3",
  "resolved": "https://registry.npmjs.org/duplexify/-/duplexify-4.1.3.tgz",
  "integrity": "sha512-0330187a0727/29KdNc06cA14Fa/LS86L/7VVt5bLXN1oDqJnQ3UCjZgP77Z6g9b8KEQc1gjHKBZIw+Xt9eY=",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "end-of-stream": "^1.4.1",
    "inherits": "^2.0.3",
    "readable-stream": "^3.1.1",
    "stream-shift": "^1.0.2"
  }
},
"node_modules/eastasianwidth": {
  "version": "0.2.0",
  "resolved": "https://registry.npmjs.org/eastasianwidth/-/eastasianwidth-0.2.0.tgz",
  "integrity": "sha512-S48wHbA+wK9rd7aq6UgfusCk0CjZC+ZgJmIuQdWlH1CLC8uOznrmvAw6HErEgRHPRqdDRmlh20IgiQZnUMw=",
  "license": "MIT"
},
"node_modules/ecdsa-sig-formatter": {
```

```
"version": "1.0.11",

"resolved": "https://registry.npmjs.org/ecdsa-sig-formatter/-/ecdsa-sig-formatter-1.0.11.tgz",

"integrity": "sha512-nagl3RYrbNv6kQkeJlpt6NJZy8twLB/2vtz6yN9Z4vRKHN4/QZJIEbqohALSgwKdnksuY3k5Addp5lg8sVoVcQ==",

"license": "Apache-2.0",

"dependencies": {

  "safe-buffer": "^5.0.1"

},

},

"node_modules/ee-first": {

  "version": "1.1.1",

  "resolved": "https://registry.npmjs.org/ee-first/-/ee-first-1.1.1.tgz",

  "integrity": "sha512-WMwm9LhRUo+WUaRN+vRuETqG89lgZphVSNkdFgeb6sS/E4OrDIN7t48CAewSHXc6C8lefD8KKfr5vY61brQlow==",

  "license": "MIT"

},

"node_modules/emoji-regex": {

  "version": "9.2.2",

  "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-9.2.2.tgz",

  "integrity": "sha512-L18DaJsXSUk2+42pv8mLs5jJT2hqFkFE4j21wOmgBUqsZ2hL72NsUU785g9RXgo3s0ZNgVl42TiHp3ZtOv/Vyg==",

  "license": "MIT"

},

"node_modules/encodeurl": {
```

```
"version": "2.0.0",

"resolved": "https://registry.npmjs.org/encodeurl/-/encodeurl-2.0.0.tgz",

"integrity": "sha512-  
Q0n9HRi4m6JuGIV1eFlmvJB7ZEVxu93lrMyiMsGC0lrMJMWzRgx6WGquyfQgZVb31vhGgXnf  
mPNNXmxnOkRBrg==",

"license": "MIT",

"engines": {

  "node": ">= 0.8"

}

},

"node_modules/end-of-stream": {

  "version": "1.4.5",

  "resolved": "https://registry.npmjs.org/end-of-stream/-/end-of-stream-1.4.5.tgz",

  "integrity": "sha512-  
ooEGc6HP26xXq/N+GCGOT0JKCLDGrq2bQUZrQ7gyrJiZANJ/8YDTxTpQBXGMn+WbIQXNVp  
yWymm7KYVICQnyOg==",

  "license": "MIT",

  "optional": true,

  "dependencies": {

    "once": "^1.4.0"

  }

},

"node_modules/es-define-property": {

  "version": "1.0.1",

  "resolved": "https://registry.npmjs.org/es-define-property/-/es-define-property-  
1.0.1.tgz",
```

```
  "integrity": "sha512-e3nRfgfUZ4rNGL232gUgX06QNyyez04KdjFrF+LTRoOXmrOgFKDg4BCdsjW8EnT69eqdYGM  
RpJwiPVYNrCaW3g==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 0.4"
```

```
  }
```

```
},
```

```
"node_modules/es-errors": {
```

```
  "version": "1.3.0",
```

```
  "resolved": "https://registry.npmjs.org/es-errors/-/es-errors-1.3.0.tgz",
```

```
  "integrity": "sha512-Zf5H2Kxt2xjTvbJvP2ZWLEICxA6j+hAmMzllpy4xcBg1vKVnx89Wy0GbS+kf5cwCVFFzdCFh2  
XSCFNULS6csw==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 0.4"
```

```
  }
```

```
},
```

```
"node_modules/es-object-atoms": {
```

```
  "version": "1.1.1",
```

```
  "resolved": "https://registry.npmjs.org/es-object-atoms/-/es-object-atoms-1.1.1.tgz",
```

```
  "integrity": "sha512-FGgH2h8zKNim9lj7dankFPclClK9Cp5bm+c2gQSYePhpaG5+esrLODihlorn+Pe6FGJzWhXQ  
otPv73jTaldXA==",
```

```
  "license": "MIT",
```

```
  "dependencies": {
```

```
    "es-errors": "^1.3.0"
```

```
,
"engines": {
  "node": ">= 0.4"
}
},
"node_modules/es-set-tostringtag": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/es-set-tostringtag/-/es-set-tostringtag-2.1.0.tgz",
  "integrity": "sha512-j6vWzfrGVfyXxge+O0x5sh6cvxAog0a/4Rdd2K36zCMV5eJ+/+tOAngRO8cODMNWbVRdVImGZQL2YS3yR8bIUA==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "es-errors": "^1.3.0",
    "get-intrinsic": "^1.2.6",
    "has-tostringtag": "^1.0.2",
    "hasown": "^2.0.2"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/escalade": {
  "version": "3.2.0",
  "resolved": "https://registry.npmjs.org/escalade/-/escalade-3.2.0.tgz",
```

```
  "integrity": "sha512-
WUj2qlxaQtO4g6Pq5c29GTcWGDyd8itL8zTlipgECz3JesAiiOKotd8JU6otB3PACgG6xkJUyVh
boMS+bje/jA==",

  "license": "MIT",

  "optional": true,

  "engines": {

    "node": ">=6"

  }
},

"node_modules/escape-html": {

  "version": "1.0.3",

  "resolved": "https://registry.npmjs.org/escape-html/-/escape-html-1.0.3.tgz",

  "integrity": "sha512-
NiSupZ4OeuGwr68lGleym/kslZMJodUGOS CZ/FSnTxcrekbvqrgdUxUOMpijaKZVjAJrWrGs/6J
y8OMuyj9ow==",

  "license": "MIT"

},

"node_modules/escape-string-regexp": {

  "version": "4.0.0",

  "resolved": "https://registry.npmjs.org/escape-string-regexp/-/escape-string-regexp-
4.0.0.tgz",

  "integrity": "sha512-
TtpcNJ3XAzx3Gq8sWRzJaVajRs0uVxA2YAkdb1jm2YkPz4G6egUFAYa3n5vtElZefPk5Wa4UXb
KuS5fKkJWdgA==",

  "dev": true,

  "license": "MIT",

  "engines": {

    "node": ">=10"
```

```
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/eslint": {
  "version": "10.0.2",
  "resolved": "https://registry.npmjs.org/eslint/-/eslint-10.0.2.tgz",
  "integrity": "sha512-uYixubwmmqJZH+KLVYIVKY1JQt7tysXhtj21WSvjcSmU5SVNzMus1bgLe+pAt816yQ8opKfheVVoPLqvVMGejYw==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "@eslint-community/eslint-utils": "^4.8.0",
    "@eslint-community/regexpp": "^4.12.2",
    "@eslint/config-array": "^0.23.2",
    "@eslint/config-helpers": "^0.5.2",
    "@eslint/core": "^1.1.0",
    "@eslint/plugin-kit": "^0.6.0",
    "@humanfs/node": "^0.16.6",
    "@humanwhocodes/module-importer": "^1.0.1",
    "@humanwhocodes/retry": "^0.4.2",
    "@types/estree": "^1.0.6",
    "ajv": "^6.14.0",
    "cross-spawn": "^7.0.6",
    "debug": "^4.3.2",
```



```
"escape-string-regexp": "^4.0.0",
"eslint-scope": "^9.1.1",
"eslint-visitor-keys": "^5.0.1",
"espree": "^11.1.1",
"esquery": "^1.7.0",
"esutils": "^2.0.2",
"fast-deep-equal": "^3.1.3",
"file-entry-cache": "^8.0.0",
"find-up": "^5.0.0",
"glob-parent": "^6.0.2",
"ignore": "^5.2.0",
"imurmurhash": "^0.1.4",
"is-glob": "^4.0.0",
"json-stable-stringify-without-jsonify": "^1.0.1",
"minimatch": "^10.2.1",
"natural-compare": "^1.4.0",
"optionator": "^0.9.3"
},
"bin": {
  "eslint": "bin/eslint.js"
},
"engines": {
  "node": "^20.19.0 || ^22.13.0 || >=24"
},
"funding": {
  "url": "https://eslint.org/donate"
```

```
,
"peerDependencies": {
  "jiti": "*"
},
"peerDependenciesMeta": {
  "jiti": {
    "optional": true
  }
}
},
"node_modules/eslint-config-google": {
  "version": "0.14.0",
  "resolved": "https://registry.npmjs.org/eslint-config-google/-/eslint-config-google-0.14.0.tgz",
  "integrity": "sha512-WsbX4WbjuMvTdeVL6+J3rK1RGhCTqjsFjX7UMSMgZiyxxaNLkoJENbrGExzERFeoTpGw3F3FypTiWAP9ZXzkEw==",
  "dev": true,
  "license": "Apache-2.0",
  "engines": {
    "node": ">=0.10.0"
  },
  "peerDependencies": {
    "eslint": ">=5.16.0"
  }
},
"node_modules/eslint-scope": {
```

```
"version": "9.1.1",

"resolved": "https://registry.npmjs.org/eslint-scope/-/eslint-scope-9.1.1.tgz",

"integrity": "sha512-
GaUN0sWim5qc8KVErfPBWmc31LEsOkrUJbvJZV+xuL3u2phMUK4HlvXlWAakfC8W4nzlK+
chPEAkYOYb5ZSclw==",

"dev": true,

"license": "BSD-2-Clause",

"dependencies": {

"@types/esrecurse": "^4.3.1",

"@types/estree": "^1.0.8",

"esrecurse": "^4.3.0",

"estraverse": "^5.2.0"

},

"engines": {

"node": "^20.19.0 || ^22.13.0 || >=24"

},

"funding": {

"url": "https://opencollective.com/eslint"

}

},

"node_modules/eslint-visitor-keys": {

"version": "3.4.3",

"resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-3.4.3.tgz",

"integrity": "sha512-
wpc+LXeiyisxPLEkUzU6svyS1frlO3Mgxj1fdy7Pm8Ygzguax2N3Fa/D/ag1WqbOprdl+uY6wMU
l8/a2G+iag==",

"dev": true,
```

```
"license": "Apache-2.0",
"engines": {
  "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
},
"funding": {
  "url": "https://opencollective.com/eslint"
}
},
"node_modules/eslint/node_modules/eslint-visitor-keys": {
  "version": "5.0.1",
  "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-5.0.1.tgz",
  "integrity": "sha512-1Uu35WZ46B50Yqg2O10311dG5gPQ704Ih37BJm4dd4/1e3UwHXvR2LHhX/g0wut6nQ1v7KbFrW44U10g==",
  "dev": true,
  "license": "Apache-2.0",
  "engines": {
    "node": "^20.19.0 || ^22.13.0 || >=24"
  },
  "funding": {
    "url": "https://opencollective.com/eslint"
  }
},
"node_modules/esprez": {
  "version": "11.1.1",
  "resolved": "https://registry.npmjs.org/esprez/-/esprez-11.1.1.tgz",
```

```
"integrity": "sha512-
AVHPqQoZYc+RUM4/3Ly5udlZY/U4LS8plG05jEjWM2lQMU/oaZ7qshzAl2YP1tfNmXfftH3oh
urfwNAug+MnsQ==",

"dev": true,

"license": "BSD-2-Clause",

"dependencies": {

  "acorn": "^8.16.0",

  "acorn-jsx": "^5.3.2",

  "eslint-visitor-keys": "^5.0.1"

},

"engines": {

  "node": "^20.19.0 || ^22.13.0 || >=24"

},

"funding": {

  "url": "https://opencollective.com/eslint"

}

},

"node_modules/esprez/node_modules/eslint-visitor-keys": {

  "version": "5.0.1",

  "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-5.0.1.tgz",

  "integrity": "sha512-
tD40eHxA35h0PElZNeIjkHoDR4YjjJp34biM0mDvplBe//mB+IHCqHDGV7pxF+7MklTvighcCP
PZC7ynWyjdTA==",

  "dev": true,

  "license": "Apache-2.0",

  "engines": {

    "node": "^20.19.0 || ^22.13.0 || >=24"

  }

}
```

```
  },
  "funding": {
    "url": "https://opencollective.com/eslint"
  },
},
"node_modules/esquery": {
  "version": "1.7.0",
  "resolved": "https://registry.npmjs.org/esquery/-/esquery-1.7.0.tgz",
  "integrity": "sha512-1001G0UgApDgXlG0DORGL9YKPYaDl9l/1smG2MkbV78xum3s2G8M3S1+F613U/Ip9gJ94S2sUHqGfEG8h7YU=",
  "dev": true,
  "license": "BSD-3-Clause",
  "dependencies": {
    "estraverse": "^5.1.0"
  },
  "engines": {
    "node": ">=0.10"
  }
},
"node_modules/esrecurse": {
  "version": "4.3.0",
  "resolved": "https://registry.npmjs.org/esrecurse/-/esrecurse-4.3.0.tgz",
  "integrity": "sha512-KmfKL3b6G+RXvP8N1vr3Tq1kL/oCFgn2NYXEtaP8/L3pKapUA4G8cFVaoF3SU323CD4XypR/ffioHmkti6/Tag==",
  "dev": true,
```

```
"license": "BSD-2-Clause",
"dependencies": {
  "estraverse": "^5.2.0"
},
"engines": {
  "node": ">=4.0"
}
},
"node_modules/estraverse": {
  "version": "5.3.0",
  "resolved": "https://registry.npmjs.org/estraverse/-/estraverse-5.3.0.tgz",
  "integrity": "sha512-MMdARuVEQziNTEJD8DgMqmhWR11BRQ/cBP+pLtYdSTnf3MIO8fFeilNEbX36ZdNlfU/7A9f3gUw49B3oQsvwBA==",
  "dev": true,
  "license": "BSD-2-Clause",
  "engines": {
    "node": ">=4.0"
  }
},
"node_modules/esutils": {
  "version": "2.0.3",
  "resolved": "https://registry.npmjs.org/esutils/-/esutils-2.0.3.tgz",
  "integrity": "sha512-kVscQXk4OCp68SZ0dkgEKVi6/8ij300KBWTJq32P/dYeWTSwK41WyTxaIn1eRmA5Z9UU/LX9D7FWSmV9SAYx6g==",
  "dev": true,
```

```
"license": "BSD-2-Clause",

"engines": {
  "node": ">=0.10.0"
}
},
"node_modules/etag": {
  "version": "1.8.1",
  "resolved": "https://registry.npmjs.org/etag/-/etag-1.8.1.tgz",
  "integrity": "sha512-aLL5F7mawVa300aL2BnEE4iNvo1qETxLrPI/o05L7z6go7fCw1J6EQmbK4FmJ2AS7kgVF/KEZ
WufBfdClMcPg==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/event-target-shim": {
  "version": "5.0.1",
  "resolved": "https://registry.npmjs.org/event-target-shim/-/event-target-shim-5.0.1.tgz",
  "integrity": "sha512-i/2XbnSz/uxRCU6+NdVJgKWDTM427+MqYbkQzD321DuCQJUqOuJKIA0IM2+W2xtYHdKOm
Z4dR6fExsd4SXL+WQ==",
  "license": "MIT",
  "optional": true,
  "engines": {
    "node": ">=6"
  }
}
```



```
},  
"node_modules/express": {  
  "version": "4.22.1",  
  "resolved": "https://registry.npmjs.org/express/-/express-4.22.1.tgz",  
  "integrity": "sha512-F2X8g9P1X7uCPZMA3MVf9wcTqlyNp7lhH5qPCI0izhaOIYXaW9L535tGA3qmqjRzpH+bZczqq  
7hVKxTR4NWnu+g==",  
  "license": "MIT",  
  "dependencies": {  
    "accepts": "~1.3.8",  
    "array-flatten": "1.1.1",  
    "body-parser": "~1.20.3",  
    "content-disposition": "~0.5.4",  
    "content-type": "~1.0.4",  
    "cookie": "~0.7.1",  
    "cookie-signature": "~1.0.6",  
    "debug": "2.6.9",  
    "depd": "2.0.0",  
    "encodeurl": "~2.0.0",  
    "escape-html": "~1.0.3",  
    "etag": "~1.8.1",  
    "finalhandler": "~1.3.1",  
    "fresh": "~0.5.2",  
    "http-errors": "~2.0.0",  
    "merge-descriptors": "1.0.3",  
    "methods": "~1.1.2",  
    "on-finished": "~2.4.1",
```

```
"parseurl": "~1.3.3",
"path-to-regexp": "~0.1.12",
"proxy-addr": "~2.0.7",
"qs": "~6.14.0",
"range-parser": "~1.2.1",
"safe-buffer": "5.2.1",
"send": "~0.19.0",
"serve-static": "~1.16.2",
"setprototypeof": "1.2.0",
"statuses": "~2.0.1",
"type-is": "~1.6.18",
"utils-merge": "1.0.1",
"vary": "~1.1.2"
},
"engines": {
  "node": ">= 0.10.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/express"
}
},
"node_modules/express/node_modules/debug": {
  "version": "2.6.9",
  "resolved": "https://registry.npmjs.org/debug/-/debug-2.6.9.tgz",
```

```
  "integrity": "sha512-
bC7ElrdJaInPbAP+1EotYvqZsb3ecl5wi6Bfi6BJTUcNowp6cvspg0jXznRTKDjm/E7AdgFBVeAP
VMNcKGsHMA==",

  "license": "MIT",

  "dependencies": {

    "ms": "2.0.0"

  },

  },

  "node_modules/express/node_modules/ms": {

    "version": "2.0.0",

    "resolved": "https://registry.npmjs.org/ms/-/ms-2.0.0.tgz",

    "integrity": "sha512-
Tpp60P6IUJDTuOq/5Z8cdskzJujfwqfOTkrwIwj7IRISpnkJnT6SyJ4PCPnGMofJc9ddhal5KVIYt
At97ix05A==",

    "license": "MIT"

  },

  "node_modules/extend": {

    "version": "3.0.2",

    "resolved": "https://registry.npmjs.org/extend/-/extend-3.0.2.tgz",

    "integrity": "sha512-
fjquC59cD7CyW6urNXK0FBufkZcoiGG80wTuPujX590cB5Ttln20E2UB4S/WARVqhXffZl2LNg
S+gQdPllim/g==",

    "license": "MIT"

  },

  "node_modules/farmhash-modern": {

    "version": "1.1.0",

    "resolved": "https://registry.npmjs.org/farmhash-modern/-/farmhash-modern-1.1.0.tgz",
```

```
  "integrity": "sha512-
6ypT4XfgqJk/F3Yuv4SX26l3doUjt0GTG4a+JgWxXQpxXzTBq8fPUeGHfcYMMDPHJHm3yPOSj
aeBwBGAHWXCdA==",

  "license": "MIT",

  "engines": {

    "node": ">=18.0.0"

  }

},

"node_modules/fast-deep-equal": {

  "version": "3.1.3",

  "resolved": "https://registry.npmjs.org/fast-deep-equal/-/fast-deep-equal-3.1.3.tgz",

  "integrity": "sha512-
f3qQ9oQy9j2AhBe/H9VC91wLmKBCCU/gDOnKNAYG5hswO7BLKj09Hc5HYNz9cGI++xlpD
ClgDaitVs03ATR84Q==",

  "license": "MIT"

},

"node_modules/fast-json-stable-stringify": {

  "version": "2.1.0",

  "resolved": "https://registry.npmjs.org/fast-json-stable-stringify/-/fast-json-stable-
stringify-2.1.0.tgz",

  "integrity": "sha512-
lhd/wF+Lk98HZoTCtlVraHtfh5XYijljalXck7saUtuanSDyLMxnHhSXEDJqHxD7msR8D0uCmql
kwjCV8xvwHw==",

  "dev": true,

  "license": "MIT"

},

"node_modules/fast-levenshtein": {

  "version": "2.0.6",
```

```
"resolved": "https://registry.npmjs.org/fast-levenshtein/-/fast-levenshtein-2.0.6.tgz",  
  
  "integrity": "sha512-DCXu6lfhqcks7TZKY3Hxp3y6qphY5SJZmrWMDrKcERSOXWQdMhU9Ig/PYrzyw/ul9jOlyh0N4M0tbC5hodg8dw==",  
  
  "dev": true,  
  
  "license": "MIT"  
  
},  
  
"node_modules/fast-xml-parser": {  
  
  "version": "5.3.7",  
  
  "resolved": "https://registry.npmjs.org/fast-xml-parser/-/fast-xml-parser-5.3.7.tgz",  
  
  "integrity": "sha512-JzVLro9NQv92pOM/jTCR6mHUh2FGwtomH8ZQjhFj/R29P2Fnj38OgPJVtcvYw6SuKClhgYuWUZf5b3rd8u2mA==",  
  
  "funding": [  
  
    {  
  
      "type": "github",  
  
      "url": "https://github.com/sponsors/NaturalIntelligence"  
  
    }  
  
  ],  
  
  "license": "MIT",  
  
  "optional": true,  
  
  "dependencies": {  
  
    "strnum": "^2.1.2"  
  
  },  
  
  "bin": {  
  
    "fxparser": "src/cli/cli.js"  
  
  }  
  
}
```

```
  },
  "node_modules/faye-websocket": {
    "version": "0.11.4",
    "resolved": "https://registry.npmjs.org/faye-websocket/-/faye-websocket-0.11.4.tgz",
    "integrity": "sha512-CzbClwlXAUuIRQAUyfqPgvPoNKTckTPGfwZV4ZdAhVcP2lh9KUxJg2b5GkE7XbjKQ3YJnQ9z6D9ntLAlB+tP8g==",
    "license": "Apache-2.0",
    "dependencies": {
      "websocket-driver": ">=0.5.1"
    },
    "engines": {
      "node": ">=0.8.0"
    }
  },
  "node_modules/fetch-blob": {
    "version": "3.2.0",
    "resolved": "https://registry.npmjs.org/fetch-blob/-/fetch-blob-3.2.0.tgz",
    "integrity": "sha512-7yAQpD2UMJzLi1Dqv7qFYnPbaPx7ZfFK6PilxQ4PfkGPyNyl2Ugx+a/umUonmKqjhM4DnfbMvdX6otXq83soQQ==",
    "funding": [
      {
        "type": "github",
        "url": "https://github.com/sponsors/jimmywarting"
      }
    ],
    {
```

```
    "type": "paypal",
    "url": "https://paypal.me/jimmywarting"
  },
],
"license": "MIT",
"dependencies": {
  "node-domexception": "^1.0.0",
  "web-streams-polyfill": "^3.0.3"
},
"engines": {
  "node": "^12.20 || >= 14.13"
}
},
"node_modules/file-entry-cache": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/file-entry-cache/-/file-entry-cache-8.0.0.tgz",
  "integrity": "sha512-XTTUwCvisa5oacNGRP9SfNtYBNAMi+RPwBFmbIZEF7N7swHYQS6/Zfk7SRwx4D5j3CH211YNRco1DEMNVfZCnQ==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "flat-cache": "^4.0.0"
  },
  "engines": {
    "node": ">=16.0.0"
  }
}
```

```
},
"node_modules/finalhandler": {
  "version": "1.3.2",
  "resolved": "https://registry.npmjs.org/finalhandler/-/finalhandler-1.3.2.tgz",
  "integrity": "sha512-aA4RyPcd3badbdABGDuTXCMTtOneUCAYH/gxoYRTZIIJdF0YPWuGqiAsIrhNnnqdXGswYk6dGujem4w80UJFhg==",
  "license": "MIT",
  "dependencies": {
    "debug": "2.6.9",
    "encodeurl": "~2.0.0",
    "escape-html": "~1.0.3",
    "on-finished": "~2.4.1",
    "parseurl": "~1.3.3",
    "statuses": "~2.0.2",
    "unpipe": "~1.0.0"
  },
  "engines": {
    "node": ">= 0.8"
  }
},
"node_modules/finalhandler/node_modules/debug": {
  "version": "2.6.9",
  "resolved": "https://registry.npmjs.org/debug/-/debug-2.6.9.tgz",
  "integrity": "sha512-bC7ElrdJaInPbAP+1EotYvqZsb3ecl5wi6Bfi6BJTUcNowp6cvspg0jXznRTKDjm/E7AdgFBVeAPVMNcKGsHMA==",
```



```
"license": "MIT",
"dependencies": {
  "ms": "2.0.0"
},
"node_modules/finalhandler/node_modules/ms": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/ms/-/ms-2.0.0.tgz",
  "integrity": "sha512-  
Tpp60P6IUJDTuOq/5Z8cdskzJujfwqfOTkrwIwj7IRISpnkJnT6SyJ4PCPnGMoFjC9ddhal5KVIYt  
At97ix05A==",
  "license": "MIT"
},
"node_modules/find-up": {
  "version": "5.0.0",
  "resolved": "https://registry.npmjs.org/find-up/-/find-up-5.0.0.tgz",
  "integrity": "sha512-  
78/PXT1wLLDgTzDs7sjq9hzz0vXD+zn+7wypEe4fXQxCmdmqfGsEPQxmiCSQI3ajFV91bVSs  
vNtrJRiW6nGng==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "locate-path": "^6.0.0",
    "path-exists": "^4.0.0"
  },
  "engines": {
    "node": ">=10"
```

```
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  },
},
"node_modules/firebase-admin": {
  "version": "13.6.1",
  "resolved": "https://registry.npmjs.org/firebase-admin/-/firebase-admin-13.6.1.tgz",
  "integrity": "sha512-Zgc6yPtMPxAZo+FoK6LMG6zpSEsoSK8iflR+IqF4oWuC3uWZU40OjxgfLTsFcsRlj/k/wD66zNv2UiTRreCNSw==",
  "license": "Apache-2.0",
  "dependencies": {
    "@fastify/busboy": "^3.0.0",
    "@firebase/database-compat": "^2.0.0",
    "@firebase/database-types": "^1.0.6",
    "@types/node": "^22.8.7",
    "farmhash-modern": "^1.1.0",
    "fast-deep-equal": "^3.1.1",
    "google-auth-library": "^9.14.2",
    "jsonwebtoken": "^9.0.0",
    "jwks-rsa": "^3.1.0",
    "node-forge": "^1.3.1",
    "uuid": "^11.0.2"
  },
},
"engines": {
  "node": ">=18"
```

```
,
"optionalDependencies": {
  "@google-cloud/firestore": "^7.11.0",
  "@google-cloud/storage": "^7.14.0"
},
"node_modules/firebase-functions": {
  "version": "7.0.5",
  "resolved": "https://registry.npmjs.org/firebase-functions/-/firebase-functions-7.0.5.tgz",
  "integrity": "sha512-uG2dR5AObLuUrWWjj/de5XxNHCVi+Ehths0DSRcLjHJdgw1TSejwoZZ5na6gVrl3znNjRdBry5Br5UlhalU3Ww==",
  "license": "MIT",
  "dependencies": {
    "@types/cors": "^2.8.5",
    "@types/express": "^4.17.21",
    "cors": "^2.8.5",
    "express": "^4.21.0",
    "protobufjs": "^7.2.2"
  },
  "bin": {
    "firebase-functions": "lib/bin/firebase-functions.js"
  },
  "engines": {
    "node": ">=18.0.0"
  },
  "peerDependencies": {
```

```
"@apollo/server": "^5.2.0",
"@as-integrations/express4": "^1.1.2",
"firebase-admin": "^11.10.0 || ^12.0.0 || ^13.0.0",
"graphql": "^16.12.0"
},
"peerDependenciesMeta": {
  "@apollo/server": {
    "optional": true
  },
  "@as-integrations/express4": {
    "optional": true
  },
  "graphql": {
    "optional": true
  }
}
},
"node_modules/firebase-functions-test": {
  "version": "2.1.0",
  "resolved": "https://registry.npmjs.org/firebase-functions-test/-/firebase-functions-test-2.1.0.tgz",
  "integrity": "sha512-USA1iwprxStMSidxBxBVydQOEUrByVdDSD83oUFX5+e/sdeLY3qFWk3naC2XhfoXw9T/RF9m+p8EarupS+Xo9w==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
```

```
"@types/lodash": "^4.14.104",
"lodash": "^4.17.5",
"ts-deepmerge": "^2.0.1"
},
"engines": {
  "node": ">=14.0.0"
},
"peerDependencies": {
  "firebase-admin": ">=6.0.0",
  "firebase-functions": ">=3.20.1"
}
},
"node_modules/flat-cache": {
  "version": "4.0.1",
  "resolved": "https://registry.npmjs.org/flat-cache/-/flat-cache-4.0.1.tgz",
  "integrity": "sha512-f7ccFPK3SXFHpx15UIGyRJ/FJQctuKZ0zVuN3frBo4HnK3cay9VEW0R6yPYFHC0AgqhukPzKj
q22t5DmAyqGyw==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "flatted": "^3.2.9",
    "keyv": "^4.5.4"
  },
  "engines": {
    "node": ">=16"
  }
}
```

```
  },
  "node_modules/flatted": {
    "version": "3.3.3",
    "resolved": "https://registry.npmjs.org/flatted/-/flatted-3.3.3.tgz",
    "integrity": "sha512-GX+ysw4PBCz0PzosHDEpZGANEuFCMLrnRTiEy9McGjmkCQYwRq4A/X786G/fjM/+OjsWSU1ZrY5qyARZmO/uwg==",
    "dev": true,
    "license": "ISC"
  },
  "node_modules/foreground-child": {
    "version": "3.3.1",
    "resolved": "https://registry.npmjs.org/foreground-child/-/foreground-child-3.3.1.tgz",
    "integrity": "sha512-gIXjKqtFuWEgzFRJA9WCQeSJLZDjgJUOMCMzxtvFq/37KojM1BFGufqsCy0r4qSQmYLsZYMe yRqzIWOMup03sw==",
    "license": "ISC",
    "dependencies": {
      "cross-spawn": "^7.0.6",
      "signal-exit": "^4.0.1"
    }
  },
  "engines": {
    "node": ">=14"
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
}
```

```
},  
"node_modules/form-data": {  
  "version": "2.5.5",  
  "resolved": "https://registry.npmjs.org/form-data/-/form-data-2.5.5.tgz",  
  "integrity": "sha512-  
jqdObeR2rxZZbPSGL+3VckHMYtu+f9//KXBsVny6JSX/pa38Fy+bGjuG8eW/H6USNQWhLi8N  
um++cU2yOCNz4A==",  
  "license": "MIT",  
  "optional": true,  
  "dependencies": {  
    "asynckit": "^0.4.0",  
    "combined-stream": "^1.0.8",  
    "es-set-tostringtag": "^2.1.0",  
    "hasown": "^2.0.2",  
    "mime-types": "^2.1.35",  
    "safe-buffer": "^5.2.1"  
  },  
  "engines": {  
    "node": ">= 0.12"  
  }  
},  
"node_modules/formdata-polyfill": {  
  "version": "4.0.10",  
  "resolved": "https://registry.npmjs.org/formdata-polyfill/-/formdata-polyfill-4.0.10.tgz",  
  "integrity": "sha512-  
buewHzMvYL29jdeQTVILecSaZKnt/RJWjoZCF5OW60Z67/GmSLBkOFM7qh1PI3zFNtJbaZL5  
eQu1vLfazOwj4g==",
```

```
"license": "MIT",
"dependencies": {
  "fetch-blob": "^3.1.2"
},
"engines": {
  "node": ">=12.20.0"
}
},
"node_modules/forwarded": {
  "version": "0.2.0",
  "resolved": "https://registry.npmjs.org/forwarded/-/forwarded-0.2.0.tgz",
  "integrity": "sha512-buRG0fpBtRHSTCOASe6hD258tEubFoRLb4ZNA6NxMVHNw2gOcwHo9wyablzMzOA5z9xA9L1KNjk/Nt6MT9aYow==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/fresh": {
  "version": "0.5.2",
  "resolved": "https://registry.npmjs.org/fresh/-/fresh-0.5.2.tgz",
  "integrity": "sha512-twtTm2B1Da382j9e1s7K0oqj/JH02u9SqhrhO3H6o8P6E7JpQAI5Ztt5Aq2xtQ8S62c53qBX05lH0rQ==",
  "license": "MIT",
  "engines": {
```



```
"node": ">= 0.6"
}
},
"node_modules/function-bind": {
  "version": "1.1.2",
  "resolved": "https://registry.npmjs.org/function-bind/-/function-bind-1.1.2.tgz",
  "integrity": "sha512-7XHNxH7qX9xG5mlwxkhumTox/MIRNcOgDrxWsMt2pAr23WHp6MrRlN7FBSFpCpr+oVO0F744iUgR82nJMfG2SA==",
  "license": "MIT",
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/functional-red-black-tree": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/functional-red-black-tree/-/functional-red-black-tree-1.0.1.tgz",
  "integrity": "sha512-dsKNQNdj6xA3T+QLADDA7mOSlX0qiMINjn0cgr+eGHGsbSHzTabclogz2+p/iqP1Xs6EP/sS2SbqH+brGTbq0g==",
  "license": "MIT",
  "optional": true
},
"node_modules/gaxios": {
  "version": "6.7.1",
  "resolved": "https://registry.npmjs.org/gaxios/-/gaxios-6.7.1.tgz",
```

```
"integrity": "sha512-
LDODD4TMYx7XXdpwxAVRAIAuB0bzbv0s+ywFonY46k126qzQHT9ygyoa9tncmOiQmmDrik6
5UYsEkv3lbfqQ3yQ==",

"license": "Apache-2.0",

"dependencies": {
  "extend": "^3.0.2",
  "https-proxy-agent": "^7.0.1",
  "is-stream": "^2.0.0",
  "node-fetch": "^2.6.9",
  "uuid": "^9.0.1"
},

"engines": {
  "node": ">=14"
}
},

"node_modules/gaxios/node_modules/uuid": {
  "version": "9.0.1",
  "resolved": "https://registry.npmjs.org/uuid/-/uuid-9.0.1.tgz",
  "integrity": "sha512-
b+1eJOlsR9K8HJpow9Ok3fiWOWSIclzXodvv0rQjVoOVNpWMpxf1wZNpt4y9h10odCNrqnY
p1OBzRktckBe3sA==",

  "funding": [
    "https://github.com/sponsors/broofa",
    "https://github.com/sponsors/ctavan"
  ],

  "license": "MIT",

  "bin": {
```

```
    "uuid": "dist/bin/uuid"
  }
},
"node_modules/gcp-metadata": {
  "version": "6.1.1",
  "resolved": "https://registry.npmjs.org/gcp-metadata/-/gcp-metadata-6.1.1.tgz",
  "integrity": "sha512-a4tiq7E0/5fTjxPAaH4jpjKsv/uCaU2p5KC6HVGrvl0cDjA8iBZv4vv1gyzlmK0ZUKqwpOyQMKzZQe3lTit77A==",
  "license": "Apache-2.0",
  "dependencies": {
    "gaxios": "^6.1.1",
    "google-logging-utils": "^0.0.2",
    "json-bigint": "^1.0.0"
  },
  "engines": {
    "node": ">=14"
  }
},
"node_modules/get-caller-file": {
  "version": "2.0.5",
  "resolved": "https://registry.npmjs.org/get-caller-file/-/get-caller-file-2.0.5.tgz",
  "integrity": "sha512-DatFPiaaIv5BIhGG3DQVUqKBGfiN2EGvr47YuRuMy6I24gEcO9XTHm0zGWs1WbpSLk539+3MCBGVknJ0xv8wag==",
  "license": "ISC",
  "optional": true,

```

```
"engines": {
  "node": "6.* || 8.* || >= 10.*"
},
"node_modules/get-intrinsic": {
  "version": "1.3.0",
  "resolved": "https://registry.npmjs.org/get-intrinsic/-/get-intrinsic-1.3.0.tgz",
  "integrity": "sha512-9fSjSaos/fRIVlp+xSJlE6lfwhES7LNtKaCBlamHsjr2na1BiABJPo0mOjjz8GJDURarmCPGqaiVg5mfjb98CQ==",
  "license": "MIT",
  "dependencies": {
    "call-bind-apply-helpers": "^1.0.2",
    "es-define-property": "^1.0.1",
    "es-errors": "^1.3.0",
    "es-object-atoms": "^1.1.1",
    "function-bind": "^1.1.2",
    "get-proto": "^1.0.1",
    "gopd": "^1.2.0",
    "has-symbols": "^1.1.0",
    "hasown": "^2.0.2",
    "math-intrinsics": "^1.1.0"
  },
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
```

```

    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/get-proto": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/get-proto/-/get-proto-1.0.1.tgz",
  "integrity": "sha512-
sTSfBjoXBp89JvIKIefqw7U2CCebSc74kiY6awiGogKtoSGbgjYE/G/+l9sF3MWFPNc9IcoOC4
ODfKHfxFmp0g==",
  "license": "MIT",
  "dependencies": {
    "dunder-proto": "^1.0.1",
    "es-object-atoms": "^1.0.0"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/glob": {
  "version": "10.5.0",
  "resolved": "https://registry.npmjs.org/glob/-/glob-10.5.0.tgz",
  "integrity": "sha512-
DfXN8DfhJ7NH3Oe7cFmu3NCu1wKbkReJ8TorzSAFbSKrlNaQSKflzqYqVY8zlbs2NLBbWpRi
U52GX2PbaBVNkg==",
  "deprecated": "Old versions of glob are not supported, and contain widely publicized
security vulnerabilities, which have been fixed in the current version. Please update.
Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me",
  "license": "ISC",

```

```
"dependencies": {
  "foreground-child": "^3.1.0",
  "jackspeak": "^3.1.2",
  "minimatch": "^9.0.4",
  "minipass": "^7.1.2",
  "package-json-from-dist": "^1.0.0",
  "path-scurry": "^1.11.1"
},
"bin": {
  "glob": "dist/esm/bin.mjs"
},
"funding": {
  "url": "https://github.com/sponsors/isaacs"
}
},
"node_modules/glob-parent": {
  "version": "6.0.2",
  "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-6.0.2.tgz",
  "integrity": "sha512-XxwI8EOhVQgWp6iDL+3b0r86f4d6AX6zSU55HfB4ydCEuXLXc5FcYeOu+nnGftS4TEju/11rt4KJPTMgbfmv4A==",
  "dev": true,
  "license": "ISC",
  "dependencies": {
    "is-glob": "^4.0.3"
  },
  "engines": {
```

```
  "node": ">=10.13.0"
}
},
"node_modules/glob/node_modules/minimatch": {
  "version": "9.0.6",
  "resolved": "https://registry.npmjs.org/minimatch/-/minimatch-9.0.6.tgz",
  "integrity": "sha512-  
kQAVowdR33eulqeA0+VZTDqU+qo1leVY+hrKYtZMio3Pg0P0vuh/kwRylLUddJhB6pf3q/botc  
OvRtx4IN1wqQ==",
  "license": "ISC",
  "dependencies": {
    "brace-expansion": "^5.0.2"
  },
  "engines": {
    "node": ">=16 || 14 >=14.17"
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
},
"node_modules/google-auth-library": {
  "version": "9.15.1",
  "resolved": "https://registry.npmjs.org/google-auth-library/-/google-auth-library-  
9.15.1.tgz",
  "integrity": "sha512-  
Jb6Z0+nvECVz+2lzSMt9u98UsoakXxA2HGHMCxh+so3n90XgYWkq5dur19JAJV7ONiJY22yB  
TyJB1TSkvPq9Ng==",
```

```
"license": "Apache-2.0",
"dependencies": {
  "base64-js": "^1.3.0",
  "ecdsa-sig-formatter": "^1.0.11",
  "gaxios": "^6.1.1",
  "gcp-metadata": "^6.1.0",
  "gtoken": "^7.0.0",
  "jws": "^4.0.0"
},
"engines": {
  "node": ">=14"
}
},
"node_modules/google-gax": {
  "version": "4.6.1",
  "resolved": "https://registry.npmjs.org/google-gax/-/google-gax-4.6.1.tgz",
  "integrity": "sha512-V6eky/xz2mCkFAd1Ioxyd6nmA61gao3n01C+Yeulwu3vzM9EDR6wcVzMSIbLMDXWeoi9SHYctXuKYC5uJUT3eQ==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "@grpc/grpc-js": "^1.10.9",
    "@grpc/proto-loader": "^0.7.13",
    "@types/long": "^4.0.0",
    "abort-controller": "^3.0.0",
    "duplexify": "^4.0.0",
```



```
"google-auth-library": "^9.3.0",
"node-fetch": "^2.7.0",
"object-hash": "^3.0.0",
"proto3-json-serializer": "^2.0.2",
"protobufjs": "^7.3.2",
"retry-request": "^7.0.0",
"uuid": "^9.0.1"
},
"engines": {
  "node": ">=14"
}
},
"node_modules/google-gax/node_modules/uuid": {
  "version": "9.0.1",
  "resolved": "https://registry.npmjs.org/uuid/-/uuid-9.0.1.tgz",
  "integrity": "sha512-b+eJOlsR9K8HJpow9Ok3fiWOWSIclzXodvv0rQjVoOVNpWMpxf1wZNpt4y9h10odCNrqnYp1OBzRktckBe3sA==",
  "funding": [
    "https://github.com/sponsors/broofa",
    "https://github.com/sponsors/ctavan"
  ],
  "license": "MIT",
  "optional": true,
  "bin": {
    "uuid": "dist/bin/uuid"
  }
}
```

```
  },
  "node_modules/google-logging-utils": {
    "version": "0.0.2",
    "resolved": "https://registry.npmjs.org/google-logging-utils/-/google-logging-utils-0.0.2.tgz",
    "integrity": "sha512-NEgUnEcBiP5HrPzufUkBzJOD/Sxsco3rLNo1F1TNf7ieU8ryUzBhqba8r756CjLX7rn3fHl6iLEwPYuqpoKgQQ==",
    "license": "Apache-2.0",
    "engines": {
      "node": ">=14"
    }
  },
  "node_modules/googleapis": {
    "version": "171.4.0",
    "resolved": "https://registry.npmjs.org/googleapis/-/googleapis-171.4.0.tgz",
    "integrity": "sha512-xybFL2SmmUglifgsbsRQYRdNrSAYwxWZDmkZTGjUIaRnX5jPqR8eL/cEvo6rCqh7iaZx6MfEPs/lrDgZ0bymkg==",
    "license": "Apache-2.0",
    "dependencies": {
      "google-auth-library": "^10.2.0",
      "googleapis-common": "^8.0.0"
    },
    "engines": {
      "node": ">=18"
    }
  }
```

```

},
"node_modules/googleapis-common": {
  "version": "8.0.1",
  "resolved": "https://registry.npmjs.org/googleapis-common/-/googleapis-common-8.0.1.tgz",
  "integrity": "sha512-eCzNACUXPb1PW5l0ULTzMHaL/ttPRADoPgjBlT8jWsTbxkCp6siv+qKJ/1ldaybCthGwsYFYallF7u9AkU4L+A==",
  "license": "Apache-2.0",
  "dependencies": {
    "extend": "^3.0.2",
    "gaxios": "^7.0.0-rc.4",
    "google-auth-library": "^10.1.0",
    "qs": "^6.7.0",
    "url-template": "^2.0.8"
  },
  "engines": {
    "node": ">=18.0.0"
  }
},
"node_modules/googleapis-common/node_modules/gaxios": {
  "version": "7.1.3",
  "resolved": "https://registry.npmjs.org/gaxios/-/gaxios-7.1.3.tgz",
  "integrity": "sha512-YGGYuEdVljqxxVH1pUTMY/XtmmsApXrCVv5EU25iX6inEPbV+VakJfLealkBtJN69AQmh1eG OdCl9Sm1UP6XQ==",
  "license": "Apache-2.0",

```

```
"dependencies": {
  "extend": "^3.0.2",
  "https-proxy-agent": "^7.0.1",
  "node-fetch": "^3.3.2",
  "rimraf": "^5.0.1"
},
"engines": {
  "node": ">=18"
},
"node_modules/googleapis-common/node_modules/gcp-metadata": {
  "version": "8.1.2",
  "resolved": "https://registry.npmjs.org/gcp-metadata/-/gcp-metadata-8.1.2.tgz",
  "integrity": "sha512-3oNOGaQ5PhLFkg==",
  "license": "Apache-2.0",
  "dependencies": {
    "gaxios": "^7.0.0",
    "google-logging-utils": "^1.0.0",
    "json-bigint": "^1.0.0"
  },
  "engines": {
    "node": ">=18"
  },
  "node_modules/googleapis-common/node_modules/google-auth-library": {
```

```
"version": "10.5.0",

"resolved": "https://registry.npmjs.org/google-auth-library/-/google-auth-library-10.5.0.tgz",

"integrity": "sha512-7ABviyMOIX5hIVD60YOfHw4/CxOfBhyduaYB+wbFWCWoni4N7SLcV46hrVRktuBbZjFC9ONyqamZITN7q3n32w==",

"license": "Apache-2.0",

"dependencies": {

  "base64-js": "^1.3.0",

  "ecdsa-sig-formatter": "^1.0.11",

  "gaxios": "^7.0.0",

  "gcp-metadata": "^8.0.0",

  "google-logging-utils": "^1.0.0",

  "gtoken": "^8.0.0",

  "jws": "^4.0.0"

},

"engines": {

  "node": ">=18"

},

"node_modules/googleapis-common/node_modules/google-logging-utils": {

  "version": "1.1.3",

  "resolved": "https://registry.npmjs.org/google-logging-utils/-/google-logging-utils-1.1.3.tgz",

  "integrity": "sha512-eAmLkJDjAFCVXg7A1unxHsLf961m6y17QFqXqAXGj/gVkkFrEiCfStRfwUIGNfeCEjNRa32JEWOUTlYXPyKvA==",

  "license": "Apache-2.0",
```

```
"engines": {
  "node": ">=14"
},
"node_modules/googleapis-common/node_modules/gtoken": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/gtoken/-/gtoken-8.0.0.tgz",
  "integrity": "sha512-CQg3RPhVhHByAlw==",
  "license": "MIT",
  "dependencies": {
    "gaxios": "^7.0.0",
    "jws": "^4.0.0"
  },
  "engines": {
    "node": ">=18"
  },
  "node_modules/googleapis-common/node_modules/node-fetch": {
    "version": "3.3.2",
    "resolved": "https://registry.npmjs.org/node-fetch/-/node-fetch-3.3.2.tgz",
    "integrity": "sha512-dRB78srN/l6gqWulah9SrxYnxeddlG30+GOqK/9OILVYLg3HPnr6SqOWTWOXKRwC2eGYCkZ59NNuSgvSrpGOA==",
    "license": "MIT",
    "dependencies": {

```

```
"data-uri-to-buffer": "^4.0.0",
"fetch-blob": "^3.1.4",
"formdata-polyfill": "^4.0.10"
},
"engines": {
  "node": "^12.20.0 || ^14.13.1 || >=16.0.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/node-fetch"
}
},
"node_modules/googleapis/node_modules/gaxios": {
  "version": "7.1.3",
  "resolved": "https://registry.npmjs.org/gaxios/-/gaxios-7.1.3.tgz",
  "integrity": "sha512-YGGyuEdVljqxkxVH1pUTMY/XtmmsApXrCVv5EU25iX6inEPbV+VakJfLealkBtJN69AQmh1eG
OdCl9Sm1UP6XQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "extend": "^3.0.2",
    "https-proxy-agent": "^7.0.1",
    "node-fetch": "^3.3.2",
    "rimraf": "^5.0.1"
  },
  "engines": {
    "node": ">=18"
```

```

    }
  },
  "node_modules/googleapis/node_modules/gcp-metadata": {
    "version": "8.1.2",
    "resolved": "https://registry.npmjs.org/gcp-metadata/-/gcp-metadata-8.1.2.tgz",
    "integrity": "sha512-3oNOGaQ5PhLFkg==",
    "license": "Apache-2.0",
    "dependencies": {
      "gaxios": "^7.0.0",
      "google-logging-utils": "^1.0.0",
      "json-bigint": "^1.0.0"
    },
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/googleapis/node_modules/google-auth-library": {
    "version": "10.5.0",
    "resolved": "https://registry.npmjs.org/google-auth-library/-/google-auth-library-10.5.0.tgz",
    "integrity": "sha512-7ABviyMOIX5hIVD60YOfHw4/CxOfBhyduaYB+wbFWCWoni4N7SLcV46hrVRktuBbZjFC9ONyqamZITN7q3n32w==",
    "license": "Apache-2.0",
    "dependencies": {

```



```
"base64-js": "^1.3.0",
"ecdsa-sig-formatter": "^1.0.11",
"gaxios": "^7.0.0",
"gcp-metadata": "^8.0.0",
"google-logging-utils": "^1.0.0",
"gtoken": "^8.0.0",
"jws": "^4.0.0"
},
"engines": {
  "node": ">=18"
}
},
"node_modules/googleapis/node_modules/google-logging-utils": {
  "version": "1.1.3",
  "resolved": "https://registry.npmjs.org/google-logging-utils/-/google-logging-utils-1.1.3.tgz",
  "integrity": "sha512-eAmLkJDjAFCVXg7A1unxHsLf961m6y17QFqXqAXGj/gVkkFrEICfStRfwUlGNfeCEjNRa32JEWOUTlYXPyyKvA==",
  "license": "Apache-2.0",
  "engines": {
    "node": ">=14"
  }
},
"node_modules/googleapis/node_modules/gtoken": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/gtoken/-/gtoken-8.0.0.tgz",
```

```
  "integrity": "sha512-
+CqsMbHPiSTdtSO14O51eMNIrp9N79gmeqmXeuJOhfucAedHw9noVe/n5uJk3tbKE6a+6Z
CQg3RPhVhHByAlw==",

  "license": "MIT",

  "dependencies": {

    "gaxios": "^7.0.0",

    "jws": "^4.0.0"

  },

  "engines": {

    "node": ">=18"

  }

},

"node_modules/googleapis/node_modules/node-fetch": {

  "version": "3.3.2",

  "resolved": "https://registry.npmjs.org/node-fetch/-/node-fetch-3.3.2.tgz",

  "integrity": "sha512-
dRB78srN/l6gqWulah9SrxeYnxeddlG30+GOqK/9OILVyLg3HPnr6SqOWTWQXKRwC2eGYC
kZ59NNuSgvSrpGOA==",

  "license": "MIT",

  "dependencies": {

    "data-uri-to-buffer": "^4.0.0",

    "fetch-blob": "^3.1.4",

    "formdata-polyfill": "^4.0.10"

  },

  "engines": {

    "node": "^12.20.0 || ^14.13.1 || >=16.0.0"

  },

}
```

```
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/node-fetch"
},
"node_modules/gopd": {
  "version": "1.2.0",
  "resolved": "https://registry.npmjs.org/gopd/-/gopd-1.2.0.tgz",
  "integrity": "sha512-1ZU19xZbBNjwRgWg+uXy18e2Y8oLW7AjF7eHkPcUoUg8QfUdzN3DfKcChp5ZrfYUdYOgWzOoWQ06vRfA==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  },
  "node_modules/gtoken": {
    "version": "7.1.0",
    "resolved": "https://registry.npmjs.org/gtoken/-/gtoken-7.1.0.tgz",
    "integrity": "sha512-0bF0B3H0WU0339b1nMkV6YUk31F9WU5l3RMv85wXo5jI430/0XG1s9s5rI53H1t81UoU8o3XqL6Pw==",
    "license": "MIT",
    "dependencies": {

```

```
"gaxios": "^6.0.0",
"jws": "^4.0.0"
},
"engines": {
  "node": ">=14.0.0"
}
},
"node_modules/has-symbols": {
  "version": "1.1.0",
  "resolved": "https://registry.npmjs.org/has-symbols/-/has-symbols-1.1.0.tgz",
  "integrity": "sha512-1cDNdwJ2Jaohmb3sg4OmKaMBwuC48sYni5HUw2DvsC8LjGTLK9h+eb1X6RyuOHe4hT0U
LCW68iomhjUoKUqlPQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  }
},
"node_modules/has-tostringtag": {
  "version": "1.0.2",
  "resolved": "https://registry.npmjs.org/has-tostringtag/-/has-tostringtag-1.0.2.tgz",
  "integrity": "sha512-1Q41A2Bt4Nkz7YqP64F4Ml2w41Zu3Fm0W0P24f41a2F1b3Df4QWUwCvW4qrYBJZUz1gh6bUA1gM1Kt4=
NqADB8VjPFLM2V0VvHUewwwwsw0ZWBaldgo+ieHtK3hasLz4qeCRjYcqfB6AQrBggRKppKF
8L52/VqdVsO47Dlw==",
```

```
"license": "MIT",
"optional": true,
"dependencies": {
  "has-symbols": "^1.0.3"
},
"engines": {
  "node": ">= 0.4"
},
"funding": {
  "url": "https://github.com/sponsors/ljharb"
}
},
"node_modules/hasown": {
  "version": "2.0.2",
  "resolved": "https://registry.npmjs.org/hasown/-/hasown-2.0.2.tgz",
  "integrity": "sha512-0hJU9SCPvmMzIBdZFqNPXWa6dqh7WdH0cIl9y+CyS8rG3nL48Bclra9HmKhVVUHyPWNH5Y7xDwAB7bfgSjkUMQ==",
  "license": "MIT",
  "dependencies": {
    "function-bind": "^1.1.2"
  },
  "engines": {
    "node": ">= 0.4"
  }
},
"node_modules/html-entities": {
```

```
"version": "2.6.0",

"resolved": "https://registry.npmjs.org/html-entities/-/html-entities-2.6.0.tgz",

"integrity": "sha512-kig+rMn/QOVRvr7c86gQ8lWXq+Hkv6CbAH1hLu+RG338StTpE8Z0b44SDVaqVu7HGKf27frdmUYEs9hTUX/cLQ==",

"funding": [

  {

    "type": "github",

    "url": "https://github.com/sponsors/mdevils"

  },

  {

    "type": "patreon",

    "url": "https://patreon.com/mdevils"

  }

],

"license": "MIT",

"optional": true

},

"node_modules/http-errors": {

  "version": "2.0.1",

  "resolved": "https://registry.npmjs.org/http-errors/-/http-errors-2.0.1.tgz",

  "integrity": "sha512-4FbRdAX+bSdmo4AUFuS0WNPz8NgFt+r8ThgNWmlrjQjt1Q7ZR9+zTlce2859x4KSXrwlSaeTqDoKQmtP8pLmQ==",

  "license": "MIT",

  "dependencies": {

    "depd": "~2.0.0",
```

```
"inherits": "~2.0.4",
"setprototypeof": "~1.2.0",
"statuses": "~2.0.2",
"toidentifier": "~1.0.1"
},
"engines": {
  "node": ">= 0.8"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/express"
}
},
"node_modules/http-parser-js": {
  "version": "0.5.10",
  "resolved": "https://registry.npmjs.org/http-parser-js/-/http-parser-js-0.5.10.tgz",
  "integrity": "sha512-Pysuw9XpUq5dVc/2SMHpuTY01RFl8fttgcyunJL7eEMhGM3cl4eOmiCycJDVCo/7O7ClfQD3Sal6ftDzqOXYMA==",
  "license": "MIT"
},
"node_modules/http-proxy-agent": {
  "version": "5.0.0",
  "resolved": "https://registry.npmjs.org/http-proxy-agent/-/http-proxy-agent-5.0.0.tgz",
  "integrity": "sha512-68G012a805b00EaYp6W7S06+0Z03G1206363Fq3C00pYU0213h3Q83170q624733jYp0X38e233n8pGw==",
  "license": "MIT"
}
```

```
"license": "MIT",
"optional": true,
"dependencies": {
  "@tootallnate/once": "2",
  "agent-base": "6",
  "debug": "4"
},
"engines": {
  "node": ">= 6"
}
},
"node_modules/http-proxy-agent/node_modules/agent-base": {
  "version": "6.0.2",
  "resolved": "https://registry.npmjs.org/agent-base/-/agent-base-6.0.2.tgz",
  "integrity": "sha512-RZNwNclF7+MS/8bDg70amg32dyeZGZxiDuQmZxKLAlQjr3jGyLx+4Kkk58UO7D2QdgFIQCo
vuSuZESne6RG6XQ==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "debug": "4"
  },
  "engines": {
    "node": ">= 6.0.0"
  }
},
"node_modules/https-proxy-agent": {
```



```
"version": "7.0.6",

"resolved": "https://registry.npmjs.org/https-proxy-agent/-/https-proxy-agent-7.0.6.tgz",

"integrity": "sha512-vK9P5/iUfdl95Al+JVyUulcVtd4ofvtrOr3HNtM2yxC9bnMbEdp3x01OhQNnjb8IJYi38VITE3mBXwcfvywuSw==",

"license": "MIT",

"dependencies": {

  "agent-base": "^7.1.2",

  "debug": "4"

},

"engines": {

  "node": ">= 14"

}

},

"node_modules/iconv-lite": {

  "version": "0.4.24",

  "resolved": "https://registry.npmjs.org/iconv-lite/-/iconv-lite-0.4.24.tgz",

  "integrity": "sha512-v3MXnZACvnywkTUEZomlActle7RXeedOR31wwl7VlyoXO4Qi9arvSenNQWne1TcRwhCL1HwLI21bEqdpj8/rA==",

  "license": "MIT",

  "dependencies": {

    "safer-buffer": ">= 2.1.2 < 3"

  },

  "engines": {

    "node": ">=0.10.0"

  }

}
```

```
},
"node_modules/ignore": {
  "version": "5.3.2",
  "resolved": "https://registry.npmjs.org/ignore/-/ignore-5.3.2.tgz",
  "integrity": "sha512-
hsBTNUqQTDwkWtcdYI2i06Y/nUBEsNEDJKjWdigLvegy8kDuJAS8uRlpkkcQpyEXL0Z/pjDy5H
BmMjRCJ2gq+g==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">= 4"
  }
},
"node_modules/imurmurhash": {
  "version": "0.1.4",
  "resolved": "https://registry.npmjs.org/imurmurhash/-/imurmurhash-0.1.4.tgz",
  "integrity": "sha512-
JmXMZ6wuvDmLiHEml9yzqO6lwFbof0GG4lkcGaENdCRDDmMVnny7s5HsIgHCbaq0w2M
yPhDqkhTUgS2LU2PHA==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=0.8.19"
  }
},
"node_modules/inherits": {
  "version": "2.0.4",
```

```
"resolved": "https://registry.npmjs.org/inherits/-/inherits-2.0.4.tgz",

"integrity": "sha512-
k/vGaX4/Yla3WzyMCvTQOXyelHvqOKtnqBduzTHpZpQZzAskKMhZ2K+EnBiSM9zGSoIFeMp
XKxa4dYeZIQqewQ==",

"license": "ISC"

},

"node_modules/ipaddr.js": {

"version": "1.9.1",

"resolved": "https://registry.npmjs.org/ipaddr.js/-/ipaddr.js-1.9.1.tgz",

"integrity": "sha512-
0KI/607xoxSToH7GjN1FfSbLoU0+btTicjsQSWQlh/hZykN8KpmMf7uYwPW3R+akZ6R/w18Zl
XSHBYXiYUPO3g==",

"license": "MIT",

"engines": {

"node": ">= 0.10"

}

},

"node_modules/is-extglob": {

"version": "2.1.1",

"resolved": "https://registry.npmjs.org/is-extglob/-/is-extglob-2.1.1.tgz",

"integrity": "sha512-
SbKbANkN603Vi4jEZv49LeVJMn4yGwsbzZworEoyEiutsN3nJYdbO36zfhGJ6QEDpOZIFkDtnq
5JRxmvl3jsoQ==",

"dev": true,

"license": "MIT",

"engines": {

"node": ">=0.10.0"

}

}
```

```

},
"node_modules/is-fullwidth-code-point": {
  "version": "3.0.0",
  "resolved": "https://registry.npmjs.org/is-fullwidth-code-point/-/is-fullwidth-code-point-3.0.0.tgz",
  "integrity": "sha512-nV2v94rO6y2p0Kk+6As/62z98iCfBye2d97qhC7byA8v6y67vZnC5D9LJ9G04aC0ufQqT4P6Ic8856g==",
  "license": "MIT",
  "engines": {
    "node": ">=8"
  }
},
"node_modules/is-glob": {
  "version": "4.0.3",
  "resolved": "https://registry.npmjs.org/is-glob/-/is-glob-4.0.3.tgz",
  "integrity": "sha512-xelSayHH36ZgE7ZWhli7pW34hNbNl8Ojv5KVmkJD4hBdD3th8Tfk9vYasLM+mXWOZhFkgZfxhLSnrwRr4elSSg==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "is-extglob": "^2.1.1"
  },
  "engines": {
    "node": ">=0.10.0"
  }
}

```

```

},
"node_modules/is-stream": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/is-stream/-/is-stream-2.0.1.tgz",
  "integrity": "sha512-43I12R+Dza2ItyrleA3Udjf3rIKer8NItpW5f1uX39v5LrYR/8ccpWWo9wbiJ6pNUdF3A6b3jWY1+c+Sj98==",
  "license": "MIT",
  "engines": {
    "node": ">=8"
  },
},
"funding": {
  "url": "https://github.com/sponsors/sindresorhus"
}
},
"node_modules/isexe": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/isexe/-/isexe-2.0.0.tgz",
  "integrity": "sha512-3C7ecf746429e7a41e33f9cc05e7c5e14865fc084f823900d18b376d0d0d40eb",
  "license": "ISC"
},
"node_modules/jackspeak": {
  "version": "3.4.3",
  "resolved": "https://registry.npmjs.org/jackspeak/-/jackspeak-3.4.3.tgz",

```

```
"integrity": "sha512-
OGLZQpz2yfahA/Rd1Y8Cd9SIEsqvXkLVoS/cgwhnhFMDbsQFeZYJJ7bIZBS9BcamUW96as
q/npPWugM+RQBw==",

"license": "BlueOak-1.0.0",

"dependencies": {
  "@isaacs/cliui": "^8.0.2"
},

"funding": {
  "url": "https://github.com/sponsors/isaacs"
},

"optionalDependencies": {
  "@pkgjs/parseargs": "^0.11.0"
}
},

"node_modules/jose": {
  "version": "4.15.9",
  "resolved": "https://registry.npmjs.org/jose/-/jose-4.15.9.tgz",
  "integrity": "sha512-
1vUQX+IdDMVPj4k8kOxgUqlcK518yluMuGZwqlr44FS1ppZB/5GWh4rZG89erpOBOJjU/OBs
nCVFfapsRz6nEA==",

  "license": "MIT",

  "funding": {
    "url": "https://github.com/sponsors/panva"
  }
},

"node_modules/json-bigint": {
  "version": "1.0.0",
```

```
"resolved": "https://registry.npmjs.org/json-bigint/-/json-bigint-1.0.0.tgz",
"integrity": "sha512-SiPv/8VpZuWbvLSMtTDOU8hEfrZWg/mH/nV/b4o0CYbSxu1UIQPLdwKOClyLQX+VIPO5vrLX3i8qtqFyhdPSUSQ==",
"license": "MIT",
"dependencies": {
  "bignumber.js": "^9.0.0"
},
"node_modules/json-buffer": {
  "version": "3.0.1",
  "resolved": "https://registry.npmjs.org/json-buffer/-/json-buffer-3.0.1.tgz",
  "integrity": "sha512-4bV5BfR2mqfQTJm+V5tPPdf+ZpuhivTuAB5g8kcrXOZpTT/QwwVRWBywX1ozr6lEuPdbHxwaJlm9G6ml2sfSQ==",
  "dev": true,
  "license": "MIT"
},
"node_modules/json-schema-traverse": {
  "version": "0.4.1",
  "resolved": "https://registry.npmjs.org/json-schema-traverse/-/json-schema-traverse-0.4.1.tgz",
  "integrity": "sha512-xbbCH5dCYU5T8LcEhhuh7HJ88HXuW3qsI3Y0zOZFKfZEHcpWiHU/Jxzk629Brsab/mMiHQti9wMP+845RPe3Vg==",
  "dev": true,
  "license": "MIT"
},
```

```
"node_modules/json-stable-stringify-without-jsonify": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/json-stable-stringify-without-jsonify/-/json-stable-stringify-without-jsonify-1.0.1.tgz",
  "integrity": "sha512-Bdboy+l7tA3OGW6FjyFHWkP5LuByj1Tk33Ljyq0axyzdk9//JSi2u3fP1QSmd1KNwq6VOKYGLAu87CisVir6Pw==",
  "dev": true,
  "license": "MIT"
},
"node_modules/jsonwebtoken": {
  "version": "9.0.3",
  "resolved": "https://registry.npmjs.org/jsonwebtoken/-/jsonwebtoken-9.0.3.tgz",
  "integrity": "sha512-901B41P1DPC6BHE0QOABvDEvqGIVU5FqooIw4T0Igu020Y+g8PdlVjn81YDknp4X3BzI2ncJ+Mkq8DmfI8A=",
  "license": "MIT",
  "dependencies": {
    "jws": "^4.0.1",
    "lodash.includes": "^4.3.0",
    "lodash.isboolean": "^3.0.3",
    "lodash.isinteger": "^4.0.4",
    "lodash.isnumber": "^3.0.3",
    "lodash.isplainobject": "^4.0.6",
    "lodash.isstring": "^4.0.1",
    "lodash.once": "^4.0.0",
    "ms": "^2.1.1",
  }
}
```



```
    "semver": "^7.5.4"
  },
  "engines": {
    "node": ">=12",
    "npm": ">=6"
  }
},
"node_modules/jsonwebtoken/node_modules/semver": {
  "version": "7.7.4",
  "resolved": "https://registry.npmjs.org/semver/-/semver-7.7.4.tgz",
  "integrity": "sha512-  
vFKC2IEtQnVhpT78h1Yp8wzwr8CM+MzKMHGJZfBtzhZNycRfnXsHk6E5TxIkkMsgNS7mdX3  
AGB7x2QM2di4IA==",
  "license": "ISC",
  "bin": {
    "semver": "bin/semver.js"
  },
  "engines": {
    "node": ">=10"
  }
},
"node_modules/jwa": {
  "version": "2.0.1",
  "resolved": "https://registry.npmjs.org/jwa/-/jwa-2.0.1.tgz",
  "integrity": "sha512-  
hRF04fqJIP8Abbkq5NKGNOBbr3JxlQ+qhZufXVr0DvujKy93ZCbXZMHDL4EOtodSbCWxOqR8  
MS1tXA5hwqCXDg==",
```

```
"license": "MIT",
"dependencies": {
  "buffer-equal-constant-time": "^1.0.1",
  "ecdsa-sig-formatter": "1.0.11",
  "safe-buffer": "^5.0.1"
},
"node_modules/jwks-rsa": {
  "version": "3.2.2",
  "resolved": "https://registry.npmjs.org/jwks-rsa/-/jwks-rsa-3.2.2.tgz",
  "integrity": "sha512-BqTyEDV+ls8F2trk3A+qJnxV5Q9EqKCBJOPTi3W97r7qTympCZjb7h2X6f2kc+0K3rsSTY1/6YG2eaXKoj497w==",
  "license": "MIT",
  "dependencies": {
    "@types/jsonwebtoken": "^9.0.4",
    "debug": "^4.3.4",
    "jose": "^4.15.4",
    "limiter": "^1.1.5",
    "lru-memoizer": "^2.2.0"
  },
  "engines": {
    "node": ">=14"
  }
},
"node_modules/jws": {
  "version": "4.0.1",
```

```
"resolved": "https://registry.npmjs.org/jws/-/jws-4.0.1.tgz",

"integrity": "sha512-
EKI/M/yqPncGUUh44xz0PxSidXFr/+r0pA70+glYhjv+et7yxM+s29Y+VGDKovRofQem0fs7Uvf4
+YmAdyRduA==",

"license": "MIT",

"dependencies": {

  "jwa": "^2.0.1",

  "safe-buffer": "^5.0.1"

}

},

"node_modules/keyv": {

  "version": "4.5.4",

  "resolved": "https://registry.npmjs.org/keyv/-/keyv-4.5.4.tgz",

  "integrity": "sha512-
oxVHkHR/EJf2CNXnWxRLW6mg7JyCCUcG0DtEGmL2ctUo1PNTin1PUil+r/+4r5MpVgC/fn1k
jsx7mjSujKqlpw==",

  "dev": true,

  "license": "MIT",

  "dependencies": {

    "json-buffer": "3.0.1"

  }

},

"node_modules/levn": {

  "version": "0.4.1",

  "resolved": "https://registry.npmjs.org/levn/-/levn-0.4.1.tgz",

  "integrity": "sha512-
+bT2uH4E5LGE7h/n3evcS/sQUXCpIp6ym8OWJ5eV6+67Dsql/LaaT7qJBAt2rzfoa/5QBGbHx
Dix1dMt2kQKQ==",
```

```
"dev": true,

"license": "MIT",

"dependencies": {

  "prelude-ls": "^1.2.1",

  "type-check": "~0.4.0"

},

"engines": {

  "node": ">= 0.8.0"

}

},

"node_modules/limiter": {

  "version": "1.1.5",

  "resolved": "https://registry.npmjs.org/limiter/-/limiter-1.1.5.tgz",

  "integrity": "sha512-FWWMIEOxz3GwUI4Ts/lvgVy6LPvoMPgjMdQ185nN6psJyBJ4yOpzqm695/h5umdLJg2vW3GR5iG11MAkR2AzJA=="

},

"node_modules/locate-path": {

  "version": "6.0.0",

  "resolved": "https://registry.npmjs.org/locate-path/-/locate-path-6.0.0.tgz",

  "integrity": "sha512-iPZK6eYjbxRu3uB4/WZ3EsEIMJFMqAoopl3R+zuq0UjcAm/MO6KCweDgPfP3elTztoKP3KtnVHxTn2NHBSDVUw==",

  "dev": true,

  "license": "MIT",

  "dependencies": {

    "p-locate": "^5.0.0"
```

```
  },
  "engines": {
    "node": ">=10"
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/lodash": {
  "version": "4.17.23",
  "resolved": "https://registry.npmjs.org/lodash/-/lodash-4.17.23.tgz",
  "integrity": "sha512-  
LgVTMpQtIopCi79SJeDiP0TfWi5CNEc/L/aRdTh3ylvmZXThheWpKjSZhmvMl8iXbC1tFg9gdH  
HDMLoV7CnG+w==",
  "dev": true,
  "license": "MIT"
},
"node_modules/lodash.camelcase": {
  "version": "4.3.0",
  "resolved": "https://registry.npmjs.org/lodash.camelcase/-/lodash.camelcase-  
4.3.0.tgz",
  "integrity": "sha512-  
TwuEnCnxbc3rAvhf/LbG7tJUDzhqXyFv3dtzLOPgCG/hODL7WFnsbwktkD7yUV0RrreP/l1PA  
Lq/YSg6VvjIA==",
  "license": "MIT",
  "optional": true
},
```

```
"node_modules/lodash.clonedeep": {
  "version": "4.5.0",
  "resolved": "https://registry.npmjs.org/lodash.clonedeep/-/lodash.clonedeep-4.5.0.tgz",
  "integrity": "sha512-H5ZhCF25riFd9uB5UCkVKo61m3S/xZk1x4wA6yp/L3RFP6Z/eHH1ymQcGLo7J3GMPfm0V/7m1tryHuGVxpqEBQ==",
  "license": "MIT"
},
"node_modules/lodash.includes": {
  "version": "4.3.0",
  "resolved": "https://registry.npmjs.org/lodash.includes/-/lodash.includes-4.3.0.tgz",
  "integrity": "sha512-N7p1e4i3lJqBqDv32Fq4qZqQJwO82qR/230Y3BvErZpBpWYFfU63FB5qF8q8/89/733H4BZxY780Wg==",
  "license": "MIT"
},
"node_modules/lodash.isboolean": {
  "version": "3.0.3",
  "resolved": "https://registry.npmjs.org/lodash.isboolean/-/lodash.isboolean-3.0.3.tgz",
  "integrity": "sha512-Bz5mupy2SVbPHURB98VAcw+aHh4vRV5IPNhILUCsOzRmsTmSQ17jluqopAentWoehktxGd9e/hbIXq980/1QJg==",
  "license": "MIT"
},
"node_modules/lodash.isinteger": {
  "version": "4.0.4",
  "resolved": "https://registry.npmjs.org/lodash.isinteger/-/lodash.isinteger-4.0.4.tgz",
```

```
    "integrity": "sha512-
DBwtEWN2caHQ9/imiNeEA5ys1JoRtRfY3d7V9wkqtbycnAmTvRRmbHKDV4a0EYc678/dia0j
rte4tjYwVBaZUA==",
    "license": "MIT"
  },
  "node_modules/lodash.isnumber": {
    "version": "3.0.3",
    "resolved": "https://registry.npmjs.org/lodash.isnumber/-/lodash.isnumber-3.0.3.tgz",
    "integrity": "sha512-
QYqzpfwO3/CWf3XP+Z+tkQsfaLL/EnUIXWVklk5FUPc4sBdTehEqZONuyRt2P67PXAk+NXmT
Bcc97zw9t1FQrw==",
    "license": "MIT"
  },
  "node_modules/lodash.isplainobject": {
    "version": "4.0.6",
    "resolved": "https://registry.npmjs.org/lodash.isplainobject/-/lodash.isplainobject-
4.0.6.tgz",
    "integrity": "sha512-
oSXzaWypCMHkPC3NvBEaPHf0KsA5mvPrOPgQWDsbg8n7orZ290M0BmC/jgRZ4vcJ6DTA
hjrSYgdsW/F+MFOBA==",
    "license": "MIT"
  },
  "node_modules/lodash.isstring": {
    "version": "4.0.1",
    "resolved": "https://registry.npmjs.org/lodash.isstring/-/lodash.isstring-4.0.1.tgz",
    "integrity": "sha512-
0wJxfxH1wgO3GrbuP+dTTk7op+6L41QCXbGINEmD+ny/G/eCqGzxyCsh7159S+mgDDcoar
nBw6PC1PS5+wUGgw==",
    "license": "MIT"
  }
}
```

```
  },
  "node_modules/lodash.once": {
    "version": "4.1.1",
    "resolved": "https://registry.npmjs.org/lodash.once/-/lodash.once-4.1.1.tgz",
    "integrity": "sha512-Sb487aTOCr9drQVL8plxOzVhafOjZN9UU54hiN8PU3uAiSV7lx1yYNpbNmex2PK6dSJoNTSJUUswT651yww3Mg==",
    "license": "MIT"
  },
  "node_modules/long": {
    "version": "5.3.2",
    "resolved": "https://registry.npmjs.org/long/-/long-5.3.2.tgz",
    "integrity": "sha512-5PbScuNBniYUaT8WU8Wq3H41t7uYU8p3iXjSsQ48QDqBz056PqUz828UzrR0pE0sBQIMj1F0u6IzBp==",
    "license": "Apache-2.0"
  },
  "node_modules/lru-memoizer": {
    "version": "2.3.0",
    "resolved": "https://registry.npmjs.org/lru-memoizer/-/lru-memoizer-2.3.0.tgz",
    "integrity": "sha512-8o0R3tXK8yF3P75d6X1965B88L6D/8Z48W2W517l0xtw1GJf4Csi61aCb2w+s0Z6t810tXWn4eR0g==",
    "license": "MIT",
    "dependencies": {
      "lodash.clonedeep": "^4.5.0",
      "lru-cache": "6.0.0"
    }
  }
}
```



```
  },
  "node_modules/lru-memoizer/node_modules/lru-cache": {
    "version": "6.0.0",
    "resolved": "https://registry.npmjs.org/lru-cache/-/lru-cache-6.0.0.tgz",
    "integrity": "sha512-Jo6dJ04CmSjuznwJSS3pUeWmd/H0ffTlkXXgwZi+eq1UCmqQwCh+eLsYOYCwY991i2Fah4h1BEMCx4qThGbsiA==",
    "license": "ISC",
    "dependencies": {
      "yallist": "^4.0.0"
    },
    "engines": {
      "node": ">=10"
    }
  },
  "node_modules/lru-memoizer/node_modules/yallist": {
    "version": "4.0.0",
    "resolved": "https://registry.npmjs.org/yallist/-/yallist-4.0.0.tgz",
    "integrity": "sha512-3wdGidZyq5PB084XLES5TpOSRA3wjXAlIWMhum2kRcv/41Sn2emQ0dycQW4uZXLejwKvg6EsvbdlVL+FYEct7A==",
    "license": "ISC"
  },
  "node_modules/math-intrinsics": {
    "version": "1.1.0",
    "resolved": "https://registry.npmjs.org/math-intrinsics/-/math-intrinsics-1.1.0.tgz",
```

```
  "integrity": "sha512-
/IXtbwEk5HTPyEwyKX6hGkYXxM9nbj64B+ilVJnC/R6B0pH5G4V3b0pVbL7DBj4tkhBAppbQU
lf6F6Xl9LHu1g==",

  "license": "MIT",

  "engines": {

    "node": ">= 0.4"

  }

},

"node_modules/media-typer": {

  "version": "0.3.0",

  "resolved": "https://registry.npmjs.org/media-typer/-/media-typer-0.3.0.tgz",

  "integrity": "sha512-
dq+qelQ9akHpcOl/gUVRTxVIOkAJ1wR3QAvb4RsVjS8oVoFjDGTc679wJYmUmknUF5HwML
Ogb5O+a3KxfWapPQ==",

  "license": "MIT",

  "engines": {

    "node": ">= 0.6"

  }

},

"node_modules/merge-descriptors": {

  "version": "1.0.3",

  "resolved": "https://registry.npmjs.org/merge-descriptors/-/merge-descriptors-
1.0.3.tgz",

  "integrity": "sha512-
gaNvAS7TZ897/rVaZ0nMtAyxNyi/pdbjbAwUpFQpN70GqnVfOiXpeUUMKRBmzXaSQ8DdTX4
/0ms62r2K+hE6mQ==",

  "license": "MIT",

  "funding": {
```

```
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/methods": {
  "version": "1.1.2",
  "resolved": "https://registry.npmjs.org/methods/-/methods-1.1.2.tgz",
  "integrity": "sha512-iclAHeNqNm68zFtnZ0e+1L2yUldvzNoauKU4WBA3VvH/vPFieF7qfRlwUZU+DA9P9bPXIS90ulxoUoCH23sV2w==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/mime": {
  "version": "3.0.0",
  "resolved": "https://registry.npmjs.org/mime/-/mime-3.0.0.tgz",
  "integrity": "sha512-jSCU7/VB1loIWBZe14aEYHU/+1UMEH0aO7qxCOVJOW9GgH72VAWppxNcjU+x9a2k3GSIBXNKxXQFqRvvZ7vr3A==",
  "license": "MIT",
  "optional": true,
  "bin": {
    "mime": "cli.js"
  },
  "engines": {
    "node": ">=10.0.0"
  }
}
```

```
}  
  
,  
  
"node_modules/mime-db": {  
  
  "version": "1.52.0",  
  
  "resolved": "https://registry.npmjs.org/mime-db/-/mime-db-1.52.0.tgz",  
  
  "integrity": "sha512-sPU4uV7dYltWJxwwwHD0PuihVNiE7TyAbQ5SWxDCB9mUYvOgroQOwYQQOKPJ8CIbE+1E  
TVlOoK1UC2nU3gYvg==",  
  
  "license": "MIT",  
  
  "engines": {  
  
    "node": ">= 0.6"  
  
  }  
  
,  
  
"node_modules/mime-types": {  
  
  "version": "2.1.35",  
  
  "resolved": "https://registry.npmjs.org/mime-types/-/mime-types-2.1.35.tgz",  
  
  "integrity": "sha512-ZDY+bPm5zTTF+YpCrAU9nK0UglCYPT0QtT1NZWFv4s++TNkcgVaT0g6+4R2uI4MjQjzysHB1  
zxuWL50hzaeXiw==",  
  
  "license": "MIT",  
  
  "dependencies": {  
  
    "mime-db": "1.52.0"  
  
  },  
  
  "engines": {  
  
    "node": ">= 0.6"  
  
  }  
  
},
```

```
"node_modules/minimatch": {
  "version": "10.2.2",
  "resolved": "https://registry.npmjs.org/minimatch/-/minimatch-10.2.2.tgz",
  "integrity": "sha512-  
+G4CpNBxa5MprY+04MbgOw1v7So6n5JY166pFi9KfYwT78fxScCeSNQSNzp6dpPSW2rON  
Ops6Ocam1wFhCgoVw==",
  "dev": true,
  "license": "BlueOak-1.0.0",
  "dependencies": {
    "brace-expansion": "^5.0.2"
  },
  "engines": {
    "node": "18 || 20 || >=22"
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
},
"node_modules/minipass": {
  "version": "7.1.3",
  "resolved": "https://registry.npmjs.org/minipass/-/minipass-7.1.3.tgz",
  "integrity": "sha512-  
tEBHqDnIoM/1rXME1zgka9g6Q2lcoCkxHLuc7ODJ5BxbP5d4c2Z5cGgtXAKu59200Cx7diuH  
TOYfSBD8n6mm8A==",
  "license": "BlueOak-1.0.0",
  "engines": {
    "node": ">=16 || 14 >=14.17"
```

```

    }
  },
  "node_modules/ms": {
    "version": "2.1.3",
    "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
    "integrity": "sha512-6FlzubTLZG3J2a/NVCAleEhJzq5oxgHyaCU9yYXvcLsvoVaHJq/s5xXI6/XXP6tz7R9xAOtHnSO/tXtF3WRTIA==",
    "license": "MIT"
  },
  "node_modules/natural-compare": {
    "version": "1.4.0",
    "resolved": "https://registry.npmjs.org/natural-compare/-/natural-compare-1.4.0.tgz",
    "integrity": "sha512-Lhv5iR1pG+XV9pO5jFwYp29W2i4LWNRG4xgA5YXv7vBvLxS6z5oJkH1845y36JbEwFv8bv89dY8p8Uw==",
    "dev": true,
    "license": "MIT"
  },
  "node_modules/negotiator": {
    "version": "0.6.3",
    "resolved": "https://registry.npmjs.org/negotiator/-/negotiator-0.6.3.tgz",
    "integrity": "sha512-+EUsqGPLsM+j/zdChZjsnX51g4XrHFOIXwfnCVPGLQk/k5giakcKsuxCOBBRu6DSm9opw/O6slWbJdghQM4bBg==",
    "license": "MIT",
    "engines": {
      "node": ">= 0.6"
    }
  }
}

```

```
}  
  
,  
  
"node_modules/node-domexception": {  
  
  "version": "1.0.0",  
  
  "resolved": "https://registry.npmjs.org/node-domexception/-/node-domexception-  
1.0.0.tgz",  
  
  "integrity": "sha512-  
/jKZoMpw0F8GRwl4/eLROPA3cfcXtLApP0QzLmUT/HuPCZWYB7IY9ZrMeKw2O/nFlqPQB3P  
VM9aYm0F312AXDQ==",  
  
  "deprecated": "Use your platform's native DOMException instead",  
  
  "funding": [  
  
    {  
  
      "type": "github",  
  
      "url": "https://github.com/sponsors/jimmywarting"  
  
    },  
  
    {  
  
      "type": "github",  
  
      "url": "https://paypal.me/jimmywarting"  
  
    }  
  
  ],  
  
  "license": "MIT",  
  
  "engines": {  
  
    "node": ">=10.5.0"  
  
  }  
  
},  
  
"node_modules/node-fetch": {  
  
  "version": "2.7.0",
```

```
"resolved": "https://registry.npmjs.org/node-fetch/-/node-fetch-2.7.0.tgz",
"integrity": "sha512-c4FRfUm/dbcWZ7U+1Wq0AwCyFL+3nt2bEw05wfxSz+DWpWsitgmSgYmy2dQdWyKC1694
ELPqMs/YzUSNozLt8A==",
"license": "MIT",
"dependencies": {
  "whatwg-url": "^5.0.0"
},
"engines": {
  "node": "4.x || >=6.0.0"
},
"peerDependencies": {
  "encoding": "^0.1.0"
},
"peerDependenciesMeta": {
  "encoding": {
    "optional": true
  }
},
"node_modules/node-forge": {
  "version": "1.3.3",
  "resolved": "https://registry.npmjs.org/node-forge/-/node-forge-1.3.3.tgz",
  "integrity": "sha512-SF9vQ3U4qKfSo4rBg6yG9O1Uu8W46+qOW6dI6DE8TvV1kHFs87X40qF3060S2sU0qQ8E01iQ1Df111Q==",
  "license": "(BSD-3-Clause OR GPL-2.0)",
```



```
"engines": {
  "node": ">= 6.13.0"
},
"node_modules/nodemailer": {
  "version": "8.0.1",
  "resolved": "https://registry.npmjs.org/nodemailer/-/nodemailer-8.0.1.tgz",
  "integrity": "sha512-5kcldlXmaEjZcHR6F28IKGSgpmZHaF1IXLWFTG+Xh3S+Cce4MiakLtWY+PIBU69fLbRa8HlaGlrC/QolUpHkhg==",
  "license": "MIT-0",
  "engines": {
    "node": ">=6.0.0"
  },
"node_modules/object-assign": {
  "version": "4.1.1",
  "resolved": "https://registry.npmjs.org/object-assign/-/object-assign-4.1.1.tgz",
  "integrity": "sha512-rJgTQnkUnH1sFw8yT6VSU3zD3sWmu6sZhlseY8VX+GRu3P6F7Fu+JNDoXfklElbLJSnc3FUQHVe4cU5hj+BcUg==",
  "license": "MIT",
  "engines": {
    "node": ">=0.10.0"
  },
"node_modules/object-hash": {
```

```
"version": "3.0.0",

"resolved": "https://registry.npmjs.org/object-hash/-/object-hash-3.0.0.tgz",

"integrity": "sha512-RSn9F68PjH9HqtltsSnqYC1XXoWe9Bju5+213R98cNGttag9q9yAOTzdbsqvla7aNm5WffBZFpWYr2aWrklWAw==",

"license": "MIT",

"optional": true,

"engines": {

  "node": ">= 6"

}

},

"node_modules/object-inspect": {

  "version": "1.13.4",

  "resolved": "https://registry.npmjs.org/object-inspect/-/object-inspect-1.13.4.tgz",

  "integrity": "sha512-W67iLl4J2EXEGTbfeHCffrjDfitvLANg0UIX3wFUUSTx92KXRFegMHUVgSqE+wwhAbi4WqjGg9czysTV2Epbew==",

  "license": "MIT",

  "engines": {

    "node": ">= 0.4"

  },

  "funding": {

    "url": "https://github.com/sponsors/ljharb"

  }

},

"node_modules/on-finished": {

  "version": "2.4.1",
```

```
"resolved": "https://registry.npmjs.org/on-finished/-/on-finished-2.4.1.tgz",
"integrity": "sha512-oVlzkg3ENAhCk2zdv7IJwd/QUD4z2RxRwpkcGY8psCVcCYZNq4wYnVWALHM+brtuJjePWiyF/CImuDr8Ch5+kg==",
"license": "MIT",
"dependencies": {
  "ee-first": "1.1.1"
},
"engines": {
  "node": ">= 0.8"
}
},
"node_modules/once": {
  "version": "1.4.0",
  "resolved": "https://registry.npmjs.org/once/-/once-1.4.0.tgz",
  "integrity": "sha512-5y5y88088e0i6339eUq55v1w1WY1CtNp+eX4zH04/Jqxi7Vv9Xp4eHm984VaZM40FkZfU9FMDjtBIk=",
  "license": "ISC",
  "optional": true,
  "dependencies": {
    "wrappy": "1"
  }
},
"node_modules/optionator": {
  "version": "0.9.4",
  "resolved": "https://registry.npmjs.org/optionator/-/optionator-0.9.4.tgz",
```

```
"integrity": "sha512-6lpQ7mKUxRcZNLIObR0hz7lxsapSSIYNZJwXPGeF0mTVqGKFIXj1DQcMoT22S3ROcLyY/rz0PWaWZ9ayWmad9g==",
```

```
"dev": true,
```

```
"license": "MIT",
```

```
"dependencies": {
```

```
  "deep-is": "^0.1.3",
```

```
  "fast-levenshtein": "^2.0.6",
```

```
  "levn": "^0.4.1",
```

```
  "prelude-ls": "^1.2.1",
```

```
  "type-check": "^0.4.0",
```

```
  "word-wrap": "^1.2.5"
```

```
},
```

```
"engines": {
```

```
  "node": ">= 0.8.0"
```

```
}
```

```
},
```

```
"node_modules/p-limit": {
```

```
  "version": "3.1.0",
```

```
  "resolved": "https://registry.npmjs.org/p-limit/-/p-limit-3.1.0.tgz",
```

```
  "integrity": "sha512-TYOanM3wGwNGsZN2cVTYPArw454xnXj5qmWF1bEoAc4+cU/ol7GVh7odevjp1FNHduHc3KZMcFduxU5Xc6uJRQ==",
```

```
  "devOptional": true,
```

```
  "license": "MIT",
```

```
  "dependencies": {
```

```
    "yocto-queue": "^0.1.0"
```

```
  },
  "engines": {
    "node": ">=10"
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/p-locate": {
  "version": "5.0.0",
  "resolved": "https://registry.npmjs.org/p-locate/-/p-locate-5.0.0.tgz",
  "integrity": "sha512-  
LaNjtRWUBY++zB5nE/NwcaoMylSPk+S+ZHNb1TzdbMJMny6dynpAGt7X/tl/QYq3TleE6nxHp  
pbo2LGymrG5Pw==",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "p-limit": "^3.0.2"
  },
  "engines": {
    "node": ">=10"
  },
  "funding": {
    "url": "https://github.com/sponsors/sindresorhus"
  }
},
"node_modules/package-json-from-dist": {
```

```
"version": "1.0.1",

"resolved": "https://registry.npmjs.org/package-json-from-dist/-/package-json-from-dist-1.0.1.tgz",

"integrity": "sha512-UEZIS3/by4OC8vL3P2dTXRETpebLI2Nil5vIrjaD/5UtrkFX/tNbwjTSRAGC/+7CAo2plcBaRgWmcBBHcsaClw==",

"license": "BlueOak-1.0.0"

},

"node_modules/parseurl": {

"version": "1.3.3",

"resolved": "https://registry.npmjs.org/parseurl/-/parseurl-1.3.3.tgz",

"integrity": "sha512-CiyeOxFT/JZyN5m0z9PfXw4SCBJ6Sygz1Dpl0wqjlhDEGGBP1GnsUVEL0p63hoG1fcj3fHynXi9NYO4nWOL+qQ==",

"license": "MIT",

"engines": {

"node": ">= 0.8"

}

},

"node_modules/path-exists": {

"version": "4.0.0",

"resolved": "https://registry.npmjs.org/path-exists/-/path-exists-4.0.0.tgz",

"integrity": "sha512-ak9Qy5Q7jYb2Wwcey5Fpvg2KoAc/ZIhLSLOSBmRmygPsGwkVVt0fZa0qrtMz+m6tJTAHfZQ8FnmB4MG4LWy7/w==",

"dev": true,

"license": "MIT",

"engines": {
```

```
    "node": ">=8"
  }
},
"node_modules/path-key": {
  "version": "3.1.1",
  "resolved": "https://registry.npmjs.org/path-key/-/path-key-3.1.1.tgz",
  "integrity": "sha512-ojmeN0qd+y0jszEtoY48r0Peq5dwMEklICOu6Q5f41lfkswXuKtYrhgoTpLnyIcHm24UhqX+5Tq
m2InSwLhE6Q==",
  "license": "MIT",
  "engines": {
    "node": ">=8"
  }
},
"node_modules/path-scurry": {
  "version": "1.11.1",
  "resolved": "https://registry.npmjs.org/path-scurry/-/path-scurry-1.11.1.tgz",
  "integrity": "sha512-Xa4Nw17FS9ApQFJ9umLiJS4orGjm7ZzwUrwamcGQuHSzDyth9boKDaycYdDcZDuqYATXw
4HFXgaqWTctW/v1HA==",
  "license": "BlueOak-1.0.0",
  "dependencies": {
    "lru-cache": "^10.2.0",
    "minipass": "^5.0.0 || ^6.0.2 || ^7.0.0"
  },
  "engines": {
    "node": ">=16 || 14 >=14.18"
  }
}
```

```
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
},
"node_modules/path-scurry/node_modules/lru-cache": {
  "version": "10.4.3",
  "resolved": "https://registry.npmjs.org/lru-cache/-/lru-cache-10.4.3.tgz",
  "integrity": "sha512-JNAzZcXrCt42VGLuYz0zfAzDfAvJWW6AfYIDBQyDV5DCll2m5sAmK+OI07s59XfsRsWHp02jAJrRadPRGTt6SQ==",
  "license": "ISC"
},
"node_modules/path-to-regexp": {
  "version": "0.1.12",
  "resolved": "https://registry.npmjs.org/path-to-regexp/-/path-to-regexp-0.1.12.tgz",
  "integrity": "sha512-RL8xGT2A2jIwL3R/H1gTvlv1J3b8tYUzj8Dy6YkAbt5qVF5s+ZlF71HnVUW64KjL239em+R6cSeDfZqUw==",
  "license": "MIT"
},
"node_modules/prelude-ls": {
  "version": "1.2.1",
  "resolved": "https://registry.npmjs.org/prelude-ls/-/prelude-ls-1.2.1.tgz",
  "integrity": "sha512-vYJ2iTMlo/Z8j3nE26AQfdHzgP4GdwT9P92Yo9CvTj9KI0VvWvX8tmUj2KqjH3qVOXf8HOyoK5TGv8o0VzEW==",
  "dev": true,

```



```
"license": "MIT",

"engines": {
  "node": ">= 0.8.0"
}
},
"node_modules/proto3-json-serializer": {
  "version": "2.0.2",
  "resolved": "https://registry.npmjs.org/proto3-json-serializer/-/proto3-json-serializer-2.0.2.tgz",
  "integrity": "sha512-SAzp/O4Yh02jGdRc+uIrgoe87dkN/XtwxfZ4ZyafJHymd79ozp5VG5nyZ7ygqPM5+cpLDjjGnYFUkngonyDPOQ==",
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "protobufjs": "^7.2.5"
  },
  "engines": {
    "node": ">=14.0.0"
  }
},
"node_modules/protobufjs": {
  "version": "7.5.4",
  "resolved": "https://registry.npmjs.org/protobufjs/-/protobufjs-7.5.4.tgz",
  "integrity": "sha512-  
CvexbZtbov6jW2eXAvLukXjXUW1TzFaivC46BpWc/3BpcCysb5Vffu+B3XHMm8lVEuy2Mm4  
XGex8hBSg1yapPg==",
```

```
"hasInstallScript": true,
"license": "BSD-3-Clause",
"dependencies": {
  "@protobufjs/aspromise": "^1.1.2",
  "@protobufjs/base64": "^1.1.2",
  "@protobufjs/codegen": "^2.0.4",
  "@protobufjs/eventemitter": "^1.1.0",
  "@protobufjs/fetch": "^1.1.0",
  "@protobufjs/float": "^1.0.2",
  "@protobufjs/inquire": "^1.1.0",
  "@protobufjs/path": "^1.1.2",
  "@protobufjs/pool": "^1.1.0",
  "@protobufjs/utf8": "^1.1.0",
  "@types/node": ">=13.7.0",
  "long": "^5.0.0"
},
"engines": {
  "node": ">=12.0.0"
}
},
"node_modules/proxy-addr": {
  "version": "2.0.7",
  "resolved": "https://registry.npmjs.org/proxy-addr/-/proxy-addr-2.0.7.tgz",
  "integrity": "sha512-LLQsMLSUDUPT44jdrU/O37qlnifitDP+ZwrmmZcoSKyLKvtZxpyV0n2/bD/N4tBAAZ/gJEdZU7KMraoK1+XYAg==",
  "license": "MIT",
```

```
"dependencies": {
  "forwarded": "0.2.0",
  "ipaddr.js": "1.9.1"
},
"engines": {
  "node": ">= 0.10"
}
},
"node_modules/punycode": {
  "version": "2.3.1",
  "resolved": "https://registry.npmjs.org/punycode/-/punycode-2.3.1.tgz",
  "integrity": "sha512-
vYt7UD1U9Wg6138shLtLOvdAu+8DsC/ilFtEVHcH+wydcSpNE20AfSOduf6MkRFahL5FY7X1
oU7nKVZFtfq8Fg==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=6"
  }
},
"node_modules/qs": {
  "version": "6.14.2",
  "resolved": "https://registry.npmjs.org/qs/-/qs-6.14.2.tgz",
  "integrity": "sha512-
V/yCWTTF7VJ9hIh18Ugr2zhJMP01MY7c5kh4J870L7imm6/DIzBsNLTXzMwUA3yZ5b/KBqLx8
Kp3uRvd7xSe3Q==",
  "license": "BSD-3-Clause",
```

```
"dependencies": {
  "side-channel": "^1.1.0"
},
"engines": {
  "node": ">=0.6"
},
"funding": {
  "url": "https://github.com/sponsors/ljharb"
}
},
"node_modules/range-parser": {
  "version": "1.2.1",
  "resolved": "https://registry.npmjs.org/range-parser/-/range-parser-1.2.1.tgz",
  "integrity": "sha512-5b3G1uicY8G3WwKf1WtK13H9Y0qKEH42YTv18a0i23OZl43XNSqPq4HhW3I9ZI9XUWY3qfGuhZ87w2744g==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.6"
  }
},
"node_modules/raw-body": {
  "version": "2.5.3",
  "resolved": "https://registry.npmjs.org/raw-body/-/raw-body-2.5.3.tgz",
  "integrity": "sha512-222657194f7417aC0aR641Z5yYQxQC3vtC4/0VY+43UW95E4xUIX2DRhoyvxSlXidCUfwNgdp0G4M6LlJ1ag==",
  "license": "MIT",
  "dependencies": {
    "bytes": "3.1.2",
    "http-errors": "2.0.0",
    "iconv-lite": "0.4.24",
    "unpipe": "1.0.0"
  },
  "engines": {
    "node": ">= 0.8"
  }
}
```

```
"license": "MIT",
"dependencies": {
  "bytes": "~3.1.2",
  "http-errors": "~2.0.1",
  "iconv-lite": "~0.4.24",
  "unpipe": "~1.0.0"
},
"engines": {
  "node": ">= 0.8"
}
},
"node_modules/readable-stream": {
  "version": "3.6.2",
  "resolved": "https://registry.npmjs.org/readable-stream/-/readable-stream-3.6.2.tgz",
  "integrity": "sha512-9u/sniCrY3D5WdsERHzHE4G2YCXqoG5FTHUiCC4SIbr6XcLZBY05ya9EKjYek9O5xOAwjGq+1JdGBAS7Q9ScoA==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "inherits": "^2.0.3",
    "string_decoder": "^1.1.1",
    "util-deprecate": "^1.0.1"
  },
  "engines": {
    "node": ">= 6"
  }
}
```

```
  },
  "node_modules/require-directory": {
    "version": "2.1.1",
    "resolved": "https://registry.npmjs.org/require-directory/-/require-directory-2.1.1.tgz",
    "integrity": "sha512-5bA9W7G83B89/GmTU5Xb0hF56B9Q8/9D/YKFWQK44qV4lUKsT/bC2UcLYPZHk1g10899+B2Qz40LzkIUaQ==",
    "license": "MIT",
    "optional": true,
    "engines": {
      "node": ">=0.10.0"
    }
  },
  "node_modules/retry": {
    "version": "0.13.1",
    "resolved": "https://registry.npmjs.org/retry/-/retry-0.13.1.tgz",
    "integrity": "sha512-XBQI8W1Cge0Seh+6gjj03LbmRfWuoszgK9ooCpwYlrrhoO80pfq4cUkU5DkknwfOfFteRwLZ56PYOGYyFWdg==",
    "license": "MIT",
    "optional": true,
    "engines": {
      "node": ">= 4"
    }
  },
  "node_modules/retry-request": {
    "version": "7.0.2",
```

```
"resolved": "https://registry.npmjs.org/retry-request/-/retry-request-7.0.2.tgz",
"integrity": "sha512-dUOVLMJ0/JJYEn8NrpOaGNE7X3vpI5XlZS/u0ANjqtcZVKnlxP7IgCFwrKTxENw29emmwug53awKtaMm4i9g5w==",
"license": "MIT",
"optional": true,
"dependencies": {
  "@types/request": "^2.48.8",
  "extend": "^3.0.2",
  "teeny-request": "^9.0.0"
},
"engines": {
  "node": ">=14"
},
"node_modules/rimraf": {
  "version": "5.0.10",
  "resolved": "https://registry.npmjs.org/rimraf/-/rimraf-5.0.10.tgz",
  "integrity": "sha512-l0OE8wL34P4nJH/H2ffoaniAokM2qSmrtXHmlpvYr5AVVX8msAyW0l8NVJFDxlSK4u3Uh/f41cQheDVdnYijwQ==",
  "license": "ISC",
  "dependencies": {
    "glob": "^10.3.7"
  },
  "bin": {
    "rimraf": "dist/esm/bin.mjs"
  }
}
```

```
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
},
"node_modules/safe-buffer": {
  "version": "5.2.1",
  "resolved": "https://registry.npmjs.org/safe-buffer/-/safe-buffer-5.2.1.tgz",
  "integrity": "sha512-sha512-
rp3So07KcdmmKbGvgaNxQJSr7bGVSVk5S9Eq1F+ppbRo70+YeaDxkw5Dd8NPN+GD6bjnY
m2VuPuCXmpuYvmCXQ==",
  "funding": [
    {
      "type": "github",
      "url": "https://github.com/sponsors/feross"
    },
    {
      "type": "patreon",
      "url": "https://www.patreon.com/feross"
    },
    {
      "type": "consulting",
      "url": "https://feross.org/support"
    }
  ],
  "license": "MIT"
},
```



```
"node_modules/safer-buffer": {
  "version": "2.1.2",
  "resolved": "https://registry.npmjs.org/safer-buffer/-/safer-buffer-2.1.2.tgz",
  "integrity": "sha512-YYZ03K82SD7Riyi0E1EQPojLz7kpepnSQI9IyPbHHg1XXXevb5dJI7tpyN2ADxGcQbHG7vcyRH
k0cbwqcQriUtg==",
  "license": "MIT"
},
"node_modules/send": {
  "version": "0.19.2",
  "resolved": "https://registry.npmjs.org/send/-/send-0.19.2.tgz",
  "integrity": "sha512-YSU5V2s68GEz/17rF7CYO0whNCjYtRZ4kK+bI/Kzli9HqYFyR+z6K+T90qZtH3I7s6Xvdz6VBR6aU83elg==",
  "license": "MIT",
  "dependencies": {
    "debug": "2.6.9",
    "depd": "2.0.0",
    "destroy": "1.2.0",
    "encodeurl": "~2.0.0",
    "escape-html": "~1.0.3",
    "etag": "~1.8.1",
    "fresh": "~0.5.2",
    "http-errors": "~2.0.1",
    "mime": "1.6.0",
    "ms": "2.1.3",
    "on-finished": "~2.4.1",

```

```
    "range-parser": "~1.2.1",
    "statuses": "~2.0.2"
  },
  "engines": {
    "node": ">= 0.8.0"
  }
},
"node_modules/send/node_modules/debug": {
  "version": "2.6.9",
  "resolved": "https://registry.npmjs.org/debug/-/debug-2.6.9.tgz",
  "integrity": "sha512-bC7ElrdJaJnPbAP+1EotYvqZsb3ecl5wi6Bfi6BJTUcNowp6cvspg0jXznRTKDjm/E7AdgFBVeAPVMNcKGsHMA==",
  "license": "MIT",
  "dependencies": {
    "ms": "2.0.0"
  }
},
"node_modules/send/node_modules/debug/node_modules/ms": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/ms/-/ms-2.0.0.tgz",
  "integrity": "sha512-Tpp60P6IUJDTuOq/5Z8cdskzJujfwqfOTkrwLwj7IRISpnkJnT6SyJ4PCPnGMofJC9ddhal5KVIYtAt97ix05A==",
  "license": "MIT"
},
"node_modules/send/node_modules/mime": {
```

```
"version": "1.6.0",

"resolved": "https://registry.npmjs.org/mime/-/mime-1.6.0.tgz",

"integrity": "sha512-x0Vn8spl+wuJ1O6S7gnbaQg8Pxx4NNHb7KSINmEWKiPE4RKOPlvijn+NkmYmmRgP68mc70j2EbeTFRsrswaQeg==",

"license": "MIT",

"bin": {

  "mime": "cli.js"

},

"engines": {

  "node": ">=4"

}

},

"node_modules/serve-static": {

  "version": "1.16.3",

  "resolved": "https://registry.npmjs.org/serve-static/-/serve-static-1.16.3.tgz",

  "integrity": "sha512-x0RTqQel6g5SY7Lg6ZreMmsOzncHFU7nhnRWkKgWuMTu5NN0DR5oruckMqRvacAN9d5w6ARnRBXl9xhDCgfMeA==",

  "license": "MIT",

  "dependencies": {

    "encodeurl": "~2.0.0",

    "escape-html": "~1.0.3",

    "parseurl": "~1.3.3",

    "send": "~0.19.1"

  },

  "engines": {
```

```
    "node": ">= 0.8.0"
  }
},
"node_modules/setprototypeof": {
  "version": "1.2.0",
  "resolved": "https://registry.npmjs.org/setprototypeof/-/setprototypeof-1.2.0.tgz",
  "integrity": "sha512-E5LDX7Wrp85Kil5bhZv46j8jOeboKq5JMmYM3gVGdGH8xFpPWxUMsNrLODCrkoXMEeNi/XZ
  lwuRvY4XNwYmJpw==",
  "license": "ISC"
},
"node_modules/shebang-command": {
  "version": "2.0.0",
  "resolved": "https://registry.npmjs.org/shebang-command/-/shebang-command-
  2.0.0.tgz",
  "integrity": "sha512-kHxr2zZpYtdmrN1qDjrrX/Z1rR1kG8Dx+gkpK1G4eXmvXswmcE1hTWBWYUzlraYw1/yZp6Yu
  DY77YtvbN0dmDA==",
  "license": "MIT",
  "dependencies": {
    "shebang-regex": "^3.0.0"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/shebang-regex": {
```

```
"version": "3.0.0",

"resolved": "https://registry.npmjs.org/shebang-regex/-/shebang-regex-3.0.0.tgz",

"integrity": "sha512-7++dFhtcx3353uBaq8DDR4NuxBetBzC7ZQOhmTQInHEd6bSrXdiEyzCvG07Z44UYdLShWUyXt5M/yz8ekcb1A==",

"license": "MIT",

"engines": {

  "node": ">=8"

},

"node_modules/side-channel": {

  "version": "1.1.0",

  "resolved": "https://registry.npmjs.org/side-channel/-/side-channel-1.1.0.tgz",

  "integrity": "sha512-ZX99e6tRweoUXqR+VBrslhda51Nh5MTQwou5tnUDgbtyM0dBgmhEDtWGP/xbKn6hqfPRHu jUNwz5fy/wbbhnpw==",

  "license": "MIT",

  "dependencies": {

    "es-errors": "^1.3.0",

    "object-inspect": "^1.13.3",

    "side-channel-list": "^1.0.0",

    "side-channel-map": "^1.0.1",

    "side-channel-weakmap": "^1.0.2"

  },

  "engines": {

    "node": ">= 0.4"

  },

}
```

```
"funding": {
  "url": "https://github.com/sponsors/ljharb"
},
"node_modules/side-channel-list": {
  "version": "1.0.0",
  "resolved": "https://registry.npmjs.org/side-channel-list/-/side-channel-list-1.0.0.tgz",
  "integrity": "sha512-FCLHtRD/gnpCiCHEiJLOWdmFP+wzCmDEkc9y7NsYxeF4u7Btsn1ZuwgwJGxImImHicJArLP4R0yX4c2KCrMrTA==",
  "license": "MIT",
  "dependencies": {
    "es-errors": "^1.3.0",
    "object-inspect": "^1.13.3"
  },
  "engines": {
    "node": ">= 0.4"
  },
  "funding": {
    "url": "https://github.com/sponsors/ljharb"
  },
  "node_modules/side-channel-map": {
    "version": "1.0.1",
    "resolved": "https://registry.npmjs.org/side-channel-map/-/side-channel-map-1.0.1.tgz",
```

```
"integrity": "sha512-
VCjCNfgMsby3tTdo02nbjtM/ewra6jPHmpThenkTYh8pG9ucZ/1P8So4u4FGBek/BjpOVsDCM
oLA/iuBKIFXRA==",

"license": "MIT",

"dependencies": {
  "call-bound": "^1.0.2",
  "es-errors": "^1.3.0",
  "get-intrinsic": "^1.2.5",
  "object-inspect": "^1.13.3"
},

"engines": {
  "node": ">= 0.4"
},

"funding": {
  "url": "https://github.com/sponsors/ljharb"
}
},

"node_modules/side-channel-weakmap": {
  "version": "1.0.2",
  "resolved": "https://registry.npmjs.org/side-channel-weakmap/-/side-channel-
weakmap-1.0.2.tgz",
  "integrity": "sha512-
WPS/HvHQTYnHisLo9McqBHOJk2FkHO/tlpvldyrnem4aeQp4hai3gythswg6p01oSoTl58rcpi
FAjF2br2Ak2A==",
  "license": "MIT",
  "dependencies": {
    "call-bound": "^1.0.2",
```

```
"es-errors": "^1.3.0",
"get-intrinsic": "^1.2.5",
"object-inspect": "^1.13.3",
"side-channel-map": "^1.0.1"
},
"engines": {
  "node": ">= 0.4"
},
"funding": {
  "url": "https://github.com/sponsors/ljharb"
}
},
"node_modules/signal-exit": {
  "version": "4.1.0",
  "resolved": "https://registry.npmjs.org/signal-exit/-/signal-exit-4.1.0.tgz",
  "integrity": "sha512-bzyZ1e88w9O1iINJbKnOlvyTrWPDl46O1bG0D3XInv+9tkPrxrN8jUUTiFlDkkmKWgn1M6CflA13SuGqOa9Korw==",
  "license": "ISC",
  "engines": {
    "node": ">=14"
  },
  "funding": {
    "url": "https://github.com/sponsors/isaacs"
  }
},
"node_modules/statuses": {
```



```

"version": "2.0.2",

"resolved": "https://registry.npmjs.org/statuses/-/statuses-2.0.2.tgz",

"integrity": "sha512-
DvEy55V3DB7uknRo+4iOGT5fP1slR8wQohVdknigZPMpMstaKJQWhwiYBACJE3Ul2pTnATih
hBYnRhZQHGBiRw==",

"license": "MIT",

"engines": {

  "node": ">= 0.8"

},

},

"node_modules/stream-events": {

  "version": "1.0.5",

  "resolved": "https://registry.npmjs.org/stream-events/-/stream-events-1.0.5.tgz",

  "integrity": "sha512-
E1GUzBSgvct8Jsb3v2X15pjzN1tYebtbLaMg+eBOUOAxbLoSbT2NS91ckc5UD1KfLjld+jXJRg
o0qnV5Nerg==",

  "license": "MIT",

  "optional": true,

  "dependencies": {

    "stubs": "^3.0.0"

  },

},

"node_modules/stream-shift": {

  "version": "1.0.3",

  "resolved": "https://registry.npmjs.org/stream-shift/-/stream-shift-1.0.3.tgz",

  "integrity": "sha512-
76ORR0DO1o1hIKwTbi/DM3EXWGf3ZJYO8cXX5RJwnul2DEg2oyoZyjLNoQM8WsvZiFKCRfC
1O0J7iCvie3RZmQ==",

```

```
"license": "MIT",
"optional": true
},
"node_modules/string_decoder": {
  "version": "1.3.0",
  "resolved": "https://registry.npmjs.org/string_decoder/-/string_decoder-1.3.0.tgz",
  "integrity": "sha512-hkRX8U1WjJFd8LsDJ2yQ/wWWxaopEsABU1XfkM8A+j0+85JAGppt16cr1Whg6Klbb4okU6Mql6BOj+uup/wKeA==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "safe-buffer": "~5.2.0"
  }
},
"node_modules/string-width": {
  "version": "5.1.2",
  "resolved": "https://registry.npmjs.org/string-width/-/string-width-5.1.2.tgz",
  "integrity": "sha512-HnLOCR3vjcY8beoNLtcjZ5/nxn2afmME6lhrDrebokqMap+XbeW8n9TXpPDOqdGK5qcl3oT0GKTW6wC7EMiVqA==",
  "license": "MIT",
  "dependencies": {
    "eastasianwidth": "^0.2.0",
    "emoji-regex": "^9.2.2",
    "strip-ansi": "^7.0.1"
  }
},
```

```
"engines": {
  "node": ">=12"
},
"funding": {
  "url": "https://github.com/sponsors/sindresorhus"
}
},
"node_modules/string-width-cjs": {
  "name": "string-width",
  "version": "4.2.3",
  "resolved": "https://registry.npmjs.org/string-width/-/string-width-4.2.3.tgz",
  "integrity": "sha512-wKyQRQpJ0sIp62ErSZdGsjMJWsap5oRNihHhu6G7JVO/9jIB6UyevL+tXuOqrng8j/cxKTWYwUwvSTriiZz/g==",
  "license": "MIT",
  "dependencies": {
    "emoji-regex": "^8.0.0",
    "is-fullwidth-code-point": "^3.0.0",
    "strip-ansi": "^6.0.1"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/string-width-cjs/node_modules/emoji-regex": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-8.0.0.tgz",
```

```
  "integrity": "sha512-
MSjYzcWNOA0ewAHpz0MxpYFvwg6yjy1NG3xteoqz644VCo/RPgnr1/GGt+ic3iJTzQ8Eu3TdM
14SawnVUmGE6A==",
  "license": "MIT"
},
"node_modules/string-width/node_modules/ansi-regex": {
  "version": "6.2.2",
  "resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-6.2.2.tgz",
  "integrity": "sha512-
Bq3SmSpyFHaWjPk8If9yc6svM8c56dB5BAW4Qbw5jHTwwXXcTL0RMkpDJp6VL0XzlWaCH
TXrkFURMYmD0sLqg==",
  "license": "MIT",
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "https://github.com/chalk/ansi-regex?sponsor=1"
  }
},
"node_modules/string-width/node_modules/strip-ansi": {
  "version": "7.1.2",
  "resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-7.1.2.tgz",
  "integrity": "sha512-
gmBGslpoQJtgnMAvOVqGZpEz9dyoKTCzy2nfz/n8aIFhN/jCE/rCmcxabB6jOOHV+0WNnylO
xaxBQPSvcWklhA==",
  "license": "MIT",
  "dependencies": {
    "ansi-regex": "^6.0.1"
  }
}
```

```
  },
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "https://github.com/chalk/strip-ansi?sponsor=1"
  }
},
"node_modules/strip-ansi": {
  "version": "6.0.1",
  "resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-6.0.1.tgz",
  "integrity": "sha512-Y38VPSHcqkFrCpFnQ9vuSXMquuv5oXOKpGeT6aGrr3o3Gc9AlVa6JBfUSOCnbxGGZF+/0ooI7KrPuUSztUdU5A==",
  "license": "MIT",
  "dependencies": {
    "ansi-regex": "^5.0.1"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/strip-ansi-cjs": {
  "name": "strip-ansi",
  "version": "6.0.1",
  "resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-6.0.1.tgz",
```

```
"integrity": "sha512-
Y38VPSHcqkFrCpFnQ9vuSXmquuv5oXOKpGeT6aGrr3o3Gc9AlVa6JBfUSOCnbxGGZF+/0oo
I7KrPuUSztUdU5A==",

"license": "MIT",

"dependencies": {

  "ansi-regex": "^5.0.1"

},

"engines": {

  "node": ">=8"

}

},

"node_modules/strnum": {

  "version": "2.1.2",

  "resolved": "https://registry.npmjs.org/strnum/-/strnum-2.1.2.tgz",

  "integrity": "sha512-
l63NF9y/cLROq/yqKXSLtcMeeyOfnSQLfMSlzFt/K73olaD8DGaQWd7Z34X9GPiKqP5rbSh84
Hl4bOlLcjiSrQ==",

  "funding": [

    {

      "type": "github",

      "url": "https://github.com/sponsors/NaturalIntelligence"

    }

  ],

  "license": "MIT",

  "optional": true

},

"node_modules/stubs": {
```

```
"version": "3.0.0",

"resolved": "https://registry.npmjs.org/stubs/-/stubs-3.0.0.tgz",

"integrity": "sha512-
PdHt7hHUJKxvTCgbKX9C1V/ftOcJlQgz8BZWnfV5c4B6dcGqlpelTbJ999jBGZ2jYiPAwcX5dP6
oBwVlBlUbxw==",

"license": "MIT",

"optional": true

},

"node_modules/teeny-request": {

"version": "9.0.0",

"resolved": "https://registry.npmjs.org/teeny-request/-/teeny-request-9.0.0.tgz",

"integrity": "sha512-
resvxdc6Mgb7YEThw6G6bExlXKkv6+YbuzGg9xuXxSgxJF7Ozs+o8Y9+2R3sArdWdW8nOoko
Qb1yrpFB0pQK2g==",

"license": "Apache-2.0",

"optional": true,

"dependencies": {

"http-proxy-agent": "^5.0.0",

"https-proxy-agent": "^5.0.0",

"node-fetch": "^2.6.9",

"stream-events": "^1.0.5",

"uuid": "^9.0.0"

},

"engines": {

"node": ">=14"

}

},
```

```
"node_modules/teeny-request/node_modules/agent-base": {
  "version": "6.0.2",
  "resolved": "https://registry.npmjs.org/agent-base/-/agent-base-6.0.2.tgz",
  "integrity": "sha512-9V3881eK17qH081eLPLF5XVF6L9oHyZCf1BkrD78e3WgEhR2tj7nDQKXpSd4w3jKv01UyyoE0LFr6zrpeHs=",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "debug": "4"
  },
  "engines": {
    "node": ">= 6.0.0"
  }
},
"node_modules/teeny-request/node_modules/https-proxy-agent": {
  "version": "5.0.1",
  "resolved": "https://registry.npmjs.org/https-proxy-agent/-/https-proxy-agent-5.0.1.tgz",
  "integrity": "sha512-OCs8j1GplWtQQHsJ8B1h4pD8uH2QxW3xocEMl2ADDB3uVt1YiWwIVN1D1DlKb+h3WHV5EXHfKOq2zU5uuz1Hs=",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "agent-base": "6",
    "debug": "4"
  },
}
```



```
"engines": {
  "node": ">= 6"
},
"node_modules/teeny-request/node_modules/uuid": {
  "version": "9.0.1",
  "resolved": "https://registry.npmjs.org/uuid/-/uuid-9.0.1.tgz",
  "integrity": "sha512-b+1eJOlsR9K8HJpow9Ok3fiWOWSIclzXodvv0RQjVoOVNpWMpxf1wZNpt4y9h10odCNRqnYp1OBzRktckBe3sA==",
  "funding": [
    "https://github.com/sponsors/broofa",
    "https://github.com/sponsors/ctavan"
  ],
  "license": "MIT",
  "optional": true,
  "bin": {
    "uuid": "dist/bin/uuid"
  }
},
"node_modules/toidentifier": {
  "version": "1.0.1",
  "resolved": "https://registry.npmjs.org/toidentifier/-/toidentifier-1.0.1.tgz",
  "integrity": "sha512-o5sSPKEkg/DIQNmH43V0/uerLrpzVedkUh8tGNvaeXpfpuwjKenlSox/2O/BTlZUtEe+JG7s5YhEz608PIAHRA==",
  "license": "MIT",
```

```
"engines": {
  "node": ">=0.6"
},
"node_modules/tr46": {
  "version": "0.0.3",
  "resolved": "https://registry.npmjs.org/tr46/-/tr46-0.0.3.tgz",
  "integrity": "sha512-N3WmsSuqV66lT30CrXNbEjx4GEwlow3v6rr4mCcv6prnfwhS01rkgyFdjPNBYd9br7LpXV1+E
mh01fHnq2Gdgrw==",
  "license": "MIT"
},
"node_modules/ts-deepmerge": {
  "version": "2.0.7",
  "resolved": "https://registry.npmjs.org/ts-deepmerge/-/ts-deepmerge-2.0.7.tgz",
  "integrity": "sha512-3phiGcxPSSR47RBubQxPoZ+pqXsEsozLo4G4AlSrsMKTFg9TA3l+3he5BqpUi9wiuDbaHWWX
H/amlzQ49uEdXtg==",
  "dev": true,
  "license": "ISC"
},
"node_modules/tslib": {
  "version": "2.8.1",
  "resolved": "https://registry.npmjs.org/tslib/-/tslib-2.8.1.tgz",
  "integrity": "sha512-oJFu94HQB+KVduSUQL7wnpmqnmfmLsOA/nAh6b6EH0wCEoK0/mPeXU6c3wKDV83MkOu
HPRHtSXKKU99IBazS/2w==",
  "license": "0BSD"
```

```
  },
  "node_modules/type-check": {
    "version": "0.4.0",
    "resolved": "https://registry.npmjs.org/type-check/-/type-check-0.4.0.tgz",
    "integrity": "sha512-XleUoc9uwGXqjWwXaUTZAmzMcFZ5858QA2vwx1Ur5xlciXIP+8LnFDgRplU30us6teqdlSkFfu+ae4K79Ooew==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "prelude-ls": "^1.2.1"
    }
  },
  "engines": {
    "node": ">= 0.8.0"
  }
},
"node_modules/type-is": {
  "version": "1.6.18",
  "resolved": "https://registry.npmjs.org/type-is/-/type-is-1.6.18.tgz",
  "integrity": "sha512-TkRKr9sUTxeh8MdfuCSP7VizJyzRNMjj2J2do2Jr3Kym598JVdEksuzPQCnlFPW4ky9Q+iA+ma9BGm06XQBy8g==",
  "license": "MIT",
  "dependencies": {
    "media-typer": "0.3.0",
    "mime-types": "~2.1.24"
  }
},
```

```
"engines": {
  "node": ">= 0.6"
},
"node_modules/undici-types": {
  "version": "6.21.0",
  "resolved": "https://registry.npmjs.org/undici-types/-/undici-types-6.21.0.tgz",
  "integrity": "sha512-iwDZqg0QAGrg9Rav5H4n0M64c3mkR59cJ6wQp+7C4nI0gsmExaedaYLN044eT4AtBBwjbTiGPMlt2Md0T9H9JQ==",
  "license": "MIT"
},
"node_modules/unpipe": {
  "version": "1.0.0",
  "resolved": "https://registry.npmjs.org/unpipe/-/unpipe-1.0.0.tgz",
  "integrity": "sha512-pjy2bYhSsufwWlKwPc+l3cN7+wuJlK6uz0YdJEOlQDbl6jo/YlPi4mb8agUkVC8BF7V8NuzeyPNqRksA3hztKQ==",
  "license": "MIT",
  "engines": {
    "node": ">= 0.8"
  }
},
"node_modules/uri-js": {
  "version": "4.4.1",
  "resolved": "https://registry.npmjs.org/uri-js/-/uri-js-4.4.1.tgz",
```

```
  "integrity": "sha512-7rKUyy33Q1yc98pQ1DAmLtwX109F7TIfWlW1Ydo8Wl1ii1SeHieeh0HHfPeL2fMXK6z0s8ecKs9frCuLJvndBg==",
```

```
  "dev": true,
```

```
  "license": "BSD-2-Clause",
```

```
  "dependencies": {
```

```
    "punycode": "^2.1.0"
```

```
  }
```

```
},
```

```
"node_modules/url-template": {
```

```
  "version": "2.0.8",
```

```
  "resolved": "https://registry.npmjs.org/url-template/-/url-template-2.0.8.tgz",
```

```
  "integrity": "sha512-XdVKMF4SJ0nP/O7XIPB0JwAEuT9lDIYnNsK8yGVe43y0AWoKeJNdv3ZNWh7ksJ6KqQFjOO6
```

```
ox/VEitLnaVNufw==",
```

```
  "license": "BSD"
```

```
},
```

```
"node_modules/util-deprecate": {
```

```
  "version": "1.0.2",
```

```
  "resolved": "https://registry.npmjs.org/util-deprecate/-/util-deprecate-1.0.2.tgz",
```

```
  "integrity": "sha512-
```

```
EPD5q1uXyFxJpCrLnCc1nHnq3gOa6DZBocAlil2TaSCA7VCJ1UJDMagCzIkXNsUYfD1daK//L  
TEQ8xilbrHtcw==",
```

```
  "license": "MIT",
```

```
  "optional": true
```

```
},
```

```
"node_modules/utis-merge": {
```

```
  "version": "1.0.1",
```

```
"resolved": "https://registry.npmjs.org/utils-merge/-/utils-merge-1.0.1.tgz",
"integrity": "sha512-pMZTvlkT1d+TFGvDOqodOclx0QWkkgi6Tdoa8gC8ffGAAqz9pzPTZWAybbsHHoED/ztMtkv/V
oYTYyShUn81hA==",
"license": "MIT",
"engines": {
  "node": ">= 0.4.0"
},
"node_modules/uuid": {
  "version": "11.1.0",
  "resolved": "https://registry.npmjs.org/uuid/-/uuid-11.1.0.tgz",
  "integrity": "sha512-0/A9rDy9P7cJ+8w1c9WD9V//9Wj15Ce2MPz8Ri6032usz+NfePxx5AcN3bN+r6ZL6jEo066/yN
YB3tn4pQEx+A==",
  "funding": [
    "https://github.com/sponsors/broofa",
    "https://github.com/sponsors/ctavan"
  ],
  "license": "MIT",
  "bin": {
    "uuid": "dist/esm/bin/uuid"
  }
},
"node_modules/vary": {
  "version": "1.1.2",
  "resolved": "https://registry.npmjs.org/vary/-/vary-1.1.2.tgz",
```

```
  "integrity": "sha512-BNGbWLfd0eUPabhkXUVM0j8uuvREyTh5ovRa/dyow/BqAbZJyC+5fU+IzQOzmAKzYqYRAISoRhdQr3elZ/PXqg==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 0.8"
```

```
  }
```

```
},
```

```
"node_modules/web-streams-polyfill": {
```

```
  "version": "3.3.3",
```

```
  "resolved": "https://registry.npmjs.org/web-streams-polyfill/-/web-streams-polyfill-3.3.3.tgz",
```

```
  "integrity": "sha512-d2JWLcIvmZyTsIoge9MsgFCZrt571BikcWGYkjC1khllbTeDlGqZ2D8vD8E/Ua8WGWbb7Plm8/XJYV7IJHZZw==",
```

```
  "license": "MIT",
```

```
  "engines": {
```

```
    "node": ">= 8"
```

```
  }
```

```
},
```

```
"node_modules/webidl-conversions": {
```

```
  "version": "3.0.1",
```

```
  "resolved": "https://registry.npmjs.org/webidl-conversions/-/webidl-conversions-3.0.1.tgz",
```

```
  "integrity": "sha512-2JAn3z8AR6rjK8Sm8orRC0h/bcl/DqL7tRPdGZ4l1CjdF+EaMLmYxBHyXuKL849eucPFhvBoxMsflfOb8kxaeQ==",
```

```
  "license": "BSD-2-Clause"
```

```
  },
  "node_modules/websocket-driver": {
    "version": "0.7.4",
    "resolved": "https://registry.npmjs.org/websocket-driver/-/websocket-driver-0.7.4.tgz",
    "integrity": "sha512-b17KeDIQVjyb0ssuSDF2cYXSg2iztliJ4B9WdsuB6J952qCPKmnVq4DyW5motImXHDC1cBT/1UezrJVskw5zjg==",
    "license": "Apache-2.0",
    "dependencies": {
      "http-parser-js": ">=0.5.1",
      "safe-buffer": ">=5.1.0",
      "websocket-extensions": ">=0.1.1"
    },
    "engines": {
      "node": ">=0.8.0"
    }
  },
  "node_modules/websocket-extensions": {
    "version": "0.1.4",
    "resolved": "https://registry.npmjs.org/websocket-extensions/-/websocket-extensions-0.1.4.tgz",
    "integrity": "sha512-OqUN79upVtFddZH00L743Wsbq5AqKkZmLgJVIKqhvYUx0Xp60U4Kj01pSjYlq2D9VJVqo8517mGlq741sA==",
    "license": "Apache-2.0",
    "engines": {
      "node": ">=0.8.0"
    }
  }
}
```



```

    }
  },
  "node_modules/whatwg-url": {
    "version": "5.0.0",
    "resolved": "https://registry.npmjs.org/whatwg-url/-/whatwg-url-5.0.0.tgz",
    "integrity": "sha512-saE57nupxk6v3HY35+jzBwYa0rKSy0XR8JSxZPwgLr7ys0IBzhGviA1/TUGJLmSVqs8pb9AnvIC
    XEuOHLprYTw==",
    "license": "MIT",
    "dependencies": {
      "tr46": "~0.0.3",
      "webidl-conversions": "^3.0.0"
    }
  },
  "node_modules/which": {
    "version": "2.0.2",
    "resolved": "https://registry.npmjs.org/which/-/which-2.0.2.tgz",
    "integrity": "sha512-BLI3Tl1TW3Pvl70l3yq3Y64i+awpwXqsGBYWkkqMtnbXgrMD+yj7rhW0kuEDxzJaYXGjEW5og
    apKNMEKNMjibA==",
    "license": "ISC",
    "dependencies": {
      "isexe": "^2.0.0"
    },
    "bin": {
      "node-which": "bin/node-which"
    }
  },

```

```
"engines": {
  "node": ">= 8"
},
"node_modules/word-wrap": {
  "version": "1.2.5",
  "resolved": "https://registry.npmjs.org/word-wrap/-/word-wrap-1.2.5.tgz",
  "integrity": "sha512-BN22B5eaMMI9UMtjrGd5g5eCYPpCPDUy0FJXbYsaT5zYxjFOckS53SQDE3pWkVoWpHXVb3BrYcEN4Twa55B5cA==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=0.10.0"
  },
  "node_modules/wrap-ansi": {
    "version": "8.1.0",
    "resolved": "https://registry.npmjs.org/wrap-ansi/-/wrap-ansi-8.1.0.tgz",
    "integrity": "sha512-si7QWI6zUMq56bESFvagtmzMdGOtoxfR+Sez11Mobfc7tm+VkUckk9bW2UeffTGVUbOksxmSw0AA2gs8g71NCQ==",
    "license": "MIT",
    "dependencies": {
      "ansi-styles": "^6.1.0",
      "string-width": "^5.0.1",
      "strip-ansi": "^7.0.1"
    }
  }
}
```

```
,
"engines": {
  "node": ">=12"
},
"funding": {
  "url": "https://github.com/chalk/wrap-ansi?sponsor=1"
}
},
"node_modules/wrap-ansi-cjs": {
  "name": "wrap-ansi",
  "version": "7.0.0",
  "resolved": "https://registry.npmjs.org/wrap-ansi/-/wrap-ansi-7.0.0.tgz",
  "integrity": "sha512-YVGlj2kamLSTxw6NsZjoBxfSwsn0ycdesmc4p+Q21c5zPuZ1pl+NfxVdxPtdHvmNVOQ6XSYG4AUtyt/Fi7D16Q==",
  "license": "MIT",
  "dependencies": {
    "ansi-styles": "^4.0.0",
    "string-width": "^4.1.0",
    "strip-ansi": "^6.0.0"
  },
  "engines": {
    "node": ">=10"
  },
  "funding": {
    "url": "https://github.com/chalk/wrap-ansi?sponsor=1"
  }
}
```

```

},
"node_modules/wrap-ansi-cjs/node_modules/emoji-regex": {
  "version": "8.0.0",
  "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-8.0.0.tgz",
  "integrity": "sha512-MSjYzcWNOA0ewAHpz0MxpYFvwg6yjy1NG3xteoqz644VCo/RPgnr1/GGt+ic3iJTzQ8Eu3TdM14SawnVUmGE6A==",
  "license": "MIT"
},
"node_modules/wrap-ansi-cjs/node_modules/string-width": {
  "version": "4.2.3",
  "resolved": "https://registry.npmjs.org/string-width/-/string-width-4.2.3.tgz",
  "integrity": "sha512-wKyQRQpjJ0sIp662ErSZdGsjMJWsap5oRNihHhu6G7JVO/99IB6UyevL+tXuOqrng8j/cxKTWYwUwvSTriiZz/g==",
  "license": "MIT",
  "dependencies": {
    "emoji-regex": "^8.0.0",
    "is-fullwidth-code-point": "^3.0.0",
    "strip-ansi": "^6.0.1"
  },
  "engines": {
    "node": ">=8"
  }
},
"node_modules/wrap-ansi/node_modules/ansi-regex": {
  "version": "6.2.2",

```

```
"resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-6.2.2.tgz",
"integrity": "sha512-Bq3SmSpyFHaWjPk8If9yc6svM8c56dB5BAW4Qbw5jHTwwXXcTLoRMkpDJp6VL0XzIWaCH
TXrkFURMYmD0sLqg==",
"license": "MIT",
"engines": {
  "node": ">=12"
},
"funding": {
  "url": "https://github.com/chalk/ansi-regex?sponsor=1"
}
},
"node_modules/wrap-ansi/node_modules/ansi-styles": {
  "version": "6.2.3",
  "resolved": "https://registry.npmjs.org/ansi-styles/-/ansi-styles-6.2.3.tgz",
  "integrity": "sha512-4Dj6M28JB+oAH8kFkTLUo+a2jwOFkuqb3yucU0CANcRRUbxS0cP0nZYCGjcc3BNXwRIsUV
mDGgzawme7zvJHvg==",
  "license": "MIT",
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "https://github.com/chalk/ansi-styles?sponsor=1"
  }
},
"node_modules/wrap-ansi/node_modules/strip-ansi": {
```

```
"version": "7.1.2",

"resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-7.1.2.tgz",

"integrity": "sha512-gmBGslpoQJtgnMAvOVqGZpEz9dyoKTCzy2nfz/n8aIFhN/jCE/rCmcxabB6jOOHV+0WNnylO
xaxBQPSvcWklhA==",

"license": "MIT",

"dependencies": {

  "ansi-regex": "^6.0.1"

},

"engines": {

  "node": ">=12"

},

"funding": {

  "url": "https://github.com/chalk/strip-ansi?sponsor=1"

}

},

"node_modules/wrappy": {

  "version": "1.0.2",

  "resolved": "https://registry.npmjs.org/wrappy/-/wrappy-1.0.2.tgz",

  "integrity": "sha512-l4Sp/DRseor9wL6EvV2+TuQn63dMkPjZ/sp9XkghTEbV9KlPS1xUsZ3u7/IQO4wxtcFB4bgpQP
RcR3QCvezPcQ==",

  "license": "ISC",

  "optional": true

},

"node_modules/y18n": {

  "version": "5.0.8",
```

```
"resolved": "https://registry.npmjs.org/y18n/-/y18n-5.0.8.tgz",
  "integrity": "sha512-0pfFZegeDWJHJIAmTLRP2DwHjdF5s7jo9tuztdQxAhINCdvS+3nGINqPd00AphqJR/0LhANUS
6/+7SCb98YOfA==",
  "license": "ISC",
  "optional": true,
  "engines": {
    "node": ">=10"
  }
},
"node_modules/yargs": {
  "version": "17.7.2",
  "resolved": "https://registry.npmjs.org/yargs/-/yargs-17.7.2.tgz",
  "integrity": "sha512-7dSzzRQ++CKnNI/krKnYRV7JKKPUXMEh61soaHKg9mrWEhzFWWhFnxPxGl+69cD1Ou63C13
NUPCnmlcrvqCuM6w==",
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "cliui": "^8.0.1",
    "escalade": "^3.1.1",
    "get-caller-file": "^2.0.5",
    "require-directory": "^2.1.1",
    "string-width": "^4.2.3",
    "y18n": "^5.0.5",
    "yargs-parser": "^21.1.1"
  },
```

```
"engines": {
  "node": ">=12"
},
"node_modules/yargs-parser": {
  "version": "21.1.1",
  "resolved": "https://registry.npmjs.org/yargs-parser/-/yargs-parser-21.1.1.tgz",
  "integrity": "sha512-tVpsJW7DdjecAiFpbIB1e3qxIQsE6NoPc5/eTdrbbIC4h0LVsWhnoa3g+m2HclBlujHzsxZ4VJV
A+GUuc2/LBw==",
  "license": "ISC",
  "optional": true,
  "engines": {
    "node": ">=12"
  },
  "node_modules/yargs/node_modules/emoji-regex": {
    "version": "8.0.0",
    "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-8.0.0.tgz",
    "integrity": "sha512-MSjYzcWNOA0ewAHpz0MxpYFvwg6yjy1NG3xteoqz644VCo/RPgnr1/GGt+ic3iJTzQ8Eu3TdM
14SawnVUmGE6A==",
    "license": "MIT",
    "optional": true
  },
  "node_modules/yargs/node_modules/string-width": {
    "version": "4.2.3",
```



```
"resolved": "https://registry.npmjs.org/string-width/-/string-width-4.2.3.tgz",
"integrity": "sha512-wKyQRQpjJ0sIp62ErSZdGsjMJWsap5oRNihHhu6G7JVO/9jIB6UyevL+tXuOqrng8j/cxKTwWyW
UwvSTriiZz/g==",
"license": "MIT",
"optional": true,
"dependencies": {
  "emoji-regex": "^8.0.0",
  "is-fullwidth-code-point": "^3.0.0",
  "strip-ansi": "^6.0.1"
},
"engines": {
  "node": ">=8"
},
"node_modules/yocto-queue": {
  "version": "0.1.0",
  "resolved": "https://registry.npmjs.org/yocto-queue/-/yocto-queue-0.1.0.tgz",
  "integrity": "sha512-rVksvsnNCdJ/ohGc6xgPwyN8eheCxsiLM8mxuE/t/mOVqJewPuO1miLpTHQiRgTKCLexL4Me
AFVagts7HmNZ2Q==",
  "devOptional": true,
  "license": "MIT",
  "engines": {
    "node": ">=10"
  },
  "funding": {
```

```
    "url": "https://github.com/sponsors/sindresorhus"
  }
}
}
```

```
```
```

```
package.json
```

```
```json
{
  "dependencies": {
    "firebase-functions": "^7.0.5",
    "nodemailer": "^8.0.1"
  }
}
```

```
```
```

```
policy.yaml
```

```
```yaml
bindings:
- members:
  - serviceAccount:service-397499309217@gcp-gae-service.iam.gserviceaccount.com
```

role: roles/appengine.serviceAgent

- members:

- serviceAccount:service-397499309217@gcp-sa-artifactregistry.iam.gserviceaccount.com

role: roles/artifactregistry.serviceAgent

- members:

- serviceAccount:service-397499309217@gcp-sa-cloudaiacpanion.iam.gserviceaccount.com

role: roles/cloudaiacpanion.serviceAgent

- members:

- serviceAccount:397499309217@cloudbuild.gserviceaccount.com

role: roles/cloudbuild.builds.builder

- members:

- serviceAccount:service-397499309217@gcp-sa-cloudbuild.iam.gserviceaccount.com

role: roles/cloudbuild.serviceAgent

- members:

- serviceAccount:service-397499309217@containerregistry.iam.gserviceaccount.com

role: roles/containerregistry.ServiceAgent

- members:

- serviceAccount:firebase-app-hosting-compute@barangay-1721d.iam.gserviceaccount.com

role: roles/developerconnect.readTokenAccessor

- members:

- serviceAccount:service-397499309217@gcp-sa-devconnect.iam.gserviceaccount.com

role: roles/developerconnect.serviceAgent

- members:

- serviceAccount:397499309217-compute@developer.gserviceaccount.com

- serviceAccount:barangay-1721d@appspot.gserviceaccount.com
- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com
- role: roles/editor
- members:
- serviceAccount:service-397499309217@gcp-sa-firebase.iam.gserviceaccount.com
- role: roles/firebase.managementServiceAgent
- members:
- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com
- serviceAccount:firebase-app-hosting-compute@barangay-1721d.iam.gserviceaccount.com
- role: roles/firebase.sdkAdminServiceAgent
- members:
- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com
- role: roles/firebaseappcheck.admin
- members:
- serviceAccount:firebase-app-hosting-compute@barangay-1721d.iam.gserviceaccount.com
- role: roles/firebaseapphosting.computeRunner
- members:
- serviceAccount:service-397499309217@gcp-sa-firebaseapphosting.iam.gserviceaccount.com
- role: roles/firebaseapphosting.serviceAgent
- members:
- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com
- role: roles/firebaseauth.admin
- members:
- serviceAccount:service-397499309217@firebase-rules.iam.gserviceaccount.com

role: roles/firebaserules.system

- members:

- serviceAccount:service-397499309217@gcp-sa-firebasestorage.iam.gserviceaccount.com

role: roles/firebasestorage.serviceAgent

- members:

- serviceAccount:service-397499309217@gcp-sa-firestore.iam.gserviceaccount.com

role: roles/firestore.serviceAgent

- members:

- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com

role: roles/iam.serviceAccountTokenCreator

- members:

- user:jonladyong@gmail.com

role: roles/owner

- members:

- serviceAccount:service-397499309217@serverless-robot-prod.iam.gserviceaccount.com

role: roles/run.serviceAgent

- condition:

expression: request.time < timestamp("2025-10-17T03:31:48.458Z")

title: developer-connect-connection-setup

members:

- serviceAccount:service-397499309217@gcp-sa-devconnect.iam.gserviceaccount.com

role: roles/secretmanager.admin

- members:

- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com

role: roles/storage.admin

- members:

- serviceAccount:firebase-adminsdk-fbsvc@barangay-1721d.iam.gserviceaccount.com

role: roles/storage.objectCreator

etag: BwZFB3fZHuM=

version: 3

`,`,

requirements.txt

```text

pydantic[email]

pytest

python-decouple

fastapi==0.129.0

uvicorn==0.41.0

wsproto==1.2.0

firebase-admin==7.1.0

google-cloud-firestore==2.23.0

google-cloud-storage==3.9.0

python-dotenv==1.2.1

email-validator==2.3.0

python-dateutil==2.9.0.post0

reportlab==4.4.10

python-multipart==0.0.22

requests==2.31.0

